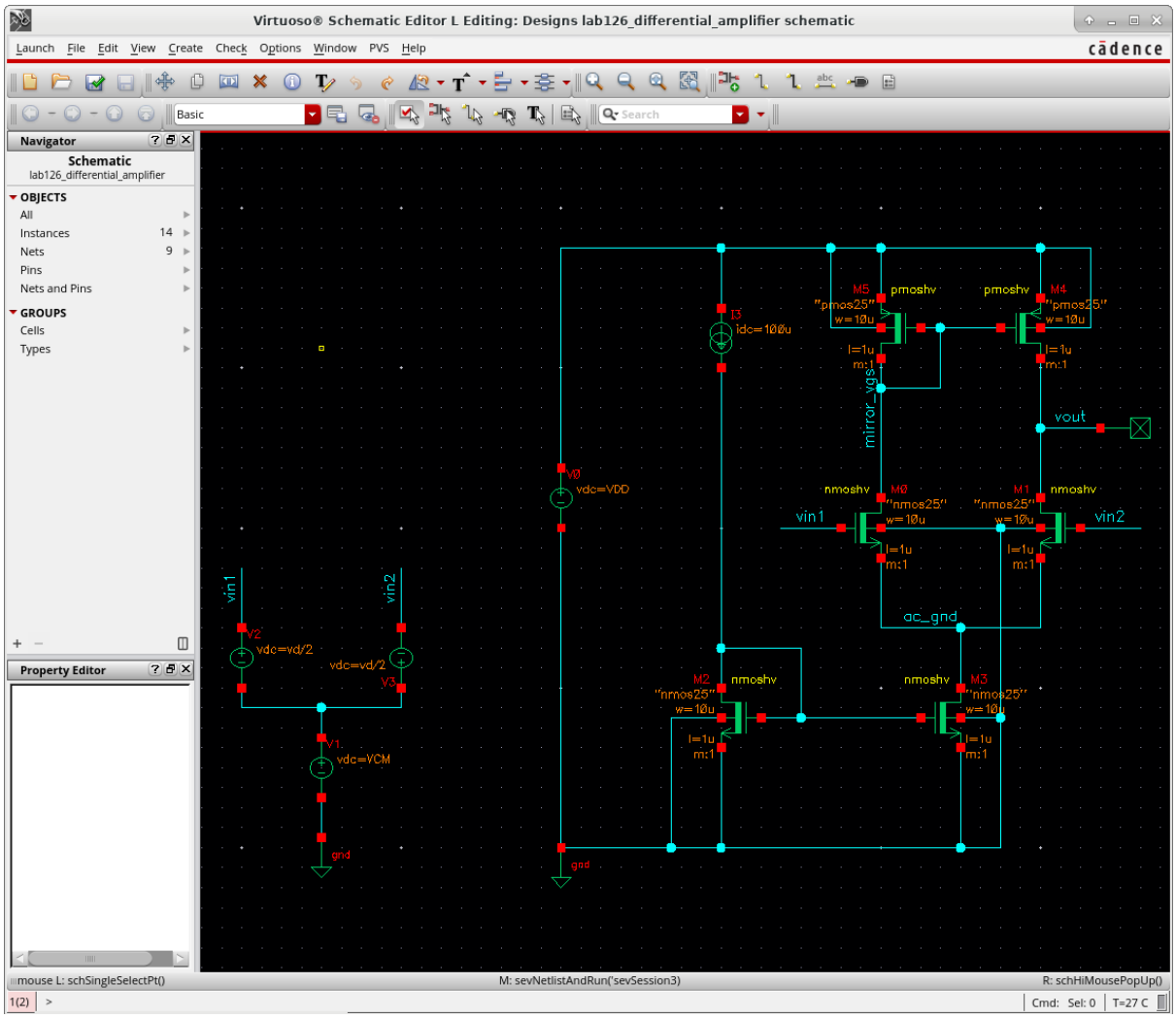
Analogue Integrated Circuits
EEMEM0006
Laboratory Instructions

The digital revolution is constructed on top of an analogue reality.



Tutorial Lab 12.1 (Formatively Assessed)

Setup, Package Introduction, Schematic Entry, and Simulation

Preliminary Matter: Setting up the Cadence Environment

Please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout before entering these commands into your terminal.

For this course, you will use the Cadence Virtuoso Design Environment (referred to simply as Cadence). Cadence is a Linux based software suite for designing full custom integrated circuits. In order to use Cadence you will first need to set-up the Cadence environment.

1. Open a terminal window and ensure that you are in your home directory using the following command:

```
cd ~
```

Note: 'cd' stands for change directory. The solitary tilde symbol (~) is a shortcut/alias for your home directory (/home/username).

2. Under your home directory, use the following commands to load the Cadence environment:

```
module use /eda/cadence/modules
module load course/EENGM6030
```

Note: The two 'module' commands load the required shell/Cadence environmental variables for this course (paths, names, aliases…) In particular, this includes a reference to the process design kit (PDK) used in this course, the GPDK180. You will need to do this every time you open a terminal, since the sourced environment only applies to the current terminal (until it's closed).

3. The first time you use Cadence, you will need to run the following command (script):

```
EENGM6030_setup
```

*Note: **This step only needs to be completed ONCE - the first time you run Cadence.** This will create a folder called AICD (short for Analogue Integrated Circuit Design) under your home directory. The AICD folder contains the files and directories needed to run Cadence. Running this setup script more than once will result in a warning message that the AICD folder already exists. It will then exit without making changes.*

4. The AICD folder will be used for all your Cadence related activities for this course. **You should always make sure you are in your AICD folder before you launch Cadence.** To run cadence, use the following commands:

```
cd ~/AICD
virtuoso &
```

1. Learning Objectives of this Lab:

With guidance, you will:

- Attain a working understanding of the environment for and basic functionality of the Cadence Design Framework for analogue integrated circuit design.

These labs are designed to illustrate the principles and practices of using a schematic entry-driven, SPICE-based* simulation program to develop analogue integrated circuits. They will cover the most important steps in the analogue IC design flow.

We will use the Cadence Virtuoso Design Environment for this course, but the principle and application is equally applicable to all contemporary platforms offering an analogue IC design flow. The underlying algorithms of simulation and the measurement techniques are generic.

What these tutorial labs are not! They are not intended as a detailed treatment of device physics or circuit analysis/design. -

*SPICE - **S**imulation **P**rogram with **I**ntegrated **C**ircuit **E**mphasis.

2. Cadence Documentation, Help, and Licensing

The Cadence Design Environment is a collection of tools – Schematic, Layout, and Text Editors, Analogue Design Environment, Library Manager, Spectre Simulation Engine, and more.

PLEASE NOTE: WHEN LAUNCHING A TOOL IN THE CADENCE DESIGN ENVIRONMENT YOU MAY SOMETIMES GET A WARNING ABOUT AN UNAVAILABLE LICENSE FOR THE TOOL, AND ASKING PERMISSION TO OBTAIN A HIGHER-TIERED LICENSE. YOU SHOULD SELECT 'ALWAYS' TO THESE MESSAGES.
If you neglect to do this you can recover from the situation by selecting the *options* tab, selecting *cds environment editor* selecting *licence* searching for ADE and change the setting to *Always*.

Each tool has its own help menu. You can explore the documentation for each tool at **Menu: Help → User Guide**. While this course is not about teaching you the specifics of Cadence, you do need to be familiar with it. We do not cover all aspects of the Cadence Virtuoso toolset. There is a wealth of local and online documentation available from the help menus, covering every aspect of the Virtuoso Design Environment. You should learn to consult this documentation if you want to know more about a specific tool. (You can also access the user guides for a specific tool by pressing F1 when a particular window, dialog box, or form is in focus – though this doesn't always work).

Alternatively, you can type 'cdnshelp &' at the command prompt (assuming you have setup your Cadence environment). This will take you to the home page of the Cadence help pages. Clicking on the 'Show Library' icon will display the contents tree for all sections.

3. Getting Started with Cadence:

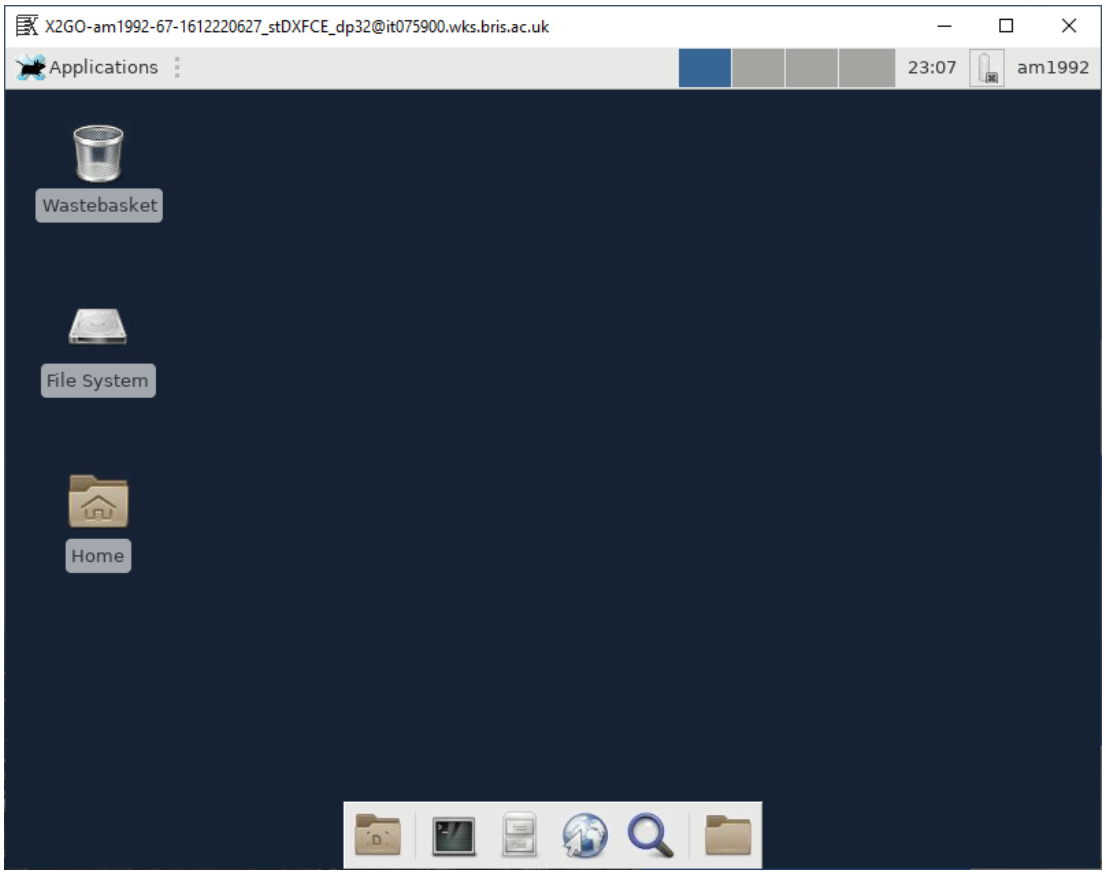
3.1 Remote Access using X2Go

(3.1.1) You may be using Cadence remotely, so you will need to log into one of the engineering Linux Lab machines (in MVB1.15 or MVB2.11) using a remote desktop client.

(3.1.2) The X2Go client runs on Windows, Mac, and Linux and provides a bandwidth efficient graphical user interface (GUI) to a remote Linux Lab machine. While you may use whichever remote desktop software you prefer (where it is compatible with the University's systems), we recommend you use X2go, especially if you are off campus. IT services provide a support page for X2go at:

https://uob.sharepoint.com/sites/itservices/SitePages/fits-engineering-linux-x2go.aspx

(3.1.3) Launch the OS. To start a remote Linux session via X2go, follow the instructions on the IT services page linked-to above (see 'Making a New Session'). When you have successfully started a remote session, you will be logged into one of the Linux Lab machines (randomly chosen from the pool of available machines). You should see your Linux desktop as in fig. 1.

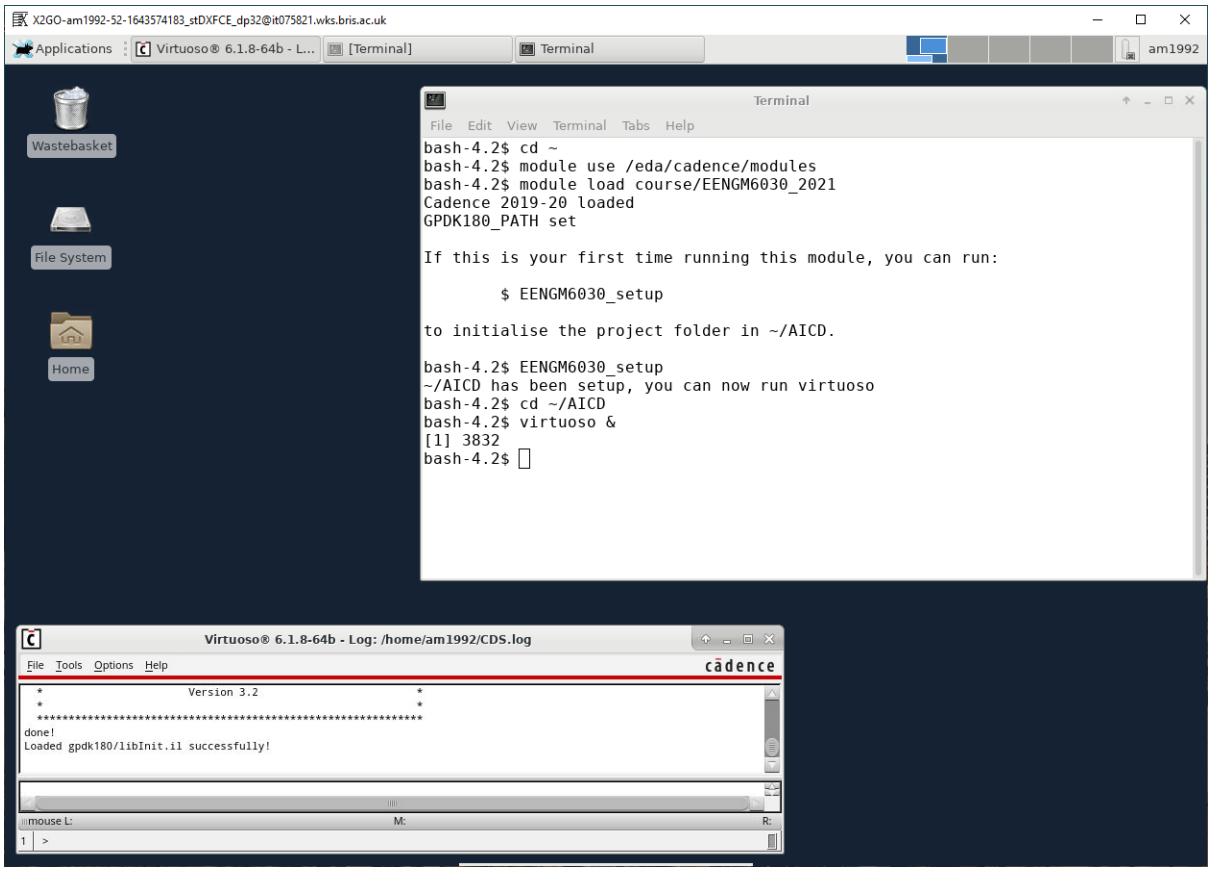


< Figure 1: Linux Desktop in an X2go window. Yours may look slightly different depending on the content of your desktop or your chosen desktop personalisations. >

3.2 Launching Cadence

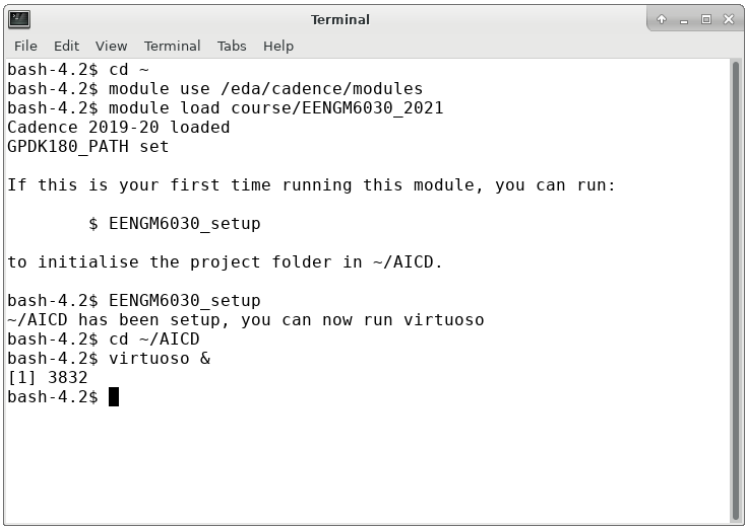
(3.2.1) In order to use Cadence, you will first need to set-up the Cadence environment. Open a terminal window (from either the launcher panel, the applications menu, or by right-clicking anywhere on the empty desktop space) and enter the commands in the order given on the 'Setting up the Cadence Environment' page at the front of this document (preliminary matter). The purpose of each command is explained in the italic text beneath each command.

(3.2.2) You will need to follow these instructions each time you start Cadence (for this course). The setup script 'EENGM6030_setup' in step (3) is used to initialise the Cadence environment before first use. In particular, it will automatically create a folder named 'AICD' directly under your home directory. The AICD folder holds all your Cadence related data for this course. **You should always make sure you are in your AICD folder before you launch Cadence. Note that you only need to run the EENGM6030_setup script the first time you run Cadence.**



< Figure 2: The Terminal and Command Interpreter Window (CIW). >

(3.2.3) Launch the Cadence Command Interpreter Window (CIW). Once you have successfully executed the commands on the 'Setting up the Cadence Environment' page, after a brief period, the Command Interpreter Window (CIW) will appear, usually on the bottom left of the screen. Depending on network activity, it may take a few minutes for the CIW to load.



< Figure 3: The Terminal Window. Setting-up and starting Cadence. Note: bash-4.2$ is the prompt for the terminal's bash shell. Do NOT type this in! >

3.3 The Command Interpreter Window (CIW)

(3.3.1) The CIW controls your design session. Take some time to explore the CIW functionality. The principal components of the CIW are

1. The **title bar.** This contains the following information:

    - The name/version of the workbench you are running (e.g. Virtuoso 6.18-64b)

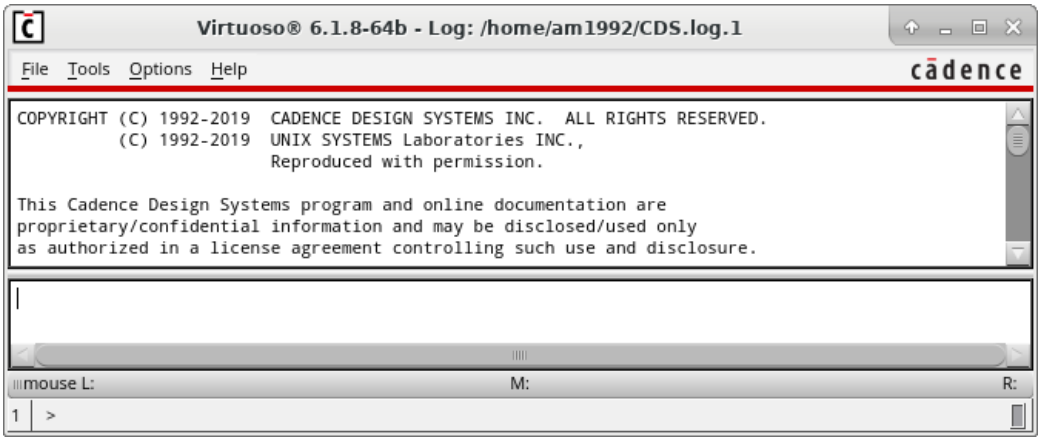
    - The path to the log file (CDS.log) that records the ongoing events of the design session. This is typically /home/username/CDS.log. The content of the log file is displayed in the output area.

    (Aside: if you used Cadence for the COMS30026 Design Verification course, the workbench would have been NCsim not Virtuoso. This would have been reflected in the CIW title bar!)

2. The **menu bar**. The items along the menu bar can change depending on any extensions used (e.g., PDKs), but the base menu items are **File**, **Tools**, **Options**, and **Help**. These are used to launch/start the various components and activities (e.g., Library Manager) of the design environment session. It is also where the online help is found.

3. The **output area**. This is the most important aspect of the CIW for our purposes. This area displays the ongoing events of the design session including command history, responses, warnings, and error messages. This is where you will come to diagnose issues/problems with your design/simulations. As well as displaying this information in the output area, the software also writes it to the CDS.log file.

4. The **edit area**. You can type a command in the edit area of the CIW below the displayed history (output area). This area is useful for running single commands or scripts.

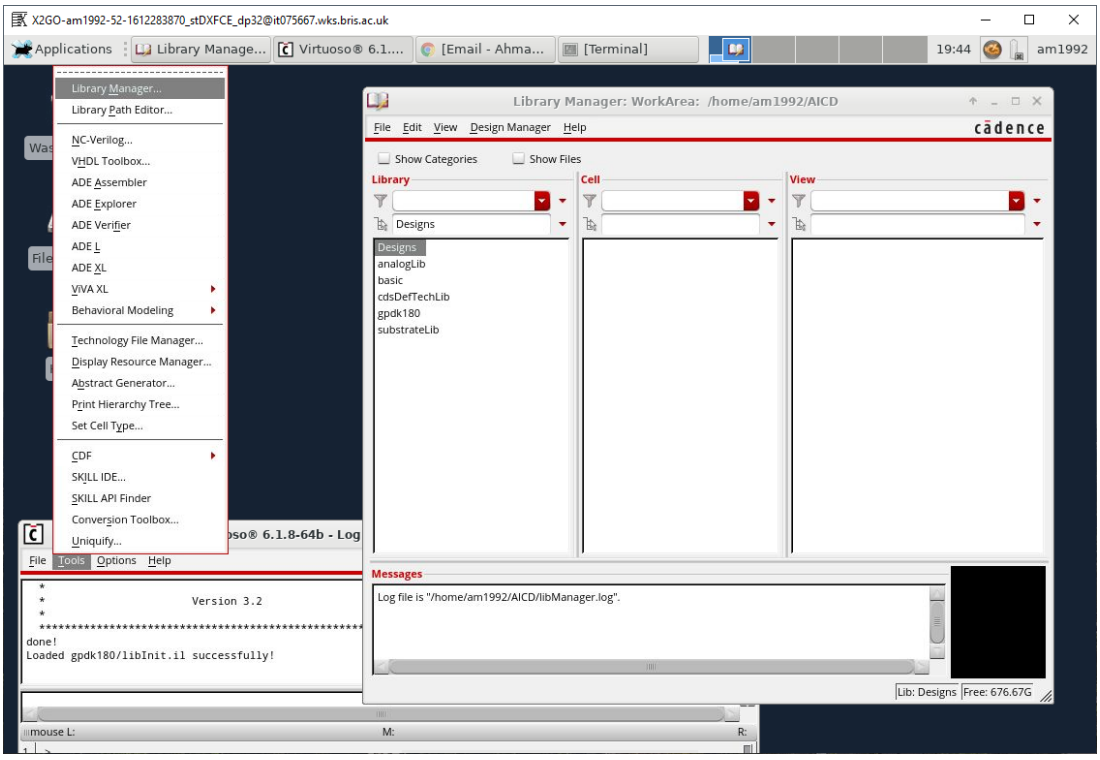


< Figure 4: The Command Interpreter Window (CIW).
The text displayed in your CIW may look different. The CIW starts up displaying only a few lines of output. You can resize the window or use the scroll bars to view more lines.>

(3.3.3) You can explore more of the CIW from the online help. From the CIW, choose **Menu: Help → User Guide.**

3.4 The Library Manager

(3.4.1). Here, we will Investigate the library manager and the different libraries and their components (e.g., res, gnd, vcvs, noConn, ISO border). From the tools menu of the CIW, launch the Library Manager from **Menu: Tools → Library Manager.**



< Figure 5: The Library Manager window with predefined libraries for this course. >

(3.4.2) You can use the Library Manager to create, add, copy, delete, and organise libraries, cells, and cellviews in a design project. The most important aspect to understand here, is how the Library Manager is organised: Each library typically holds several cells, and each cell typically has several views. We will describe each as follows.

1. **Libraries**. A Cadence design project organises itself in terms of libraries. A library in a design project serves a similar function to a folder/directory of an Operating System. Each library contains a number of components called cells. The cells in a library are typically related by a common theme (though this not strictly necessary, it is good practice). For example, you may have a library of analogue cells (FETs, BJTs, Diodes, resistors, caps…) and another library of digital cells (logic-gates, flip-flops, MUXs, registers…). Or, you may have a library of cells by one vendor/foundry (AMS, UMC, TMSC…), and another library from a different vendor/foundry. Libraries may be created by the user or predefined/included by Cadence or a third-party vendor.

2. **Cells**. Each cell in a library represents a component or part. These components may represent a passive or active primitive device (resistor, capacitor, diode, transistor…) or a system of devices (amplifiers, oscillators, regulators, gates, ALUs…). A cell may itself be composed of or reference other cells from the same or a different library. Some cells serve only the purposes of the schematic editor or simulation application (noConn, iprobe, ISO border…). These kinds of cells do not represent real devices or circuits—they are there only to facilitate the schematic entry design flow.

3. **Views**. Each cell has at least one view associated with it. Each view represents one way of modelling the component. For example, a cell may have a schematic view, which describes its logical connections, or a layout view, which describes its physical on-chip layout. A cell may also have symbol view, which is what you would use to instantiate that part in another schematic. A cell may also have a simulator view (functional or behavioural) used by the SPICE-based simulator.

(3.4.3) For this course, when you open the Library Manager you will notice six predefined libraries: **Designs**, **analogLib**, **basic**, **gpdk180**, **cdsDefTechLib**, and **substrateLib**. In practice, you would be required to create your personal design directory(s) and link to them, along with any vendor parts libraries you require, via the cds.lib library definition file in your AICD folder. However, this has already been done by the EENGM030_setup script you ran earlier. You therefore do not need to create or define any libraries.

(3.4.4) The predefined libraries for this course (that you should see in your Library manager) are explained below:

1. **Designs.** This library is where you will store all of your design work (cells/views) for this course. The Designs library is a user-defined library and can be found as a folder in you AICD directory (e.g., ~/AICD/Designs). It was created and linked-to in the cds.lib file by the EENG6030_setup script the first time you setup the Cadence environment.

2. **analogLib**. This is a Cadence provided general parts library. The cells in this library are ideal versions of the most common analogue components (e.g., resistors, capacitors,

sources, ground). They are behavioural/mathematical models. They are simulatable but not synthesizable or realizable and do not have a layout (physical) view. Each cell in this library has at least a symbol view (so that it can be instantiated for use in your schematic) and a **spectre** view (Cadence's SPICE-class simulator is called **Spectre**) that defines its simulation behaviour. (Some components have additional views for other simulators – **auCdl**, **auLvs**, **hspiceD**, **hspiceS** – which are not used in this course).

3. **basic**. This is another Cadence provided general parts library. It contains schematic and symbol views of various connectors, pins, and supplies. These are also ideal parts and are not synthesizable or realisable. Each cell in the basic library has at least a symbol view. Additionally, some cells may have a schematic and/or one or more simulator views (spectre, auCdl, auLvs, hspiceD, hspiceS) that defines its simulation behaviour. (You may also come across two other Cadence provided libraries – **ahdLib** and **rfLib** – but they are not used in this course and have therefore not been defined in the Library Manager).

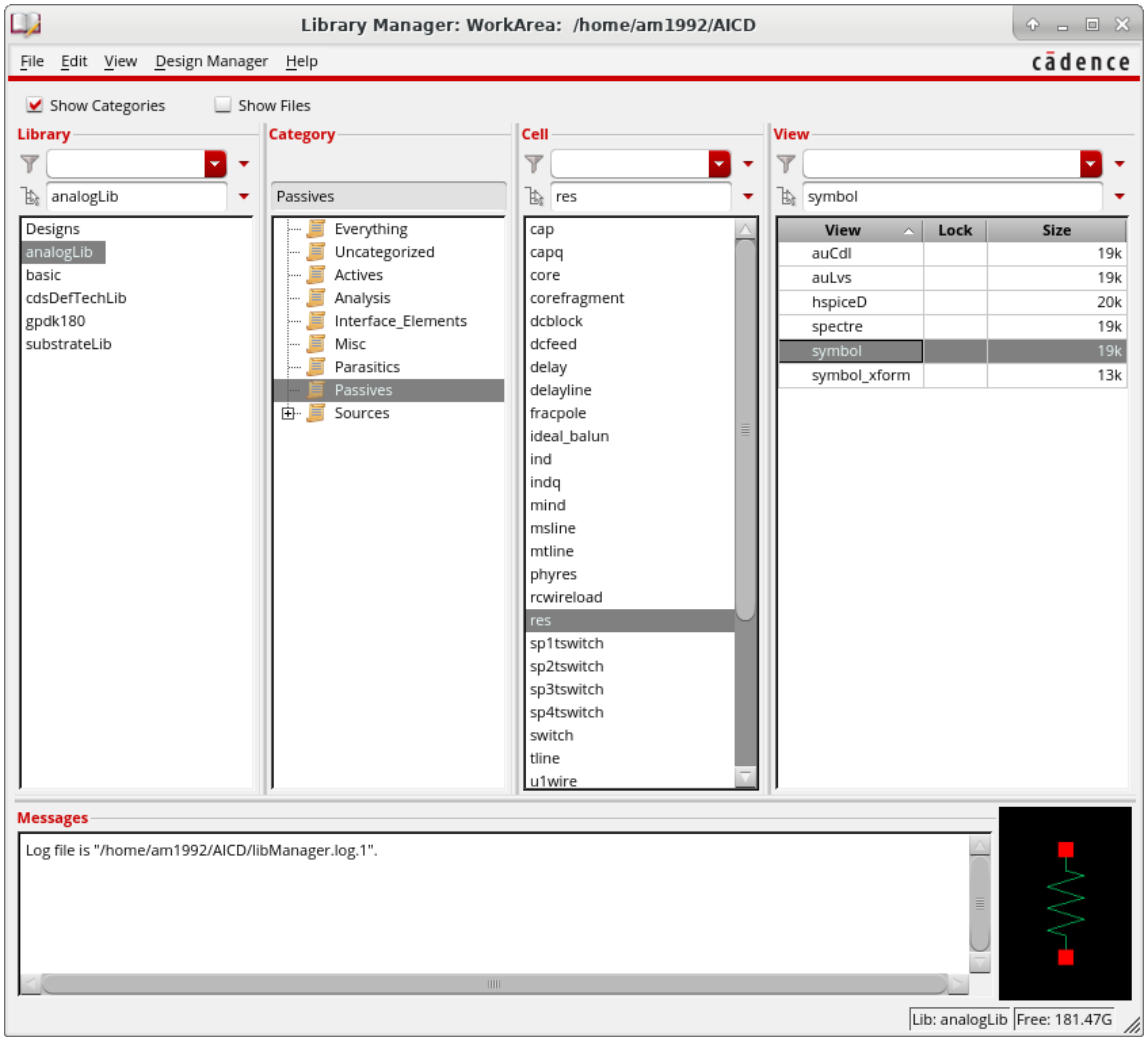
4. **gpdk180.** This is a process design kit (PDK) technology library. It contains schematic, symbol, layout, simulation models, and technology files of a vendor or foundry's 180 nm CMOS process technology. The GPDK180 is a generic PDK provided by Cadence— it is not a real process technology produced by any real foundry and therefore is not synthesisable or realisable. It emulates the behaviour of a typical (generic) 180 nm MOSFET process node. The technology files are useful if you want to do layout.

5. **cdsDefTechLib**. This is the Cadence default technology database. It is referenced by any technology library in your design project, in this case by the GPDK180 library. The details of the library are beyond the scope of this course.

6. **substrateLib.** This library is associated with the GPDK180 technology library. It contains cells which are used during substrate extraction process for parasitic and/or noise behaviour. The details of the library are beyond the scope of this course.

(3.4.5) Notice at the top of the Library Manager there are two check boxes 'Show Categories' and 'Show Files'. The Show Categories check box organises the contents of each library into specific categories. Each category can be further divided into subcategories. This can make it easier to find the component you're looking for. It is often easier to just text search for the component. Each section of the Library Manager (Library, Category, Cell, and View) has a text search field just above the listing area. The 'Show files' check box can be ignored (it lists all the internal files associated with a particular library, category, or cell).

(3.4.6) Explore the '**analogLib**' ideal parts library. In this course, you will make heavy use of **res** (passives: resistor), **cap** (passives: capacitor), **vdc** (sources: independent: DC voltage source), **idc** (sources: independent: DC current source), **gnd** (sources: globals: ground). We will also make use of the controlled source **vcvs** (sources: dependent: voltage-controlled-voltage-source). Note that the analogLib library is divided into nine categories when the 'Show Categories' box is checked (Everything, Uncategorized, Actives, Analysis, Interface_Elements, Misc, Parasitics, Passives, and Sources). The 'Sources' category is further divided into five sub-categories (Dependent, Globals, Independent, Ports, Z_S_Domain).



< Figure 6: Library Manager with 'Show Categories' check box selected. Note: when you select a schematic or symbol cellview an illustration of the cell shows in the bottom-right corner of the dialog box. This doesn't always work as it should! >

(3.4.7) Also explore the '**basic**' and '**GPDK180**' libraries. These two libraries, like analogLib, are also categorised. In these labs, we will only be using the **nmoshv** n-type MOSFET model and the **pmoshv** p-type MOSFET from the gpdk180 library. Note, that these are four-terminal devices (gate, source, drain, and body/bulk). Note the 'hv' suffix stands for high voltage. This is relative! For our purposes, high voltage means a maximum VDS of 2.5V. This is in contrast to low-voltage devices with a sub-1V VDS. (The high voltage behaviour is achieved by using a thicker gate oxide level.) Note the symbol, spectre and layout views in the gpdk180.

(3.4.8) From the '**basic**' library you will only be making use of the **noConn** (no-connection) part. Note that while the analogLib/gpdk180 parts have a layout/spectre view (layout and simulation behaviour), the noConn part does not have a layout or spectre view as it only used by the schematic editor to avoid warning messages on floating wires (it does not simulate). The noConn part will be demonstrated later in this lab.

(3.4.9) You can explore more of the Library Manager from the online help. From the Library Manager, choose **Menu: Help → User Guide.**

4. Beginning the Circuit Simulation Exercises

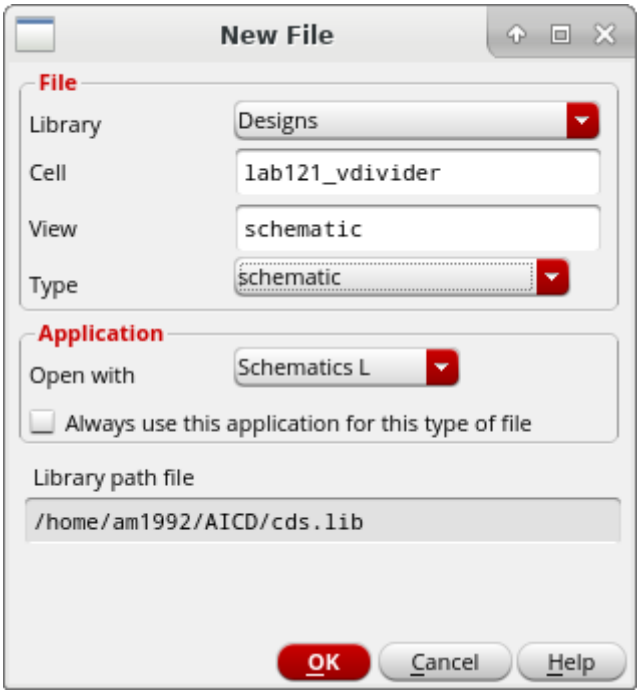
In this section, we will

- Introduce the Virtuoso Schematic Editor and the Analogue Design Environment (ADE) from the Cadence toolset.

- Cover the basic steps of the schematic entry-driven, SPICE-based IC design flow by creating and simulating a simple voltage resistive divider circuit with an ideal DC voltage source supply.

4.1 Creating a New Schematic

(4.1.1) From the Library Manager window, select the 'Designs' library and choose **Menu: Files: à New à Cell View**. You should see the 'New File' dialog box (fig. 6, this box can sometimes hide itself behind the Library Manager window – so check if it doesn't seem to appear).



< Figure 7. The New File Dialog Box >

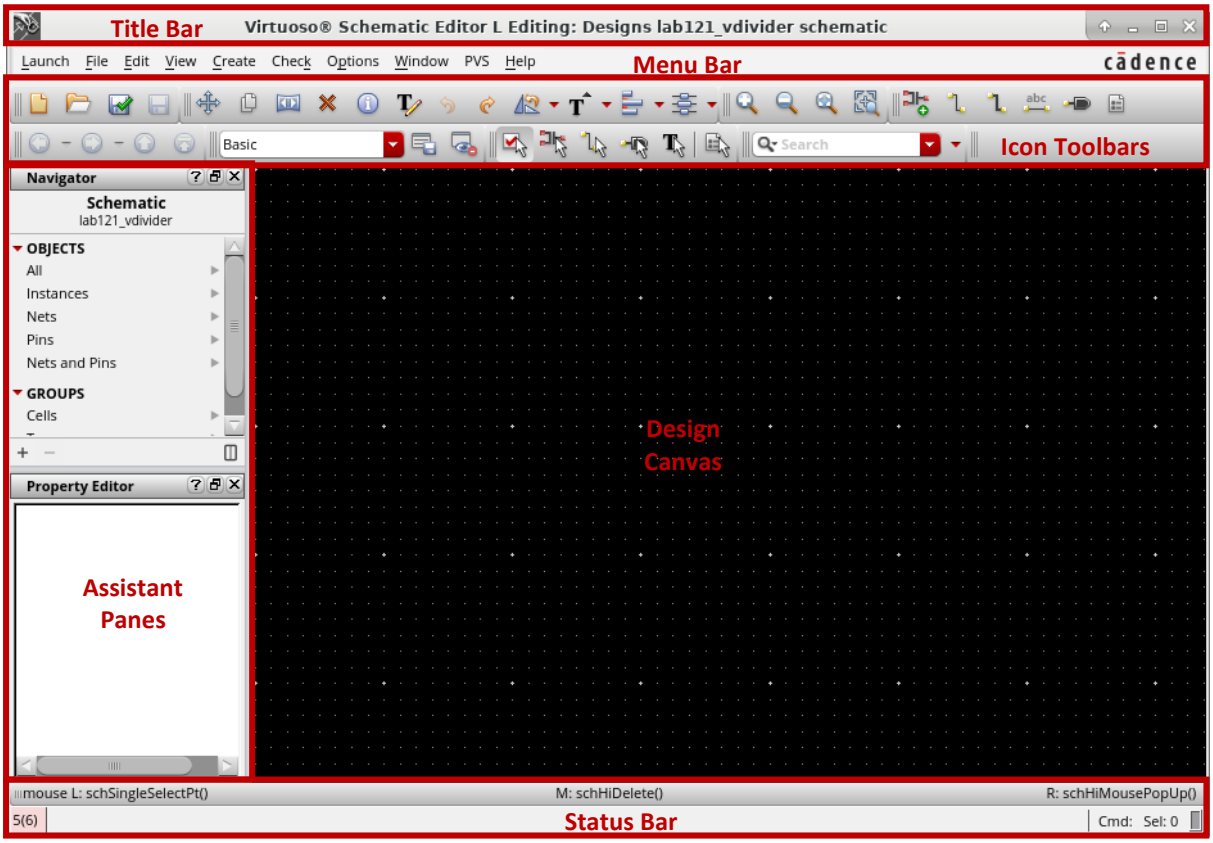
(4.1.2) In the 'Cell' text field, choose a meaningful cell name (e.g. lab121_vdivider). In the 'View' text field, type 'schematic' as the view name and from the 'Type' drop down menu choose 'schematic' as the view type. In the 'Application' section of the dialog box choose 'Schematics L' from the 'Open with' drop down menu. OK the form.

(4.1.3) You should now be presented with the **Virtuoso Schematic Editor L** schematic entry window for the cell you just created (fig. 8). If you check the Library Manager, you will see that a cell called 'lab121_vdivider' (or whatever name you chose) with view name 'schematic' has been added to the 'Designs' library. This cell view will not be saved in the Library Manager

until you check and save it in the Schematic Editor. From the Schematic Editor window, choose **Menu: File à Check and Save**. (Or, you can also **Menu: Check à Current_Cellview** followed by **Menu: File à Save** to perform the same command.)

(4.1.4) Note the many options available from the file ('type') and application ('open with') drop down menus in the new file dialog box. These are how you can add new cellviews to your pre-existing libraries, and also add new views to your pre-existing cells. With experience, you will learn that there are many different ways of adding cellviews to your libraries.

(4.1.5) Also note that some of the tools of the Cadence environment (see **Menu: Launch**) are divided into three tiers L, XL, and GXL e.g. (Schematic Editor L and Schematic Editor XL). Successive tiers (L → XL → GXL) are versions of the same tool, with additional/advanced features. You will only use Schematic Editor L and ADE L tools in this course. The L/XL/GXL don't stand for anything.



< Figure 8. The Virtuoso Schematic Editor L Window.
Note the components of the editor – Title Bar, Menu Bar, Tool Bars, Assistant Panes, Design Canvas, and Status Bar. >

PLEASE NOTE: IF A SCHEMATIC ISN'T PROPERLY CLOSED DOWN (FOR EXAMPLE, DUE TO A DROPPED REMOTE CONNECTION), THE SCHEMATIC MIGHT BECOME LOCKED. IF THIS HAPPENS YOU WILL NOT BE ABLE TO OPEN THE FILE FOR EDITING. YOU WILL ONLY BE ABLE TO OPEN THE FILE FOR READING UNTIL YOU REMOVE ANY/ALL LOCK FILES. TO DO THIS, GO TO YOUR '~/AICD/Designs' FOLDER AND FIND THE FOLDER REPRESENTING THE SCHEMATIC E.G. 'lab121_vdivider'. DELETE ALL THE .cdslck LOCK FILES AND YOU SHOULD BE ABLE TO OPEN YOUR SCHEMATIC FOR EDITING.

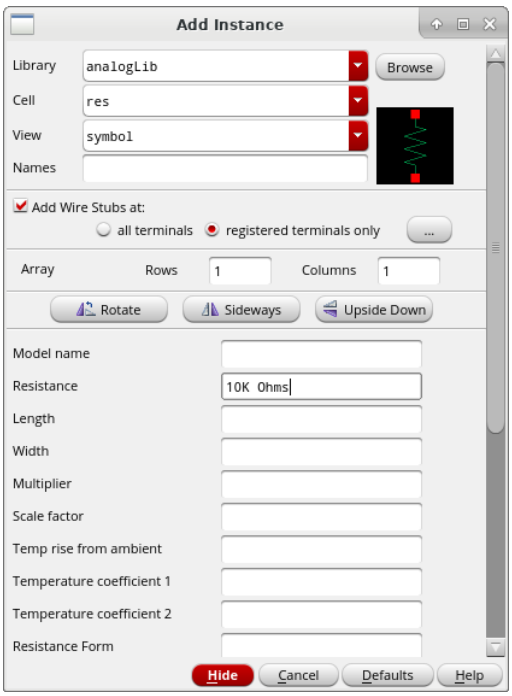
4.2 Adding Components to the Schematic

(4.2.1) On the design canvas of the Schematic Editor, we will place two 10KΩ resistors (**res**), one 10V DC voltage source (**vdc**), and a ground connection (**gnd**). We will then wire them to create a resistive voltage divider circuit. All parts are sourced from the **analogLib** library.

(4.2.2) From the Schematic Editor window, choose **Menu: Create → Instance...** This will open the '**Add instance**' form (fig. 9). Note: There are multiple ways of starting a command in the schematic editor. For example, to add a new part you can use the menu bar **Menu: Create → Instance...**, or you can use the '**Create Instance**' icon-button on the toolbar at the top of the schematic window (fig. 11), or you can use the keyboard shortcut '**I**' (the shortcut key – if one exists – will be shown next to the menu item). You can also right-click anywhere in the empty space of the design canvas to invoke a context-menu and select the **Add Instance…** command from there. Becoming familiar with all available methods will improve your productivity!

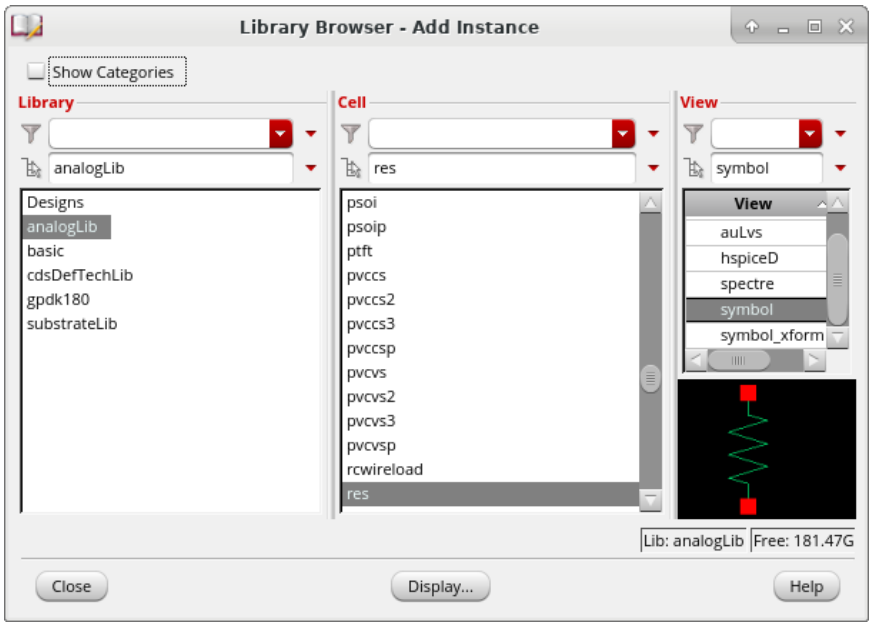
(4.2.3) In the 'Library' text field type 'analogLib', in the 'Cell' text field type 'res', and in the 'View' text field type 'symbol'. (The fields may autocomplete as you type). When you have filled in these text fields, the **Add Instance** form will expand listing the attributes/parameters (and their text fields) for the cell, in this case **res**. (You may need to click away from the current text field for the form to expand.)

(4.2.4) You will see that default values are filled-in for some fields on the **Add Instance** form, others are empty. In the case of the resistor (**res**), the only default value is for the 'Resistance' attribute (1K Ohms). Change this to 10K Ohms. **Do NOT enter the unit type 'Ohms' into the text field. This is automatically appended by the software when you click away from the text field. For a 10KΩ resistor, just enter 10k, 10K, or 10000 in the Resistance text field.** Explore the other fields. Leave them empty. Click the 'Hide' button to exit the form.

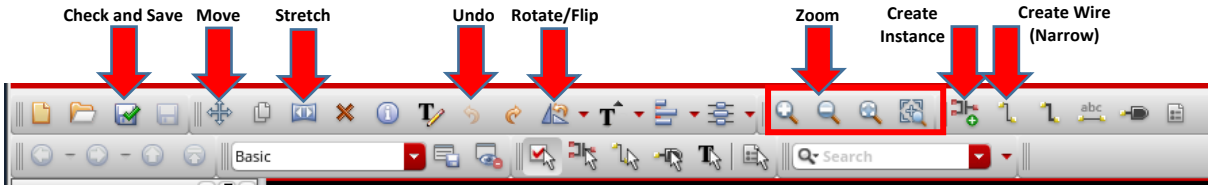


< Figure 9. The Add Instance Form. Filled in for a 10KΩ resistor.>

(4.2.5) Note: You can also use the browse button to select the required library, cell, and view. Click 'Browse'. The 'Library Browser - Add Instance' window will open. Select the **analogLib** library, **res** cell, and **symbol** view. See Figure 10. You can check the 'Show Categories' box to make the search easier or you can text search for the part using the text field in the Library, Cell, and View sections just above the listing area. After selecting the **res** cellview close the Library Browser and you will be returned to the Add Instance form.



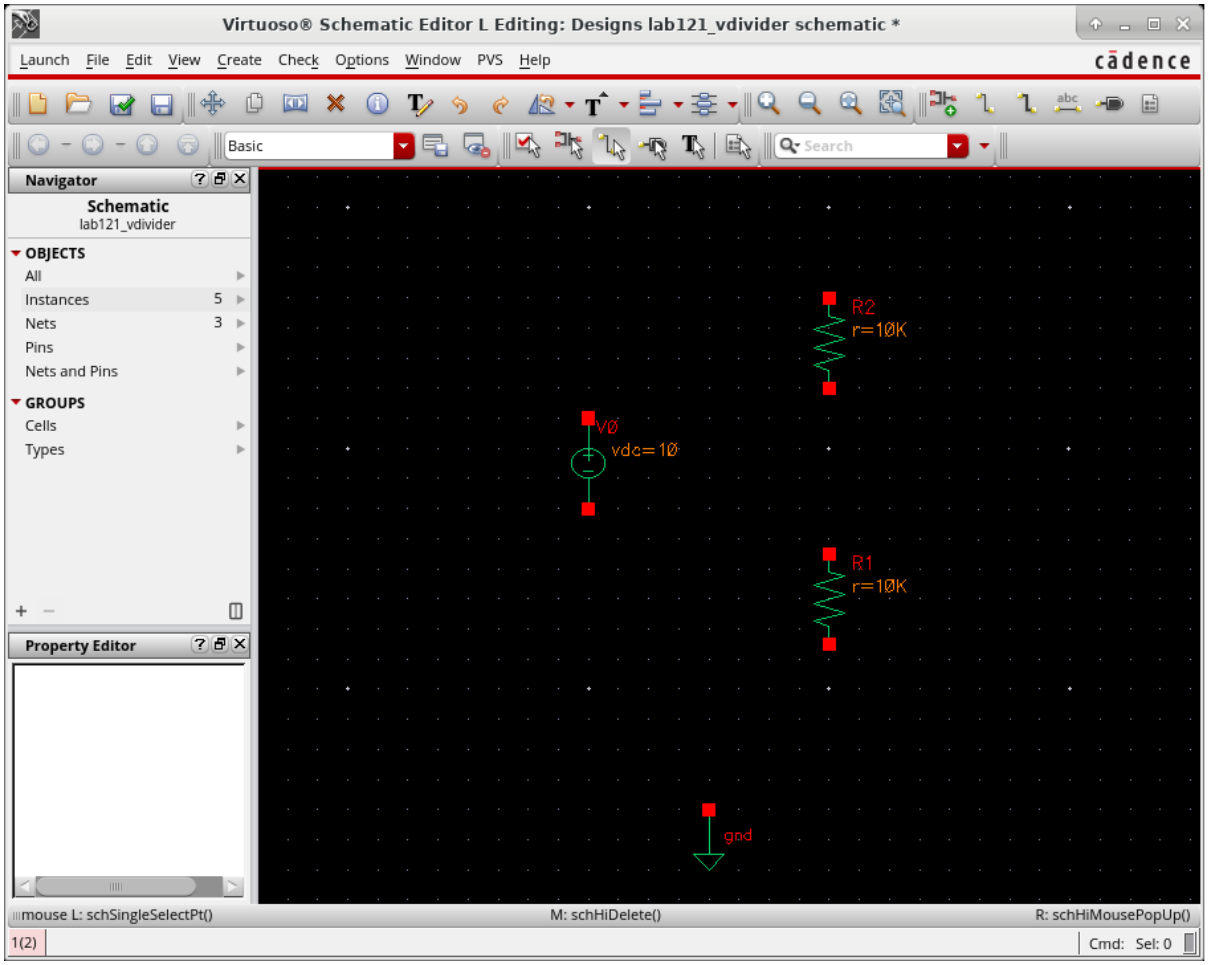
< Figure 10. The Library Browser. Note: that while the Library Browser looks similar to the Library Manager, they are not the same. You cannot use the Library Manager to place components in your schematic. Do not fall into the trap of opening an analogLib cellview from the Library Manager! >



< Figure 11. Useful icon-buttons. Note: to see what an icon button does, hover over the icon and the name of the button will appear. >

(4.2.6) On hiding the **Add Instance** form, you will see a yellow resistor symbol moving with the mouse pointer. You can click anywhere on the design canvas to place as many instances of this component as required (in our case, two). Once placed onto the canvas, the colour of the resistor will change from yellow to green/red. **To stop placing instances of the resistor component, press the 'ESC' key.**

(4.2.7) Now add the remaining two components from the analogLib library – **vdc** and **gnd** – using the same procedure as above. For the **vdc**, set the DC voltage text field to 10 (do not add the unit type), leave all other text fields unchanged (empty). Note the **gnd** component has no text fields.



< Figure 12. The analogLib components of the voltage divider circuit in the schematic editor. >

(4.2.8) Note: you can **zoom-in** and **zoom-out** of the design by using the scroll wheel. You can **pan** the design canvas in any direction by holding down the middle button (scroll wheel) and dragging the canvas in any direction. To centre the view (fit-to-screen), you can press **F**. There are a number of zoom options – zoom-in, zoom-out, zoom-selected, fit-to-screen– accessible from the 'View' menu (**Menu: View → …**) and also the icon-button toolbar.

(4.2.9) Note: You can undo a command by selecting **Menu: Edit → Undo**, using the '**Undo**' icon-button on the toolbar, or by using the keyboard shortcut '**U**'. Your command history only exists between schematic saves/checks. You can undo all commands in your command history until you save or check the schematic (i.e., Save, Check, Check and Save). Once you have saved/checked your schematic your command history is cleared, and you can no longer undo any changes you may have made to your schematic.

(4.2.10) Note: When adding an instance to your schematic, it is good practice to place them in their (approximate) intended location at instantiation. If you want to move a component after placing it on the schematic, you can select it (left-click) and drag it to a new location. Alternatively, you can move the item by selecting **Menu: Edit → Move**, using the '**Move**' icon button on the toolbar, or the keyboard shortcut '**Shift+M**' and clicking on the item. You can also use the stretch command by selecting **Menu: Edit → Stretch**, the '**Stretch**' icon-button

on the toolbar, or the keyboard shortcut '**M**' and clicking on the item. The difference between **Move** (**Shift+M**) and **Stretch** (**M**) is that 'Move' will disconnect the part from its connected wires when moved, whereas 'Stretch' (like dragging) will move the wires with it.

(4.2.11) Note: When adding an instance to your schematic you can rotate and/or flip the item before you place it on the canvas using the 'Rotate', 'Sideways', and 'Upside Down' buttons on the Add Instance form (refer back to Figure 9). Alternatively, you can rotate or flip the item before placing it on the canvas using the keyboard shortcut '**R**'.

After placing a component on the canvas, you can rotate the item using **Menu: Edit → Rotate** or you can use the **Rotate-Left/Rotate-Right/Flip-Horizontal/Flip-Vertical** multi-function icon-button on the toolbar at the top of the schematic window. Note that the functionality of the **Menu: Edit → Rotate** command and associated keyboard shortcut '**R**', is actually set by the Rotate/Flip multi-function icon-button – so whether this command Rotates-Left, Rotates-Right, Flips-Horizontal, or Flips-Vertical depends on the selection of this icon-button.

Finally, pressing F3 while a command is in progress gives you a more detailed set of options for that command. When moving an item using the **Move** or **Stretch** commands, pressing F3 will bring up a dialog box with more detailed options including 'Rotate', 'Sideways', and 'Upside Down' buttons. This is another way of rotating/flipping a component after placement.

(4.2.12) Note: You can modify the attributes of a component (e.g., Resistance or DC Voltage) after it has been placed on the design canvas, by first selecting the item (left-click on the item, you will see a pink box around the item when it is selected) and then by right-clicking on the item and selecting 'Properties…' You can also use **Menu: Edit → Properties → Objects**, the '**Edit Properties**' icon-button from the toolbar, or the keyboard shortcut '**Q**'. You can then modify the required attribute(s) and Apply/OK the change.

4.3 Nets, Nodes, and Wires.

(4.3.1) With the components placed (as in Figure 12), you can now add wires to connect them together. You can use **Menu: Create → Wire (narrow)**, the '**Create Narrow Wire**' icon-button on the toolbar, or the keyboard shortcut '**W**' to start the wire command.

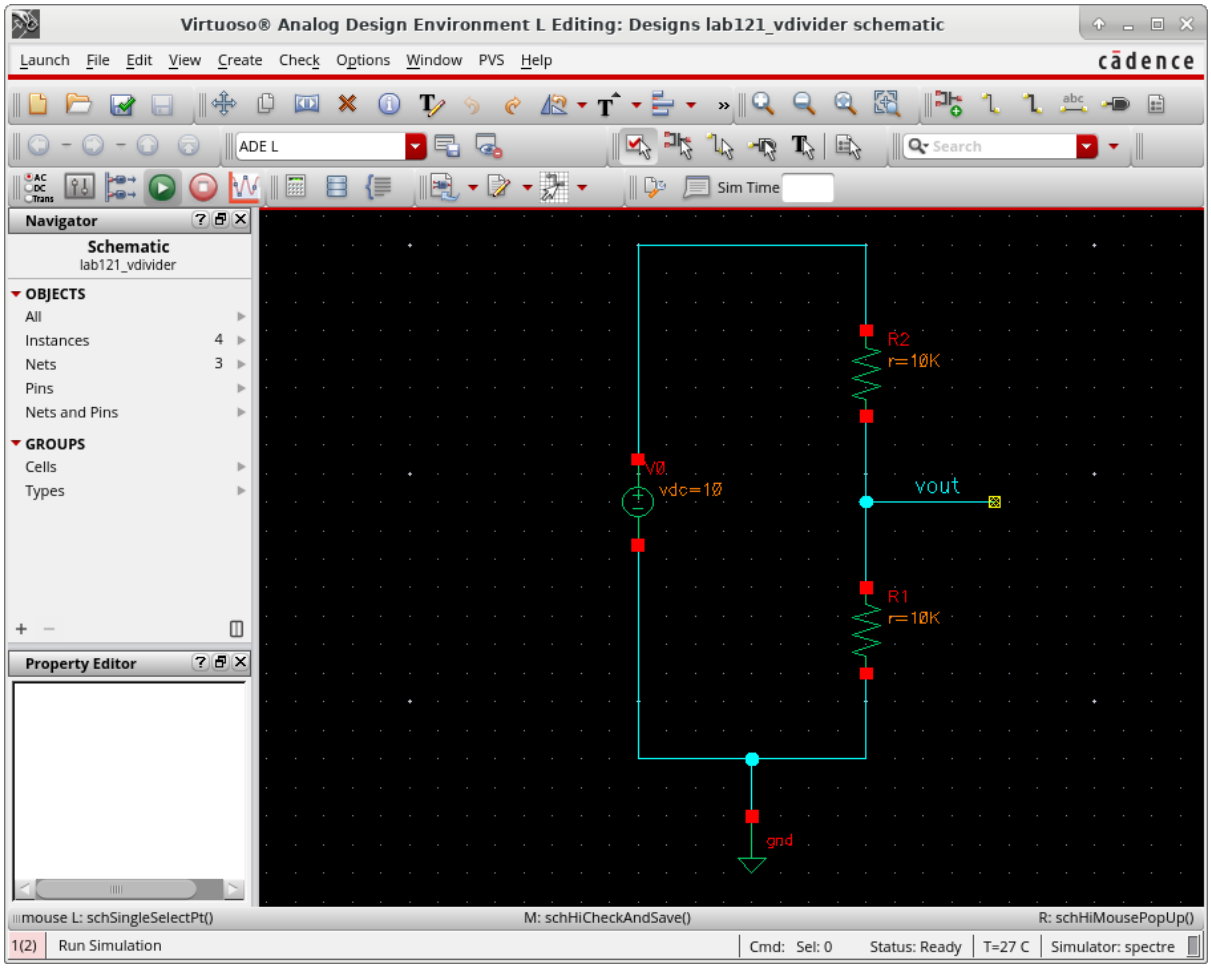
(4.3.2) A wire connects two or more **nodes** and/or **nets** together to form a single **net**. In SPICE terminology, a **net** is the interconnection between one or more components. A **net** has the same potential (voltage) with respect to ground at all points along the interconnection. A **node** identifies a terminal of a component into which a current flows. In the schematic editor, a **net** is created by the wires (blue lines) connecting one or more components. A **node** is identified by the small red squares at the terminals of a component.

(4.3.3) To wire two components together, left-click once on a terminal node of the first component (the small red squares) to begin the wire and then left-click again on a terminal node of the second component to terminate the wire. **For each component click once on the node and release, do not drag!** The nodes should now be connected together with a wire.

(4.3.4) Repeat this for all connections of the divider circuit (see Figure 13). You can also wire a component by placing your cursor on the terminal node of a component (the small red squares) and dragging away from the component while holding down the left-mouse button – begin at the terminal node of one component and terminate on the terminal node of the second component.

(4.3.5) Note: Every net is identified by a net name. By default, this is 'net' appended by a number. The net names are sequentially generated e.g., net1, net2, net3… Some nets have special names e.g., ground (gnd!). The net names are hidden by default – you can toggle on/off the visibility of net names from **Menu: View à Annotations à Net Names**. Or right-click on the canvas to invoke the context-menu and select **Annotations à Net Names**.

(4.3.6) In most cases you can change the default net name to something more helpful. Extend the output net (the net between the two resistors) to the right and name it something intuitive for graphical output (see Figure. 13).



< Figure 13. The wired voltage divider circuit in the schematic editor. >

(4.3.7) Choose **Menu: Create à Wire Name**, use the '**Create Wire Name**' icon-button, or the keyboard shortcut '**L**' to start the create wire name command. The 'Create Wire Name' form will appear, type 'vout' in the 'Names' text field and 'Hide' the form. On hiding the form, you

will see the name ‘vout’ in yellow moving with the mouse pointer. Click on the output net to assign the new name to it. The name will change from yellow to blue (the colour of the wire) when it has been successfully associated with the wire.



< Figure 14. The Create Wire Name Form. >

4.4 Checking and Saving the Schematic

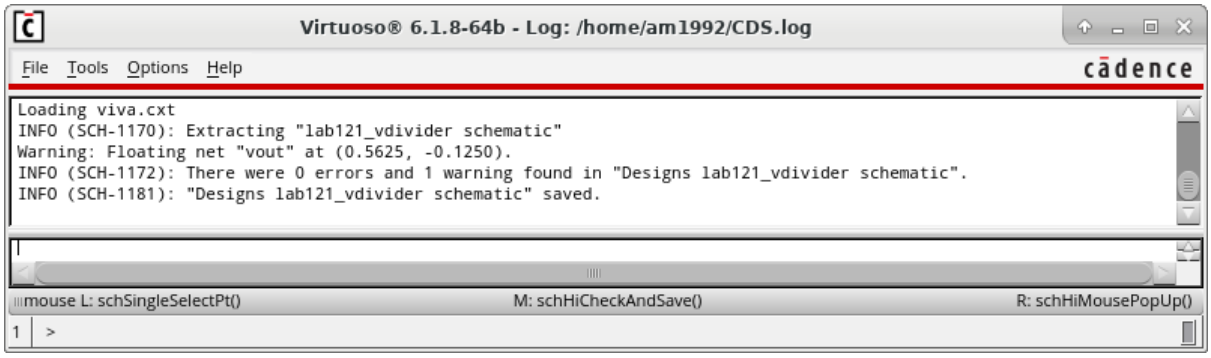
(4.4.1) Save your schematic. You can use the command **Menu: File → Check and Save**, the ‘**Check and Save**’ icon-button, or the keyboard shortcut ‘**Shift+X**’.

(4.4.2) Note: When you check and save you will see a warning message. You will also notice that the unconnected end of the output net (vout), has been highlighted with a yellow square. The schematic editor will issue warnings if there any issues with your circuit. Warnings are not fatal. Your circuit will still be saved and can still be netlisted and simulated. Warnings identify some ambiguity in your circuit such as unconnected pins, wire crossovers, overlapping components and many more. It is good practice to address and eliminate all warnings. Errors are fatal. A schematic cannot be netlisted or simulated if errors exist.

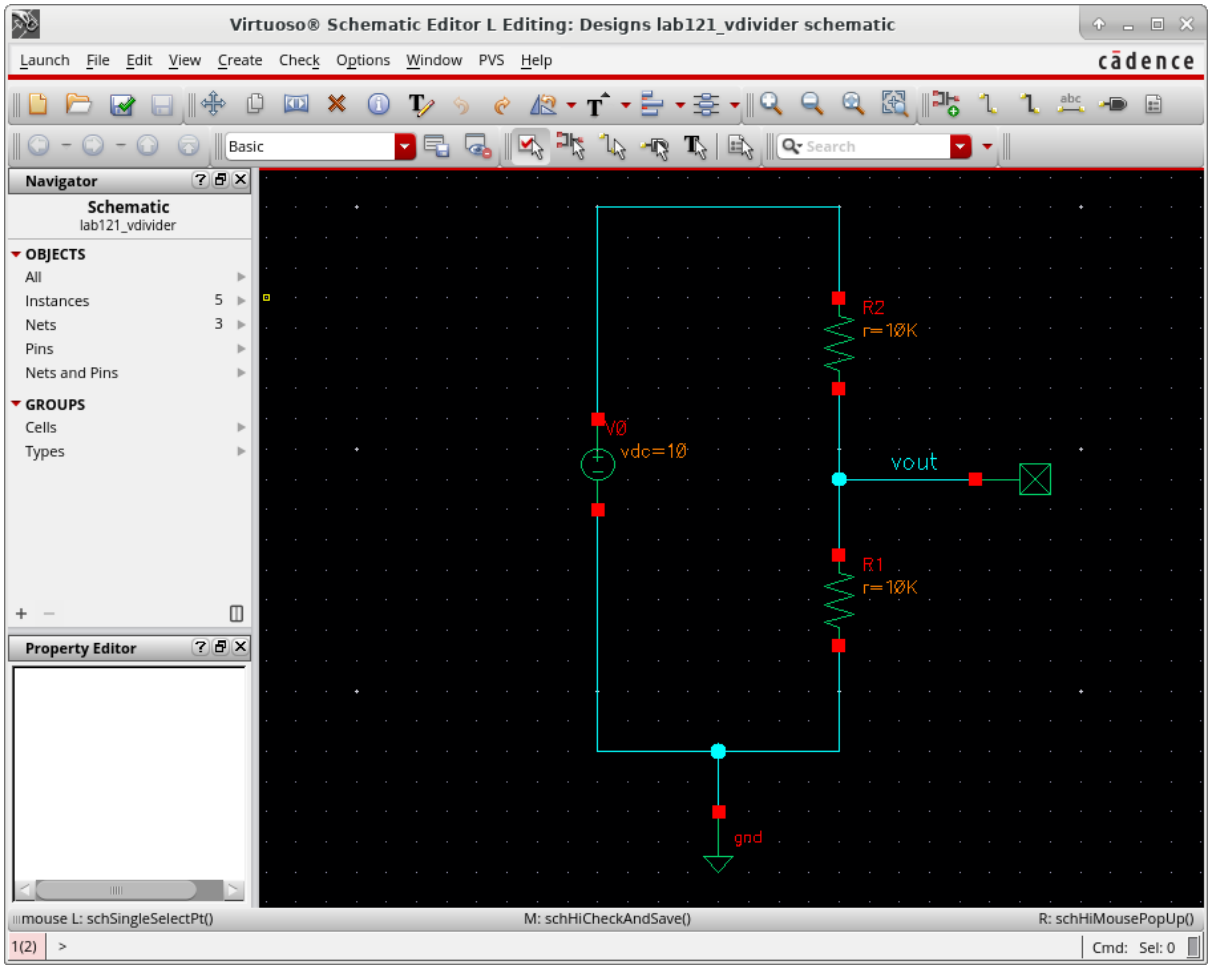
(4.4.3) Note: If there are any warnings or errors generated by your schematic, they will appear in the output area of the CIW. Always check the CIW to diagnose any errors or warnings.

(4.4.4) View the output area of the CIW. Note the ‘floating pin’ warning line (Figure 15.). To avoid floating net warnings, add the **noConn** part from the basic library (Library: **basic**, Cell: **noConn**, View: **symbol**) at the end of any unconnected nets and/or nodes. See Figure 16.

(4.4.5) **Check and Save** your schematic again. This time there should be no warning.



< Figure 15. 'Floating net' warning after Check and Save. >



< Figure 16. The complete voltage divider circuit in the schematic editor including noConn (no-connection) to avoid 'floating net' warnings. >

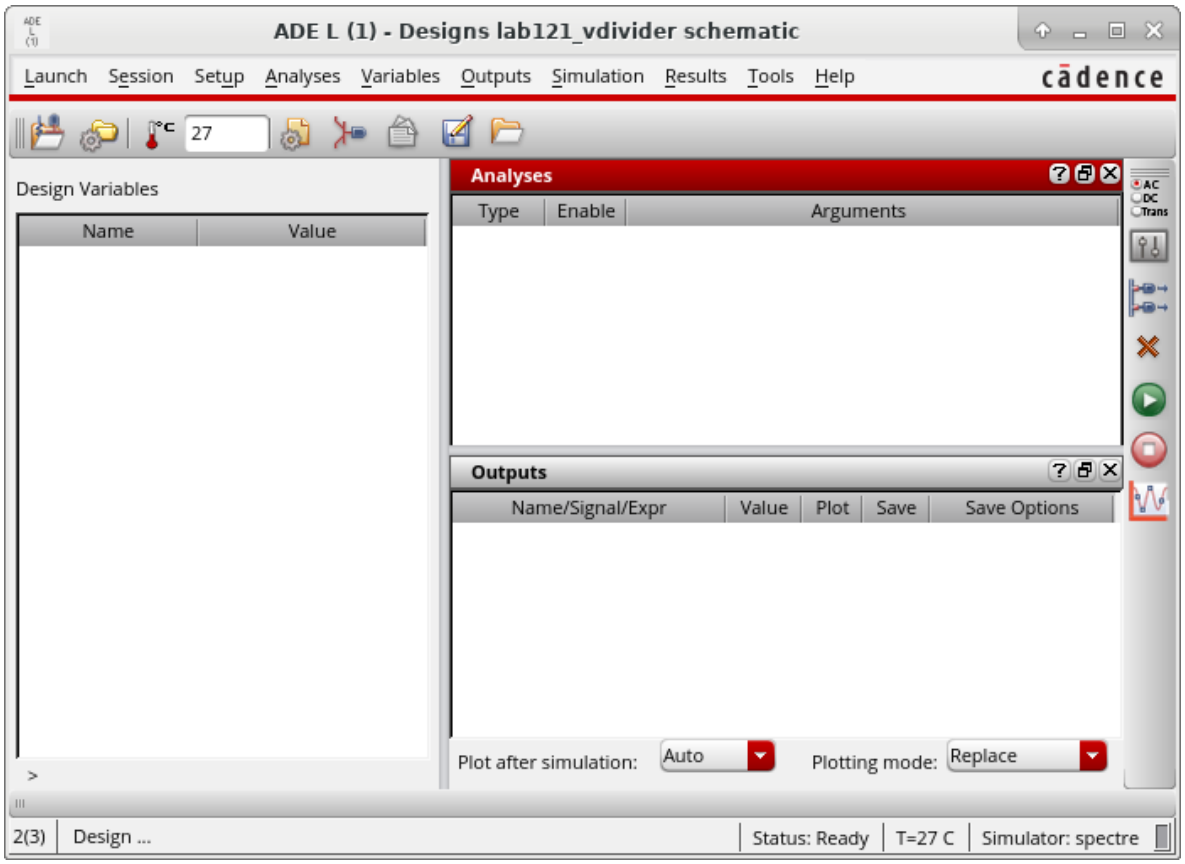
(4.4.6) Note: There is another save option **Menu: File → Save**. The difference between **Save** and **Check and Save**, is that the latter will check the schematic for correctness (errors or other issues) before it saves. If there are errors, it will ask you to confirm that you still wish to save. **Save** does not check your schematic for any issues – it will save your schematic regardless of correctness.

(4.4.7) Note: A netlist is required to simulate your design. A netlist is a text-based description of the connections of your circuit generated from your schematic. The Cadence netlisters will not generate a netlist until the schematic has been checked – if you do not check your design after making changes, it will not simulate. To only check your design, you can use **Menu: Check → Current Cellview** (or keyboard shortcut **X**). If there are any warnings or errors generated by your schematic, they will appear in the output area of the CIW.

4.5 Simulating the Schematic

(4.5.1) You should now have the completed schematic of the voltage divider circuit as shown in Figure 15. It should check and save without any errors or warnings.

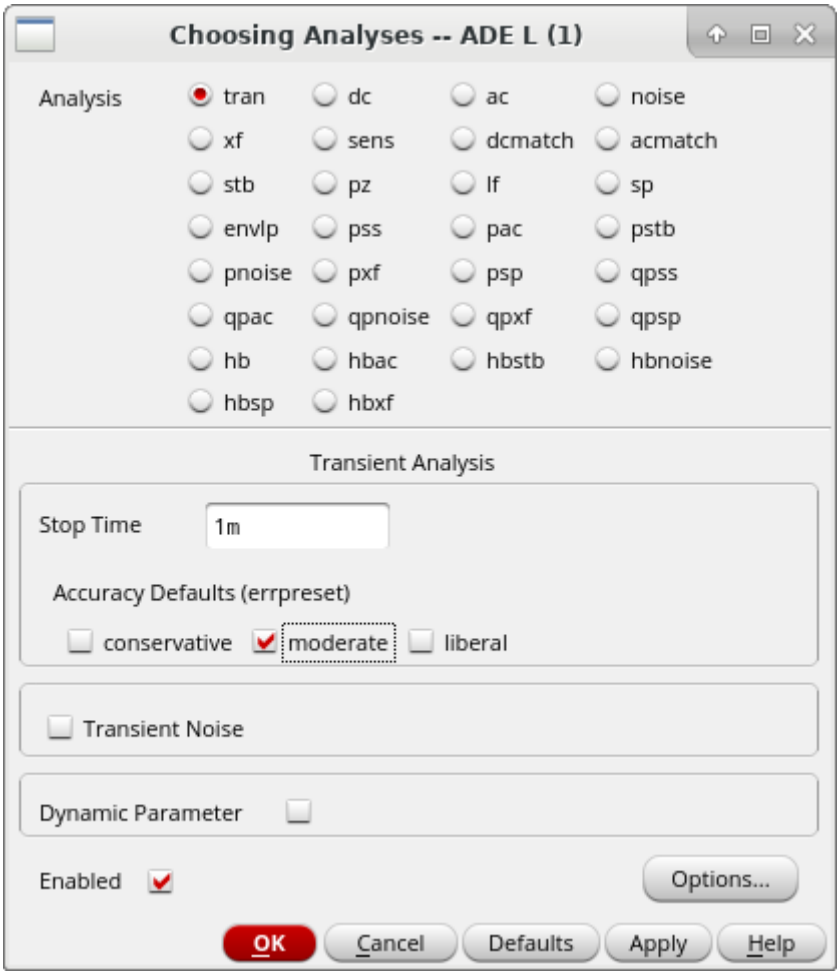
(4.5.2) We will now simulate the voltage divider circuit. From the Schematic Editor window, launch the Virtuoso Analogue Design Environment (ADE) window from **Menu: Launch → ADE L**. When you launch the ADE from a Schematic Editor window, the ADE will be pre-initialised for the design in that window, in this case the voltage divider circuit. This will be reflected in the title bar of the ADE window. (Alternatively, the ADE may be launched from the CIW, by **Menu: Tools → ADE L**. If you launch the ADE from the CIW you must manually setup the ADE for that design by selecting **Menu: Setup → Design…** and selecting the library, cell, and view).



< Figure 17. The Analogue Design Environment L (ADE L) Window. >

(4.5.3) Note: The ADE is the simulation environment for the Virtuoso toolset. The ADE is used to control all aspects of a schematic's simulation; including netlisting, starting and stopping a simulation, the types of analyses to run, which output waveforms (nets and nodes) to save and/or plot, the range of component values over which to simulate, circuit temperature and temperature variations, technology process variations, and much more. The ADE controls the settings for the **Spectre** simulator (Cadence's SPICE-class simulator).

(4.5.4) To simulate a design, we first need to choose an analysis type. From the ADE L window, go to **Menu: Analyses à Choose…** or use the '**Choose Analyses**' button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. Note: to see what an icon-button does, hover over the button and the name of the icon will appear.



< Figure 18. The Choosing Analyses Form. >

(4.5.5) Note: Many of the analyses options listed on the '**Choosing Analyses**' form are beyond the scope of this course. In this course, we will stick to the four fundamental SPICE analysis types – Transient Analysis (**tran**), DC Analysis (**dc**), AC Analysis (**ac**), and Stability Analysis (**stb**). From the Spectre user guide,

1. ***tran****. The Transient analysis computes the transient (time-domain) response of the circuit over a specified time interval.*

2. ***dc****. The DC Analysis finds the DC operating point or DC transfer curves of the circuit. To generate transfer curves, a parameter and a sweep range can be specified. The parameter can be a temperature, a design variable, a device instance parameter, or a device model parameter.*

3. ***ac****. The AC Analysis linearises the circuit about the DC operating point and computes the steady-state response of the circuit to a given small-signal sinusoidal stimulus. This analysis is useful for obtaining small-signal frequency response of a design.*

4. ***stb****. The Stability Analysis linearises the circuit about the DC operating point and computes loop gain, gain margin, and phase margin for a specific feedback loop or an active device. The stability of the circuit can be determined from the loop gain waveform.*

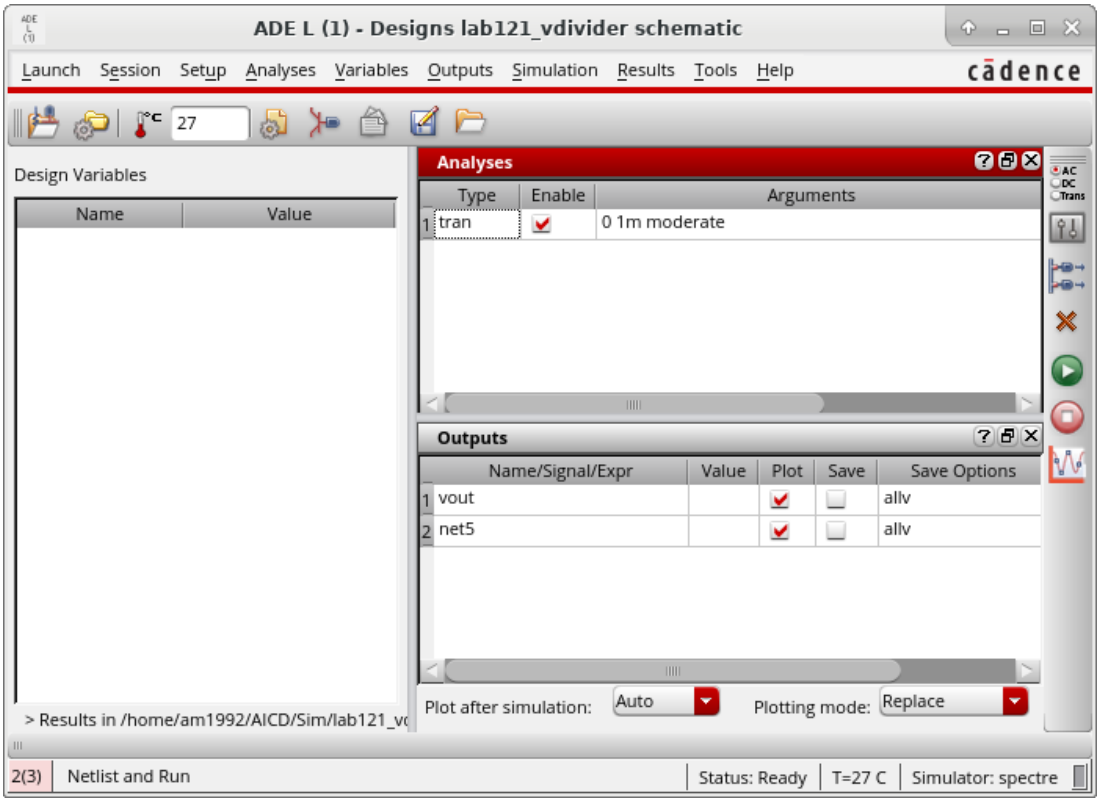
(4.5.6) We will start with a transient analysis. From the '**Choose Analyses**' form, select the '**tran**' option button and enter '1m' (one millisecond) in the 'Stop Time' text field. Set the 'Accuracy' option button to 'moderate'. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. See Figure 18. OK the form. And the **tran** analysis should appear in the 'Analyses' section of the ADE window.

(4.5.7) Note: The accuracy setting allows the designer to trade off the speed and accuracy of a transient analysis with a single setting: liberal, moderate, or conservative. The Liberal setting is fastest but has lowest accuracy. The conservative setting is the most accurate but also the slowest. Moderate is a compromise between liberal and conservative accuracy. From the spectre user guide…*The liberal setting is suitable for digital circuits or analog circuits that have only short time constants. The conservative setting is appropriate for sensitive analog circuits.* We will demonstrate the difference between these settings later in this lab.

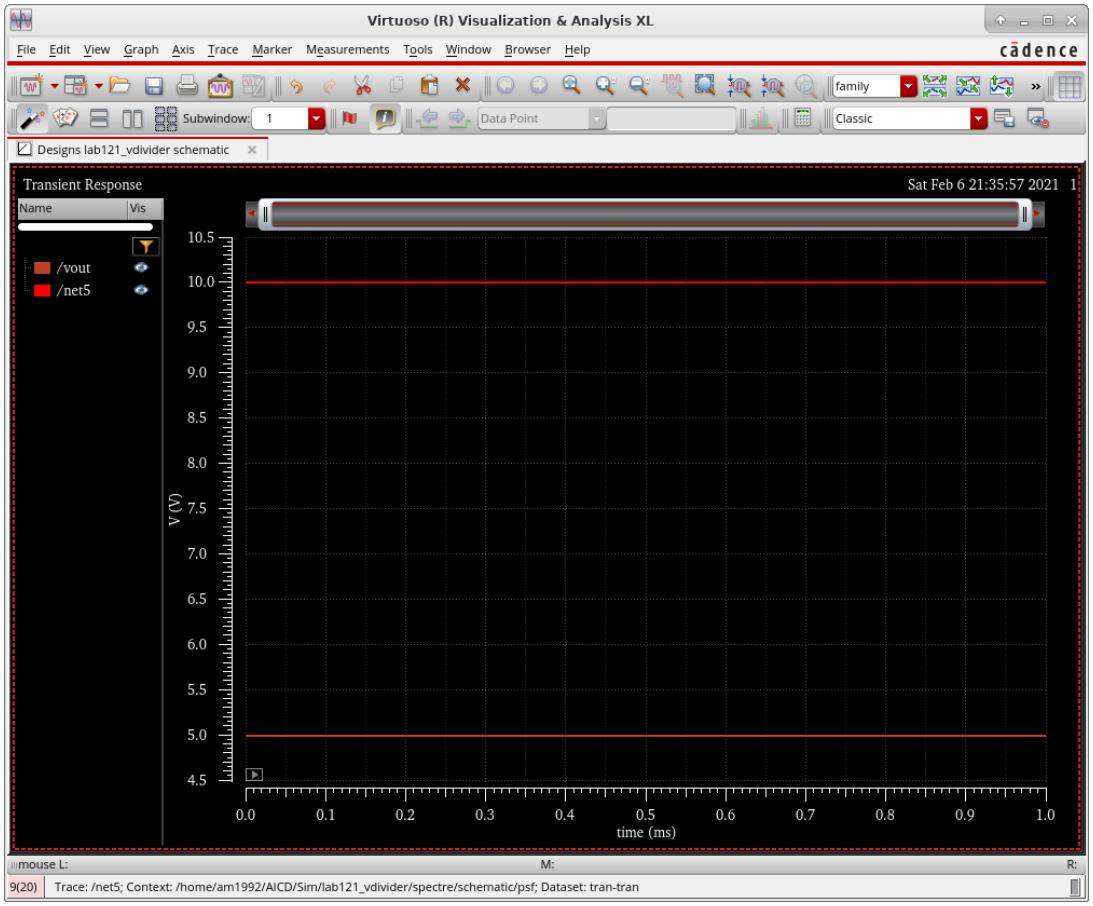
(4.5.8) We will now select the net voltages we want to plot. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** The schematic window will automatically become current. Select the voltage supply and output nets (blue wires) by clicking on them one after the other. The nets will change colour as you select them. **When you have selected the nets you want to plot, you must always press the 'ESC' key to exit the plotting mode of the ADE.** If you forget to escape the plotting mode, every time you click on the schematic window you will continue to (inadvertently) add more waveforms to your output window!

(4.5.9) Note: As you click on the nets (wires) to plot, observe that they appear in the 'Outputs' section of the ADE window. The name of the net is displayed, along with two checkboxes that determine whether the waveforms for those nets are to be saved and/or plotted. See Fig. 19.

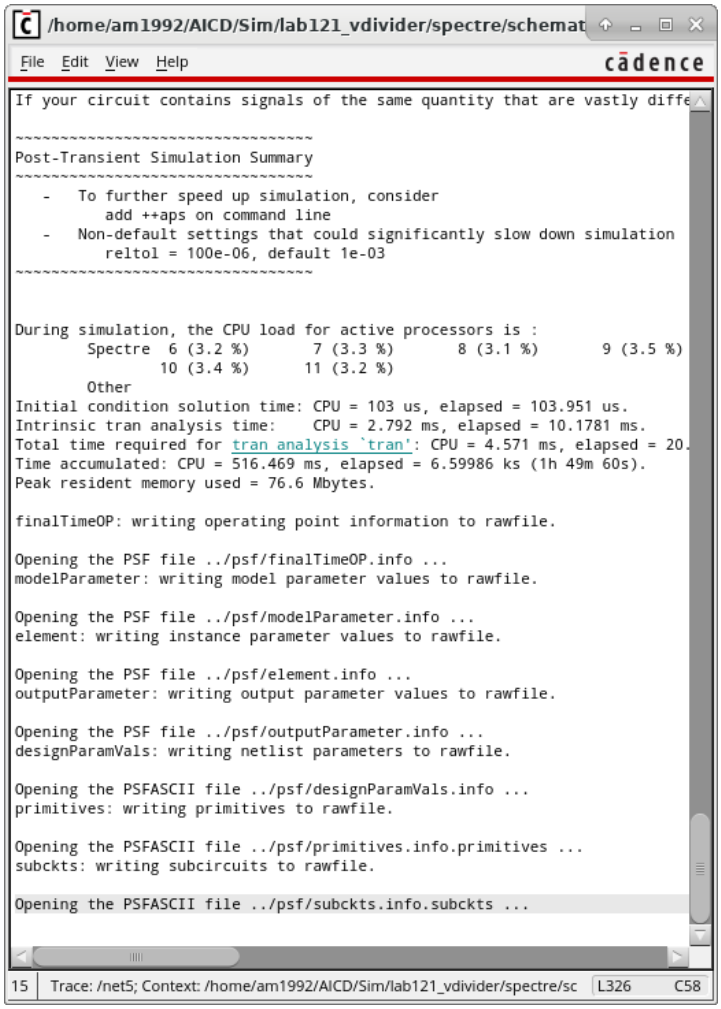
(4.5.10) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. If you followed all the instructions correctly, the simulation should run properly and both a textual output window as well as a waveform output window as shown in Figures 20 and 21 should open.



< Figure 19. The ADE with Analyses and Outputs set. >



< Figure 20. The Waveform Output Window. >



< Figure 21. The Textual Output Window. >

(4.5.9) Note: The operation of the resistive voltage divider is such that,

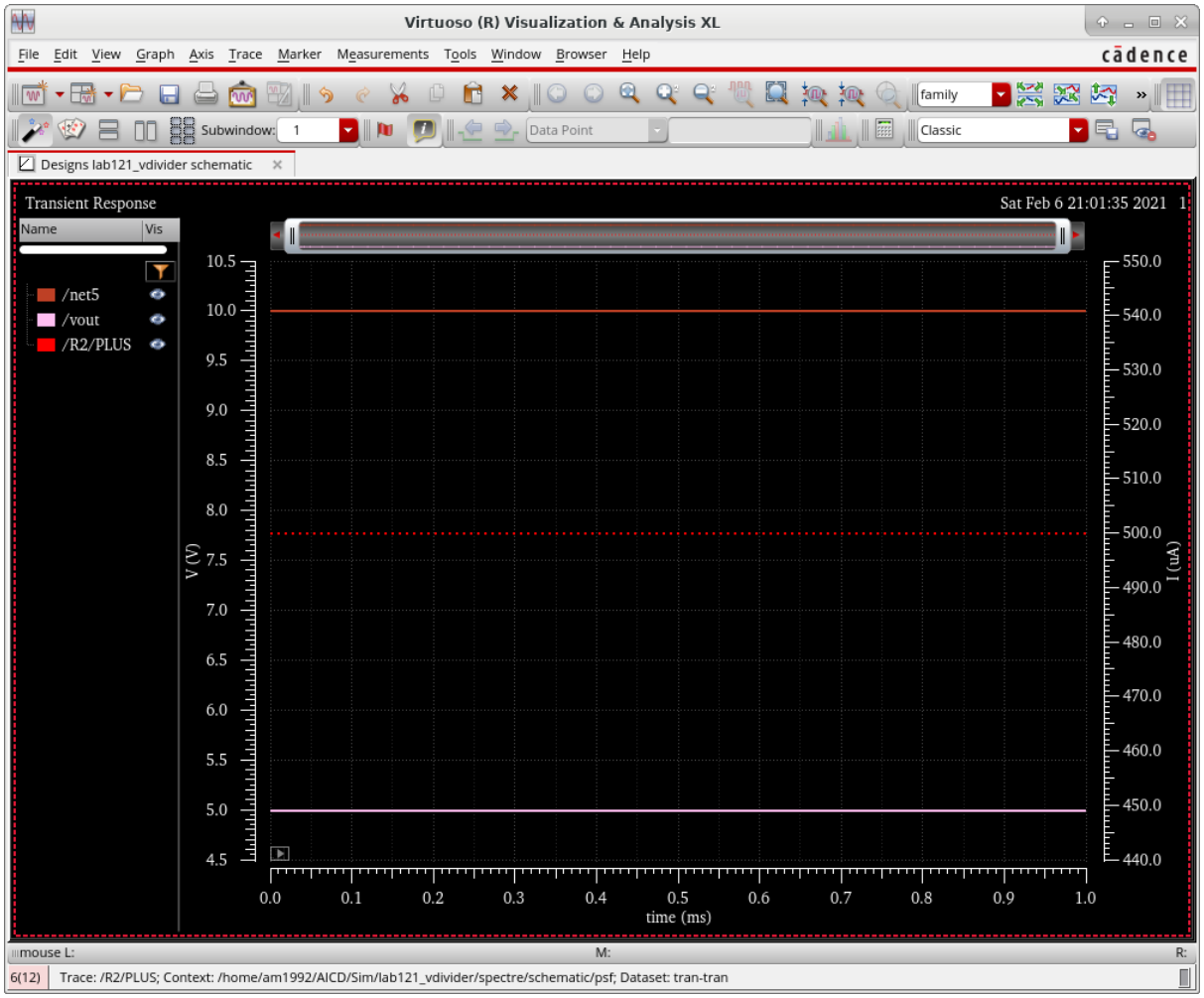
$$vout = vdc \times \frac{R1}{R1 + R2} = 5V$$

You can identify the supply (10V) and output (5V) waveforms by their net names on the left-side panel. Note that the output net is called 'vout' (or whatever name you chose) while the supply net name is the default net name 'netX' (where X is a number) since we did not name it. Since the voltage divider circuit uses only a DC voltage source, the waveforms do not vary with time.

(4.5.10) We will now add a branch current to the analysis output. Again, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode**.** If you want to save and plot the currents, you have to click a terminal **node** of a component (not the net/wire). Since, all components are in series, the current is the same at all nodes. Click on the terminal node of the upper most resistor (R2 in Figure 15 , yours may be different!). When you select a node, the node will turn pink. **Again, when you have selected all the nodes you wish to plot, you must always press the 'ESC' key to exit the plotting mode of the ADE.** The selected node should now be added to the 'Outputs' section of the ADE.

(4.5.11) Note: If in plotting mode, and you click on the body of a symbol, all terminal nodes of the component will be selected and added to your 'Outputs' window.

(4.5.12) A re-simulation is necessary, as by default, Cadence does not save currents. In the Analog Design Environment window, re-run the '**Netlist and Run**' command by clicking on the green play button. If the simulation runs properly, both a textual output window as well as a waveform output window will appear as shown in Figure 22 should open.



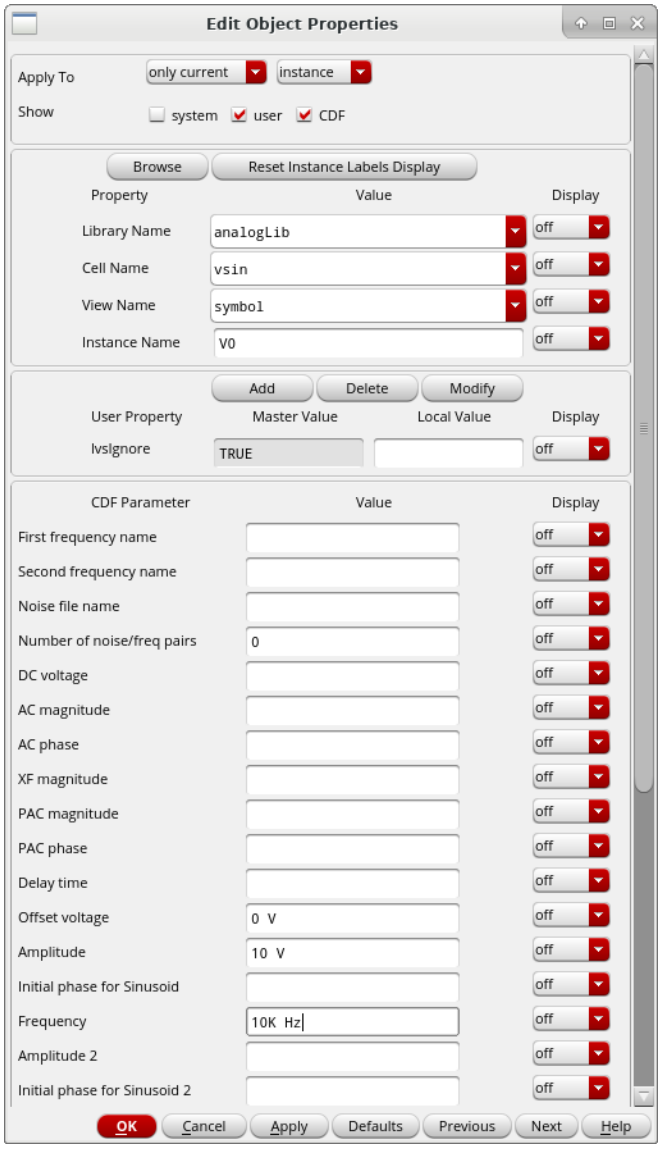
< Figure 22. The Waveform Output Window. >

(4.5.13) Note: The operation of the resistive voltage divider is such that,

$$i = \frac{vdc}{R1 + R2} = 0.5mA = 500\mu A$$

Identify the supply (10V), output (5V) waveforms, and node current (500uA) by their net/node names on the left-side panel. Note that the output node name by default is of the form '/component_name/terminal_name', in this case '/R2/PLUS' (i.e. the +ve terminal of resistor R2). Note that different waveform types/units (e.g. current uA, voltage V) will be shown on their own y-axis. In this case, voltage (V, left y-axis) and current (uA, right y-axis).

(4.5.14) In the schematic editor window, replace the DC voltage source (**vdc**) with a time-varying Sinusoidal voltage source (**vsin**). You can use **Menu: Edit à Delete**, the '**Delete'** icon-button on the toolbar, or the '**delete'** key on the keyboard to delete the **vdc** part and add a new instance of the **vsin** part. Or you can change **vdc** to a **vsin** component directly by editing its properties (**Menu: Edit à Properties à Objects**, or **Edit Properties** icon-button, or press '**Q**') by changing the cell name from vdc to vsin. The 'Add Instance' form should then change from a vdc form to a vsin form.



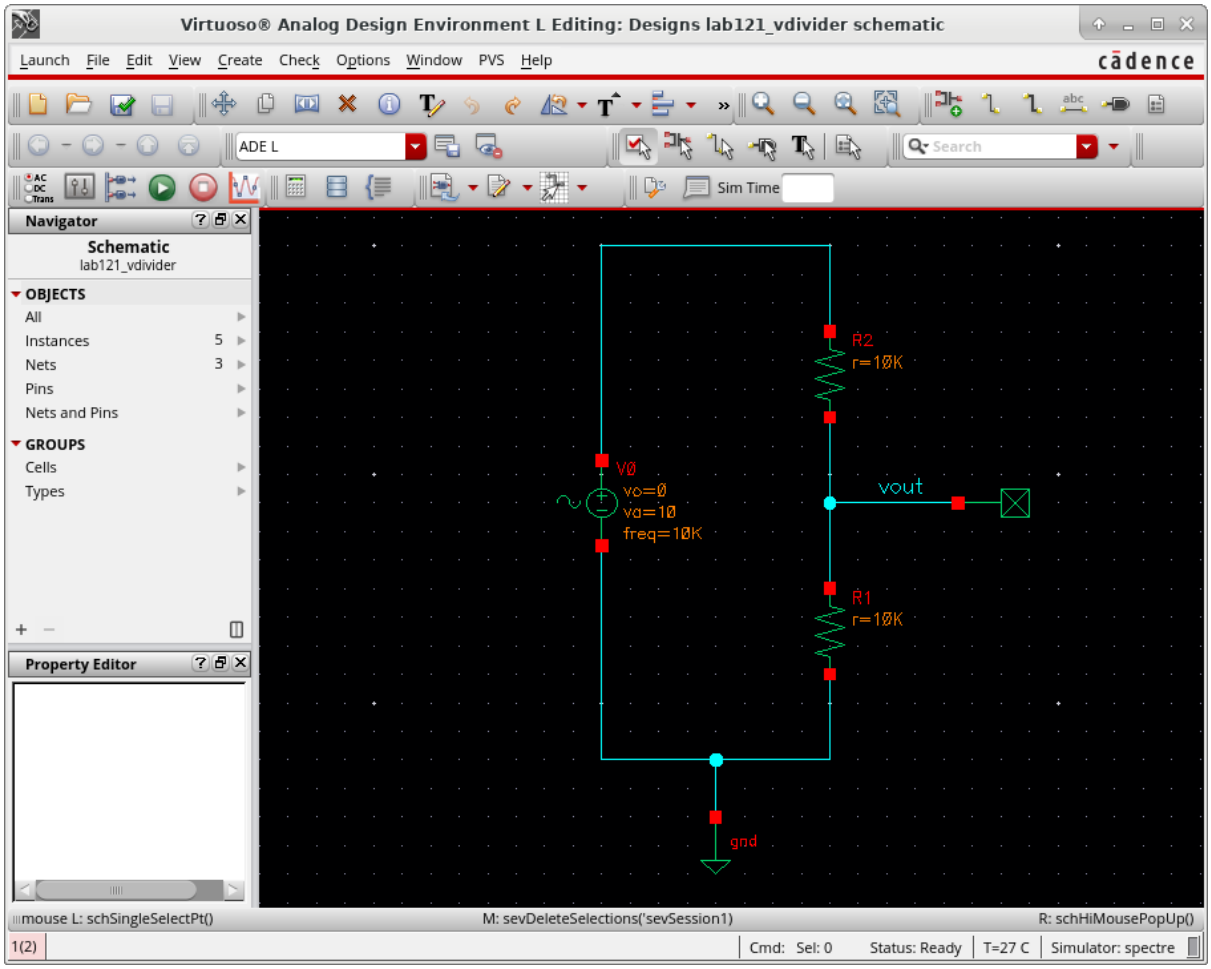
< Figure 23. The vsin Add Instance Form. >

(4.5.15) In the **vsin** instance form, set the attributes/parameters as follows: 'DC Voltage' = empty, 'Amplitude' = 10V, 'Offset' = 0V, and 'Frequency' 10K. See Figure 23. OK the form.

(4.5.16) Note: Most sources in a SPICE-based simulator, have several attributes for specifying a sinusoidal amplitude. Some specify the amplitude for use with a time-domain (transient) analysis, while others specify the amplitude for use with a small-signal (AC) analysis. In the case of the **vsin** component, make sure to set the 'Amplitude' attribute for the transient

analysis and not the 'AC Magnitude' attribute – these are not the same. The AC Magnitude is used for the various small-signal analyses of the ADE and is ignored by the transient analysis. (Similarly, the 'DC Voltage' attribute is used only by DC Analysis – it is ignored by the transient analysis. To give the source a DC bias use the 'Offset' attribute.)

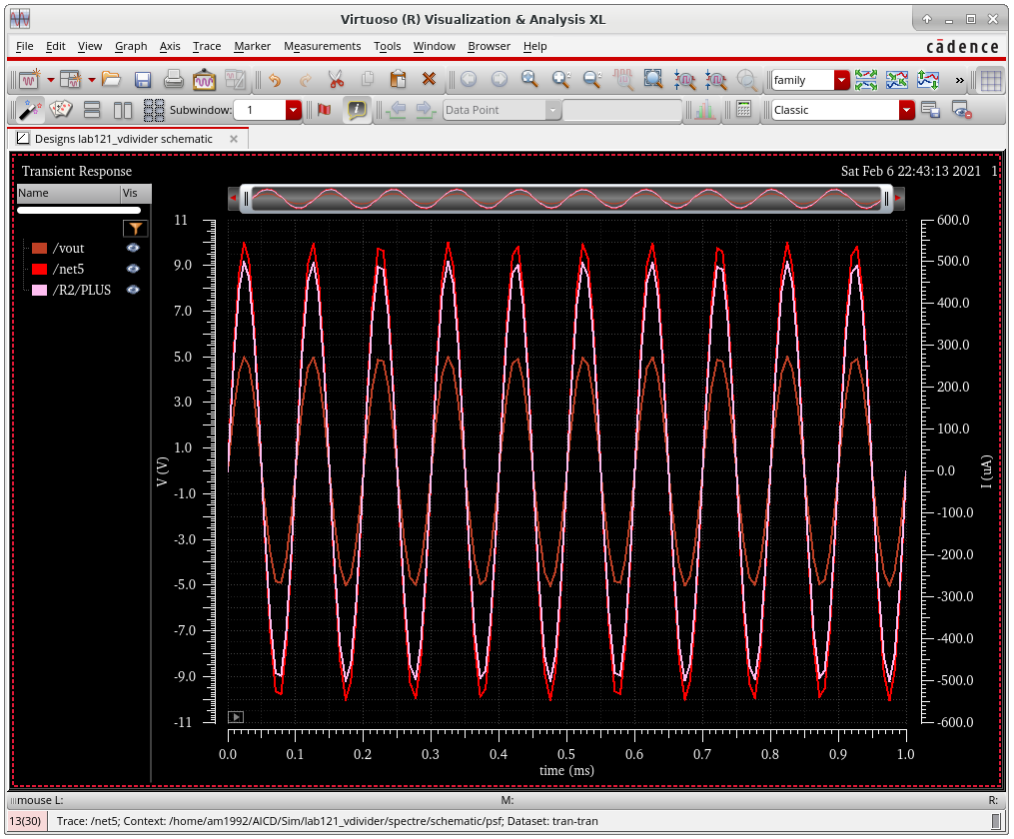
(4.5.17) **Check and Save** the schematic. There should be no errors or warnings. Use the CIW to diagnose any errors or warnings (if there are any).



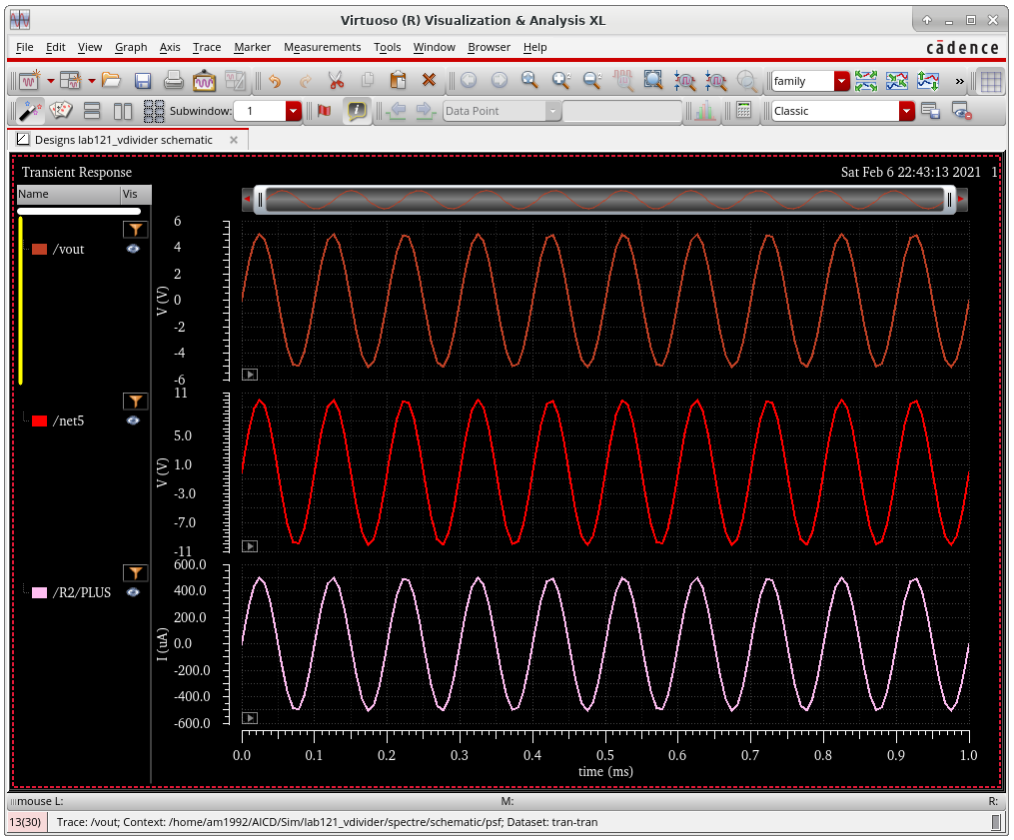
< Figure 24. The complete voltage divider circuit in the schematic editor with vsin source. Amplitude = 10V, Frequency = 10KHz, DC Offset = 0V. >

(4.5.18) We will now re-simulate the schematic. In the Analog Design Environment window, re-run the '**Netlist and Run**' command by clicking on the green play button. If the simulation runs properly, both the textual output window as well as the waveform output window will appear. See Figure 25. Note that this time the waveform varies with time over the duration of the analysis. The peak values of the voltage and current waveforms remain the same: supply net (=10V), output net (=5V), and node current (=500uA).

(4.5.19) You can separate the various plots in the waveform window to make it more readable. Right-click anywhere on the empty (black) space of the waveform window to invoke a context-menu, and select **Split Current Strip à trace**. Each trace will now be presented in its own space.



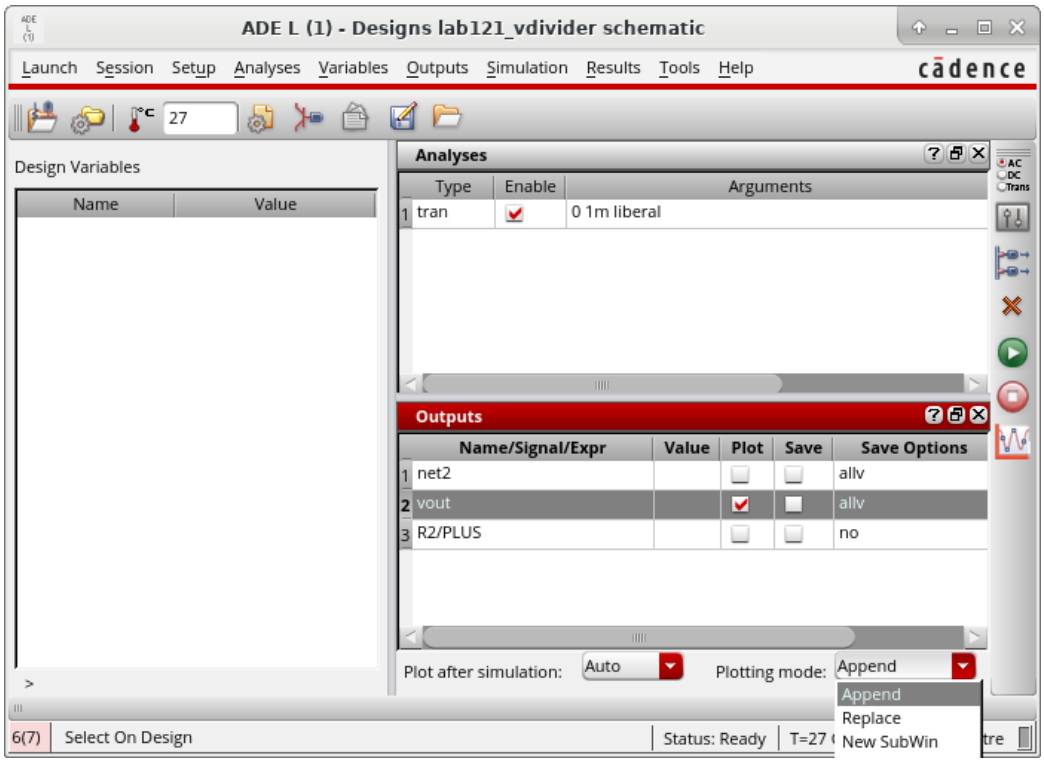
< Figure 25. The Waveform Output Window for voltage divider with vsin source. >



< Figure 26. The Waveform Output Window – with split traces. >

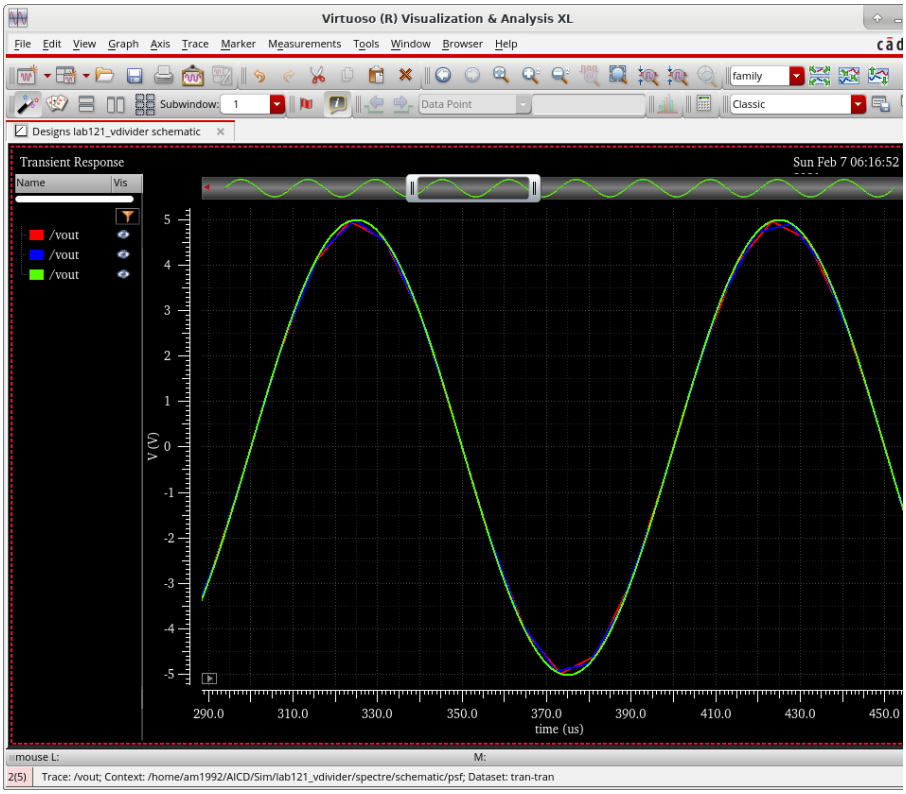
(4.5.20) Finally, re-run the transient analysis for the three different transient analysis accuracy options: liberal, moderate, and conservative. Observe, the output waveforms. Note that for the liberal option, the number of sample points that make-up the trace is fewer (i.e. larger time-step between solved points), leading to a piecewise linear (coarse, jagged) appearance. With the conservative option, the appearance of the trace is far smoother due to the high number of sample points (shorter time-step between solved points).

To plot all three waveforms in the same plot window, you will need to change the plotting mode of the ADE to 'Append'. On the bottom right corner of the ADE window there is a drop-down menu called '**Plotting mode**'. By default, this is set to replace; this means every time you run the simulation the contents of the plot will be overwritten with the new simulation results. By setting this to Append the previous simulation results are preserved in the plotting window and are displayed along the current simulation data. See Figure 27.

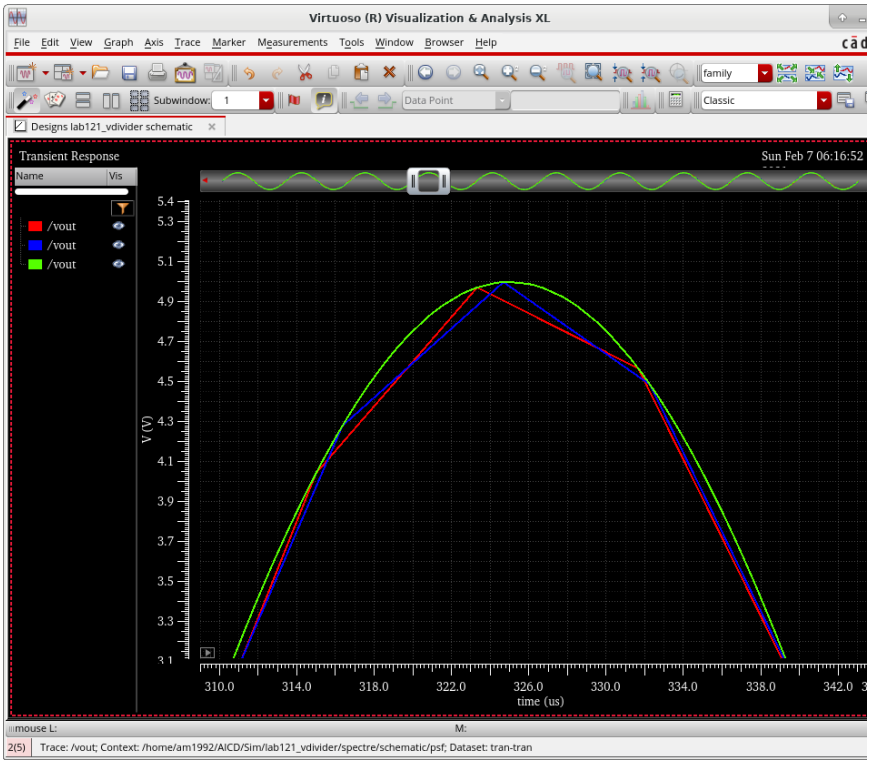


< Figure 277. ADE window showing plotting mode drop down menu (bottom-right) set to 'Append'. >

For a simple circuit like the voltage divider, the increase in simulation time is not noticeable so the trade-off between the liberal, moderate, and conservative accuracy setting is not immediately apparent. However, for larger and more complex circuits, a loss of accuracy for a faster simulation time may be worthwhile – at least during the early stages of the design flow where the focus may be on getting the design working rather than accuracy. In the latter stages of the design flow, high accuracy simulations may be preferred.



< Figure 288. The Waveform Output Window – traces with liberal (red), moderate (blue), and conservative (red) accuracy settings. >



< Figure 29. The Waveform Output Window – traces with liberal (red), moderate (blue), and conservative (red) accuracy settings. Close-up >

End of Lab!

T Tutorial Lab 12.2 (Formatively Assessed):

FET Terminal Characteristics and a Single-stage FET amplifier

Preliminary Matter: Setting up the Cadence Environment

Please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout before entering these commands into your terminal.

For this course, you will use the Cadence Virtuoso Design Environment (referred to simply as Cadence). Cadence is a Linux based software suite for designing full custom integrated circuits. In order to use Cadence you will first need to set-up the Cadence environment.

5. Open a terminal window and ensure that you are in your home directory using the following command:

```
cd ~
```

Note: 'cd' stands for change directory. The solitary tilde symbol (~) is a shortcut/alias for your home directory (/home/username).

6. Under your home directory, use the following commands to load the Cadence environment:

```
module use /eda/cadence/modules
module load course/EENGM6030
```

Note: The two 'module' commands load the required shell/Cadence environmental variables for this course (paths, names, aliases…) In particular, this includes a reference to the process design kit (PDK) used in this course, the GPDK180. You will need to do this every time you open a terminal, since the sourced environment only applies to the current terminal (until it's closed).

a. ***Note: This step only needs to be completed ONCE - If you already ran this script in lab 12.1, you don't need to run it again here.*** The first time you use Cadence, you will need to run the following command (script):

```
EENGM6030_setup
```

This will create a folder called AICD (short for Analogue Integrated Circuit Design) under your home directory. The AICD folder contains the files and directories needed to run Cadence. Running this setup script more than once will result in a warning message that the AICD folder already exists. It will then exit without making changes.

7. The AICD folder will be used for all your Cadence related activities for this course. **You should always make sure you are in your AICD folder before you launch Cadence.** To run cadence, use the following commands:

```
cd ~/AICD
virtuoso &
```

Note: The 'virtuoso &' command starts Cadence. The ampersand symbol (&) causes the Cadence process to run in the background freeing up the terminal for other commands the '&' symbol is a feature of the terminal's shell (bash) not Cadence.

1. Learning Objectives of this Lab:

From the previous lab, you should know how to:

- Start a remote Linux Lab session using the X2go remote desktop client.
- Set up the Linux terminal environment for Cadence and launch the Command Interpreter Window (CIW).
- Open the Library Manager. Understand the hierarchy of libraries, cells, and views. Identify the four main libraries used in this course (Designs, analogLib, basic, and gpdk180) and their uses. (cdsDefTechLib and subtrateLib can be ignored.)
- Create, edit, check, and save a new schematic using the Library Manager and the Virtuoso Schematic Editor. Instantiate components from the analogLib library. Edit component properties before and after instantiation. Move, drag, stretch, rotate, and flip components before and after instantiation. Delete components. Zoom and pan the schematic editor canvas. Understand the role of nets and nodes. Add wires and create wire names.
- List the four basic SPICE analysis types (transient, DC, AC, Stability).
- Launch the Analogue Design environment (ADE). Set up a transient analysis. Simulate a schematic using the ADE. Plot the voltage and current waveforms at given nets and nodes.

In this lab, you will:

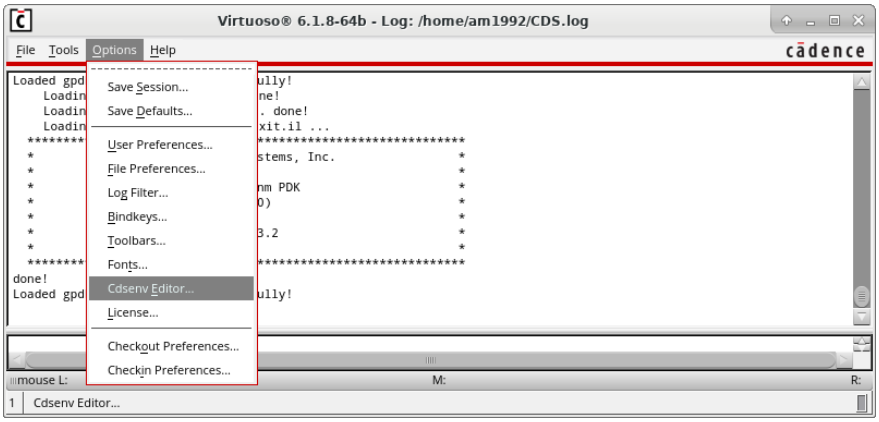
- Instantiate and characterise an NMOS FET from the GPDK180 library.
- Create and simulate a simple resistor-loaded single-stage NMOS FET voltage amplifier.
- Address the inadequacies of using a resistive load, and implement a current-source loaded NMOS FET voltage amplifier.

2. Cadence Documentation, Help, and Licensing

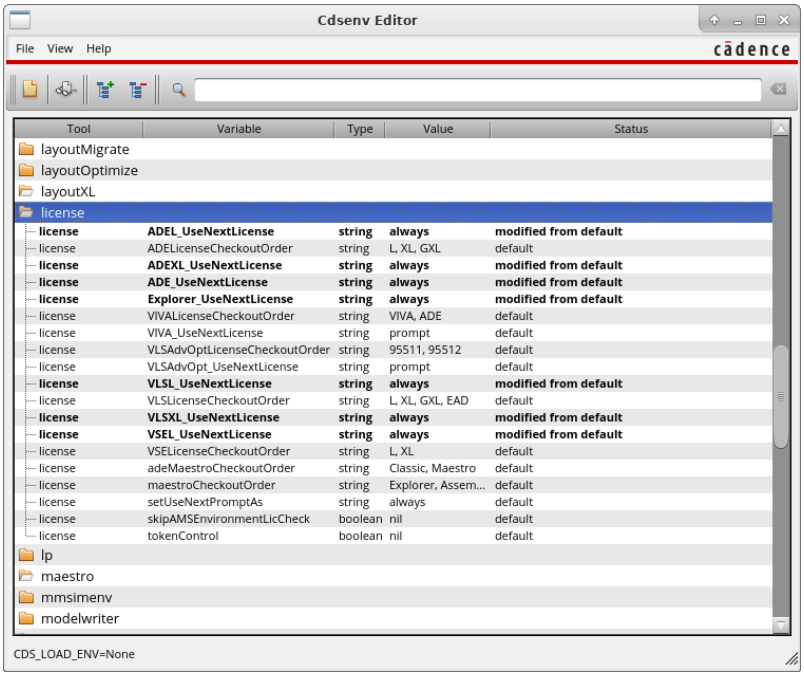
Local and online help is available from the help menus (**Menu: Help → User Guide** or by pressing **F1**), or by typing 'cdnshelp &' at the command prompt (assuming you have setup your terminal environment).

PLEASE NOTE: WHEN LAUNCHING A TOOL IN THE CADENCE DESIGN ENVIRONMENT YOU MAY SOMETIMES GET A WARNING ABOUT A LICENSE BEING UNAVAILABLE FOR THE TOOL, AND REQUESTING PERMISSION TO OBTAIN A HIGHER-TIERED LICENSE. YOU SHOULD SELECT 'ALWAYS' TO THESE MESSAGES.

If you accidently deny a license request, or you cannot access a license, you will need to modify the Cadence environment licence/permissions from the CIW. From the CIW menu go to **Menu: Options → Cdsenv Editor**. Once in the environment editor, go to the license folder, and make sure all licenses (in bold) are set to '**always'** – if any entries are listed as '**never**' you will need to manually change/retype them to '**always**'.



< Figure X.1. The Cadence Environment Menu Option from the CIW Window. >



< Figure X.2. The Cadence Environment Editor. Note all licenses should be set to 'always' >

3. Getting Started with Cadence:

For a refresher/introduction on how to start a Cadence session please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout.

3.1 Dealing with Lock files

PLEASE NOTE: IF A SCHEMATIC ISN'T PROPERLY CLOSED DOWN (FOR EXAMPLE, DUE TO A DROPPED REMOTE CONNECTION), THE SCHEMATIC MIGHT BECOME EDIT LOCKED. IF THIS HAPPENS YOU WILL NOT BE ABLE TO OPEN THE FILE FOR EDITING. YOU WILL ONLY BE ABLE TO OPEN THE FILE FOR READING UNTIL YOU REMOVE ANY/ALL LOCK FILES.

There are two ways you can unlock files:

(1) The first is to manually delete any edit lock files. To do this, go to your '~/AICD/Designs' folder and find the folder representing the schematic e.g., for lab1 this would be '~/AICD/Designs/lab121_vdivider/schematic' if this is what you called your schematic. Delete all the .cdslck lock files in this folder and you should be able to open your schematic for editing.

(2) You can also automatically release edit locks with the command:

```
clsAdminTool -are .
```

To use this command, you have to be in your AICD folder (the full-stop is shorthand for the current directory path). If you are not in your AICD folder, you have to specify the path (relative or absolute) to your AICD directory.

Since this is a Cadence command, you have to ensure that you have set up the terminal environment for Cadence (in particular, the modules must be loaded as in step 2, 'Preliminary Matter'). It is best to use the terminal window that you used to start Cadence, since the environment was already set up. Note that you will not be able enter any commands into your terminal if you did not start cadence with the '&' character. By example, to release any lock files after opening a new terminal, the full command set would be,

```
cd~
module use /eda/cadence/modules
module load course/EENGM6030_2021
clsAdminTool -are AICD
```

For more information on this command, use
`clsAdminTool -help`

4. Circuit Simulation Exercises: Part 1

With guidance, you will:

- Instantiate and parameterise an NMOS FET from the GPDK180 library
- Create and simulate a test circuit for the evaluation of the NMOS FET V_{GS}-I_{DS} and V_{DS}-I_{DS} terminal characteristics.

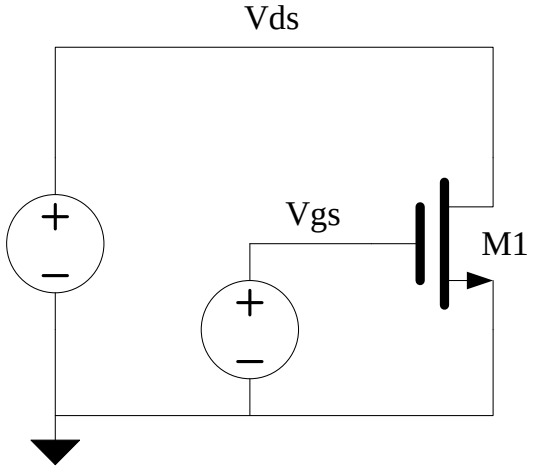
The ideal MOSFET is typically modelled as a Voltage-Controlled Current-Source (VCCS); by controlling the voltage at the input—the gate-to-source voltage V_{GS}—we can control the current that flows at the output—the drain-to-source current I_{DS}.

Here, a distinction must be made between the large-signal nonlinear relationship of V_{GS} and I_{DS}, and the small-signal (idealised, linearised) relationship of the same variables. The small-signal transconductance gm of a MOSFET relates to its large-signal terminal characteristics by $\mathrm{gm}(V_{GS}) = \mathrm{d}I_{DS}/\mathrm{d}V_{GS}$. Further, the non-ideal MOSFET deviates from the VCCS model by a dependency of the output current I_{DS} on the output voltage V_{DS}. This non-ideal behaviour is modelled in the small-signal domain by a parasitic resistance r_o between the output terminals of the VCCS, where $r_o = \mathrm{d}V_{DS}/\mathrm{d}I_{DS}$.

In this lab, we will analyse the MOSFET in the large-signal domain; that is, we will consider the input-output relationship of the MOSFET for a large, full-scale swing of the input signal. We will characterise – through simulation – the V_{GS}-I_{DS} and V_{DS}-I_{DS} large-signal terminal behaviour of an NMOS device from the GPDK180 library. In the following lab (12-3) we will relate these large-signal characteristics to the small-signal model of the NMOS device.

4.1 NMOS Terminal Characteristics

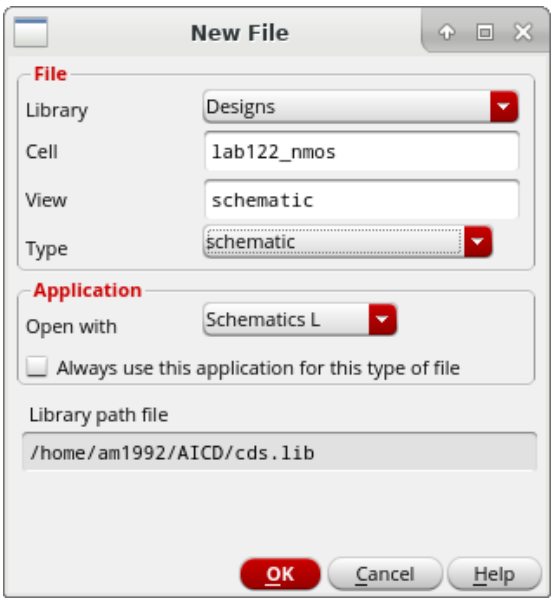
(4.1.1) Create the schematic for the NMOS testbench circuit in Figure 1. A summary of the circuit's operation is given in section 4.1 of the background material.



< Figure 30. Circuit for Large Signal Transconductance Terminal Characteristics Evaluation. Reproduced from Figure 14, section 4.1 of the background material. >

(4.1.2) From the Library Manager window, select the 'Designs' library and choose **Menu: Files à New à Cell View** to invoke the New File form (Figure 2).

In the 'Cell' text field, choose a meaningful cell name (e.g., lab122_nmos). In the 'View' text field, type 'schematic' as the view name and from the 'Type' drop down menu choose 'schematic' as the view type. In the 'Application' section of the dialog box, choose 'Schematics L' from the 'Open with' drop down menu. OK the form.



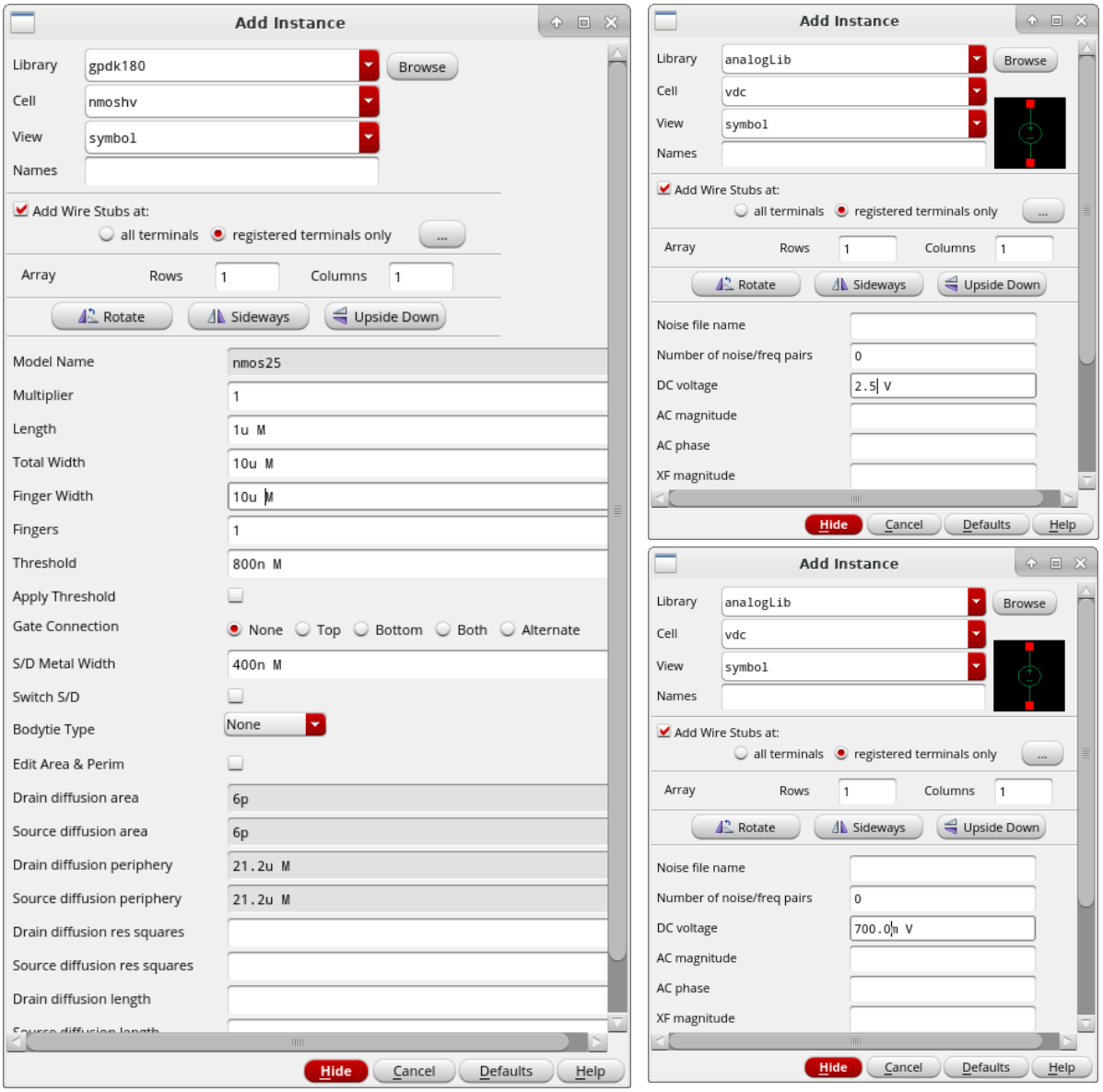
< Figure 31. The New File Form >

(4.1.3) You should be presented with a blank **Virtuoso Schematic Editor L** schematic entry window for the new cellview. This cellview will not be saved in the Library Manager until you check and save it in the Schematic Editor. From the Schematic Editor window, choose **Menu: File à Check and Save**. (Or you can also **Menu: Check à Current_Cellview** followed by **Menu: File à Save** to perform the same command.)

(4.1.4) In your schematic editor window, construct the circuit given in Figure 1. Instantiate and place two DC voltage sources (**vdc**) from the analogLib library, a ground connection (**gnd**) from the analogLib library, and one high-voltage NMOS FET (**nmoshv**) from the GPDK180 library. Set the DC supply voltage to 2.5V (**DC Voltage** = *2.5*), set the gate-to-source voltage to 0.7V (**DC Voltage** = *0.7*), and dimension the NMOS device 10u/1u (**width** = *10u*, **length** = 1u). Leave all other attributes unchanged.

*Recall, to add a new part you can use **Menu: Create → Instance...**, the '**Create Instance**' icon-button on the toolbar at the top of the schematic window, or the keyboard shortcut '**I**'. You can also right-click anywhere in the empty space of the design canvas to invoke a context-menu and select the **Add Instance…** command from there. Refer to lab 12-1 for a refresher on how to add components to the schematic (sec. 4.2 Adding Components to the Schematic).*

*You can set the component properties at instantiation from the Add Instance form before placing them on the design canvas. Alternatively, you can modify the component properties after it has been placed on the design canvas, by first selecting the item and then by right-clicking on the item and selecting 'Properties…' You can also use **Menu: Edit → Properties → Objects**, the '**Edit Properties**' icon-button from the toolbar, or the keyboard shortcut '**Q**'. You can then modify the required attribute(s) and Apply/OK the change.*



< Figure 32. The nmoshv (GPDK180) and vdc (analogLib) Add Instance Forms. >

(4.1.5) Note that there are low-voltage MOS devices and high-voltage MOS devices in the GPDK180 library. In these labs, we will only be using the **nmoshv** n-type MOSFET and the **pmoshv** p-type MOSFET models. These are four-terminal devices (gate, source, drain, and body/bulk). Note the 'hv' suffix stands for high voltage. This is relative! For our purposes, high voltage means a maximum V_{DS} of 2.5V. This is in contrast to low-voltage devices with a sub-1V V_{DS}. (The high voltage behaviour is achieved by using a thicker gate oxide level.) Both the **pmoshv** and **nmoshv** devices have a threshold voltage V_T of ~0.5V.

When adding the **nmoshv** or **pmoshv** from the GPDK180 library, there are five parameters under the designer's control that set their operation and characteristics: the **length**, **total width**, **finger width**, **number of fingers**, and **multiplier**. This is typical of most MOSFET process development kits (PDKs). We address these briefly below:

1. The **length** attribute specifies the channel length of the device. For the GPDK180 library, the minimum size of the channel (its feature size, technology node, or process node) is 180nm. You can specify any size above this value (to the nearest 5nm) but not below. In these labs, since we are not operating at particularly high-frequencies, we choose not to use the smallest node – we will always default to a value of 1um. This ensures the first-order design equations we used to describe MOSFET operation (for the most part) provide parity with the simulations.

2. The **total width** attribute specifies the channel width of the device. For the GPDK180 library, the width may be set in the range 400nm to 50um (to the nearest 5nm). The **length** (L) attribute and the **total width** (W) attribute together specifies the *effective* width-to-length (W/L) ratio of the device.

3. The **finger size** and **number of fingers**. When laying out a MOSFET with a high W/L ratio, the designer has two options regarding the **shape of the gate**. Rather than a single, wide polysilicon run forming the gate (i.e., a single 'finger'), the gate is distributed into several equally spaced parallel fingers. This approach allows for better performance at high-frequencies (due to the reduction in gate parasitics and IR drop along the length of the gate) and greater layout flexibility (e.g. compactness) when using very wide devices. The effective W/L ratio (total-width/length) stays the same (though some aspects of circuit performance will be affected).

   For the GPDK180 library, the designer specifiers the number of fingers and, either, the total-width or the finger-width. Whichever attribute is not specified will automatically adjust according to the relation:

$$finger\ width = \frac{total\ width}{number\ of\ fingers}$$

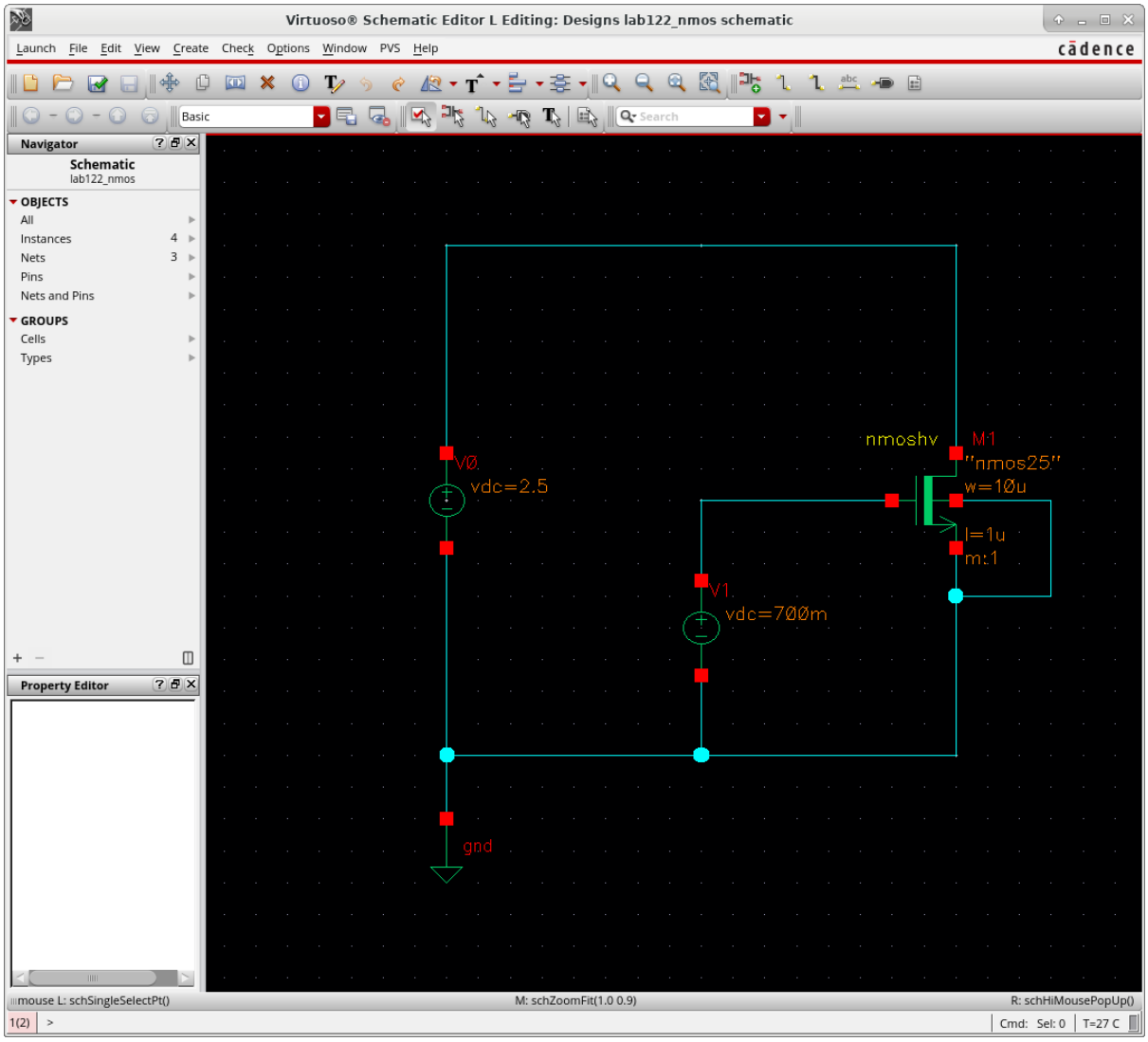
   In some PDKs, fingers are referred to as **stripes.** We will only use single fingers/stripes in this course.

4. The **multiplier** attribute allows several identical devices to be connected in parallel. This is used when the W/L ratio is high. For example, a device with a 50u/1u width/length ratio could also be constructed from five 10u/1u devices connected in parallel. In this case: length = 1um, total width = 10um, multiplier = 5. The multiplier is particularly useful for those cases where we want to scale current sources – for example in a current mirror. In such cases, better matching will be obtained by using multiple identical single-gate devices rather than one large device. We will make extensive use of the multiplier in the coming labs.

(4.1.6) With the components placed, you can now add wires to connect them together. You can use **Menu: Create → Wire (narrow)**, the '**Create Narrow Wire**' icon-button on the toolbar, or the keyboard shortcut '**W**' to start the wire command (see Figure 4).

*Recall that you can drag, move or stretch the components and wires after they have been placed on the design canvas. Left-click-and-hold the mouse button and drag the component or wire, or select the component and use the **stretch** or **move** commands. You can also rotate and flip the components before placing them using the **rotate**, **sideways**, and **upside-down** buttons on the add instance form. Or you can rotate/flip the components after placing them by selecting the component and using the **rotate** or **flip** commands, or by using **F3** with the **move/stretch** commands.*

(4.1.7) Check and save your schematic using the **Check and Save** command! Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.



< Figure 33. The wired NMOS testbench circuit in the schematic editor. >

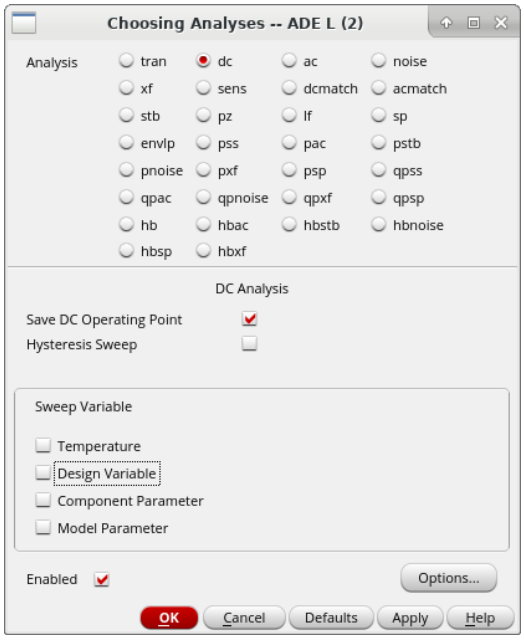
(4.1.8) NOTE: FOR AN NMOS DEVICE, YOU MUST ALWAYS CONNECT THE BODY/BULK TERMINAL TO THE MOST NEGATIVE NET (WIRE) OF THE CIRCUIT. IN FIGURE 4, THIS IS THE GROUND TERMINAL. FOR A PMOS DEVICE (NOT SHOWN) THE BODY/BULK TERMINAL MUST ALWAYS BE CONNECTED TO THE MOST POSITIVE NET (WIRE) OF THE CIRCUIT, USUALLY THE SUPPLY VDD. THIS IS NECESSARY TO ENSURE THAT THE CHANNEL WILL BE PROPERLY ESTABLISHED. THERE IS AN EXCEPTION TO THIS THAT WE WILL COME TO LATER.

4.2 Simulating the NMOS FET DC operating Point

(4.2.1) You should now have the completed schematic of the NMOS testbench circuit as shown in Figure 4. It should check and save without any errors or warnings.

(4.2.2) We will simulate the NMOS testbench using a DC Analysis. From the Schematic Editor window, launch the Analogue Design Environment (ADE) from **Menu: Launch à ADE L**.

(4.2.3) From the ADE window, go to **Menu: Analyses à Choose…** or use the **Choose Analyses** button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. On the form, select the '**dc**' option button. Check the 'Save DC Operating Point' box. OK the form.



< Figure 34. The Choosing Analyses Form. Only DC Operating Point selected. >

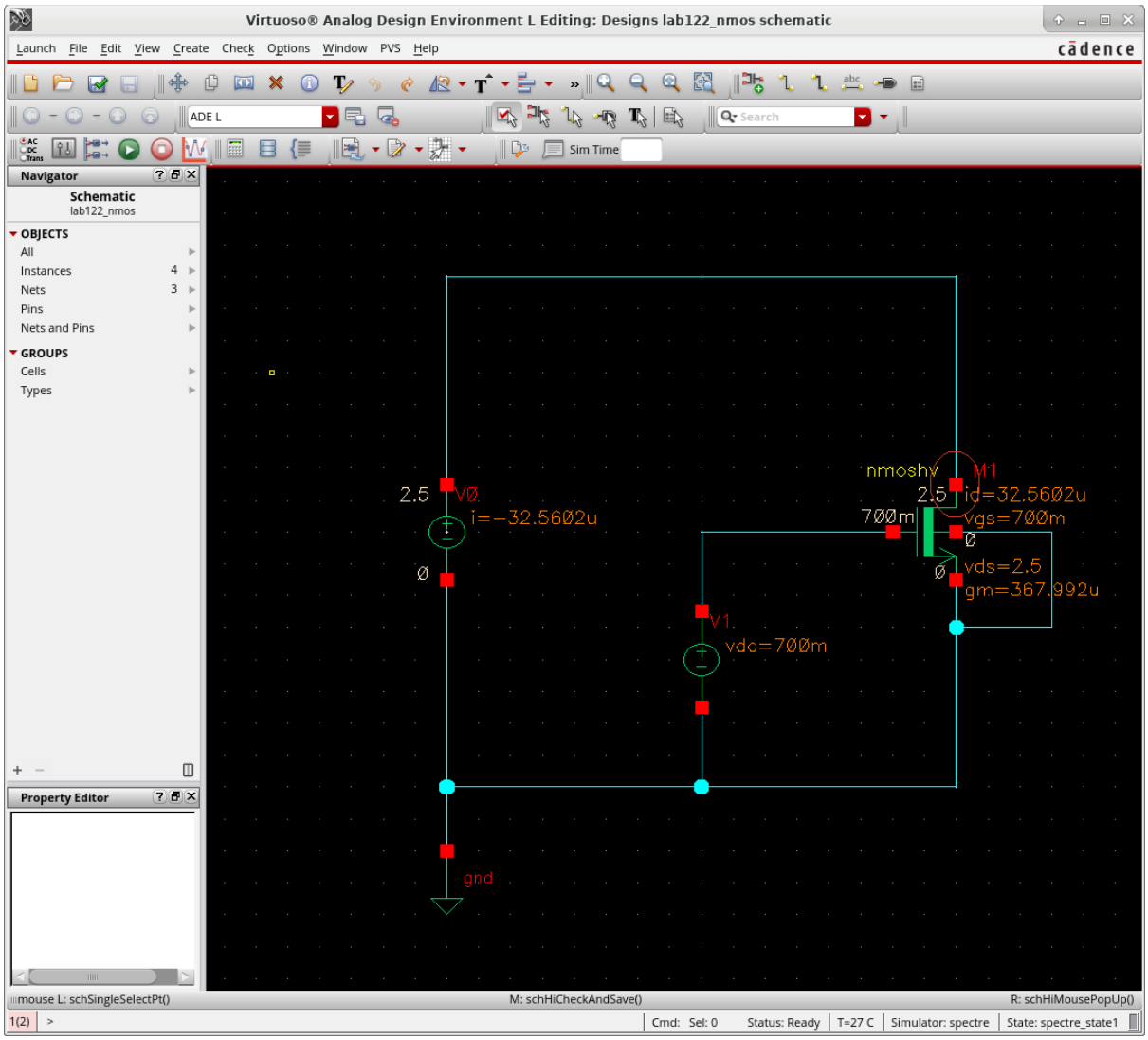
The DC operating point calculates the various node voltages and currents around the circuit, as well as various NMOS device parameters such as gm, IDS, VGS, VDS and more. The DC operating point will be calculated from the DC supply voltage (2.5V) and the NMOS gate-to-source voltage (0.7V).

(4.2.4) At this stage we are only simulating for the DC operating point. Simulating the DC operating point does not produce an output waveform, so we do not need to select any outputs to be plotted.

(4.2.5) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. The text output window should appear but there will be no waveform window.

(4.2.6) From the ADE window choose **Menu: Results à Annotate à DC node voltages…** If you now look in the schematic window, you should see the node voltages annotated directly on the schematic. Again, from the ADE window, choose **Menu: Results à Annotate à DC operating points…** You should now see (along with the node voltages) additional DC operating point information annotated on the schematic including the DC bias currents, NMOS device transconductance gm, IDS, VGS, and VDS. See Figure 5.

Note: You can also turn annotations on and off by right-clicking on the canvas in the schematic editor window and selecting **Right-Click: Annotations…** There are number of annotations to choose from.



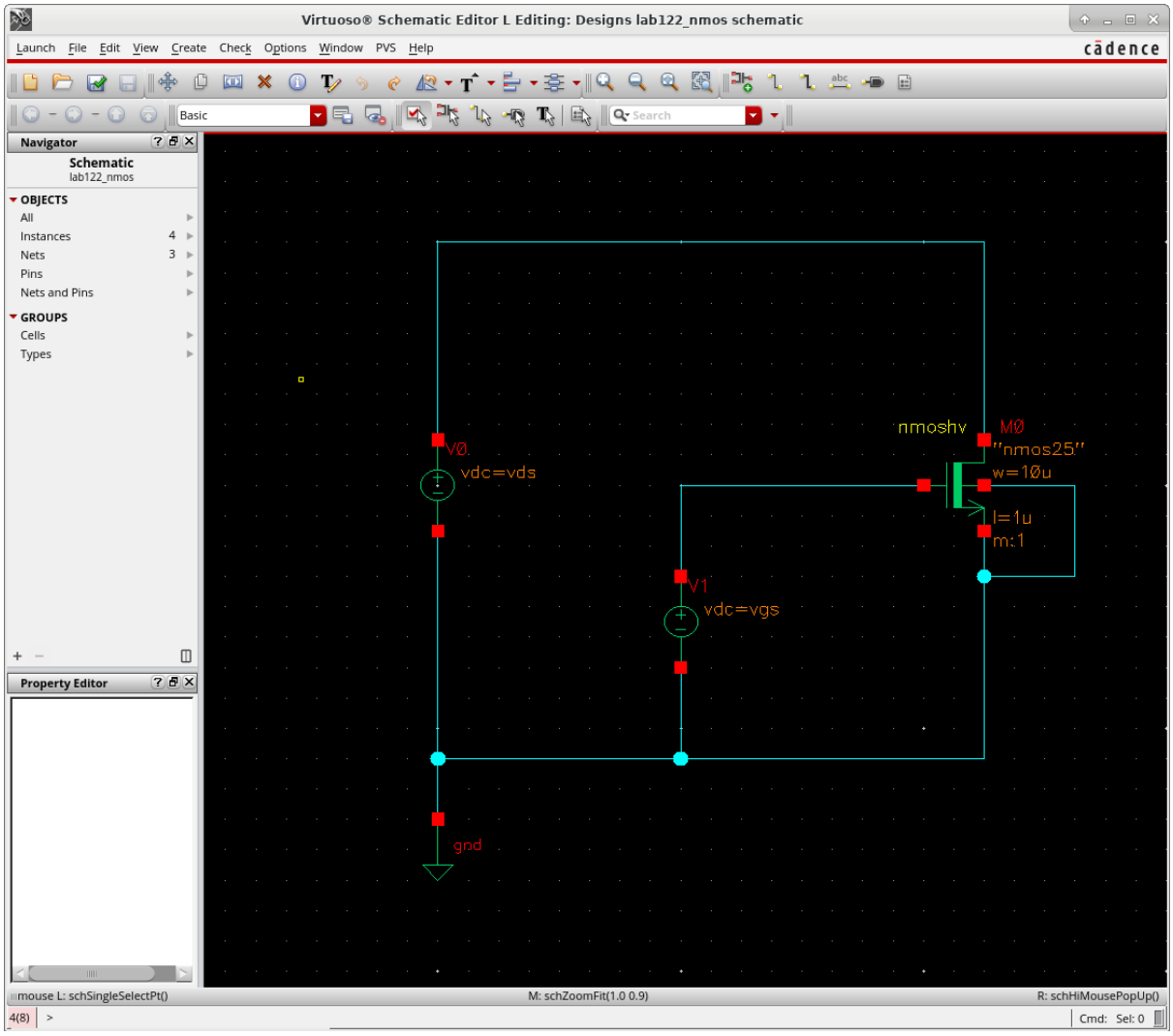
< Figure 35. DC Analysis – Save DC Operating Point. The wired NMOS testbench circuit with annotated DC node voltages and DC operating points. >

4.3 Parameterising the NMOS FET for a Swept-DC Analysis of V_{GS} vs. I_{DS}

(4.3.1) We will now simulate the testbench circuit using a swept-variable DC Analysis, aka a DC-sweep. In order to do this, we first need to modify the circuit slightly. Instead of giving each voltage source a fixed value, we will specify a parameter variable (aka a design variable).

(4.3.2) In the schematic window, modify both the DC voltage sources (**vdc**). For the DC supply voltage, change the 2.5 V value to the string vds (**DC Voltage** = *vds*) and for the gate-to-source voltage change the 0.7 V value to the string vgs (**DC Voltage** = *vgs*). This gives us two design variables – vdc=vds and vdc=vgs. **Check and save** your schematic after any modifications.

Note: you may need to annotate the component parameters so you can see the design variable names. From the ADE, choose **Menu: Results → Annotate → Component Parameters…** or from the schematic editor window, choose **Right-Click: Annotations → Component Parameters**.



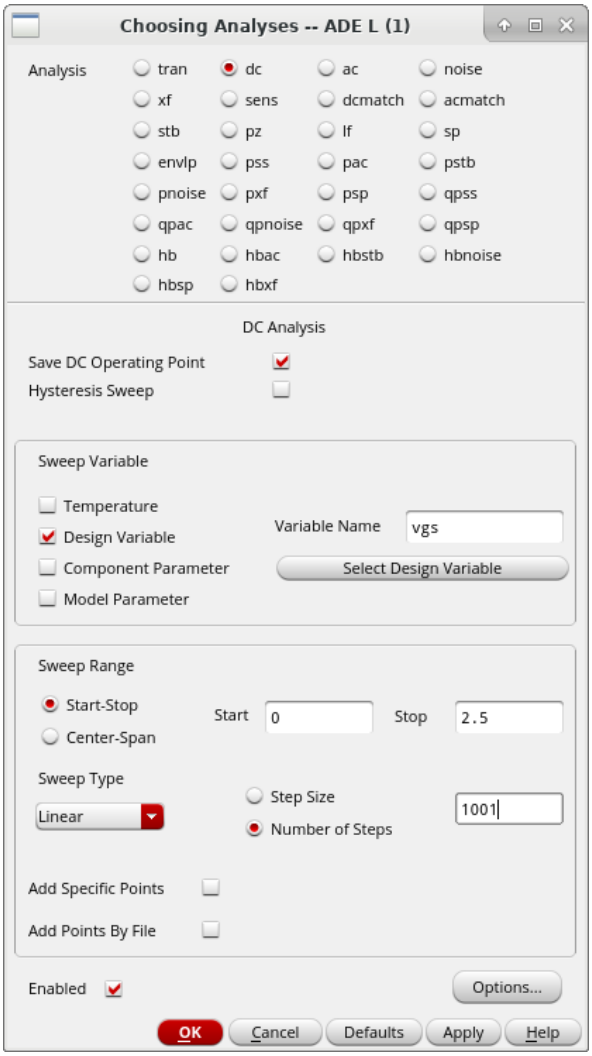
< Figure 36. The wired NMOS testbench circuit with dDesign Variables for the DC supply voltage (vdc=vds) and the DC gate-to-source voltage (vdc=vgs). >

In Figure 7, note that the DC supply voltage and DC gate-to-source voltage are now set using a design variable – vds and vgs respectively. During a simulation, the design variable(s) can be incrementally changed (according to a specified rule) and the simulation run for each and every increment. This allows a circuit to be simulated (swept) over a range of parameter values – and not at just one fixed bias/operating point.

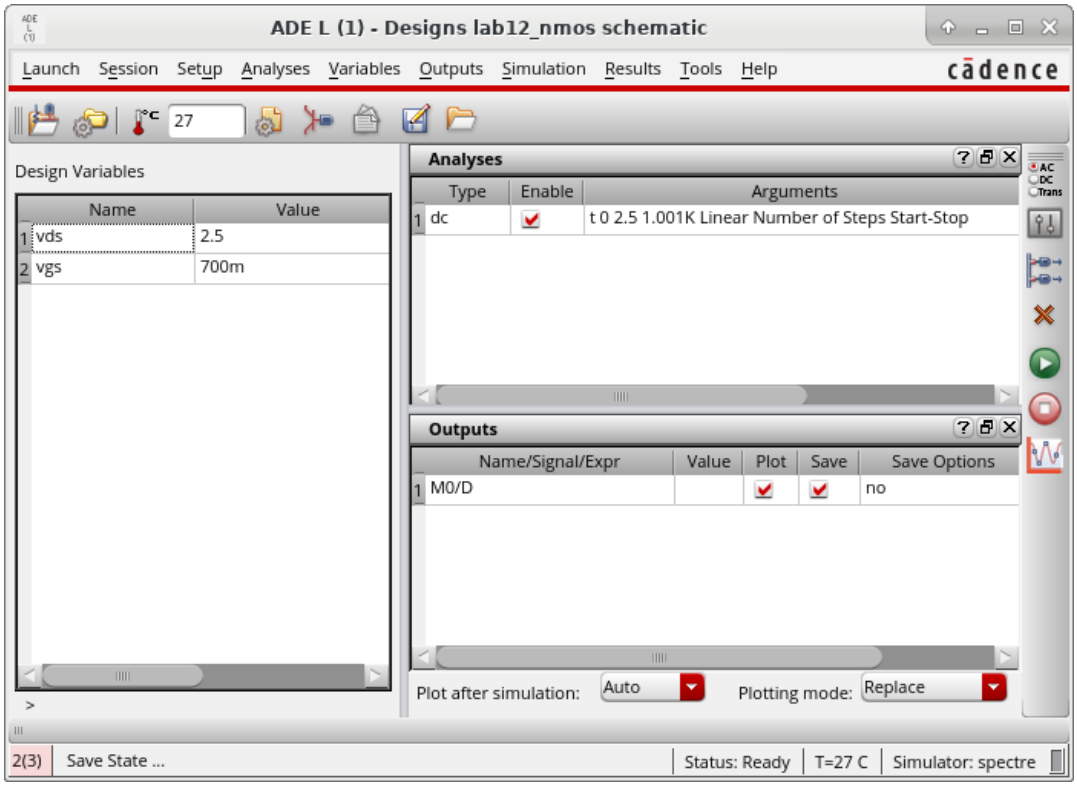
(4.3.3) Copy the design variables from the schematic into the ADE. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the *vds* and *vgs* variables in the 'Design Variables' list of the ADE. Initialise *vds = 2.5* and *vgs = 0.7*. These values are only used to calculate the DC operating point of the circuit.

(4.3.4) From the ADE window, edit the DC analysis (**dc**) by double clicking on it, or right-clicking on it and choosing edit from the context menu. We wish to sweep the input voltage (vgs) of the NMOS device from 0V to 2.5V while observing the drain current (IDS). We will use 1001 steps in the sweep to give the output waveform sufficient resolution.

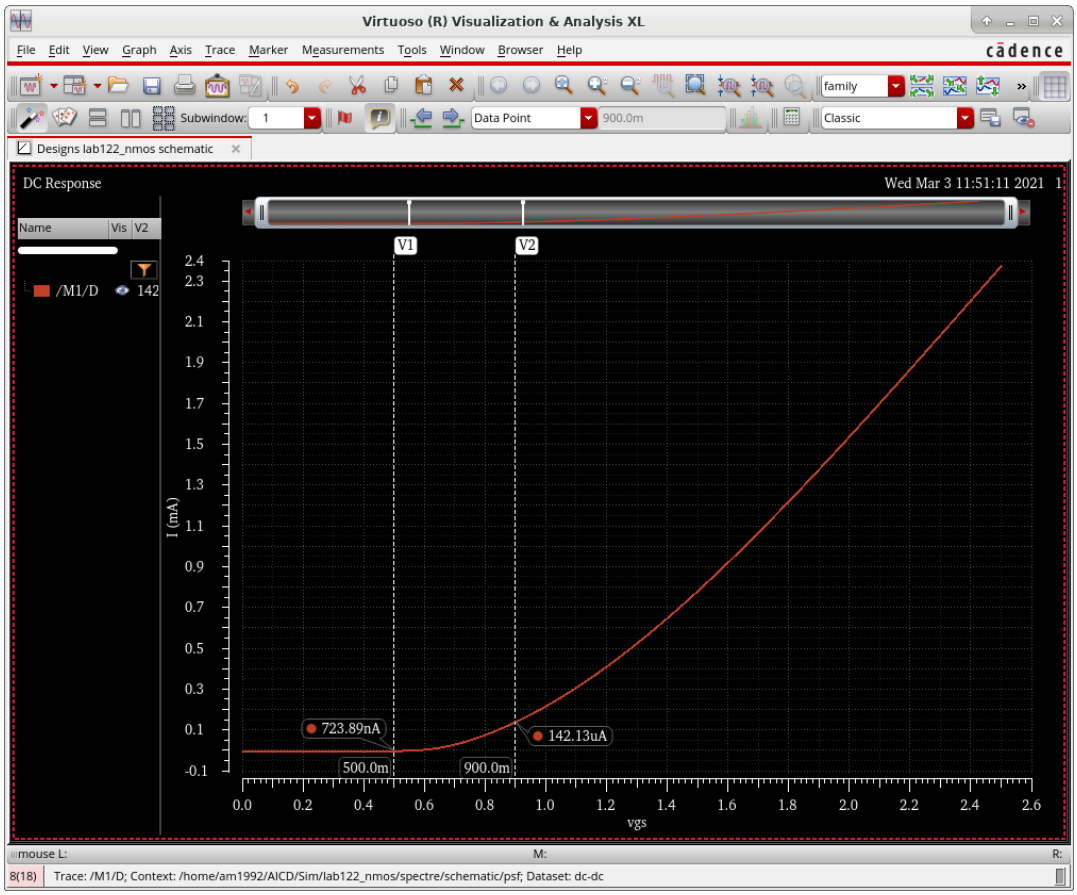
In the choosing analyses form. Check the ‘Save DC Operating Point’ box. Check the ’Design Variable’ box as the sweep variable. Type *vgs* as the variable name. Choose the ‘Start-Stop’ option button with Start = 0 and Stop = 2.5 for the sweep range. Select ‘Linear’ from the drop down menu as the sweep type. Choose the ‘Number of Steps’ option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the ‘Enabled’ box near the bottom of the form. See Figure 8. OK the form. The **dc** analysis should now appear in the ‘Analyses’ list of the ADE window.



< Figure 37. The Choosing Analyses Form for a DC sweep. >



< Figure 38. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 39. The waveform output of IDS vs. VGS with vertical markers at 0.5V and 0.9V. >
(4.3.5) Select the drain node current (IDS) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** The schematic window will automatically become current. Select only the drain node of the NMOS device. **Press the 'ESC' key to exit the plotting mode of the ADE**.

*Recall**,** if you want to plot a particular voltage, you click on the net/wire. If you want to save and plot the currents, you have to click a terminal **node** of a component (not the net/wire).*

(4.3.6) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error and both the text output window as well as the waveform output window should open automatically. You should get a plot of V_{GS} vs. I_{DS} – see Figure 10.

(4.3.7) Note: You can add markers to your graph. From the waveform output window, choose **Menu: Markers à Create Marker…** to bring up the 'Create Graph Marker' form.

From the marker form, you can choose the marker type (**point**, **horizontal**, or **vertical**). You can specify the precise location of the markers at instantiation, otherwise they will initialise/ appear at the origin and can be dragged to the wanted trace position. Double-clicking on a marker brings up the marker properties form, where you can refine its behaviour (you can also use **Menu: Marker à properties** to bring up the marker properties form – select the marker first.)

Alternatively, you can use the keyboard shortcuts to add a marker to your plot: '**m'** (point marker), '**h'** (horizontal marker), and '**v'** (vertical marker). When using the keyboard shortcuts, the marker will appear at the location of you mouse cursor.

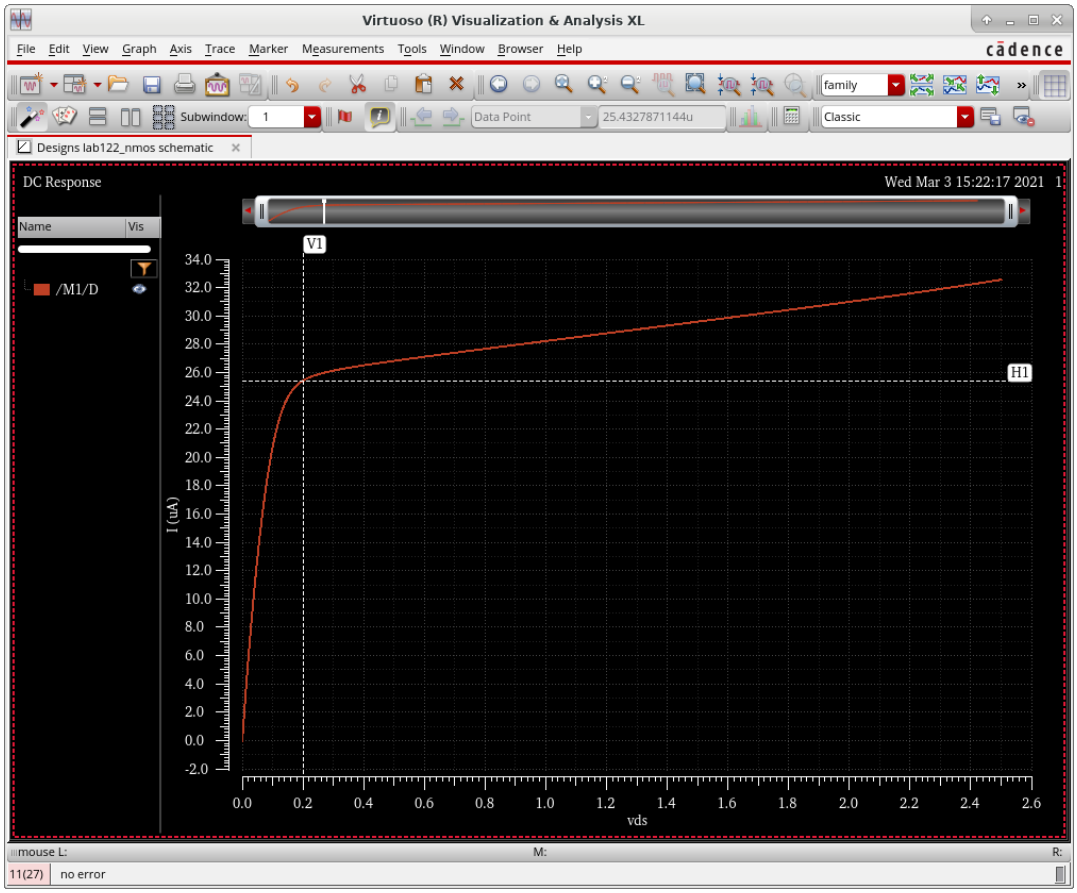
4.4 Parametric Swept-DC Analysis of V_{DS} vs. I_{DS} vs. V_{GS}

(4.4.1) We will now plot a DC sweep of V_{DS} vs. I_{DS} (for nominal V_{GS} = 0.7V). From the ADE window, edit the DC analysis (**dc**) to make $\underline{V_{DS}}$ the sweep variable. Either double click on the analysis or right-click on the analysis and choose 'Edit' from the context menu. Change the sweep variable name from V_{GS} to V_{DS}. Leave the rest of the form unchanged. OK the form.

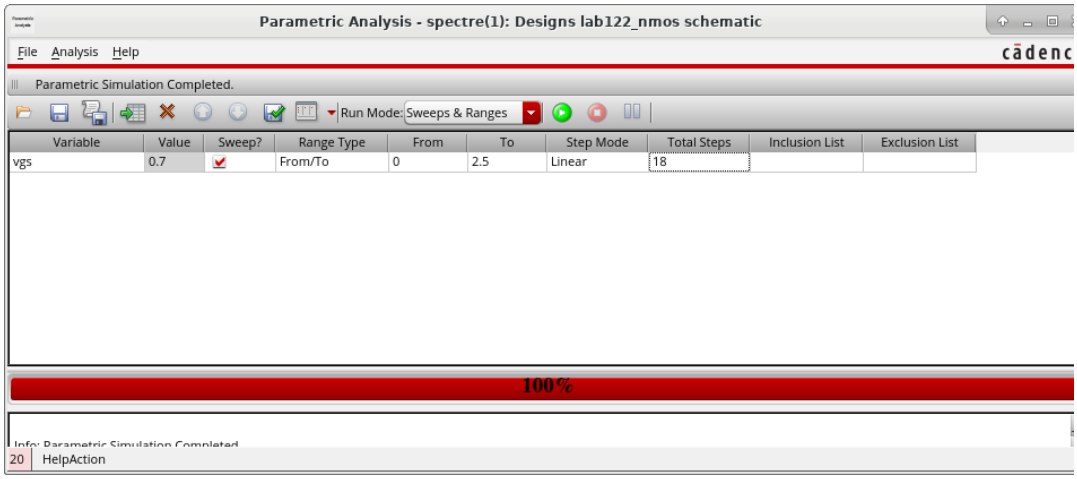
(4.4.2) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figure 11.

(4.4.3) Ultimately, we want to plot the V_{DS} vs. I_{DS} family of curves for increasing values of V_{GS}. (V_{DS} vs. I_{DS} vs. V_{GS}) This requires two sweeps – for V_{DS} from 0V to 2.5V and for V_{GS} from 0V to 2.5V. The DC analysis form only allows you to sweep one variable at a time, so we will need

to use the Parametric Analysis tool. From the ADE window, choose **Menu: Tools → Parametric Analysis…** to invoke the Parametric Analysis tool. See Figure 12.



< Figure 40. Single DC-sweep of IDS vs. VDS (VGS = 0.7V, VOV = ~0.2V, W/L = 10um/1um) >

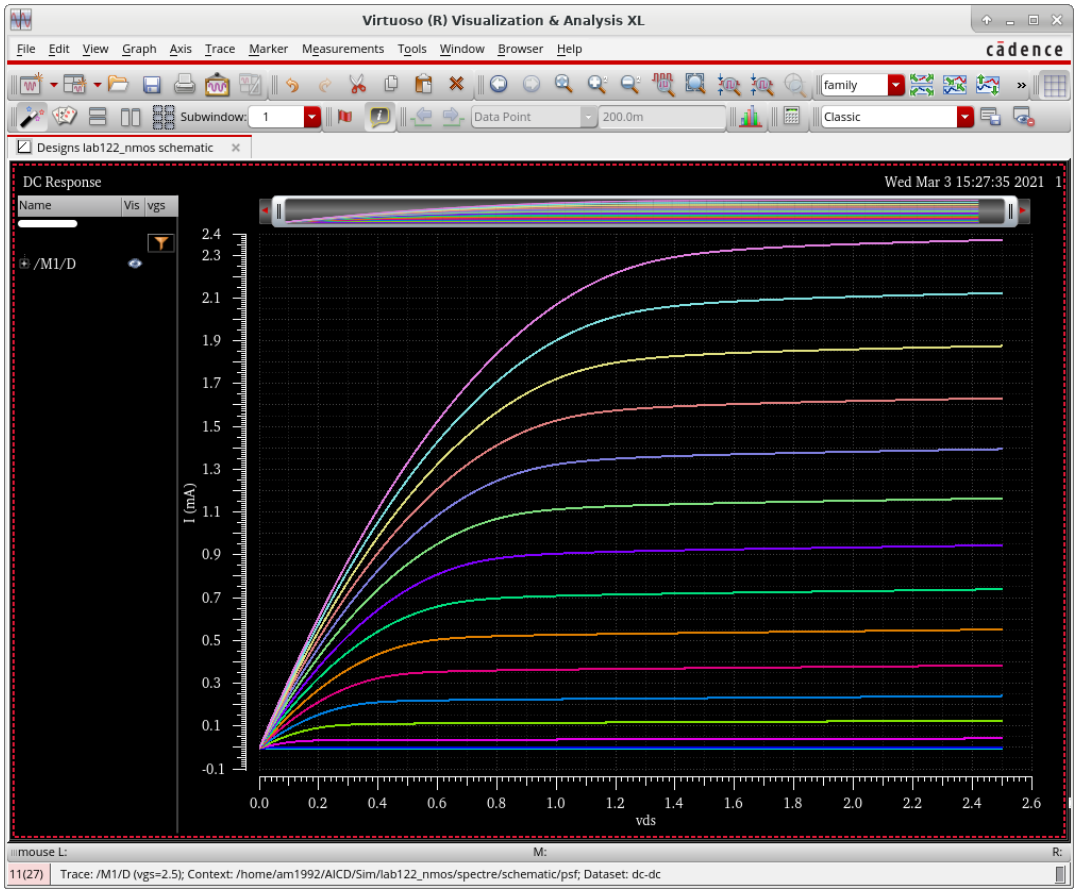


< Figure 41. Parametric Analysis Tool. Set up for a linear sweep of VGS → 0V to 2.5V in 18 steps. This will run the swept DC-analysis (dc) 18 times to create a family of curves. >

Note: The parametric tool works by repeating an analysis/simulation a specified number of times. While the parametric analysis tool allows us to specify additional variables to sweep, it is typically used to vary a parameter using far fewer sweep points than a DC-sweep, as it is much slower. The parametric analysis will run an analysis in the Analyses section of the ADE, once for each step of a parameter variation. A swept DC-analysis can efficiently handle sweeps of tens of thousands of sweep points. For the parametric tool, sweeps are best kept below 100 sweep points. A parametric sweep can be specified as a range to sweep or as an explicit list/set of values over which to iterate. (Run Mode: Sweeps and Ranges, or Parametric Set).

(4.4.4) In the Parametric Analysis window, select Sweeps and Ranges from the '**Run Mode'** drop down menu in the tool bar. In the '**Variable'** column, you should get a drop-down menu of the available design variables. Select V_{GS} as the parametric sweep variable. Note, only the design variables in the 'Design Variables' section of the ADE will be available in the drop-down list along with the system variable temp. So, make sure you have copied all design variables from the schematic into the ADE using the **Menu: Variables → Copy From Cellview…** command.

In the '**Range Type**' drop-down menu select 'From/To'. In the 'From' column type 0 as the start value of the parametric sweep, and in the 'To' column type 2.5 as the stop value of the parametric sweep. In the '**Step Mode**' column select 'linear' and in the **Total Steps** type '18'.



< Figure 42. Parametric DC-sweep of IDS vs. VDS for VGS à 0V to 2.5V. (18 steps, W/L = 10um/1um) >

(4.4.4) The parametric analysis is run from the parametric analysis window not from the ADE window. In the Parametric Analysis window, run the '**Run Selected Sweeps**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figure 13. With the above settings, the ADE will perform a swept-DC analysis of V_{DS} from 0V to 2.5V, a total of 18 times – one simulation for each value of V_{GS} (V_{GS} will be linearly incremented from 0V to 2.5V with each increment of V_{GS} given by 0.134V (=2.5V/18) as specified in the parametric form).

(4.4.5) **Discussion**. The plots of Figure 10 (V_{GS} vs. I_{DS}), Figure 11 (V_{DS} vs. I_{DS}), and Figure 13 (V_{DS} vs. I_{DS} vs. V_{GS}), should not be new to you. Refer to section 1.2.4 of the background material – for a discussion on the large-signal terminal characteristics of the MOS device.

As pointed out in the introduction to this part of the lab, the point of the above exercises was to simulate the large-signal behaviour of the MOSFET device from which the small-signal model of the MOSFET *as an amplifier* is derived. Recall, the fundamental gain A_V of a MOSFET amplifier is given by,

$$A_V = g_m \times r_O || R_D$$

where g_m is the small-signal transconductance of the MOSFET, r_o is its inherent output resistance, and R_D is the load. The small-signal transconductance g_m and output impedance r_o relate to the large-signal characteristics we have just simulated by,

$$g_m(V_{GS}) = \frac{\partial I_{DS}}{\partial V_{GS}} \qquad r_o(V_{DS}) = \left[\frac{\partial I_{DS}}{\partial V_{DS}}\right]^{-1}$$

When using a MOSFET as an amplifier, we must first bias the device at an appropriate DC operating point – the fixed DC values of V_{GS}, I_{DS}, and V_{DS} – to maximise the performance of the device (to a given specification). This comes down to operating the MOSFET at a point along the curves of V_{DS} vs. I_{DS} and V_{GS} vs. I_{DS}, that maximise both g_m and r_o, while giving due consideration to other factors (often equally as important) such as linearity, signal-swing, frequency response, stability, and power consumption. We will develop the background required for making such a decision in the rest of this lab and the coming labs.

An amplifier comprises a transconductor and a load. In the next section, we will move from the characterisation of the MOSFET as a VCCS (the transconductor), to the basic MOSFET amplifier – by the addition of a load.

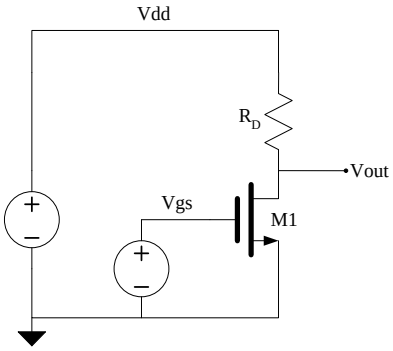
5. Circuit Simulation Exercises: Part 2

With guidance, you will:

- Create and simulate a simple resistor-loaded single-stage NMOS FET voltage amplifier.
- Address the inadequacies of using a resistive load for IC design. Create and simulate a current-source loaded (or active-loaded) NMOS FET voltage amplifier.

5.1 A Simple Voltage Amplifier

(5.1.1) Create the schematic for the basic resistor-loaded voltage amplifier circuit in Figure 14. A summary of the circuit's operation is given in section 4.2.1 of the background material.



< Figure 43. Basic Resistor-Loaded Voltage Amplifier
Reproduced from Figure 15, section 4.2.1 of the background material. >

(5.1.2) Note: Since you have already built most of this circuit in the first part of this lab, there are a number of ways you could approach creating this new schematic. You could:

1. Modify your existing schematic (saving it under the same name).
2. Save or create a copy of your existing schematic with a new name, and either continue to modify your existing schematic or switch to working on the new copy.
3. Create a new schematic from scratch.

Option (1) is quick and easy – but in doing so you lose the history or progression of your work. For example, if you wanted to replot the characterisation exercises you completed in the first part of this lab, you would have to rebuild the circuit as it was then, as you will have since modified the circuit. For simple circuits, with few modifications, this may not be much of a problem. For complex circuits, however, it is good practice to keep a stable working copy of your design at various stages of completion that you can revert to if things go wrong.

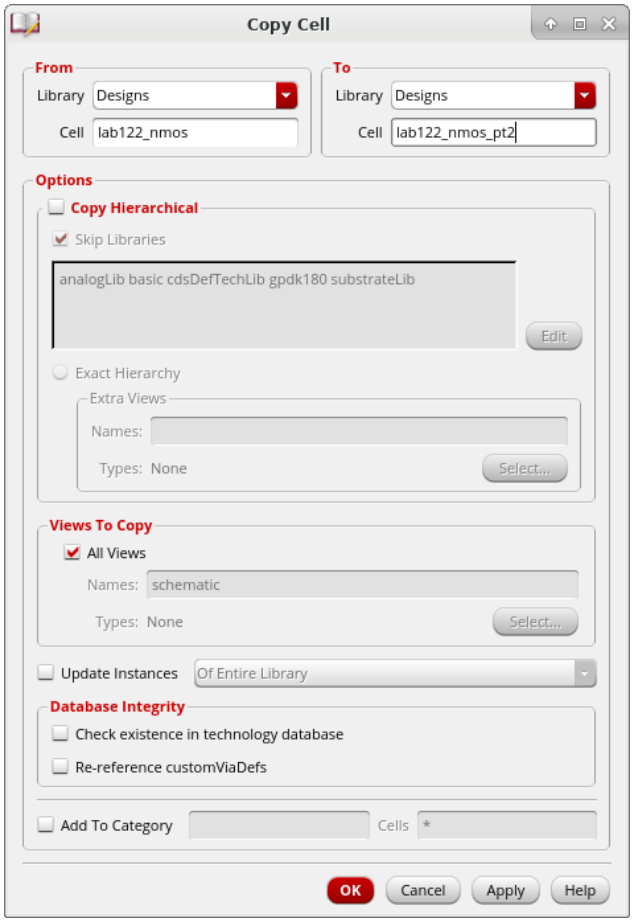
Option (2) is the most efficient way of starting a new design – by reusing as much of an existing design as possible while keeping the existing working design safe (unmodified). In this case,

give each copy of the schematic a meaningful name and keep the modified versions in one library to make any changes to the circuit easier to identify (assuming you are not using a version control system). In cadence, there are two ways of creating a new copy of an existing schematic – from the Library Manager or from within the Schematic Editor.

Option (3) is recommended as you are still learning Cadence. Since you are still learning simple theoretical concepts of the analogue IC design flow, and your circuits are simple, take the opportunity to remind yourself of how to perform the basic operations of the software (e.g., starting a new schematic and/or simulation from scratch) until it becomes second nature. This will ensure that when you come to learn the more difficult concepts in the analogue IC design flow you are not held back by your lack of familiarity with the software.

(5.1.3) To start a new schematic from scratch, use the New File form as you have done previously – ensuring you are in your Designs library – giving your new schematic a meaningful name such a 'lab122_nmos_pt2'.

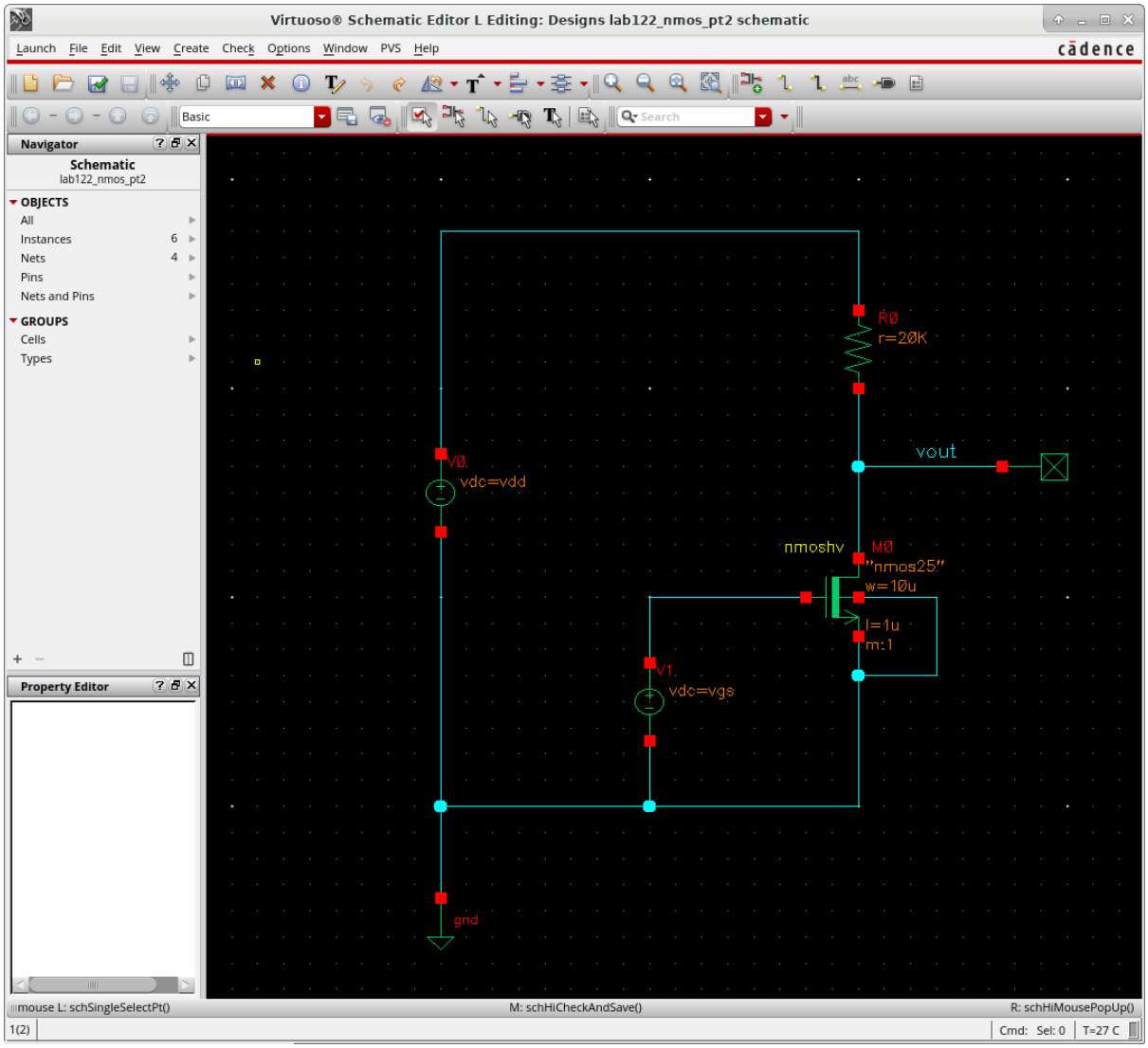
Alternatively, to create a copy from an existing schematic, go to the Library Manager window and select the cell you want to copy e.g. lab122_nmos from the Designs library. Right-click on the cell and select 'Copy…' from the context menu to invoke the 'Copy Cell' form. (You could also select **Menu: Edit à Copy** from the menu bar.)



< Figure 44: The Copy Cell Form >

In the ‘From’ section of the copy cell form, select the source library and cell name and in the ‘To’ section choose the destination library and cell name (in this lab, use the Designs library in both the From/To sections). Give the new cell a meaningful name e.g. ‘lab122_nmos_pt2’. Leave the rest of the form unchanged and OK the form. In the library manager, in the Designs library, you should now see a new schematic cellview called ‘lab122_nmos_pt2’ (or whatever name you chose).

You could also save a copy from the Schematic Editor window from **Menu: File à Save a Copy…**, specifying the library, cell name, and cell type. Note, that in this case when you save a copy you remain working in your original schematic – you do not automatically switch to the new copy. You should see the new copy in the Library Manager.



< Figure 45. The Resistor-Loaded Voltage Amplifier Schematic as in Figure 14>

(5.1.4) In your schematic editor window, create the circuit given in Figure 15. Instantiate two DC sources (**vdc**), a resistor (**res**), and an NMOS FET from the GPDK180 library (**nmoshv**). Parameterise the DC supply voltage as *vdd* (**DC Voltage** = *vdd*), parameterise the gate-to-source voltage as vgs (**DC Voltage** = *vgs*), set the load resistor to 20KΩ (**Resistance** = *20K*), and dimension the NMOS device 10u/1u (**width** = *10u*, **length** = 1u). Leave all other attributes unchanged.

(5.1.6) Wire the components as in Figure 15 using the **Wire (narrow)** command. Name the output net *vout* using the **Create Wire Name** command. Use the **noConn** connector from the **basic** library to avoid floating-net warnings. Keep the schematic neat.

(5.1.7) Check and save you schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.

5.2 Simulating the Voltage Amplifier using a Swept-DC Analysis

(5.2.1) You should now have the completed schematic of the resistor-loaded voltage amplifier circuit as shown in Figure 16. It should check and save without any errors or warnings.

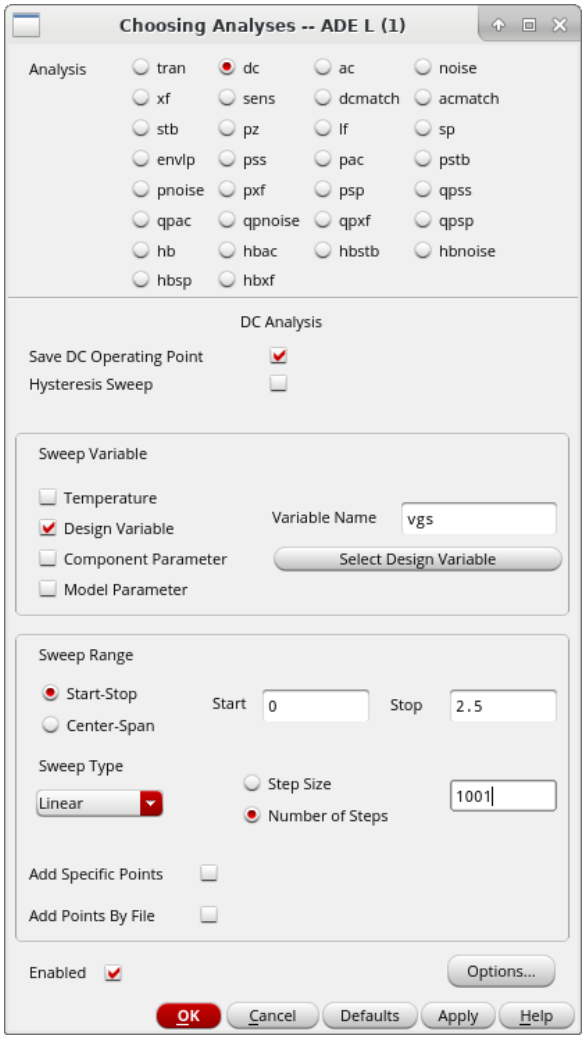
(5.2.2) We will simulate the basic voltage amplifier circuit using a swept-variable DC Analysis, aka a DC-sweep. From the Schematic Editor window, launch the Virtuoso Analogue Design Environment (ADE) window from **Menu: Launch à ADE L**.

(5.2.3) Copy the design variables from the schematic into the ADE. For this circuit, these are the DC voltages from the two **vdc** components - *vdd* and *vgs*. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the *vdd* and *vgs* variables in the 'Design Variables' list of the ADE. Initialise *vdd = 2.5V* and *vgs = 0.7V*. These values are used to calculate the DC operating point of the amplifier circuit.

(5.2.4) From the ADE window, go to **Menu: Analyses à Choose…** or use the '**Choose Analyses**' button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. From the form, select the '**dc**' option button.

(5.2.5) We wish to sweep the input voltage (vgs) of the NMOS device from 0V to 2.5V while observing the output voltage (vout). We will use 1001 steps in the sweep to give the output waveform sufficient resolution.

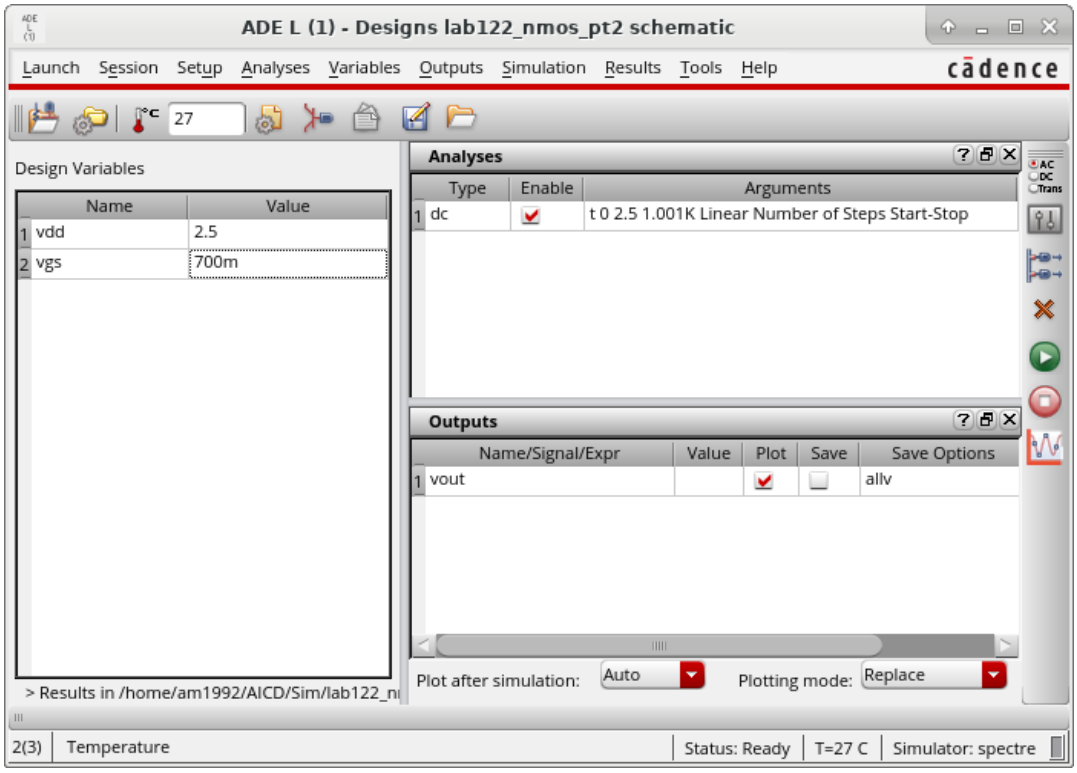
In the choosing analyses form. Check the ‘Save DC Operating Point’ box. Check the ’Design Variable’ box as the sweep variable. Type ‘vgs’ as the variable name. Choose the ‘Start-Stop’ option button with Start = 0 and Stop = 2.5 for the sweep range. Select ‘Linear’ from the drop-down menu as the sweep type. Choose the ‘Number of Steps’ option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the ‘Enabled’ box near the bottom of the form. See Figure 17. OK the form. The **dc** analysis should now appear in the ‘Analyses’ list of the ADE window.



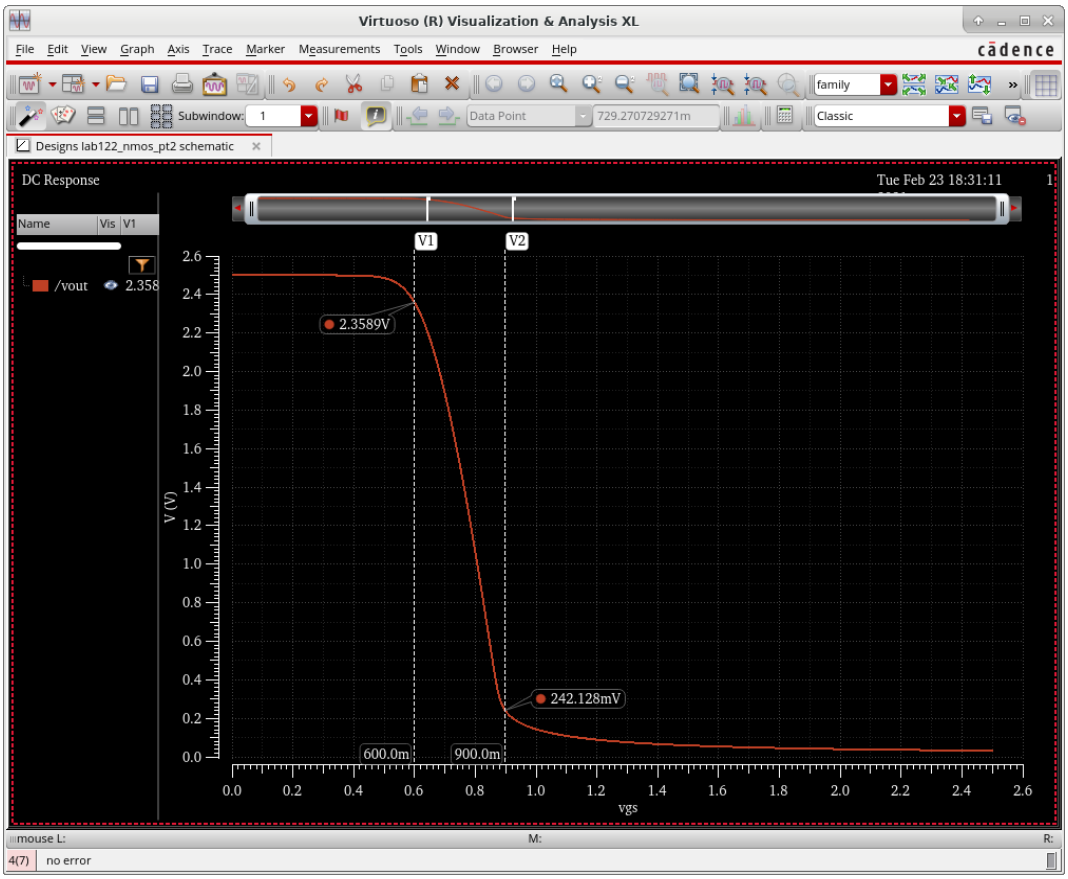
< Figure 46. The Choosing Analyses Form. >

(5.2.6) Select the output net voltage (vout) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** The schematic window will automatically become current. Select only the output net *vout*. **Press the ‘ESC’ key to exit the plotting mode of the ADE**.

(5.2.7) In the Analog Design Environment window, run the ‘**Netlist and Run**’ command by clicking on the green play button. If you followed all the instructions correctly, the simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figure 19.



< Figure 47. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 48. The waveform output window with vertical markers at 0.6V and 0.9V. >

5.3 Discussion: The Inadequacies of Using a Resistive-Load

The Voltage Transfer Curve

Both the PMOS and NMOS devices from the GPDK180 library that we will be using in these labs are high-voltage devices (**nmoshv** and **pmoshv**) and have a threshold voltage ~0.5V.

Observe the DC voltage transfer curve (VTC) of the basic voltage amplifier (Figure 19). With V_{GS} below the threshold voltage, the NMOS device is cut-off. As no current flows through the resistor, there is no voltage drop across it, and so V_{OUT} sits at the 2.5V supply voltage. As V_{GS} approaches 0.5V, the threshold voltage, we begin to turn on the device. And, as V_{GS} pushes over and above 0.5V, very readily we get some transfer from the power-rail (2.5V) at the top down to the ground-rail (0V) at the bottom. Notice that everything goes on in between the first 0.3V of the overdrive voltage ($0V \leq V_{OV} \leq 0.34V$) – that is, between 0.6V and 0.9V. Not a lot happens before or after that.

Given the DC voltage transfer curve – we pose the question: where on the transfer-curve do we bias or operate the device? For analogue design we want to operate in the middle of the sweep/transition, at $V_{OUT} = V_{DD}/2$, as this is where we can support an analogue signal transfer through the system with maximum swing (positive and negative excursions about the output operating point). Given that the transfer curve is somewhat non-linear, we may adjust the above statement to: we want to operate the device in the middle of the most linear part of the sweep/transition. We, therefore, want to bias the device at the input such that, for the nominal input voltage, the output sits close to mid-rail where we'll get the maximum, linear voltage swing. For the VTC in Figure 19, we would set the input DC bias at about 0.7V to 0.8V.

(As an aside, if we were interested in digital electronics we would only be interested in the extremes – 2.5V and 0V, where the transistor is wholly-on or (very nearly) wholly-off – since it's a binary system. In fact, the original logic gates were implemented by precisely this type of circuit – so-called Resistor-Transistor Logic. Nowadays, of course, CMOS based digital logic has become mainstream. You will have seen the CMOS previously on the programme.

Note that the output does not fully go to 0V. We've got a complete mapping to the 2.5V – this is simply because when we are below (or significantly below) the threshold voltage, we are not passing any current though the NMOS device or through the resistor, and with no voltage drop across the resistor the output is at the supply-rail.

However, we can't quite reach zero because there is always some residual resistance of the device. That is, at the lower end of the transfer curve, we have pushed the device out of the saturation region ($V_{DS} \geq V_{GS} - V_T$), into the triode region ($V_{DS} < V_{GS} - V_T$), and into the linear region ($V_{DS} << V_{GS} - V_T$) where we've effectively got some resistance. This leads to a voltage drop across the device that prevents the output going fully to 0V rail. If the output were to go

fully to the 0V rail, we would have a V_{GS} but no V_{DS}, and we would have effectively turned off the device (V_{DS} = I_{DS} = 0).
Note: If we were to use a PMOS device as the transductor and have the resistor as the load connected to the 0V rail – the opposite would be true – we could reach the 0V rail but not the 2.5V rail.

Limitations of the Resistive Load

From the VTC, we have a voltage gain of around $\Delta V_{OUT}/\Delta V_{IN}$ = 1V/0.1V = 10 (although not perfectly linear across the sweep). Note, that this is not how we normally measure the small-signal gain of a circuit – we will use an AC analysis for that in the next lab – but we should be able to extract, graphically, the DC operating point information (chiefly *gm* and r_o) from the DC-sweep simulation.

There is, however, a problem with the resistor-loaded architecture. We know the relationship between current, voltage and resistance. If we want to make the transfer curve steeper – that is, increase the voltage gain – we need a larger load resistance (assuming we have a fixed transconductance). But, if we were to increase the load resistance, while keeping the same drain current, we upset (lower) the DC operating point at V_{OUT}—lowering the available signal swing. We can, in turn, reduce the I_{DS} bias current. However, for large gains we require large resistive loads, this may require a very small drain current and therefore a very small V_{GS}. As we decrease V_{GS} we come out of the predictable square-law region of MOSFET operation, where all the design equations/approximations work, and enter the exponential region. In this region, the paper design equations no longer work, device models are no longer accurate, and we operate awkwardly close to the threshold voltage – where process, voltage, and/or temperature (PVT) variations may accidentally turn the MOS device off completely. In addition, the exponential region of operation is noisy due to the low drain current density.

So, whilst we can increase the gain by increasing the resistance, this leads to many problems with respect to the DC operating point. (The DC bias point at V_{OUT} will drop and we are no-longer at mid-rail. We now have to reduce the drain current to bring that mid-rail voltage up, but we've got a problem with pushing less current through the device – operating in the exponential region / close to the threshold voltage, susceptibility to PVT, questionable model accuracy, and device noise.) Alternatively, we could increase the transconductance to increase gain. But this means drawing more DC bias current – lowering overall power efficiency. Increasing the drain current also decreases r_o, which appears in parallel with the load resistor. As a result, the increase in gain is limited.

The Resistor-loaded voltage amplifier, while simple, is beset with problems for IC design giving a relatively low gain and a lack of configurability of gain. So, what would make an ideal load? ***We need to decouple the setting of the gain, from the setting of the DC bias/operating point. We want a very high impedance load (ideally infinite), while being able to set the drain-bias current arbitrarily. We do not want the DC voltage across the load to be a function of the load current.***

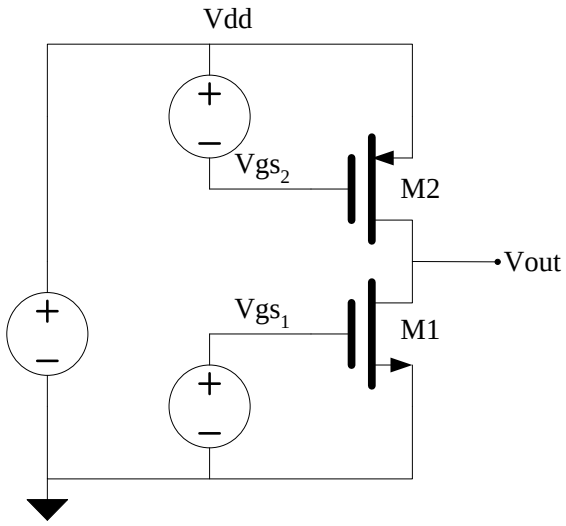
The ***ideal current-source*** has an infinite small-signal (incremental) impedance. Yet, the DC voltage across it is not dependent on the current through the source. If the load of the basic voltage amplifier was an ideal current source, it would define the current through the device while presenting an (ideally) infinite impedance. This would give us a very high gain. Note that despite the infinite impedance of the load, the gain would not be infinite since it appears in parallel with the r_o of the transconductance device (NMOS) which is finite.

Recall that the MOSFET device is a transconductance device. It therefore behaves as a voltage controlled current source (VCCS). A constant-current source can thus be implemented from a MOSFET device with a constant gate-to-source bias. In the circuit of Figure 14, we can use a PMOS device to implement a current-source load. Together, the NMOS transconductor and PMOS load, form the basic building-block of the IC voltage amplifier – the CMOS gain cell. We will explore this in the next section.

5.4 Replacing the Resistor-Load with a Current-Source Load

(5.4.1) In this part of the lab, we will implement a current-source loaded single-stage NMOS FET voltage amplifier to redress the inadequacies of using a resistive-load.

(5.4.2) Modify or copy your existing voltage amplifier schematic, or create a new schematic for the current-source loaded voltage amplifier circuit. You should now be able to create, edit, and modify a schematic with minimal direction.



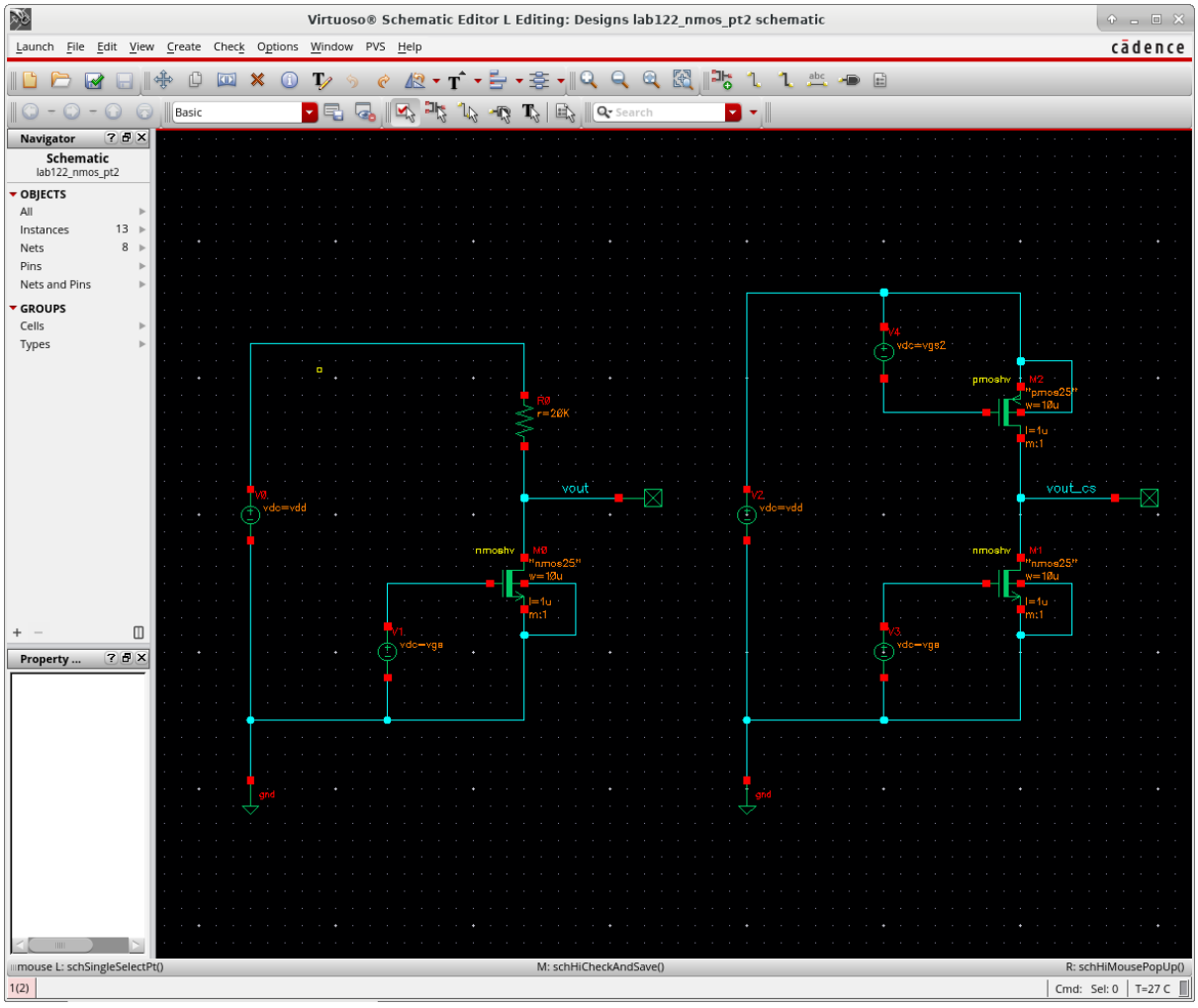
< Figure 49. Current-Source Loaded Voltage Amplifier
(Reproduced from Figure 16, section 4.2.2 of the background notes). >

(5.4.3) In this case, we would suggest building the new current-source loaded amplifier circuit alongside your existing resistor loaded amplifier circuit so you can compare the performance of the two circuits side-by-side (two circuits in one schematic). See Figure 21.

(5.4.4) Copying the circuit. Since the circuits of Figures 15 and 16 are similar (except for the load device), if you want to create the new circuit alongside your previous circuit, you can save time/effort by duplicating part or all of the previous circuit.

In the schematic window, on an empty part of the design canvas, left-click-and-hold the mouse button and drag a selection box around the circuit you want to duplicate. When you release the mouse button, the selected circuit will be highlighted pink/purple. Choose the **Copy** command (**Menu: Edit → Copy** or press '**c**') and click on a highlighted part of the circuit. A copy of the circuit should now attach itself to and move with the cursor. You can instantiate the copy by left-clicking anywhere on the design canvas.

(5.4.5) Modify you second circuit by replacing the resistor-load with a PMOS FET device from the GPDK180 library (**pmoshv**). Add a dc voltage source (**vdc**) for the PMOS gate-to-source bias. We will keep all other component properties the same as they were for the resistor-loaded amplifier. (Note the orientation of the vdc component providing vgs for the PMOS device – for a PMOS device, the gate must be negative with respect to the source.)



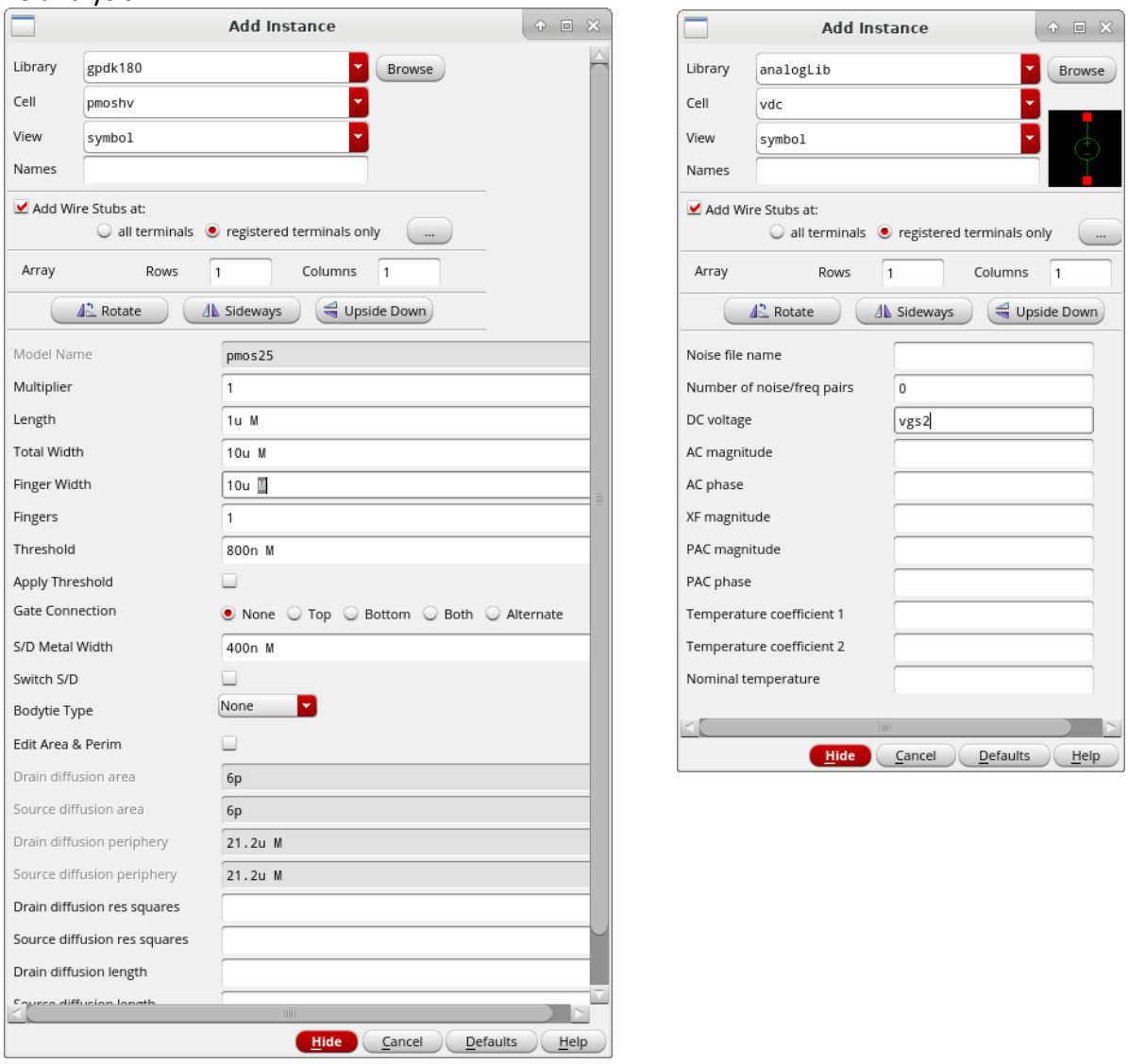
< Figure 50. The completed Current-Source Loaded Voltage Amplifier schematic alongside the original Resistor-Loaded Voltage Amplifier>

(5.4.6) Give the PMOS load device a W/L ratio of 10u/1u ('**Total Width**' = *10u*, '**Length**' = *1u*). Parameterise the PMOS gate-to-source voltage as 'vgs2' (**'DC Voltage'** = *vgs2*). Leave all other attributes/properties unchanged.

(5.4.7) Name the output net of the current-source loaded amplifier circuit '*vout_cs*' to distinguish it from the output net of the resistor loaded amplifier *vout*. Note: When you name the output net of the new circuit, it must not have the same name as the output net of your previous circuit (vout). Giving two separate wires or nets the same name will create a logical connection between those wires/nets – they will become one shared net. This behaviour can be useful to avoid excess wiring in your schematic reducing clarity, but is not wanted here.

(5.4.8) Check and save your schematic using the **Check and Save** command.

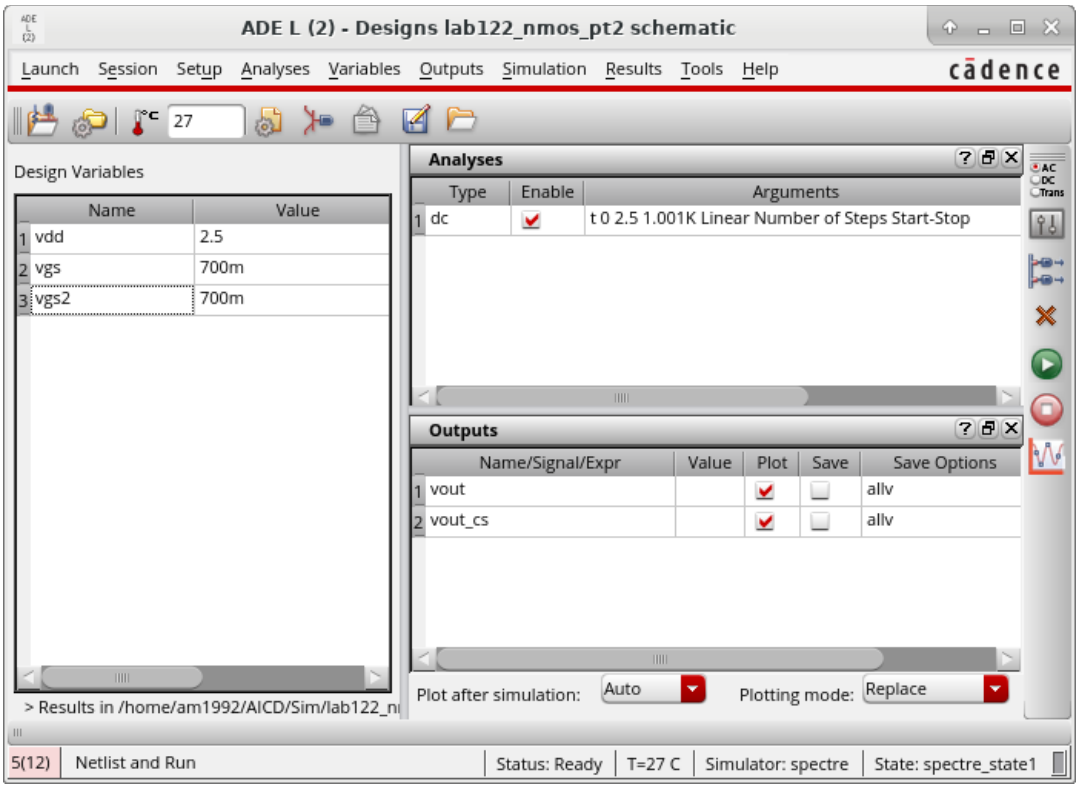
(5.4.9) If you chose to modify your existing circuit, rather than create a new schematic from scratch, you should still have your ADE set up for simulation. If not, or you have created a new schematic, launch the ADE and follow the steps in the previous section to set up the swept-DC analysis.



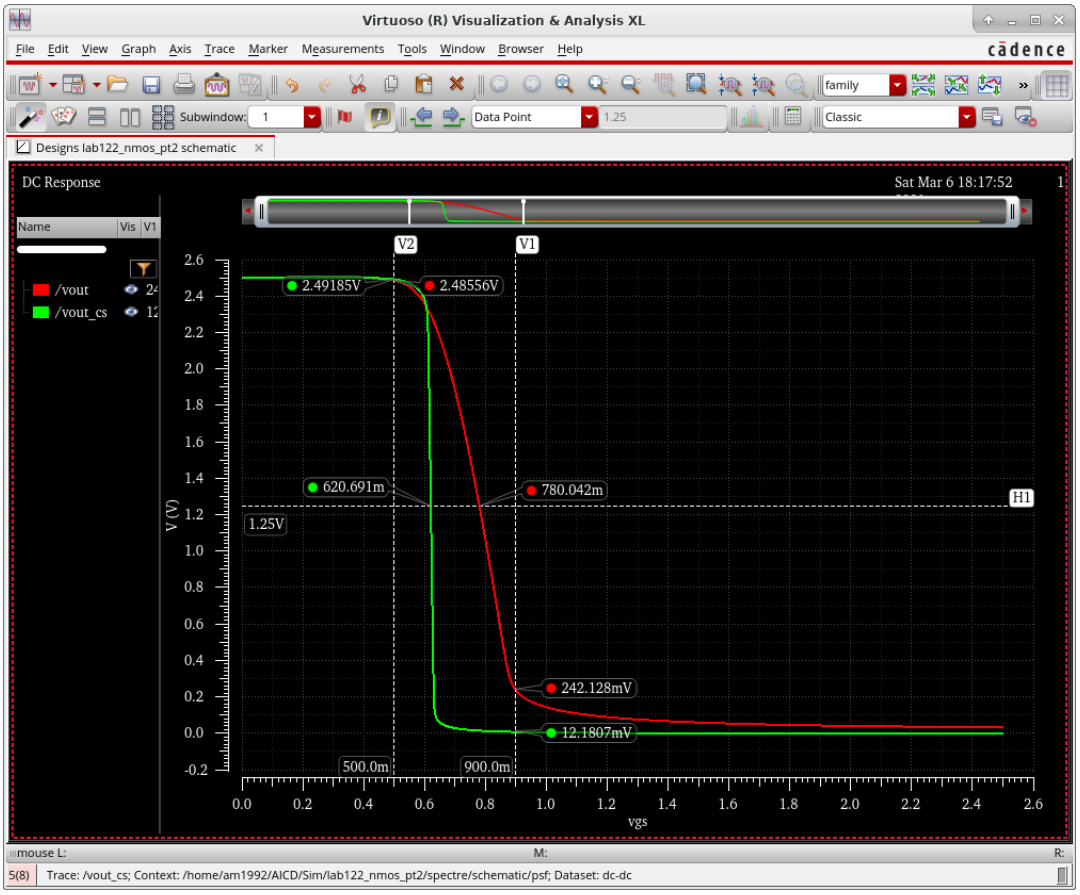
< Figure 51. The pmoshv (GPDK180) and vdc (analogLib) Add Instance Forms. >

(5.4.10) Since you have added a new design variable to your schematic (vgs2) you will need to copy it from the schematic to the ADE (**Menu: Variables à Copy From Cellview**). Initialise vgs2 to 0.7V in the Design Variables section of the ADE. You will also need to add the output net (vout_cs) of the new circuit to the Outputs section of the ADE (**Menu: Outputs à To Be Plotted à Select On Schematic**). Press the 'ESC' key to exit the plotting mode of the ADE.

(5.4.11) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. If you followed all the instructions correctly, the simulation should run properly and both a textual output window as well as a waveform output window should open. See Figure 24 (Makers have been added post simulation.)



< Figure 52. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 53. The Resistor-Loaded Voltage Amplifier Schematic as in Figure 20. >

(5.4.13) **Discussion**. If we compare the voltage transfer curves of the resistor-loaded amplifier and the current-source loaded amplifier, we see the transition of the VTC is far steeper for the latter circuit. As discussed previously, the slope of the VTC is the graphical measure of gain – from the VTC, we have a voltage gain of around $\Delta V_{OUT}/\Delta V_{IN}$ = 1V/0.01V = 100, at least a factor of ten increase over the resistor-loaded amplifier.

Referring back to part 1 of the lab, recall the graph of I_{DS} vs. V_{DS}. The small-signal resistance of the MOS device was given by the slope of the large-scale I-V curve. In the saturation region, where we operate both the PMOS and NMOS devices for amplifier design, when $V_{DS} \geq V_{GS} - V_T$, the I-V curve becomes horizontal and (in the ideal case) independent of increasing drain-to-source voltage. This infers a very-high drain-to-source resistance (represented by r_o in the small signal model of the MOSFET) with $\Delta I_{DS}/\Delta V_{DS} = 0$ (or equivalently $\Delta V_{DS}/\Delta I_{DS} = \infty$). The key difference, however, between the resistive and PMOS loads is that (as you can observe from the graphs) the I-V curves of the PMOS device (specifically in the saturation region of operation) do not need to pass through the origin. That is, the high impedance of the PMOS load is independent of the DC bias level of either IDS or VDS. We have decoupled the setting of the gain from the setting of the DC operating point.

From the horizontal marker set in Figure 24, we can see from the output plot what the required VGS for the NMOS device should be for the correct current demand for VOUT = VDD/2. For the 20KΩ resistor-load a VGS of ~0.78V and for the current-source load a VGS of around ~0.62V will give both amplifier configurations a VOUT of around 1.3V (~VDD/2). We will make use of these values in the next section – when we simulate the time-domain responses of both circuits.

We can think of the current source loaded circuit in terms of trying to balance the current being 'sourced' by the PMOS device and the current being 'sunk' by the NMOS device. The only time the output VOUT will be mid-rail (VDD/2) is if the current sourced by the PMOS is the same as the current sunk by the NMOS. There are two scenarios if the sourced/sunk currents are not equal. (1) if the current sourced by the PMOS is greater than the current sunk by the NMOS, the excess current will be pushed through the very high output resistance of the NMOS and the output voltage will be forced up to the supply rail. (2) If the current sunk by the NMOS is greater than the current sourced by the PMOS, the excess current will be pulled through the very high output resistance of the PMOS and the output voltage will be forced down to the ground rail. Note that the difference between the currents being sourced (PMOS) and sunk (NMOS) will be influenced by different carrier mobilities of the two types of device. The PMOS device may be scaled (W/L ratio) to compensate for its lower carrier mobility.

We also note that with the PMOS load, V_{OUT} now extends from the power rail (2.5V) closer to the ground rail (0V) due to the much larger PMOS drain-to-source resistance. In this case, the residual resistance of the NMOS device is much smaller, where comparably less voltage is dropped across it (than when compared to the 20KΩ resistive load).

In the next section we will simulate the time-domain (transient) responses of the voltage amplifier circuits.

5.5 Time-Domain Response of the Single-Stage NMOS FET Amplifier

(5.5.1) We will now carry out a transient simulation of both the resistor-loaded and current source loaded single-stage NMOS FET voltage amplifiers. We will apply a DC bias (as specified above) to the gate of each NMOS transconductance device to set the DC operating point V_{OUT} = V_{DD}/2. We will apply a 10mV sinusoidal perturbation to the input and observe the output.

(5.5.2) In the schematic editor window, replace the gate-to-source DC voltage sources (**vdc**) of the NMOS FET transconductance devices with a time-varying Sinusoidal voltage source (**vsin**). Leave the gate-to-source DC voltage source of the PMOS unchanged.

Recall, you can use the **Delete** command to delete the **vdc** part and add a new instance of the **vsin** part. Or you can change **vdc** to a **vsin** component directly by editing its properties (**Menu: Edit à Properties à Objects**, or **Edit Properties** icon-button, or press '**Q**') and changing the cell name from vdc to vsin. You can also select the part, right-click and choose Properties from the context menu. The 'Add Instance' form will then change from a vdc form to a vsin form.

(5.5.3) For the **resistor loaded amplifier**, set the **vsin** attributes as follow: ‘DC Voltage’ = vgs, ‘Amplitude’ = 10mV, ‘Offset voltage’ = 780mV, and ‘Frequency’ = 10KHz. OK the form.

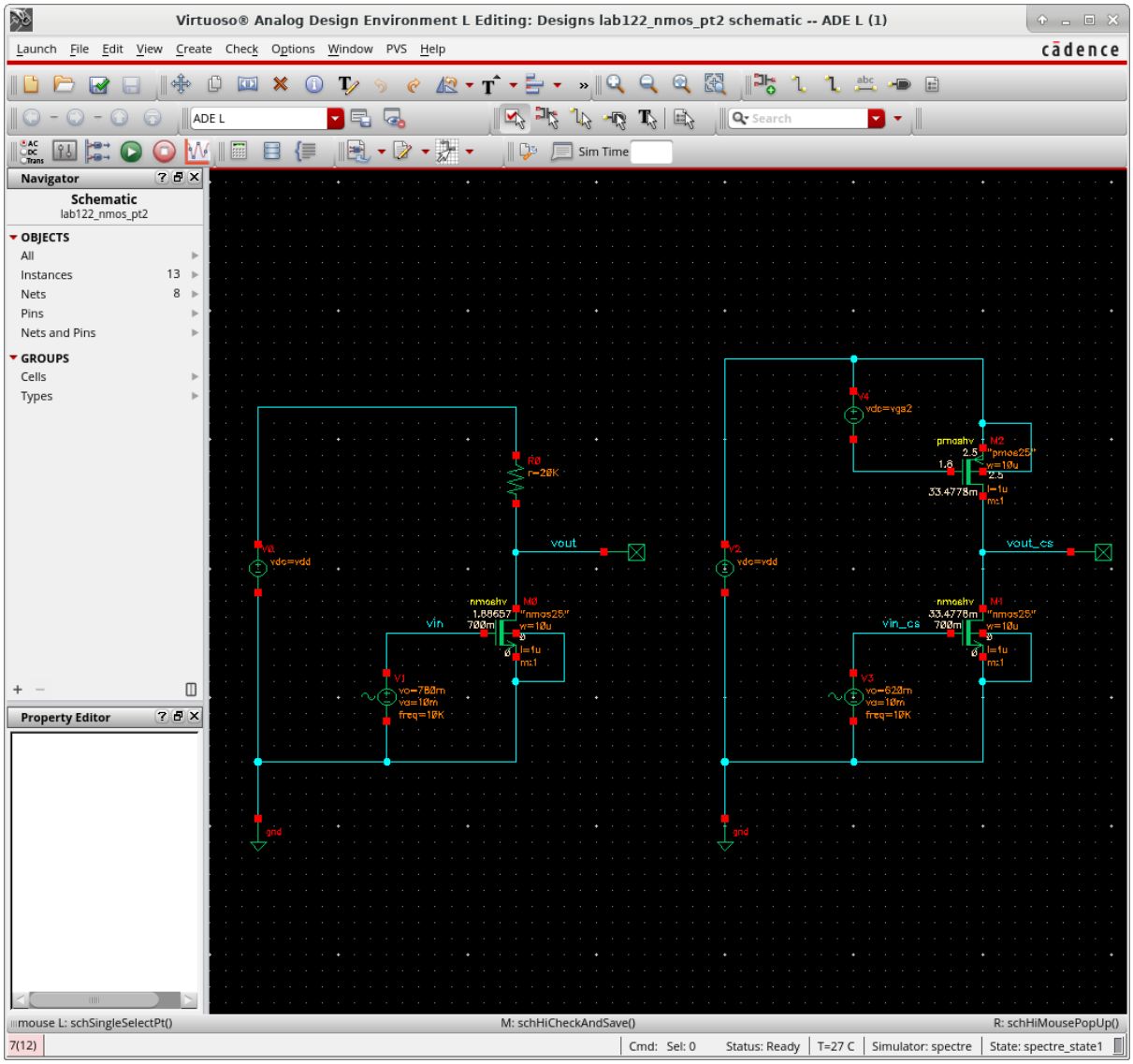
(5.5.4) For the **current-source loaded amplifier**, set the **vsin** attributes as follow: ‘DC Voltage’ = vgs, ‘Amplitude’ = 10mV, ‘Offset voltage’ = 620.7mV, and ‘Frequency’ = 10KHz. Ok the form.

(5.5.5) Name the input nets of the two amplifiers *vin* (for the resistor-loaded amplifier) and *vin_cs* (for the current-source loaded amplifier). See Figure 25.

(5.5.6) **Check and Save** the schematic. There should be no errors or warnings. Use the CIW to diagnose any errors or warnings (if there are any).



< Figure 54. The vsin Edit Object Instance Form. >

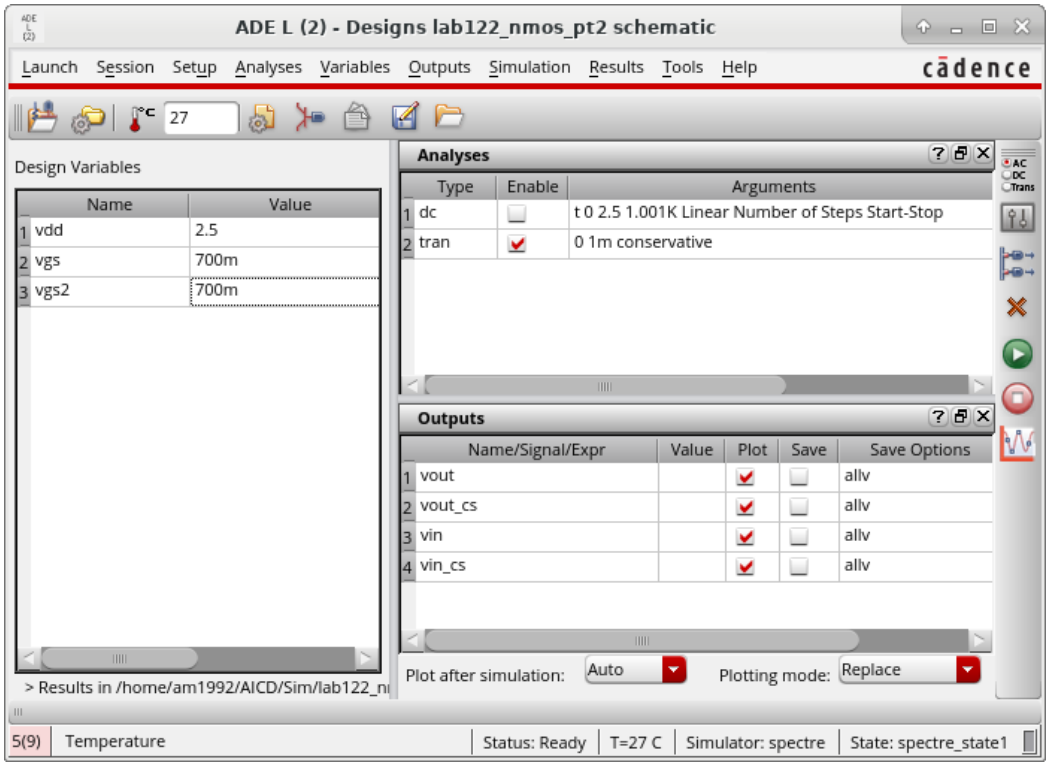


< Figure 55. The voltage amplifier circuits with vsin excitation. Amplitude = 10mV, Frequency = 10KHz, DC Offset = 780mV & 620mV. >

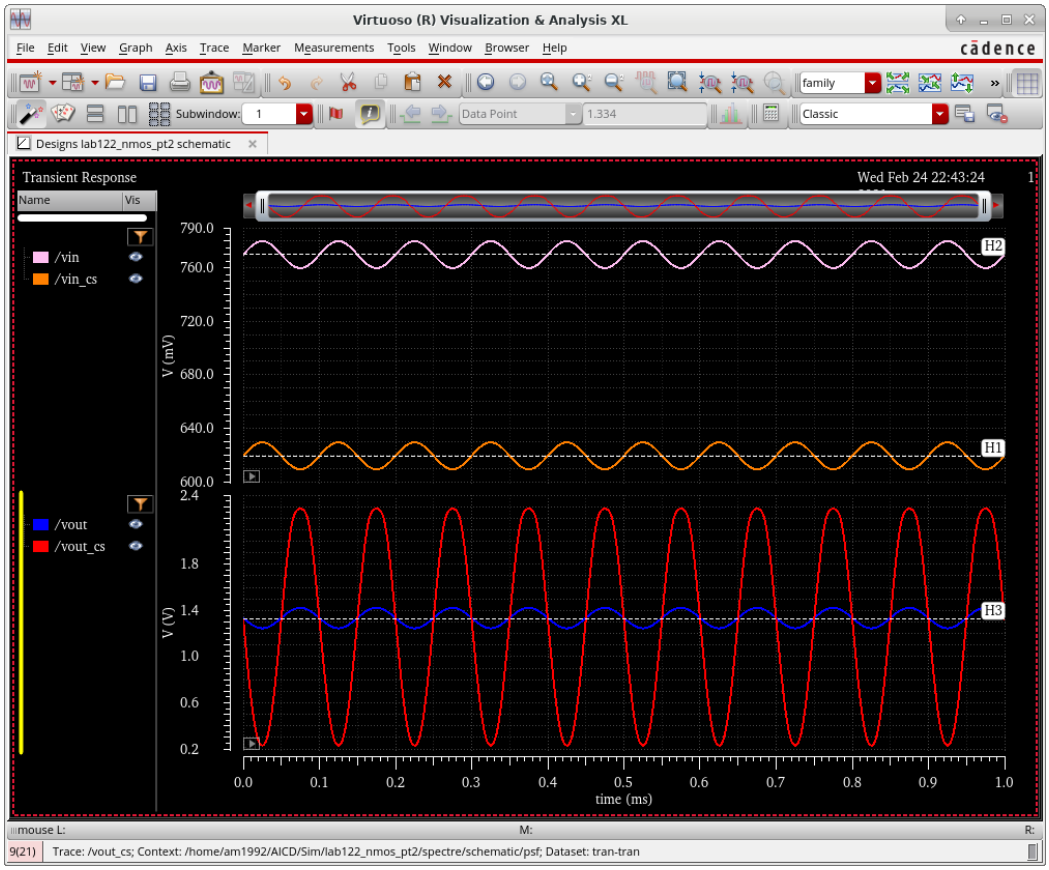
(5.5.7) We will add a transient analysis to the ADE Analyses list. From the '**Choose Analyses**' form, select the '**tran**' option button and enter '1ms' in the 'Stop Time' text field. Set the 'Accuracy' option button to 'moderate'. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. OK the form. And the **tran** analysis should appear in the 'Analyses' section of the ADE window next to the DC Analysis. Turn the DC analysis OFF by unchecking the 'Enable' option.

(5.5.8) Add the two input nets – *vin* and *vin_cs* – to the Outputs list section of the ADE window to be plotted. You should already have *vout* and *vout_cs* in the list – if not, add them.

(5.5.9) Run the **Netlist and Run** command by clicking on the green play button. If the simulation runs correctly, you should see the text output window as well as the waveform output window as shown in Figure 28.



< Figure 56. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 57. The waveform output window with horizontal markers. >

(5.5.10) **Discussion**. In Figure 27, the input signal (top) and output (bottom) waveforms have been separated into their own graph spaces. Note that the input signals are the same for both resistor-loaded and current-source loaded amplifiers except for the DC input bias/offsets -780mV and 620.7mV respectively. From the output waveforms we can see both are riding on a VOUT ~ 1.25V (approximately VDD/2) satisfying the maximum swing requirement. The output waveform of the 20kΩ resistor-loaded circuit (blue) is considerably smaller in amplitude than the output waveform of the PMOS current-source loaded circuit (red).

5.6 Plotting Modes of the ADE

(5.5.1) Finally, notice at the bottom-right of the ADE window, under the Outputs section, there are two drop down menus: **Plot after simulation** and **Plotting mode**. Both of these options together determine the behaviour of the waveform output window (the Virtuoso Visualisation & Analysis XL tool by default) after running a simulation.

(5.6.2) From the ADE user guide. ‘Plot After Simulation’ menu options:

***Auto**: Automatically plots the outputs after a simulation is run. This is the default option. For every subsequent run, the new graph replaces, appends, or opens a new (sub) graph window according to the Plotting Mode option selected. Every time a graph is plotted, the default settings are used.*

***Refresh**: Updates open graphs in the output waveform window with new simulation results and maintains graph and trace settings. If no graph is open, the results are plotted in a new graph window and in subsequent runs, outputs are refreshed in the same graph. The Refresh option ignores the modes set in the Plotting Modes list.*

***None**: Does not plot output. You have to manually invoke the waveform output widow using the Plot Outputs button*

(5.6.3) From the ADE user guide. ‘Plotting Mode’ menu options.

***Replace**: Replaces the existing graph with the new graph. This is the default option.*

***Append**: Appends the new graph to the existing graph.*

***New SubWin**: Plots the new graph in a new sub window of the existing graph.*

***New Win**: Plots the new graph in a new window. Keeps the existing graph unchanged.*

These plotting options only work when the Plot after simulation list is set to **Auto**.

End of Lab!

Tutorial Lab 12.3 (Formatively Assessed):

Large Signal and Small Signal Operation of FETs

Preliminary Matter: Setting up the Cadence Environment

Please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout before entering these commands into your terminal.

For this course, you will use the Cadence Virtuoso Design Environment (referred to simply as Cadence). Cadence is a Linux based software suite for designing full custom integrated circuits. In order to use Cadence you will first need to set-up the Cadence environment.

8. Open a terminal window and ensure that you are in your home directory using the following command:

```
cd ~
```

Note: 'cd' stands for change directory. The solitary tilde symbol (~) is a shortcut/alias for your home directory (/home/username).

9. Under your home directory, use the following commands to load the Cadence environment:

```
module use /eda/cadence/modules
module load course/EENGM6030
```

Note: The two 'module' commands load the required shell/Cadence environmental variables for this course (paths, names, aliases…) In particular, this includes a reference to the process design kit (PDK) used in this course, the GPDK180. You will need to do this every time you open a terminal, since the sourced environment only applies to the current terminal (until it's closed).

a. ***Note: This step only needs to be completed ONCE - If you already ran this script in lab 12.1, you don't need to run it again here.*** The first time you use Cadence, you will need to run the following command (script):

```
EENGM6030_setup
```

This will create a folder called AICD (short for Analogue Integrated Circuit Design) under your home directory. The AICD folder contains the files and directories needed to run Cadence. Running this setup script more than once will result in a warning message that the AICD folder already exists. It will then exit without making changes.

10. The AICD folder will be used for all your Cadence related activities for this course. **You should always make sure you are in your AICD folder before you launch Cadence.** To run cadence, use the following commands:

```
cd ~/AICD
virtuoso &
```

Note: The 'virtuoso &' command starts Cadence. The ampersand symbol (&) causes the Cadence process to run in the background freeing up the terminal for other commands the '&' symbol is a feature of the terminal's shell (bash) not Cadence.

1. Learning Objectives of this Lab:

From the previous lab, you should know how to:

- Parameterise components in a schematic using design variables. Copy design variables from the schematic into the ADE. Initialise the design variables in the ADE.

- Carry out a DC operating point analysis. Annotate a schematic with DC node voltages, DC operating point information, and other model/circuit parameters.

- Carry out a DC sweep analysis and a parametric analysis (using design variables). Use the parametric analysis tool to add more variables to a DC-sweep.

- Simulate the terminal characteristics of the nmoshv/pmoshv FETs from the GPDK180 technology library. Plot the I_{DS} vs. V_{GS}, I_{DS} vs. V_{DS}, and I_{DS} vs. V_{DS} vs. V_{GS} family of curves.

- Create and simulate a resistor-loaded NMOS FET single-stage amplifier. Understand the conflicting requirements of the simple resistor-loaded FET amplifier – specifically, the need to maximise the small-signal gain of the amplifier by using a large load resistance but at the same time the need to constrain the size of the load resistance in order to set an appropriate DC operating point.

- Create and simulate a PMOS current source loaded NMOS FET single-stage amplifier. Understand how the use of a current source load allows us to simultaneously and independently maximise the small-signal gain of the amplifier (infinite small-signal resistance) and set the DC operating point of the circuit at an appropriate level (i.e., that its DC current is independent of its small-signal resistance).

In this lab, you will:

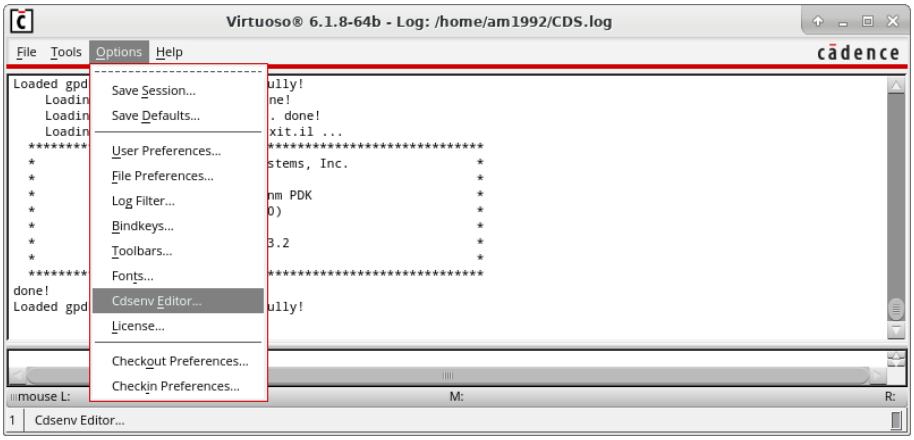
- Develop the small-signal model of the FET amplifier from the large-signal terminal characteristics simulated in the previous lab. Determine how to set the bias point to maximise performance. Understand the role of operating point calculations in AC analysis. Carry out a frequency sweep analysis of a resistor-loaded NMOS FET single-stage amplifier using an AC analysis.

2. Cadence Documentation, Help, and Licensing

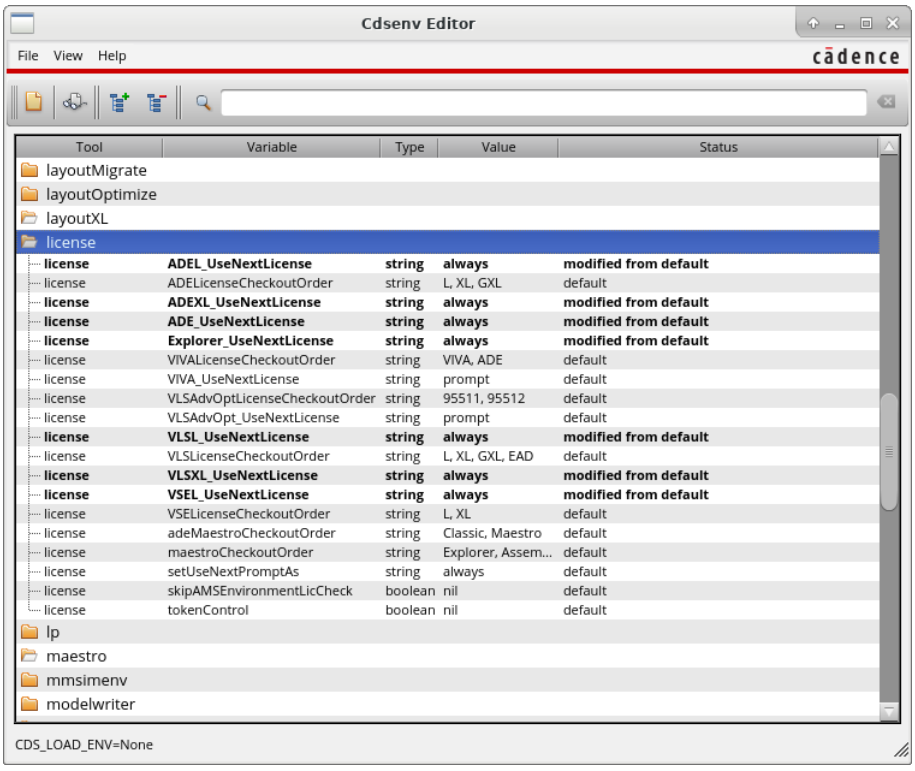
Local and online help is available from the help menus (**Menu: Help → User Guide** or by pressing **F1**), or by typing 'cdnshelp &' at the command prompt (assuming you have setup your terminal environment).

PLEASE NOTE: WHEN LAUNCHING A TOOL IN THE CADENCE DESIGN ENVIRONMENT YOU MAY SOMETIMES GET A WARNING ABOUT A LICENSE BEING UNAVAILABLE FOR THE TOOL, AND REQUESTING PERMISSION TO OBTAIN A HIGHER-TIERED LICENSE. YOU SHOULD SELECT 'ALWAYS' TO THESE MESSAGES.

If you accidently deny a license request, or you cannot access a license, you will need to modify the Cadence environment licence/permissions from the CIW. From the CIW menu go to **Menu: Options → Cdsenv Editor**. Once in the environment editor, go to the license folder, and make sure all licenses (in bold) are set to '**always'** – if any entries are listed as '**never**' you will need to manually change/retype them to '**always**'.



< Figure X.1. The Cadence Environment Menu Option from the CIW Window. >



< Figure X.2. The Cadence Environment Editor. Note, all licenses should be set to 'always'.>

3. Getting Started with Cadence:

For a refresher/introduction on how to start a Cadence session please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout.

3.1 Dealing with Lock files

PLEASE NOTE: IF A SCHEMATIC ISN'T PROPERLY CLOSED DOWN (FOR EXAMPLE, DUE TO A DROPPED REMOTE CONNECTION), THE SCHEMATIC MIGHT BECOME EDIT LOCKED. IF THIS HAPPENS YOU WILL NOT BE ABLE TO OPEN THE FILE FOR EDITING. YOU WILL ONLY BE ABLE TO OPEN THE FILE FOR READING UNTIL YOU REMOVE ANY/ALL LOCK FILES.

There are two ways you can unlock files:

(3) The first is to manually delete any edit lock files. To do this, go to your '~/AICD/Designs' folder and find the folder representing the schematic e.g., for lab1 this would be '~/AICD/Designs/lab121_vdivider/schematic' if this is what you called your schematic. Delete all the .cdslck lock files in this folder and you should be able to open your schematic for editing.

(4) You can also automatically release edit locks with the command:

```
clsAdminTool -are .
```

To use this command, you have to be in your AICD folder (the full-stop is shorthand for the current directory path). If you are not in your AICD folder, you have to specify the path (relative or absolute) to your AICD directory.

Since this is a Cadence command, you have to ensure that you have set up the terminal environment for Cadence (in particular, the modules must be loaded as in step 2, 'Preliminary Matter'). It is best to use the terminal window that you used to start Cadence, since the environment was already set up. Note that you will not be able enter any commands into your terminal if you did not start cadence with the '&' character. By example, to release any lock files after opening a new terminal, the full command set would be,

```
cd~
module use /eda/cadence/modules
module load course/EENGM6030_2021
clsAdminTool -are AICD
```

For more information on this command, use

```
clsAdminTool -help
```

4. Circuit Simulation Exercises: Part 1

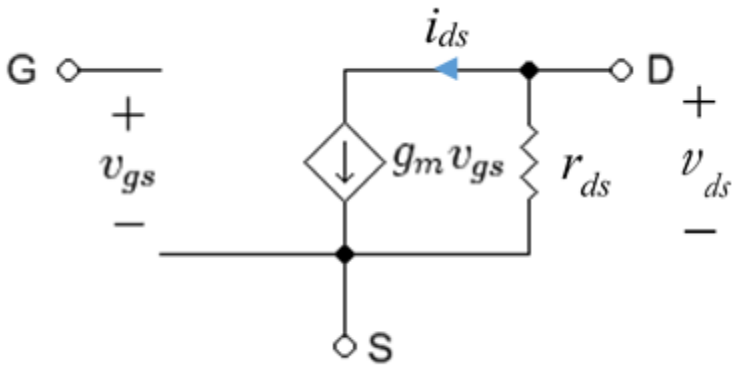
With guidance, you will:

- Examine the relationship between the overdrive voltage and the transconductance. Understand weak channel-inversion, strong channel-inversion and carrier-velocity saturation.

- Understand the small-signal model of a FET amplifier at a bias point. Determine how best to choose a bias point given the large-signal terminal characteristics of an NMOS/PMOS device.

4.1 The Small-Signal Model

Consider, the simplest small-signal model of the MOSFET,



< Figure 58. >

Where g_m is the small-signal transconductance, the proportionality constant that relates the output signal current i_{ds} to the input signal voltage v_{gs}, and where the small-signal resistance r_{ds} models the non-ideal dependency of i_{ds} on the output signal voltage v_{ds}. In this lab, we will relate the large-signal characteristics of the NMOS device to the small-signal model and determine – through simulation – how to choose the DC operating point. We will revisit the small-signal model (Figure 1) in part 2 of this lab when we introduce the AC analysis.

4.2 Plotting the Small-Signal Transconductance and Output Impedance Curves

(4.2.1) In the large-signal domain, I_{DS} is a non-linear function of V_{GS} and V_{DS},

$$I_{DS} = f(V_{GS}, V_{DS})$$

The small-signal transconductance g_m and output impedance r_{ds} are defined as,

$$g_m(V_{GS}) = \frac{\partial I_{DS}}{\partial V_{GS}} \qquad r_{ds}(V_{DS}) = \left[\frac{\partial I_{DS}}{\partial V_{DS}}\right]^{-1}$$

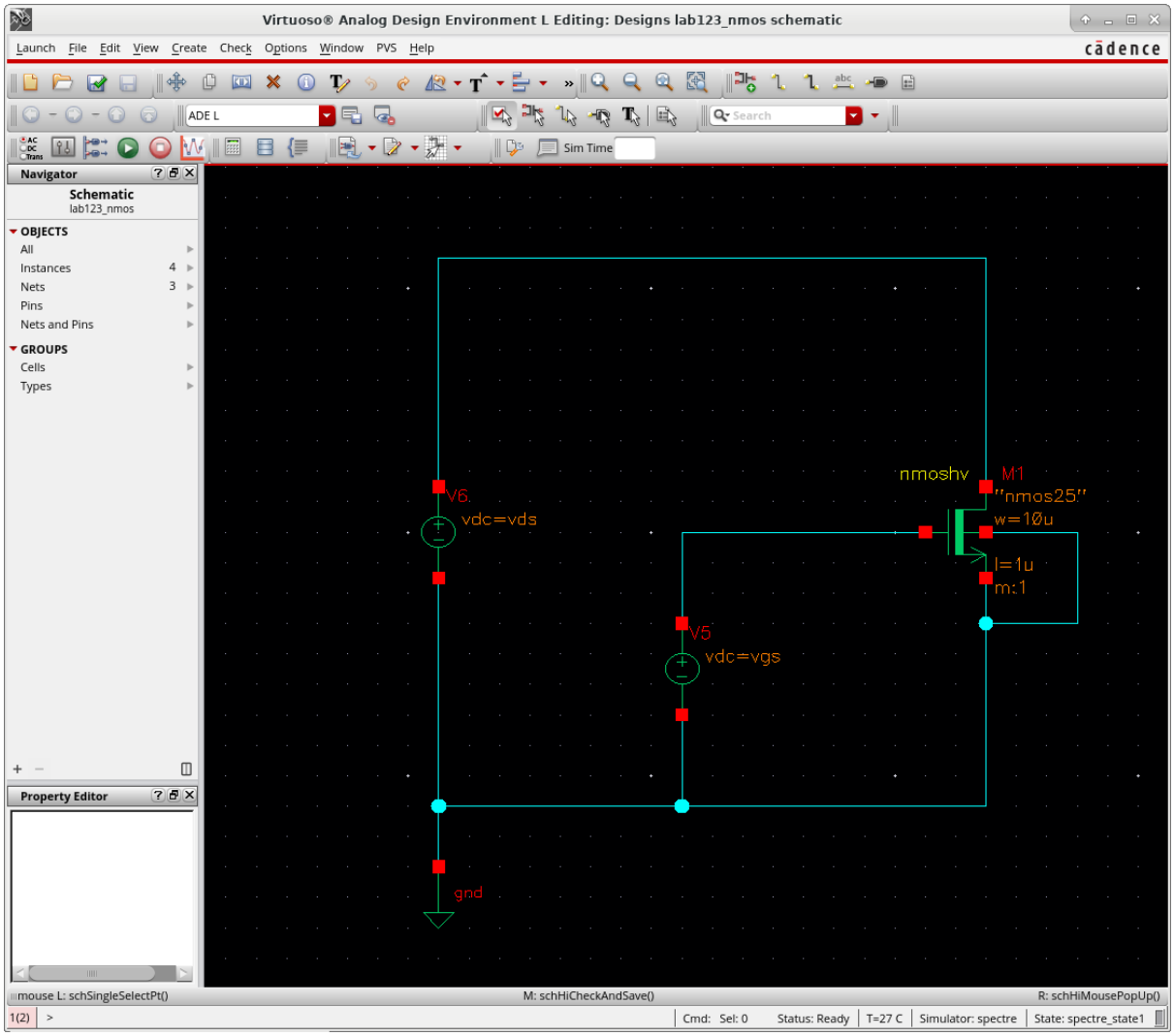
The derivative of the large-signal I_{DS} vs. V_{GS} plot will therefore give us a plot of g_m vs. V_{GS}. Likewise, the derivative of the large-signal I_{DS} vs. V_{DS} plot will give us a plot of r_{ds} vs. V_{DS}. For a fixed bias point, the small-signal parameters g_m and r_{ds}, are simply a single point along their respective curves. (Note that in the case of g_m, we assume V_{DS} is held constant and in the case of r_{ds} we assume V_{GS} is held constant.)

(4.2.2) In part 1 of the previous lab [12.2], we plotted the curves of I_{DS} vs. V_{GS} (with V_{DS}=2.5V) and I_{DS} vs. V_{DS} (with V_{GS}=0.7V) for an unloaded NMOS FET. From these graphs we want to obtain the curves of g_m vs. V_{GS} and r_{ds} vs. V_{DS}. Cadence virtuoso has an in-built calculator that allows us to post-process the results of our simulations. We re-run last lab's simulations and use the cadence calculator to obtain the required curves.

(4.2.3) Open the schematic you created in part 1 of lab 12.2 ('lab122_nmos' or whatever name you chose). You can also copy the cell to create a new file e.g. lab123_nmos. (You should now know how to do this without instruction.)

(4.2.4) Ensure that your schematic is properly parameterised using design variables: recall, for the DC supply voltage vdc=vds and for the DC gate-to-source voltage vdc=vgs. The NMOS device should be W/L = 10u/1u. See Figure 2. Ensure that it checks and saves without warnings or errors. Use the CIW to diagnose any warnings or errors.

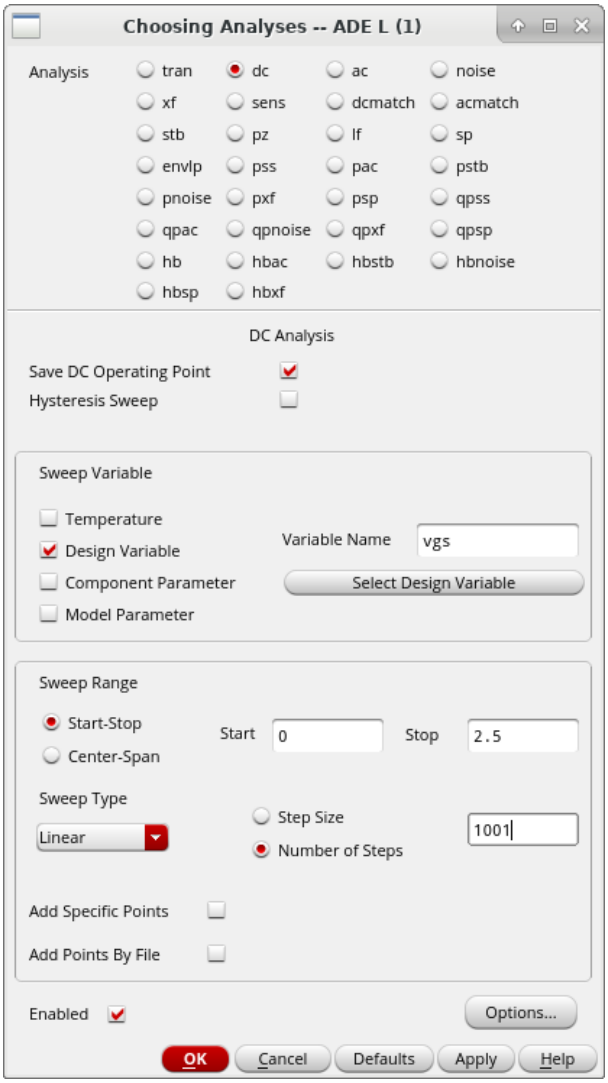
(4.2.5) From the schematic window, launch the ADE tool (**Menu: Launch à ADE L**). From the ADE window, copy the design variables from the schematic into the ADE (**Menu: Variables à Copy From Cellview**). You should now see the *vds* and *vgs* variables in the 'Design Variables' list of the ADE. Initialise *vds = 2.5* and *vgs = 0.7*. Note, that these initial values are only used to calculate the DC operating point of the circuit.



< Figure 59. The NMOS (10u/1u) testbench circuit with Design Variables for the DC supply voltage (vdc=vds) and the DC gate-to-source voltage (vdc=vgs). >

(4.2.6) From the ADE window, setup the DC sweep analysis for I_{DS} vs. V_{GS} (**Menu: Analyses à Choose**). We wish to sweep the input voltage (vgs) of the NMOS device from 0V to 2.5V while observing the drain current (IDS). We will use 1001 steps in the sweep to give the output waveform sufficient resolution.

In the choosing analyses form, select the '**dc**' option button. Check the 'Save DC Operating Point' box. Check the 'Design Variable' box as the sweep variable. Type *vgs* as the variable name. Choose the 'Start-Stop' option button with Start = 0 and Stop = 2.5 for the sweep range. Select 'Linear' from the drop down menu as the sweep type. Choose the 'Number of Steps' option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. See Figure 3. OK the form. The **dc** analysis should appear in the 'Analyses' list of the ADE window.



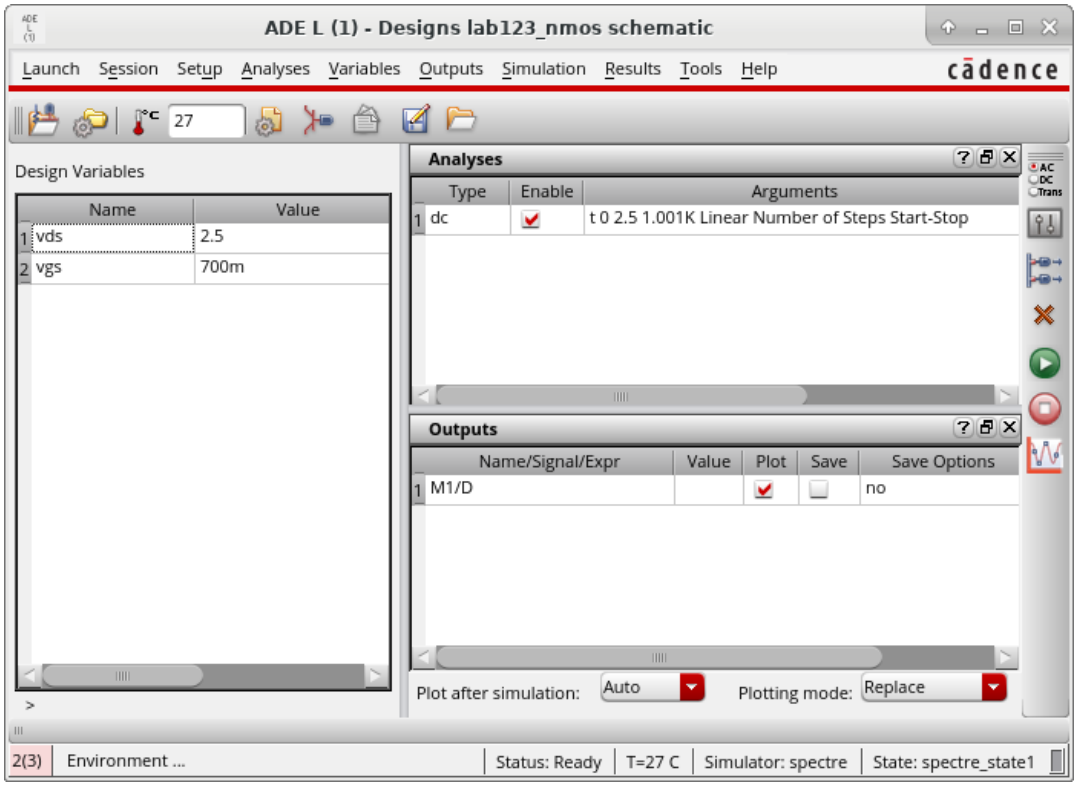
< Figure 60. The Choosing Analyses Form for a DC sweep. >

(4.2.7) Select only the drain node current (I_{DS}) for plotting. From the ADE, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** Select only the drain node of the NMOS device (M1/D). **Press the 'ESC' key to exit the plotting mode of the ADE**.

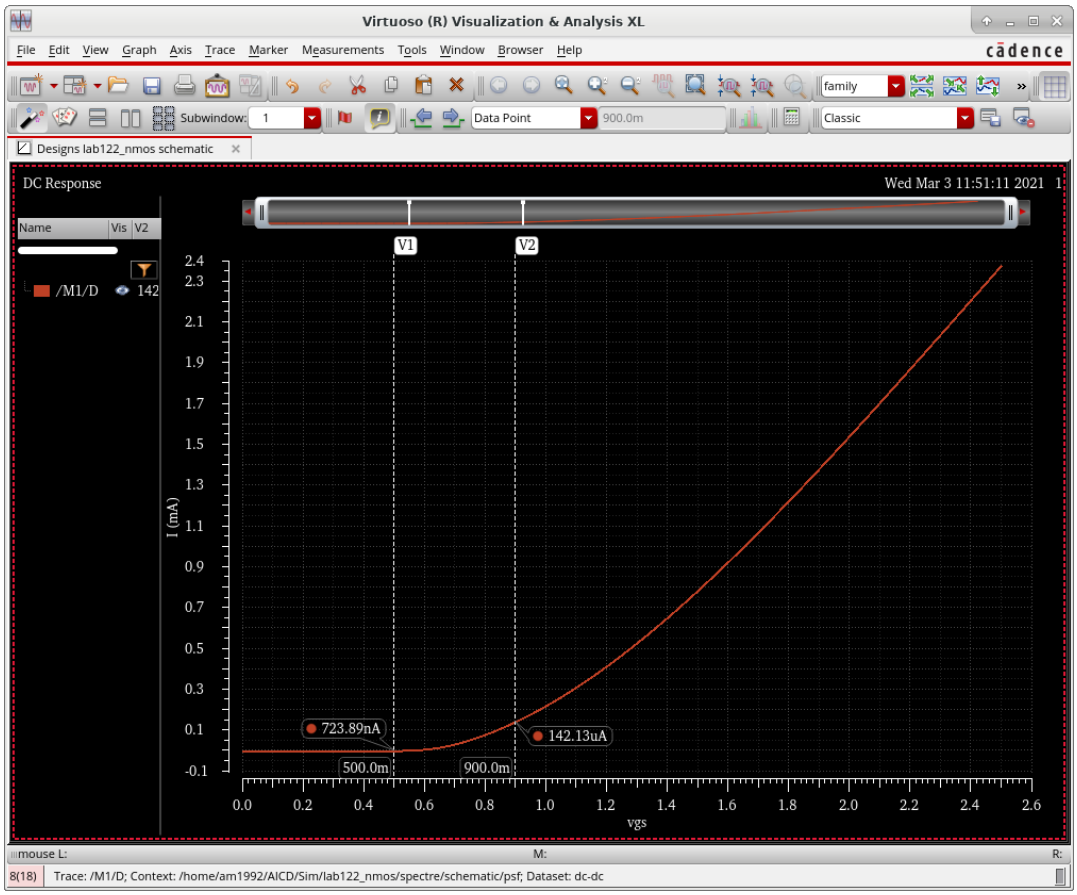
*Recall**,** if you want to plot a particular voltage, you click on the net/wire. If you want to save and plot the currents, you have to click a terminal **node** of a component (not the net/wire).*

(4.2.8) In the ADE window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error and both the text output window as well as the waveform output window should open automatically. You should get a plot of I_{DS} vs. V_{GS}. See Figure 5.

*Recall, you can add point, horizontal, and/or vertical markers to your graph. From the waveform output window, choose **Menu: Markers à Create Marker…** to bring up the 'Create Graph Marker' form. Or use the keyboard shortcuts to add a marker to your plot: '**m**' (point marker), '**h**' (horizontal marker), and '**v**' (vertical marker).*



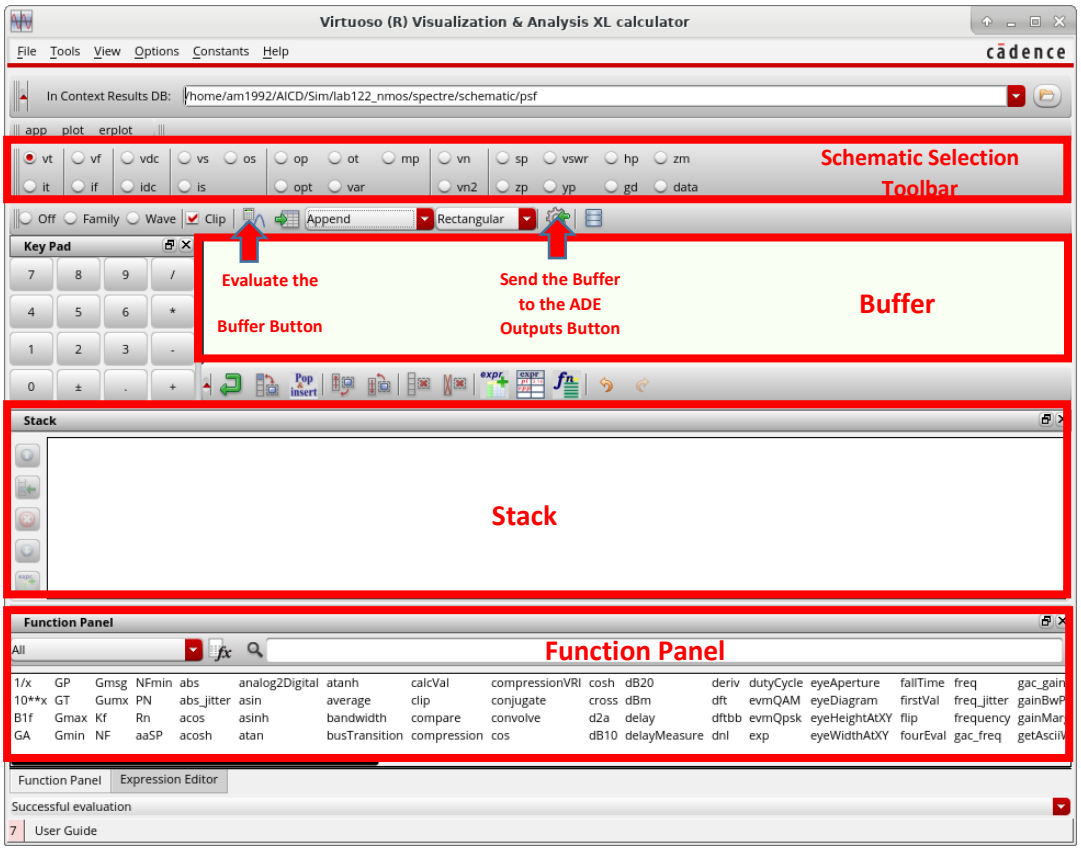
< Figure 61. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 62. The waveform output of IDS vs. VGS with vertical markers at 0.5V and 0.9V. >

4.3 Using the Built-in Waveform Calculator

(4.3.1) The calculator tool allows you to post-process your results by applying mathematical operations to your simulation data. To launch the calculator tool, from the ADE window, choose **Menu: Tools à Calculator…**



< Figure 63. The VIVA XL Calculator. >

(4.3.2) The principal components of the calculator are:

1. **Schematic Selection Toolbar.** Near the top of the calculator is the Schematic Selection Toolbar. This is how we get data from the simulation into the calculator. This is similar to the 'To Be Plotted' command in the ADE. This toolbar contains a selection of option/radio buttons for selecting the nets (voltages) and/or nodes (currents) on the schematic following a particular analysis. When you select one of the option buttons, the schematic will come into focus allowing you to select the required net or node.

2. **Buffer**. When you select the net or node using the schematic selection toolbar, or by sending a trace from the waveform output window directly to the calculator, the expression underlying the net, node, or waveform will appear in the buffer of the calculator. Any mathematical operations performed on the expression will also be shown in the buffer.

3. **Stack**. You can push/store any expression you have built in the buffer on to the stack by either pressing enter on the keyboard or using the enter icon button (just beneath the buffer area). You can also load/pop an expression (assuming you have any stored) from the stack into the buffer. You can also swap an expression on the stack with the expression in the buffer. The stack can be up to eight instructions deep.

4. **Function Panel**. The function panel contains a large number of standard mathematical and engineering functions that can be applied to the expression in the buffer. Expressions of any complexity can be built, using as many functions as required. The function you are applying has to be compatible with the data from the simulation. The list of functions are categorised according to the drop down menu in the function panel. Take some time to explore the function list/categories.

5. **Evaluate the Buffer Button**. This button evaluates the expression in the buffer. Depending on the expression, and the mathematical functions applied to the expression, the buffer may evaluate as a scalar or as a waveform. A scalar evaluation will be displayed in the buffer area – replacing the original expression. A waveform evaluation will be displayed in a waveform output window. The waveform will be plotted according to the calculator plotting mode in the drop down menu next to the evaluate button (i.e. replace, append, new Win, new SubWin). Append is the default.

6. **Send Buffer Expression to ADE Outputs**. Once an expression has been built in the buffer of the calculator, it can be sent to the Outputs section of the ADE. This expression will then be treated the same as any other output in the list.

(4.4.3) Note: The calculator has two input modes: Reverse Polish Notation (RPN) and Algebraic mode. The calculator mode is set to RPN by default. Change this to algebraic. From the calculator window choose **Menu: Options à Mode à Algebraic Mode**. This setting doesn't change how the calculator operates, but it does change the behaviour of the calculator when adding or manipulating expressions to/in the buffer. The most important behaviours to note are (1) in algebraic mode, adding a function (from the function panel) to the buffer will place that function wherever the cursor is located, and (2) in algebraic mode, when applying a function you need to manually add the closing bracket.

(4.3.4) Note: An expression in the calculator will only successfully evaluate if the simulation data is compatible with the expression in the buffer. When you use the Schematic Selection Toolbar, each button can only be used to acquire the simulation data for a specific analysis type. The following rules apply:

vt – select transient analysis data at a net (voltage).
it – select transient analysis data at a node (current).

vf – select AC 'frequency response' analysis data at a net (voltage).
if – select AC 'frequency response' analysis data at a node (current).

vdc – select DC operating point analysis data at a net (voltage).
idc – select DC operating point analysis data at a node (current).
vs – select DC sweep analysis data at a net (voltage).
is – select DC sweep analysis data at a node (current).
os – select DC sweep analysis data (operating point information) of a device.

It is possible to build up an expression using the wrong option buttons. For example, if you use **vt** (transient data) when you have only carried out a DC analysis, you will get an error on evaluation ('`Expression evaluates to nil`'). So, make sure you use the correct schematic selection button for the simulation. The other option buttons are associated with analyses we do not use in this course. Feel free to peruse the user guide for more information (**Menu: Help à VIVA XL User Guide**)

(4.3.5) Note: You can also send a signal to the calculator directly from the waveform output window (assuming you have plotted any waveforms). This is often the easiest method. Right-click on the trace in the waveform output window, and select **Send To à Calculator** from the context menu. This will send the underlying expression for the trace to the buffer of the calculator.

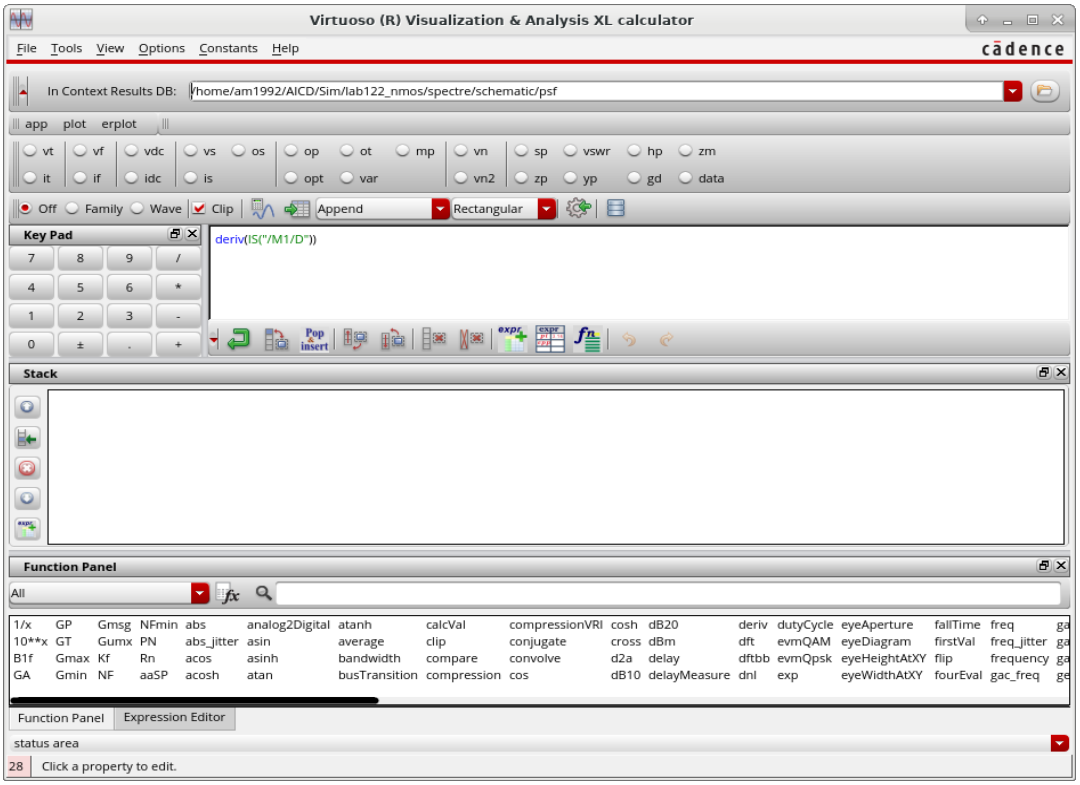
Plotting g_m vs. V_{GS}

(4.3.6) Use the schematic Selection Toolbar to get the plot of I_{DS} vs. V_{GS} into the calculator. Select the '**is**' option button (for current sweep) on the schematic selection toolbar. When you select the '**is**' button the schematic editor window will come into focus. Select only the drain node current (I_{DS}) by clicking on the drain node of the NMOS device (M1/D). The calculator should return to focus and you should see the expression for the drain current IS("/M1/D") in the buffer area of the calculator.

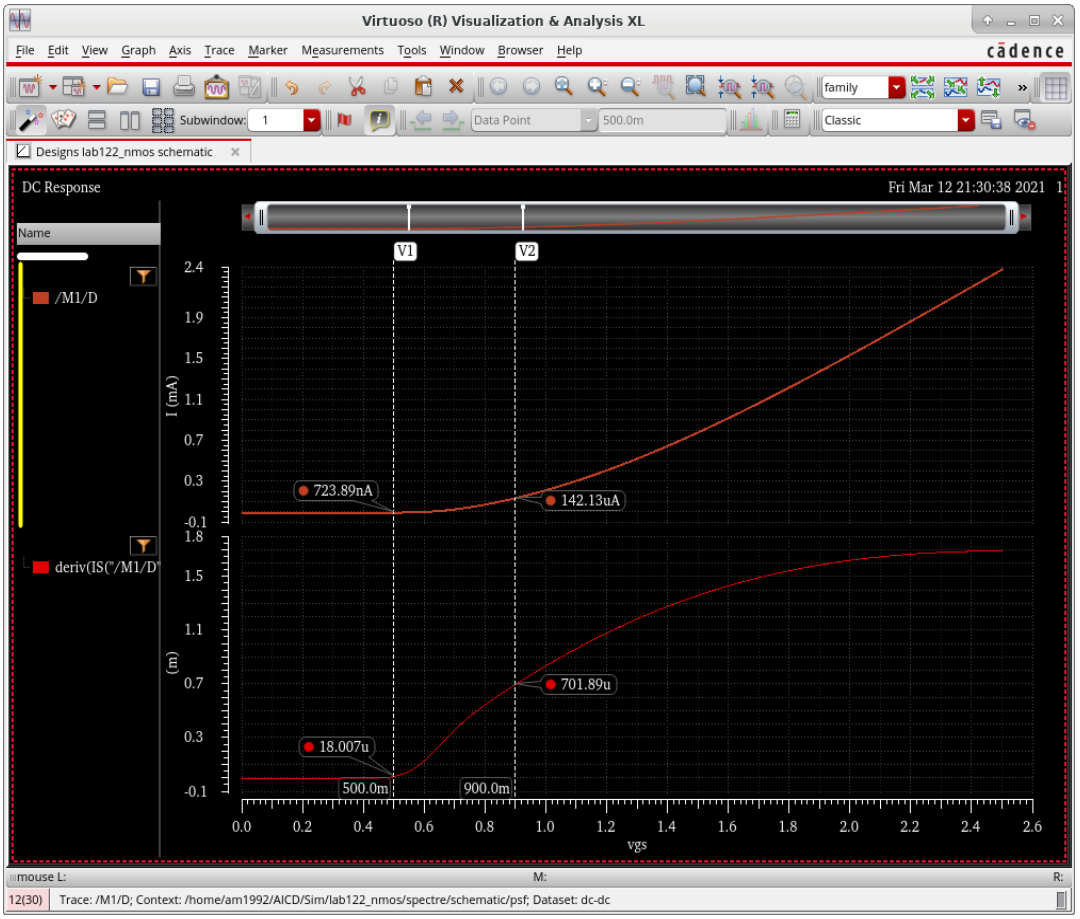
(4.3.7) With the expression for I_{DS} vs. V_{GS} in the buffer, apply the derivative function from the function panel. In the function panel of the calculator window, select '**All**' from the drop down menu. Place the cursor in the correct positon (or highlight the entire expression) in the buffer and select the '**deriv**' function from the list of functions. The functions are listed alphabetically so either search the list or use the text search box just above the listing area. (Note that the **deriv** function is categorised under special functions.) Make sure to close the brackets after you select the **deriv** function. The expression in the buffer should be deriv(IS("/M1/D")). See Figure 7.

(4.3.8) In the calculator window, evaluate the buffer expression by using the **Evaluate the Buffer** button. If you set up the expression correctly and the simulation data exists (you should have run the DC sweep simulation before opening the calculator) the waveform of dI_{DS}/dV_{GS} vs. V_{GS} should be plotted alongside the original plot (assuming you didn't close it down and that the calculator plotting mode). This is a plot of the small-signal transconductance of the device versus the DC gate-to-source bias voltage - g_m vs. V_{GS}.

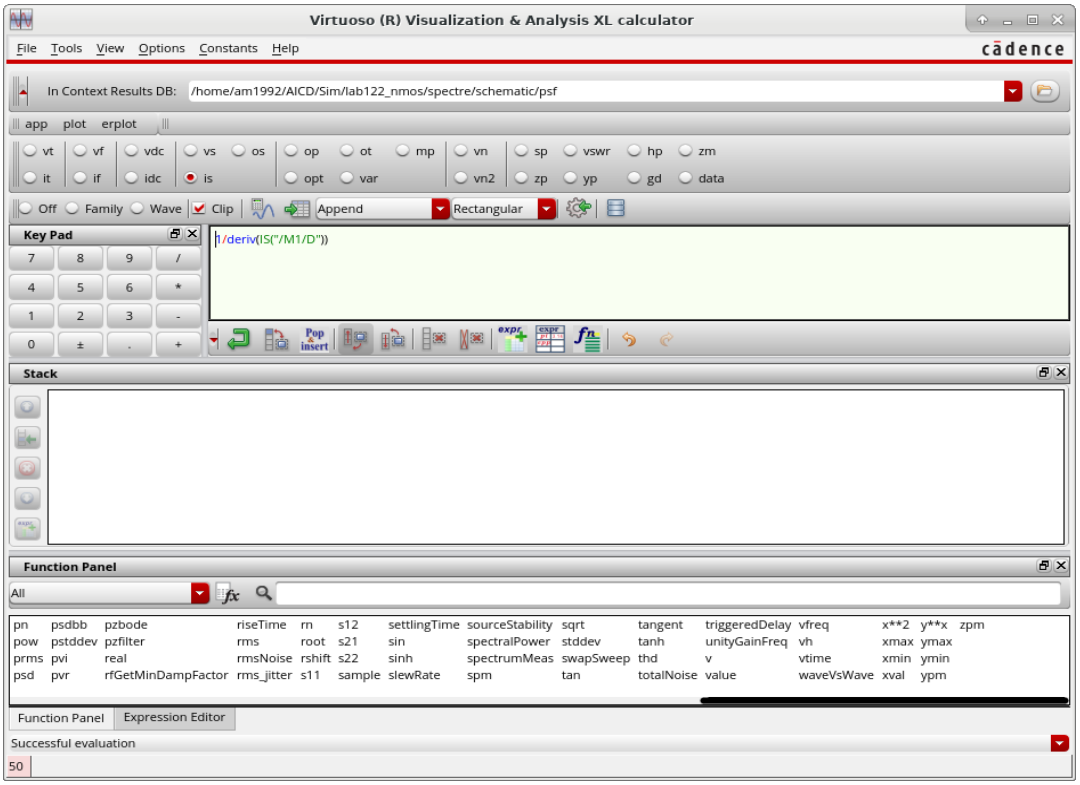
(4.3.9) Split the traces for clarity. Right-click on the empty area of the graph window and select S**plit Current Strip à trace** from the context menu.



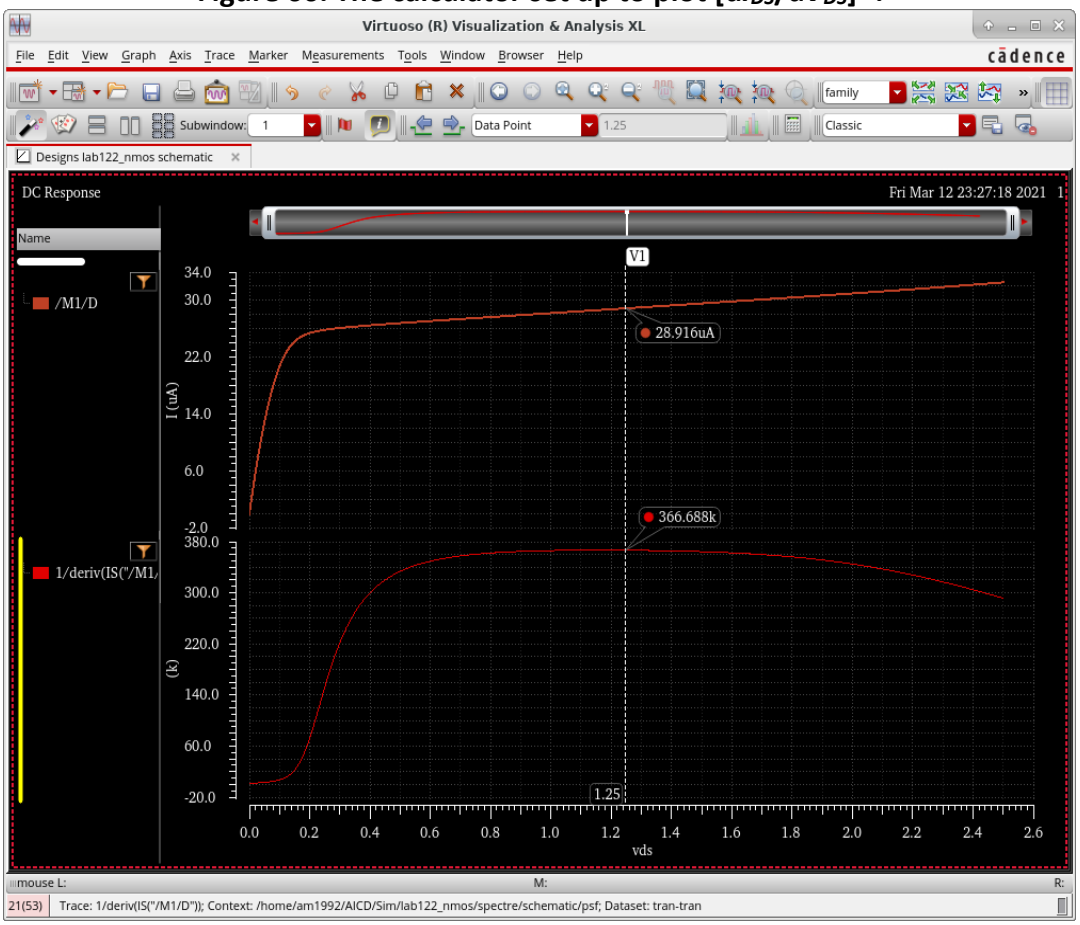
< Figure 64. The calculator set up to plot dI_{DS}/dV_{GS}. >



< Figure 65. The plots of I_{DS} vs. V_{GS} (top) and g_m=dI_{DS}/dV_{GS} vs. V_{GS} (bottom). V_{DS}=2.5. >



< Figure 66. The calculator set up to plot [dI_{DS}/dV_{DS}]$^{-1}$. >



< Figure 67. The plots of I_{DS} vs. V_{DS} (top) and r_{ds}=dI_{DS}/dV_{DS} vs. V_{DS} (bottom). V_{GS}=0.7. >

Plotting r_{ds} vs. V_{DS}

(4.3.10) Plot I_{DS} vs. V_{DS}. From the ADE window, edit the DC analysis (**dc**) to make V_{DS} the sweep variable. Either double click on the analysis or right-click on the analysis and choose edit from the context menu. See Figure 3. Change the sweep variable name from V_{GS} to V_{DS}. Leave the rest of the form unchanged. Make sure V_{GS} = 0.7V in the ADE. OK the form.

(4.3.11) In the ADE window, run the DC sweep simulation. The simulation should run properly and both the text output window as well as the waveform output window should open automatically.

(4.3.12) The expression from the previous exercise should still be in the buffer. If not, use the schematic selection toolbar to get the plot of I_{DS} vs. V_{DS} into the calculator following the same steps as above. To get the reciprocal of the expression you can manually modify the expression in the buffer to either 1/deriv(IS("/M1/D")).

(4.3.13) In the calculator window, evaluate the buffer expression by using the **Evaluate the Buffer** button. If you set up the expression correctly and the simulation data exists (you should have run the DC sweep simulation before opening the calculator) the waveform of dV_{DS}/dI_{DS} vs. V_{DS} should be plotted alongside the original plot (assuming you didn't close it down and that the calculator plotting mode). This is a plot of the small-signal output resistance of the device versus the DC drain-to-source bias voltage - r_{ds} vs. V_{DS}. At a value of $V_{DS} = V_{dd}/2$=1.25V, the output resistance is r_{ds} = 366.688KΩ (Figure 10, bottom).

4.4 Discussion: Choosing the Bias Point

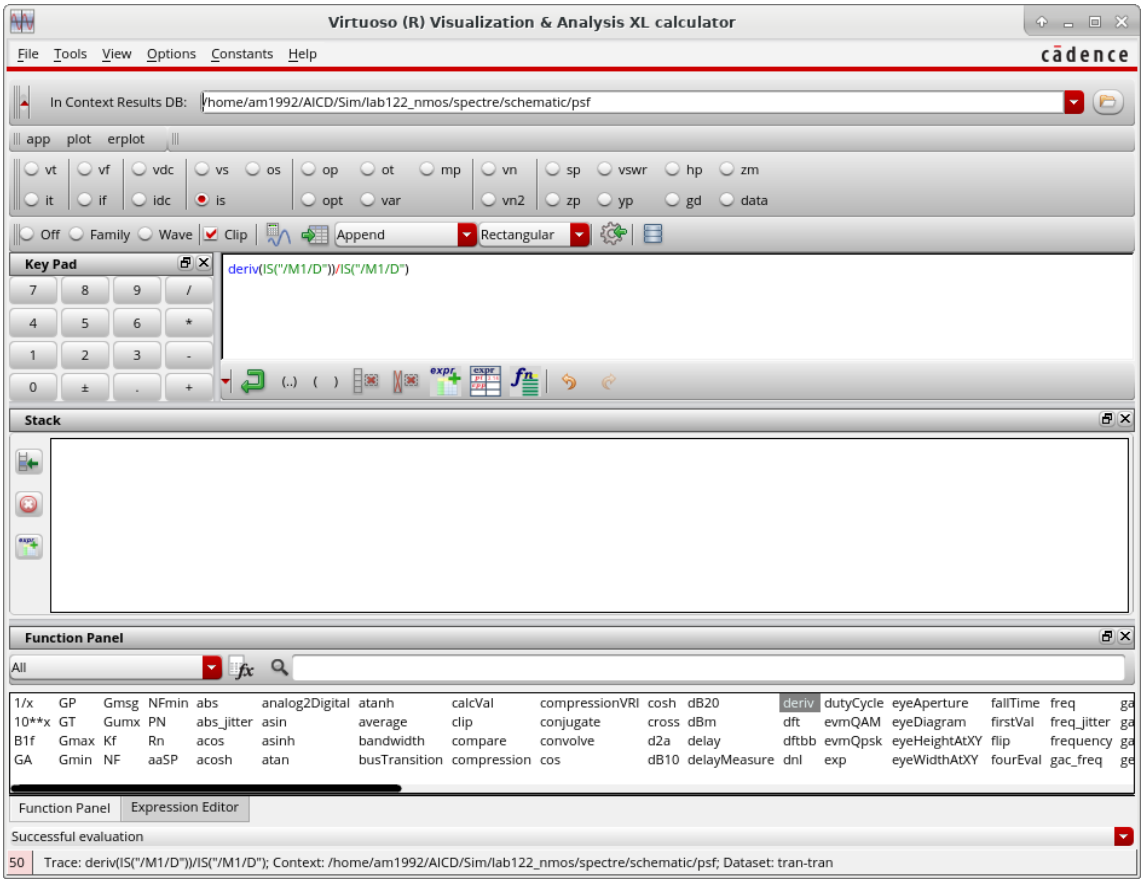
We are not just interested in the value of g_m. We also need to consider the power draw for a given g_m. As we increase the drain current, and consequently power consumption, we get a diminishing return on g_m; that is, we spend a lot of power for only a small increase in g_m. The metric g_m/I_{DS} is measure of bias efficiency – it tells us where on the curve of I_{DS} vs. V_{GS} we are getting the most return (g_m, gain) for our current draw (power consumption).

Plotting g_m/I_{DS} vs. V_{GS}

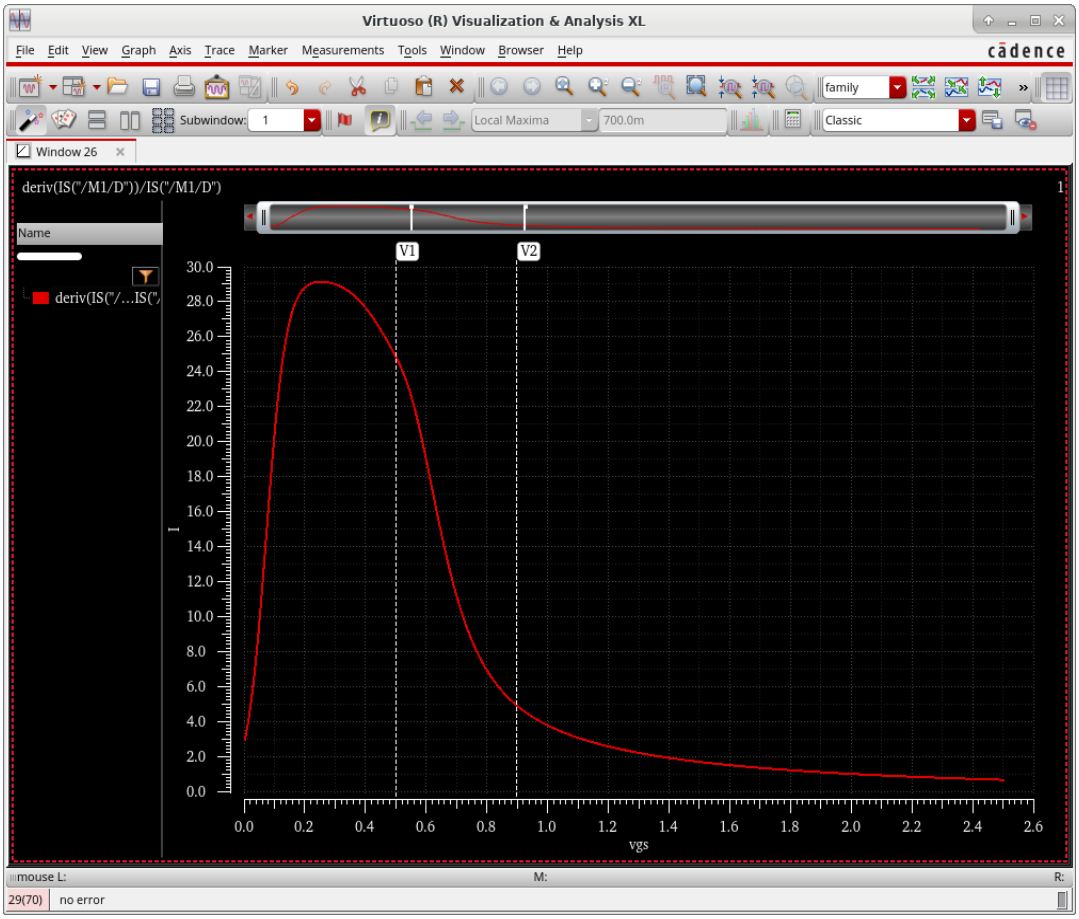
(4.4.2) We will plot the curve of g_m/I_{DS} vs. V_{GS}. using the calculator. Change the sweep variable back to V_{GS} and re-simulate the DC sweep analysis (see Figure 3). As you did above, build the expression deriv(IS("/M1/D"))/(IS("/M1/D")) in the calculator buffer. This will plot g_m/I_{DS} vs. V_{GS}. See Figure 11 and 12.

Use the schematic Selection Toolbar. Select the 'is' option button (for DC current sweep) on the schematic selection toolbar. When you select the 'is' button the schematic editor window will come into focus. Select only the drain node current (I_{DS}) by clicking on the drain node of the NMOS device (M1/D). The calculator should return to focus, and you should see the expression for the drain current IS("/M1/D") in the buffer area of the calculator. Use the ***deriv*** *function to plot the derivative of the drain current deriv(IS("/M1/D")). This will give us a plot*

*of g_m vs. V_{GS}. Add a backslash / character to the end of the expression for division. To add the final current term, you can either copy/paste the string to the end of your expression or use the schematic selection tool to append the drain current to the expression by selecting the '**is**' button and again clicking on the drain node of the NMOS device (M1/D). You should now have deriv(IS("/M1/D"))/(IS("/M1/D")) in the expression buffer. See Figure 11.*



< Figure 68. The calculator set up to plot the derivative of the g_m/I_{DS} vs. V_{GS}. >



< Figure 69. The plot of g_m/I_{DS} vs. V_{GS}. V_{DS}=2.5. >

*final current term, you can either copy/paste the string to the end of your expression or use the schematic selection tool to append the drain current to the expression by selecting the '**is**' button and again clicking on the drain node of the NMOS device (M1/D). You should now have deriv(IS("/M1/D"))/(IS("/M1/D")) in the expression buffer. See Figure 11.*

(4.4.3) **Discussion**. In section 1.2.5 (page 11) referring to fig. 3 of the background material, note that there are four regions of the I_{DS} vs. V_{GS} characteristic curve – the sub-threshold region, the exponential region, the square-law region and the velocity saturation region.

When biasing a MOSFET for use as an amplifier, we want to bias the device in its square-law region of operation, where the drain current is proportional to the square of the overdrive voltage. In this region, we have a strong channel-inversion giving us a good return in terms of a performance vs. power consumption trade-off. Recall the first-order equation for the drain current in the square-law region (assuming the device is in the saturation region) is,

$$i_{DS} = \frac{1}{2}k'\left(\frac{W}{L}\right)(V_{GS} - V_t)^2(1 + \lambda V_{DS})$$

where $k = k'\left(\frac{W}{L}\right) = \mu C_{ox}\left(\frac{W}{L}\right)$. There are three inter-related expressions for the small-signal transconductance g_m in the square-law region (repeated here).

$$g_m = k(V_{OV}) = 2\sqrt{ki_{DS}} = \frac{2i_{DS}}{V_{OV}}$$

In general, we see that the transconductance is proportional to the *k* process parameter, the drain current I_{DS}, and the overdrive voltage $V_{OV} = V_{GS} - V_t$. The higher the overdrive voltage or the higher the drain current, the higher the transconductance. This holds true up until we reach velocity saturation. In the velocity saturation region, where the maximum velocity of the charge carriers in the channel has been reached, we get increasingly small returns on g_m (and consequently gain) for any increase in overdrive voltage or drain current and therefore power consumption – lowering overall power efficiency.

Lower the drain current or overdrive voltage too much, and we come out of the predictable square-law region – where all the design equations/approximations work – and enter the exponential region. Here, we have weak channel-inversion, where we operate awkwardly close to the threshold voltage – where process, voltage, and/or temperature (PVT) variations may accidentally turn the MOS device off completely (i.e., we cross into the sub-threshold region). In addition, the exponential region of operation is noisy due to the low drain current density.

We find as a general rule, working with an overdrive voltage of 0.2V (VGS=0.7V) – we get a good transconductance for the bias current draw, g_m ~ 365uS. This puts us firmly in the square-law/strong-inversion region, sufficiently away from the weak-inversion/exponential region where noise, PVT, and model accuracy are problematic, and away from the velocity saturation region where power efficiency is poor. Moreover, aiming for a V_{OV}=0.2V holds true regardless of process node or technology.

What about VDS? In the equation for IDS above, the output resistance is determined by the channel length modulation parameter λ. From the background material, the MOSFET output resistance is then given by,

$$r_{ds} = \frac{1}{i_{DS}\lambda}$$

Recall from the previous lab that we choose VDS≈VDD/2 to give us maximum signal swing. We note from Figure 10, that the small-signal resistance r_{ds} is approximately flat over the range 0.6V to 2V (with r_{ds} ≈ 350–370kΩ). If we wanted to maximise linearity, a VDD of approximately 1.2V would be optimal – which is close enough to mid-rail.

So, for the fixed bias point VGS=0.7V and VDS=1.2V, IDS ≈ 28.8uA, r_{ds} ≈ 367kΩ, and g_m ≈ 365uS. From the large-signal characteristics (Figures 8 and 10) we have estimated the small-signal parameters g_m and r_{ds} (as in Figure 1).

In the next part of the lab, we will turn our attention to the AC analysis (aka small-signal analysis, linear analysis, or linear network analysis).

5. Circuit Simulation Exercises: Part 2

With guidance, you will:

- Understand the relationship between the AC analysis and the DC operating point.
- Simulate the frequency response of the basic resistor-loaded single-stage NMOS FET amplifier.

The AC analysis is a small-signal analysis. As we stated in the introduction (part 1), small-signal analysis assumes that a circuit has been linearised around a fixed DC operating point (or bias point). When you simulate a device using an AC analysis, you are not simulating the actual device but an idealised, linear model of the device. The input-output relationship of a circuit in an AC analysis is therefore *always* linear – regardless of the input signal size. (Note, that in an AC analysis, you can drive a circuit with any size of input – you are not limited to small perturbations of the input since you are not driving the actual circuit, but the linearised version of it. So, take care when interpreting the response of the simulation - just because you drive it with a 1V signal in AC analysis and get a linear response, this does not mean that it is realistic to do so with the actual circuit – e.g., due to clipping or distortion.)

When you carry out an AC (small-signal) analysis, the simulator first calculates the DC operating points. (Note that this happens automatically, there is no separate 'Save DC Operating Point' option, unlike the DC analysis.) The small-signal parameters of the devices/circuit are then calculated from the DC operating points. For a MOSFET, the small-signal parameters include voltages (vgs, vds, vgd…), currents (ids, igs, igd…), transconductances (gm, gmb…), parasitic capacitances (cgs, cds, cgd…), conductances (gds, gbd…), and many more. All of these parameters are used to construct the small-signal linear model of the MOSFET – which at its core is based on the circuit given in Figure 1 (although it has dozens of parameters). The linear input-output response of the device/circuit using the small-signal parameters is then calculated.

Aside: the full list of small-signal parameters for the GPDK180 NMOS is: *ids, isub, vgs, vds, vbs, vgb, vdb, vgd, vth, vdsat, vfbeff, gm, gds, gmbs, betaeff, cjd, cjs, qb, qg, qd, qbd, qbs, cgg, cgd, cgs, cgb, cdg, cdd, cds, cdb, csg, csd, css, csb, cbg, cbd, cbs, cbb, ron, id, is, ibulk, ibs, ibd, pwr, gmoverid, ueff, cgsovl, cgdovl, cgbovl, i1, i3, i4, gbd, gbs, vgsteff, qinv, igd, igs, igb, igcs, igcd, igidl, igisl, qgi, qsi, qdi, qbi, cddbi, cssbi, cggbi, cgsbi, cgdbi, cbdbi, cbsbi, qsrco, ide, ige, ise, ibe, idb, isb, vsb, gmb, vgt, vdss, vsat_marg, self_gain, rout, beff, fug, vearly, ft, vth_drive, vdsat_marg, tk, ib, ig.*

5.1 AC Analysis: DC Operating Point

(5.1.1) We will, again, plot the curve of g_m vs. V_{GS}. But, this time we will use an AC analysis to do so. Note that in this case, g_m will be calculated internally by the simulator, rather than by us explicitly as we did in part 1 (using the calculator). The point of this exercise is to demonstrate the underlying principle of the AC analysis (and its relationship to the DC operating point analysis).

(5.1.2) Recall, in the simplest small-signal model of the MOSFET, the drain current is given by,

$$i_{ds} = g_m v_{gs}$$

where the lowercase v_{gs}=1V is the AC input signal. If we sweep the DC bias V_{GS} (note the uppercase) using an AC analysis with the AC input signal set to 1V, the resulting plot of i_{ds} will also be a plot of g_m. Hence, by sweeping the DC bias point V_{GS} we will implicitly be plotting the curve of g_m vs. V_{GS} just as we did with the DC analysis.

(5.1.3) Note. We are using the same schematic and simulations as we used in part 1. If you don't have your schematic and ADE already open from the previous exercises. You will need to set up the schematic/ADE and ensure you have copied the design variables into the ADE and initialised them (see Figure 4). Also, make sure you have set up you DC sweep analysis with v_{gs} as your sweep variable (0-2.5, 1001, linear).

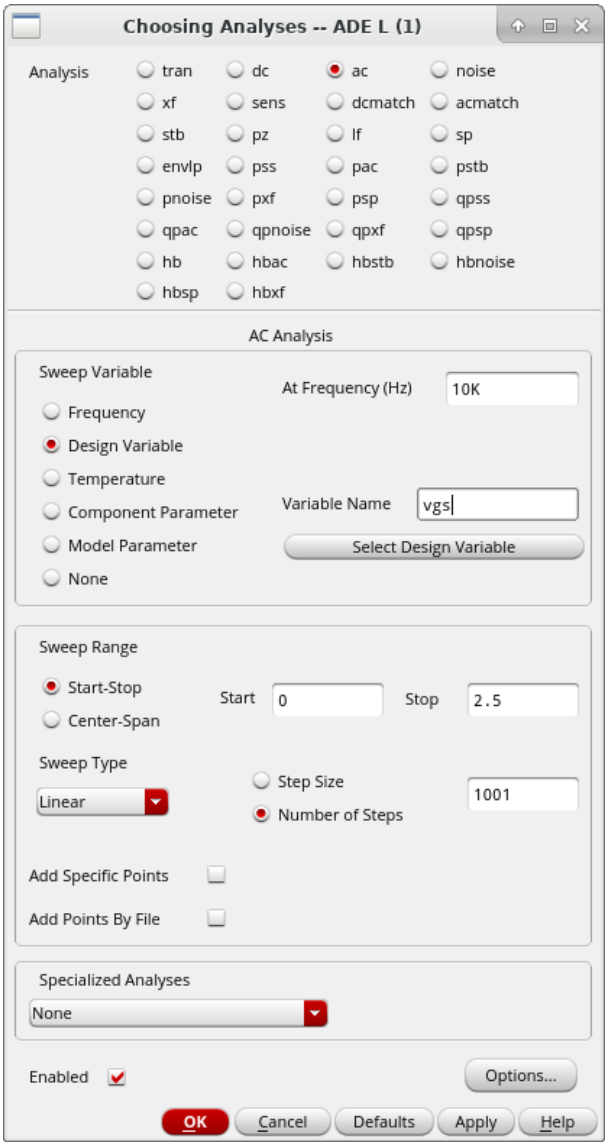
(5.1.4) In the schematic editor window, modify the gate-to-source DC voltage source (**vdc**) of the NMOS FET transconductance device to include an AC stimulus. (**Menu: Edit à Properties à Objects**, or **Edit Properties** icon-button, or press '**Q**'.) In the edit object properties form, set the 'AC magnitude' parameter to 1. Leave all other attributes unchanged. OK the form.



< Figure 70. Adding a 1V AC Stimulus to the DC Gate-to-Source Voltage >

(5.1.5) From the ADE window, setup the AC sweep analysis for I_{DS} vs. V_{GS} (**Menu: Analyses à Choose**). We wish to sweep the input voltage (V_{GS}) of the NMOS device from 0V to 2.5V while observing the drain current (I_{DS}). We will use 1001 steps in the sweep to give the output waveform sufficient resolution. We will not be sweeping the frequency since (for this exercise) we are only interested in the DC operating point behaviour of the AC analysis, so we will fix the frequency at 10KHz – at this relatively low frequency we can ignore the effects of the parasitic capacitances.

In the choosing analyses form, select the '**ac**' option button. Select the 'Design Variable' option as the sweep variable. Type *vgs* as the variable name. Set the 'At Frequency (Hz)' to 10K. Choose the 'Start-Stop' option button with Start = 0 and Stop = 2.5 for the sweep range. Select 'Linear' from the drop down menu as the sweep type. Choose the 'Number of Steps' option with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. See Figure 14. OK the form. The **ac** analysis should appear in the 'Analyses' list of the ADE window (next to the **dc** analysis we set up previously).



< Figure 71. The Choosing Analyses Form for a AC sweep. >

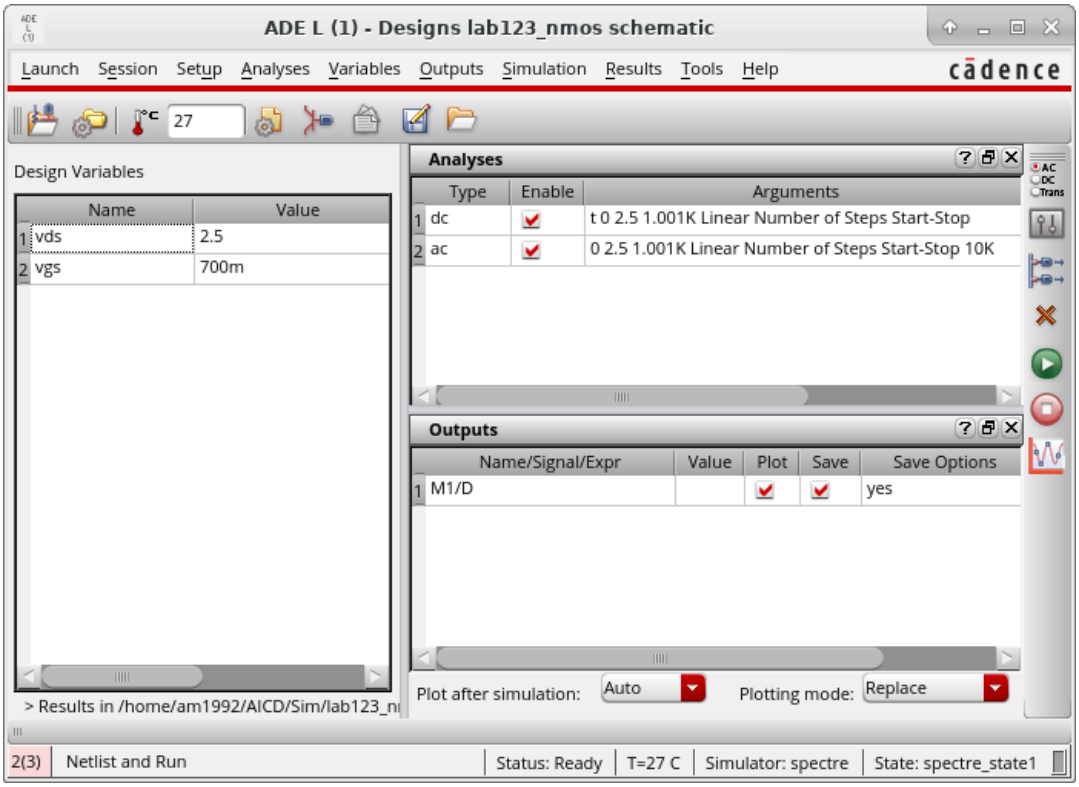
(5.1.6) If not already set up (from the last part), select the drain node current (I_{DS}) for plotting. From the ADE, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** Select only the drain node of the NMOS device (M1/D). **Press the 'ESC' key to exit the plotting mode of the ADE**.

(5.1.8) In the ADE window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error and both the text output window as well as the waveform output window should open automatically. You will see two plots, in separate sub-windows. One will be a plot of I_{DS} vs. V_{GS} for the DC analysis, and one will be a plot of I_{DS} vs. V_{GS} for the AC analysis. See Figure 16.

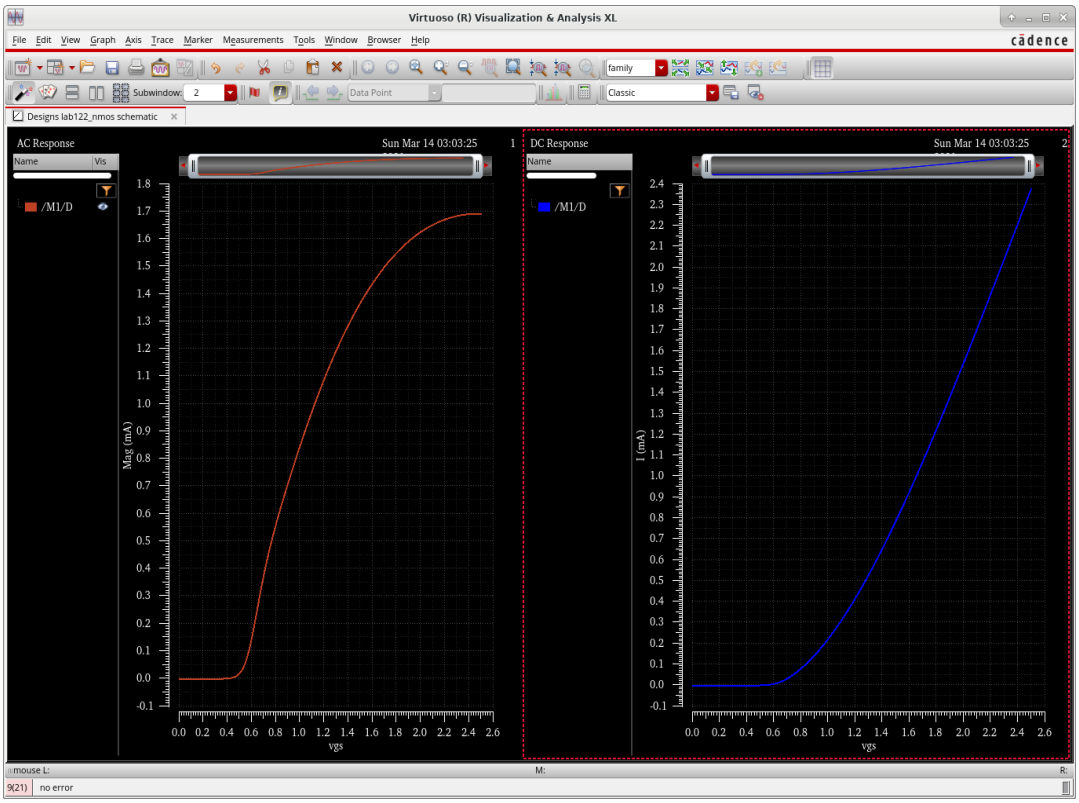
(5.1.9) Click on the DC analysis plot sub-window (note the dashed red line around the selected sub-window). Plot the curve of dI_{DS}/dV_{GS} vs. V_{GS} using the calculator just as you did previously i.e., deriv(IS("/M1/D")). Make sure the plotting mode of the calculator is set to 'append'. When you evaluate the buffer, the curve of dI_{DS}/dV_{GS} vs. V_{GS} will be added to the DC analysis plot sub window. See Figure 18.

(5.1.10) Note, the plot of dI_{DS}/dV_{GS} vs. V_{GS} in the DC analysis plot sub-window is the same as the g_m vs. V_{GS} curve in the AC analysis plot sub-window. Select the plot from the DC analysis plot sub-window and drag it over to the AC analysis plot sub-window. They will overlay *almost* exactly (the difference is miniscule). Whereas the AC analysis is implicitly calculating the g_m operating point *at* each sample of V_{GS}, the calculator derived plot from the DC analysis is calculating the differential *between* samples. So there may be a very small difference. (In fact if you use more steps per sample you see the difference between the AC and DC derived plots of gm disappear.

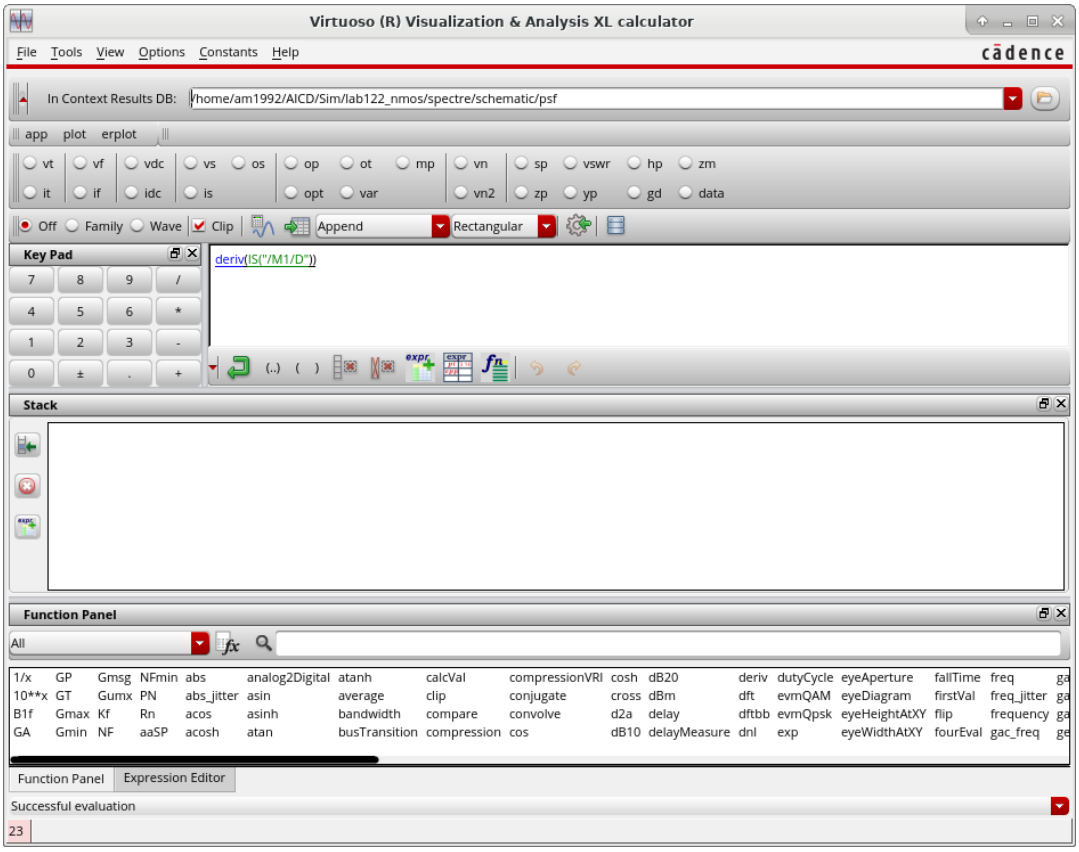
(5.1.11) In the final part of this lab, we will carry out a frequency sweep of the basic single-stage resistor-loaded NMOS FET amplifier. A frequency sweep calculates the output response to a steady-state sinusoidal input excitation over a range of frequencies.



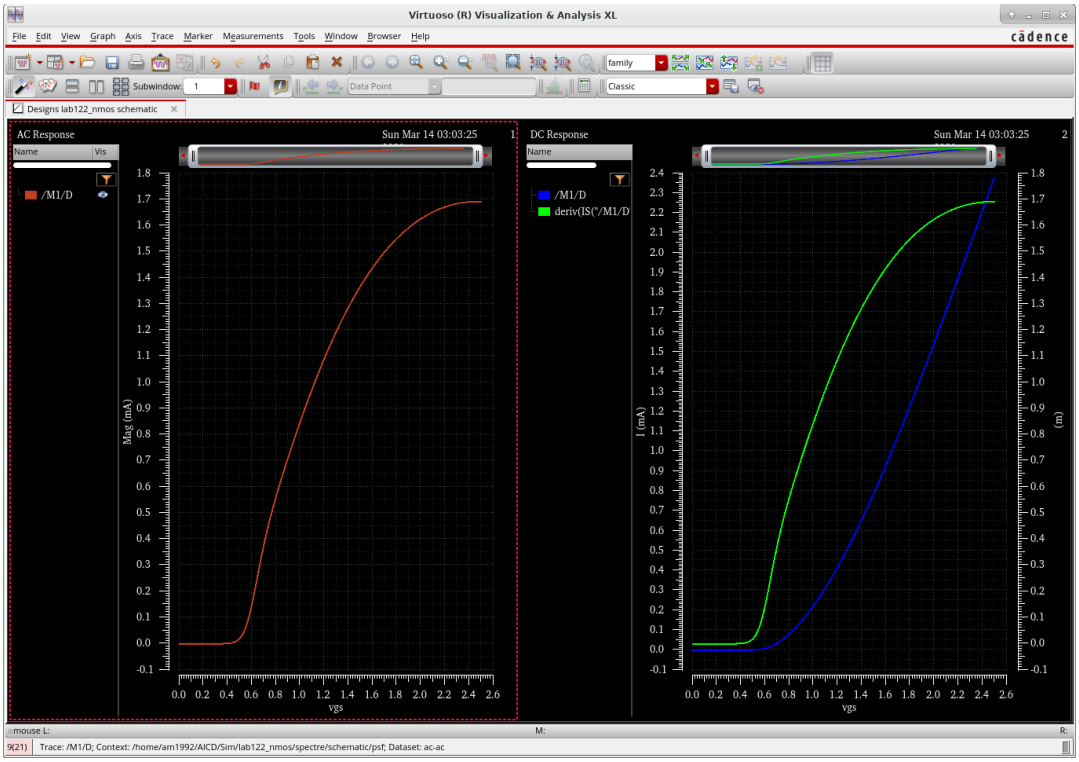
< Figure 72. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 73. AC and DC Sweep of I_{DS} vs. V_{GS}. >



< Figure 74. The calculator set up to plot dI_{DS}/dV_{GS}. >



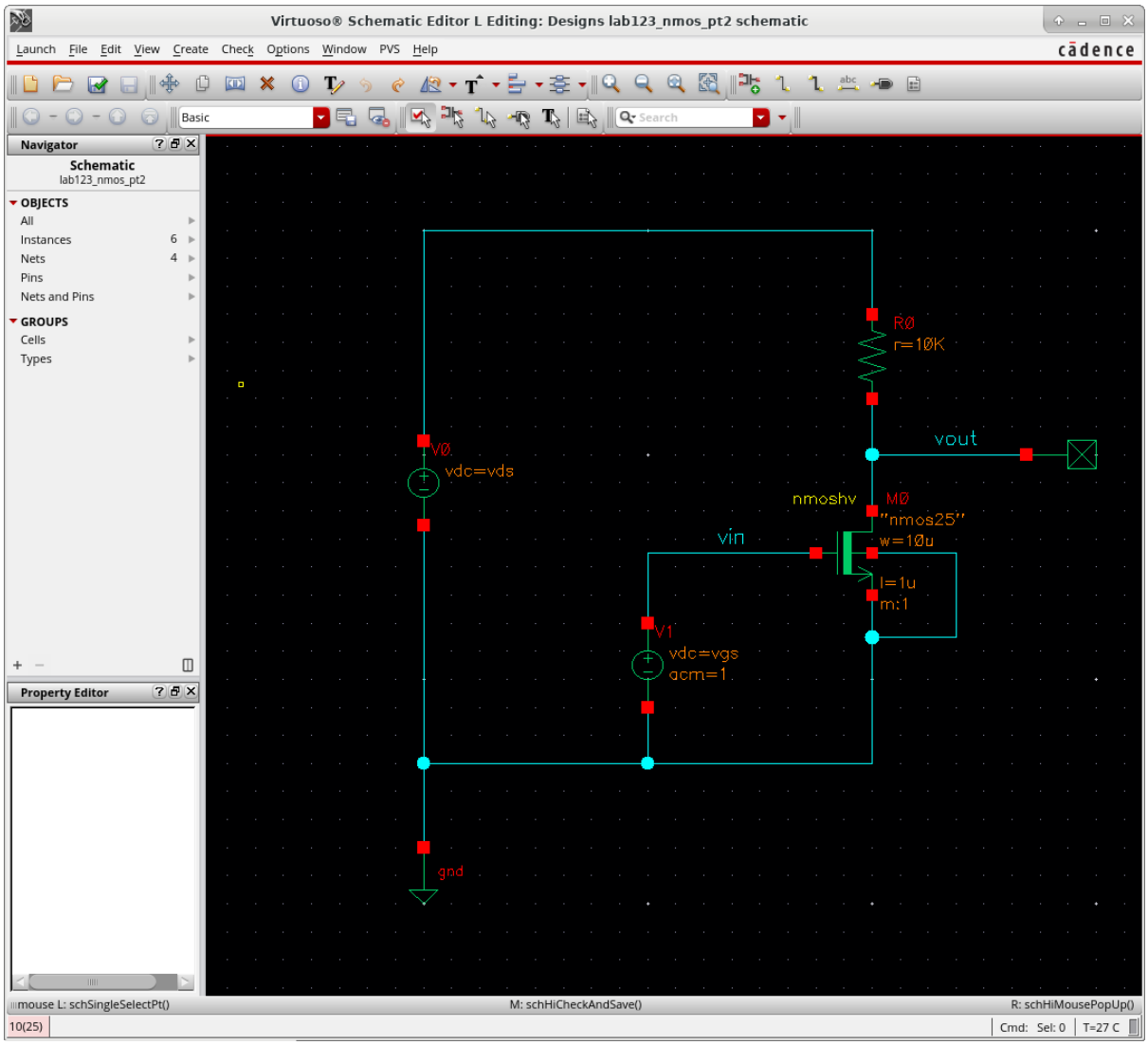
< Figure 75. AC and DC Sweep of I_{DS} vs. V_{GS}, and dI_{DS}/dV_{GS} vs. V_{GS} >

5.2 AC Analysis: Frequency Response

(5.2.1) Create a schematic of a resistor-loaded single-stage NMOS FET. See Figure 19.

(5.2.2) You can reuse your circuit from last lab (lab122_nmos_pt2 or whatever name you chose), you can copy the circuit from last lab, or you can create a new schematic from scratch. If you make a copy/new schematic, give it a meaningful name e.g., lab123_nmos_pt2.

Ensure that your schematic is properly parameterised using design variables: for the DC supply voltage vdc=vds and for the DC gate-to-source voltage vdc=vgs. The gate-to-source input voltage *vin* should have a 1V AC stimulus (AC magnitude = 1). The NMOS device should be W/L = 10u/1u. The resistor load should be 10KΩ. Name the output net *vout*. Use the noConn part to avoid floating net warnings. Ensure that it checks and saves without warnings/errors. Use the CIW to diagnose any warning or errors.

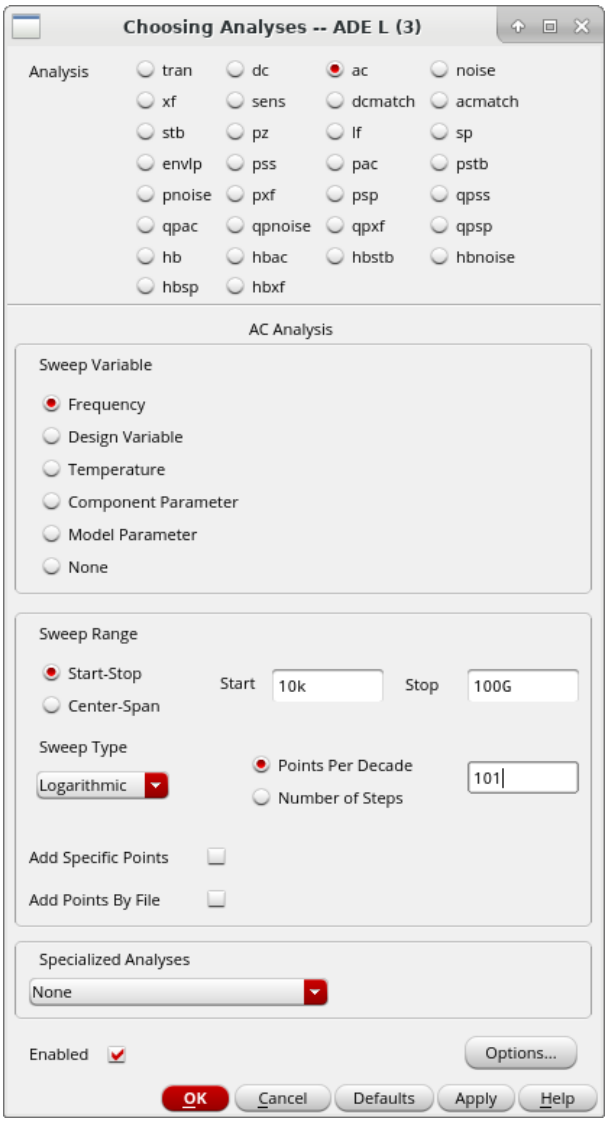


< Figure 76. Basic Resistor-Loaded Amplifier including a 1V AC Stimulus for AC Analysis>

(5.2.3) From the schematic window, launch the ADE tool (**Menu: Launch à ADE L**). From the ADE window, copy the design variables from the schematic into the ADE (**Menu: Variables à Copy From Cellview**). Initialise *vds = 2.5* and *vgs = 0.7*.

(5.2.4) From the ADE window, setup the AC sweep analysis (**Menu: Analyses à Choose**). In this simulation we will sweep the frequency from 10KHz to 100GHz. We will plot the output *vout* vs. frequency. Because of the large frequency range, we will use a logarithmic sweep of the frequency – with 101 plot points per decade of frequency.

In the choosing analyses form, select the '**ac**' option button. Select the 'Frequency' option as the sweep variable. Choose the 'Start-Stop' option button with Start = 10K and Stop = 100G for the sweep range. Select 'Logarithmic' from the drop down menu as the sweep type. Choose the 'Points Per Decade' option button with 101 plot points per decade. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. See Figure 20. OK the form. The **ac** analysis should appear in the 'Analyses' list of the ADE window (next to the **dc** analysis we set up previously).



< Figure 77. Choosing Analyses Form set up for a Frequency Sweep AC Analysis >

(5.2.5) Select the output net voltage (vout) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** The schematic window will automatically become current. Select only the output net *vout*. **Press the 'ESC' key to exit the plotting mode of the ADE**.

(5.2.6) In the ADE window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error and both the text output window as well as the waveform output window should open automatically. See Figure 22.

(5.2.7) Note that because we set *vin* = 1V, the plot of *vout* is the plot of the circuit voltage gain A_V. That is,

$$v_{out} = A_V v_{in} = A_V$$

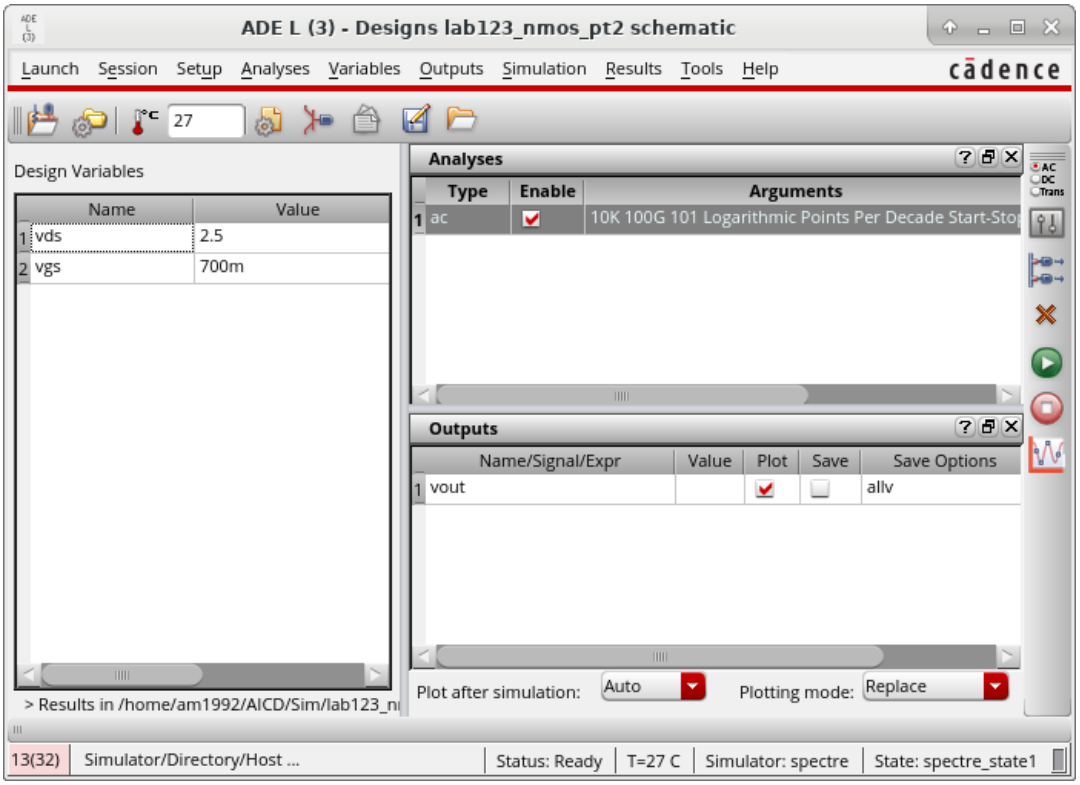
(5.2.8) The amplitude of the response by default is plotted on the linear/numerical scale – aka a **magnitude plot**. It is common practice to plot the output response of an amplifier in decibels (dB) to give a **log-log Bode** plot. (We use the dB scale as it makes it easier to graphically add responses, we can do this using the calculator.) From the ADE window, choose **Menu: Tools à Calculator…**

(5.2.9) Use the schematic Selection Toolbar to get the plot of *vout* into the calculator. Select the '**vf**' option button (for frequency sweep) on the schematic selection toolbar. When you select the '**vf**' button the schematic editor window will come into focus. Select only the output net *vout*. The calculator should return to focus, and you should see the expression for the drain current VF("/vout") in the buffer area of the calculator. (You may need to delete any expressions in the buffer from the previous exercises.)

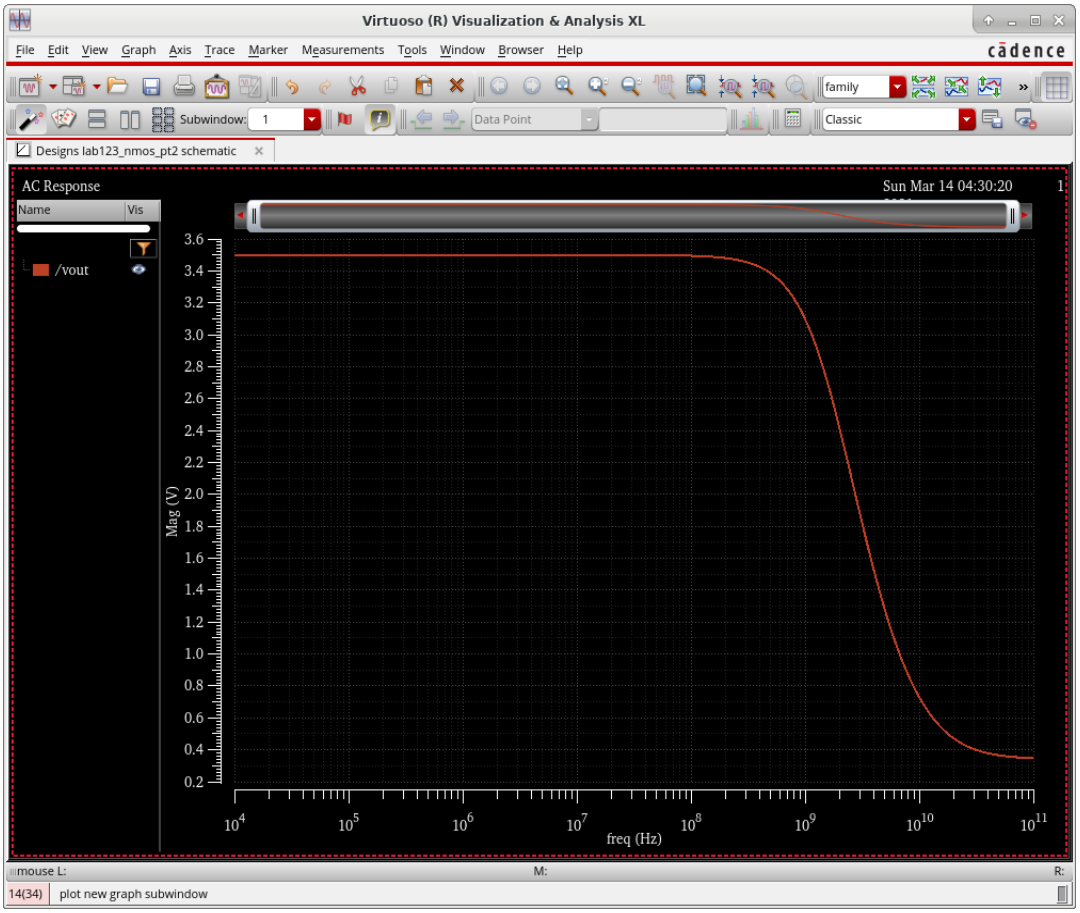
(5.2.10) With the expression for *vout* in the buffer, apply the decibel function from the function panel. In the function panel of the calculator window, select '**All**' from the drop down menu. Place the cursor in the correct positon in the buffer and select the '**dB20**' function from the list of functions. The functions are listed alphabetically so either search the list or use the text search box just above the listing area. Make sure to close the brackets after you select the **dB20** function. The expression in the buffer should be dB20(VF("/vout")). See Figure 24.

Note that there are two dB functions: dB10 and dB20. When converting a power gain *G* to decibels we use dB10 (that is $dB_{10} = 10 \log G$) and for a current/voltage gain *A* we use dB20 (that is $dB_{20} = 20 \log A$).

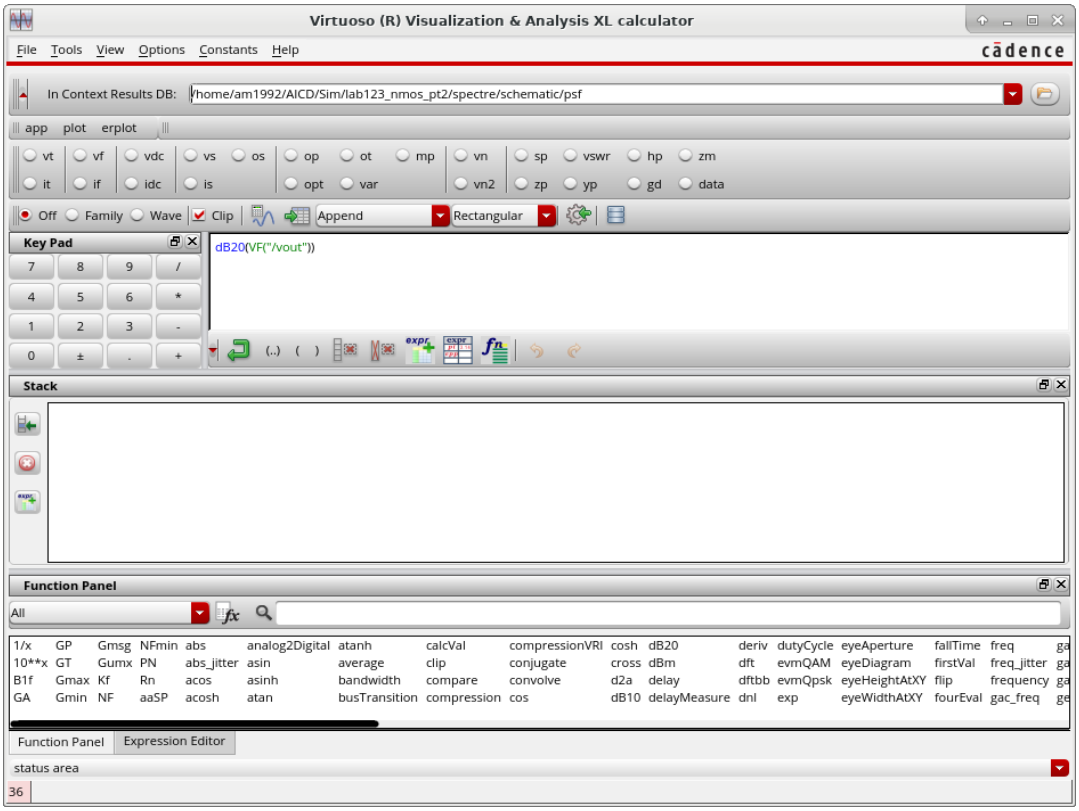
(5.2.11) In the calculator window, evaluate the buffer expression by using the **Evaluate the Buffer** button. If you set up the expression correctly and the simulation data exists (you should have run the AC sweep simulation before opening the calculator) the plot of $20 \log vout$ should be plotted alongside the original plot of vout.



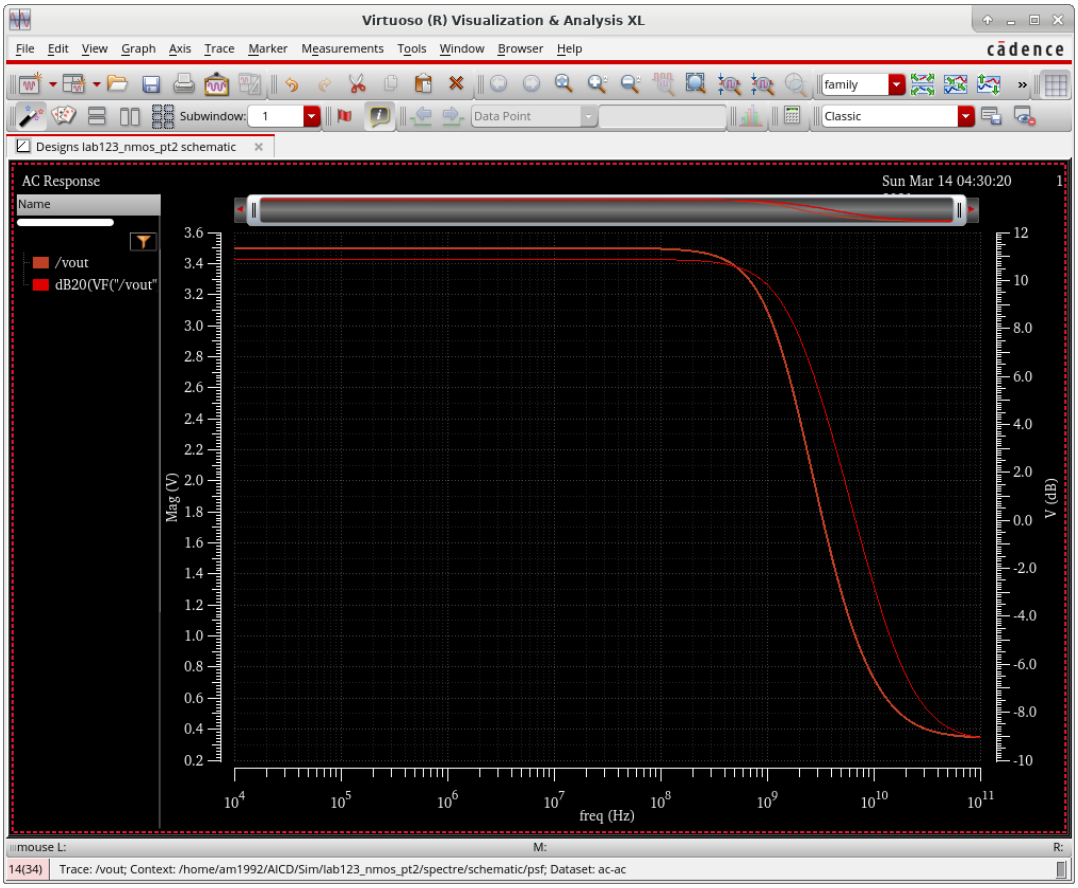
< Figure 78. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 79. Frequency sweep response of the Resistor-Loaded NMOS FET Amplifier >



< Figure 80. Calculator setup to plot vout in dB (20 log vout) >



< Figure 81. Frequency response with Magnitude and dB Scales >

(5.2.12) We will discuss the frequency response of the MOSFET in more detail in the coming labs. Here, we will point out that the frequency sweep response of Figures 22 and 24 are generated from the linearised model of the MOSFET. We note that the gain of the MOSFET falls by 3dB between 1 to 2GHz. We can therefore assume that at least one pole exists around this point – where $\omega_{-3dB} = 1/RC$ – caused by the internal parasitic capacitances/resistances within the MOSFET. And at least one zero exists beyond 10GHz responsible for the flattening out of the response.

We will investigate the role of poles and zeros in more detail – particularly in the context of negative feedback – in the coming labs.

End of Lab!

Tutorial Lab 12.4 (Formatively Assessed):

Amplifier Use, Feedback, and Stability Analysis

Preliminary Matter: Setting up the Cadence Environment

Please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout before entering these commands into your terminal.

For this course, you will use the Cadence Virtuoso Design Environment (referred to simply as Cadence). Cadence is a Linux based software suite for designing full custom integrated circuits. In order to use Cadence you will first need to set-up the Cadence environment.

11. Open a terminal window and ensure that you are in your home directory using the following command:

```
cd ~
```

Note: 'cd' stands for change directory. The solitary tilde symbol (~) is a shortcut/alias for your home directory (/home/username).

12. Under your home directory, use the following commands to load the Cadence environment:

```
module use /eda/cadence/modules
module load course/EENGM6030
```

Note: The two 'module' commands load the required shell/Cadence environmental variables for this course (paths, names, aliases…) In particular, this includes a reference to the process design kit (PDK) used in this course, the GPDK180. You will need to do this every time you open a terminal, since the sourced environment only applies to the current terminal (until it's closed).

a. ***Note: This step only needs to be completed ONCE - If you already ran this script in lab 12.1, you don't need to run it again here.*** The first time you use Cadence, you will need to run the following command (script):

```
EENGM6030_setup
```

This will create a folder called AICD (short for Analogue Integrated Circuit Design) under your home directory. The AICD folder contains the files and directories needed to run Cadence. Running this setup script more than once will result in a warning message that the AICD folder already exists. It will then exit without making changes.

13. The AICD folder will be used for all your Cadence related activities for this course. **You should always make sure you are in your AICD folder before you launch Cadence.** To run cadence, use the following commands:

```
cd ~/AICD
virtuoso &
```

Note: The 'virtuoso &' command starts Cadence. The ampersand symbol (&) causes the Cadence process to run in the background freeing up the terminal for other commands the '&' symbol is a feature of the terminal's shell (bash) not Cadence.

1. Learning Objectives of this Lab:

From the previous lab, you should be able to:

- Obtain the transconductance g_m from both the large-signal characteristics/DC-sweep and the small-signal AC analysis. (Plot the dI_{DS}/dV_{GS} vs. V_{GS}, dV_{DS}/dI_{DS} vs. V_{DS}, and g_m vs. V_{GS}, r_{DS} vs. V_{DS}, and g_m/I_{DS} vs. V_{GS} using DC and/or AC analysis.)

- Understand the need to choose an appropriate DC operating point. Appreciate the trade-off between performance and power consumption in choosing an operating point. Understand the underlying relationships between DC operating point analysis, large-signal and small-signal analyses, DC and AC analyses.

- Use the Cadence in-built calculator to post-process simulation data. Import simulation data from the schematic into the expression buffer using the schematic selection tool. Manipulate the expression, apply functions from the function panel to the expression, and evaluate the buffer. Send the expression from the calculator to the ADE. Save ADE and Calculator state.

- Carry out a basic frequency-sweep simulation using an AC analysis. Understand the use of linear vs. logarithmic frequency sweep, and the dB power scale to plot gain. Use the calculator to convert and plot the linear magnitude response on the dB power scale à $dB20(A_V) = 20 \log A_V$, where $A_V = V_{OUT}/V_{IN}$.

In this lab, you will:

- Understand the importance of feedback in (analogue) electronic circuits, the potential for instability within such circuits, and the loop gain magnitude and loop gain phase as the criteria/metrics for stability.

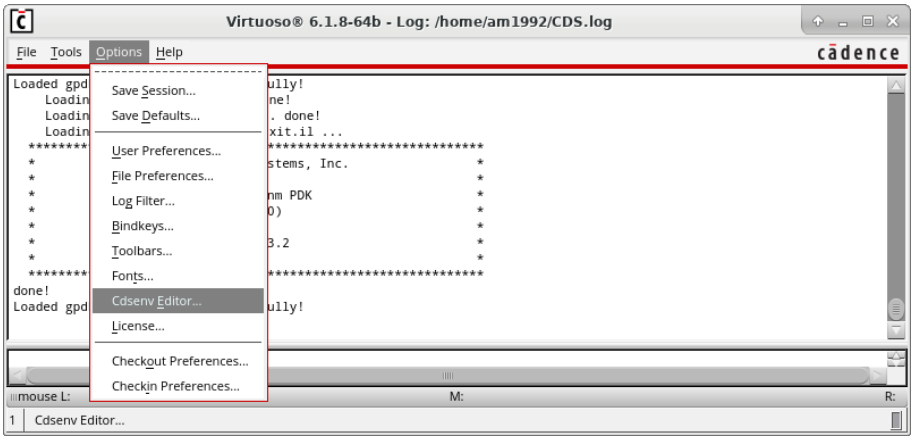
- Design and investigate a simple ideal feedback system. Investigate the SPICE/Spectre option for the analysis of stability in feedback circuits. Investigate the stability of a voltage reference circuit with modelled op-amp drive.

2. Cadence Documentation, Help, and Licensing

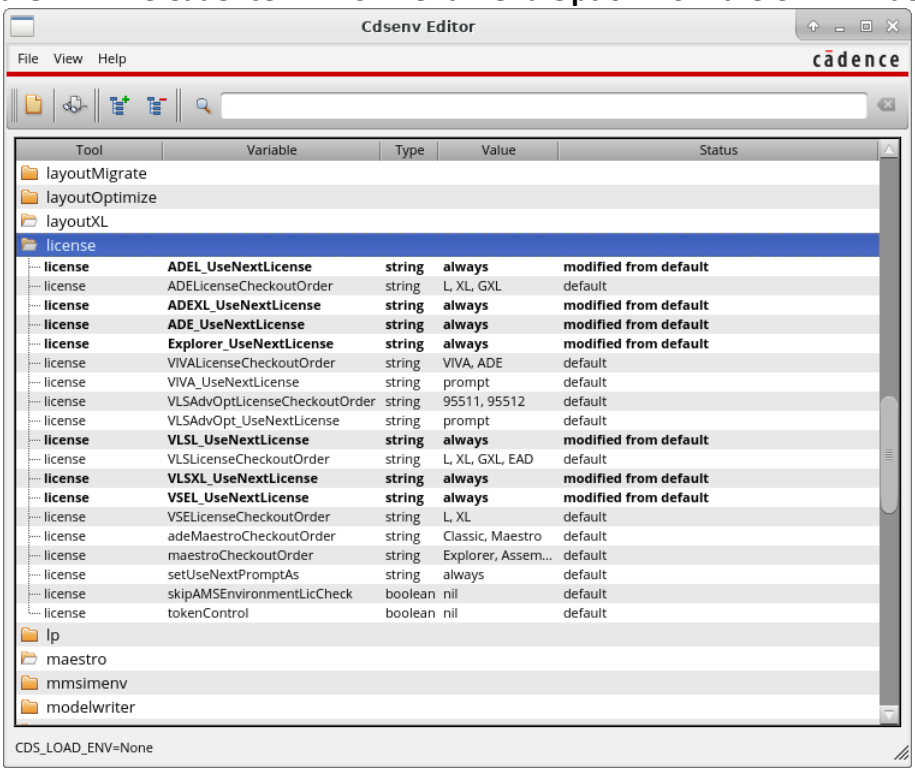
Local and online help is available from the help menus (**Menu: Help → User Guide** or by pressing **F1**), or by typing 'cdnshelp &' at the command prompt (assuming you have setup your terminal environment).

PLEASE NOTE: WHEN LAUNCHING A TOOL IN THE CADENCE DESIGN ENVIRONMENT YOU MAY SOMETIMES GET A WARNING ABOUT A LICENSE BEING UNAVAILABLE FOR THE TOOL, AND REQUESTING PERMISSION TO OBTAIN A HIGHER-TIERED LICENSE. YOU SHOULD SELECT 'ALWAYS' TO THESE MESSAGES.

If you accidently deny a license request, or you cannot access a license, you will need to modify the Cadence environment licence/permissions from the CIW. From the CIW menu go to **Menu: Options → Cdsenv Editor**. Once in the environment editor, go to the license folder, and make sure all licenses (in bold) are set to '**always'** – if any entries are listed as '**never**' you will need to manually change/retype them to '**always**'.



< Figure X.1. The Cadence Environment Menu Option from the CIW Window. >



< Figure X.2. The Cadence Environment Editor. Note, all licenses should be set to 'always'.>

3. Getting Started with Cadence:

For a refresher/introduction on how to start a Cadence session please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout.

3.1 Dealing with Lock files

PLEASE NOTE: IF A SCHEMATIC ISN'T PROPERLY CLOSED DOWN (FOR EXAMPLE, DUE TO A DROPPED REMOTE CONNECTION), THE SCHEMATIC MIGHT BECOME EDIT LOCKED. IF THIS HAPPENS YOU WILL NOT BE ABLE TO OPEN THE FILE FOR EDITING. YOU WILL ONLY BE ABLE TO OPEN THE FILE FOR READING UNTIL YOU REMOVE ANY/ALL LOCK FILES.

There are two ways you can unlock files:

(5) The first is to manually delete any edit lock files. To do this, go to your '~/AICD/Designs' folder and find the folder representing the schematic e.g., for lab1 this would be '~/AICD/Designs/lab121_vdivider/schematic' if this is what you called your schematic. Delete all the .cdslck lock files in this folder and you should be able to open your schematic for editing.

(6) You can also automatically release edit locks with the command:

```
clsAdminTool -are .
```

To use this command, you have to be in your AICD folder (the full-stop is shorthand for the current directory path). If you are not in your AICD folder, you have to specify the path (relative or absolute) to your AICD directory.

Since this is a Cadence command, you have to ensure that you have set up the terminal environment for Cadence (in particular, the modules must be loaded as in step 2, 'Preliminary Matter'). It is best to use the terminal window that you used to start Cadence, since the environment was already set up. Note that you will not be able enter any commands into your terminal if you did not start cadence with the '&' character. By example, to release any lock files after opening a new terminal, the full command set would be,

```
cd~
module use /eda/cadence/modules
module load course/EENGM6030_2021
clsAdminTool -are AICD
```

For more information on this command, use
`clsAdminTool -help`

4. Circuit Simulation Exercises: Part 1

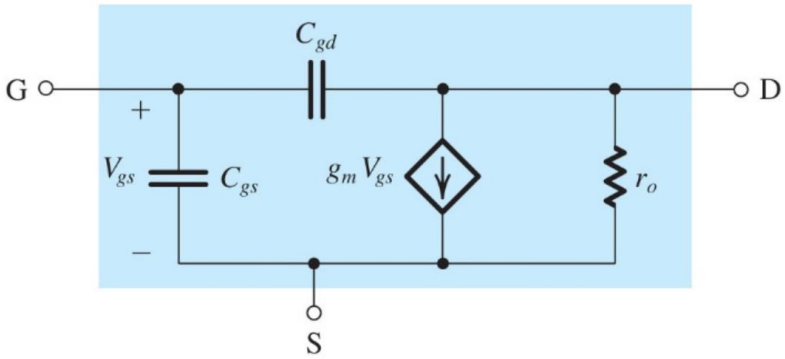
With guidance, you will:

- Understand the importance of feedback in electronic circuits for control, and the potential for instability within such circuits.
- Design and investigate a simple ideal feedback system.
- Investigate the SPICE/Spectre option for the analysis of stability in feedback circuits.

As we discussed in the previous lab, the AC analysis simulates an idealised linear model of the circuit under test – the so called small-signal model. The small-signal model of the transistor is built from ideal components – resistors, capacitors, and voltage-controlled current sources (VCCS) – representing the ideal as well as non-ideal behaviours of the circuit. The values of these ideal components are determined from a DC operating point analysis of the large-signal, non-linear circuit model.

At the end of the last lab, we carried out a frequency sweep AC analysis where we observed the frequency response of the basic resistor-loaded amplifier. We observed from the bode plot (magnitude/dB frequency response plot) that the circuit had at least one pole (at around 10^9 Hz) and one zero (at around 10^{11} Hz) caused by the parasitic reactances and resistances inside the model of the NMOS device. The most significant capacitances in the circuit are the gate-to-source c_{gs}, gate-to- drain c_{gd}, and drain-to-source c_{ds} capacitances. Due to the large-voltage variation across the gate-to-drain capacitance, it is this capacitance that dominates the frequency response and gives us the dominant pole responsible for the 3dB roll-off near 10^9 Hz. This same capacitance also gives us the zero near 10^{11} Hz (as the frequency increases the signal passes straight from input-to-output via this parasitic route through the transistor).

Note the small signal model from last week, with parasitic capacitances added.



< Figure 82. >

Fundamentally, the stability of a circuit is determined by the location of the poles and zeros of the circuit in the frequency domain. In this lab, we will see how the poles and zeros arise from the RC products in the circuit in both the forward and feedback paths of a simple

feedback system (which models an amplifier). We will explore how the poles and zeros affect the frequency response of the circuit – the input-output frequency response, as well as the frequency response of the feedback loop – and crucially what this means for the stability of the circuit. We will introduce loop gain magnitude and loop gain phase as the criteria and metric for amplifier stability.

4.1 Feedback and Stability

In the analogue IC designer's toolbox, feedback is the most useful tool. We use feedback to control our circuits – to set and maintain the bias/operating points, to compensate for non-linear and non-ideal behaviour (linearisation, compensation), to improve process-voltage-temperature (PVT) robustness, to reduce parameter sensitivity between devices and circuits, to increase or decrease input and output impedances, to compare/track/regulate/mirror currents and voltages, to shape the frequency response, and more.

But feedback is problematic. As soon as you introduce feedback into a circuit you introduce the potential for instability. When utilising feedback, we almost always use **negative feedback** – where the feedback signal is subtracted from the input signal and the difference (the error signal) is used to drive the system. When a system feeds the output back to the input such that it adds to the input, you have **positive feedback**. The output, in this case, will grow unbounded until it saturates (reaches some system limit). Consider the **Barkhausen criteria** for instability (aka Barkhausen stability criterion, Barkhausen criterion for sustained oscillation). A system that satisfies the Barkhausen criteria will oscillate – in fact, the Barkhausen criteria is the defining criteria in oscillator design. Beyond the Barkhausen criterion the system is unstable – it will not stabilise or settle at an operating point.

We must therefore ensure when using feedback in a system, that it remains stable over a very wide frequency range. If the circuit is unstable at any point in the frequency domain, it is considered unstable overall. Even where instability occurs outside of a system's intended frequency range of operation, disturbances, noise, or other unintended/unforeseen inputs outside of the specification may drive the system into instability.

To determine the stability of a circuit through simulation requires, a **stability analysis**, which is separate and distinct from the AC and DC analyses. You cannot determine whether a circuit is stable through AC or DC analysis alone. The DC analysis does not simulate time-domain or frequency-domain related behaviour such as oscillations. The AC analysis considers only the frequency domain steady-state behaviour. It is possible to spot instability in a transient analysis in the form of oscillations and exponentially increasing responses, but this does not help in characterising where the instability lies. The only meaningful way to determine if a circuit is stable is through a stability analysis. In Cadence, this is the '**stb**' analysis type. Note that the stability analysis is a small-signal analysis – it simulates an idealised linear circuit for a given DC operating/bias point. Using the stb analysis, determination of stability is made from the observation of the magnitude and phase response of the feedback loop. Specifically, we look for the point at which the feedback transitions from negative to positive.

For a fuller explanation of feedback theory refer section 5.1 of the background material and reference [3] – Appendix E (Sedra/Smith).

Operational Amplifier (Op-Amp) Circuits

Op-amp circuits are the principal example of a feedback system. The general structure of the op-amp is a **differential input stage** followed by a **high gain stage**. (Usually, the gain stage has a high output-impedance, and may be followed by a low-impedance unity-gain buffer stage.) The input stage has an inverting and non-inverting terminal – feedback is applied by taking the output (or some fraction/function of the output) and feeding it back to the inverting input. The feedback is then a comparison between the feedback signal and the input signal. Mathematically, we have a positively sensed input and a negatively sensed feedback signal. Note how this correlates to the basic model of the feedback system.

The op-amp demonstrates a core design philosophy in analogue IC design – in many cases we prefer not to create amplifiers with absolute parameter values, where due to PVT, there can be large tolerances in the values. Instead, we use a high-gain amplifier with feedback. The operation (linear gain, filtering, differentiation, integration, summing, etc.) is then set by the arrangement/configuration of external passive devices (external to the amplifier not to the IC) – namely resistors and capacitors. In this case, precision of the circuit is often determined by the ratios (and therefore the matching) of the external passive devices than by the absolute values of the passive devices. The matching of two or more devices in an IC can be done with far more accuracy compared to the absolute values. For example, consider the closed-loop gain of the inverting and non-inverting operational amplifiers. With a sufficiently high op-amp gain,

$$A_{V(-)} = -\frac{R_F}{R_{IN}} \qquad\qquad A_{V(+)} = 1 + \frac{R_F}{R_{IN}}$$

where R_F and R_{IN} are the feedback and input resistors. It is the ratio of these two devices that set the gain and not their absolute values. Moreover, when subjected to variations in process, voltage, and/or temperature as long as the ratio is maintained (using proper layout techniques), the gain accuracy is maintained. The combination of high gain and feedback can overcome many non-idealities in the circuit.

Dependent/Controlled Sources

There are four general amplifier topologies – defined by their input-output quantities: voltage amplifiers (voltage-in, voltage-out, gain=V_{OUT}/V_{IN}), transconductance amplifiers (voltage-in, current-out, gain=I_{OUT}/V_{IN}), transimpedance amplifiers (current-in, voltage-out, gain=I_{OUT}/V_{IN}) and current amplifiers (current-in, current-out, gain=I_{OUT}/I_{IN}).

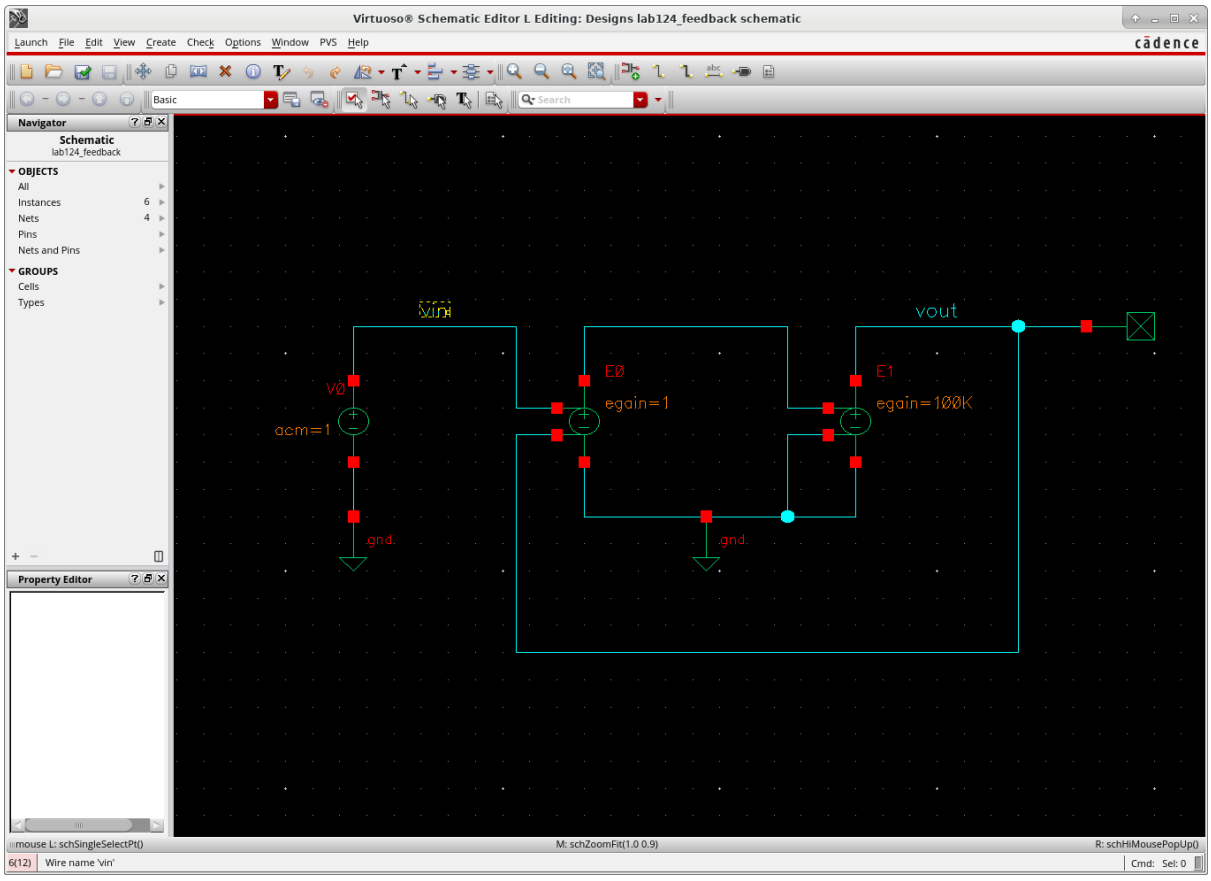
Each of the four amplifier types can be modelled by a **controlled source** from the analogLib library. Each of these controlled sources are ideal and have a linear scalar input-to-output relationship: voltage-controlled voltage-source (VCVS, voltage-gain=V_{OUT}/V_{IN}), voltage-controlled current-source (VCCS, transconductance=I_{OUT}/V_{IN}), current-controlled voltage source (CCVS, transresistance=V_{OUT}/I_{IN}), and current-controlled current source (CCCS, current-gain=I_{OUT}/I_{IN}). In this lab, we will use the VCVS to design and explore a simple feedback system. Note that we can use the VCVS to represent the Thevenin equivalent circuit of an amplifier, while the VCCS can be used to represent the Norton equivalent circuit of an amplifier.

4.2 A Simple Ideal Feedback System

(4.2.1) Create a new schematic. From the Library Manager, **Menu: File à New à Cell View**. Ensure you are in your Designs library. Use the New File form as you have done previously, giving your new schematic a meaningful name such as 'lab124_feedback'.

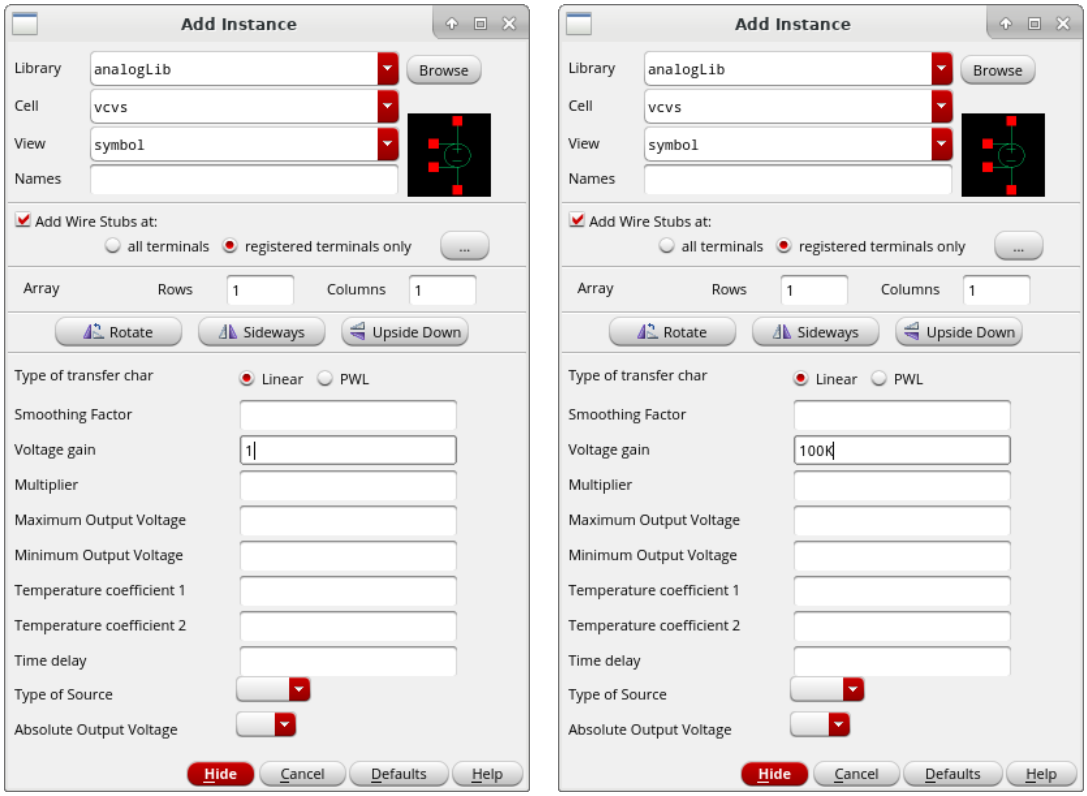
(4.2.2) In the schematic editor window, create the unity feedback system as given in Figure 2 as described below.

(4.2.3) From the analogLib library, instantiate two voltage-controlled voltage sources (**vcvs),** a DC voltage source (**vdc**), and a ground reference (**gnd**). The first VCVS in the feedback system will be used as the differential input stage with unity gain, set the voltage gain parameter to 1 (**Voltage gain** = 1). The second VCVS will be used as the high gain stage, set the voltage gain to 100K (**Voltage gain** = 100K). We will use the vdc component to provide the input ac stimulus, set the ac magnitude to 1 (**AC magnitude** = 1). Leave all other parameters/attributes empty. See Figures 2, 3 and 4.

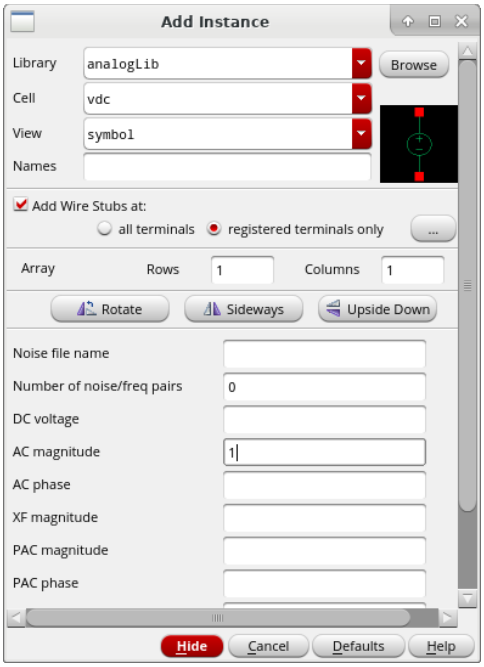


< Figure 83. Simple Feedback System. Ideal op-amp model using two VCVS components from the analogLib library. Unity gain configuration. >

(4.2.4) Wire the components as in Figure 2 using the **Wire (narrow)** command. Connect the input voltage source to the positive terminal of the differential input stage (+vcvs) and feedback the output to the negative terminal of the input stage (-vcvs). Name the output net *vout* and the input net *vin* using the **Create Wire Name** command. Use the **noConn** connector from the **basic** library to avoid floating-net warnings. Keep the schematic neat.



< Figure 84. Add Instance forms. VCVS for the input stage and gain stage, respectively.>

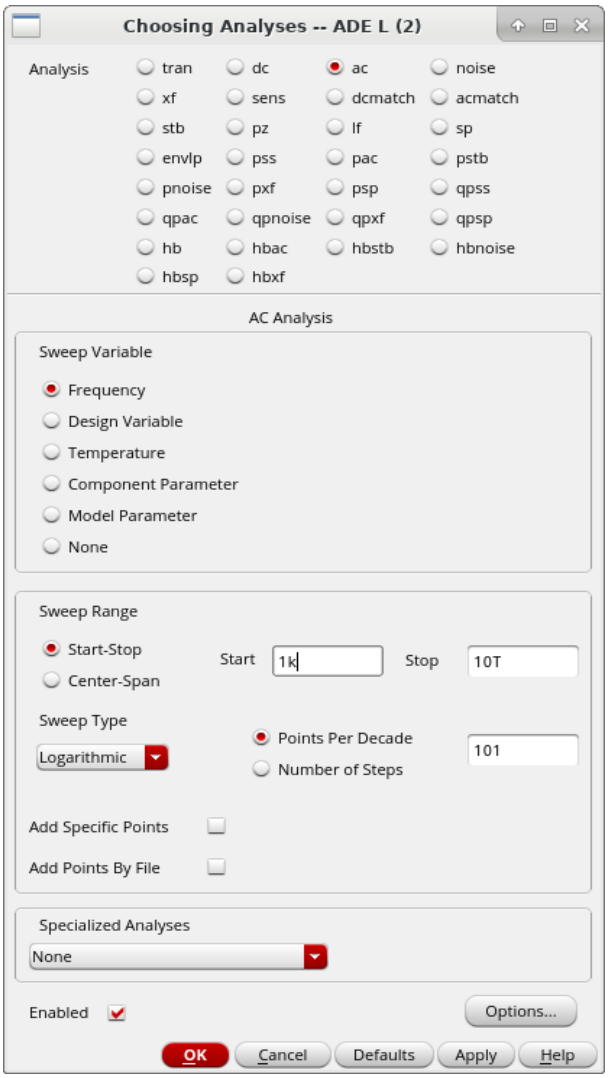


< Figure 85. Add Instance forms VDC for the AC input stimulus. >

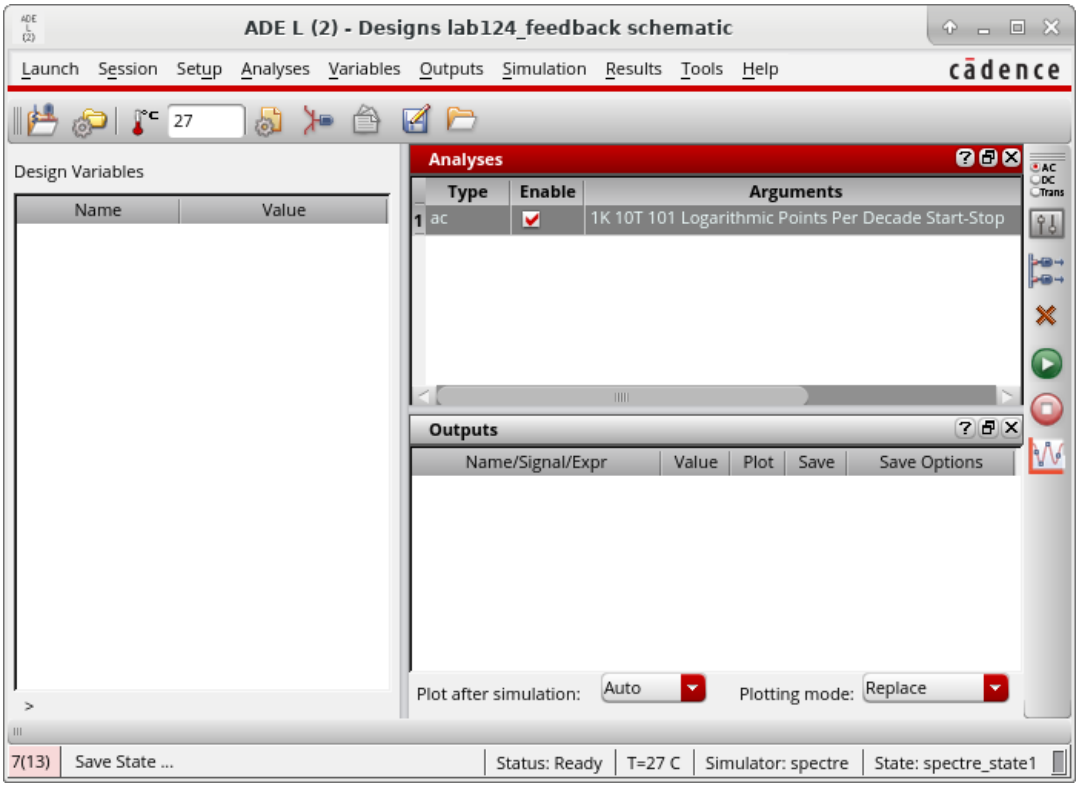
(4.2.5) Check and save your schematic using the **Check and Save** command. Ensure that it checks and saves without warnings or errors. Use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.

(4.2.6) From the ADE window, setup an AC frequency sweep analysis (**Menu: Analyses à Choose**). In this simulation we will sweep the frequency from 1KHz to 10THz. Because of the large frequency range, we will use a logarithmic sweep of the frequency – with 101 plot points per decade of frequency. At this stage, we will not select any outputs for plotting.

In the choosing analyses form, select the '**ac**' option button. Select the 'Frequency' option as the sweep variable. Choose the 'Start-Stop' option button with Start = 1K and Stop = 10T for the sweep range. Select 'Logarithmic' from the drop-down menu as the sweep type. Choose the 'Points Per Decade' option button with 101 plot points per decade. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. OK the form. The **ac** analysis should appear in the 'Analyses' list of the ADE window. See Figures 5 and 6.



< Figure 86. The Choosing Analyses Form for an AC frequency sweep. >



< Figure 87. The ADE with the only AC Analyses set up. >

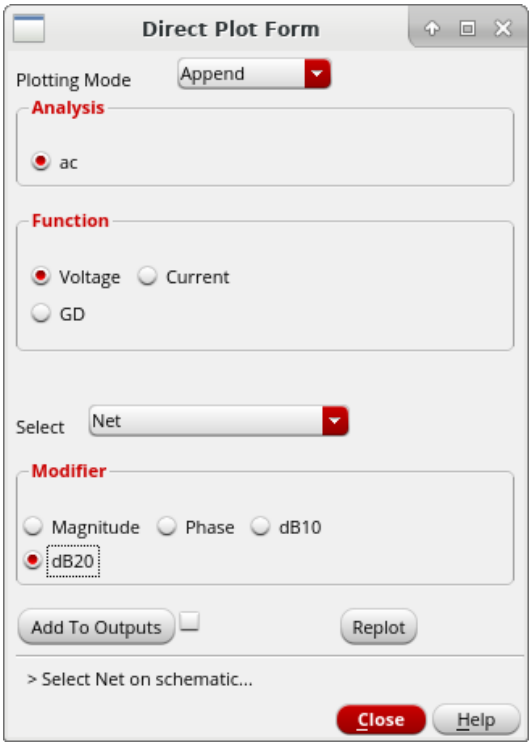
(4.2.7) In the ADE window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. Only the text output window should open automatically since we didn't select an output for plotting.

The Direct Plot Form

(4.2.8) Note: Until now, every time we have run a simulation we have set up the ADE, *before the simulation is run*, using the 'To Be Plotted' command (Menu: Outputs → To Be Plotted → Select On Schematic). This would plot one or more outputs in the waveform window upon completion of the simulation. A more general approach is to plot the outputs manually, *after a simulation has run*, using the **Direct Plot** command of the ADE. Using this option, you can quickly probe the circuit – selecting and plotting one or more waveforms at different nets and nodes one by one – without having to set them up in the ADE beforehand. You can also plot common expressions quickly without having to use the calculator.

(4.2.9) From the ADE window, select **Menu: Results → Direct Plot → Main Form** to invoke the Direct Plot Form (this may also invoke an empty waveform window). In the 'Analysis' section of the Direct Plot Form there will be an option button for every analysis type setup and run in the ADE. In this case, since we have only setup and run an AC analysis, there is only an '**ac**' option present. Associated with each analysis type (e.g., *trans, ac, dc, stb…*), will be a number of plotting options; for the ac analysis we have the function options (e.g., voltage, current, group delay) and the modifier options (e.g., magnitude, phase, dB10, or dB20). The options available can be very different for the different analysis types.

At the very top of the form is the potting mode drop-down menu, with the usual options – Replace, Append, New Sub-Window, and New Window. Leave it on the 'Append' option.



< Figure 88. The Direct Plot Form for AC Analysis. >

(4.2.10) From the Direct Plot Form, plot the output voltage magnitude and phase. Make sure the **ac** option button is selected in the analysis section. Select the **voltage** option button from the function section. In succession,

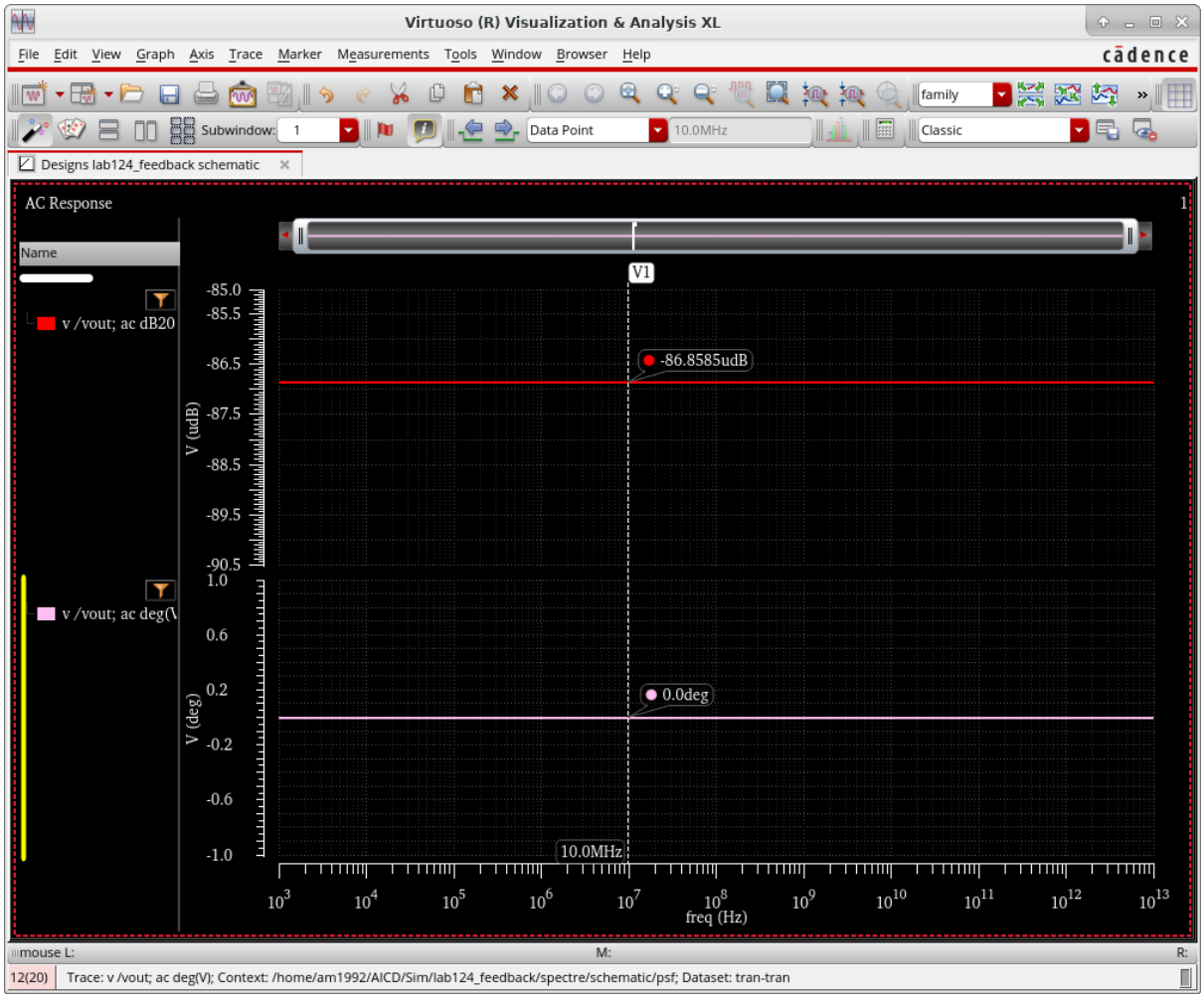
(1) Select the **dB20** option in the modifier section of the direct plot form. Then, in the schematic window, click on the output net *vout*. The output voltage magnitude plot vs. frequency should appear in the waveform window.

(2) Select the **phase** option in the modifier section of the direct plot form. Then, in the schematic window click again on the output net *vout*. The output voltage phase plot vs frequency. should be added to the waveform window (assuming the form's potting mode is set to append).

You should now have both a plot of amplitude (dB20) vs. frequency and a plot of phase (degrees) vs. frequency plotted in the waveform window. See Figure 8.

(4.2.11) From the schematic editor window, **press the 'ESC' key to exit the plotting mode of the Direct Plot Form**. Or simply close down the Direct Plot Form.

(4.2.12) Split the traces for clarity. Right-click on the empty area of the graph window and select **Split Current Strip à trace** from the context menu.



< Figure 89. Plot of the Amplitude (dB20) vs. Frequency, and Phase (degrees) vs. frequency.
1KHz to 10 THz >

(4.2.13) **Discussion**. Referring to Figure 8. The feedback system we are simulating, models a basic high gain op-amp connected in a unity-gain feedback closed-loop configuration. In this case, the closed-loop gain should be very close to one (linear scale) or 0dB (decibel scale). Recall, that the closed-loop gain *G* is given by,

$$G = \frac{A_V}{A_V + 1}$$

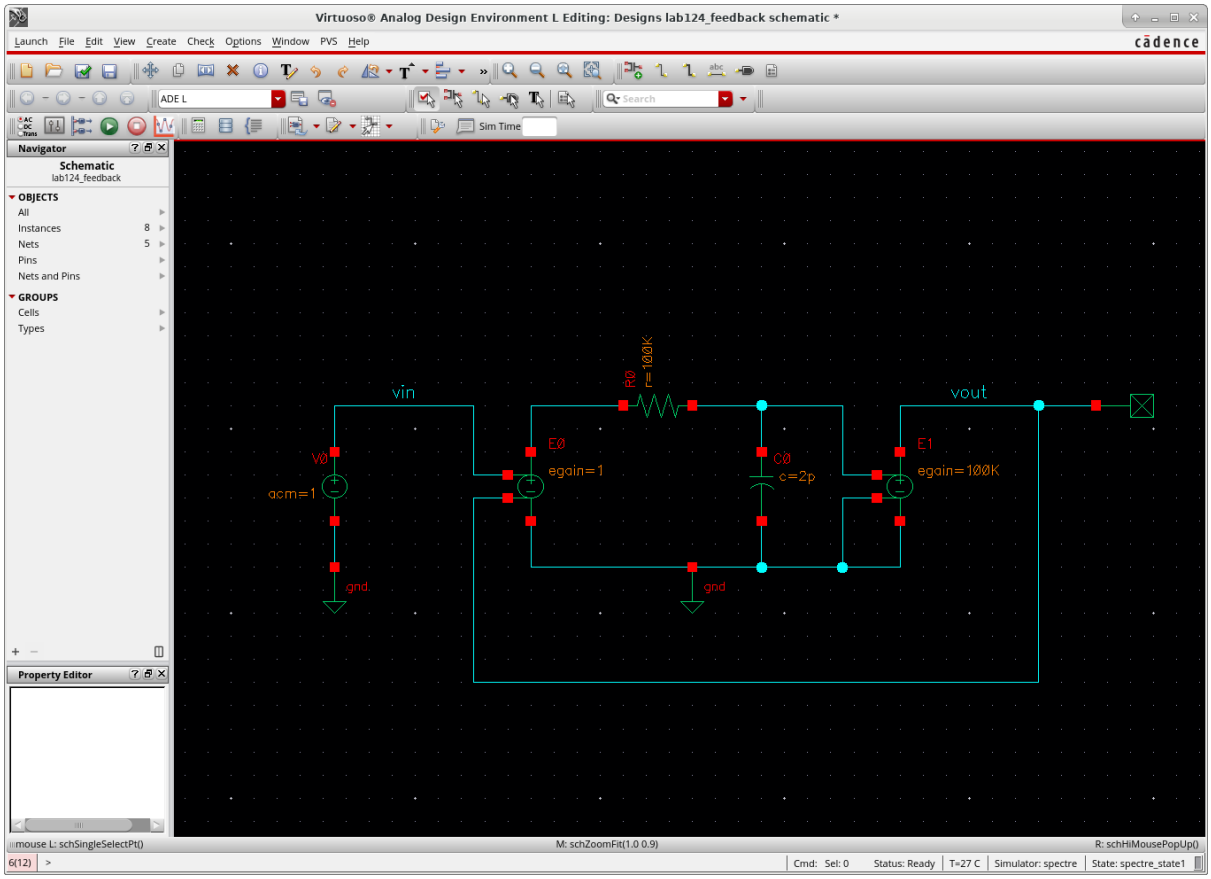
where A_V is the open loop gain of the system (100K). From the definition of dB20,

$$dB20(G) = 20\log G = 20\log\left(\frac{100K}{100K + 1}\right) = -86.86\ \mu dB$$

We see from Figure 8 (top), that the gain is indeed very close to unity (-87udB). It is not quite unity, since this would require an infinite open loop gain. We also note that since there are no reactive components in the model, the output response – magnitude and phase – has no frequency dependence i.e., zero degrees phase shift at the output across all frequencies.

4.3 Adding a Pole to the Forward Path

(4.3.1) Add a resistor and capacitor to the forward path of the simple feedback system as shown in Figure 9. From the analogLib, instantiate a resistor (**res**) and capacitor (**cap**). Set the resistor to 100KΩ (**Resistance** = 100K) and set the capacitor to 2pF (**Capacitance** = 2p).



< Figure 90. Simple Feedback System. Including a pole – caused by the RC low pass filter – in the forward path. >

(4.3.2) Check and save your schematic using the **Check and Save** command.

(4.3.3) Re-simulate the schematic. Again, use the Direct Plot form to plot both the magnitude (dB20) vs. frequency and phase (degrees) vs. frequency. See Figure 10.

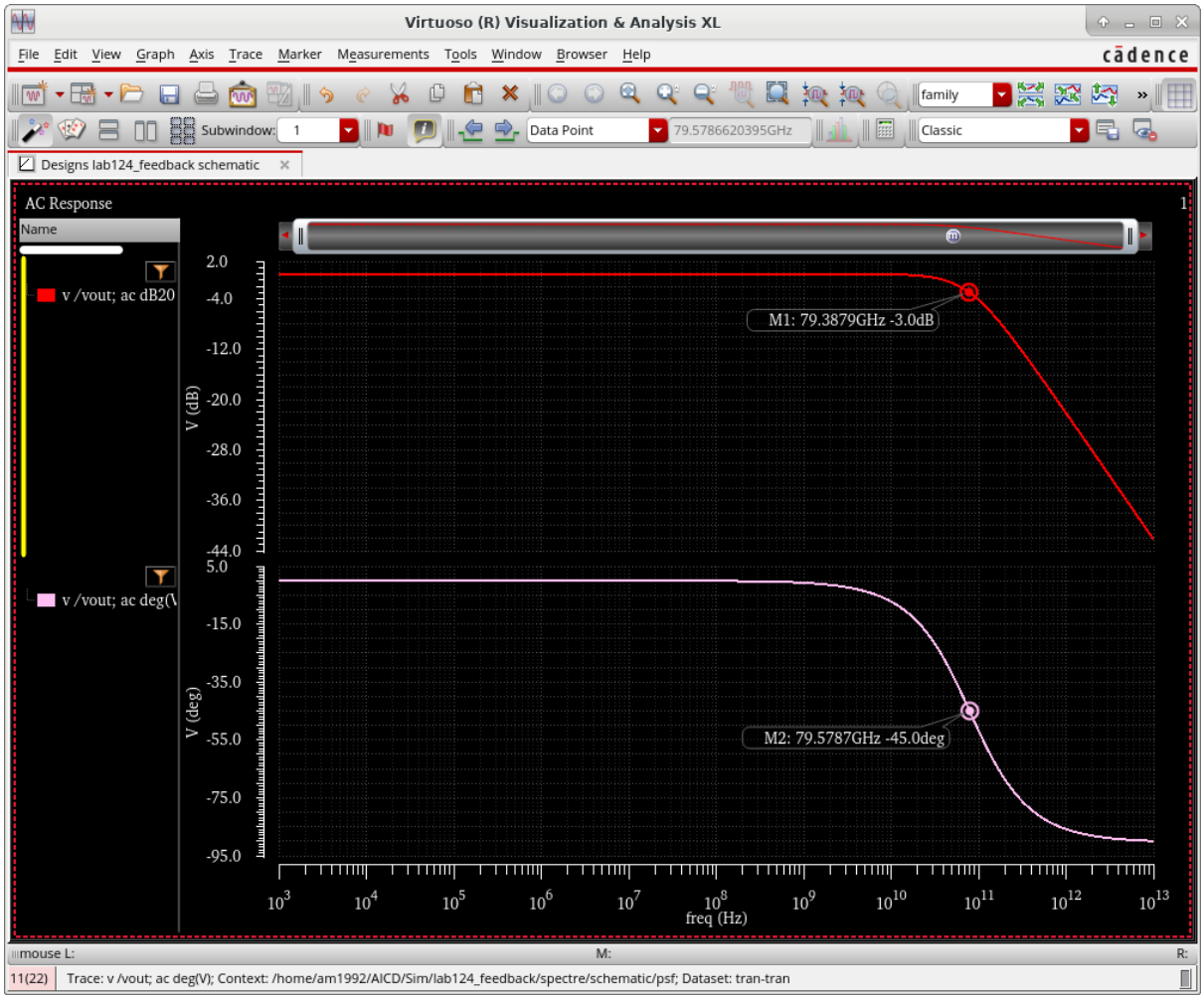
(4.3.4) Do not close down the waveform window.

(4.3.5) **Discussion**. Referring to Figure 10. We have introduced a low-pass filter into the forward path of the feedback system. The resistor/capacitor combination models a pole formed by the high output impedance of the first stage of a FET amplifier and the gate capacitance at the input of the following stage. Note that the combination of VCVS and series resistor represents the Thevenin equivalent circuit of the first stage's output – we could also do this with a VCCS and a parallel resistor (the Norton equivalent).

With the pole introduced in the forward path, caused by the RC product, we now see a frequency dependence in the output magnitude and phase response. In this case, the single pole leads to a corner-frequency located at:

$$f_{-3dB} = A_V \times \frac{1}{2\pi RC}$$

where R = 100KΩ, C = 2pF, and A_V = 100K, f_{-3dB} ≈ 79.58GHz. A marker is placed at the -3dB point on the magnitude plot and the -45° point on the phase point. See Figure 10.



< Figure 91. Bode plots for the simple feedback system with pole in the forward path. Plot of the amplitude (dB20) vs. Frequency (top), and Phase (degrees) vs. frequency (bottom). 1KHz to 10THz. >

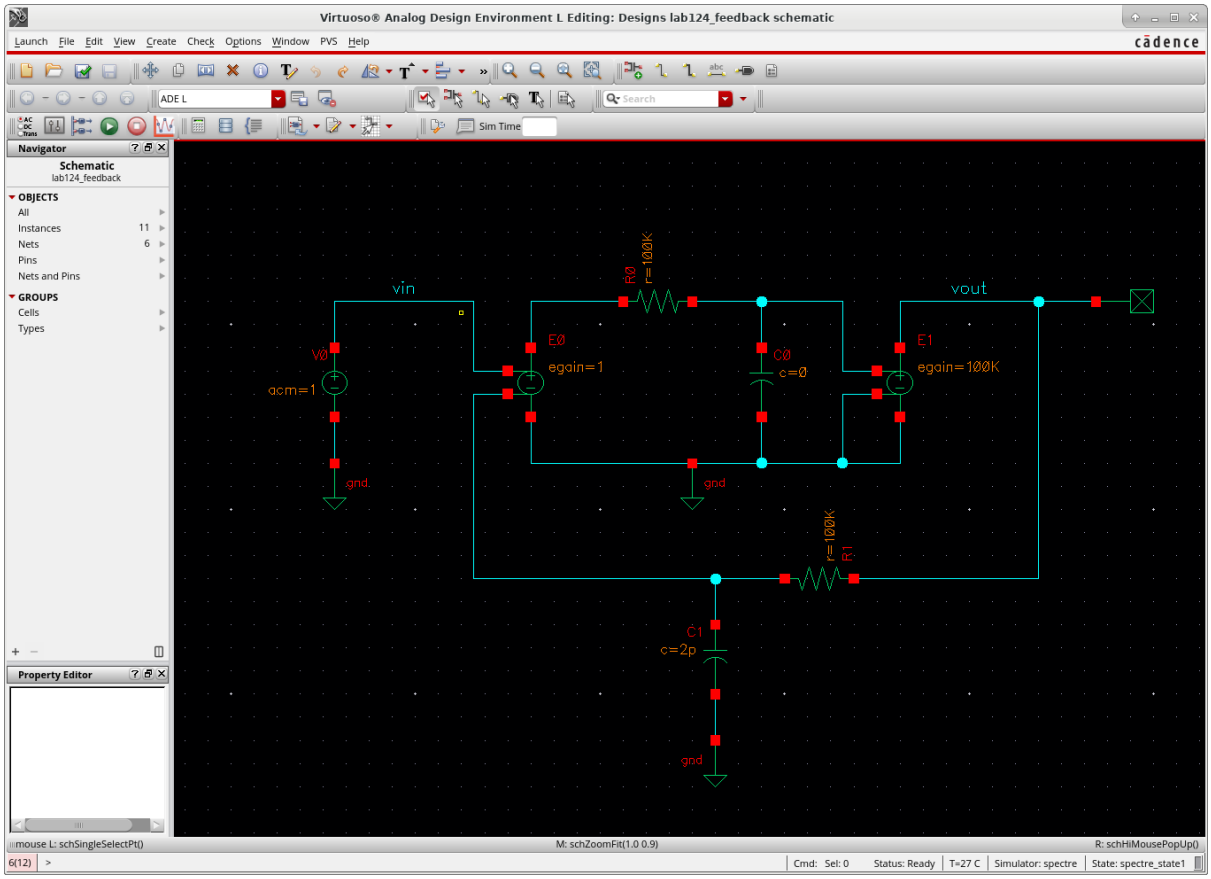
4.4 Adding a Pole to the Feedback Path

(4.4.1) Remove the capacitor in the forward path. Set the capacitor to 0 (**Capacitance** = 0). This effectively makes the capacitor look like an open circuit. The 100K resistor in the forward path has no effect since the input impedance of the following VCVS is infinite.

(4.4.2) Now, add a resistor and capacitor (low-pass filter) to the feedback path of the feedback system as shown in Figure 11. From the analogLib instantiate a resistor (**res**) and capacitor (**cap**). Set resistor to 100KΩ (**Resistance** = 100K), set capacitor to 2pF (**Capacitance** = 2p).

(4.4.3) Check and save your schematic using the **Check and Save** command.

(4.4.4) Re-simulate the schematic. Again, use the Direct Plot form to plot both the magnitude (dB20) vs. frequency and phase (degrees) vs. frequency. Make sure the plotting Mode of the Direct Plot Form is set to 'Append'.



< Figure 92. Simple Feedback System. Including a pole caused by the RC low pass filter – in the feedback path. Note the forward pole is removed since C = 0. The 100K resistor in the forward path has no effect since the input impedance of the following VCVS is infinite.>

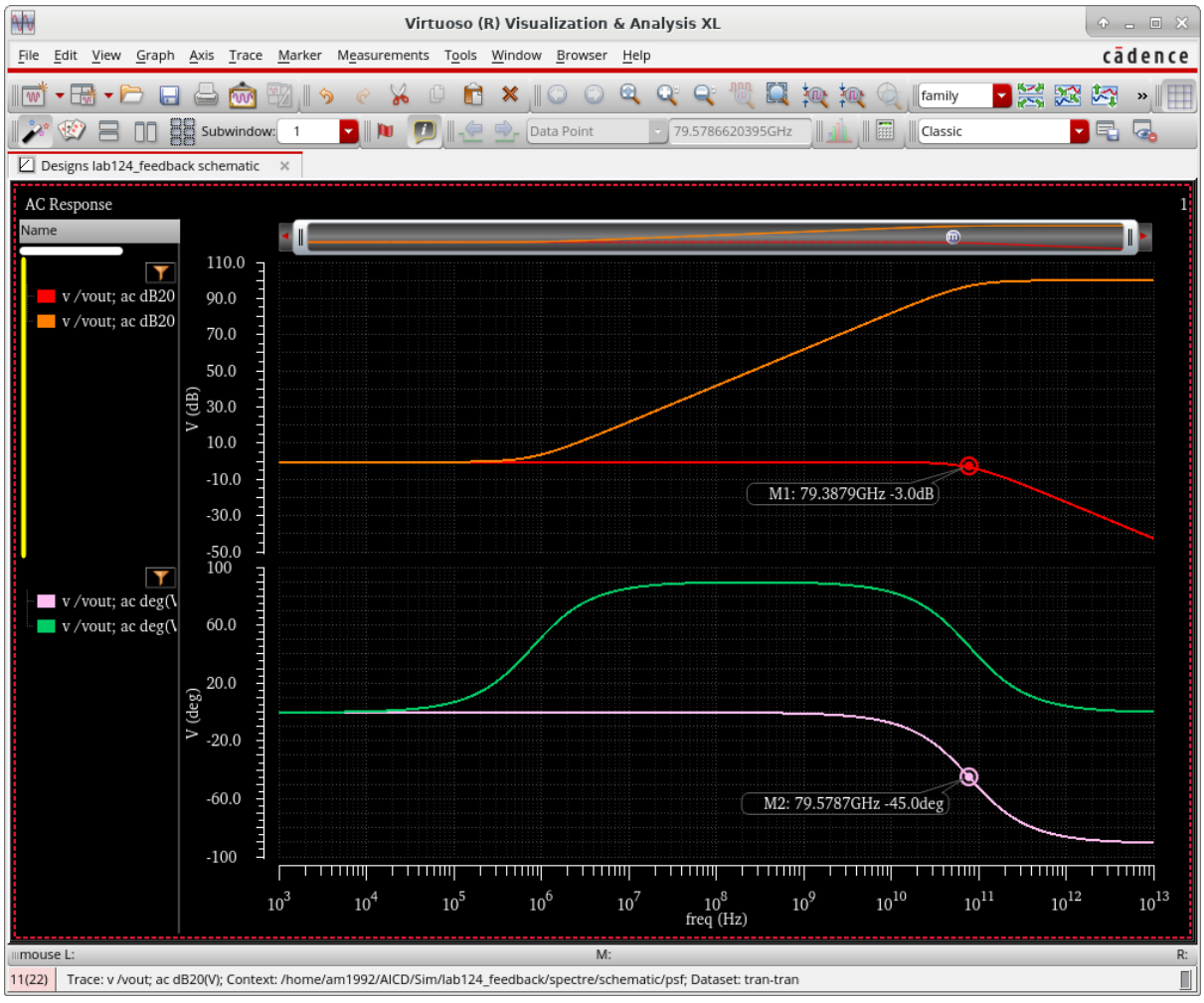
(4.4.5) **Discussion**. Referring to Figure 12 – note that the plot of the response with just the feedback pole has been appended to the plot of the response with just the forward pole. Here, we can see that the pole in the feedback path leads to a zero in the forward path located at,

$$f_1 = \frac{1}{2\pi RC}$$

This causes the output to behave like a high-pass filter. At low frequencies, below the zero frequency, the feedback system behaves like a unity-gain amplifier, with 100% of the output being fed back to the input. As the frequency increases, the pole (low-pass filter) in the feedback path begins to reduce the amount of signal being fed back to the input. As the level of feedback is reduced, the feedback system behaves more like an open-loop system. The apparent pole located at,

$$f_2 = A_V \times \frac{1}{2\pi RC}$$

occurs due to the finite open-loop gain of the amplifier. Above this frequency, the feedback pole no longer passes the feedback signal - the system effectively becomes open-loop with a gain of A_V=100K (=100 dB).

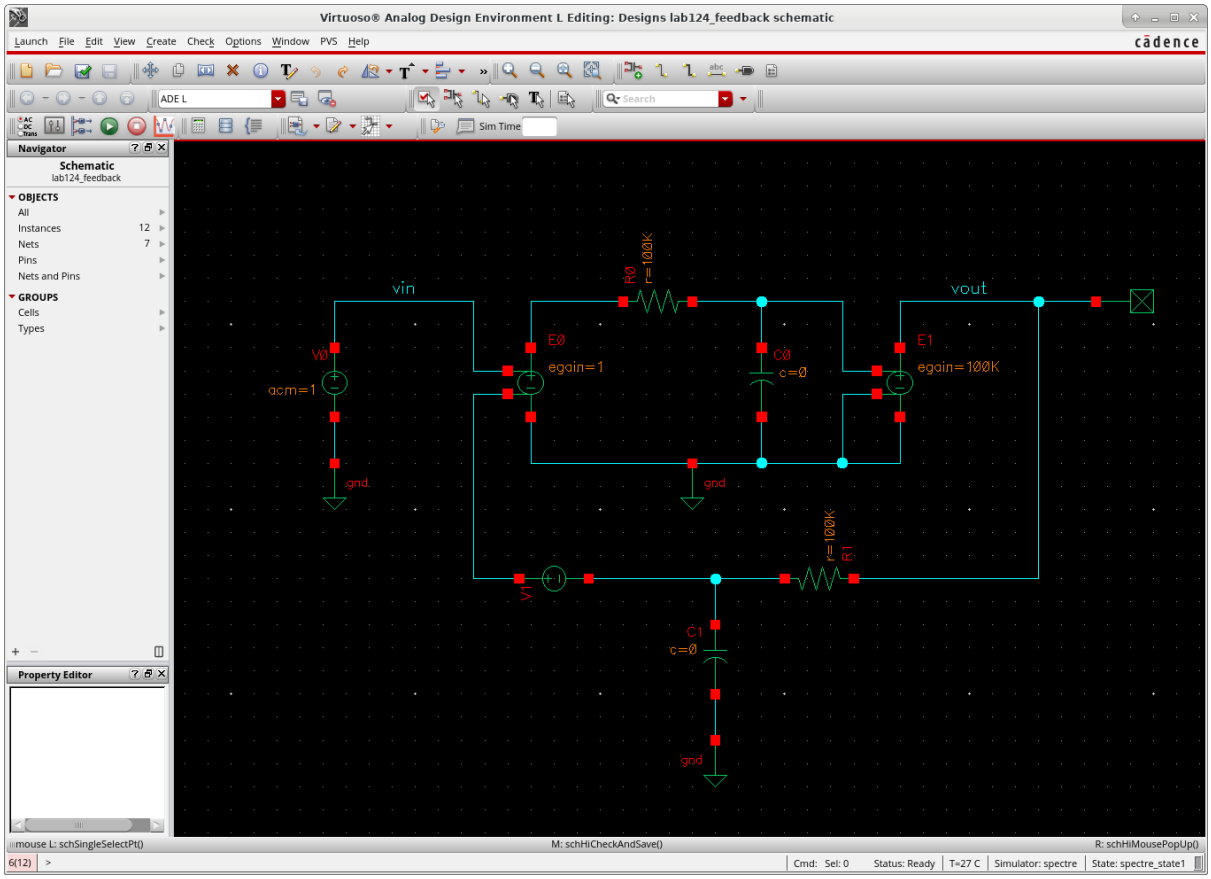


< Figure 93. Bode plots for the simple feedback system with pole in the feedback path. Plot of the magnitude (dB20) vs. Frequency (top), and Phase (degrees) vs. frequency (bottom). 1KHz to 10THz. Appended to previous plot of only forward path pole>

4.5 Stability Analysis: Loop Gain, Loop Phase, Phase Margin, and Gain Margin

(4.5.1) Here, we will explore the stability of the feedback system. Remove the capacitors in both the forward and feedback paths by setting both the capacitors to 0 (**Capacitance** = 0).

(4.5.2) Insert a null **vdc** voltage source in the feedback path after the capacitor Null means 0V, it is a short circuit element. The AC and DC magnitude values of this component should be set to zero or left empty. This vdc does not in any way change the functionality of the circuit or its output response. It is used only as a loop identifier/probe by the stability analysis tool.



< Figure 94. Simple Feedback System with all poles zeroed (C0=C1=0). A null vdc is placed in the feedback path after the capacitor (AC/DC Voltage = 0). It is used by the stability analysis to identify the feedback loop. >

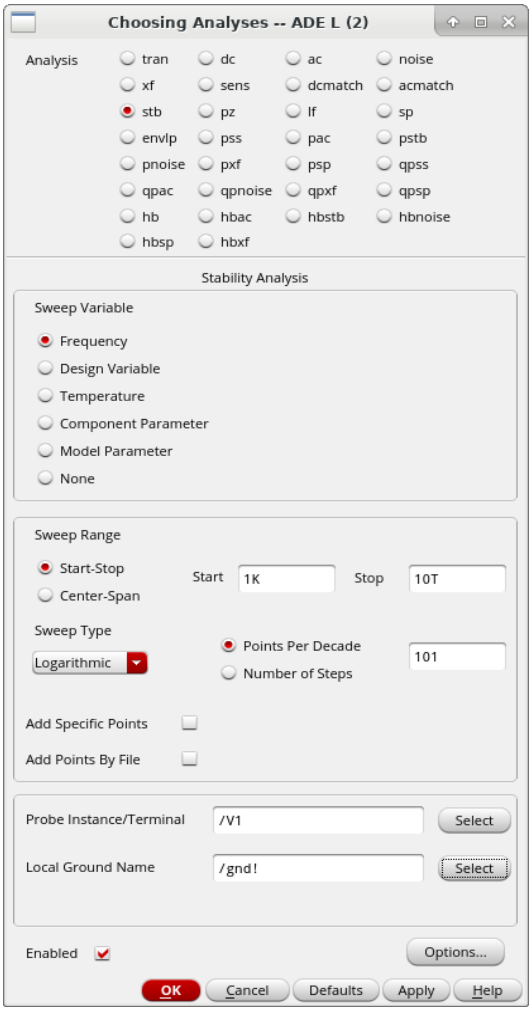
(4.5.3) We want to determine the stability of the feedback system. We do this by examining how the loop gain and phase of the circuit change with frequency. Specifically, we are looking for when the loop gain undergoes a 180° phase-shift, at which point if the gain magnitude is greater than unity (0 dB), the applied negative feedback becomes positive. As we discussed in the opening section (4.2), when the feedback becomes positive, the output will grow unbounded until it saturates. Note that the loop gain is the total gain around the feedback loop – it is the product of the forward gain ***A(ω)*** and the feedback factor ***B(ω)***. Refer to Figure 31, section 7.1.1. Recall, from the Barkhausen stability criteria that, stability is determined by the product ***AB*** being below unity (0dB) before the loop gain phase-shift reaches 180°.

(4.5.4) From the ADE window, setup a stability analysis (**Menu: Analyses à Choose**). In this simulation we will again sweep the frequency from 1KHz to 10THz using a logarithmic sweep of the frequency – with 101 plot points per decade of frequency. At this stage, we will not select any outputs for plotting (we will use the Direct Plot Form later).

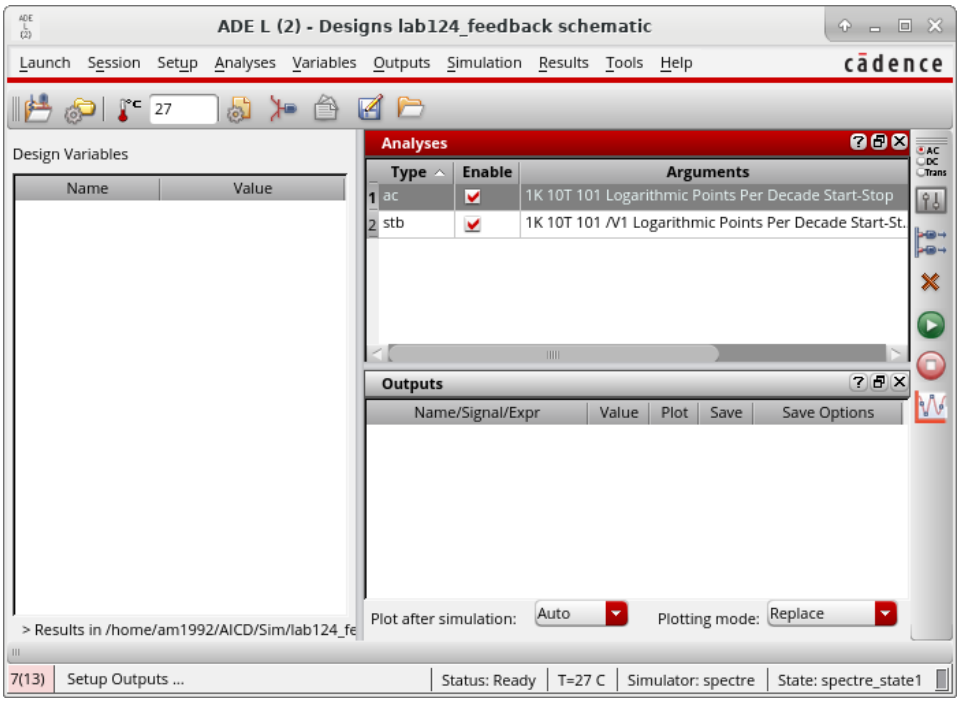
In the choosing analyses form, select the '**stb**' option button. Select the 'Frequency' option as the sweep variable. Choose the 'Start-Stop' option button with Start = 1K and Stop = 10T for the sweep range. Select 'Logarithmic' from the drop-down menu as the sweep type. Choose the 'Points Per Decade' option button with 101 plot points per decade.

At the bottom of the form, there are two fields: 'Probe Instance/Terminal' and 'Local Ground Name'. The probe instance/terminal is used to identify the feedback loop – we need to point this to the null vdc component we inserted into the feedback path. Press the select button next to the 'Probe Instance/Terminal' form field. The schematic should come into focus. Click on the null vdc component. The name of the null vdc part (/V1) should now appear in the form field. We also need to do the same with the local ground name. Press the select button next to the 'Local Ground Name' form field. The schematic should come into focus. Click on any net connected to a ground (gnd) reference. The name of the ground net (/gnd!) should now appear in the form field.

Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. OK the form. The **stb** analysis should appear in the 'Analyses' list of the ADE window next to the **ac** analysis already set up. See Figures 14 and 15.



< Figure 95. The Choosing Analyses Form for a Stability Analysis. >



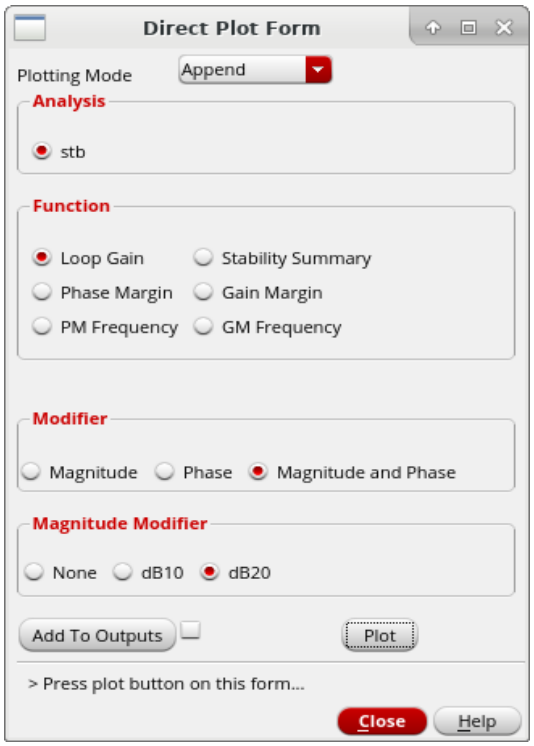
< Figure 96. The ADE with the AC and STB Analyses set up. >

(4.5.5) In the ADE window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. Only the text output window should open automatically since we didn't select an output for plotting.

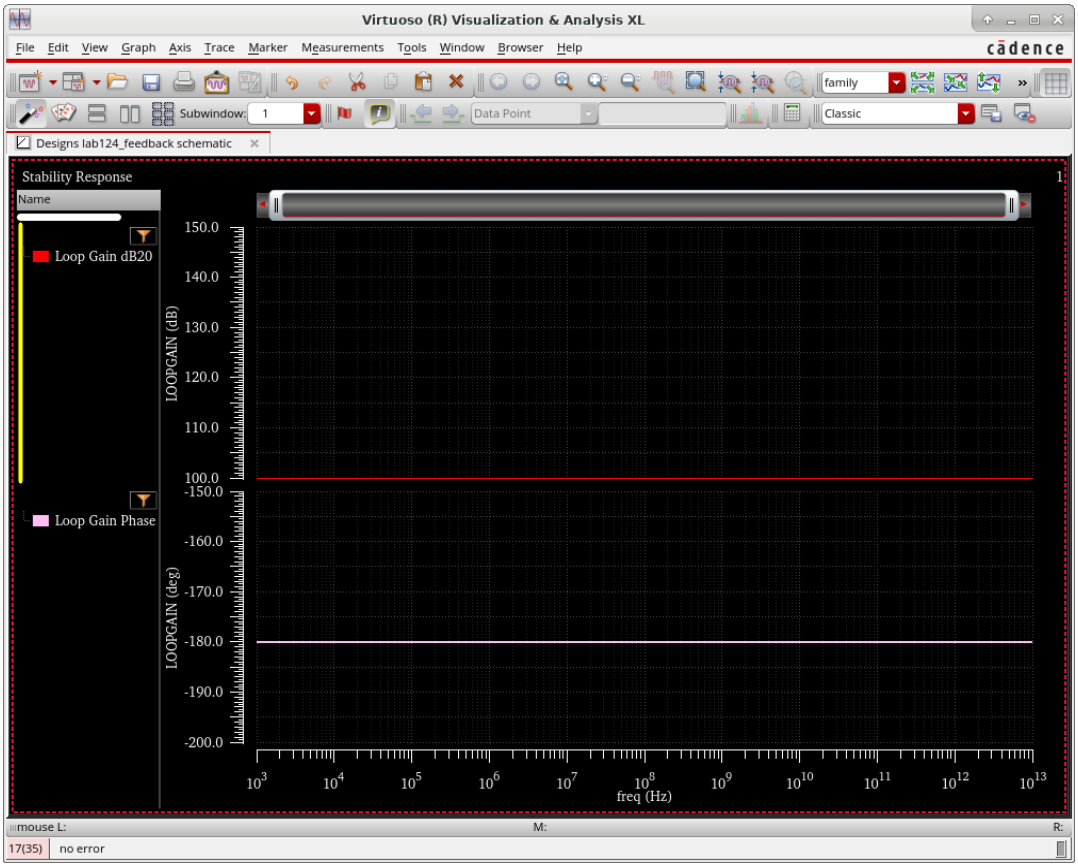
(4.5.6) From the ADE window, open the Direct Plot form (**Menu: Results → Direct Plot → Main Form**). Note that the **stb** analysis option button has been added alongside the existing **ac** option button to the 'Analysis' section of the Direct Plot form. Switch between the ac and stb options to see how the plotting options/expressions change depending on analysis type. For the stb analysis type, there are several plotting options for determining stability. At this stage we only want to plot the loop gain – magnitude and phase – versus frequency.

Make sure the **stb** option button is selected in the Analysis section of the Direct Plot Form. Select the **Loop Gain** option button from the function section. Select the **Magnitude and Phase** option in the modifier section of the direct plot form. In the magnitude modifier section, select the **dB20** option. Click the 'Plot' button near the bottom right corner of the form. You should now have both a plot of loop gain magnitude (dB20) vs. frequency and a plot of loop gain phase (degrees) vs. frequency plotted in the waveform window. See Figures 16 and 17.

(4.5.7) Split the traces for clarity. Right-click on the empty area of the graph window and select **Split Current Strip à trace** from the context menu.



< Figure 97. The Direct Plot Form for Stability Analysis. >



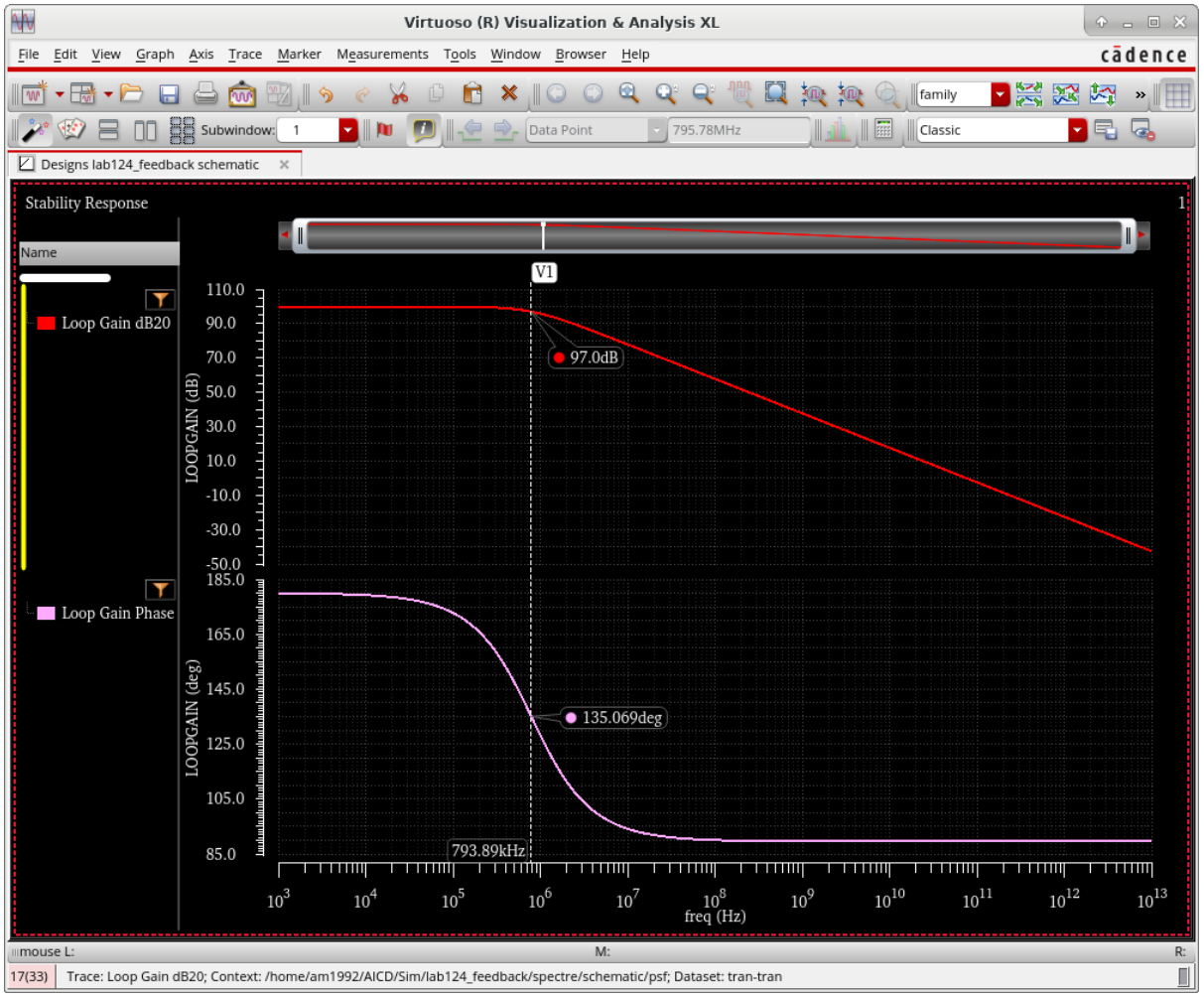
< Figure 98. Loop Gain Magnitude and Phase Plot of the Simple Feedback System. (C0=C1=0). Note that from the frequency response there are no poles or zeros. >

(4.5.8) Note: Referring to Figure 17. Since both the poles in the forward and feedback paths of the feedback system have been removed (C0=C1= 0), there is no frequency dependence of the loop gain magnitude and loop gain phase. The feedback system is unconditionally stable, since the feedback signal is not subject to any phase shift and therefore can never make the 180° transition from stable negative feedback to positive feedback.

(4.5.9) Return the pole to the forward path of the feedback system. Set the forward path capacitor back to 2pF (Capacitance = 2p).

(4.5.10) Check and save your schematic using the **Check and Save** command.

(4.5.11) Re-simulate the stability analysis. Again, use the Direct Plot form to plot the loop gain (dB20) vs. frequency and loop gain phase (degrees) vs. frequency. See Figure 18.



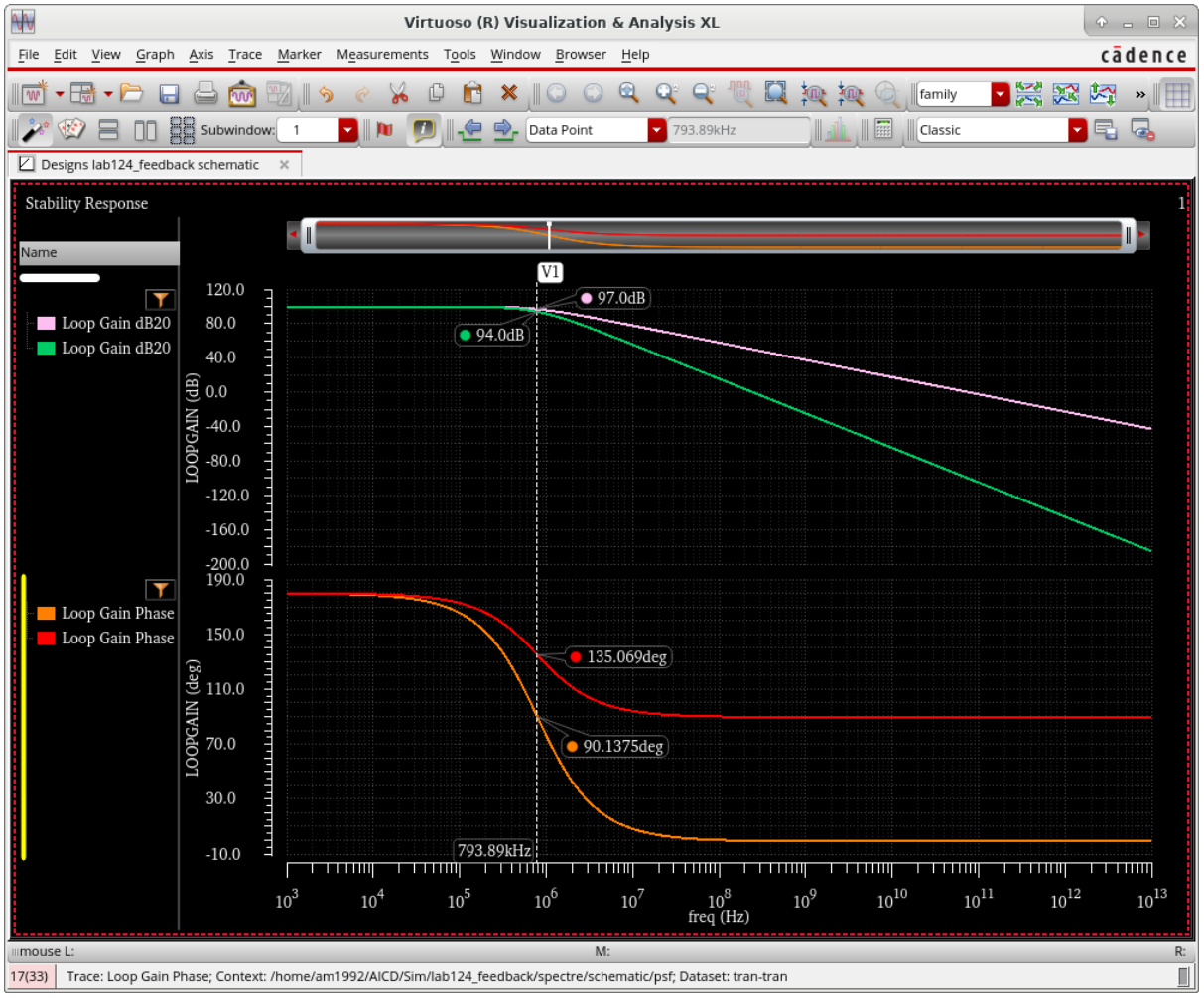
< Figure 99. Loop Gain Magnitude and Phase Plot of the Simple Feedback System. With C0=2pF and C1=0. Note that there is only one pole in the forward path ~794KHz. >

(4.5.12) Note: Referring to Figure 18. With the pole in the forward path of the feedback system (C0 = 2pF, C1 = 0), there is a frequency dependence of the loop gain magnitude and loop gain phase. On the magnitude plot the pole introduces a roll-off of 20dB per decade starting at the corner-frequency given by $\omega_{-3dB} = 1/RC$ where R=100K, C=2pF. (Giving $f_{-3dB} \approx 795.78KHz$). Since a pole can introduce a phase shift of up to 90° in the feedback signal, again, the feedback signal cannot make the 180° transition from stable negative feedback to unstable positive feedback. The feedback system is still unconditionally stable.

(4.5.13) Now, return the pole to the feedback path of the feedback system. Set the feedback path capacitor back to 2pF (Capacitance = 2p).

(4.5.14) Check and save your schematic using the **Check and Save** command.

(4.5.15) Re-simulate the stability analysis. Again, use the Direct Plot form to plot the loop gain (dB20) vs. frequency and loop gain phase (degrees) vs. frequency. See Figure 19.

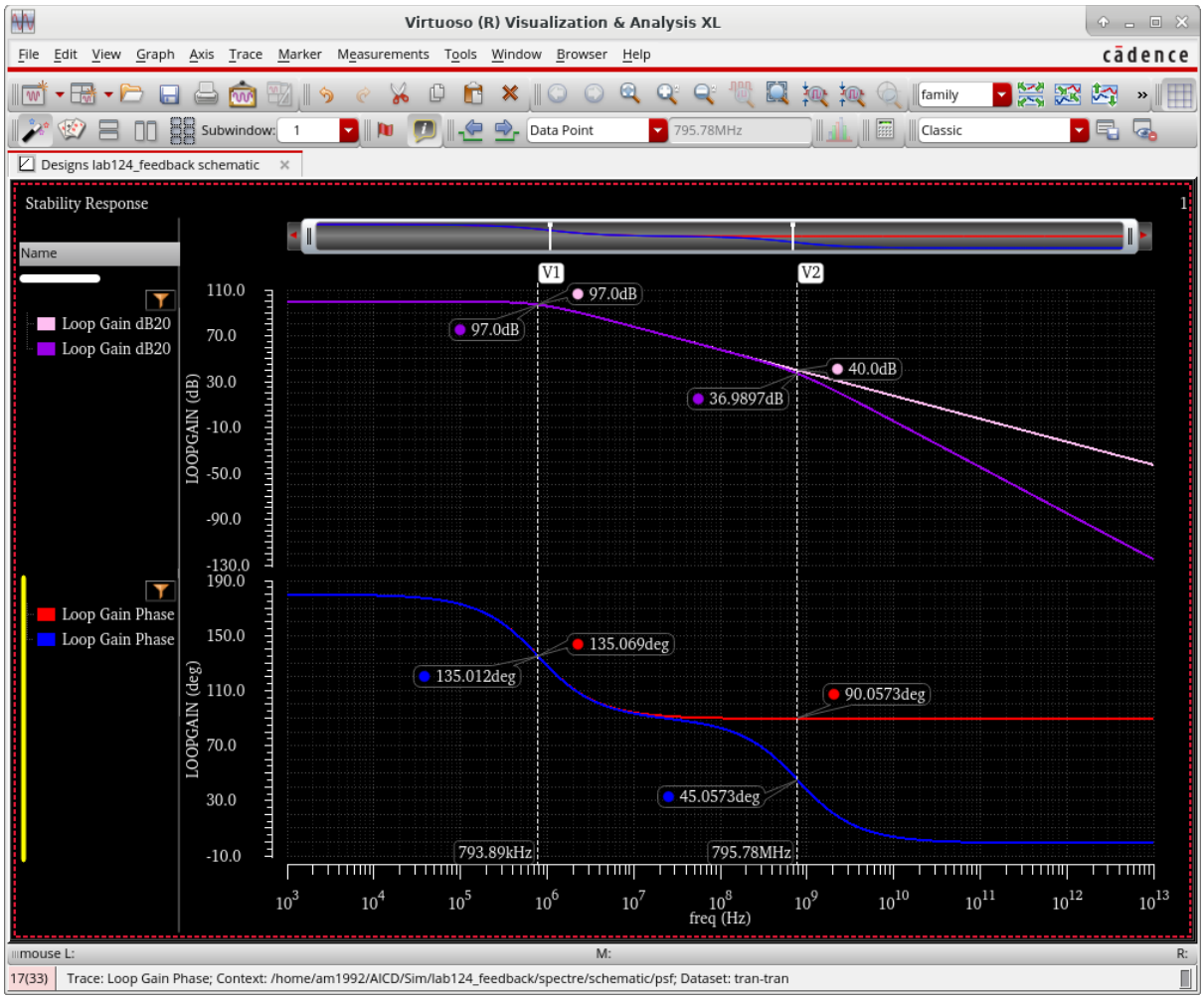


< Figure 100. Loop Gain Magnitude and Phase Plot of the Simple Feedback System. With C0=2pF and C1=2pF. Note the there are two poles: one in the forward path and one in feedback path. Both poles are located at the same frequency ~794KHz, so neither is dominant. >

(4.5.16) Note: Referring to Figure 19. With both the forward path and feedback path poles present in the feedback system (C0 = 2pF, C1 = 2pF). As we would expect, there is a frequency dependence of the loop gain magnitude and loop gain phase.

On the magnitude plot, each pole introduces a roll-off of 20dB per decade and since there are two poles closely located, we see a 40 dB roll-off overall – both poles give rise to the same the corner-frequency, $\omega_{-3dB} = 1/RC$ where R=100K, C=2pF. (Giving $f_{-3dB} \approx 795.78KHz$). Each pole also introduces a phase shift of up to 90° in the feedback signal. Since there are two poles, we now see a (close to) 180° transition in the feedback signal. The feedback system is therefore on the very edge of instability. This system is technically referred to as **marginally stable**, since the feedback never quite becomes positive. Bear in mind, however, that this is a very simple ideal system; for all practical purposes this system would be considered **unstable** (or at least **insufficiently stable**), especially when you consider that in a real system – subject to parasitics, PVT variation, parameter drift over time, etc – a marginally stable system can very easily be tipped into instability.

(4.5.17) As we noted above, both forward path and feedback path poles are closely located. To make the effect of each pole more obvious, we will make the feedback path pole non-dominant.



< Figure 101. Loop Gain Magnitude and Phase Plot of the Simple Feedback System. With C0=2pF and C1=2fF. Note the there are two poles: one in the forward path and one in feedback path. The forward path pole is still located at ~794Khz. The feedback path pole is located at ~796MHz. The forward path pole is dominant. >

(4.5.18) Set the feedback path capacitor to 2fF (Capacitance = 2f). Keep the resistor at 100KΩ.

(4.5.19) Check and save your schematic using the **Check and Save** command.

(4.5.20) Re-simulate the stability analysis. Again, use the Direct Plot form to plot the loop gain (dB20) vs. frequency and loop gain phase (degrees) vs. frequency. See Figure 20.

(4.5.21) Note: Referring to Figure 20. Both the forward path and feedback path poles are present in the feedback system (C0 = 2pF, C1 = 2fF), but one dominates. On the magnitude plot, each pole introduces a roll-off of 20dB per decade, but this time we see the two break frequencies more clearly. The pole in the forward path causes a roll-off of 20dB per decade and dominates the frequency response since it occurs at a much lower frequency, the corner-frequency set by $\omega_{-3dB} = 1/RC$ where R=100K, C=2pF. Giving $f_{1\text{-}3dB} \approx 795.78KHz$. The second pole occurs at a much higher frequency, causing the roll-off to increase to 40 dB per decade, the second break frequency occurring at $\omega = 1/RC$ where R=100K, C=2fF. Giving $f_{2\text{-}3dB} \approx 795.78MHz$ Changing the location of the second pole (in the frequency domain) does not change the stability of the feedback system. Again, each pole introduces a phase shift of 90° in the feedback signal. We still see (close to) a 180° transition in the feedback signal, with the feedback system still on the edge of instability. This system would still be considered unstable.

(4.5.22) We will continue our simulations of system stability in part two of this lab using a different design/schematic. We will discuss the general approach to compensating unstable systems and we will introduce the stability metrics of phase margin and gain margin.

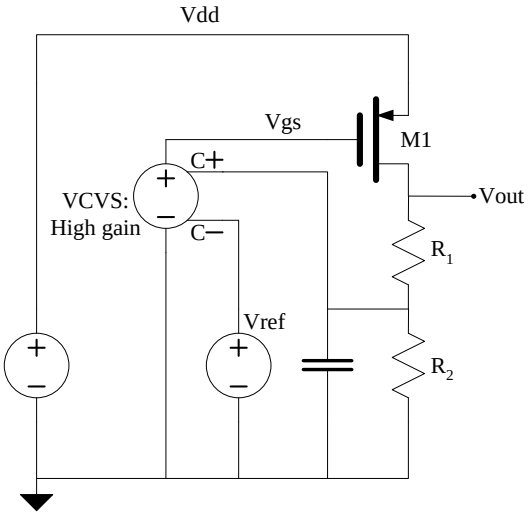
5. Circuit Simulation Exercises: Part 2

With guidance, you will:

- Investigate the stability of a voltage reference circuit with modelled op-amp drive.

5.1 A Controllable Voltage Reference Circuit

(5.1.1) Create the schematic for the IC voltage reference circuit in Figure 21. A summary of the circuit's operation is given in section 4.6 of the background material.

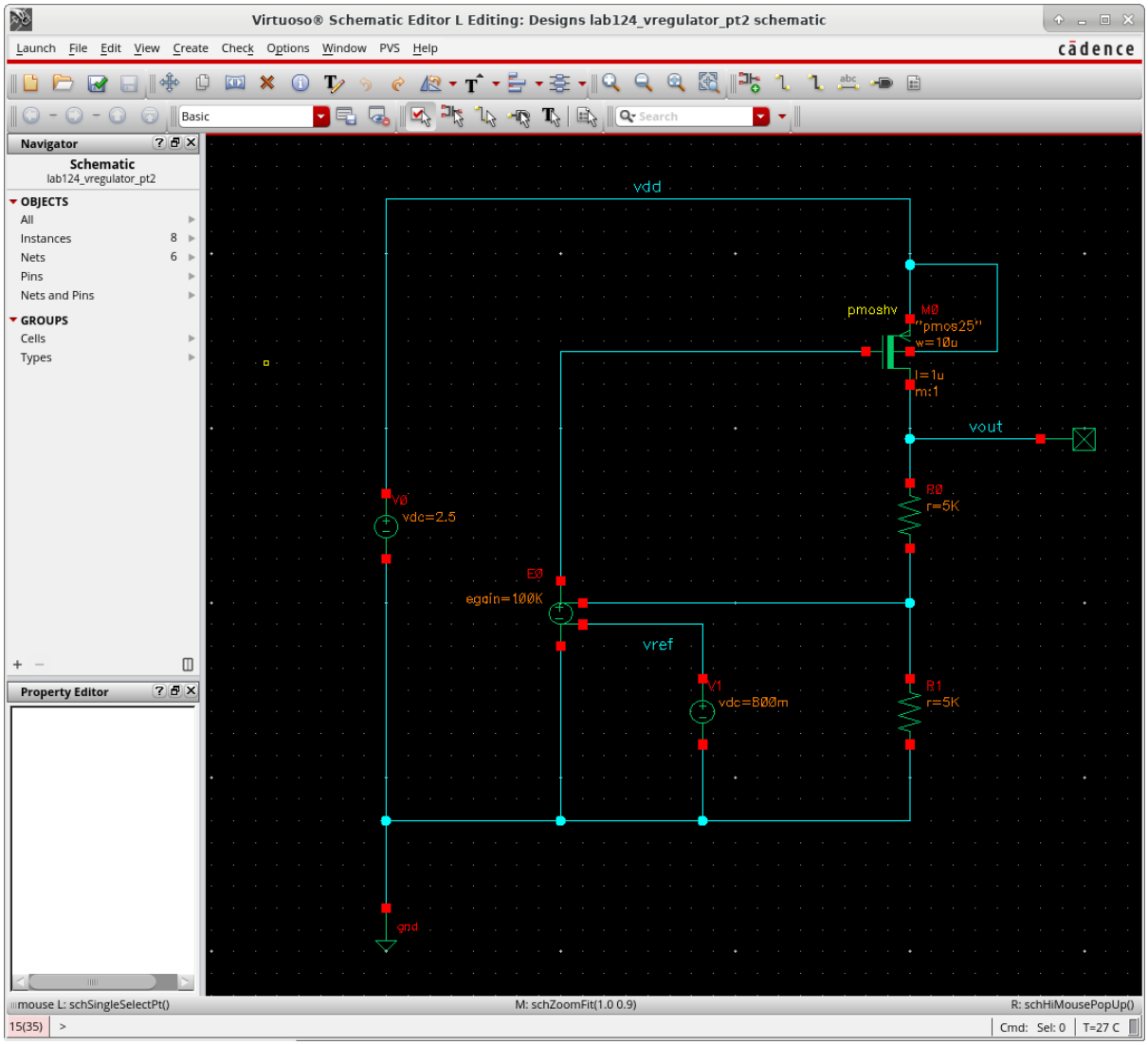


< Figure 102. IC Controllable Voltage Reference Circuit
Reproduced from Figure 30, section 4.6 of the background material. >

(5.1.2) From the Library Manager window, select the 'Designs' library and choose **Menu: Files à New à Cell View** to invoke the New File form. Give the new schematic a meaningful name such as lab124_vregulator_pt2.

(5.1.3) In the schematic editor window, construct the circuit given in Figure 21 (22). From the analogLib library instantiate and place two DC voltage sources (**vdc**), two resistors (**res**), a voltage-controlled voltage source (**vcvs**), and a ground reference (**gnd**). Also instantiate one high-voltage PMOS FET (**pmoshv**) from the GPDK180 library. Set the DC supply voltage to 2.5V (**DC Voltage** = *2.5*), set the reference voltage to 0.8V (**DC Voltage** = *0.8*), set the two resistors to 5KΩ each (**Resistance** = 5K), and dimension the PMOS device 10um/1um (**width** = *10u*, **length** = 1u). Leave all other attributes unchanged.

(5.1.4) Wire the components as in Figures 21 and 22 using the **Wire (narrow)** command. **Remember to connect the body/bulk terminal of the PMOS device to the most positive net in the circuit (VDD=2.5V)**. Using the **Create Wire Name** command, name the DC supply net *vdd*, the PMOS gate net *vgs*, and the reference input net *vref*.



< Figure 103. Controllable Voltage Reference Circuit. >

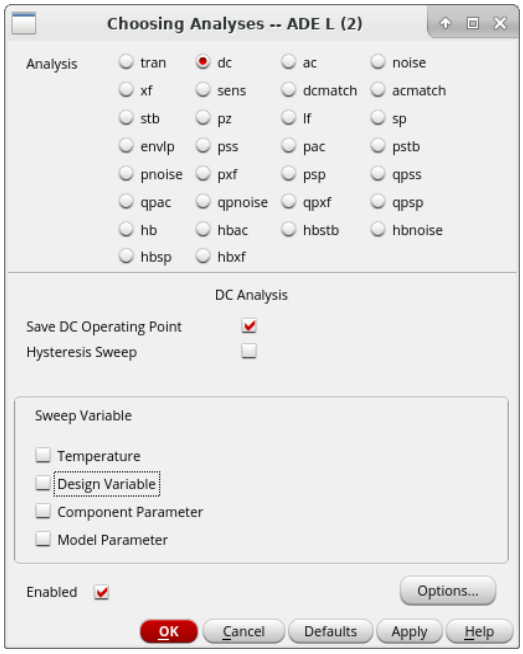
(5.1.5) Check and save your schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.

5.2 DC Operating Point Analysis of the Volage Reference Circuit

(5.2.1) You should now have the completed schematic of the voltage reference circuit as shown in Figure 22. It should check and save without any errors or warnings.

(5.2.2) We will simulate the voltage reference using a DC Analysis. From the Schematic Editor window, launch the Analogue Design Environment (ADE) from **Menu: Launch à ADE L**.

(5.2.3) From the ADE window, go to **Menu: Analyses à Choose…** or use the **Choose Analyses** button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. On the form, select the '**dc**' option button. Check the 'Save DC Operating Point' box. OK the form.

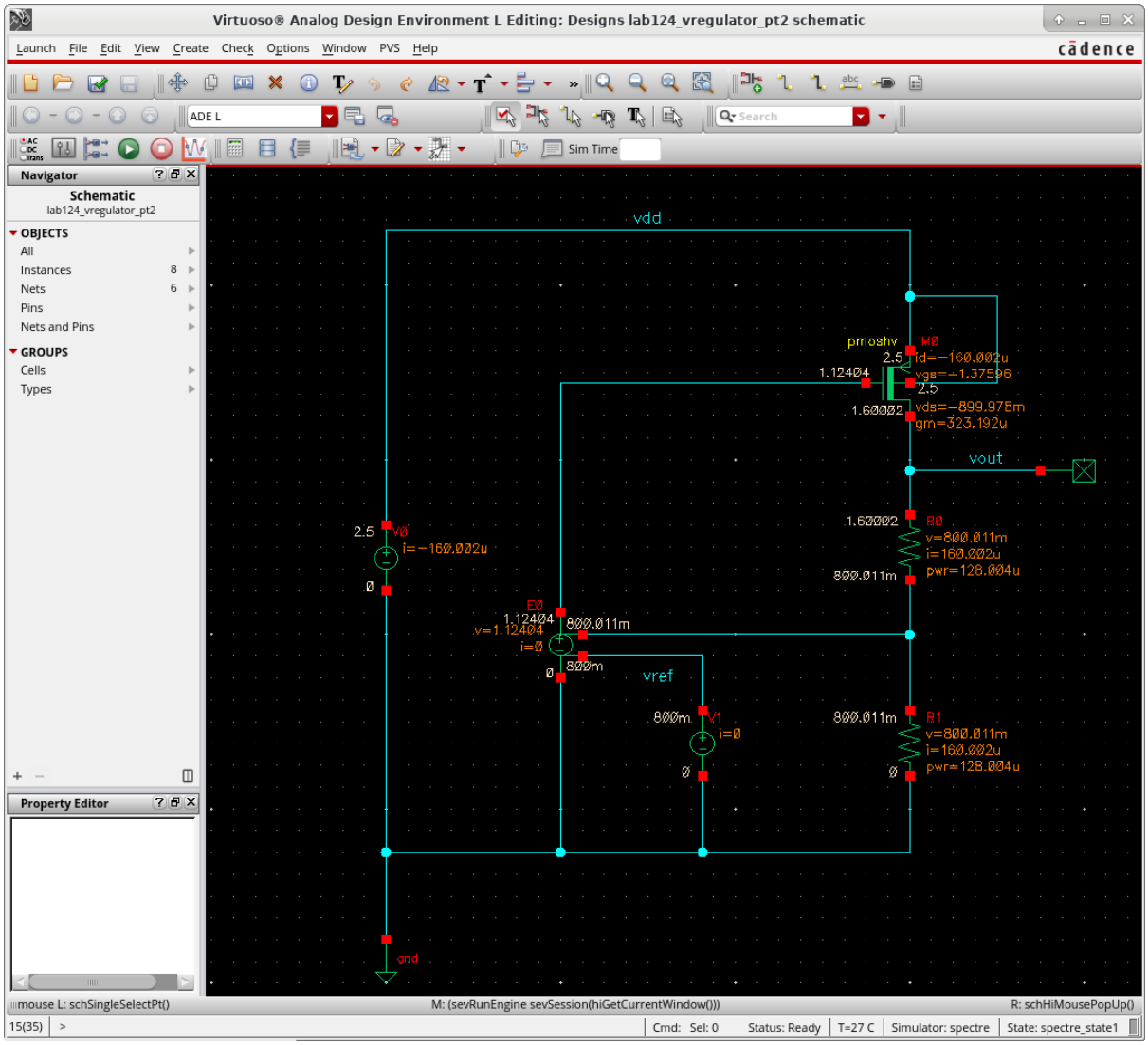


< Figure 104. The Choosing Analyses Form. Only DC Operating Point selected. >

(5.2.4) At this stage we are only simulating for the DC operating point. Simulating the DC operating point does not produce an output waveform, so we do not need to select any outputs to be plotted.

(5.2.5) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. The text output window should appear but there will be no waveform window.

(5.2.6) From the ADE window choose **Menu: Results à Annotate à DC node voltages…** If you now look in the schematic window, you should see the node voltages annotated directly on the schematic. Again, from the ADE window, choose **Menu: Results à Annotate à DC operating points…** You should now see (along with the node voltages) additional DC operating point information annotated on the schematic including the DC bias currents. See Figure 24.



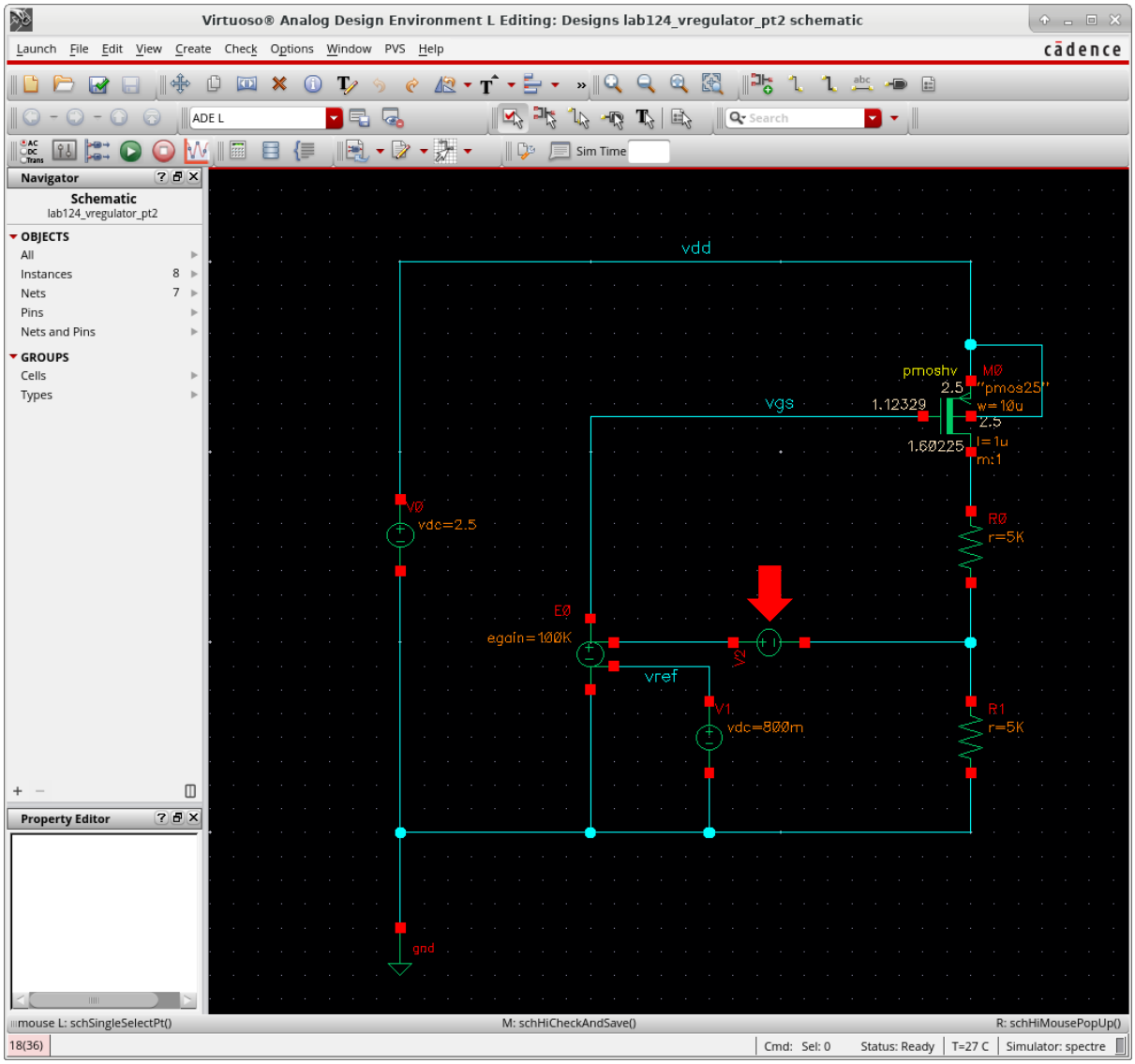
< Figure 105. Controllable Voltage Reference Circuit with Node Voltage and DC Operating Point Annotations. >

(5.2.7) **Discussion**. Refer to section 4.6 of the background material for a discussion on circuit operation. The important point to note is that the high gain of the VCVS regulates the output voltage(*vout*) to be twice that of the voltage reference (*vref*). The factor of two occurs due to the even division of the feedback signal across the potential divider. The feedback results drives the same voltage to be on both VCVS inputs, this requires an output voltage across the resistors equal to 2*vref so that vcvs(+) equals vcvs(-). The high gain of the VCVS is used to overcome any non-ideal behaviour of the PMOS – or any other disturbance. The higher the gain the more closely the output will track the voltage reference (by a factor of two.) Note that the PMOS device provides an inversion in the loop – so the input to the VCVS is the negative terminal and the feedback signal goes to the positive terminal.

5.3 Stability Analysis of the Volage Reference Circuit

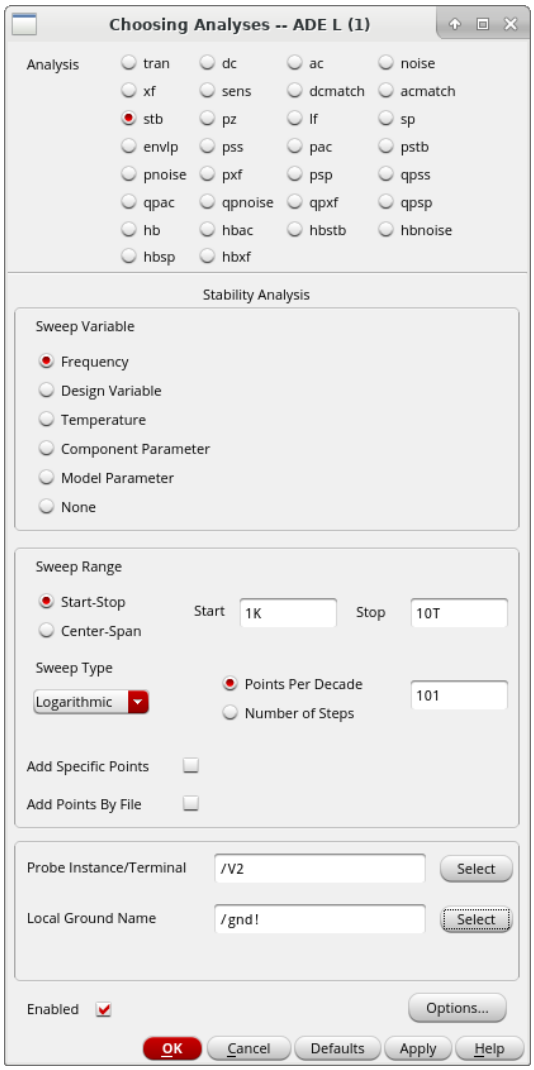
(5.3.1) Here, we will explore the stability of the voltage reference circuit.

(5.3.2) Insert a null **vdc** voltage source in the feedback path just before positive terminal of the VCVS. The AC and DC magnitude values of this component should be set to zero or left empty. See Figure 25 (highlighted by the red arrow). This vdc does not in any way change the functionality of the circuit or its output response. It is used only as a loop identifier/probe by the stability analysis tool.



< Figure 106. Annotated Controllable Voltage Reference Circuit with null VDC component in the feedback path for stability analysis. >

(5.3.3) We want to determine the stability of the feedback system by examining how the loop gain and phase of the circuit change with frequency.



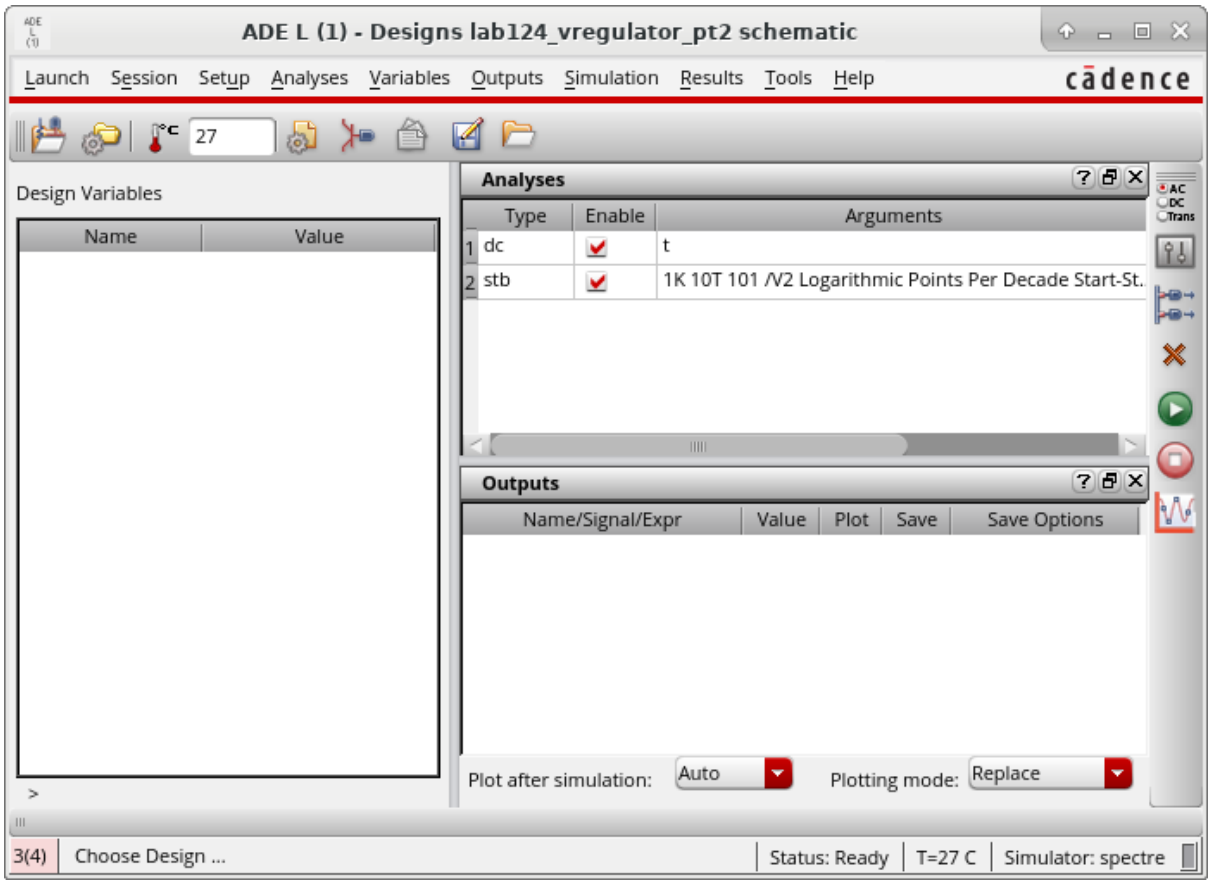
< Figure 107. The Choosing Analyses Form for a Stability Analysis. >

(5.3.4) From the ADE window, setup a stability analysis (**Menu: Analyses à Choose**). In this simulation we will again sweep the frequency from 1KHz to 10THz using a logarithmic sweep of the frequency – with 101 plot points per decade of frequency. At this stage, we will not select any outputs for plotting (we will use the Direct Plot Form later).

In the choosing analyses form, select the '**stb**' option button. Select the 'Frequency' option as the sweep variable. Choose the 'Start-Stop' option button with Start = 1K and Stop = 10T for the sweep range. Select 'Logarithmic' from the drop-down menu as the sweep type. Choose the 'Points Per Decade' option button with 101 plot points per decade.

At the bottom of the form, there are two fields: 'Probe Instance/Terminal' and 'Local Ground Name'. The probe instance/terminal is used to identify the feedback loop – we need to point this to the null vdc component we inserted into the feedback path. Press the select button next to the 'Probe Instance/Terminal' form field. The schematic should come into focus. Click on the null vdc component. The name of the null vdc part (/V2) should now appear in the form field. We also need to do the same with the local ground name. Press the select button next to the 'Local Ground Name' form field. The schematic should come into focus. Click on any net connected to a ground (gnd) reference. The name of the ground net (/gnd!) should now appear in the form field.

Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. OK the form. The **stb** analysis should appear in the 'Analyses' list of the ADE window next to the **dc** analysis already set up. See Figure 27.

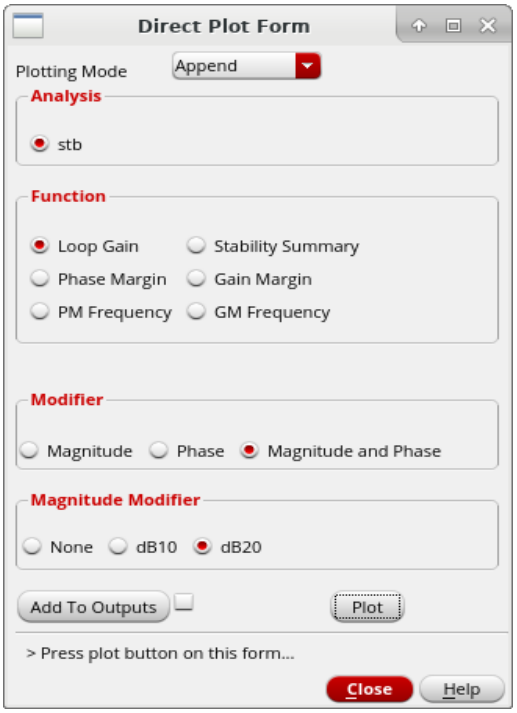


< Figure 108. The ADE with the DC OP Analysis and STB Analyses set up. >

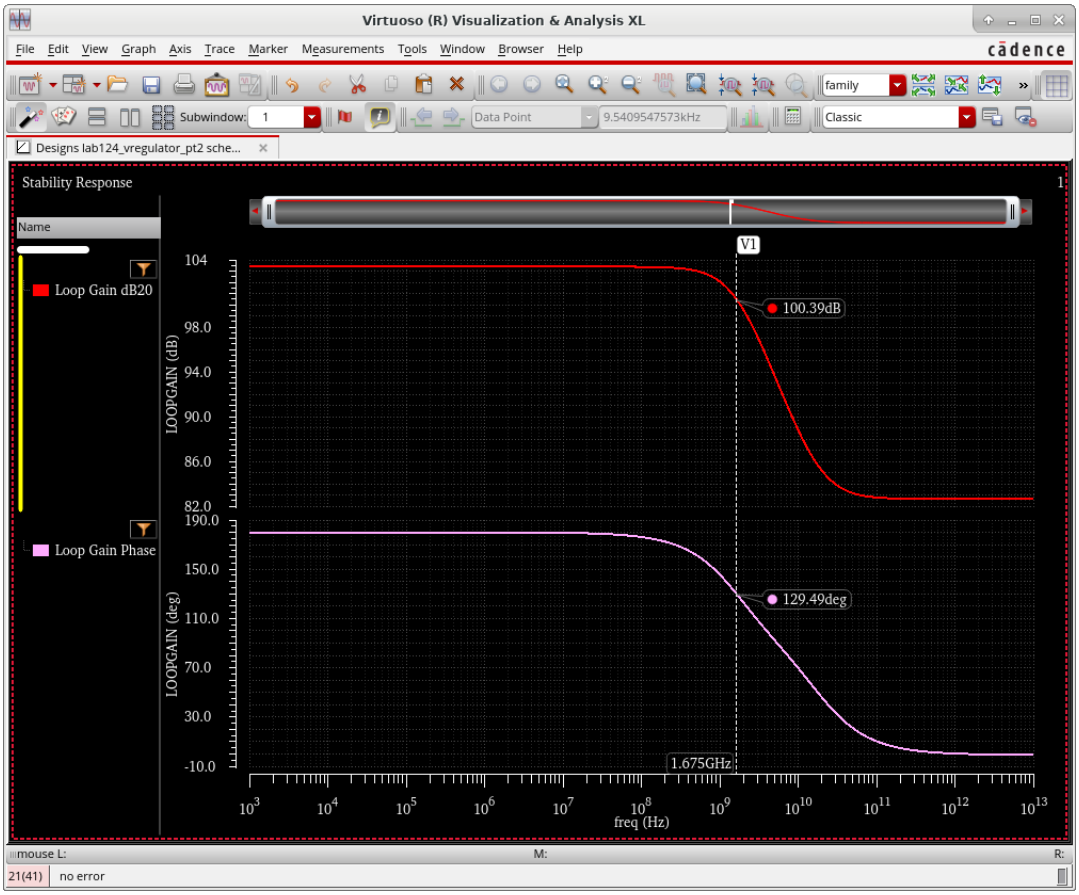
(5.3.5) In the ADE window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. Only the text output window should open automatically since we didn't select an output for plotting.

(5.3.6) From the ADE window, open the Direct Plot form (**Menu: Results → Direct Plot → Main Form**). Only the **stb** analysis will show up on the Direct Plot Form since the **dc** operating point analysis doesn't have any plotting options.

Make sure the **stb** option button is selected in the Analysis section of the Direct Plot Form. Select the **Loop Gain** option button from the function section. Select the **Magnitude and Phase** option in the modifier section of the direct plot form. In the magnitude modifier section, select the **dB20** option. Click the 'Plot' button near the bottom right corner of the form. You should now have both a plot of loop gain in dB20 vs. frequency and a plot of loop gain phase (degrees) vs. frequency plotted in the waveform window. See Figures 28 and 29.



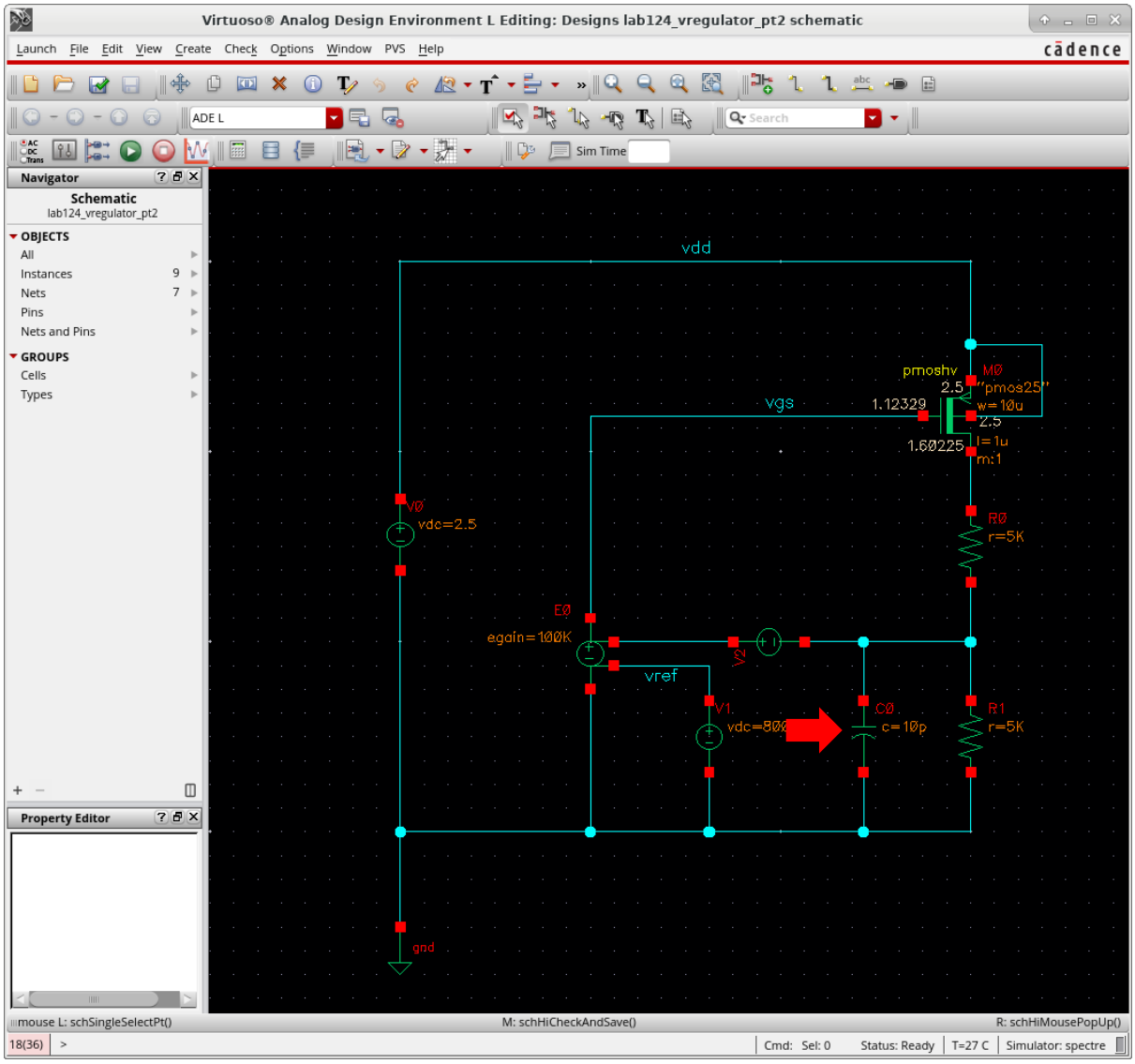
< Figure 109. The Direct Plot Form for the Stability Analysis>



< Figure 110. Loop Gain Magnitude and Phase Plot of the Controllable Voltage Reference Circuit. The DC gain is ~103.39dB. With Giving f_{-3dB}≈1.675GHz >

(5.3.7) Split the traces for clarity. Right-click on the empty area of the graph window and select **Split Current Strip à trace** from the context menu.

(5.3.8) **Disscussion**. We see from the stability analysis that the voltage reference is unstable. The feedback signal undergoes an almost 180° phase shift, with the gain remaining very high throughout. Clearly, at the higher frequencies the feedback becomes (almost) positive. Techincally, the system is marginally stable since the phase shift doesn't quite reach 180° – but for all practical perposes it would be considered unstable. As we have not introduced any recative components into the circuit, this instability occurs entirely due to the parasitic recatances modelled in the PMOS device – of which there is more than one.

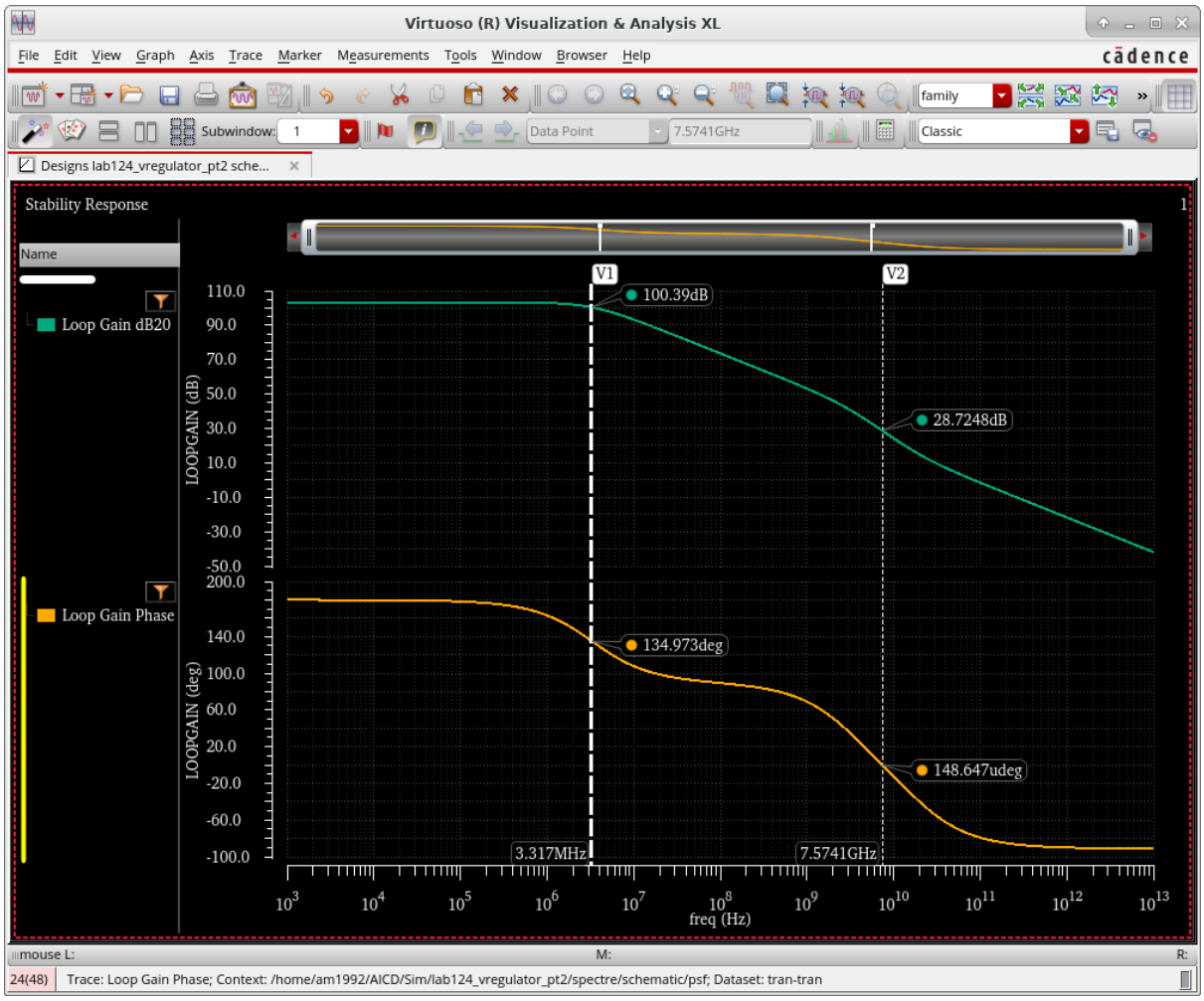


< Figure 111. Annotated controllable voltage reference circuit with null VDC and 10pF compensation capacitor in the feedback path to improve stability. >

(5.3.9) Recall, that to ensure stability we want the loop gain to fall below 0dB before the phase shift reaches 180°. We can either (1) lower the gain of the VCVS feedback amplifier (though, this would adversely affect the DC tracking accuracy of the reference), or (2) insert a dominant pole into the feedback path to bring down the gain as the frequency increases. The capacitor will form a pole with the two resistors of the potential divider and lead to a roll-off in the gain. If the gain can be brought down to below 0dB before the loop gain phase shifts 180° we would have stabilised the system.

(5.3.10) To improve the stability we will first insert a capacitor to ground in the feedback path. From the analogLib instantiate a capacitor (**cap**). Set the capacitance to 10pF (**Capacitance** = 10p). Place it into the feedback path as in Figure 30.

(5.3.11) Re-simulate the stability analysis. Again, use the Direct Plot form to plot the loop gain (dB20) vs. frequency and loop gain phase (degrees) vs. frequency. See Figure 31.

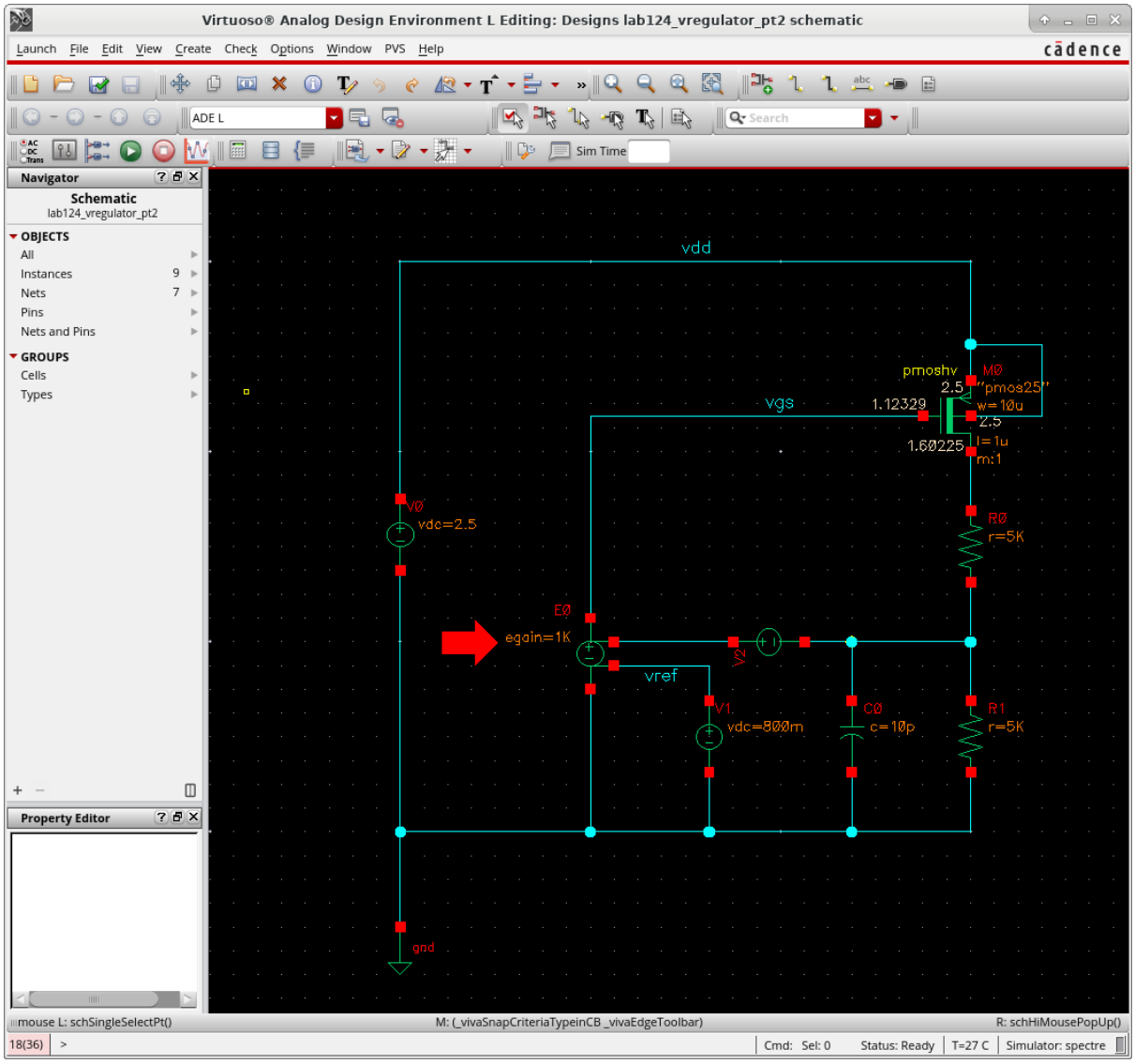


< Figure 112. < Figure 113. Loop Gain Magnitude and Phase Plot of the Controllable Voltage Reference Circuit with 10pF Compensation Capacitor. The DC gain is ~103.39dB. With Giving f_{-3dB}≈3.317MHz. Still unstable. >

(5.3.12) **Discussion**. Even with the 10pF capacitor in the feedback path the voltage reference is still unstable. We have a loop gain of over 28dB as the loop gain phase-shift reaches 180º. This is an improvement compared to the no-capacitator version (82dB, see Figure 29). Since capacitors take up a lot of space, increasing the capacitance in the feedback path isn't usually the preferred option (5pF to 10pF is an upper limit of what is acceptable). Instead, we will lower the gain of the VCVS. This is a trade-off in tracking accuracy vs. stability.

(5.3.10) Modify the VCVS voltage gain from 100K to 1K. *You can use **Menu: Edit → Properties → Objects**, the '**Edit Properties**' icon-button from the toolbar, or the keyboard shortcut '**Q**'. You can then modify the required attribute(s).*

(5.3.11) Re-simulate the stability analysis. Again, use the Direct Plot form to plot the loop gain (dB20) vs. frequency and loop gain phase (degrees) vs. frequency. See Figure 34.

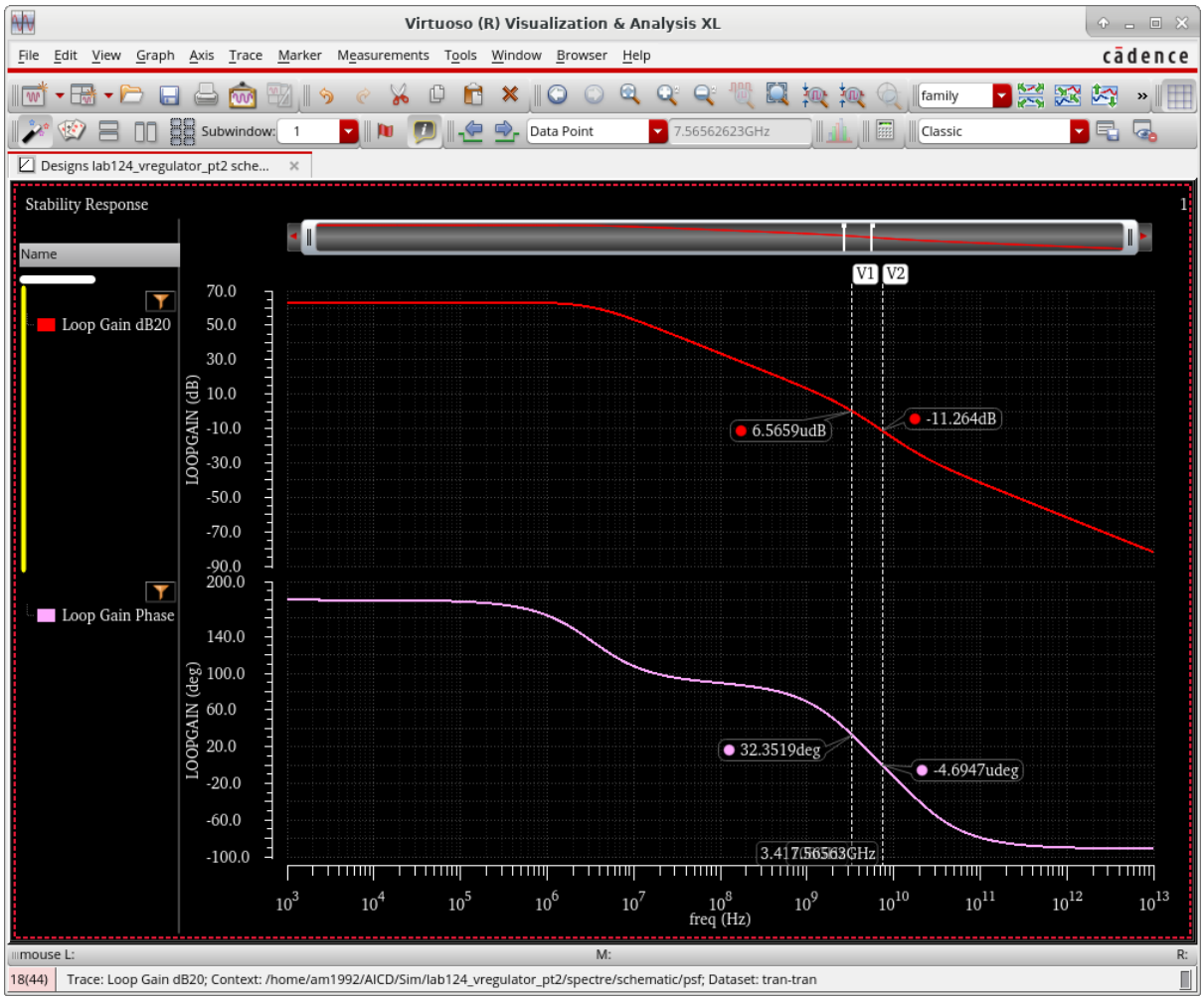


< Figure 114. Annotated controllable voltage reference circuit with null VDC. Includes 10pF compensation capacitor in the feedback path and a lowered VCVS gain (from 100K to 1K) to improve stability. >

(5.3.12) **Discussion**. With the lower VCVS gain (1K) and the 10pF capacitor in the feedback path, the voltage reference is now stable, at a cost of a slightly lower tracking accuracy. Note the new DC operating point of the voltage reference – previously 1.6002V with the VCVS=100K, now 1.602V with the VCVS=1K. This would be considered, in most cases, an acceptable trade-off (but this would really depend on the required specification of the reference.)

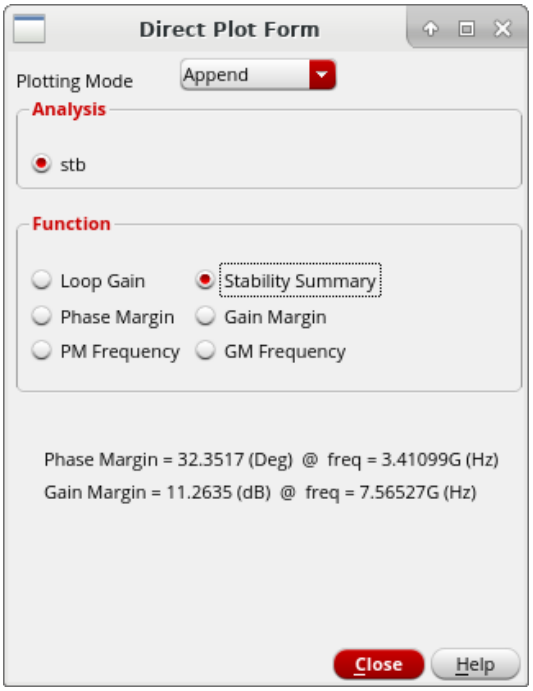
Note that two vertical markers have been placed on the magnitude and phase plots of the loop gain. When the gain falls to 0dB (6.57udB) note that the phase has shifted from 180° to 32.3519°. 32.3519° is therefore the phase margin. When the phase has shifted fully to 180° the gain has fallen to -11.264dB. 11.264dB is therefore the gain margin.

Whilst it is now stable, the phase margin is only 32°. As a rule of thumb, we would aim for a phase margin of 60° to 70° – this provides a good performance trade-off between system responsiveness (in the time domain) and stability in the face of PVT, parameter drift, and so on. (Appendix C in [2] offers a good development of the relationship between phase margin and time domain response).



< Figure 115. Loop Gain Magnitude and Phase Plot of the Controllable Voltage Reference Circuit with 10pF Compensation Capacitor and Lowered VCVS Gain (1K). Stable, with a Phase margin = 32° and a Gain Margin of 11dB. >

(5.3.13) Finally, on the direct plot form select **Stability Summary** from the function section. Note that the gain and phase margin (at their recpective frequncies) are displayed.



< Figure 116. The Direct Plot Form Displaying the Stability Summary. >

End of Lab!

Tutorial Lab 12.5 (Formatively Assessed):

Cascode Stages, Mirrors, and Symbol View Generation

Preliminary Matter: Setting up the Cadence Environment

Please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout before entering these commands into your terminal.

For this course, you will use the Cadence Virtuoso Design Environment (referred to simply as Cadence). Cadence is a Linux based software suite for designing full custom integrated circuits. In order to use Cadence you will first need to set-up the Cadence environment.

1. Open a terminal window and ensure that you are in your home directory using the following command:

2.

```
cd ~
```

Note: 'cd' stands for change directory. The solitary tilde symbol (~) is a shortcut/alias for your home directory (/home/username).

3. Under your home directory, use the following commands to load the Cadence environment:

```
module use /eda/cadence/modules

module load course/EENGM6030
```

Note: The two 'module' commands load the required shell/Cadence environmental variables for this course (paths, names, aliases…) In particular, this includes a reference to the process design kit (PDK) used in this course, the GPDK180. You will need to do this every time you open a terminal, since the sourced environment only applies to the current terminal (until it's closed).

a. ***Note: This step only needs to be completed ONCE - If you already ran this script in lab 12.1, you don't need to run it again here.*** The first time you use Cadence, you will need to run the following command (script):

```
EENGM6030_setup
```

This will create a folder called AICD (short for Analogue Integrated Circuit Design) under your home directory. The AICD folder contains the files and directories needed to run Cadence. Running this setup script more than once will result in a warning message that the AICD folder already exists. It will then exit without making changes.

4. The AICD folder will be used for all your Cadence related activities for this course. **You should always make sure you are in your AICD folder before you launch Cadence.** To run cadence, use the following commands:

```
cd ~/AICD

virtuoso &
```

1. Learning Objectives of this Lab:

From the previous lab, you should:

- Understand the importance of feedback for control and compensation in electronic circuits and the potential for instability within such circuits. Understand how the RC products (delays) in the forward and feedback paths of a circuit give rise to poles and zeros and how these affect the frequency response of the feedback circuit.

- Know how to use the SPICE/Spectre method for stability analysis (**stb**) in feedback circuits. Understand the role of Loop Gain Magnitude and Loop Gain Phase in stability analysis. Understand the Phase Margin and Gain Margin stability metrics and how to obtain them through simulation.

- Use feedback to provide voltage regulation.

In this lab, you will:

- Understand the concept of channel length modulation.

- Understand the effect of channel length on MOSFET output impedance.

- Understand the concept of cascoding to increase MOSFET output impedance.

- Simulate a telescopic cascode amplifier circuit and compare its performance with that of a non-cascode equivalent. Investigate cascoding in a FET current mirror.

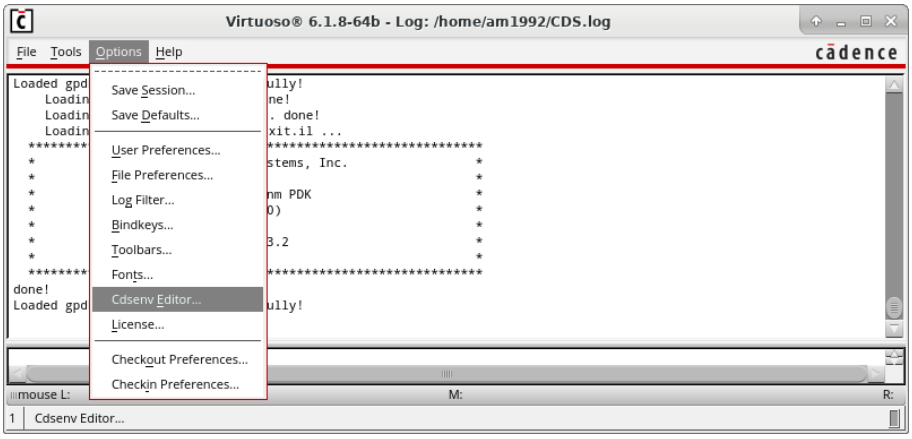
- Construct, instantiate and use in a simulation, a symbol view for a schematic circuit.

2. Cadence Documentation, Help, and Licensing

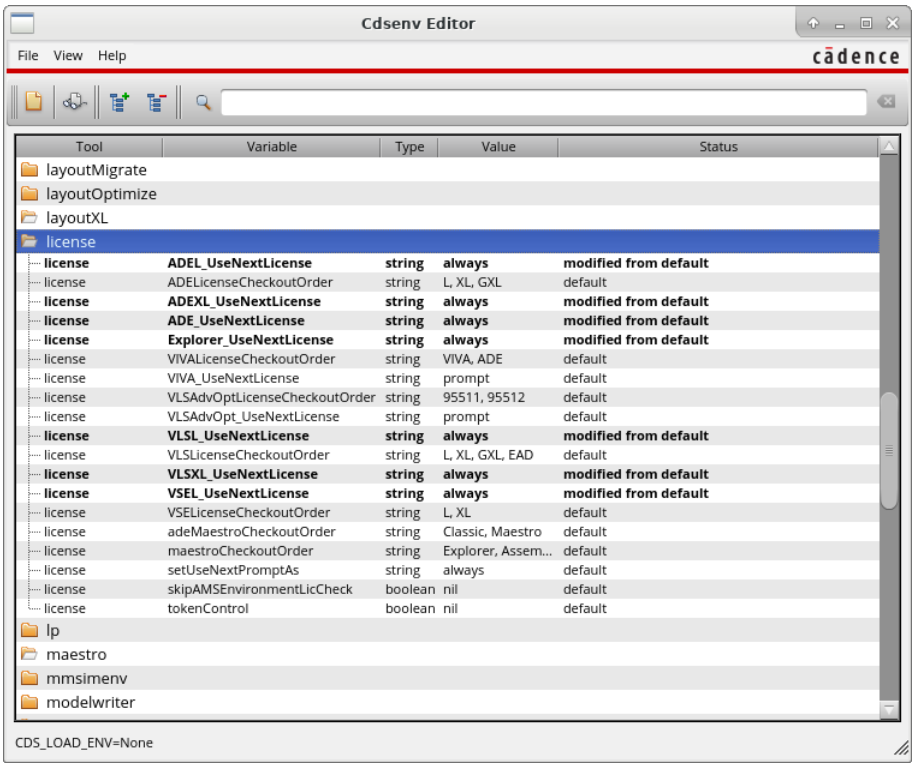
Local and online help is available from the help menus (**Menu: Help → User Guide** or by pressing **F1**), or by typing 'cdnshelp &' at the command prompt (assuming you have setup your terminal environment).

PLEASE NOTE: WHEN LAUNCHING A TOOL IN THE CADENCE DESIGN ENVIRONMENT YOU MAY SOMETIMES GET A WARNING ABOUT A LICENSE BEING UNAVAILABLE FOR THE TOOL, AND REQUESTING PERMISSION TO OBTAIN A HIGHER-TIERED LICENSE. YOU SHOULD SELECT 'ALWAYS' TO THESE MESSAGES.

If you accidently deny a license request, or you cannot access a license, you will need to modify the Cadence environment licence/permissions from the CIW. From the CIW menu go to **Menu: Options → Cdsenv Editor**. Once in the environment editor, go to the license folder, and make sure all licenses (in bold) are set to '**always'** – if any entries are listed as '**never**' you will need to manually change/retype them to '**always**'.



< Figure X.1. The Cadence Environment Menu Option from the CIW Window. >



< Figure X.2. The Cadence Environment Editor. Note, all licenses should be set to 'always'.

3. Getting Started with Cadence:

For a refresher/introduction on how to start a Cadence session please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout.

3.1 Dealing with Lock files

PLEASE NOTE: IF A SCHEMATIC ISN'T PROPERLY CLOSED DOWN (FOR EXAMPLE, DUE TO A DROPPED REMOTE CONNECTION), THE SCHEMATIC MIGHT BECOME EDIT LOCKED. IF THIS HAPPENS YOU WILL NOT BE ABLE TO OPEN THE FILE FOR EDITING. YOU WILL ONLY BE ABLE TO OPEN THE FILE FOR READING UNTIL YOU REMOVE ANY/ALL LOCK FILES.

There are two ways you can unlock files:

(1) The first is to manually delete any edit lock files. To do this, go to your '~/AICD/Designs' folder and find the folder representing the schematic e.g., for lab1 this would be '~/AICD/Designs/lab121_vdivider/schematic' if this is what you called your schematic. Delete all the .cdslck lock files in this folder and you should be able to open your schematic for editing.

(2)

(3) You can also automatically release edit locks with the command:

```
clsAdminTool -are .
```

To use this command, you have to be in your AICD folder (the full-stop is shorthand for the current directory path). If you are not in your AICD folder, you have to specify the path (relative or absolute) to your AICD directory.

Since this is a Cadence command, you have to ensure that you have set up the terminal environment for Cadence (in particular, the modules must be loaded as in step 2, 'Preliminary Matter'). It is best to use the terminal window that you used to start Cadence, since the environment was already set up. Note that you will not be able enter any commands into your terminal if you did not start cadence with the '&' character. By example, to release any lock files after opening a new terminal, the full command set would be,

```
cd~
module use /eda/cadence/modules
module load course/EENGM6030_2021
clsAdminTool -are AICD
```

For more information on this command, use

```
clsAdminTool -help
```

4. Circuit Simulation Exercises: Part 1

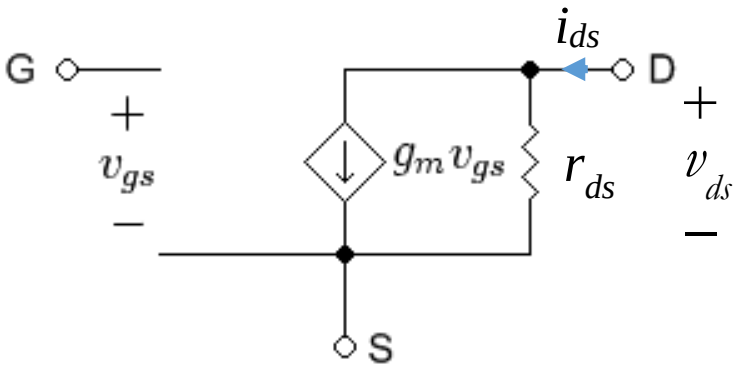
With guidance, you will:

- Understand the concept of channel length modulation. Simulate the dependency of the MOSFET output impedance on channel length.

- Understand the concept of cascoding. Simulate a telescopic cascode testbench circuit and compare its performance with that of a non-cascode equivalent.

The ideal MOSFET – modelled as a Voltage-Controlled Current-Source (VCCS) – has an infinite incremental output impedance. In this case, the drain (output) current I_{DS} is independent of the drain (output) voltage V_{DS}. As we have seen in the previous labs, the non-ideal MOSFET deviates from the VCCS model by a dependency of the output current on the output voltage. This non-ideal behaviour is modelled in the small-signal domain by a parasitic resistance r_{ds} (aka r_o) between the output terminals of the VCCS, where $r_{ds} = dV_{DS}/dI_{DS}$. At the heart of this behaviour is the concept of channel length modulation, where the effective length of the channel varies with drain-to-source voltage due to the electrical field moving the channel pinch-off point away from the drain. (We will discuss channel length modulation later in this tutorial, but for a summary of the phenomena refer to section 1.2.5 of the background material and section 4.2.3 in reference [3]).

In the previous labs, we have characterised – through simulation – the V_{DS}-I_{DS} large-signal terminal behaviour of an NMOS device from the GPDK180 library, and from this, plotted r_{ds} as a function of the DC operating point. Consider, the simplest small-signal model of the MOSFET



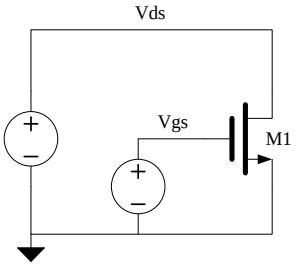
< Figure 117. Voltage controlled current source with (non-ideal) output impedance r_{ds}. >

Ultimately, the presence of r_{ds} will lower the voltage gain of the MOSFET amplifier for a given transconductance as it appears in parallel with the infinite output impedance of the ideal MOSFET (and also load device). In this lab, we will focus on mitigating the dependency of I_{DS} on V_{DS} in order to increase the output impedance of the MOSFET – and ultimately the gain of the MOSFET voltage amplifier. We will introduce the concept of channel length modulation and observe – through simulation – how the choice of channel length effects the output impedance of the MOSFET. We will also introduce the circuit technique of amplifier cascoding

as a way of increasing the MOSFET output impedance, and how this technique avoids the adverse consequences of selecting a longer channel length.

4.1 NMOS Terminal Characteristics

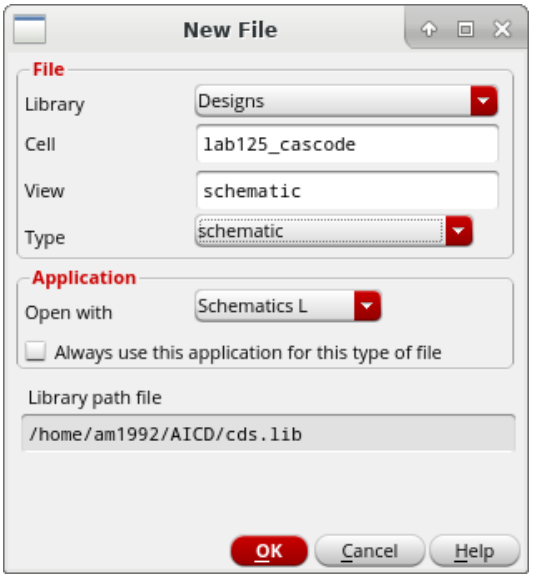
(4.1.1) Create the schematic for the NMOS testbench circuit in Figure 2. A summary of the circuit's operation is given in section 4.1 of the background material. Note: You will have created this circuit in labs 12.2 and 12.3. You can re-use your circuit, copy it, or build it from scratch. We recommend that you create a new schematic since this is a new lab (you should be able to do this quickly).



< Figure 118. Circuit for Large Signal Transconductance Terminal Characteristics Evaluation.
Reproduced from Figure 14, section 4.1 of the background material. >

(4.1.2) From the Library Manager window, select the 'Designs' library and choose **Menu: Files à New à Cell View** to invoke the New File form (Figure 3).

In the 'Cell' text field, choose a meaningful cell name (e.g. lab125_cascode). In the 'View' text field, type 'schematic' as the view name and from the 'Type' drop down menu choose 'schematic' as the view type. In the 'Application' section of the dialog box, choose 'Schematics L' from the 'Open with' drop down menu. OK the form.



< Figure 119. The New File Form. >

(4.1.3) You should be presented with a blank **Virtuoso Schematic Editor L** schematic entry window for the new cellview. This cellview will not be saved in the Library Manager until you save it in the Schematic Editor. From the schematic editor window, use either **Menu: File à Save** or **Menu: File à Check and Save**.

(4.1.4) In your schematic editor window, construct the circuit given in Figure 2. Instantiate and place two DC voltage sources (**vdc**) from the analogLib library, a ground connection (**gnd**) from the analogLib library, and one high-voltage NMOS FET (**nmoshv**) from the GPDK180 library.

(4.1.5) Parameterise your design. For the DC supply voltage, set the voltage attribute to the string vds (**DC Voltage** = *vds*) and for the gate-to-source voltage set the voltage attribute to the string vgs (**DC Voltage** = *vgs*), and dimension the NMOS device 10um/1um (**width** = *10u*, **length** = 1u). Leave all other attributes unchanged.

(4.1.6) With the components placed, you can now add wires to connect them together. You can use **Menu: Create → Wire (narrow)**, the '**Create Narrow Wire**' icon-button on the toolbar, or the keyboard shortcut '**W**' to start the wire command (see Figure 5).

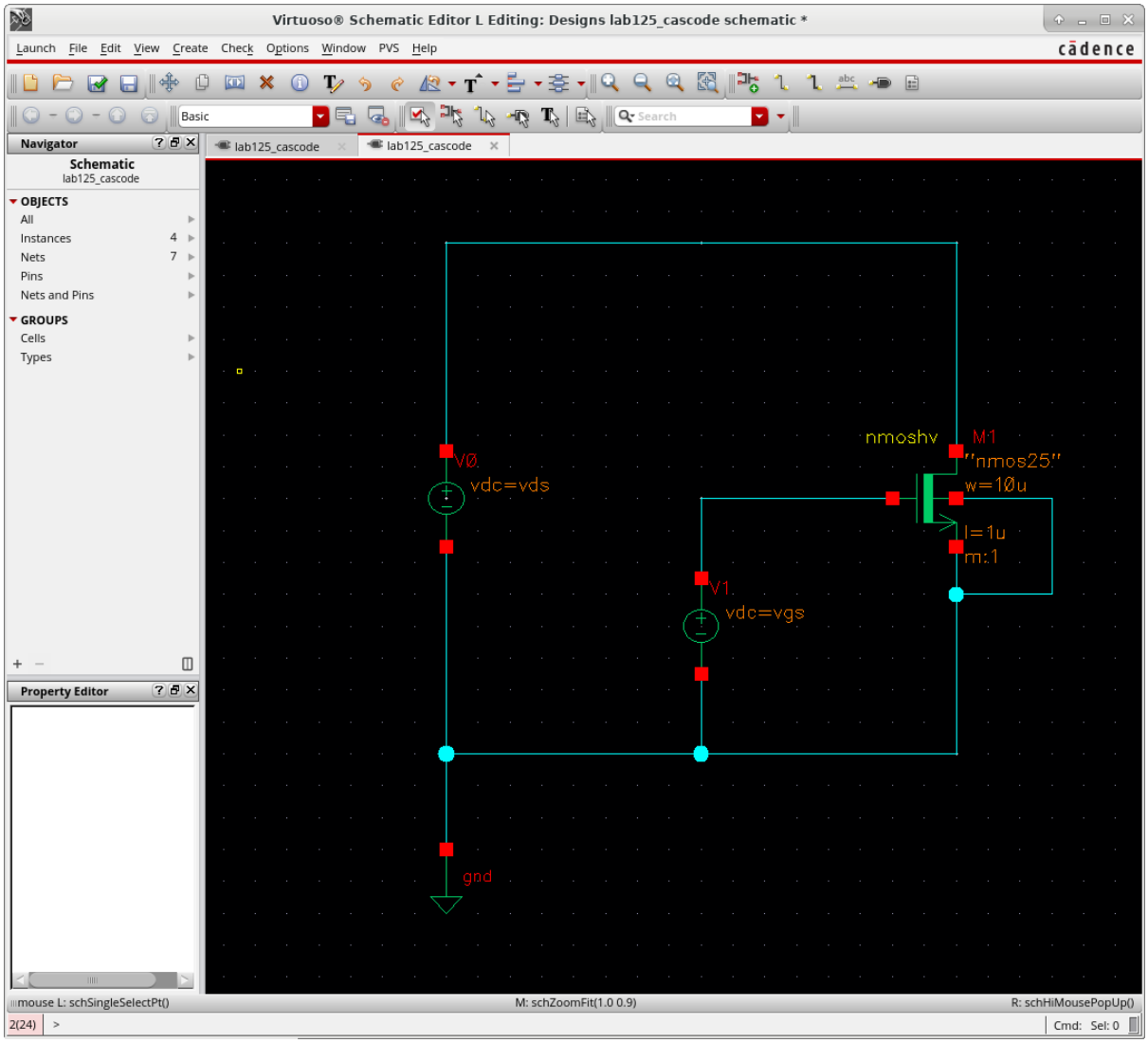
(4.1.7) Check and save your schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.
*Recall, to add a new part you can use **Menu: Create → Instance...**, the '**Create Instance**' icon-button on the toolbar at the top of the schematic window, or the keyboard shortcut '**I**'. You can also right-click anywhere in the empty space of the design canvas to invoke a context-menu and select the **Add Instance…** command from there. Refer to lab 12-1 for a refresher on how to add components to the schematic (sec. 4.2 Adding Components to the Schematic).*

*You can set the component properties at instantiation from the Add Instance form before placing them on the design canvas. Alternatively, you can modify the component properties after it has been placed on the design canvas, by first selecting the item and then by right-clicking on the item and selecting 'Properties…' You can also use **Menu: Edit → Properties → Objects**, the '**Edit Properties**' icon-button from the toolbar, or the keyboard shortcut '**Q**'. You can then modify the required attribute(s) and Apply/OK the change.*

*You can drag, move or stretch the components and wires after they have been placed on the design canvas. Left-click-and-hold the mouse button and drag the component or wire, or select the component and use the **stretch** or **move** commands. You can also rotate and flip the components before placing them using the **rotate**, **sideways**, and **upside-down** buttons on the add instance form. Or you can rotate/flip the components after placing them by selecting the component and using the **rotate** or **flip** commands, or by using **F3** with the **move/stretch** commands.*



< Figure 120. The nmoshv (GPDK180) and vdc (analogLib) Add Instance Forms. >



< Figure 121. The wired NMOS testbench circuit with Design Variables for the DC supply voltage (vdc=vds) and the DC gate-to-source voltage (vdc=vgs). nmoshv=10um/1um. >

(4.1.8) RECALL: FOR AN NMOS DEVICE, YOU MUST ALWAYS CONNECT THE BODY/BULK TERMINAL TO THE MOST NEGATIVE NET (WIRE) OF THE CIRCUIT. IN FIGURE 5, THIS IS THE GROUND TERMINAL. FOR A PMOS DEVICE (NOT SHOWN) THE BODY/BULK TERMINAL MUST ALWAYS BE CONNECTED TO THE MOST POSITIVE NET (WIRE) OF THE CIRCUIT, USUALLY THE SUPPLY VDD. THIS IS NECESSARY TO ENSURE THAT THE CHANNEL WILL BE PROPERLY ESTABLISHED.

4.2 Simulating the NMOS FET DC operating Point

(4.2.1) You should now have the completed schematic of the NMOS testbench circuit as shown in Figure 5. It should check and save without any errors or warnings.

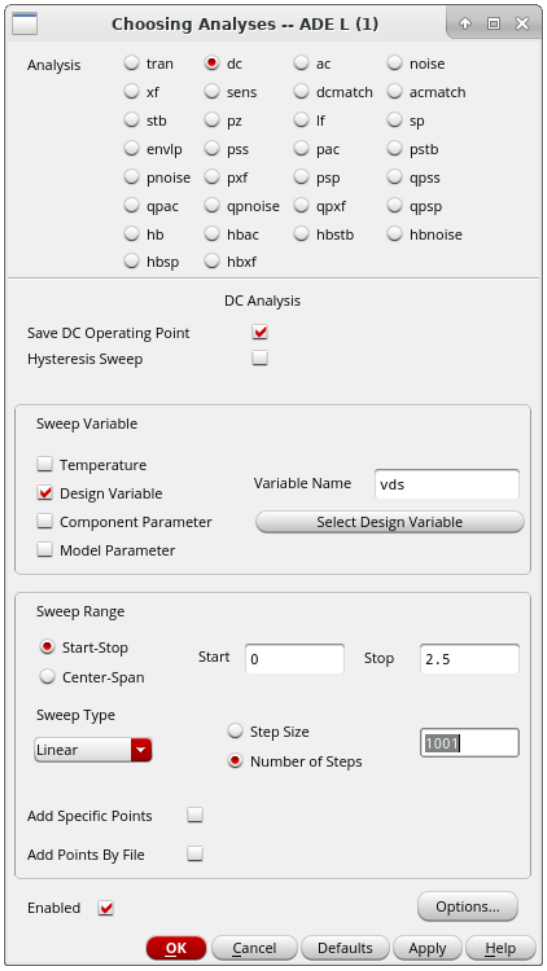
(4.2.2) We will simulate the NMOS testbench using a DC Sweep Analysis. From the Schematic Editor window, launch the Analogue Design Environment (ADE) – **Menu: Launch à ADE L**.

(4.2.3) Copy the design variables from the schematic into the ADE. For this circuit, these are the DC voltages from the two **vdc** components - *vds* and *vgs*. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the *vds* and *vgs* variables in the 'Design Variables' list of the ADE. Initialise *vds = 2.5V* and *vgs = 0.7V*. These values are used only to calculate the DC operating point of the amplifier circuit.

(4.2.4) We want to plot the I_{DS} vs. V_{DS} family of curves for increasing values of V_{GS} (that is I_{DS} vs. V_{DS} vs. V_{GS}). This requires two sweeps – for V_{DS} from 0V to 2.5V and for V_{GS} from 0V to 2.5V. The DC analysis form only allows you to sweep one variable at a time, so we will also need to use the Parametric Analysis tool as we did in lab 12.2.

(4.2.5) From the ADE window, go to **Menu: Analyses à Choose…** or use the '**Choose Analyses**' button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. From the form, select the '**dc**' option button. Check the 'Save DC Operating Point'. We wish to sweep the input voltage (vds) of the NMOS device from 0V to 2.5V while observing the drain current (IDS). We will use 1001 steps in the sweep to give the output waveform sufficient resolution.

In the choosing analyses form. Check the 'Save DC Operating Point' box. Check the 'Design Variable' box as the sweep variable. Type *vds* as the variable name. Choose the 'Start-Stop' option button with Start = 0 and Stop = 2.5 for the sweep range. Select 'Linear' from the drop down menu as the sweep type. Choose the 'Number of Steps' option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. See Figure 6. OK the form. The **dc** analysis should now appear in the 'Analyses' list of the ADE window.



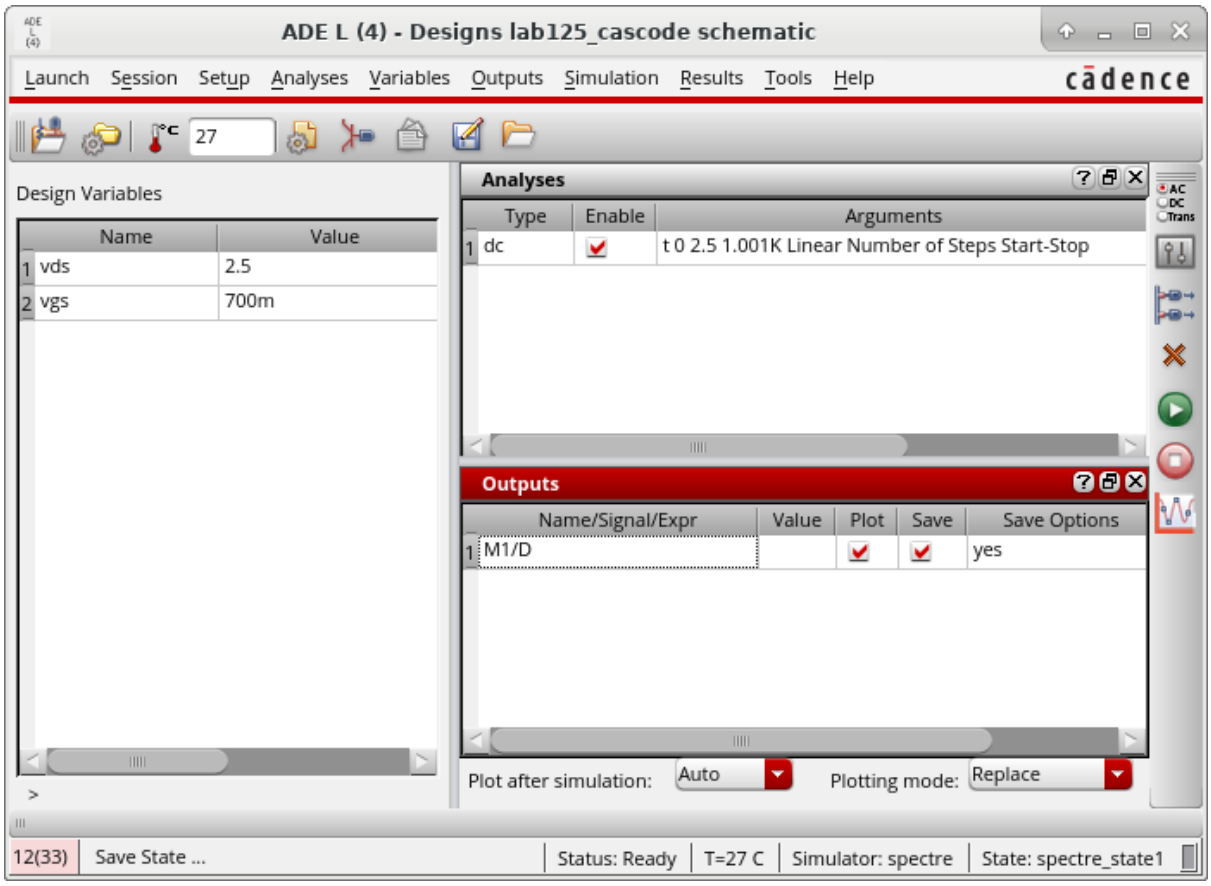
< Figure 122. The Choosing Analyses Form. >

(4.2.7) Select the drain node current (IDS) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** The schematic window will automatically become current. Select only the drain node of the NMOS device. **Press the 'ESC' key to exit the plotting mode of the ADE**.

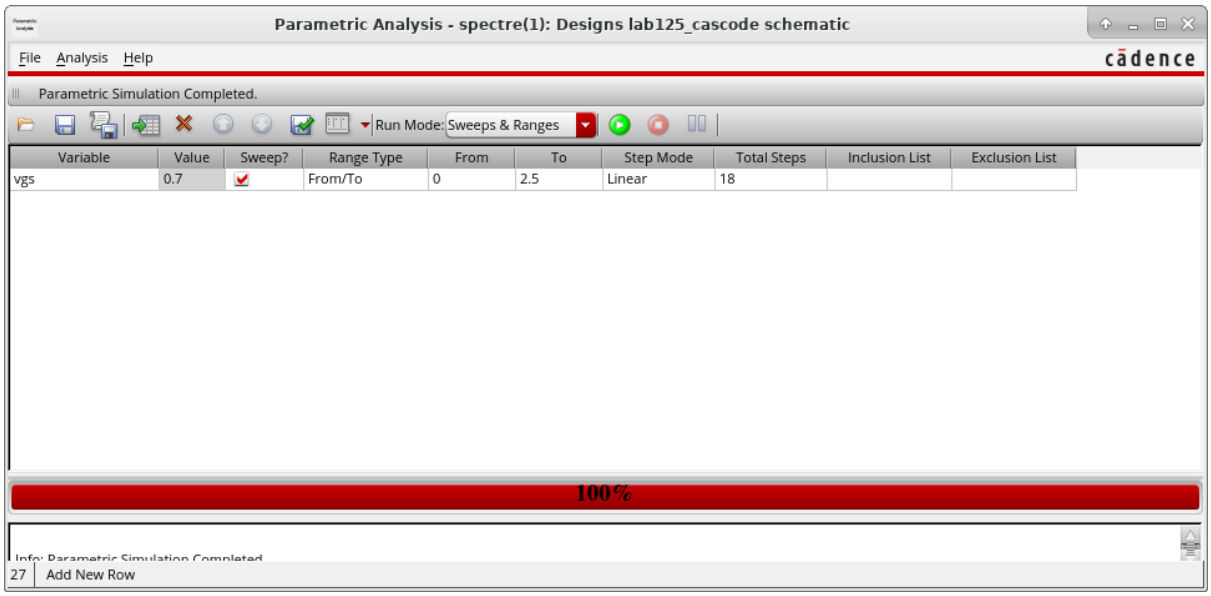
*Recall**,** if you want to plot a particular voltage, you click on the net/wire. If you want to save and plot the currents, you have to click a terminal **node** of a component (not the net/wire).*

(4.2.6) To sweep *vgs* we will use the parametric analysis tool. From the ADE window, choose **Menu: Tools → Parametric Analysis…** to invoke the Parametric Analysis tool. See Figure 8. In the Parametric Analysis window, select Sweeps and Ranges from the '**Run Mode'** drop down menu in the tool bar. In the '**Variable'** column, you should get a drop-down menu of the available design variables. Select vgs as the parametric sweep variable. Note, only the design variables in the 'Design Variables' section of the ADE will be available in the drop-down list along with the system variable temp. So, make sure you have copied the design variables from the schematic into the ADE using the Menu: Variables → Copy From Cellview… command.

In the '**Range Type**' drop down menu select 'From/To'. In the 'From' column type 0 as the start value of the parametric sweep, and in the 'To' column type 2.5 as the stop value of the parametric sweep. In the '**Step Mode**' column select 'linear' and in the **Total Steps** type 18.



< Figure 123. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 124. Parametric Analysis Tool. Set up for a linear sweep of VGS → 0V to 2.5V in 18 steps. This will run the swept DC-analysis (dc) 18 times to create a family of curves. >

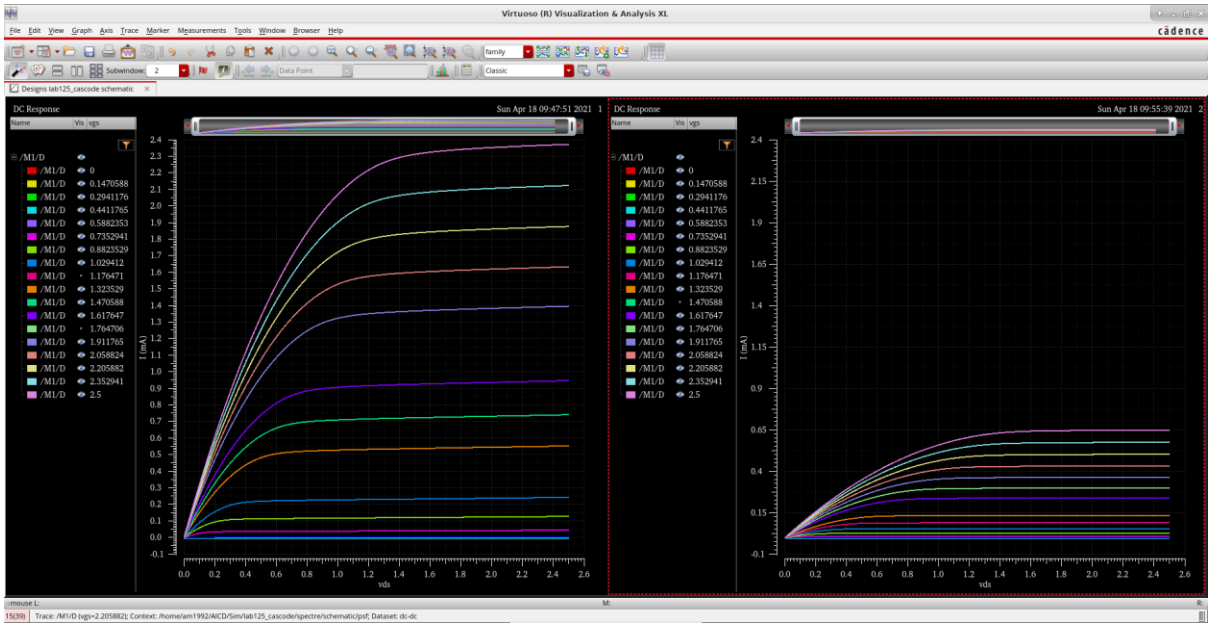
Note: The parametric tool works by repeating an analysis/simulation a specified number of times. While the parametric analysis tool allows us to specify additional variables to sweep, it is typically used to vary a parameter using far fewer sweep points than a DC-sweep, as it is much slower. The parametric analysis will run an analysis in the Analyses section of the ADE, once for each step of a parameter variation. A swept DC-analysis can efficiently handle sweeps of tens of thousands of sweep points. For the parametric tool, sweeps are best kept below 100 sweep points. A parametric sweep can be specified as a range to sweep or as an explicit list/set of values over which to iterate. (Run Mode: Sweeps and Ranges, or Parametric Set).

(4.2.8) The parametric analysis is run from the parametric analysis window not from the ADE window. In the Parametric Analysis window, run the '**Run Selected Sweeps**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. With the above settings, the ADE will perform a swept-DC analysis of V_{DS} from 0V to 2.5V, a total of 18 times – one simulation for each value of V_{GS} (V_{GS} will be linearly incremented from 0V to 2.5V with each increment of V_{GS} given by 0.134V (=2.5V/18) as specified in the parametric form). See Figure 9 (left, reproduced in Figure 10). **Leave the waveform output window open.**

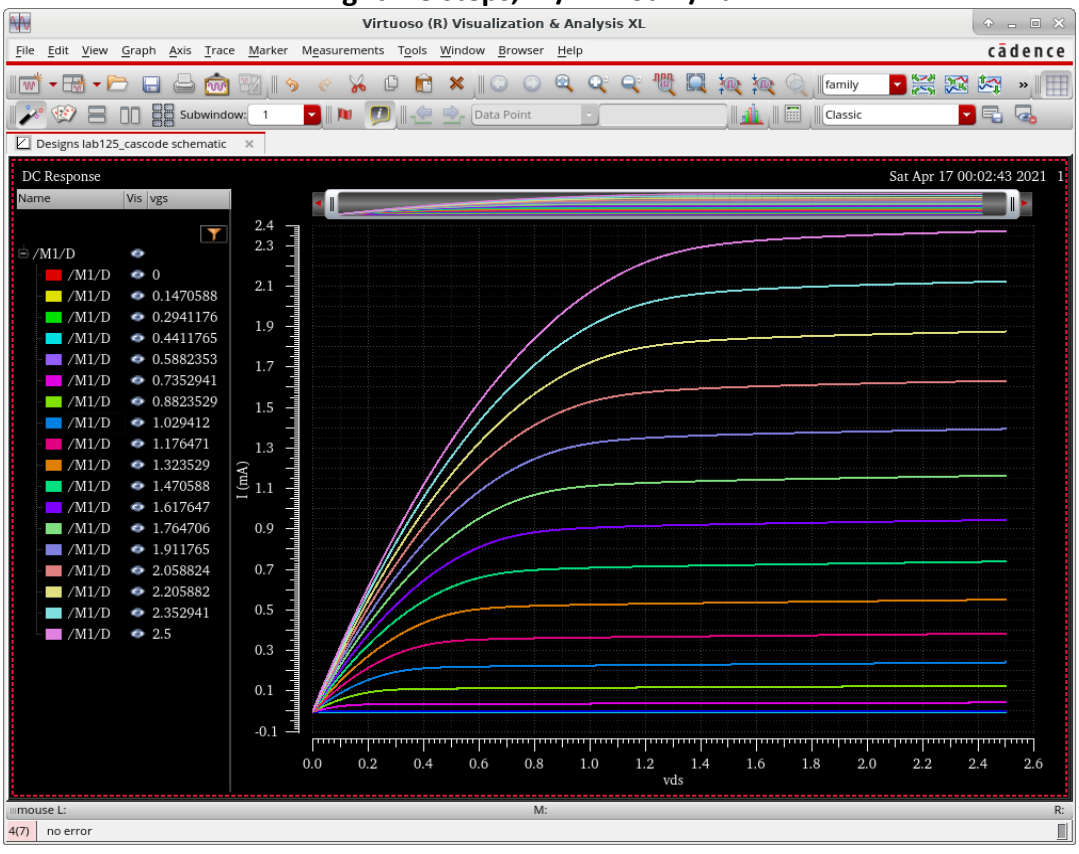
4.3 Increasing the Channel Length to Increase Output Impedance

(4.3.1) We will re-run the parametric simulation with the MOSFET channel length increased from 1um to 4um. Dimension the NMOS device 10u/4u (**width** = *10u*, **length** = 4u). Leave all other attributes unchanged.

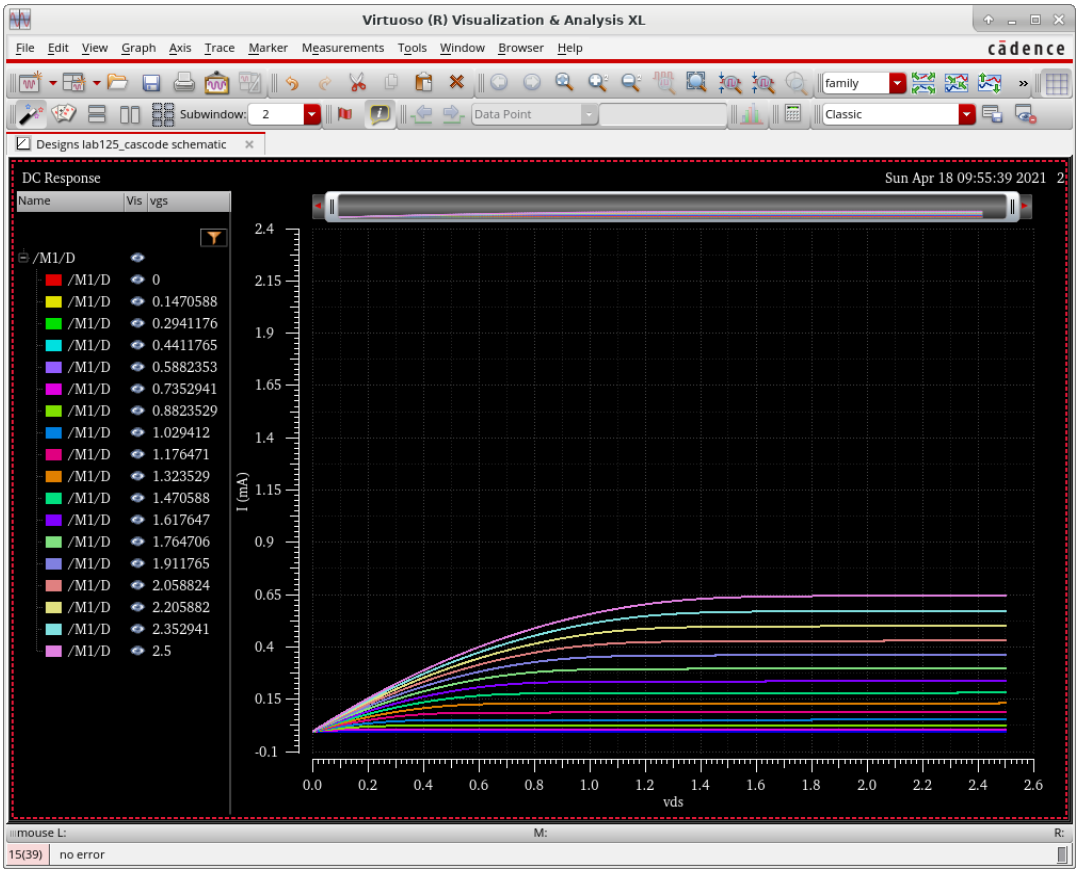
(4.3.2) Set the plotting mode of the ADE (lower right corner) to **New Sub-Window**. Re-run the parametric analysis; in the Parametric Analysis window, run the '**Run Selected Sweeps**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figure 9 (right, reproduced in Figure 11). These charts are on the same y-axis scale.



< Figure 125. Parametric DC-sweep of IDS vs. VDS vs. VGS.
Left: 18 steps, W/L = 10um/1um.
Right: 18 steps, W/L = 10um/4um.

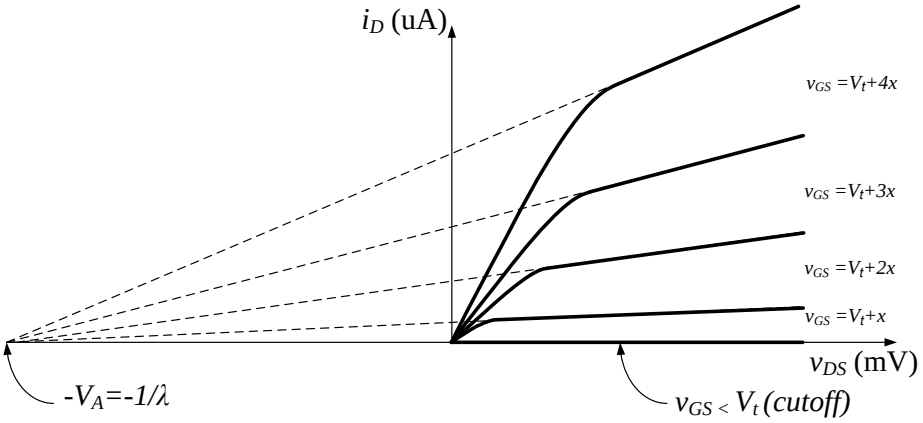


< Figure 126. Parametric DC-sweep of IDS vs. VDS vs. VGS. (18 steps, W/L = 10um/1um) >



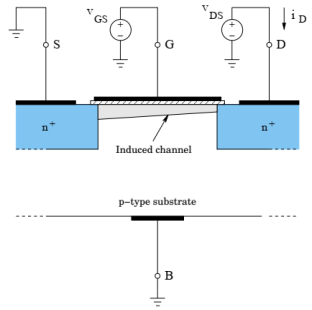
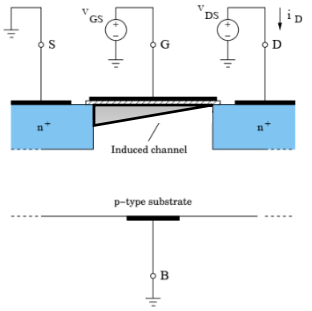
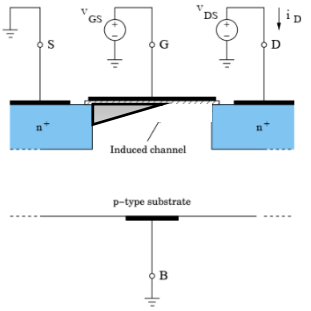
< Figure 127. Parametric DC-sweep of IDS vs. VDS vs. VGS. (18 steps, W/L = 10um/4um) >

(4.2.10) **Discussion**. From the slope of the large-signal I_{DS} vs. V_{DS} curves in the saturation region, we see that there is some dependency of I_{DS} on V_{DS} (as we simulated in labs 12.2 and 12.3). As we mentioned in this introduction, this arises due to **channel length modulation**.



< Dependency of I_{DS} on V_{DS} in the saturation region. >

In the triode region (1 in the Figure below), below pinch-off, the MOSFET channel is continuous from source-to-drain. As the MOSFET enters saturation (2 in the Figure below) where $V_{DS}(sat)=V_{GS}-V_T$, the MOSFET channel becomes pinched-off at the drain region. As the drain voltage V_{DS} increases beyond the pinch-off voltage (3 in the Figure below), the electric field pushes the channel pinch-off point away from the drain and down the channel, *effectively* making the drain-to-source channel length shorter.

1.	2.	3.
		
Below Pinch-Off	*At Pinch-Off*	*Above Pinch-Off*
$V_{DS} < V_{GS}-V_T$	$V_{DS}(sat)=V_{GS}-V_T$	$V_{DS} > V_{GS}-V_T$
$$i_{DS} = \frac{1}{2}k'\left(\frac{W}{L}\right)(V_{GS}-V_t)^2(1+\lambda V_{DS})$$ $$= \frac{1}{2}k'\left(\frac{W}{L}\right)(V_{GS}-V_t)^2\left(1+\frac{\Delta L}{L}\right)$$		

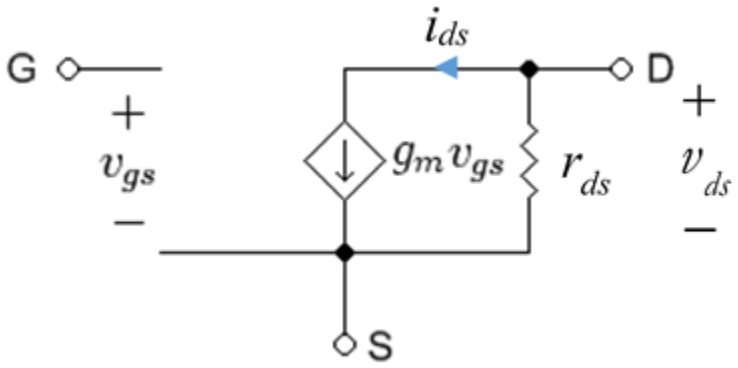
Since the drain current is inversely proportional to channel length L, I_{DS} increases with increasing V_{DS}. A variation of the drain voltage (in saturation) will therefore lead slight variation in drain current. Note that,

$$\lambda V_{DS} = \frac{\Delta L}{L}$$

where $\lambda = 1/(V_E L)$ is the channel length modulation parameter, a parameter that accounts for the change in I_{DS} with change in V_{DS}. This variation of I_{DS} with V_{DS} can be modelled as a resistance ***rds*** between the drain and source of the ideal MOSFET model. As we observed in lab 12.3, the effective resistance ***rds*** is given by,

$$r_{ds} = \left[\frac{\partial I_{DS}}{\partial V_{DS}}\right]^{-1} = \frac{V_A}{I_{DS}} = \frac{V_E L}{I_{DS}} = \frac{1}{I_{DS}\lambda}$$

The process parameter $\boldsymbol{V_E}$ is generally 5V/µmL for N-channel MOSFETs and 8V/µmL for P-channel MOSFETs. (For a summary of the phenomena refer to section 1.2.5 of the background material and section 4.2.3 in reference [3].)



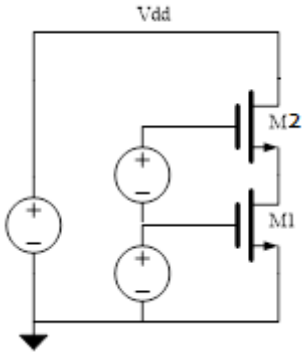
< Voltage controlled current source with (non-ideal) output impedance r_{ds}. >

r_{ds} lowers the voltage gain for a given transconductance as it appears in parallel with the load resistance (which is infinite for an ideal current-source load). We note from the above equation that r_{ds} is dependent on the channel length *L*. We can increase r_{ds} by increasing *L*. We saw this in the previous simulation when we repeated the parametric sweep with *L*=4um. The increased channel length makes the variation in effective length less sensitive as a function of overall length. However, we also see that by increasing L we also decrease the transconductance. To keep the transconductance broadly similar we therefore need to increase the width proportionally. This leads to a larger device with the accompanying trade-offs (namely, power and speed.)

4.3 Using a Cascode Device to Increase Output Impedance

(4.3.1) In this part of the lab, we will add a cascode stage to the NMOS testbench circuit in Figure 5. This is a commonly used circuit technique to mitigate the effects of channel length modulation and increase output impedance.

(4.3.2) Modify your existing testbench schematic by constructing a second testbench alongside your existing testbench circuit. The new circuit is the same as the previous one except for the addition of a cascode device. See Figure 12.



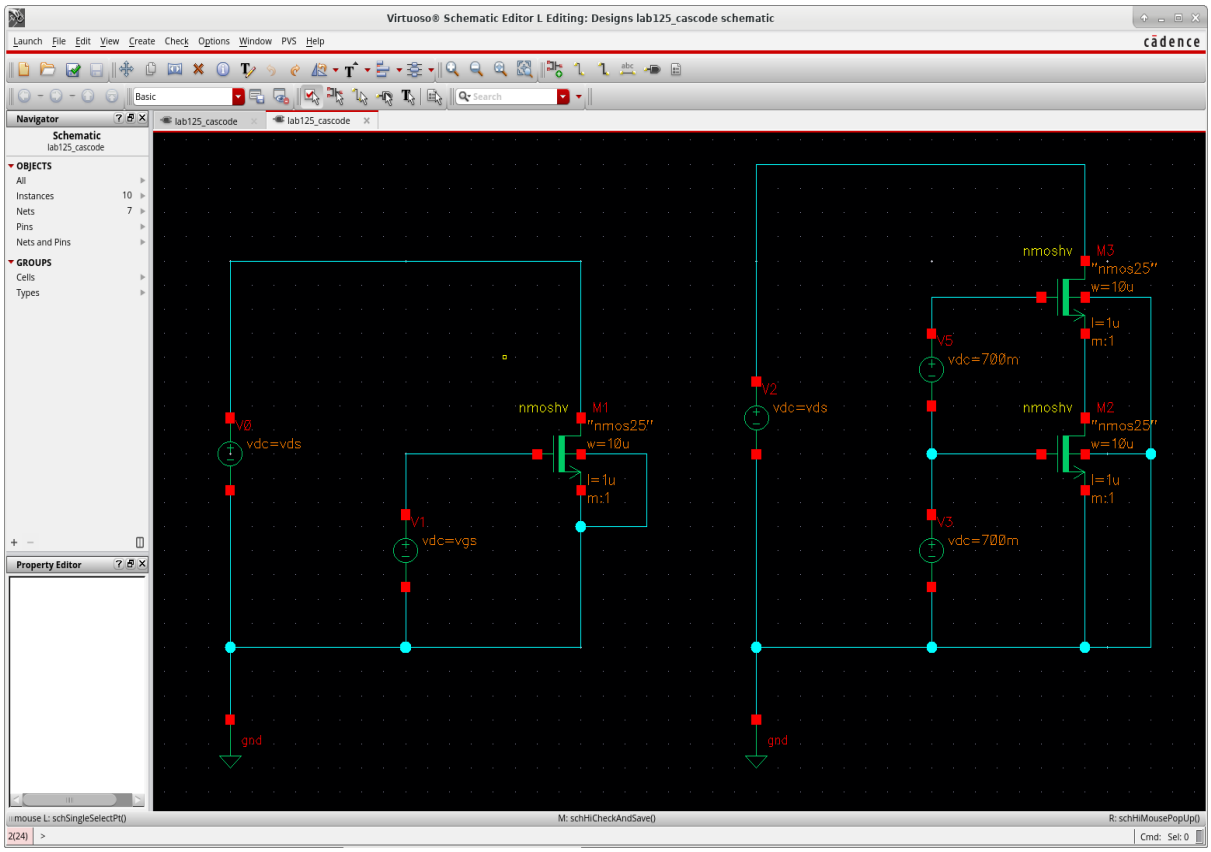
< Figure 128. Telescopic Cascode NMOS Testbench Circuit for Large Signal Transconductance Terminal Characteristics Evaluation. >

(4.3.3) Copying the circuit. Since the new circuit is similar to the previous one (except for the cascode device) you can save time/effort by copying part or all of the previous circuit. In the schematic window, on an empty part of the design canvas, left-click-and-hold the mouse button and drag a selection box around the circuit you want to duplicate. When you release the mouse button, the selected circuit will be highlighted pink/purple. Choose the **Copy** command (**Menu: Edit → Copy** or press '**c**') and click on a highlighted part of the circuit. A copy of the circuit should now attach itself to and move with the cursor. You can instantiate the copy by left-clicking anywhere on the design canvas.

(4.3.4) Modify you second circuit by adding a second NMOS FET device from the GPDK180 library (**nmoshv**). Add a telescopic repeated DC voltage source (**vdc**) for the additional NMOS gate-to-source bias.

(4.3.5) Set all NMOS devices (in both circuits) to a W/L ratio of 10um/1um (**Total Width** = *10u*, **Length** = *1u*). Set the gate-to-source voltages of all the NMOS devices to 0.7V (**DC Voltage** = *0.7*) – you can do this be setting the voltage attribute to 0.7V or by parametrising all vdc's to *vgs* and setting them to 0.7 in the ADE. Leave all other attributes/properties unchanged.

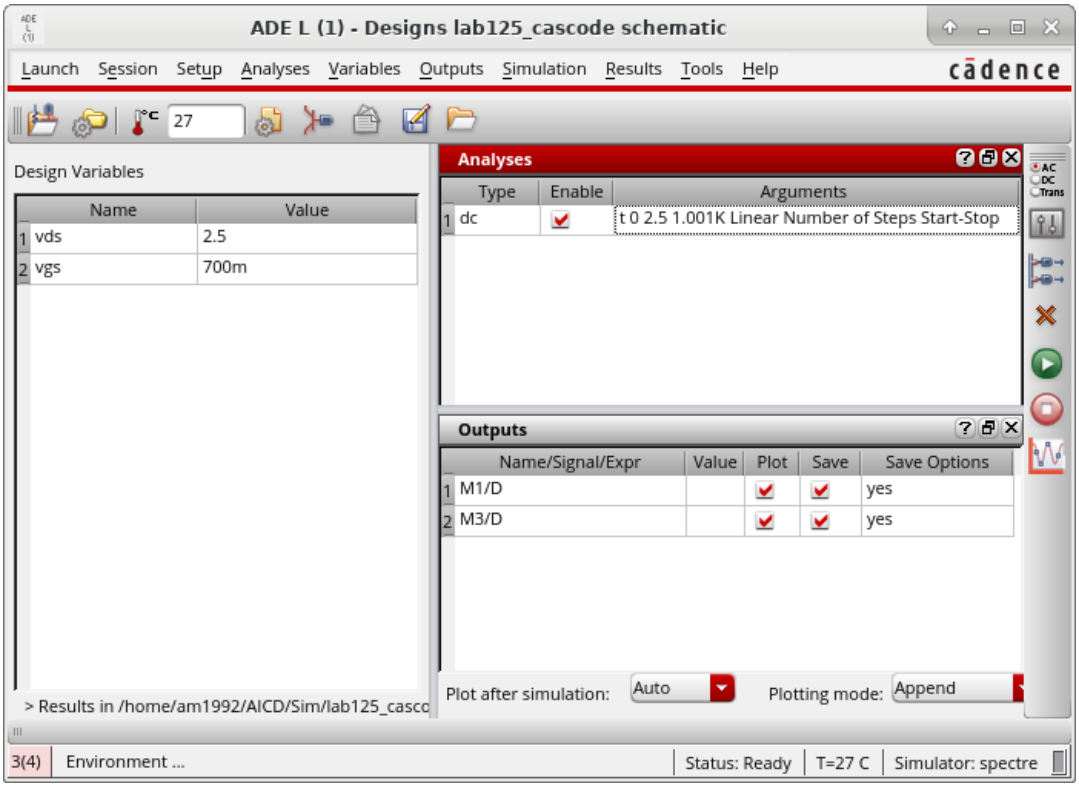
(4.3.6) Check and save your schematic using the **Check and Save** command.



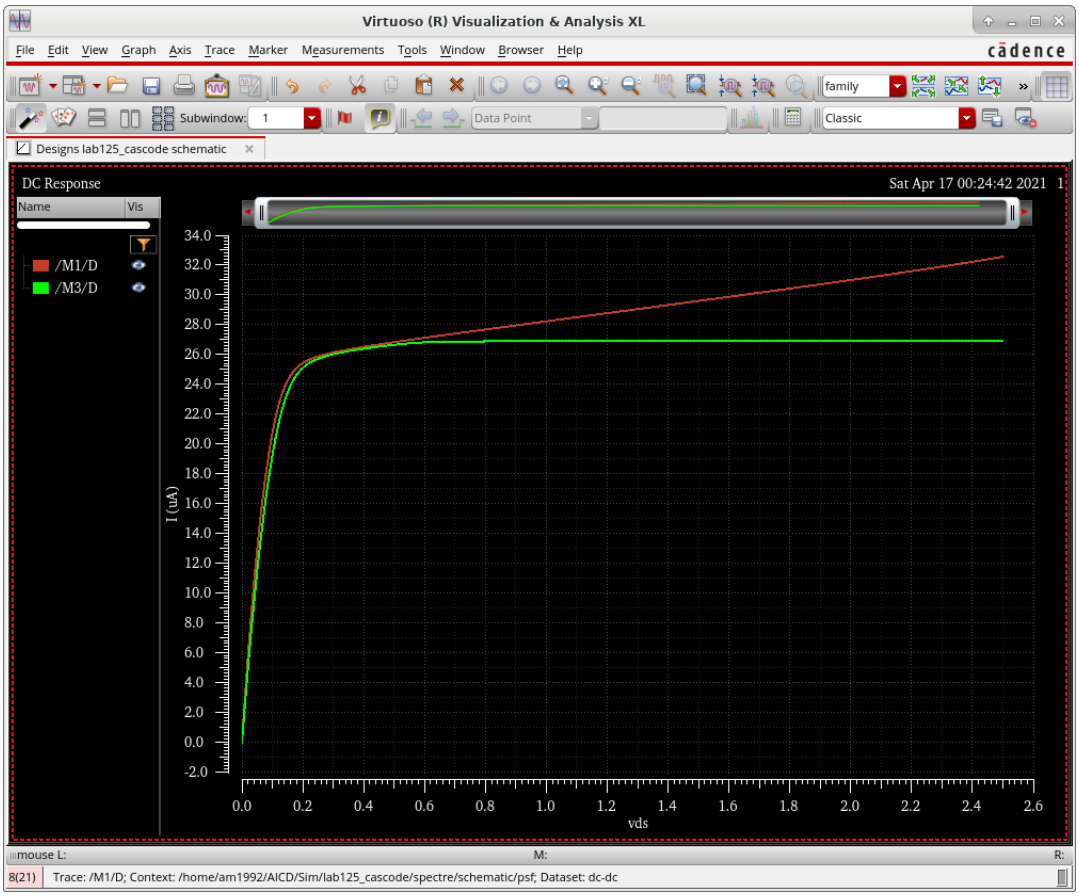
< Figure 129. Schematic with Non-Cascoded (left) and Cascoded (right) NMOS Testbench Circuit. >

(4.3.7) You should still have your ADE set up for simulation. (We will again sweep the input voltage *vds* of the NMOS device from 0V to 2.5V while observing the drain current IDS. We will use 1001 steps in the sweep to give the output waveform sufficient resolution.) If not, or you have created a new schematic, launch the ADE and follow the steps in the previous section to set up the swept-DC analysis (of VDS).

(4.3.8) For plotting, select the drain node current of the original testbench circuit (M1/D) and the drain node of the cascode device in the new testbench circuit (M3/D). **Press the 'ESC' key to exit the plotting mode of the ADE**. Run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error and both the text output window as well as the waveform output window should open automatically. You should get a plot of I_{DS} vs. V_{DS} for the cascoded and non-cascoded NMOS devices. See Figure 15.



< Figure 130. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 131. Plot of IDS vs. VDS for the Non-Cascoded (M1) and Cascoded Devices (M3).>

(4.3.9) **Discussion**. Consider the voltage at the drain of **M1** in the non-cascoded circuit (fig. 2). It is the variation of this voltage that modulates the length of the channel of **M1**. If this voltage is fixed, then no modulation occurs leading to the obviation of the need to consider the output resistance.

Now consider the voltage at the drain of **M2** in the cascoded circuit (fig. 12). This voltage is fixed by the required overdrive voltage in **M3** and (the fixed) V_{GSM3} forming a low impedance node, whilst the drain voltage of **M3** is free to vary as it is a high impedance node. Effectively, one may consider that **M3** is buffering the drain of **M2** from the high impedance node so that only a small amount of voltage variation is observed at this point. The low voltage variation leads to a low level of channel length modulation for a given process and device geometry. There will be some voltage variation as the current through the **M2**-**M3** cascode changes. It is left to the reader to consider the meaning of 'low impedance node' in terms of how the circuit driving the node may be modelled as a voltage source.

From Figure 12 with M2 and M3. Cascoding drastically lowers the V_{DS} variation seen by the transconductance device if both devices (transconductance and cascode) are in saturation. As a result, the variation in I_{DS} is reduced. The transconductance plus cascode pair will thus have a much greater *effective* r_{ds} and consequently a greater implicit small-signal voltage gain for a given device transconductance (g_m). Referring to section 6.8.1 in [3] shows that, the r_{ds} of the cascoded pair is equal to the r_{ds} of the common source device (M2, r_{ds2}) multiplied by the small-signal intrinsic gain of the cascode device (M3, A_{O3}),

$$r_{ds} = r_{ds1} \cdot A_{O2} = r_{ds1} \cdot r_{ds2} \cdot g_{m2}$$

where the intrinsic gain of the cascode device M3 is the product of its transconductance and output resistance ($A_{O3} = r_{ds3} \cdot g_{m3}$).

5. Circuit Simulation Exercises: Part 2

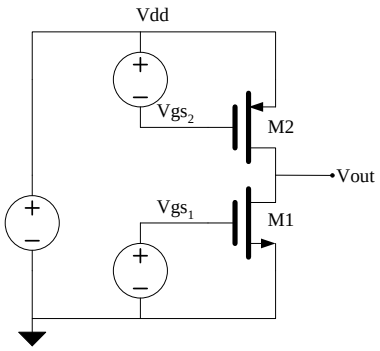
With guidance, you will:

- Simulate a telescopic cascode current-source loaded voltage amplifier circuit and compare its performance with that of a non-cascode equivalent.

5.1 The Current-Source Loaded Voltage Amplifier

(5.1.1) In this part of the lab, we will implement a current-source loaded single-stage NMOS FET voltage amplifier and observe the effect of cascoding on amplifier performance (gain).

(5.1.2) Create a new schematic for the current-source loaded voltage amplifier circuit as in Figure 16. Choose a meaningful cell name e.g., 'lab125_cascode_pt2'.



< Figure 132. Current-Source Loaded Voltage Amplifier
(Reproduced from Figure 16, section 4.2.2 of the background notes). >

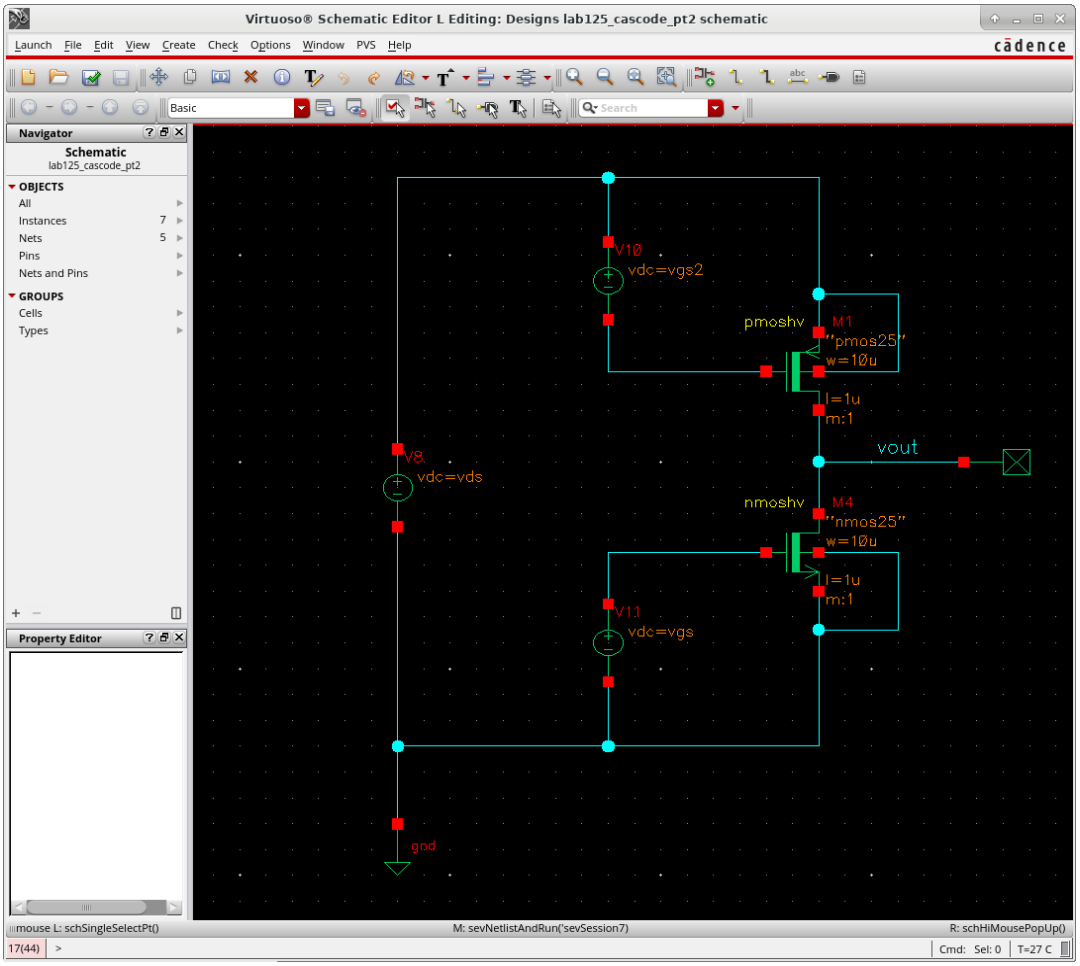
(5.1.3)) In your schematic editor window, create the circuit given in Figure 16. Instantiate three DC sources (**vdc**), a ground reference (**gnd**), and an NMOS FET and PMOS FET from the GPDK180 library (**nmoshv** and **pmoshv**).

Dimension both the NMOS and PMOS devices 10um/1um (**width** = *10u*, **length** = 1u). Leave all other attributes /properties unchanged.

Parameterise the DC supply voltage as *vds* (**DC Voltage** = vds), parameterise the gate-to-source voltage of the NMOS FET as vgs (**DC Voltage** = vgs), and parameterise the PMOS gate-to-source voltage as *vgs2* (**DC Voltage** = vgs2).

(5.1.4) Wire the components as in Figure 17 using the **Wire (narrow)** command. Name the output net *vout* using the **Create Wire Name** command. Use the **noConn** connector from the **basic** library to avoid floating-net warnings. Keep the schematic neat. See Figure 17.

(5.1.5) Check and save you schematic using the **Check and Save** command. It should check and save without any errors or warnings.



< Figure 133. Current-Source Loaded Voltage Amplifier. Variables for the DC supply voltage (vdc=vds) and the DC gate-to-source voltages (vdc=vgs, vdc=vgs2). >

5.2 Simulating the Voltage Amplifier using a Swept-DC Analysis

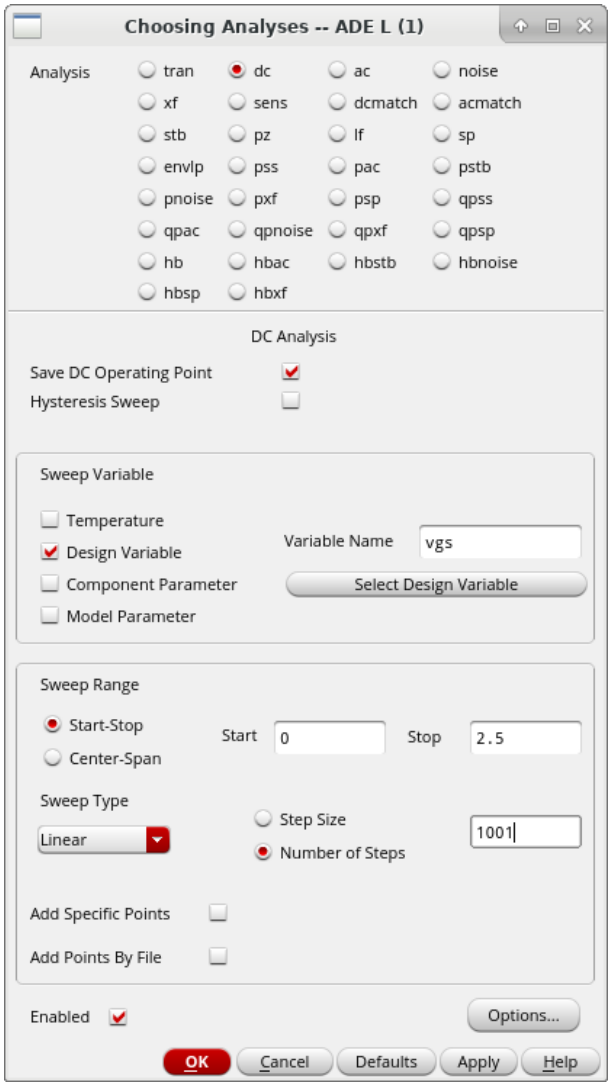
(5.2.1) We will simulate the basic voltage amplifier circuit using a DC-sweep of *vgs*. From the Schematic Editor window, launch the Virtuoso Analogue Design Environment (ADE) window from **Menu: Launch à ADE L**.

(5.2.2) Copy the design variables from the schematic into the ADE. For this circuit, these are the DC voltages from the three **vdc** components - vds, vgs, and vgs2. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the vds and vgs variables

in the ‘Design Variables’ list of the ADE. Initialise vds = *2.5V*, vgs = *0.7V*, and vgs2 = *0.7V*. These values are used only to calculate the DC operating point of the amplifier circuit.

(5.2.3) From the ADE window, go to **Menu: Analyses à Choose…** or use the ‘**Choose Analyses**’ button on the right-side of the ADE window (top button). The ‘Choosing Analyses – ADE L’ form will open. From the form, select the ‘**dc**’ option button.

(5.2.4) We wish to sweep the input voltage (V_{gs}) of the NMOS device from 0V to 2.5V while observing the output voltage (V_{out}) – the input/output voltage transfer curve. We will use 1001 steps in the sweep to give the output waveform sufficient resolution.
In the choosing analyses form. Check the ‘Save DC Operating Point’ box. Check the ’Design Variable’ box as the sweep variable. Type ‘vgs’ as the variable name. Choose the ‘Start-Stop’ option button with Start = 0 and Stop = 2.5 for the sweep range. Select ‘Linear’ from the drop down menu as the sweep type. Choose the ‘Number of Steps’ option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the ‘Enabled’ box near the bottom of the form. See Figure 17. OK the form. The **dc** analysis should now appear in the ‘Analyses’ list of the ADE window.

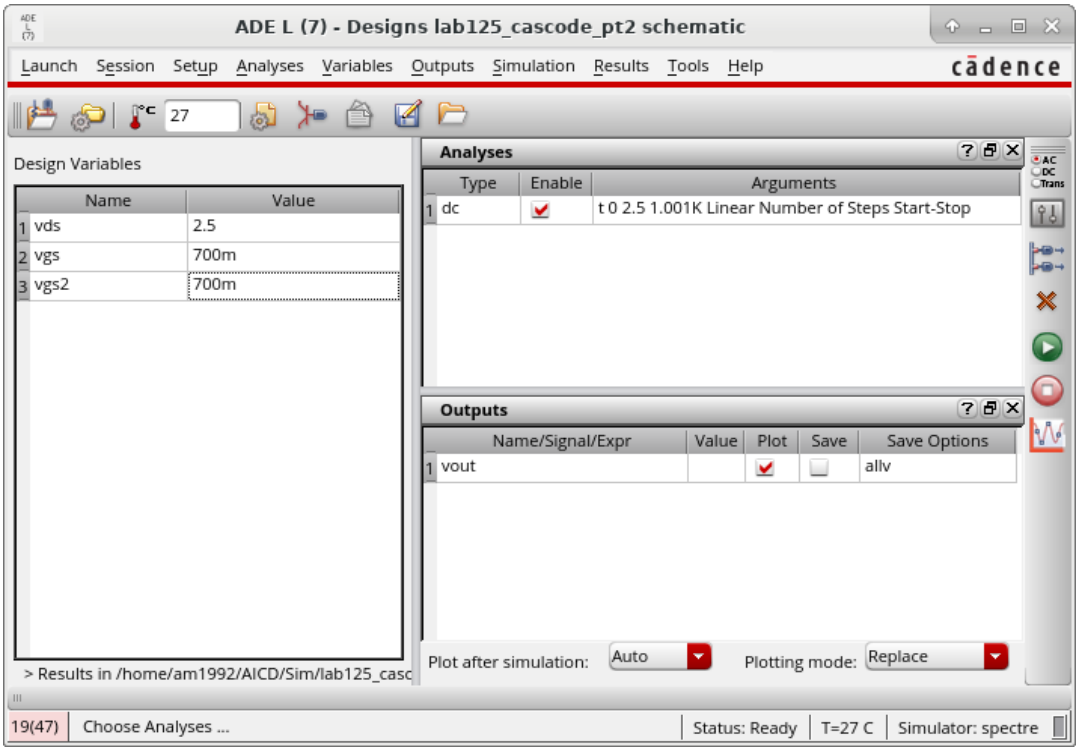


< Figure 134. The Choosing Analyses Form. >

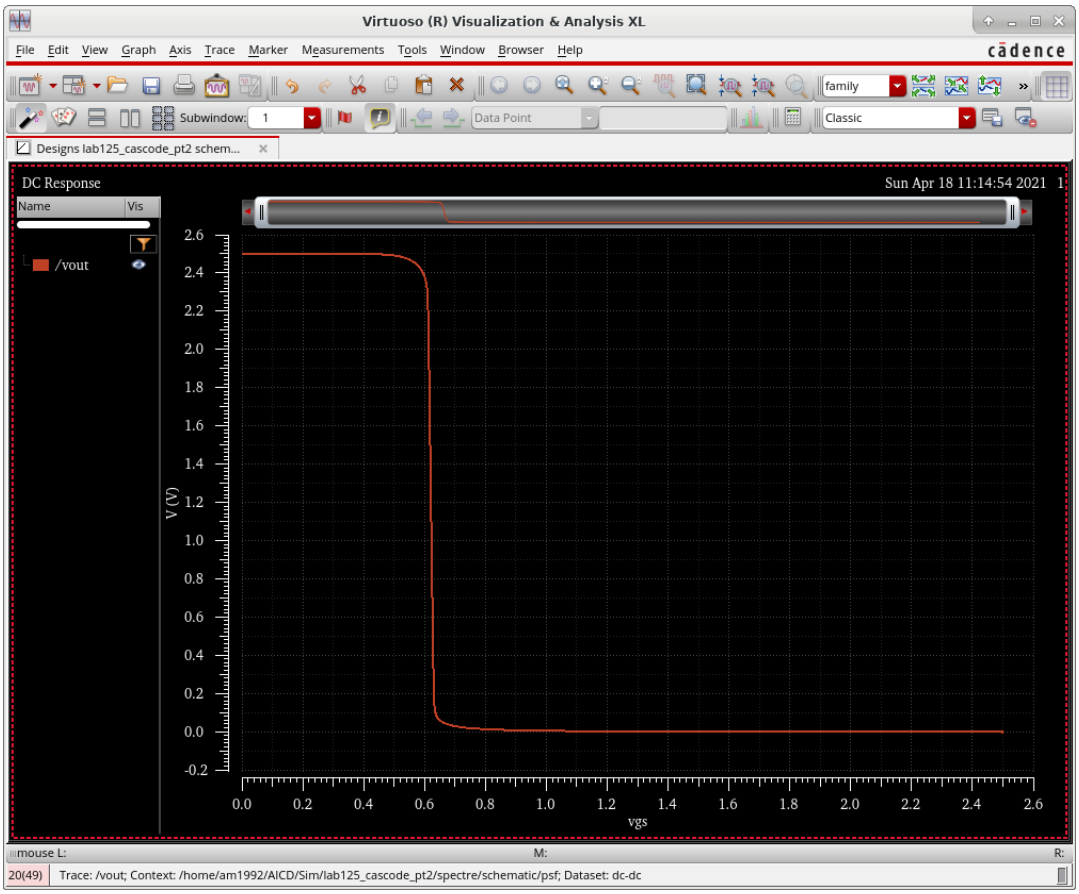
(5.2.5) Select the output net voltage (vout) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**.** The schematic window will automatically become current. Select only the output net vout. **Press the 'ESC' key to exit the plotting mode of the ADE**.

(5.2.6) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figures 19 and 20.

(5.2.7) Repeat the simulation again with vgs swept between 0.5V and 0.7V for a better picture of the analogue region of the voltage transfer curve. See Figures 21 and 22.

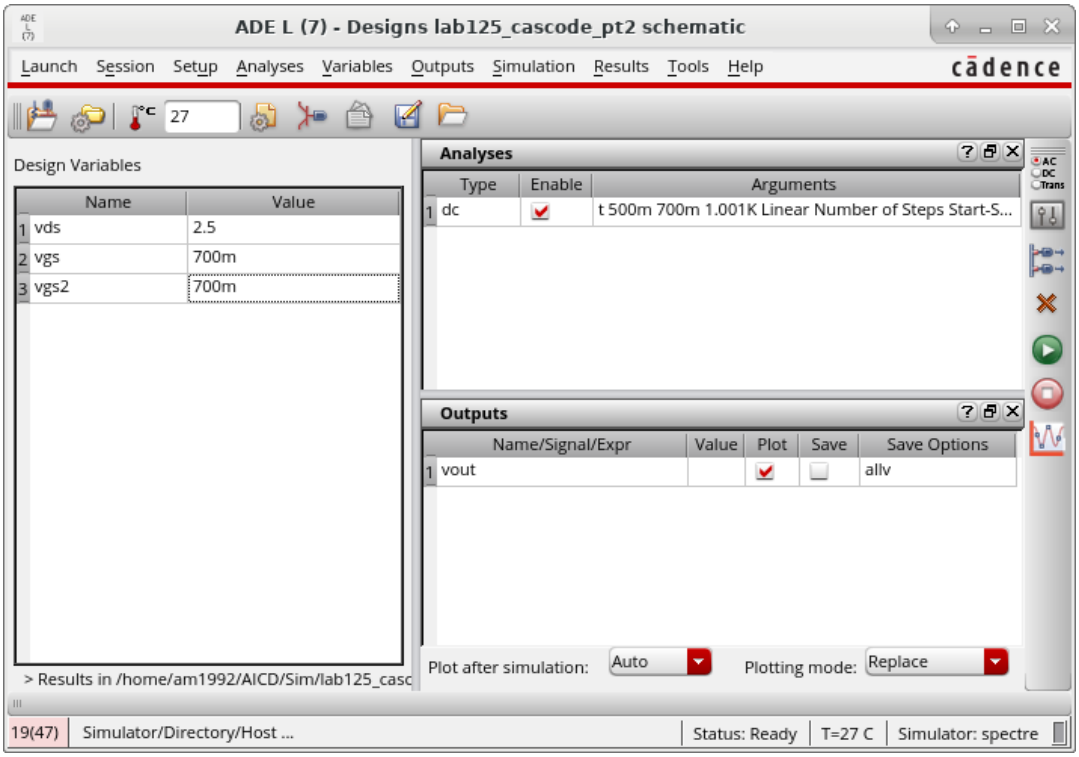


< Figure 135. The ADE with the Design Variables, Analyses and Outputs. >

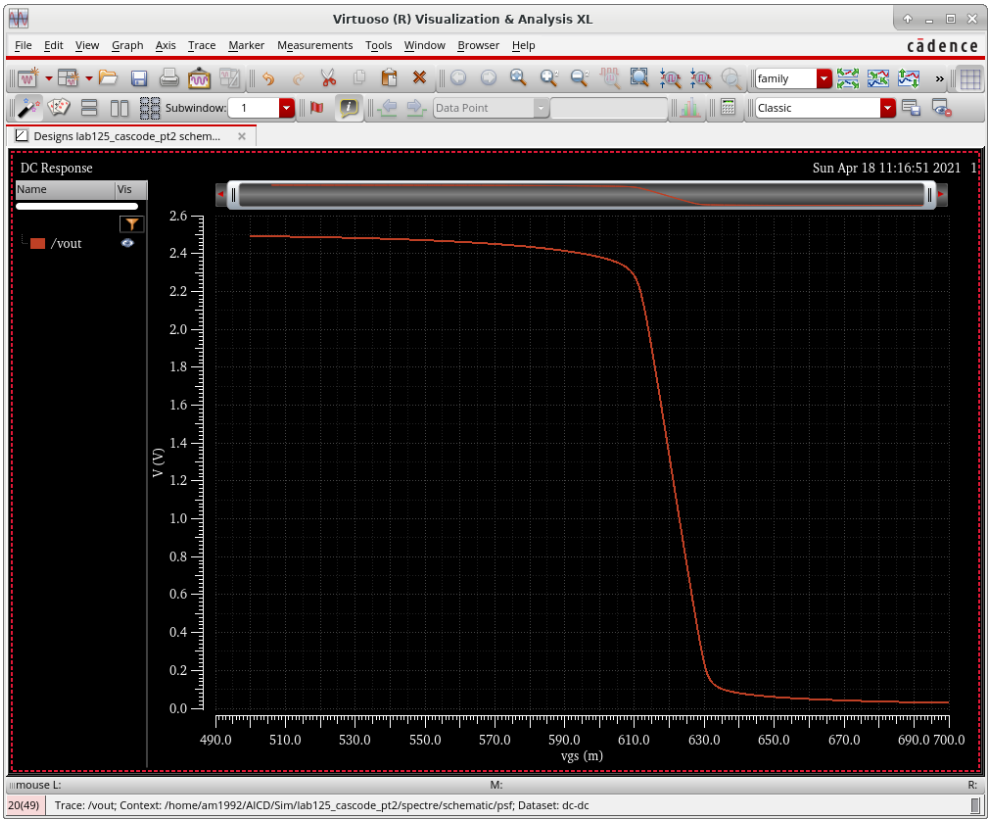


< Figure 136. Voltage Transfer Curve for the Current-Source Loaded Voltage Amplifier.

VDS=2.5V. VGS 0V → 2.5V. >



< Figure 137. The ADE with the Design Variables, Analyses and Outputs. >



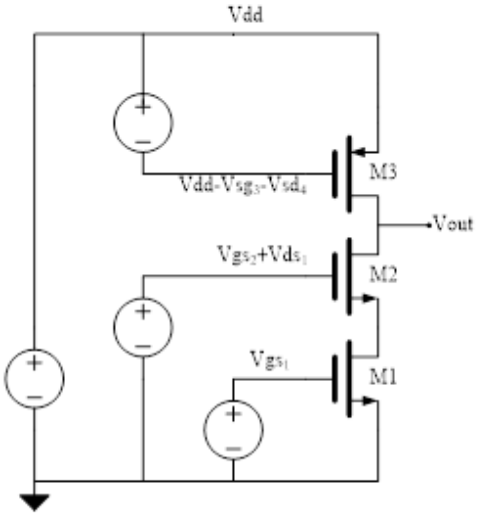
< Figure 138. Voltage Transfer Curve for the Current-Source Loaded Voltage Amplifier.

VDS=2.5V. VGS 0.5V → 0.7V. >

5.3 Cascoding the Transconductor Device to Increase Amplifier Gain

(5.3.1) We next will add a cascode stage to the NMOS transconductance device. This will increase the output impedance of the transconductance stage and consequently gain.

(5.3.2) Modify the schematic by constructing a second amplifier circuit alongside your existing amplifier circuit. The new circuit is the same as the previous one except for the addition of a cascode device in the transconductance stage. See Figure 23.



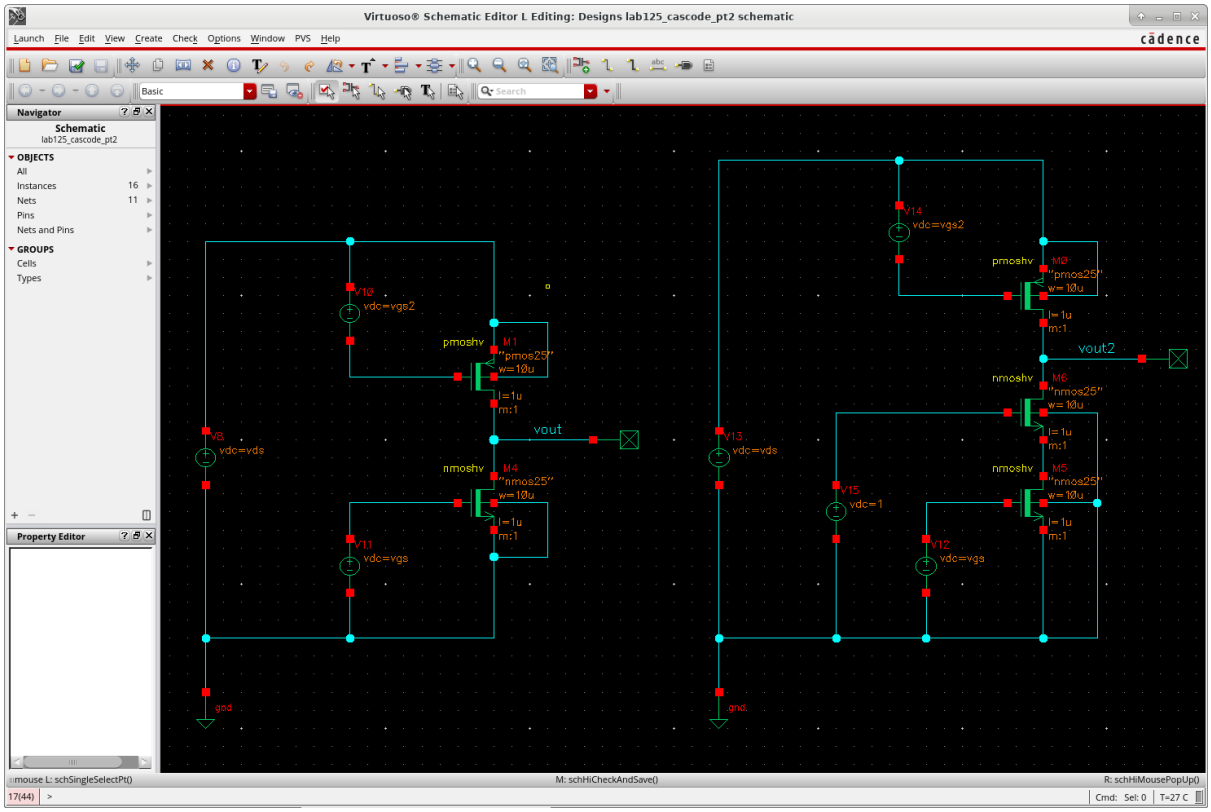
< Figure 139. Current-Source Loaded Voltage Amplifier with Cascoded Transconductance Stage>

(5.3.3) Copying the circuit. As before, in the schematic window, on an empty part of the design canvas, left-click-and-hold the mouse button and drag a selection box around the circuit you want to duplicate. Choose the **Copy** command (**Menu: Edit → Copy** or press '**c**') and click on a highlighted part of the circuit. A copy of the circuit should now attach itself to and move with the cursor. Instantiate the copy by left-clicking anywhere on the design canvas. See Figure 24.

(5.3.4) For the new circuit, all FET devices should be 10um/1um (**width** = *10u*, **length** = 1u). Add a separate DC voltage source (**vdc**) for the NMOS cascode device. Set the vdc for the cascode device to 1V (**DC Voltage** = *1*). Leave all other attributes/properties unchanged. See Figure 24. You can zoom in for closer inspection if needed.

(5.3.5) Change the name of the output net to vout2 (this avoids creating an implicit connection between the outputs of the two circuits.) Select the second output net voltage (vout2) for plotting alongside the first one (vout). **Press the 'ESC' key to exit the plotting mode of the ADE**.

(5.3.6) Check and save you schematic using the **Check and Save** command. It should check and save without any errors or warnings.

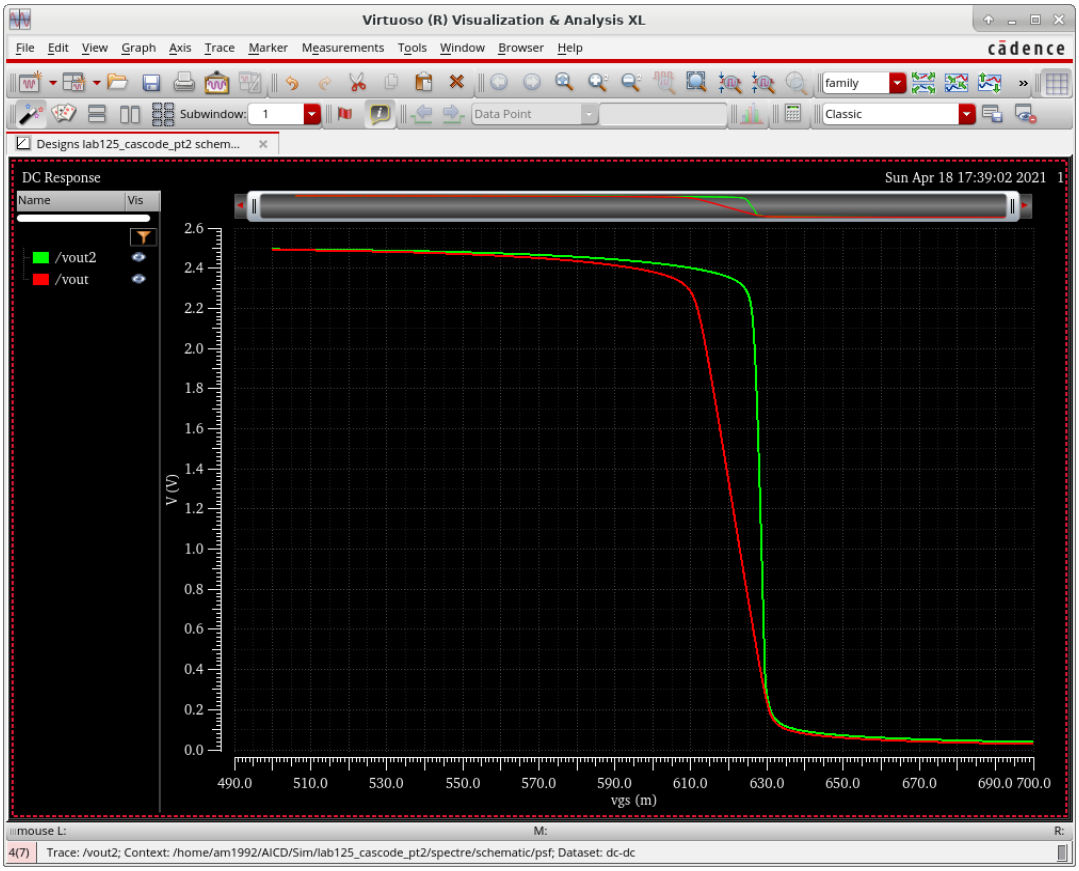


< Figure 140. Schematic with Non-Cascoded (left) and Cascoded (right) Current-Source Loaded Voltage Amplifiers. Only the Transconductance Stage is Cascoded. >

(5.3.6) From the Analog Design Environment window, re-run the DC sweep. Run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figure 25.

(5.3.7) **Discussion**. Comparing the output of the non-cascoded (red) and cascoded (green) stages, we see the increased voltage gain of the latter (indicated by the steeper slope of the green VTC curve) is due to the higher effective output resistance of the transconductance stage. In this case, the output impedance of the non-cascoded stage is given by ($r_{op}||r_{on}$) and for the cascoded stage it is given by ($r_{op}||r_{on}.r_{on}.g_{mn}$), where r_{op} is the output resistance of the PMOS load device and r_{on} is the output resistance of the NMOS transconductance and identical cascode devices. g_{mn} is the transconductance of the cascode device. $A_{on}=r_{on}.g_{mn}$.

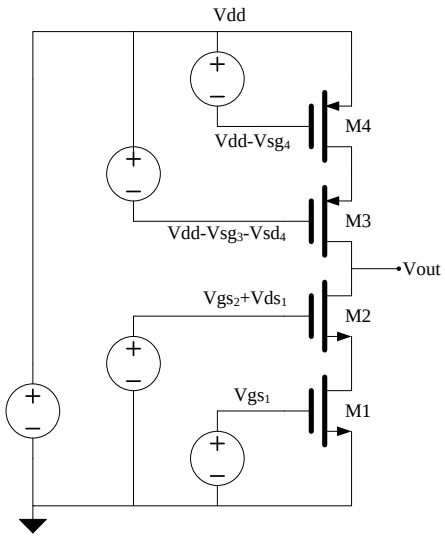
We will also note (briefly at this stage) that with the increased gain, comes the need to accurately set the DC operating point (the steeper the slope of the VTC the more sensitive the amplifier is to imprecise biasing). This cannot be done by designing for nominal device parameters in the same way we may design discrete circuits; instead, we must use feedback to set the DC operating point. We will discuss this important topic in the coming labs.



< Figure 141. Voltage Transfer Curves for the Non-Cascoded and Cascoded Current-Source Loaded Voltage Amplifiers. VDS=2.5V. VGS 0.5V → 0.7V. >

5.4 Cascoding the Transconductor and Load Devices to Increase Amplifier Gain

(5.4.1) Finally, we will add a second cascode stage, this time for the load device. Cascoding just the transconductance device makes the load device the weak link. Cascoding both the transconductor and load will maximise the gain.

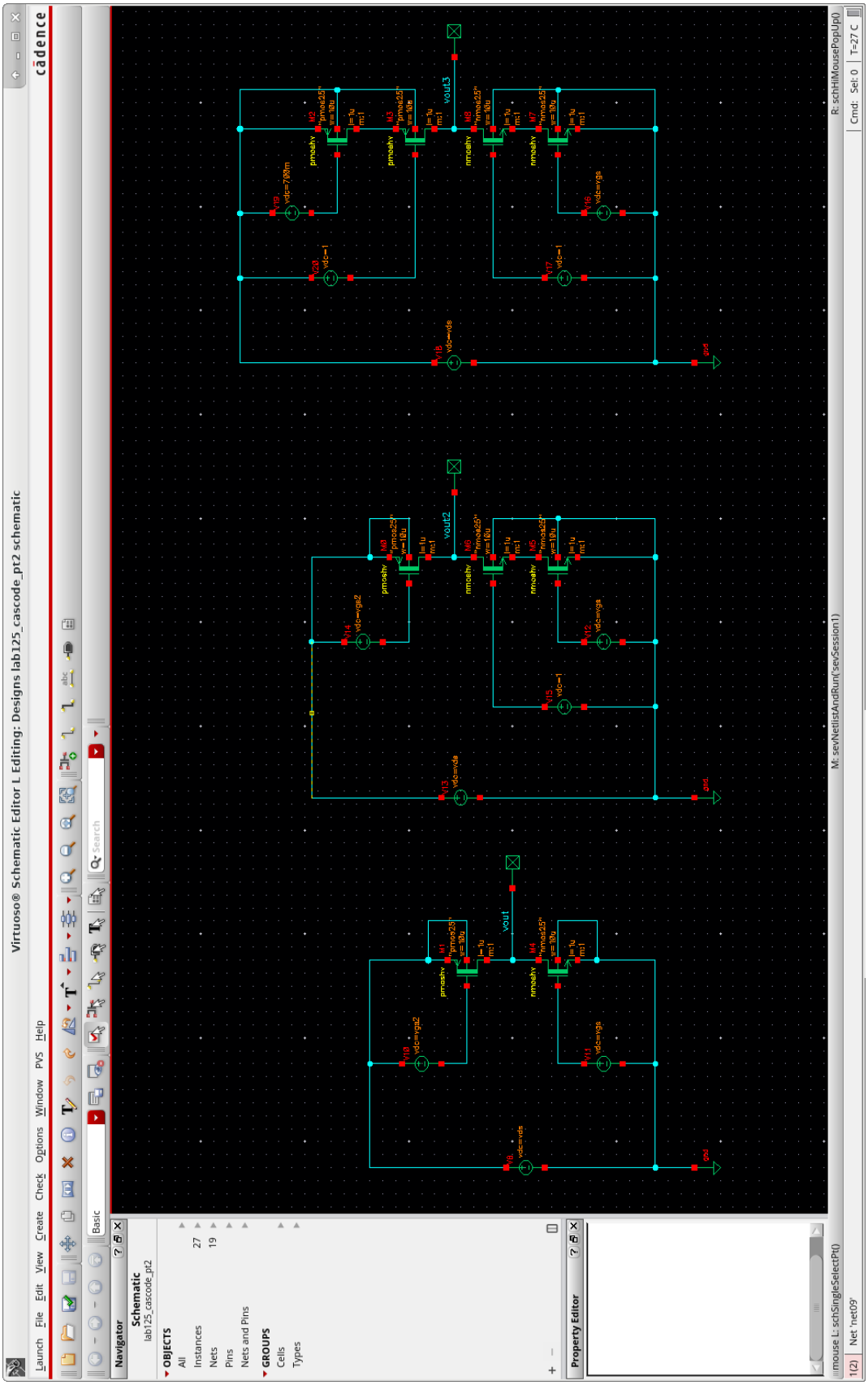


< Figure 26. Cascode NMOS Characteristics Evaluation. Reproduced from Figure 14, section 4.1 of the background material. >

(5.4.2) Again, for the new circuit, all FET devices should be 10um/1um (**width** = *10u*, **length** = 1u). Add another separate DC voltage source (**vdc**) for the PMOS cascode device. Set the vdc for the cascode device to 1V (**DC Voltage** = 1). Leave all other attributes/properties unchanged. See Figure 27.

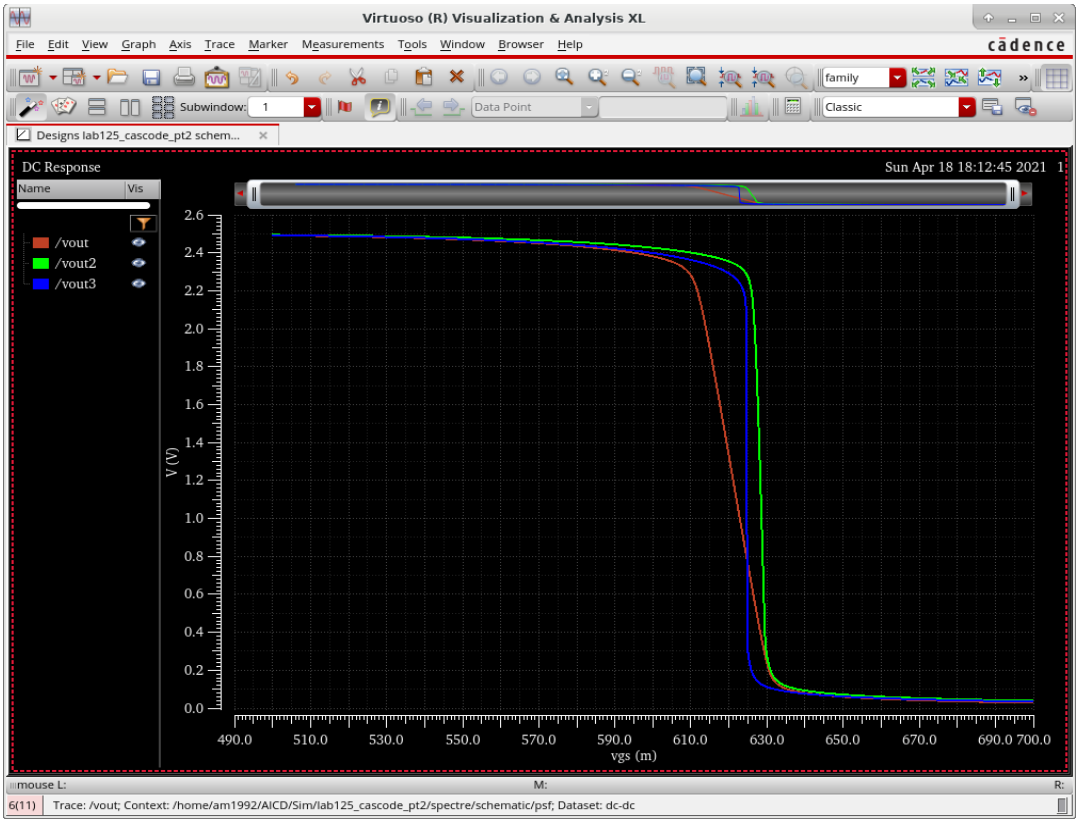
(5.4.3) Change the name of the output net to vout3. Select the third output net voltage (vout3) for plotting alongside the previous circuits. **Press the 'ESC' key to exit the plotting mode of the ADE**.

(5.4.4) Repeat the previous steps to add a third amplifier circuit alongside your existing circuits. You should now have three versions of the current-source loaded amplifier circuit: (1) an amplifier with no cascoding, (2) an amplifier with only the transconductance stage cascoded, and (3) an amplifier both the transconductance and load devices cascoded. See Figure 27.

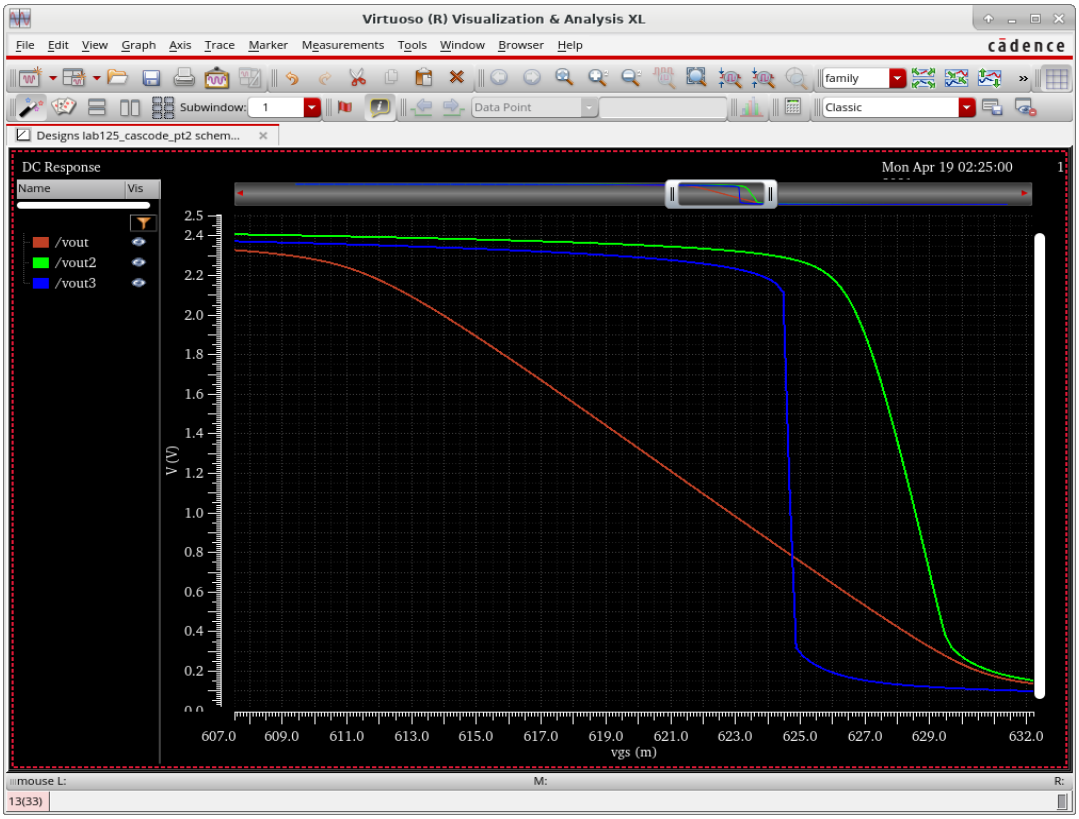


< Figure 142. Schematic with Non-Cascoded (left), Transconductor-Only Cascoded (middle) and Fully Cascoded (right) Current-Source Loaded Voltage Amplifier. >

(5.4.5) From the Analog Design Environment window, re-run the DC sweep. Run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically. See Figures 28 and 29.



< Figure 143. Voltage Transfer Curves for the Non-Cascoded (red), Transconductor-Only Cascoded (green), and Fully Cascoded Current-Source Loaded Voltage Amplifiers. VDS=2.5V. VGS 0.5V → 0.7V. >



< Figure 144. Close-Up of Figure 28 in the Analogue Region. >

(5.4.6) **Discussion**. Again, we see the increased voltage gain of the fully cascoded amplifier (blue) is due to the higher total output resistance $(r_{op}.r_{op}.g_{mp})||(r_{on}.r_{on}.g_{mn})$.

STUDENT TASK: Given the need for accurate/precise biasing (as discussed above), keep vgs=700mv and perform an **ac** simulation to establish that there is no gain (ac 1k-1M). establish accurate vgs-quiescent = 624.7mv – gain is now 15k (83dB)]

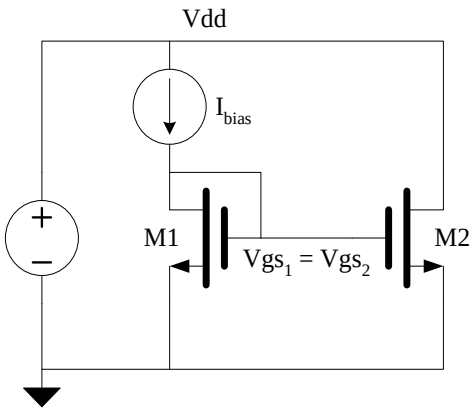
6. Circuit Simulation Exercises: Part 3

With guidance, you will:

- Investigate cascoding in a MOSFET current mirror.

6.1 A Basic Current Mirror

(6.1.1) Create a new schematic for the basic current mirror circuit in Figure 30. A summary of the circuit's operation is given in section 4.3.1 of the background material.



< Figure 145. Basic Current Mirror. Reproduced from Figure 18, section 4.3.1 of the background material. >

(6.1.2) From the library manager, invoke the New File form as you have done previously – ensuring you are in your Designs library – giving your new schematic a meaningful name such as 'lab125_cascode_pt3'.

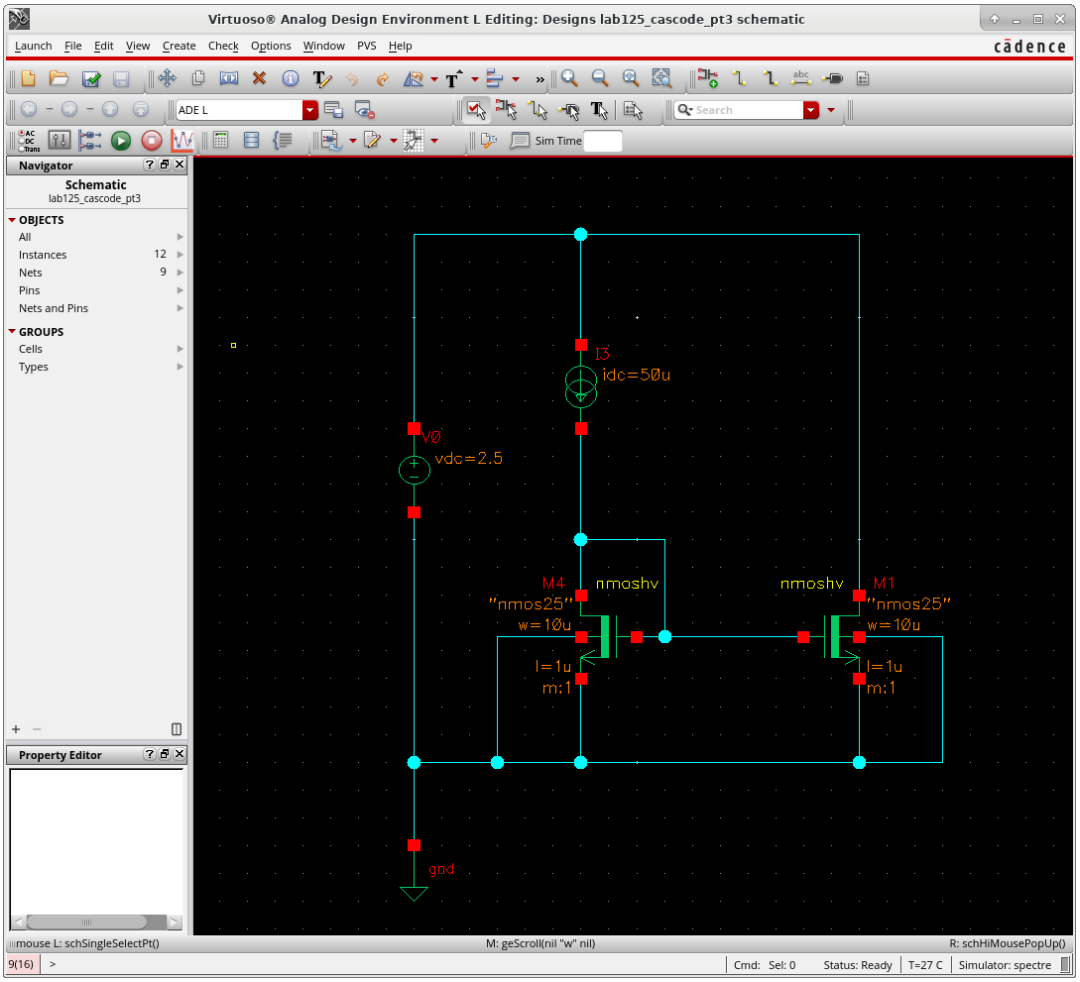
(6.1.3) In your schematic editor window, create the circuit given in Figure 30. Instantiate one ideal DC voltage source (**vdc**) and one ideal DC current source (**idc**) from the analogLib library. Instantiate two NMOS FETs from the GPDK180 library (**nmoshv**). Set the DC supply voltage to 2.5V (**DC Voltage** = 2.5), set the DC current reference to 50uA (**DC current** = 50u). Dimension both NMOS devices 10um/1um (**width** = *10u*, **length** = 1u). Leave all other attributes unchanged.

(6.1.4) Wire the components as in Figure 30 (31) using the **Wire (narrow)** command.

(6.1.6) Check and save your schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.

6.2 Simulating the DC operating Point of the Current Mirror

(6.2.1) We will simulate the current mirror using a DC Analysis. From the Schematic Editor window, launch the Analogue Design Environment (ADE) from **Menu: Launch à ADE L**.

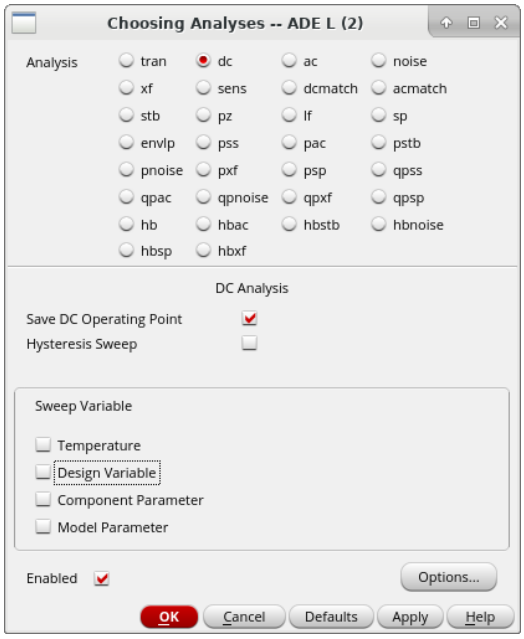


< Figure 146. Schematic of the Basic Current Mirror. 10um/1um. >

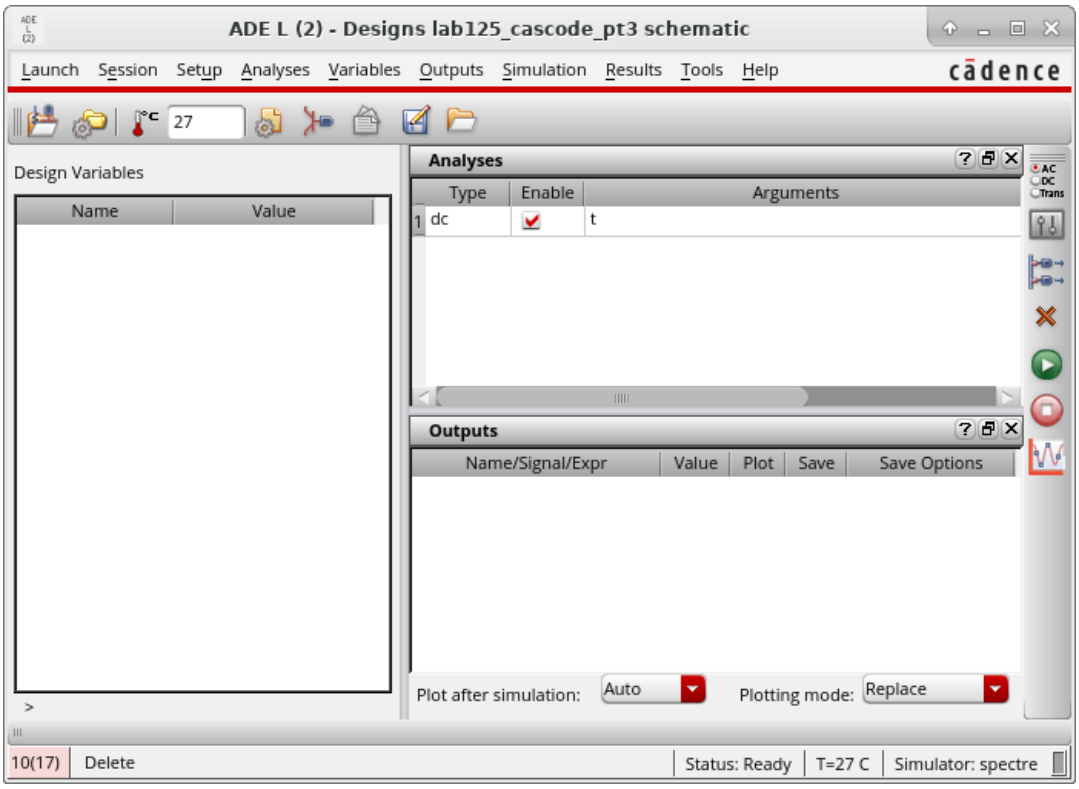
(6.2.2) From the ADE window, go to **Menu: Analyses à Choose…** or use the **Choose Analyses** button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. On the form, select the '**dc**' option button. Check the 'Save DC Operating Point' box. OK the form.

(6.2.3) At this stage we are only simulating for the DC operating point. Simulating the DC operating point does not produce an output waveform, so we do not need to select any outputs to be plotted.

(6.2.4) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. The text output window should appear but there will be no waveform window.



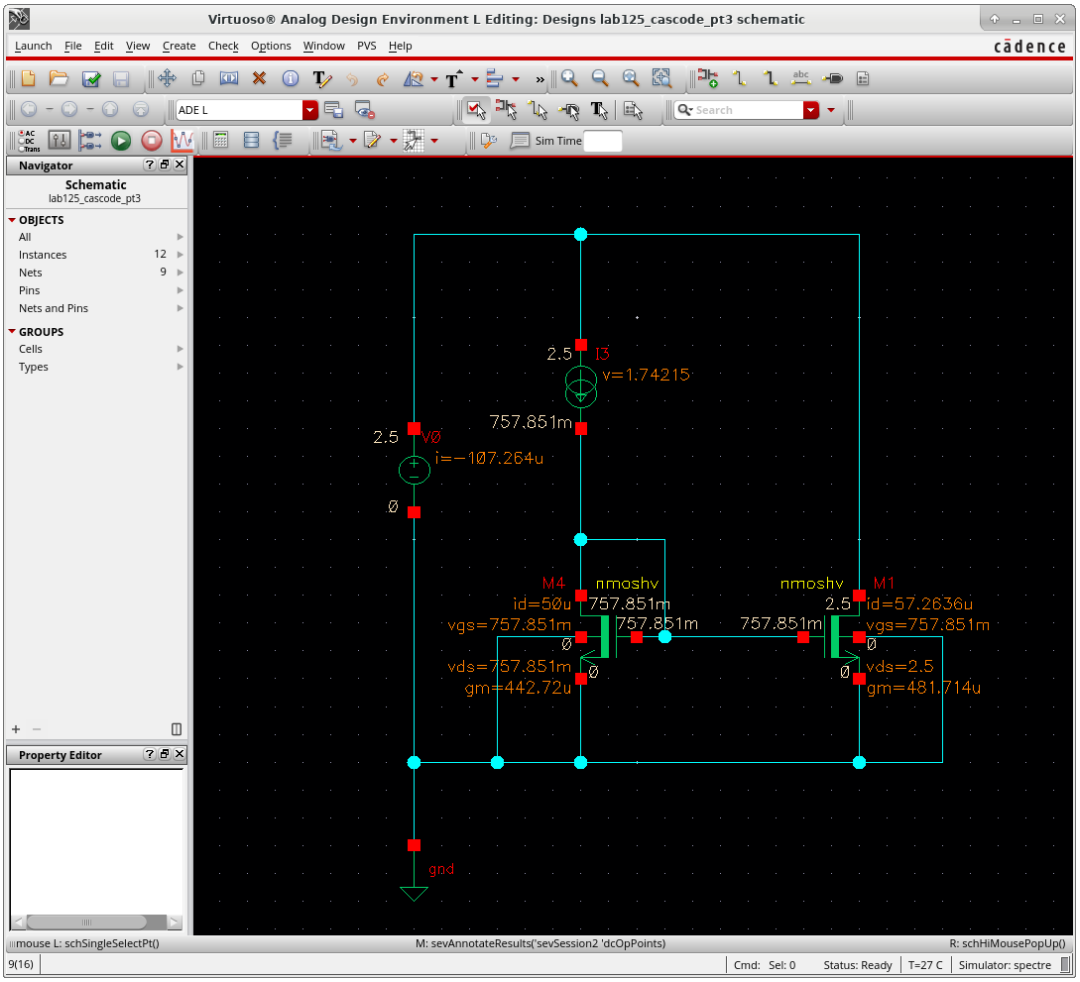
< Figure 147. The Choosing Analyses Form. Only DC Operating Point selected. >



< Figure 148. The ADE for a DC operating Point only Analysis. >

(6.2.5) From the ADE window choose **Menu: Results à Annotate à DC node voltages…** If you now look in the schematic window, you should see the node voltages annotated directly on the schematic. Again, from the ADE window, choose **Menu: Results à Annotate à DC operating points…** You should now see (along with the node voltages) additional DC operating

point information annotated on the schematic including the DC bias currents, NMOS device transconductance gm, IDS, VGS, and VDS. See Figure 5.



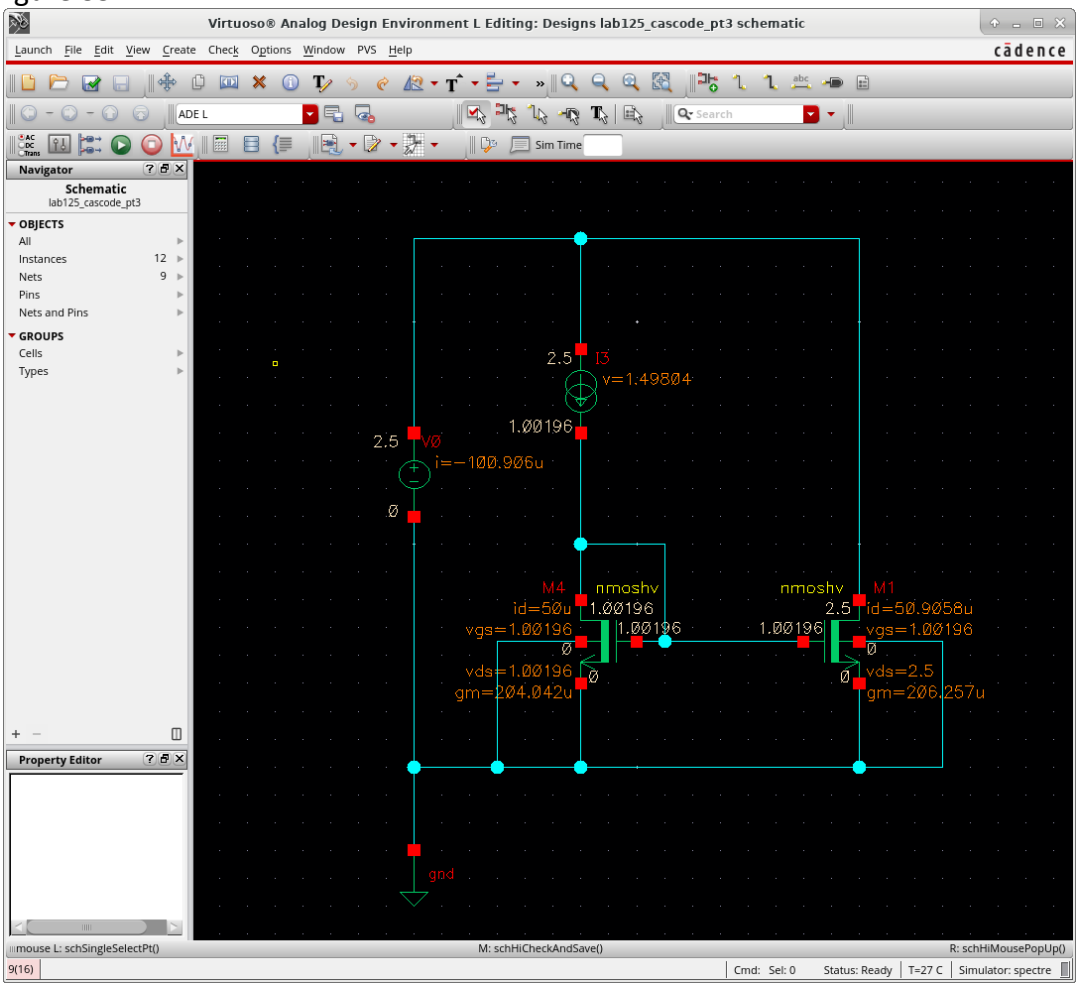
< Figure 149. DC Analysis – Save DC Operating Point. Annotated Schematic of the Basic Current Mirror. 10um/1um >

(6.2.6) **Discussion**. Note the annotated values for the drain (id) current for both devices. Despite both NMOS devices being identical, with identical V_{GS}, poor current mirroring (**50uA vs 57.3uA**) results from the vastly different values of vds (**0.758V vs 2.5V**) and the consequent effect of channel length modulation. As before, we can reduce the dependency of I_{DS} on V_{DS} by either increasing the length of the channel (making the effect of channel length modulation less sensitive) or by cascoding (shielding the mirror devices from any differences in V_{DS}).

6.3 Increasing the Channel Length to Improve Current Mirroring

(6.3.1) We will re-run the previous simulations but with the MOSFET channel length increased from 1um to 4um. Dimension both NMOS devices 10um/4um (**width** = *10u*, **length** = 4u). Leave all other attributes unchanged.

(6.3.2) From the Analog Design Environment window, re-run the DC Operating point analysis. See Figure 35.



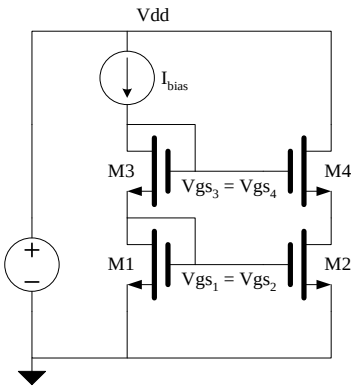
< Figure 150. DC Analysis – Save DC Operating Point. Annotated Schematic of the Basic Current Mirror. 10um/4um >

(6.3.3) **Discussion**. Increasing the channel length reduces the effect of channel length modulation despite the large difference in VDS (**1.002V vs 2.5V**). r_{ds} is increased (by reducing the dependency of IDS on VDS) such that the output device behaves more like an ideal current source. This improves current mirroring (**50uA vs 50.9uA**). However, since r_{ds} =Ve*L/ I_{ds} this strategy is highly current dependent. Where a current mirror is used as an active load for high-frequency circuits – a longer channel length may be an issue (especially where a proportional increase in device width is required to achieve a given gm).

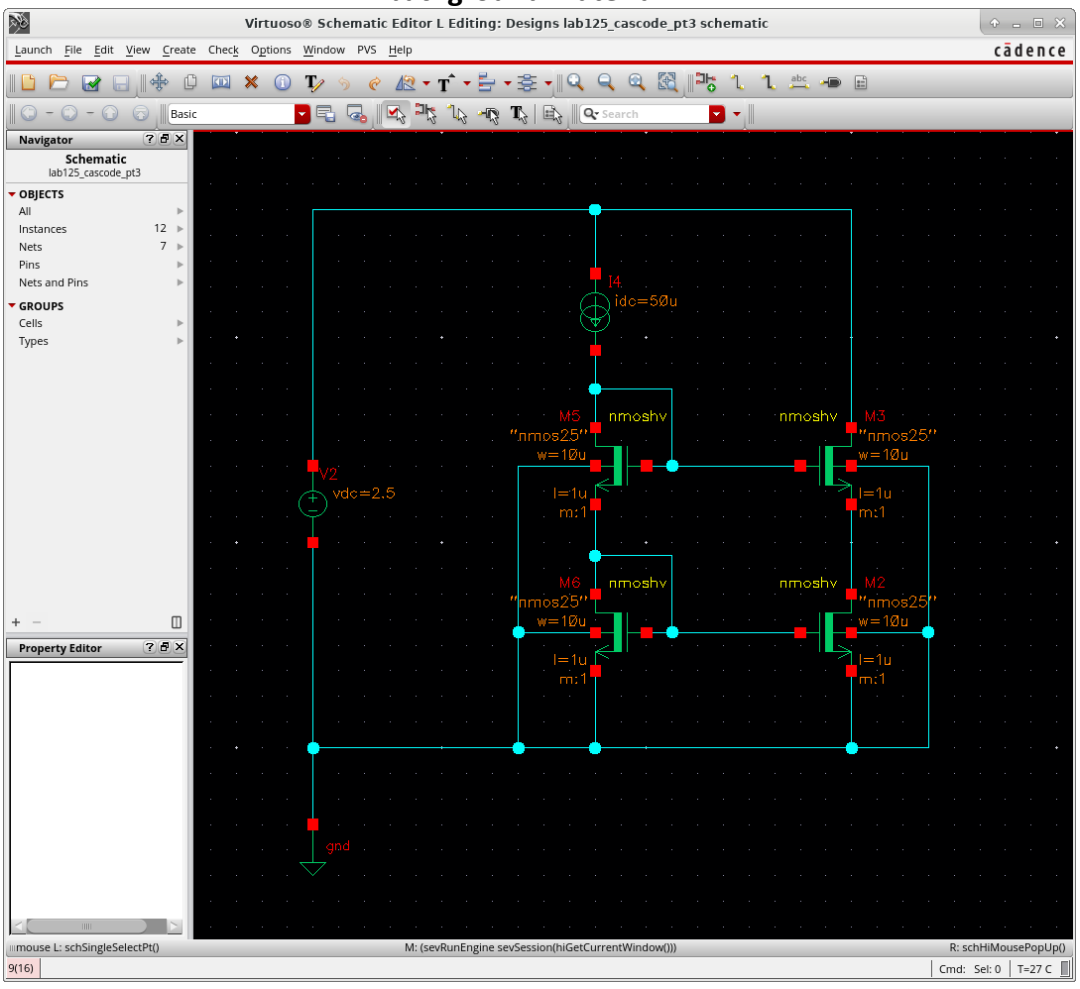
6.4 Cascoding the Basic Current Mirror to Improve Current Mirroring

(6.4.1) We will add a cascode stage to the basic current mirror in Figure 30 (31).

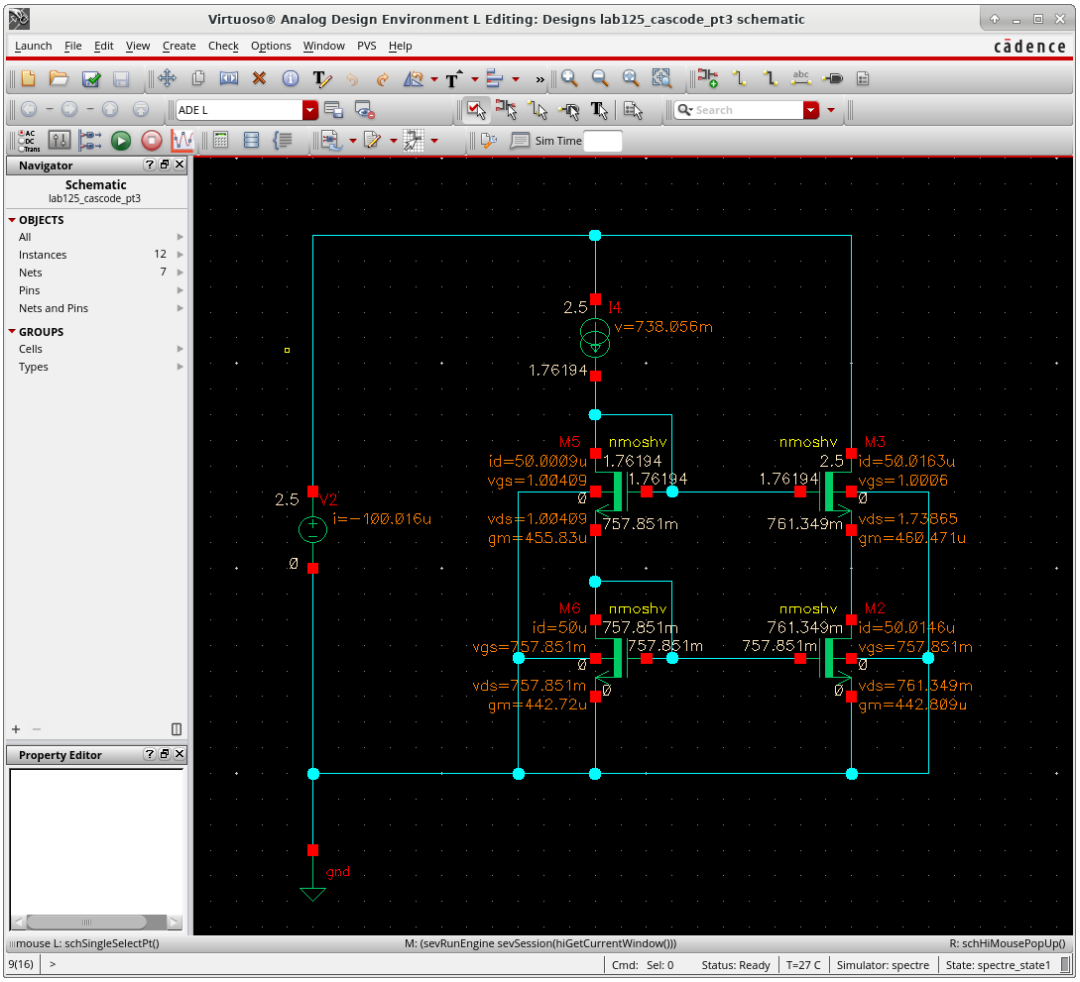
(6.4.2) Modify your existing testbench schematic by constructing a second mirror circuit alongside your existing circuit. The new circuit is the same as the previous one except for the addition of the identical cascode devices. See Figure 36.



< Figure 151. Cascode Current Mirror. Reproduced from Figure 19, section 4.3.2 of the background material. >



< Figure 152. Schematic of the Cascode Current Mirror. 10um/1um >



< Figure 153. DC Analysis – Save DC Operating Point. Annotated Schematic of the Cascode Current Mirror. 10um/1um >

(6.4.3) From the Analog Design Environment window, re-run the DC Operating point analysis. See Figure 38.

(6.4.4) **Discussion**. Note that the v_{ds} of the two lower devices are now much closer (**0.758V vs 0.761V**). This gives much better matching in mirror current (**50uA vs. 50.02uA**) for half the total gate area (with respect to the pair of 10u*4u devices) but at the expense of less output voltage headroom due to the saturation requirement of cascode devices. The designer will commonly want very accurate matching with currents and so will be willing to tolerate this. Note that other versions of the cascoded current mirror exist that offer greater voltage headroom for a similar mirror performance.

STUDENT TASK: Insert a separate, parameter-controlled, Vd for the output of the mirror and examine Iout/Vout. Then, by setting plot mode to 'append' plot the un-cascoded mirror and compare. Note that the mirrors are forced out of saturation near the same Vd but the cascode mirror has much better immunity to variations in Vd.

End of Lab!

Tutorial Lab 12.5 (Formatively Assessed): (…continued)

Symbol View Generation

7. Circuit Simulation Exercises: Part 4

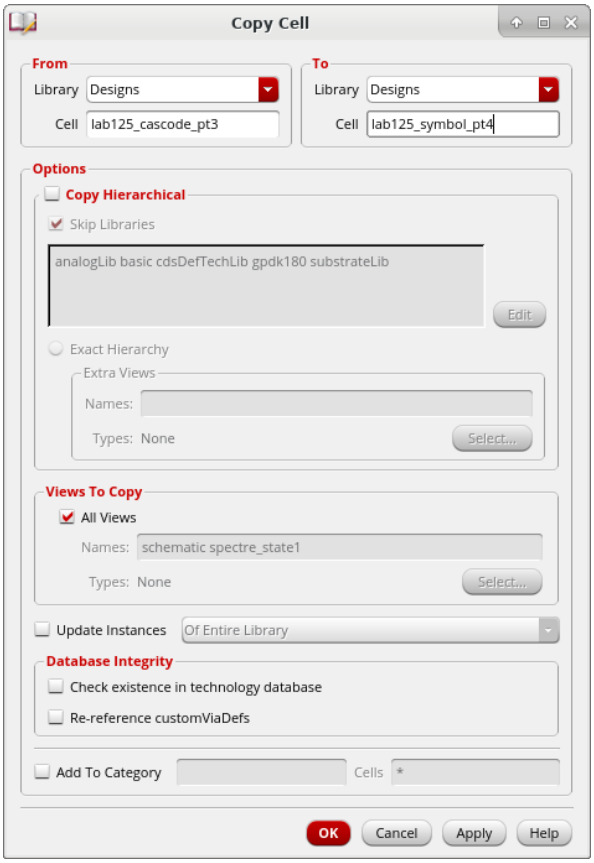
With guidance, you will

- Construct, instantiate and use in a simulation, a symbol for a current mirror circuit.

In electronic design, we will often want to abstract a complex schematic function or sub-function behind a symbol. This ensures only the interface to the function is made available to the rest of the design, with the complex (unnecessary) implementation details hidden away. This promotes the concept of the hierarchical design procedure (as opposed to a flat design), as a way of improving design reliability, manageability and reuse of components. In this part of the lab, we will look at the how we generate a symbol cellview for a given schematic.

7.1 Creating the Pinout of the Basic Current Mirror

(7.1.1) Make a copy of the basic current mirror schematic you created in the previous part of this lab (lab125_cascode_pt3). To create a copy from an existing schematic, go to the Library Manager window and select the cell you want to copy – in this case, lab125_cascode_pt3 from the Designs library. Right-click on the cell and select 'Copy…' from the context menu to invoke the 'Copy Cell' form. (You could also select **Menu: Edit à Copy** from the menu bar.)

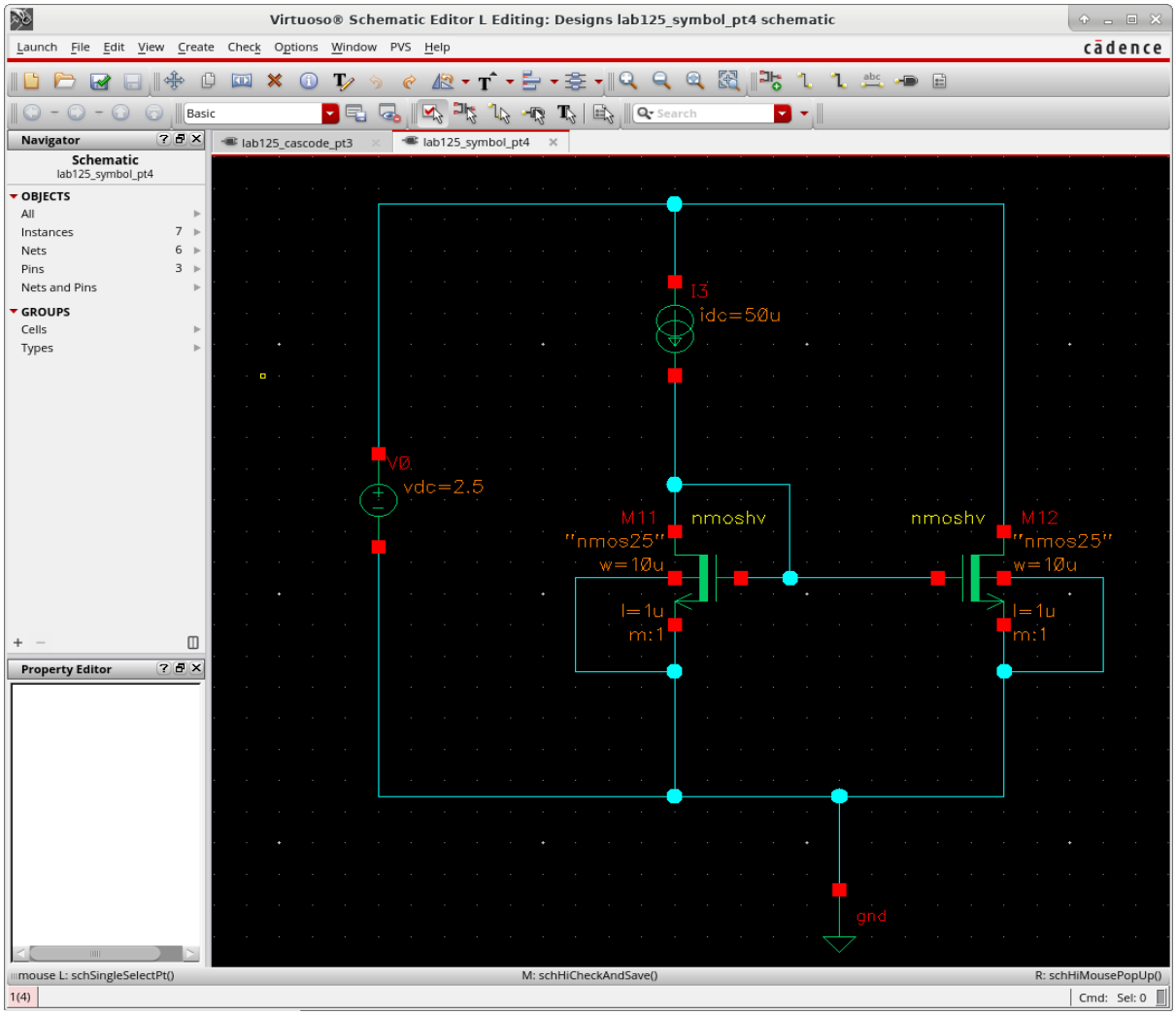


< Figure 154: The Copy Cell Form >

In the 'From' section of the copy cell form, select the source library and cell name and in the 'To' section choose the destination library and cell name (in this lab, use the Designs library in both the From/To sections). Give the new cell a meaningful name e.g., lab125_symbol_pt4. Leave the rest of the form unchanged and OK the form. In the library manager, in the Designs library, you should now see a new schematic cellview called 'lab125_symbol_pt4' (or whatever name you chose). See Figure 1.

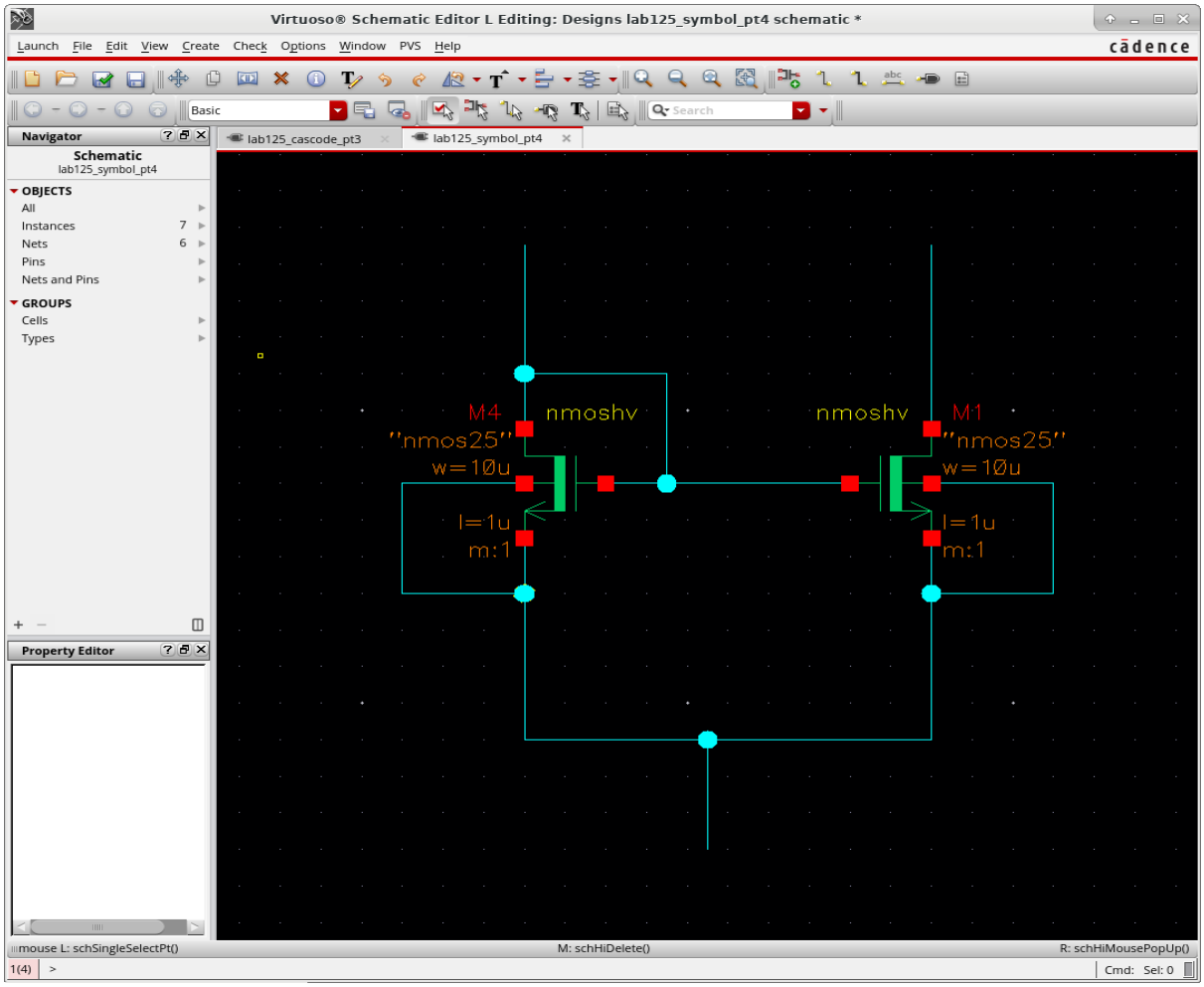
(7.1.2) **Note**: An alternative method of copying a schematic cell view, is to save a copy from the original schematic (lab125_cascode_pt3). In the Schematic Editor window from **Menu: File à Save a Copy…**, specifying the library, cell name, and cell type of the new cellview (e.g., lab125_symbol_pt4). Note, that in this case, once you have saved, a copy you remain working in your original schematic – you do not automatically switch to the new copy. You should see the new copy in the Library Manager – close the original (current) schematic and open the new schematic cellview.

(7.1.3) In your new schematic editor window, delete the cascode current mirror created previously. You should now only have the basic current mirror circuit in your schematic (lab125_symbol_pt4). See Figure 2.



< Figure 155. Schematic of the Basic Current Mirror. 10um/1um. >

(7.1.4) Delete the sundry/support components in the schematic – the 2.5V DC voltage supply (**vdc**), 50uA DC current reference (**idc**), and the ground reference (**gnd!**). See Figure 3.



< Figure 156. Schematic of the Basic Current Mirror with sundry/support components removed. >

(7.1.5) Add pins for the input, output, and ground (input/output) connections of the mirror circuit. From the schematic window, invoke the Create Pin form: use **Menu: Create à Pin...**, the '**Create Pin**' icon-button on the toolbar at the top of the schematic window, or the keyboard shortcut 'P'. You can also right-click anywhere in the empty space of the design canvas to invoke a context-menu and select the **Add Pin…** command from there.

When creating a pin you need to specify the name, direction, usage, and signal type of the pin. From the user manual,

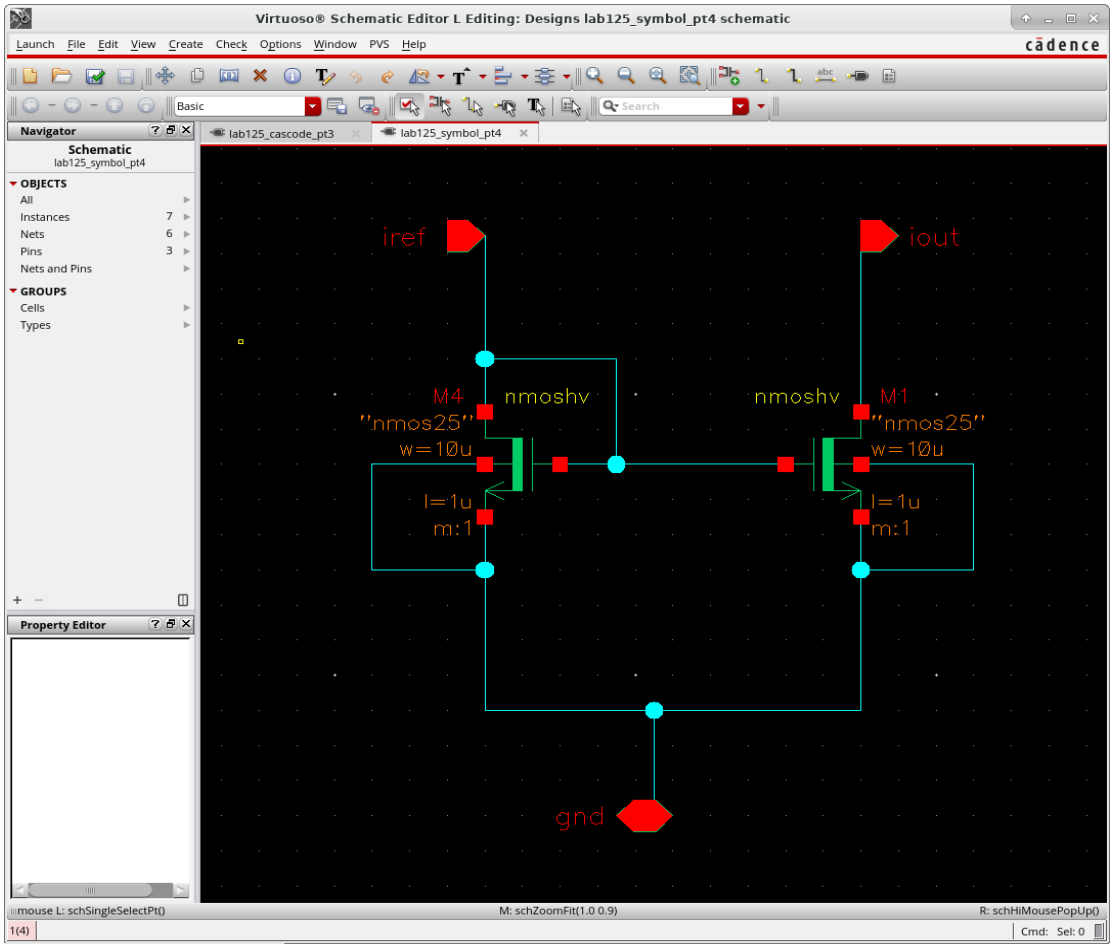
- **Names** specifies the names for the pins you want to add to the design and the order in which you want to place them. You can add more than one pin at a time as long as they are of all the same type/direction. To add more than one pin name, use a space to separate each name.

- **Direction** contains a drop-down list of the directions you can specify for the pins you want to add. For AICD, we will use only input pins, output pins, and inputOutput pins (for bidirectional connections such as for ground). Other options are available for specific applications.

- **Usage** contains a list of the pin types you can specify for the pins you want to add. The default is *schematic*. Use *offsheet* only when you are going to connect the added pins to another sheet in a multisheet schematic.

- **Signal Type** specifies the new pin's signal type. For this unit we will use 'signal' and 'ground' types only. Signal type setting is important in the areas of design verification, consistency checking for mixed-signal designs, and to also set routing constraints. Signal types to select from are *analog, clock*, *ground*, *power*, *reset*, *scan*, *signal* (default), *tieHi*, *tieLo*, *tieOff*, *optical*, *singleModeOptical*, or *multiMod*.

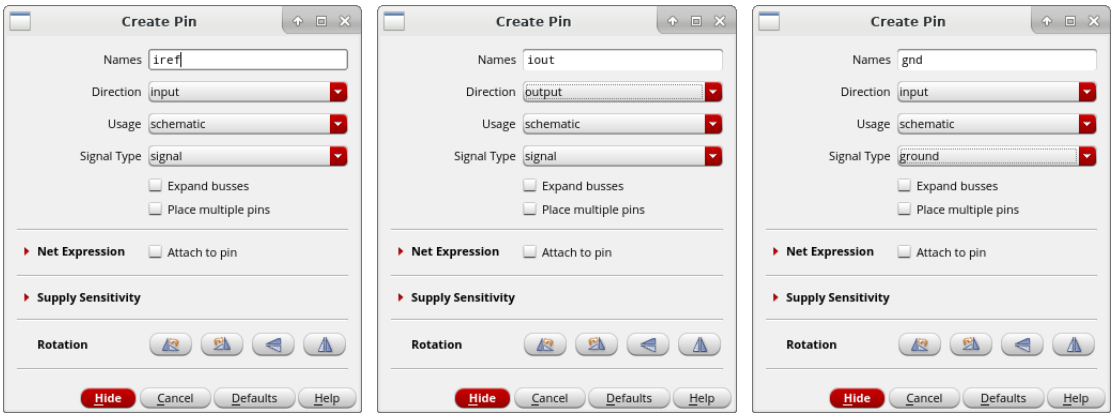
For further details of the Create Pin form, click the 'Help' button at bottom right corner of the form.



< Figure 157. Schematic of the Basic Current Mirror with IO and Ground Pins. >

(7.1.6) For the current reference branch of the mirror (the control branch), use an '*input*' pin, of '*signal*' type, and name it '***iref***'. For the output branch of the mirror (the controlled branch), use an '*output*' pin, of '*signal*' type, and name it '***iout***'. For the ground connection of the mirror, use an '*inputOuput*' pin, of '*ground*' type, and name it '***gnd***'. All pin usage types should be set to '*schematic*'. See Figures 4 and 5.

(7.1.7) Check and save you schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.



< Figure 158. Create Pin Forms for Input, Output, and Ground >

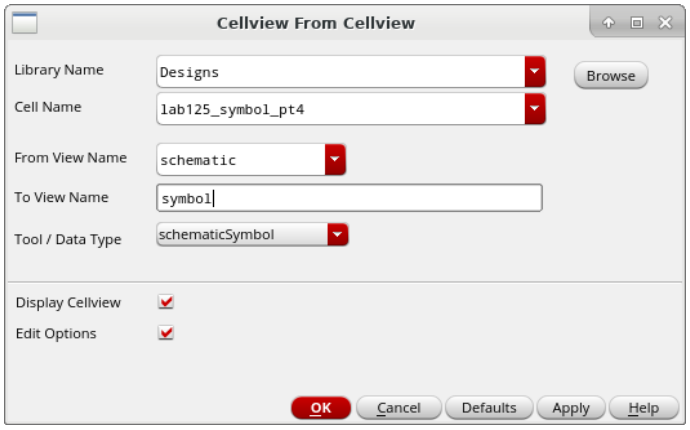
7.2 Generating the Symbol Cellview

(7.2.1) You should now have the completed schematic of the current mirror pinout as shown in Figure 4. It should check and save without any errors or warnings.

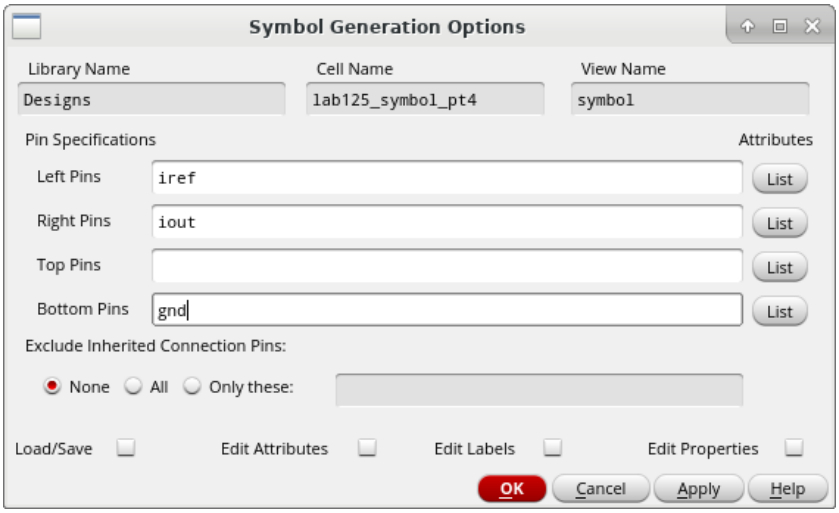
(7.2.2) From your schematic window (Figure 4), generate the symbol cellview using **Menu: Create à Cellview à From Cellview**… This will invoke the **Cellview From**. In this case, we will generate a symbol Cellview from a schematic cellview. The form may already be auto-filled in.

In the Cellview form, select the library and cell name of the schematic for which we want to generate a symbol. For the 'Library Name' select *Designs*, for the 'Cell Name' select *lab125_symbol_pt4*. Set the 'From View Name' to *schematic*, and the 'To View Name' to *symbol*. For the 'Tool / Data Type' select *schematic Symbol*. OK the form. See Figure 6.

When you OK the form, the **Symbol Generation Options form** will open. This is where you can set the initial properties and specifications of the symbol pinout. The pertinent aspect of this form are the pin positions on the symbol: left, right, top, and bottom. In schematic layout we generally arrange the pins on a component assuming a signal flow from left to right, and a current flow from top to bottom. In the associated text fields, put the *iref* pin (input) on the left, put the *iout* pin (output) on the right, and put the ground pin (inputOutput) on the bottom. Leave all other properties unchanged. OK the form. After a short while the symbol editor should open (there may be a slight delay). See Figure 7 and 8.



< Figure 159. Cellview From Cellview Form >



< Figure 160. Symbol Generation Options Form >

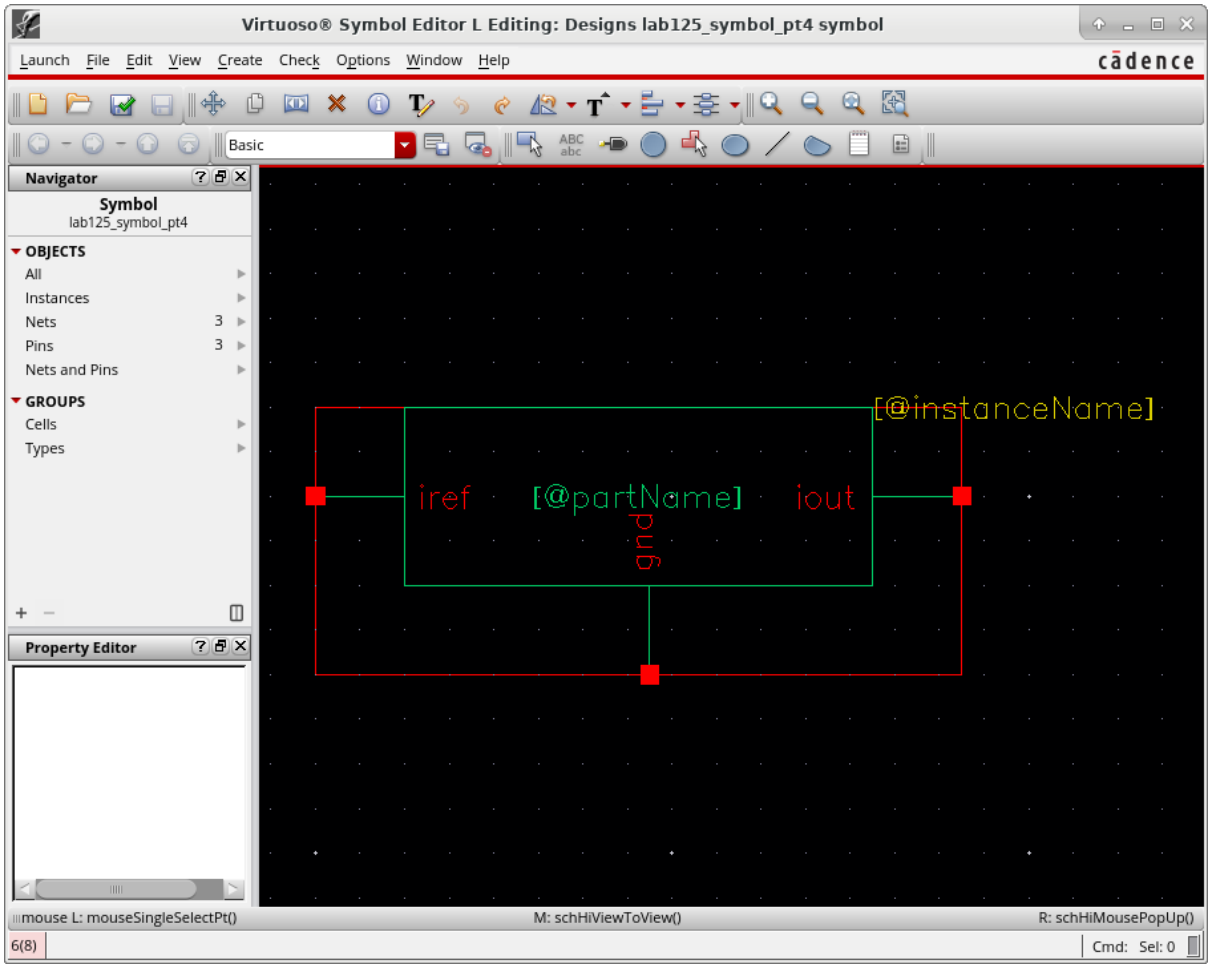
(7.2.3) Note: You can place multiple pins on a side using a name list separated by spaces.

(7.2.4) The symbol editor allows you to create and/or modify the symbol appearance. You can add or modify the basic shape of the symbol using lines/arcs, circles/ovals, rectangles, and polygons. Add or edit symbol pins. Add or edit symbol labels. Add or edit the symbol boundary/selection-box (the red outline box). Add or edit the part name and instance name display. Add/edit notes for documentation. And more.

One word of warning when adding or editing the symbol pins. The pin attributes were set-up in the schematic (using the create pin command). The symbol was then generated from the schematic. The number and properties of the pins (name, type, direction, and usage) of the symbol must match the number and properties of the pins defined in the schematic. If you change the pin definitions in the symbol view you must also update the schematic view to match – and vice versa. If the pin specifications do not match, a warning will be generated, and the simulation results may not be as expected.

The **@partName** and **@instanceName** are place holders for, respectively, the name of the cellview (in this case lab125_symbol_pt4), and the instance identifier that is automatically assigned to each device instantiated in a schematic (e.g., V1, R0, M0). Both of these @names will be set on the schematic at instantiation.

Finally, the red selection box around the symbol (Figure 8), sets the boundary of symbol when instantiated within a schematic. When you hover over or click on a symbol, the yellow/pink box that appears around the device is defined in the symbol editor by this red boundary / selection box. This also determines if a warning is generated if two symbols overlap (try overlapping to symbols and then check and save). You can delete the selection box altogether, but this is not recommended.



< Figure 161. Generated Symbol of the Basic Current Mirror. >

(7.2.5) You are free to modify the symbol as you wish – resizing, moving the names, adding shapes to the symbol – to observe the effect on the symbol layout. Remember to **check and save** your symbol (when it is first generated, and every time you make a modification).

7.3 Simulating the Symbol Cellview Using a Testbench

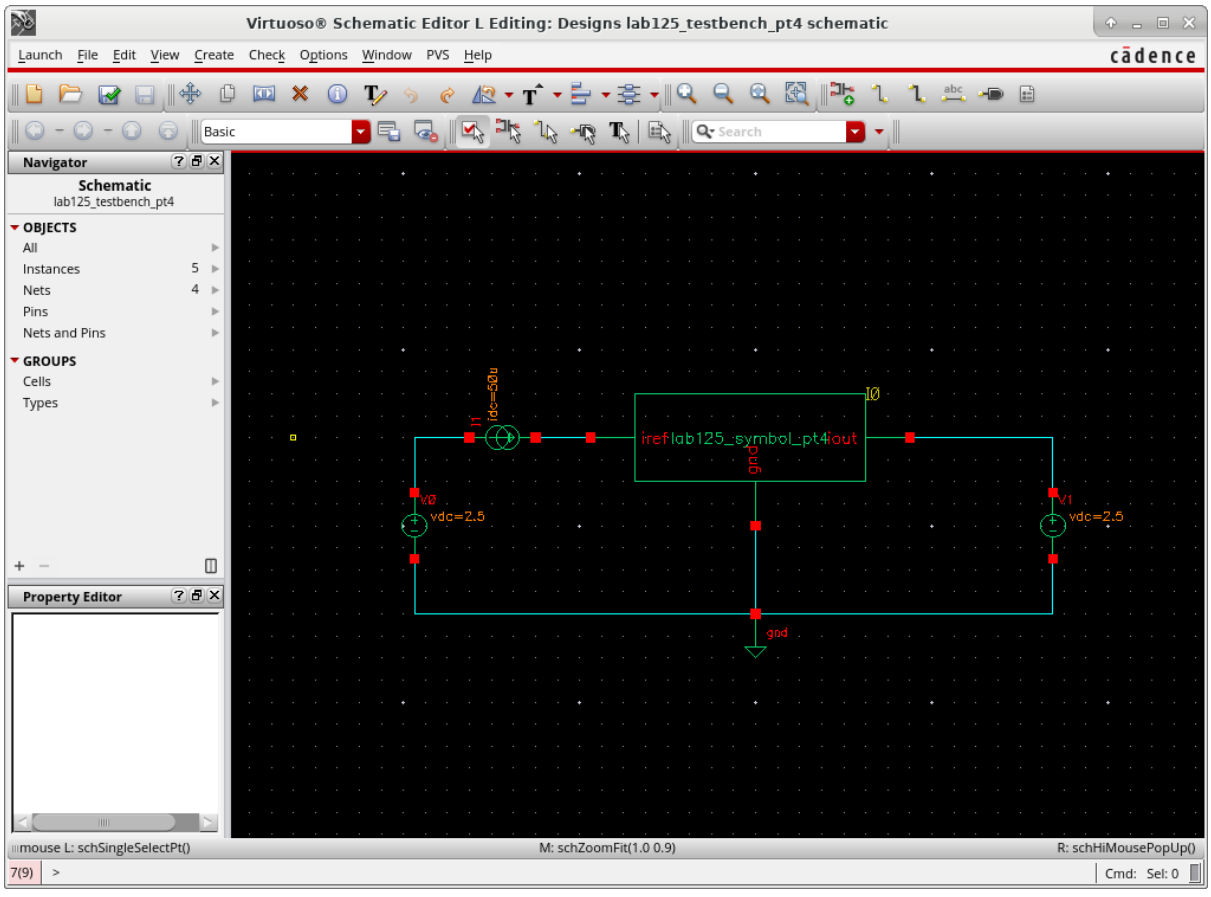
(7.3.1) In order to demonstrate the behaviour of the basic current mirror symbol view, we will create a testbench.

(7.3.2) Create a new schematic. Use the New File form as you have done previously – ensuring you are in your Designs library – giving your new schematic a meaningful name such a 'lab125_testbench_pt4'.

(7.3.3)) Using the Add Instance form, in your testbench schematic editor window, create the circuit given in Figure 9. Instantiate two ideal DC voltage sources (**vdc**) and one ideal DC current source (**idc**) from the analogLib library. Also, instantiate one instance of the symbol you have just created – this will be the symbol view of part *lab125_symbol_pt4* in your *Designs* library.

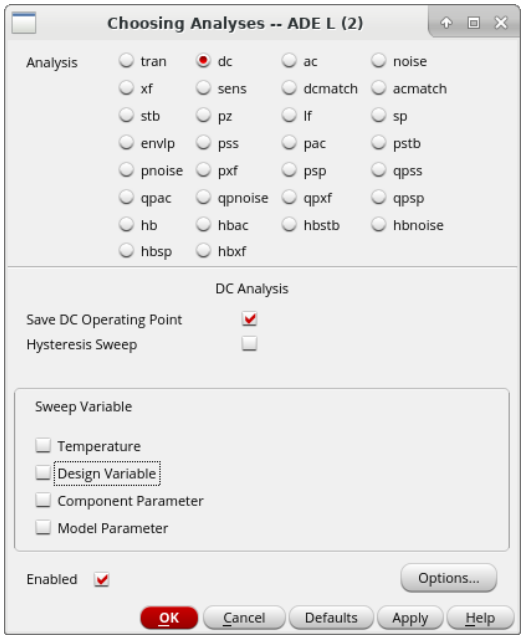
Set the DC supply voltages to 2.5V (**DC Voltage** = 2.5) and set the DC current reference to 50uA (**DC current** = 50u).

Note: In Figure 9, we are using the second 2.5V **vdc** as both the supply to the output branch of the mirror and as a current sink. The latter not strictly necessary, but it simplifies the layout of the testbench.

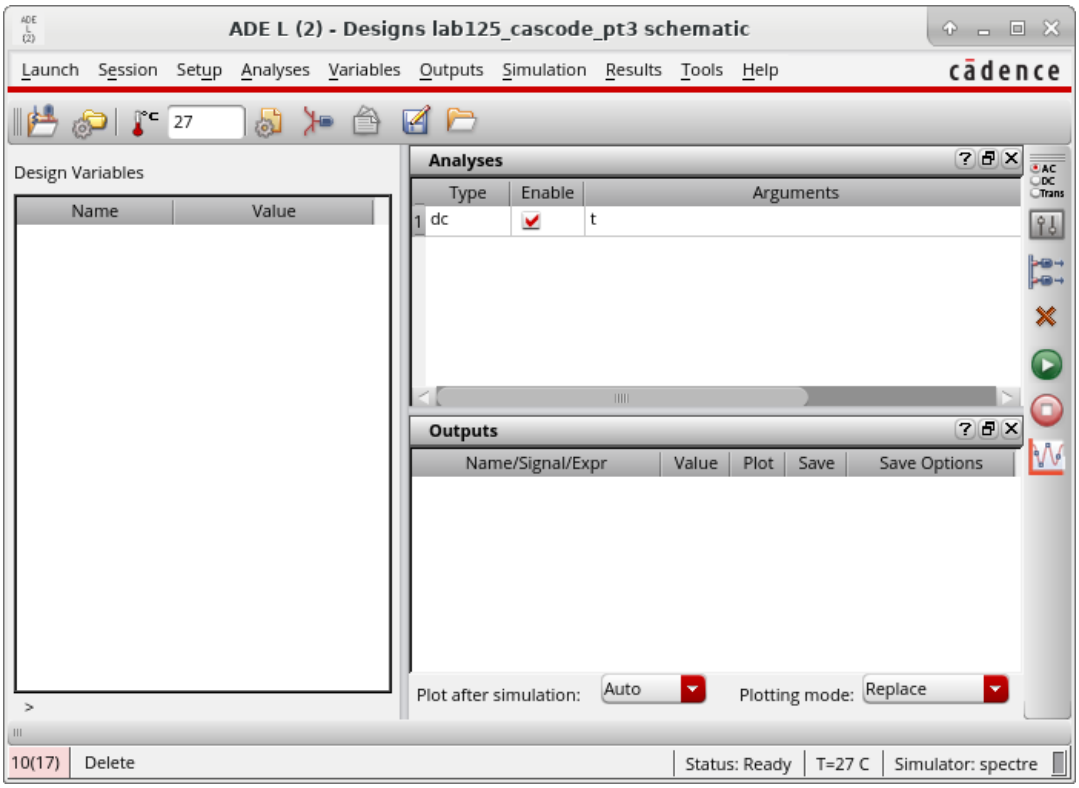


< Figure 162. Testbench for the Basic Current Mirror Symbol. >

(7.3.4) We will simulate the current mirror using a DC Analysis. From the Schematic Editor window, launch the Analogue Design Environment (ADE) from **Menu: Launch à ADE L**.



< Figure 163. The Choosing Analyses Form. Only DC Operating Point selected. >



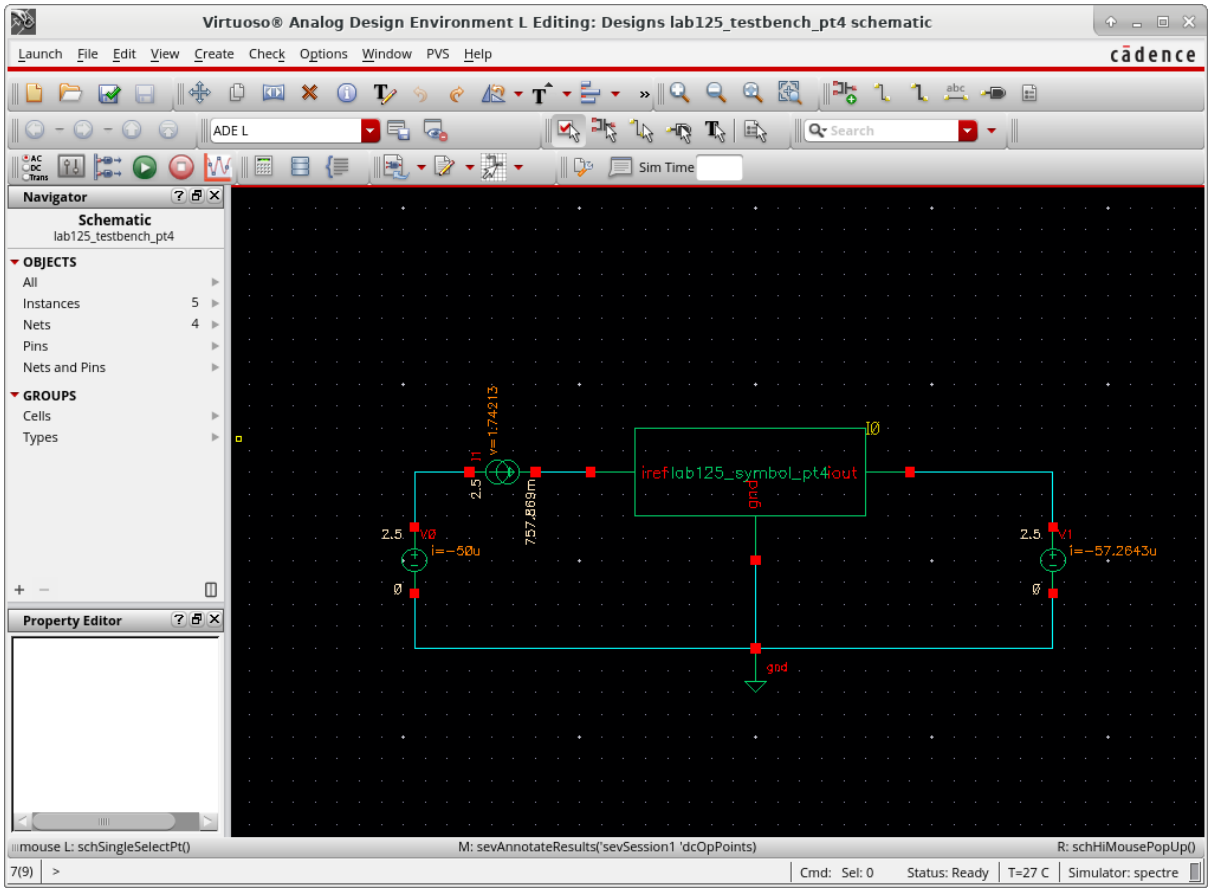
< Figure 164. The ADE for a DC operating Point only Analysis. >

(7.3.5) From the ADE window, go to **Menu: Analyses à Choose…** or use the **Choose Analyses** button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. On the form, select the '**dc**' option button. Check the 'Save DC Operating Point' box. OK the form.

(7.3.6) We are only simulating for the DC operating point, so we do not need to select any outputs to be plotted.

(7.3.7) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. The text output window should appear but there will be no waveform window.

(7.3.8) From the ADE window choose **Menu: Results à Annotate à DC node voltages…** If you now look in the schematic window, you should see the node voltages annotated directly on the schematic. Again, from the ADE window, choose **Menu: Results à Annotate à DC operating points…** You should now see (along with the node voltages) additional DC operating point information annotated on the schematic including the DC bias currents.



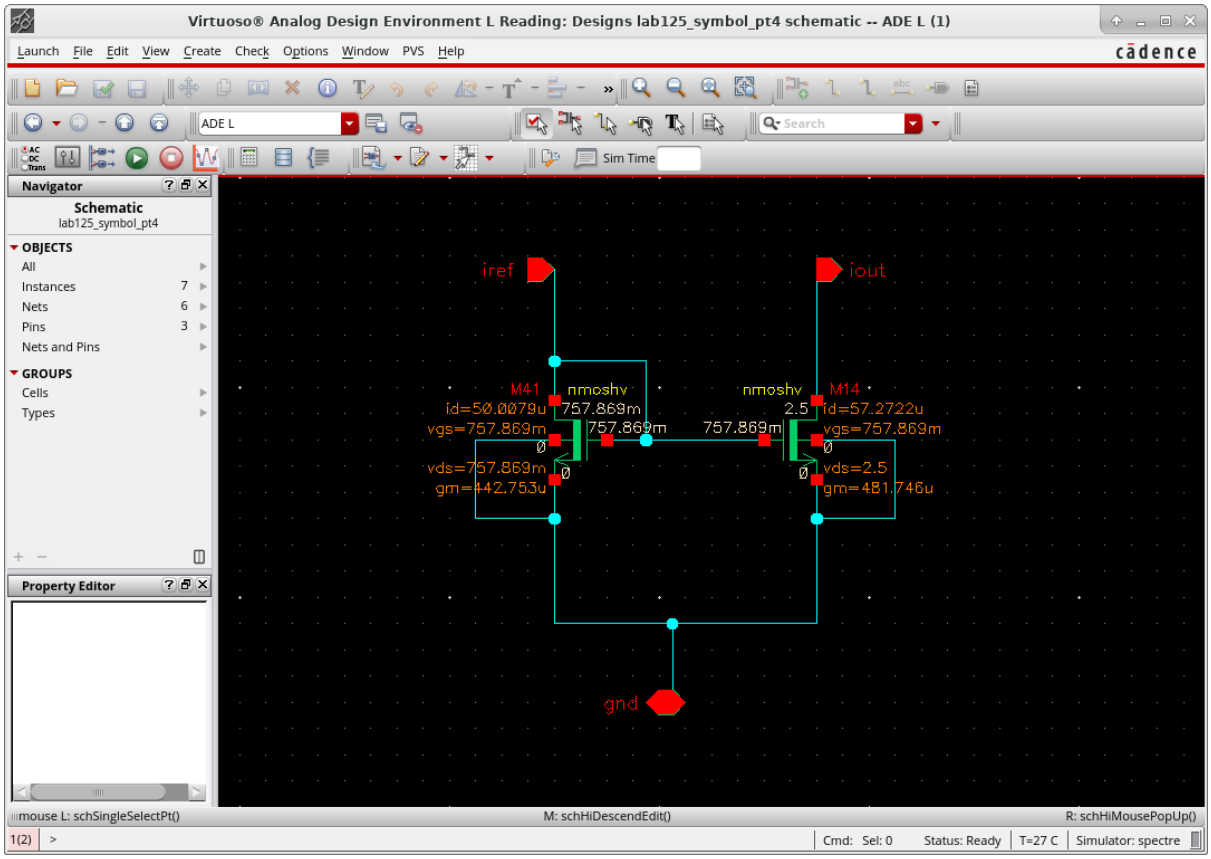
< Figure 165. DC Analysis – Save DC Operating Point. Annotated Testbench for the Basic Current Mirror Symbol. >

(7.3.9) You can look beneath the symbol abstraction by right-clicking on the symbol and choosing **Descend Edit** (keyboard shortcut shift+E) or **Descend Read** (keyboard shortcut E). You can also select the symbol and use **Menu: Edit → Hierarchy → Descend Edit**… or **Menu: Edit → Hierarchy → Descend Read**… This will invoke the Descend form, where you can choose to open the underlying schematic in the current window tab or in a new window tab.



< Figure 166. The Descend (into sub-level) Form. >

Note that Descend edit allows you to edit the schematic beneath the symbol when you descend into it (this can be convenient when you want to make quick changes to a symbol and have it applied to every instance in the schematic). Descend read does not allow you to modify the schematic when you descend into it (this is useful if you don't want to change the schematic or if a schematic is protected i.e. read only).



< Figure 167. Descended View of the Basic Current Mirror Symbol. >

If you descended into the symbol within the current window tab, you can return to the schematic one level above by right-click on the blank canvas and selecting the **Return** command (keyboard shortcut ctrl+E). You can also use **Menu: Edit → Hierarchy → Return**… If you have descended several levels, you can return straight to the top-level module using **Menu: Edit → Hierarchy → Return To Top…**

(7.3.10) **Discussion**. Note that the DC voltages and currents are (almost) the same as in the original simulation (Figure 34) – VGS = 0.757869V (0.757851V) and IDS = 57.2643uA (57.2636uA). The slight difference in the final two digits occurs since we are using two separate **vdc**'s for the supply in testbench, as opposed to one **vdc** in the original simulation.

(7.3.11) Finally, now that you have a hierarchical (multi-level) schematic, you have to make sure when checking and saving that all levels are properly checked/saved. If a lower level (within a symbol) isn't checked and saved, even if the top-level module is, you will get an error when trying to simulate.

Warning: making a change in a subcircuit schematic results in a change in every instantiation of its symbol.

End of Lab!

Tutorial Lab 12.6 (Formatively Assessed):

The FET Differential Amplifier

Preliminary Matter: Setting up the Cadence Environment

Please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout before entering these commands into your terminal.

For this course, you will use the Cadence Virtuoso Design Environment (referred to simply as Cadence). Cadence is a Linux based software suite for designing full custom integrated circuits. In order to use Cadence you will first need to set-up the Cadence environment.

5. Open a terminal window and ensure that you are in your home directory using the following command:

```
cd ~
```

Note: 'cd' stands for change directory. The solitary tilde symbol (~) is a shortcut/alias for your home directory (/home/username).

6. Under your home directory, use the following commands to load the Cadence environment:

```
module use /eda/cadence/modules
module load course/EENGM6030
```

Note: The two 'module' commands load the required shell/Cadence environmental variables for this course (paths, names, aliases…) In particular, this includes a reference to the process design kit (PDK) used in this course, the GPDK180. You will need to do this every time you open a terminal, since the sourced environment only applies to the current terminal (until it's closed).

7. The AICD folder will be used for all your Cadence related activities for this course. **You should always make sure you are in your AICD folder before you launch Cadence.** To run cadence, use the following commands:

```
cd ~/AICD
virtuoso &
```

.

1. Learning Objectives of this Lab:

From the previous lab, you should:

- Understand the concept of channel length modulation and the dependency of the MOSFET output impedance on channel length.

- Understand the concept of and rationale for cascoding. Apply cascoding to voltage amplifiers and current mirrors in order to increase output impedance and hence reduce the dependency of output current on output voltage.

- Understand the need for a symbol cellview. Generate the symbol cellview for a schematic. Create and simulate a testbench for a user generated symbol. Descend into and out of a symbol cellview.

In this lab, you will:

- Create, and simulate in terms of DC-sweep, a CMOS differential voltage amplifier with resistive and current source loads.

- Understand the need for feedback to set the DC operating point of a fully differential amplifier.

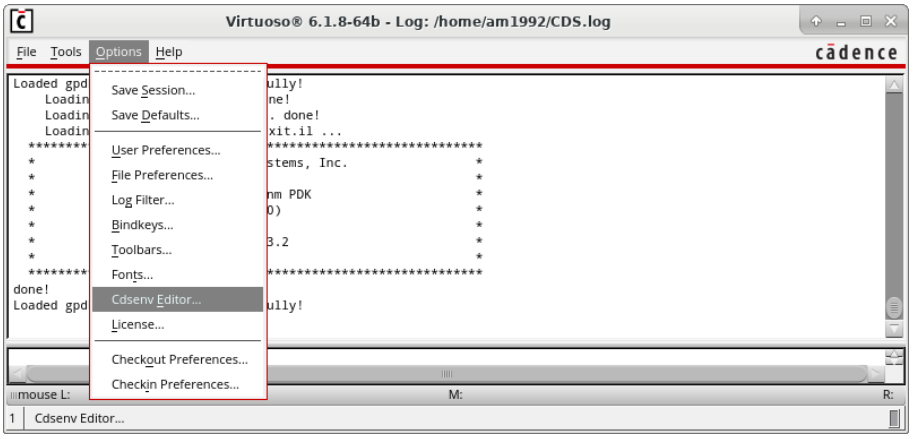
- Create, and simulate, a CMOS FET differential-to-single-ended transconductance amplifier (TCA) with a current mirror load and PFET input devices.

2. Cadence Documentation, Help, and Licensing

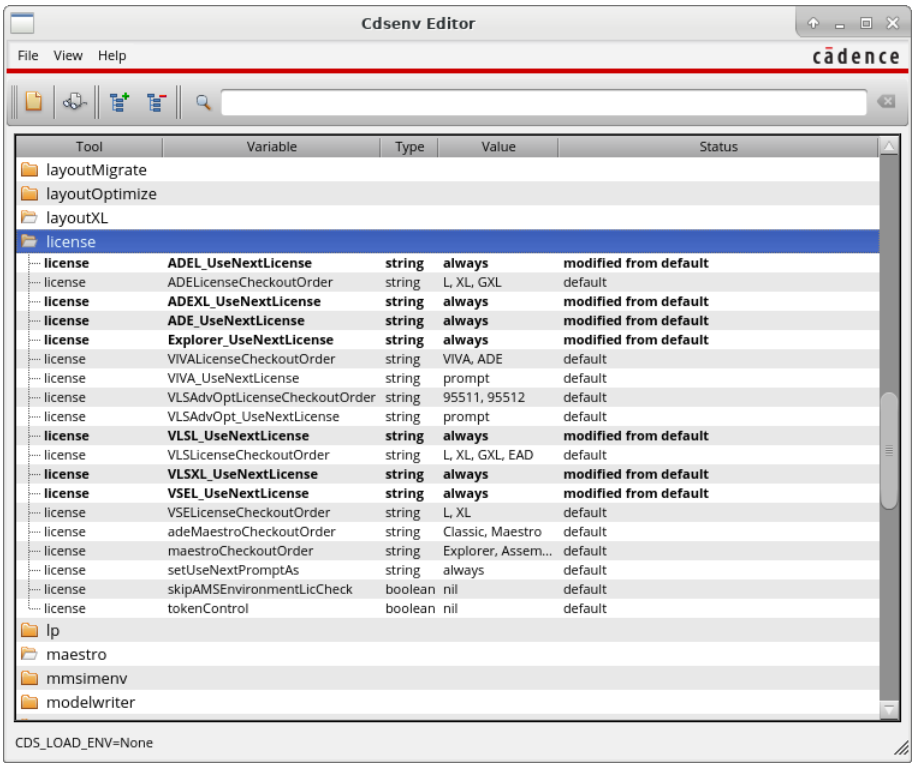
Local and online help is available from the help menus (**Menu: Help → User Guide** or by pressing **F1**), or by typing 'cdnshelp &' at the command prompt (assuming you have setup your terminal environment).

PLEASE NOTE: WHEN LAUNCHING A TOOL IN THE CADENCE DESIGN ENVIRONMENT YOU MAY SOMETIMES GET A WARNING ABOUT A LICENSE BEING UNAVAILABLE FOR THE TOOL, AND REQUESTING PERMISSION TO OBTAIN A HIGHER-TIERED LICENSE. YOU SHOULD SELECT 'ALWAYS' TO THESE MESSAGES.

If you accidently deny a license request, or you cannot access a license, you will need to modify the Cadence environment licence/permissions from the CIW. From the CIW menu go to **Menu: Options → Cdsenv Editor**. Once in the environment editor, go to the license folder, and make sure all licenses (in bold) are set to '**always'** – if any entries are listed as '**never**' you will need to manually change/retype them to '**always**'.



< Figure X.1. The Cadence Environment Menu Option from the CIW Window. >



< Figure X.2. The Cadence Environment Editor. Note, all licenses should be set to 'always'.>

3. Getting Started with Cadence:

For a refresher/introduction on how to start a Cadence session please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout.

3.1 Dealing with Lock files

PLEASE NOTE: IF A SCHEMATIC ISN'T PROPERLY CLOSED DOWN (FOR EXAMPLE, DUE TO A DROPPED REMOTE CONNECTION), THE SCHEMATIC MIGHT BECOME EDIT LOCKED. IF THIS HAPPENS YOU WILL NOT BE ABLE TO OPEN THE FILE FOR EDITING. YOU WILL ONLY BE ABLE TO OPEN THE FILE FOR READING UNTIL YOU REMOVE ANY/ALL LOCK FILES.

There are two ways you can unlock files:

(4) The first is to manually delete any edit lock files. To do this, go to your '~/AICD/Designs' folder and find the folder representing the schematic e.g., for lab1 this would be '~/AICD/Designs/lab121_vdivider/schematic' if this is what you called your schematic. Delete all the .cdslck lock files in this folder and you should be able to open your schematic for editing.

(5) You can also automatically release edit locks with the command:

```
clsAdminTool -are .
```

To use this command, you have to be in your AICD folder (the full-stop is shorthand for the current directory path). If you are not in your AICD folder, you have to specify the path (relative or absolute) to your AICD directory.

Since this is a Cadence command, you have to ensure that you have set up the terminal environment for Cadence (in particular, the modules must be loaded as in step 2, 'Preliminary Matter'). It is best to use the terminal window that you used to start Cadence, since the environment was already set up. Note that you will not be able enter any commands into your terminal if you did not start cadence with the '&' character. By example, to release any lock files after opening a new terminal, the full command set would be,

```
cd~
module use /eda/cadence/modules
module load course/EENGM6030_2021
clsAdminTool -are AICD
```

For more information on this command, use

```
clsAdminTool -help
```

4. Circuit Simulation Exercises: Part 1

With guidance, you will:

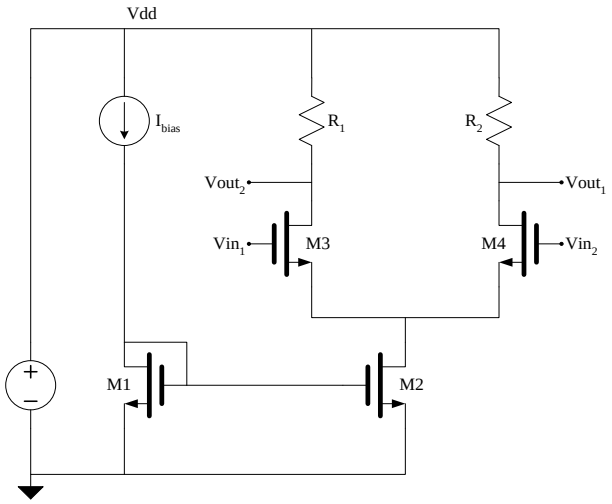
- Create, and simulate in terms of DC-sweep, a CMOS differential voltage amplifier with resistive and current source loads.

Differential amplifiers are fundamental components in Analogue IC design. At their simplest, these circuits amplify the difference between their inputs (the differential mode of operation) and reject/supress what is common between their inputs (the common mode of operation). Hence, interfering signals – internal or external – can be eliminated where they are common to both inputs of the amplifier. This gives differential amplifiers an advantage in sensing and test-and-measurement applications.

Differential amplifiers are most commonly differential-input/differential-output or differential-input/single-ended-output. Single-ended-input/differential-output amplifiers do exist but are usually conFigured from fully differential amplifiers by grounding one of the inputs. For a more detailed overview of differential amplifiers refer to sections 4.4 (Balanced-Output, Balanced Input Amplifiers) and 4.5 (Single-Ended Output Amplifiers and Operational Amplifiers) of the background material.

4.1 The Basic Resistor Loaded Balanced Amplifier

(4.1.1) Create the schematic for the basic balanced amplifier circuit in Figure 1. A summary of the circuit's operation is given in section 4.4 of the background material.

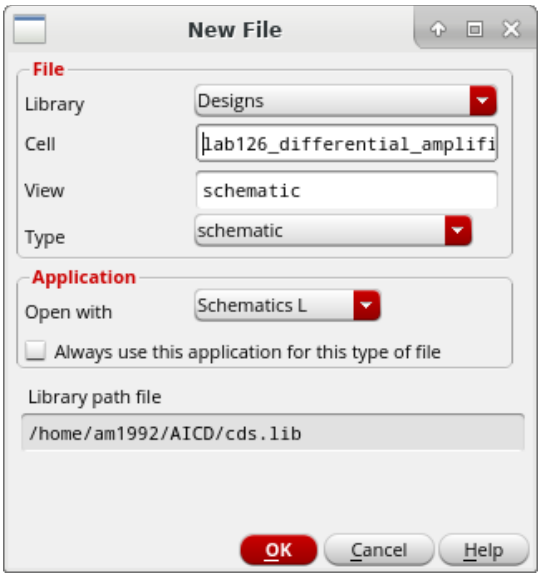


< Figure 168. The Basic Resistor Loaded Balanced Amplifier.
Reproduced from Figure 21, section 4.4 of the background material. >

(4.1.2) From the Library Manager window, select the 'Designs' library and choose **Menu: Files à New à Cell View** to invoke the New File form (Figure 2).

In the 'Cell' text field, choose a meaningful cell name (e.g., lab126_differential_amplifier). In the 'View' text field, type 'schematic' as the view name and from the 'Type' drop down menu choose 'schematic' as the view type. In the 'Application' section of the dialog box, choose 'Schematics L' from the 'Open with' drop down menu. OK the form.

(4.1.3) You should be presented with a blank **Virtuoso Schematic Editor L** schematic entry window for the new cellview.



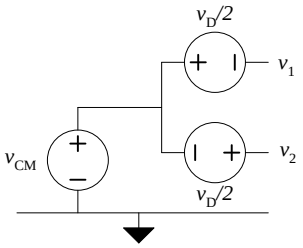
< Figure 169. The New File Form >

(4.1.4) In your schematic editor window, from the GPDK180 library, instantiate and place four **nmoshv** NMOS FETs (two for the input pair, and two for the tail bias current mirror).

*Recall, there are several ways of rotating/flipping a part. When adding an instance to your schematic you can rotate and/or flip the item before you place it on the canvas using the **Rotate**, **Sideways**, and **Upside-Down** buttons on the Add Instance form. Alternatively, you can rotate/flip the components after placing them by selecting the component and using the **rotate** or **flip** commands from the menu/toolbar. You can also flip or rotate a component by using **F3** with the **move/stretch** commands. In general, pressing F3 while a command is in progress gives you a more detailed set of options for that command. When moving an item using the **Move** or **Stretch** commands, pressing F3 will bring up a dialog box with more detailed options including **Rotate**, **Sideways**, and **Upside-Down** buttons.*

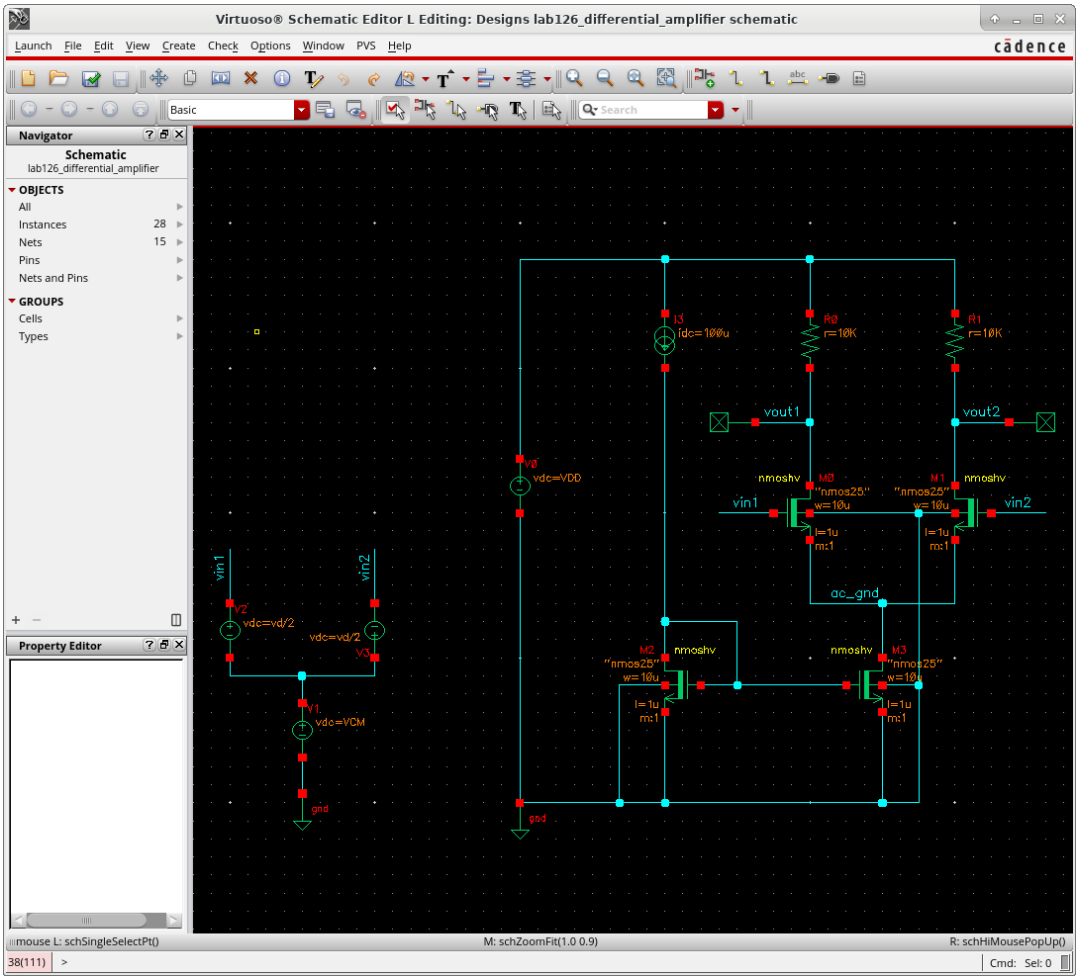
(4.1.5) Instantiate and place an ideal **vdc** source for the supply voltage, an ideal **idc** current source for the tail current reference, and two ideal resistors (**res**) for the balanced loads. All ideal components are from the analogLib library.

(4.1.6) Finally, to drive the circuit, use a single **vdc** for the common-mode input voltage (VCM) and add two split **vdc** inputs for to the differential-mode input voltage (v_D). This will enforce the common-mode voltage (CMV) principle rather than having separate VCMs driven by the same parameter for the two inputs. **Note the opposing orientation of the two v_D/2 vdc supplies – this gives a total voltage of v_D.** See Figure 3 (4).



< Figure 170. Common-mode voltage driver circuit. $v_D = (v_1 + v_2)$. >

(4.1.7) Parameterise your design. Set the tail current reference to 100uA (**DC Current** = 100u) For the DC supply voltage, set the voltage attribute to the string '*VDD*' (**DC Voltage** = *VDD*). For the input set the common-mode voltage component to the string '*VCM*' (**DC Voltage** = *VCM*), and the two differential-mode components to the string '*vd/2*' (**DC Voltage** = vd*/2*). Set both load resistors to 10KΩ (**Resistance** = 10K). Dimension all NMOS devices 10um/1um (**width** = 10u, **length** = 1u). Leave all other attributes unchanged.



< Figure 171. The Resistor-Loaded Differential Amplifier Schematic as in Figure 1>

(4.1.8) Wire the components as in Figure 4 using the **Wire (narrow)** command.

(4.1.9) Using the **Create Wire Name** command: name the two input nets vin1 and vin2 (the gates of the differential input pair), name the two output nets vout1 and vout2 (the drains of the differential input pair), and name the tail bias point ac_gnd (the drain of the tail device / the common sources of the differential input pair). Use the **noConn** connector from the **basic** library on the outputs to avoid floating-net warnings. Keep the schematic neat.

(4.1.10) To connect the inputs to the amplifier to the driver circuit, we will create an implicit connection using wire names. Name the outputs of the common-mode driver circuit vin1 and vin2. This will create a connection between the vin1 of the driver circuit and vin1 of the amplifier, and a connection between vin2 of the driver and vin2 of the amplifier. See Figure 4.

REMEBER: FOR AN NMOS DEVICE, YOU MUST ALWAYS CONNECT THE BODY/BULK TERMINAL TO THE MOST NEGATIVE NET (WIRE) OF THE CIRCUIT. IN FIGURE 4, THIS IS THE GROUND TERMINAL. FOR A PMOS DEVICE (NOT SHOWN) THE BODY/BULK TERMINAL MUST ALWAYS BE CONNECTED TO THE MOST POSITIVE NET (WIRE) OF THE CIRCUIT, USUALLY THE SUPPLY VDD. THIS IS NECESSARY TO ENSURE THAT THE CHANNEL WILL BE PROPERLY ESTABLISHED.

(4.1.11) Check and save your schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.

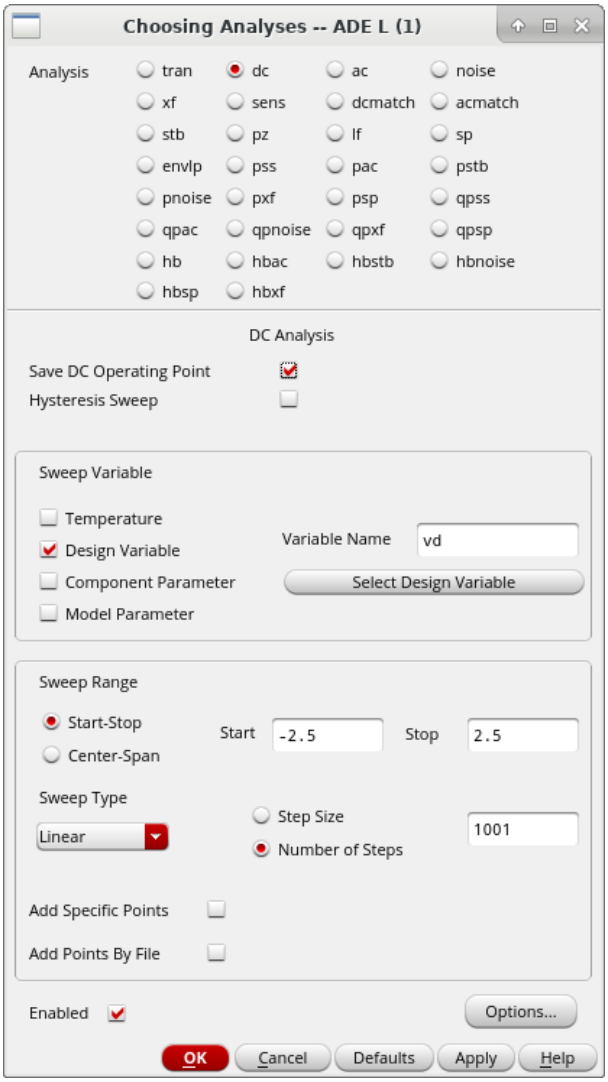
4.2 Simulating the Basic Differential Amplifier

(4.2.1) We will simulate the amplifier using a DC Sweep Analysis. From the Schematic Editor window, launch the Analogue Design Environment (ADE) – **Menu: Launch à ADE L**.

(4.2.2) Copy the design variables from the schematic into the ADE. For this circuit, these are the DC voltages from the four **vdc** components – 'VDD', 'VCM', and 'vd'. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the VDD, VCM, and vd variables in the 'Design Variables' list of the ADE. Initialise VDD *= 2.5V*, VCM *= 1.25V,* and vd = 0V. These values are used to set the DC operating point of the amplifier circuit.

(4.2.3) We want to plot the differential output voltage (vout1 and vout2) and the AC ground point (ac_gnd) as we sweep vd from -2.5 to 2.5V. From the ADE window, go to **Menu: Analyses à Choose…** or use the '**Choose Analyses**' button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open.

In the choosing analyses form: Select the '**dc**' option button. Check the 'Save DC Operating Point' box. Check the 'Design Variable' box as the sweep variable. Type vd as the variable name. Choose the 'Start-Stop' option button with Start = -2.5 and Stop = 2.5 for the sweep range. Select 'Linear' from the drop-down menu as the sweep type. Choose the 'Number of Steps' option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. See Figure 5. OK the form. The **dc** analysis should now appear in the 'Analyses' list of the ADE window.

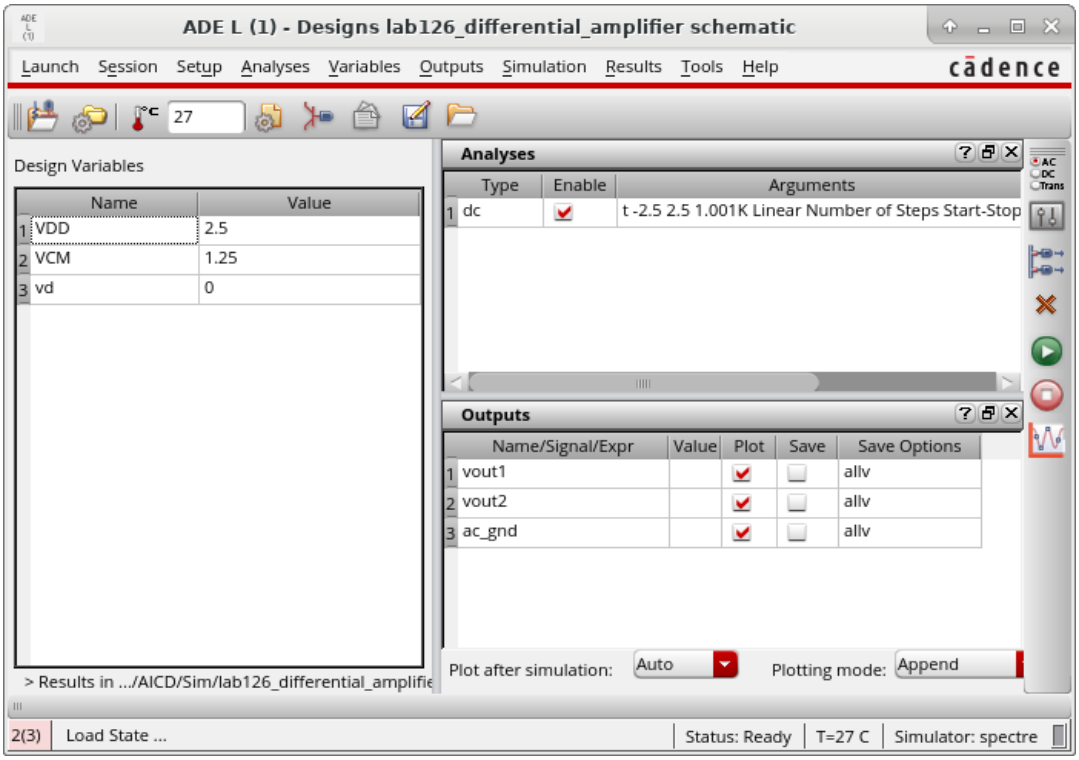


< Figure 172. The Choosing Analyses Form. >

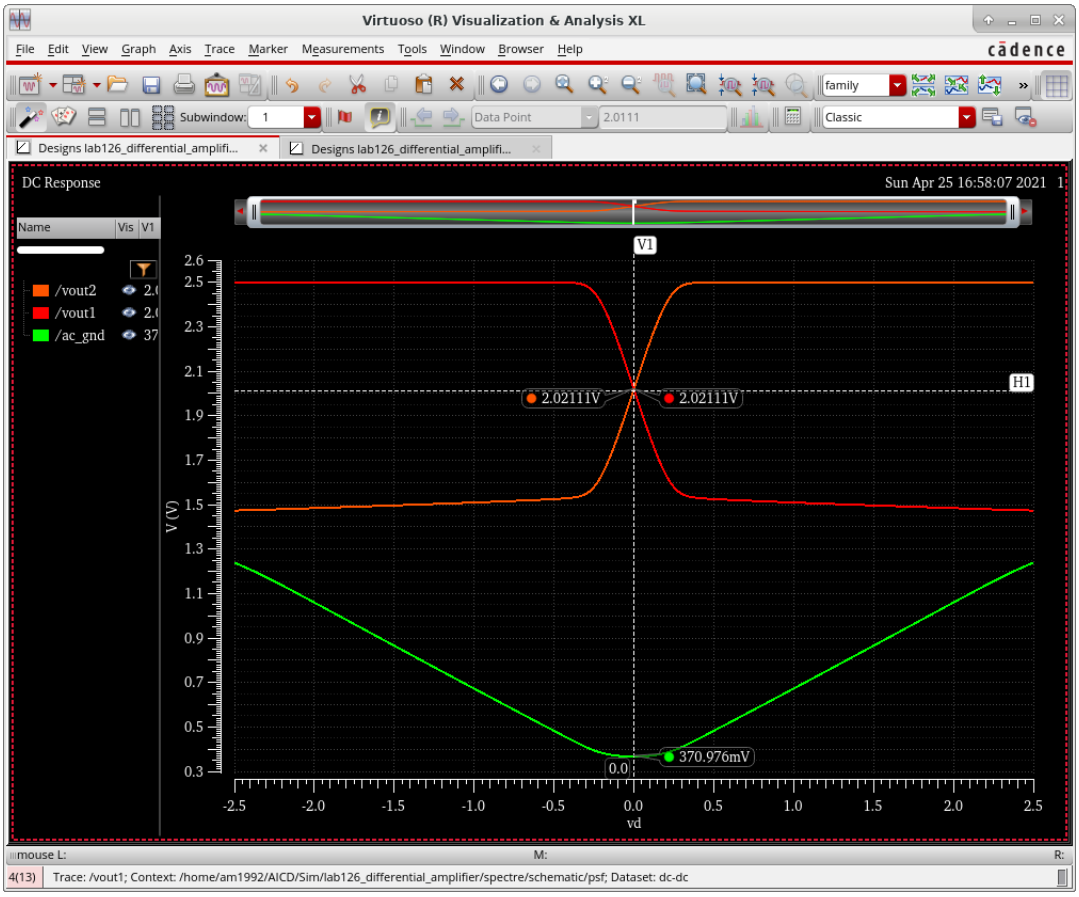
(4.2.4) Select the two output voltages (vout1 and vout2) and the AC ground point (ac_gnd) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**. Press the 'ESC' key to exit the plotting mode of the ADE**.

Recall, if you want to plot a particular voltage, you click on the net/wire. If you want to save and plot the currents, you have to click a terminal ***node*** *of a component (not the net/wire).*

(4.2.5) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. Both the text output window as well as the waveform output window should open automatically. See Figures 7 and 8.

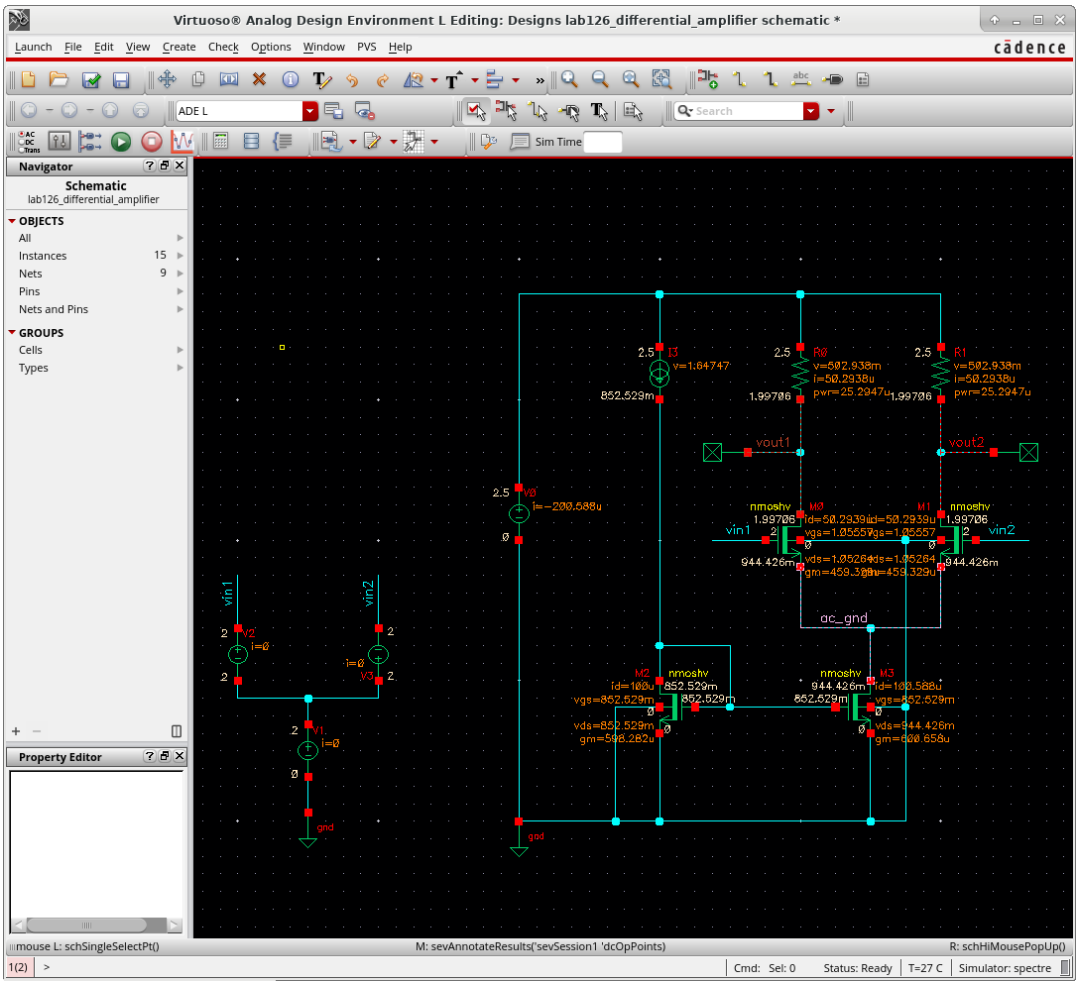


< Figure 173. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 174. Resistor-Loaded Differential Amplifier Output Voltages (vout1, vout2) and the AC Ground Point (ac_gnd). VCM=1.25. VCM=1.25V. VD=-2.5V to +2.5V >

(4.2.6) You can also annotate the schematic, **Menu: Results à Annotate à DC node voltages…** and **Menu: Results à Annotate à DC operating points…** See Figure 6.



< Figure 175 (out of Figure numbering order). Annotated. Resistor-Loaded Differential Amplifier Schematic, this is actually showing the node voltages for a VCM = 2V as used below>

(4.2.7) **Discussion**. Referring to the plot of Figure 8. The differential output (vout1, vout2) sits on top of the common mode output voltage (V_{CMO}) – this is the DC operating point when the input voltage (vd) is zero.

When **vd=0V**, the tail current (100uA) is split equally between the two branches of the differential amplifier (50uA through each). In this case, the common-mode output voltage is 2.0111V (indicated by the vertical marker at the *vout1/vout2* cross point). This can be calculated from,

$$V_{OUT} = V_{DD} - I_{DS}R_L = 2.5 - \frac{1}{2}(100\mu A)10k\Omega = 2V$$

Where I_{DS} is the current in the tail device (the NFET that is biasing the differential amplifier, M3). In these equations we assume that is the same as the circuit bias current for simplicity.

When **vd > |0.3V|**, the tail current is steered almost entirely into one side of the differential amplifier. At one extreme, the maximum vout is defined by zero current through the load, in which case

$$V_{OUT} = V_{DD} - I_{DS}R_L = 2.5 - (0)10k\Omega = 2.5V$$

At the other extreme, the minimum vout is defined (mostly) by the maximum voltage drop across load when all of the bias current is drawn through the load, in which case

$$V_{OUT} = V_{DD} - I_{DS}R_L = V_{DD} - (100uA)10k\Omega = 1.5V.$$

In analogue design, it is the transitory region around vd=0V in which we are interested. This is the analogue region of operation. In this region, when **vd < |0.3V|**, we see the typical input/output voltage transfer curve that we have previously studied with the single-ended amplifier designs. However, in this case, we have two complementary outputs – the inverting output and the non-inverting output. (Which output is inverting/non-inverting depends on which input you are comparing it to – the output on the same side as the input is inverted with respect to that input. The output on the opposite side to the input is non-inverting with respect to that input.)

The slope of the transfer curve defines the gain of the amplifier. As with the single-ended amplifier design, the differential gain is given by the product of the transconductance (g_m) of the input devices and the load resistance,

$$A_V = 2g_mR_L$$

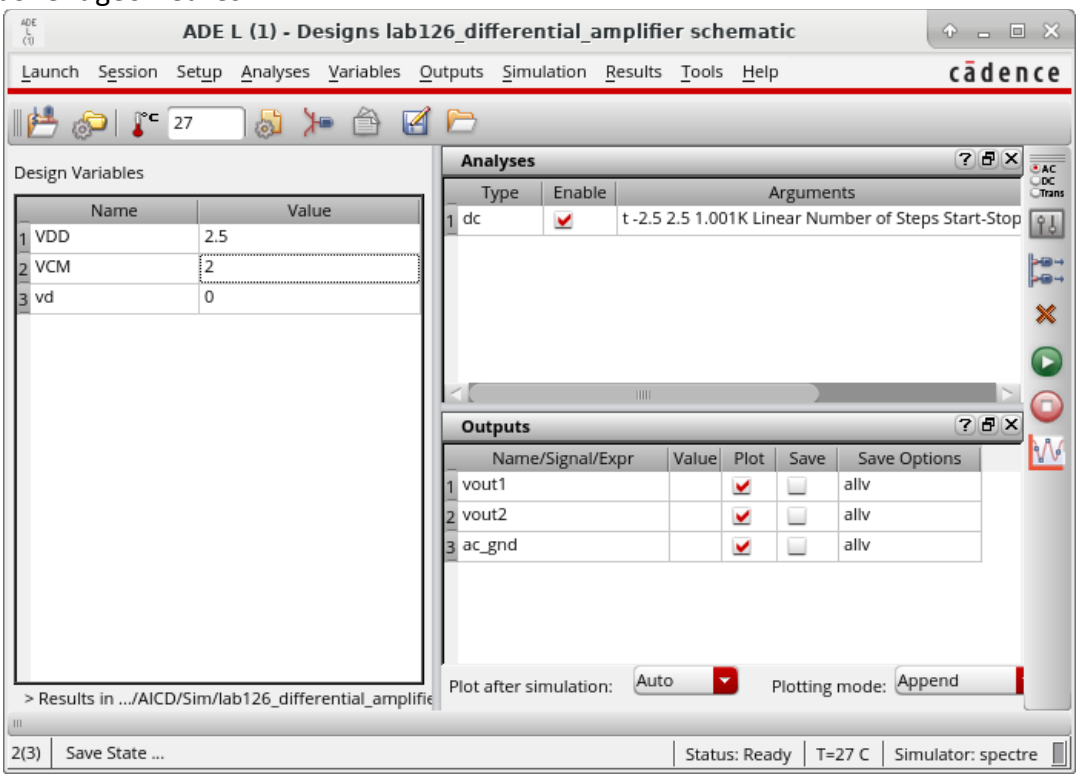
Note that the transconductance is halved at vd=0 because the tail current is shared between the input devices.

The output of the differential amplifier may be taken single-ended or differentially. A single-ended output may be attained by taking one output with respect to ground. A balanced or differential output may be attained by taking the difference between the two outputs. The balanced output is double the gain of the unbalanced gain – hence the '2' multiplier in the gain equation above. Later in this lab, we will use the calculator to plot the differential voltage transfer curve, from which the gain can be calculated.

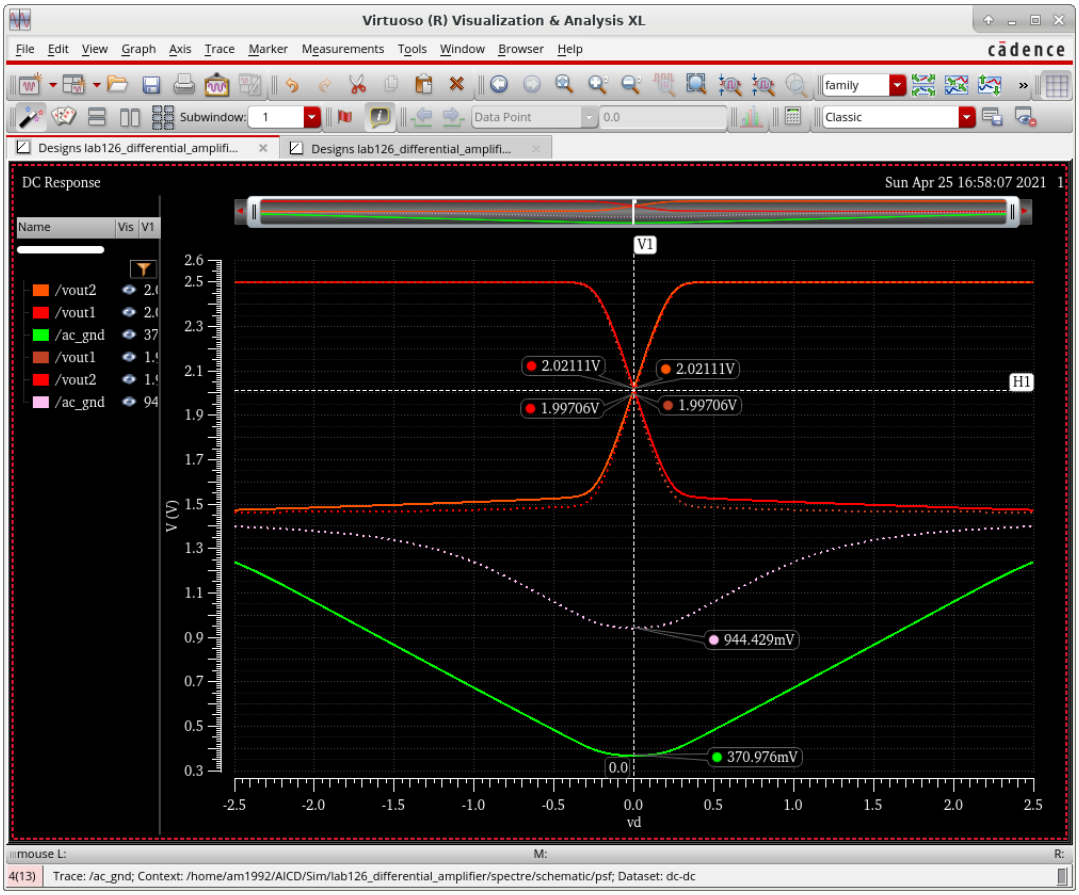
One final point to note, the common-source AC ground point of the differential amplifier is also plotted (ac_gnd) alongside the output voltages (vout1, vout2). Here, we see that there is some variation of the voltage at this point as vd is swept from -2.5V to 2.5V. This is caused by the source voltage of the dominant input FET carrying the current, 'pulling up' the AC ground point as the voltage on its gate is increased (This is a high impedance node). If the NMOS input pair were ideal linear devices, we would indeed see no variation of the voltage at the AC ground point. However, their linearity is only assumed over small excursions of the input voltage, so as the voltage on the gate of the dominant NMOS FET increases, its source will follow at a v_{gs} below the gate.

Furthermore, the tail bias device has a current that is dependent on its drain voltage – due to channel length modulation. This non-ideal behaviour of the bias device will compound the variation on the AC ground point. We can lower the variation of the AC ground node by increasing the common mode input voltage (VCM). If we re-simulate with a common mode input of VCM=2V (Figures 9 and 10), we can see how this lowers the variation on the AC ground node, with little change in the output voltages (vout1 and vout2). Set the plotting mode of the ADE to append (bottom-right drop-down menu.)

Note that the variation of the AC ground point is not normally a problem and so does not define what the VCM should be. If it is a problem, then the designer will redesign the component geometries.



< Figure 176. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 177. Resistor Loaded Differential Amplifier Output Voltages and AC Ground Point. VCM=1.25V (solid) & VCM=2V (dotted). VCM=1.25V. VD=-2.5V to +2.5V >

5. Circuit Simulation Exercises: Part 2

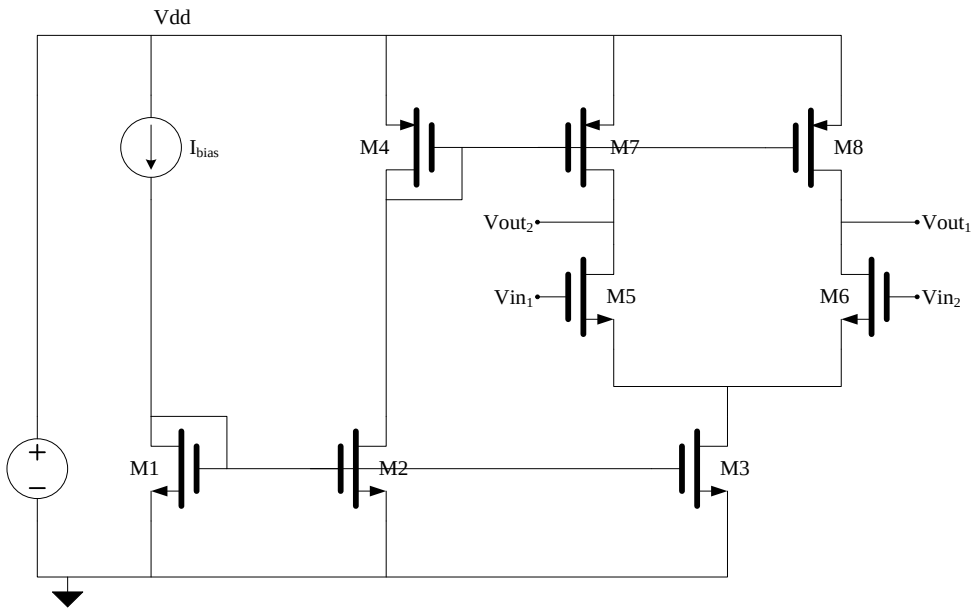
With guidance, you will:

- Create and simulate a current-source loaded (active-loaded) differential amplifier.
- Understand the need for feedback in order to set the DC operating point of a fully differential amplifier.

5.1 The Current-Source Loaded Balanced Amplifier

(5.1.1) In this part of the lab, we will replace the two 10kΩ resistor loads with PMOS current sources biased by an expanded current mirror.

(5.1.2) Modify your existing schematic by constructing the current source loaded amplifier alongside your existing circuit. We will use a proper expansion of the current reference rather than the quick but unrealistic version (with the current source placed in between the two driving FETs, M2 and M4). See Figure 11.



< Figure 178. The PMOS Current-Source Loaded Balanced Amplifier (with Current Mirror Expansion). Reproduced from section 4.4 of the background material. >

(5.1.3) Set all NMOS devices to a W/L ratio of 10um/1um (**Total Width** = 10u, **Length** = 1u). The multipliers of all NMOS devices should be set to one (**Multiplier** = 1).

(5.1.4) Set all PMOS devices to a W/L ratio to 25um/1um (**Total Width** = 25u, **Length** = 1u) to approximately match the transconductances to the NMOS devices (since the NMOS devices have a carrier mobility approximately 2.5 times that of the PMOS devices).

(5.1.5) For the PMOS current mirror/bias device (M4 in Figure 11), set the multiplier to two (**Multiplier** = 2). The multiplier of the PMOS load devices (M7, M8 in Figure 11) should be set to one (**Multiplier** = 1).

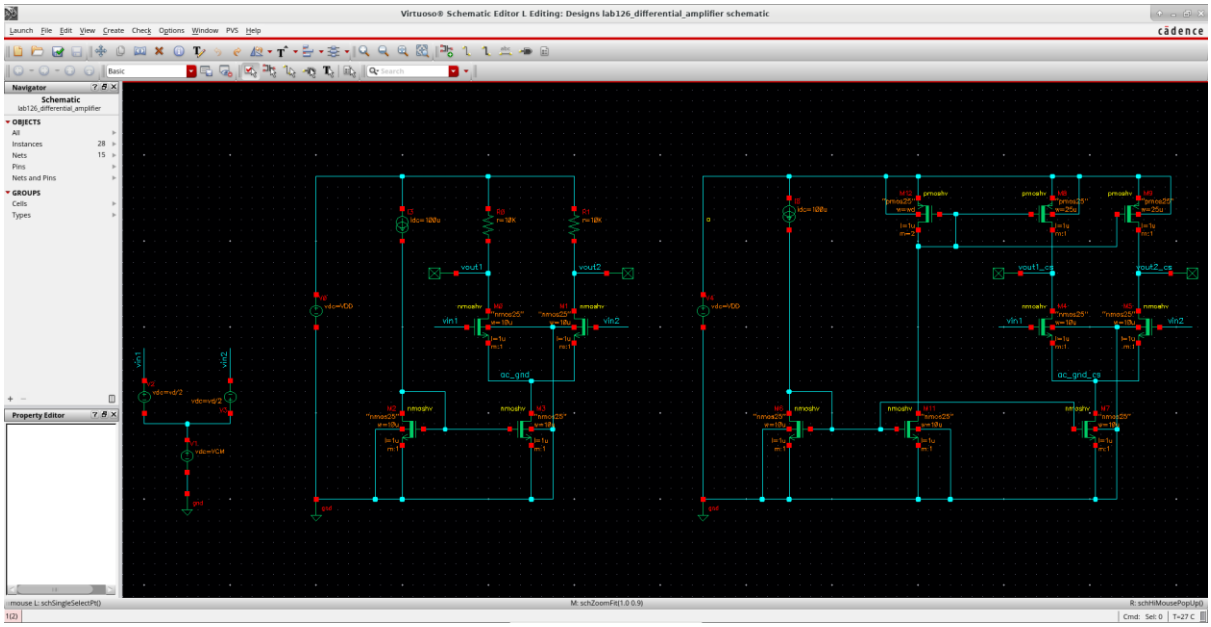
(5.1.6) **Discussion**. Recall, from our discussion in lab 12.2 (paragraph 5.4.13), when biasing the amplifier, we want to set the output DC operating point to approximately mid-rail (VDD/2). For a current-source loaded amplifier, we will only set the output (vout1, vout2) to be mid-rail if the current sourced by the PMOS load devices is the same as the current sunk by the NMOS input devices. That is, we must balance the currents of the NMOS and PMOS devices.

Here, we achieve this balance of the PMOS (source) and NMOS (sink) currents by scaling the PMOS mirror/bias device M4 in fig. 11. Since the tail current is set to 100uA, the current in each branch/side of the differential input pair must be 50uA. We therefore want the PMOS load devices to source 50uA each. We do this by adjusting the PMOS current mirror/bias device (M4) and load devices (M7, M8) to have a ratio of 2:1:1, respectively. This ratio means the PMOS load devices will source half the current that is mirrored into the PMOS mirror device (100uA:50uA:50uA).

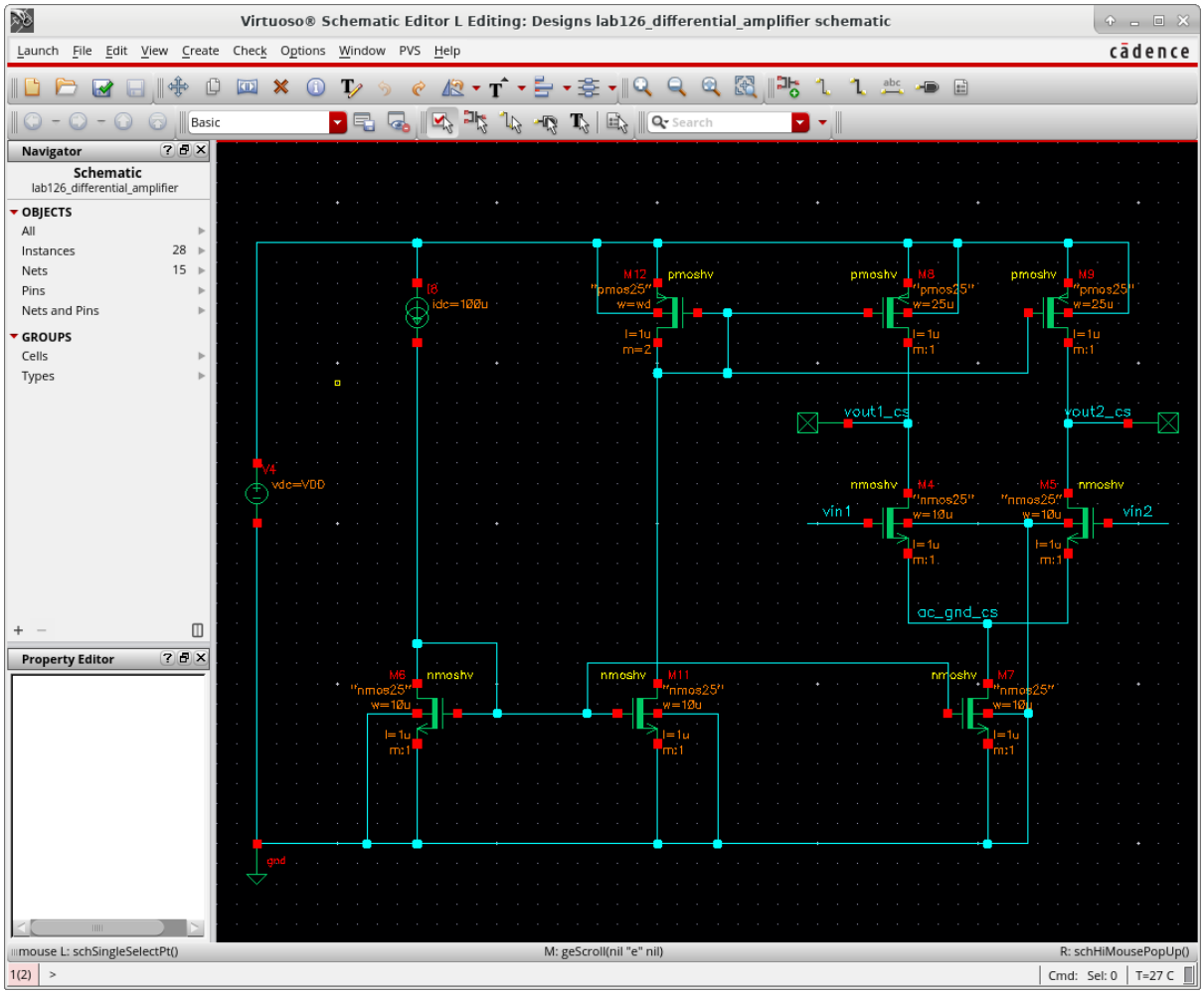
Note: This way of balancing the NMOS/PMOS currents is purely for teaching purposes. In practice this would not work.We would use feedback to drive the PMOS load devices to establish the output DC operating point at approximately mid-rail. See section 5.3.

(5.1.7) Change the name of the output nets to vout1_cs and vout2_cs (this avoids creating an implicit connection between the outputs of the two circuits.) Do the same for the AC ground point – ac_gnd_cs. The inputs can be left as vin1 and vin2 (this allows both amplifiers to be driven by the same driver circuit). See Figures 12 and 13.

(5.1.8) Check and save you schematic using the **Check and Save** command.



< Figure 179. Resistor-Loaded and Current-Source Loaded Differential Amplifiers. >



< Figure 180. Close-Up of the Current-Source Loaded Differential Amplifiers. Note the multiplier values of the PMOS devices from left to right m=2, m=1, m=1. (Ratio 2:1:1). >

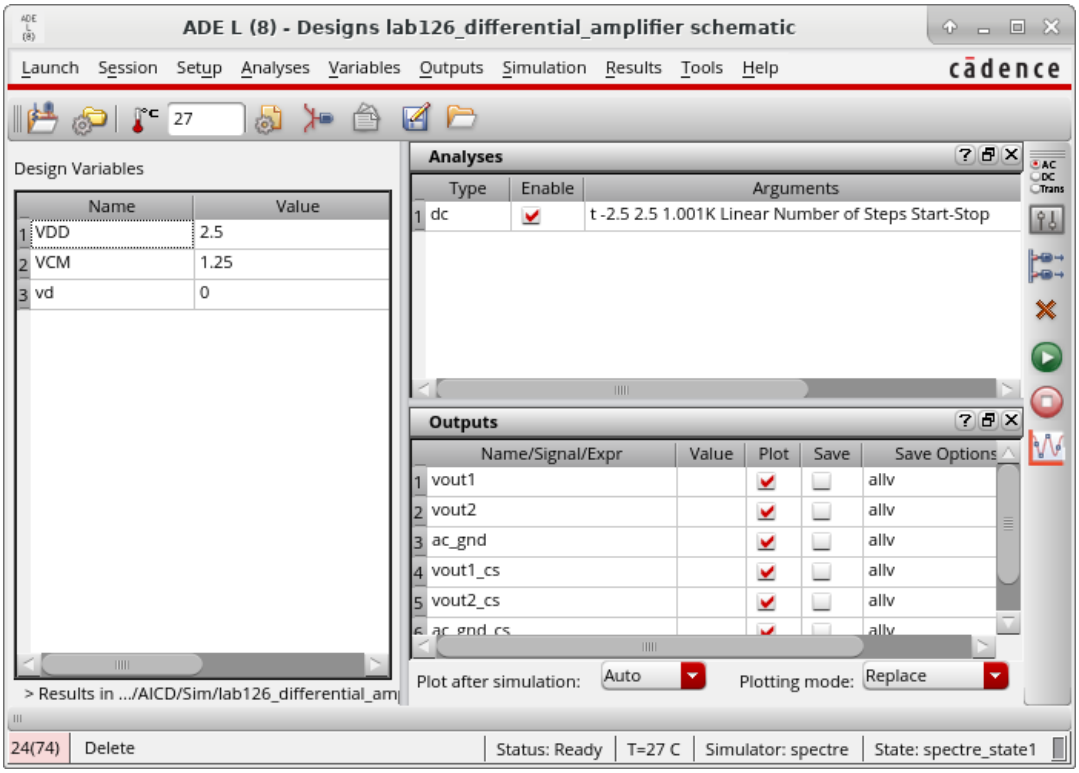
5.2 Simulating the Differential Amplifier using a Swept-DC Analysis

(5.2.1) Return VCM to 1.25V (if you haven't already done so).

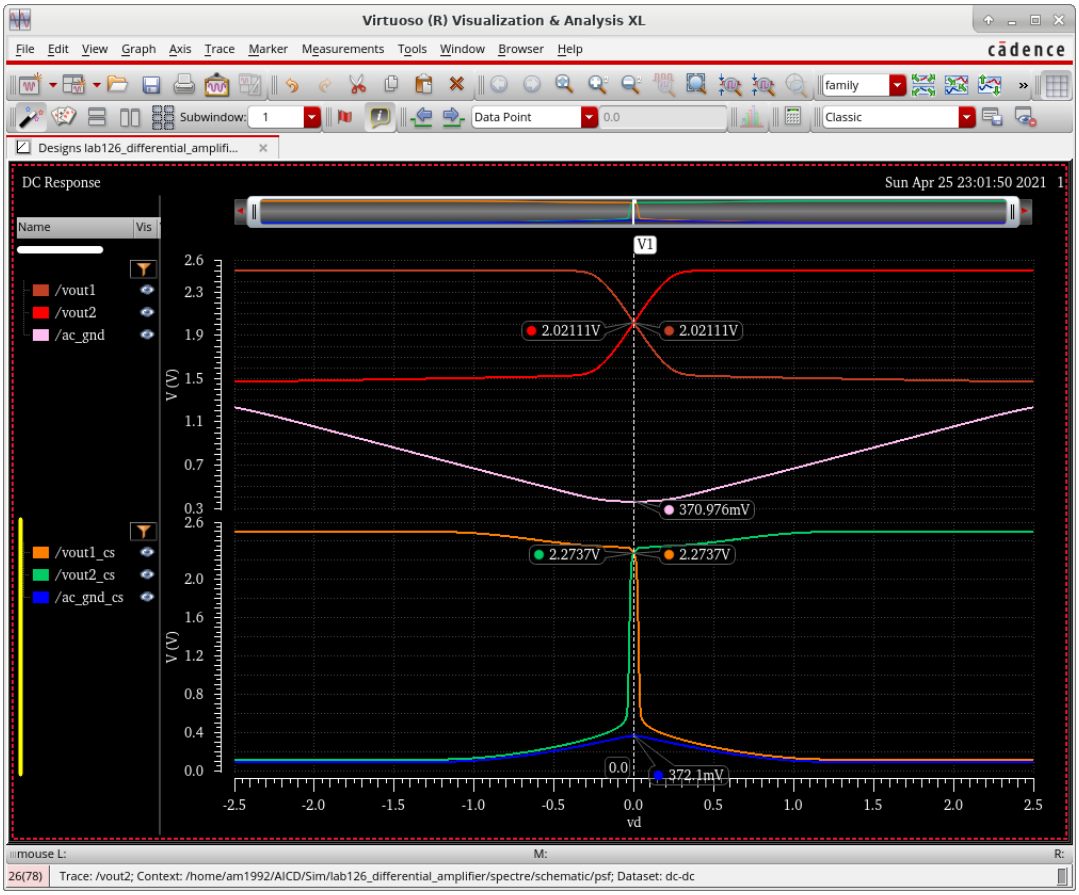
(5.2.2) Select the second amplifier's output nets (vout1_cs, vout2_cs) and AC ground point (ac_gnd_cs) for plotting alongside the first amplifier (vout1, vout2, ac_gnd). **Press the 'ESC' key to exit the plotting mode of the ADE**. See Figure 14.

(5.2.3) Re-run the DC sweep (this should still be set-up form the previous part – if not repeat step 4.2.3). When the simulation has completed, separate the traces for the resistor-loaded and current source loaded differential amplifies. Figures 14 and 15.

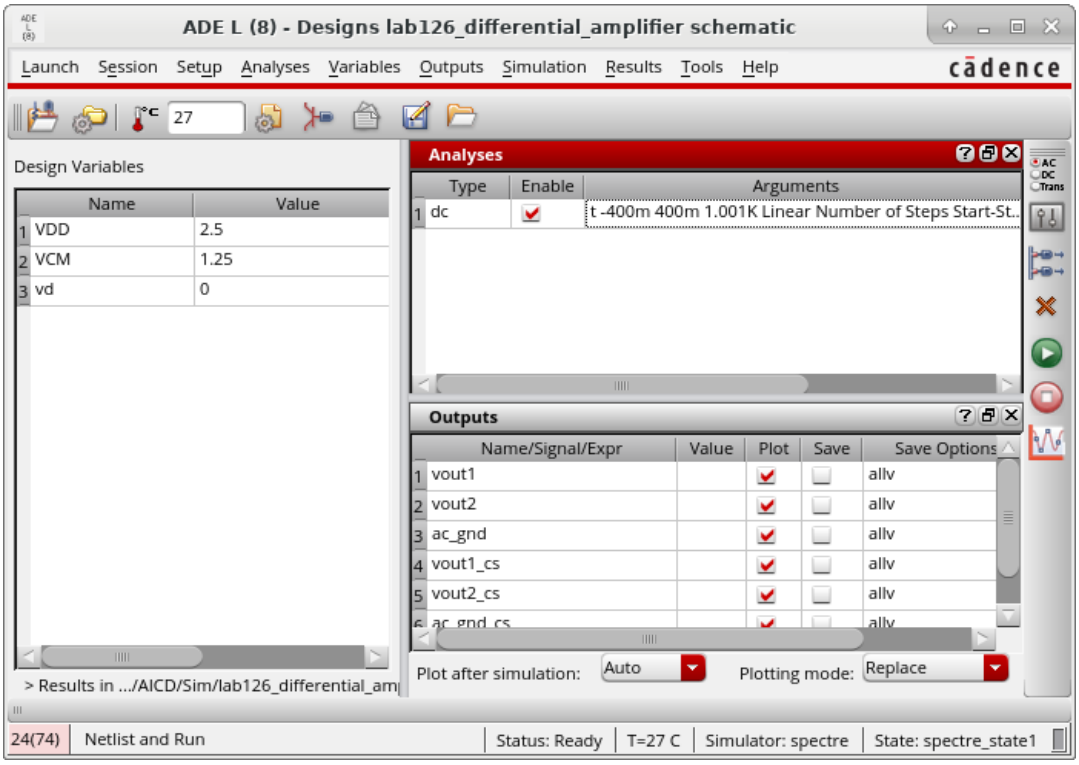
(5.2.4) We are only interested in the transition area around vd=0V – the so-called analogue region of operation. Repeat the simulation again with vd swept between -0.4V and 0.4V for better picture of analogue region of the voltage transfer curve. Figures 16 and 17.



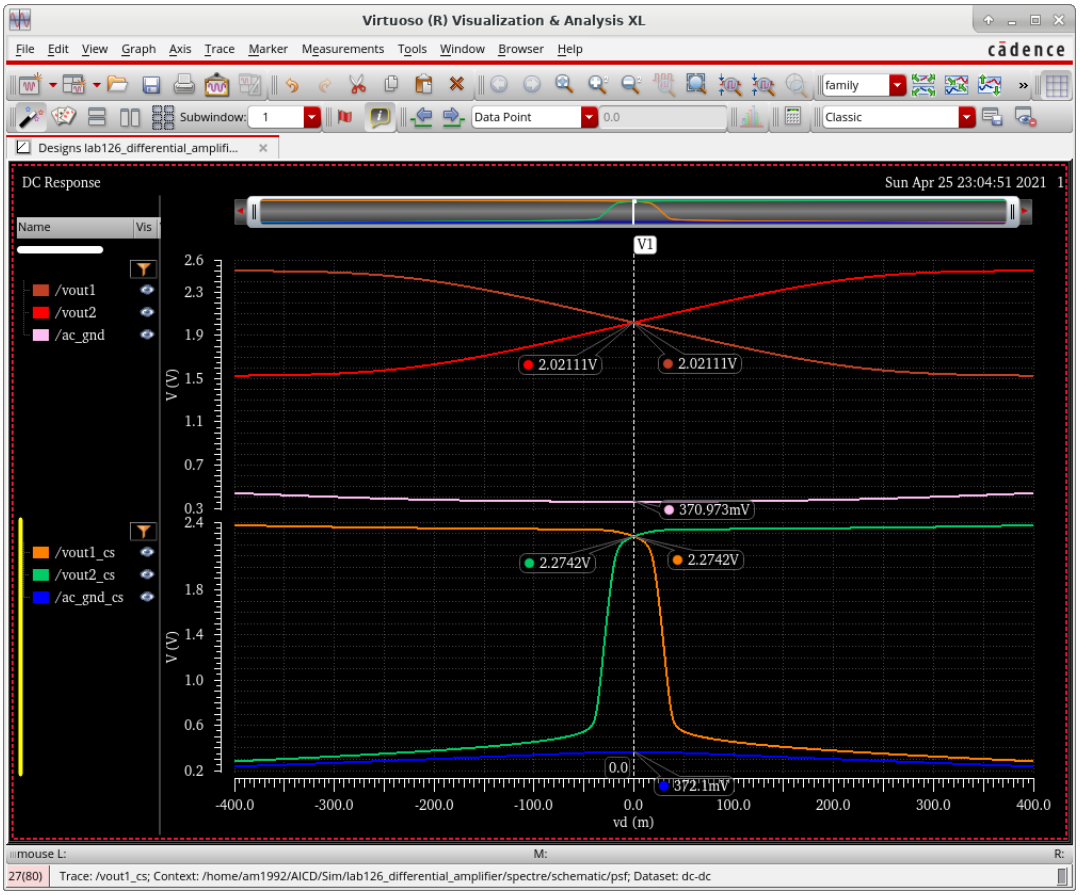
< Figure 181. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 182. Resistor-Loaded and Current-Source Loaded Differential Amplifier Output Voltages. VCM=1.25V. VD=-2.5V to +2.5V >



< Figure 183. The ADE with the Design Variables, Analyses and Outputs. >



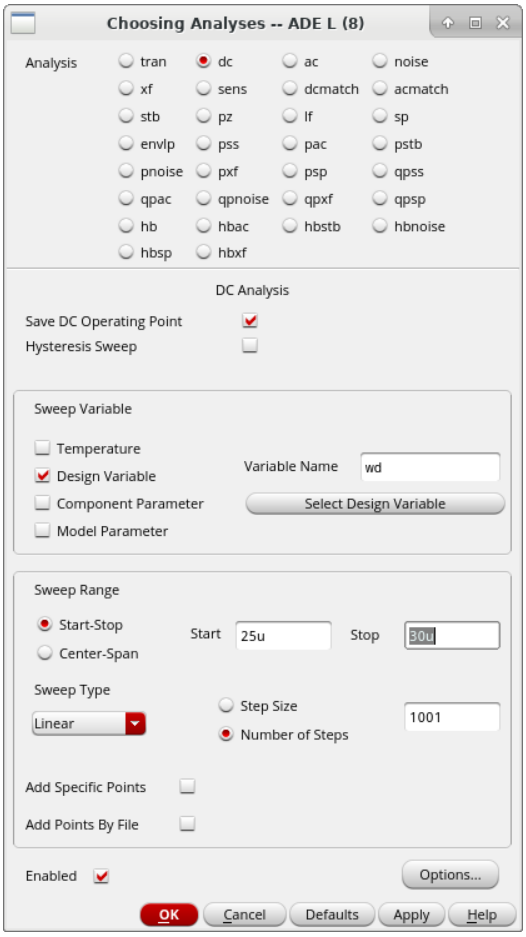
< Figure 184. Resistor-Loaded and Current-Source Loaded Differential Amplifier Output Voltages. VCM=1.25V. VD=-0.4V to +0.4V >

(5.2.5) **Discussion**. Since the gain of an amplifier is given by the slope of its transfer curve, we see that greater load resistance of the PMOS current source gives more gain (as we would expect). However, we can also see there is a significant offset in the output – that is, at vd=0V, we are not operating in the middle of the voltage transfer curve where the gain is at its greatest. In fact, at vd=0V, we are operating very close to the supply rail (2.274V) with no headroom for signal swing and low gain (just where the VTC begins to flatten out).

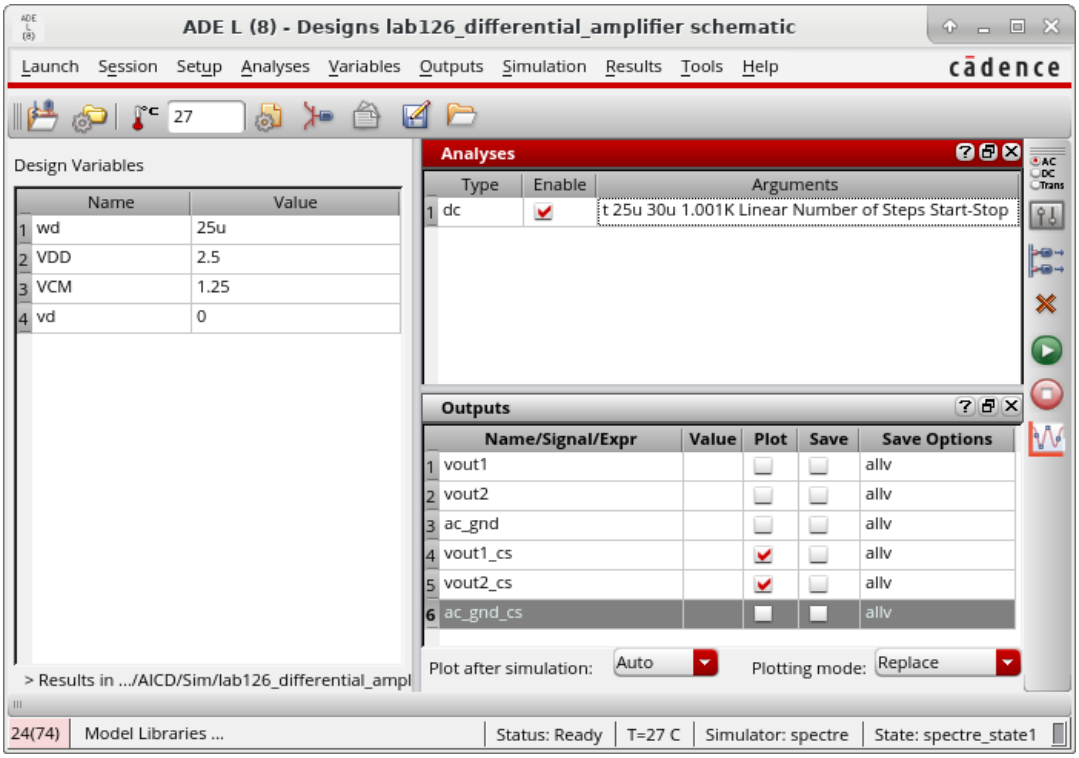
There is an additional problem with design. The vds of the tail device is only 372mV. The device is operating in the triode region. Whilst this isn't a problem for this illustration, in practice the designer would redesign the component geometries and increase the input common mode.

As we have discussed previously, we want to set the output voltage at around mid-rail (VDD/2). In order achieve this we have to precisely match the currents being sourced by the PMOS loads and sunk by the NMOS input devices.

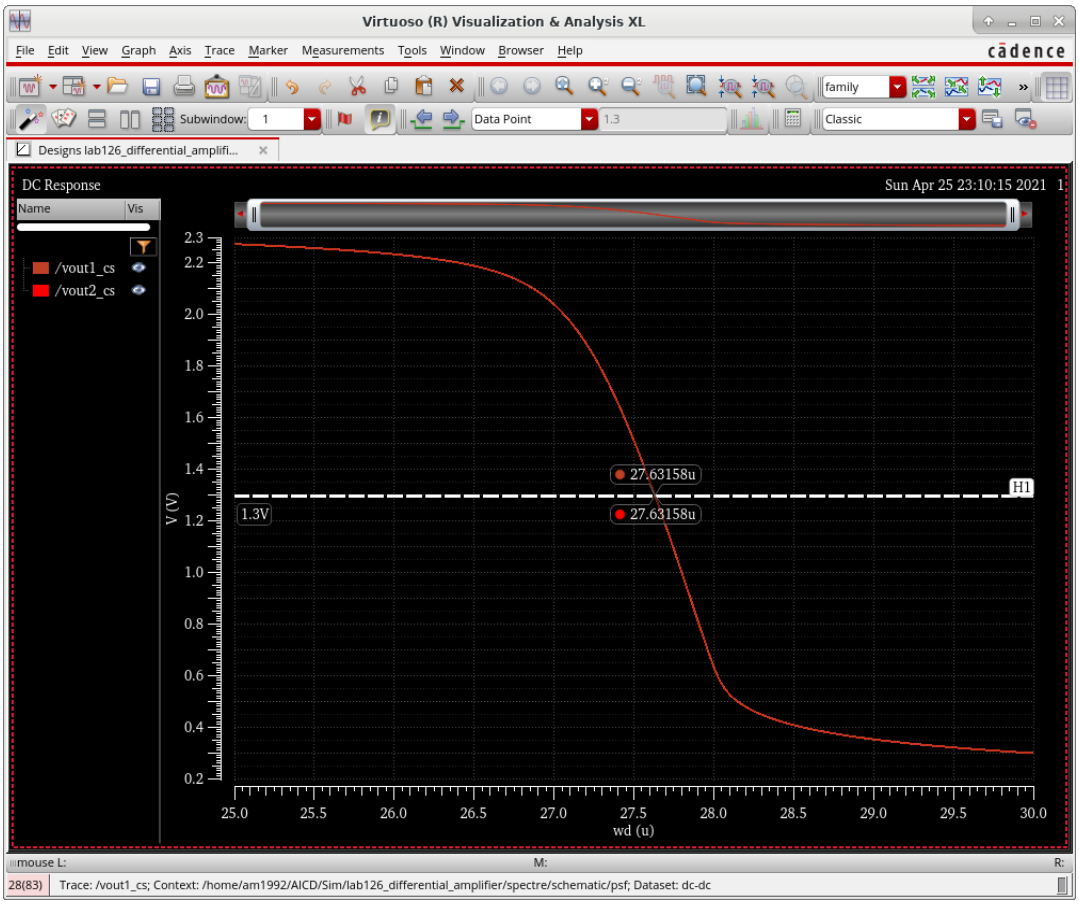
We will run a DC sweep, by parameterising the width of the PMOS mirror/bias device (M4 in fig. 11) to ‘wd’. Modfiy the DC sweep analysis to sweep the width of the PMOS device from 25um to 30um (start-stop, linear, number of steps 1001 - see figs. 18 and 19) and observing when the common mode output voltage (vout1 and vout2 when vd=0V). We see a device width of 27.63um gives a VCMO of 1.3V. See Figure 20.



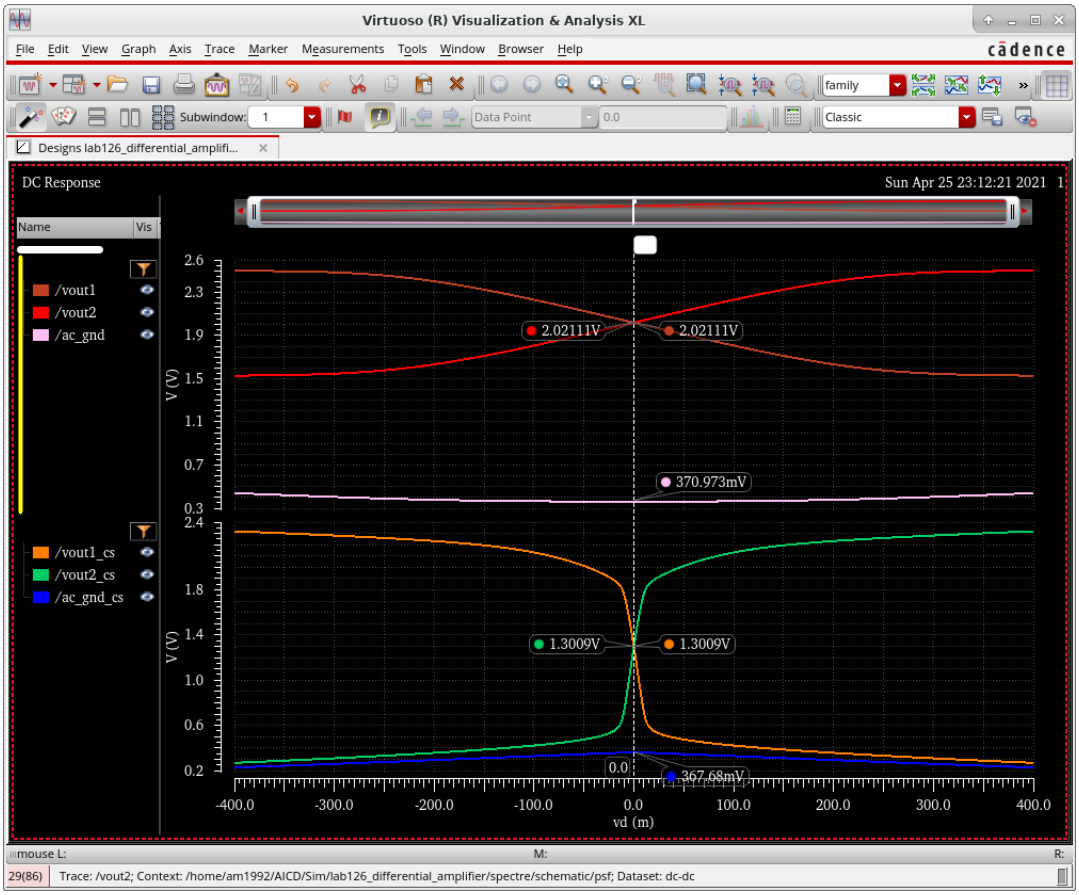
< Figure 185. The Choosing Analyses Form. >



< Figure 186. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 187. Common-Mode Output Voltage of the Current-Source Loaded Differential Amplifier as PMOS mirror/Bias Device is Swept from 25um to 30um. At 27.63um VCMO=1.3V >

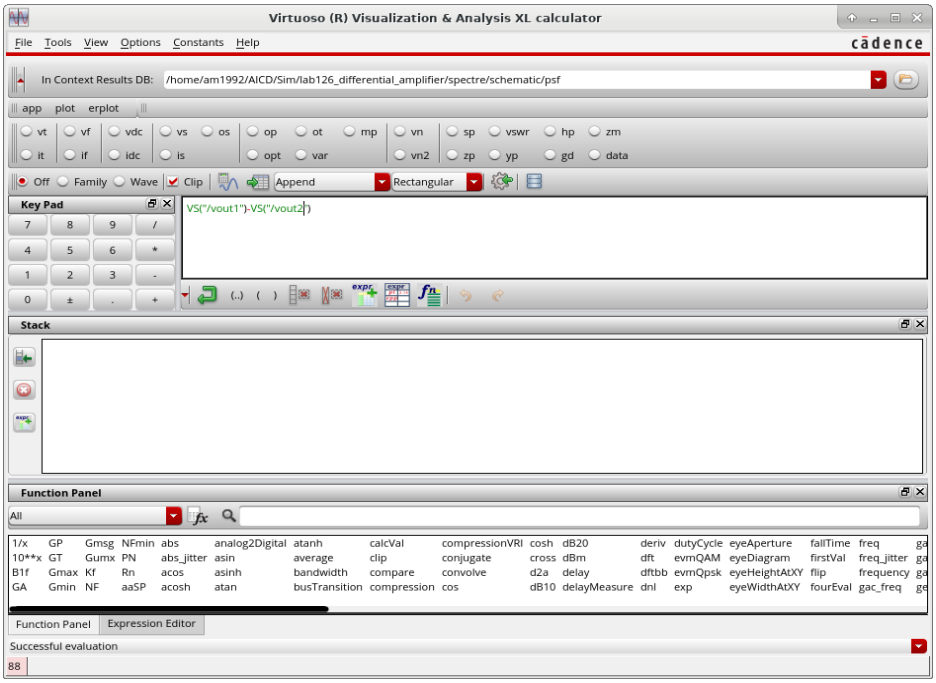


< Figure 188. Resistor-Loaded and Current-Source Loaded Differential Amplifier Output Voltages. VCM=1.25V. VD=-0.4V to +0.4V. PMOS mirror/Bias Device Width = 27.63um >

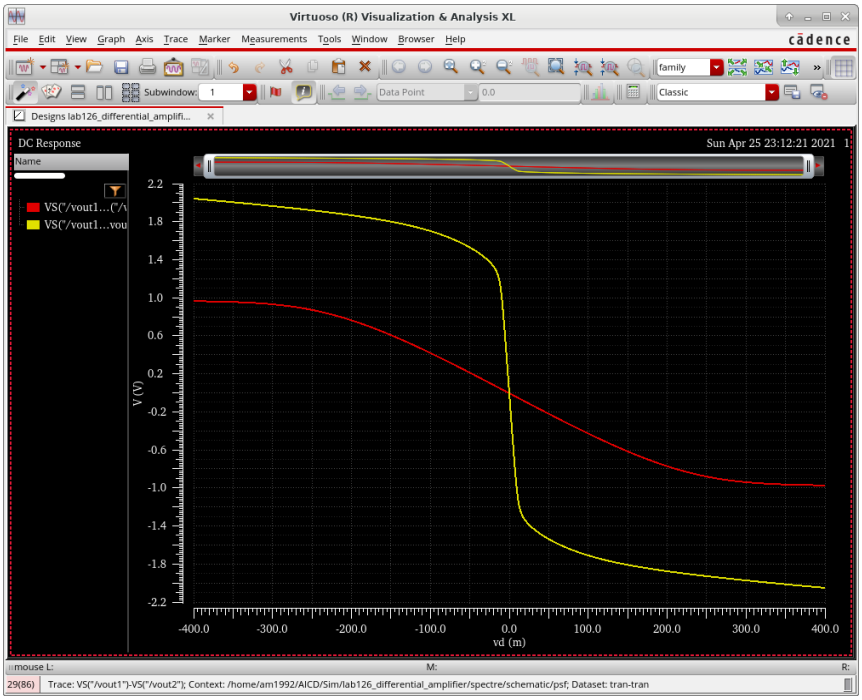
(5.2.6) **Discussion**. Set the PMOS mirror/bias device width the 27.63um and re-run the DC sweep for the differential input voltage vd from -2.5V to +2.5V (see step 4.2.3).

Having precisely adjusted the PMOS mirror/bias device width to 27.63um (length 1um), we see the common-mode output voltage (the DC operating point, at vd=0V) has been centred to approximately mid-rail 1.3009V. **It must be stressed that this method of setting the common-mode output voltage is totally unrealistic in practice. There is simply no way of setting the device widths/lengths with such precision due to variations in process technology. Furthermore, even if we could set the device dimensions with such precision, the circuit would not be able to maintain the DC operating point over a wide temperature range, typically -40°C to 125°C, nor over a varying voltage supply (+/- 10%). Prectical circuits use feedback to establish the common mode output voltage. We will demonstrate this in section 5.3.**

(5.2.7) Use the calculator to plot the difference between the two outputs and note that the gain is increased by 2 (compared to a single output). (If you need a refresher on how to use the calculator, refer to section 4.3 - Using the Built-in Waveform Calculator - of lab 12.3.) In the calculator buffer, construct the expression for the differential gain: VS("/vout1")-VS("/vout2"). Do this for both the resistor-loaded and current source-loaded amplifiers. Plot the VTC differential gain curves. See Figures 22 and 23.



< Figure 189. Calculator Expression for Calculating Differential Output Voltage. >



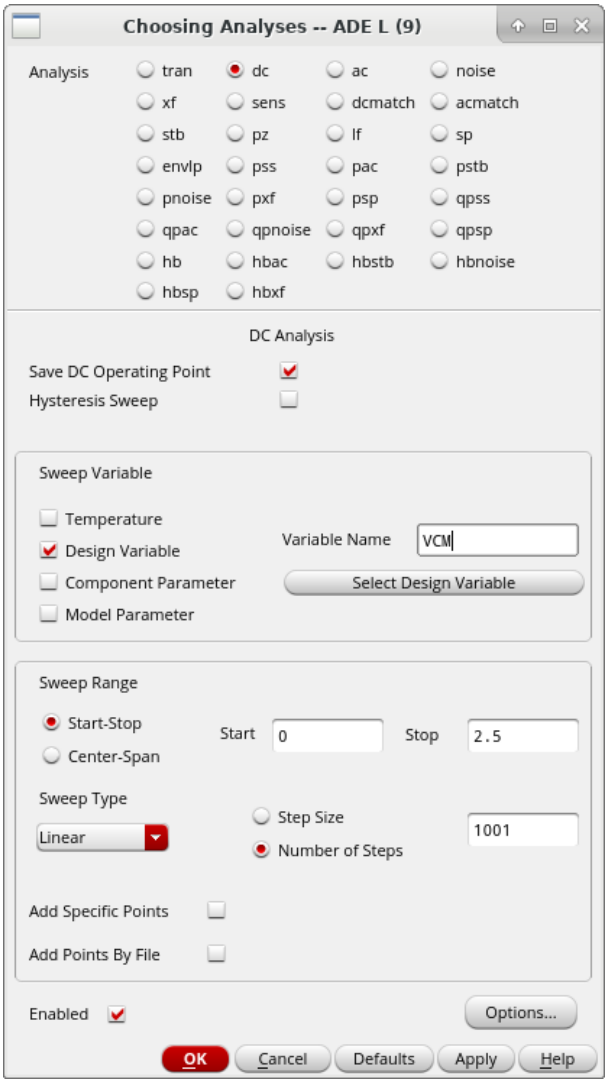
< Figure 190. Differential Output Voltage for Resistor-Loaded (red) and Current-Source Loaded (yellow) Differential Amplifiers. >

(5.2.8) Finally, we want to establish the approximate common-mode output voltage range of the differential amplifiers.

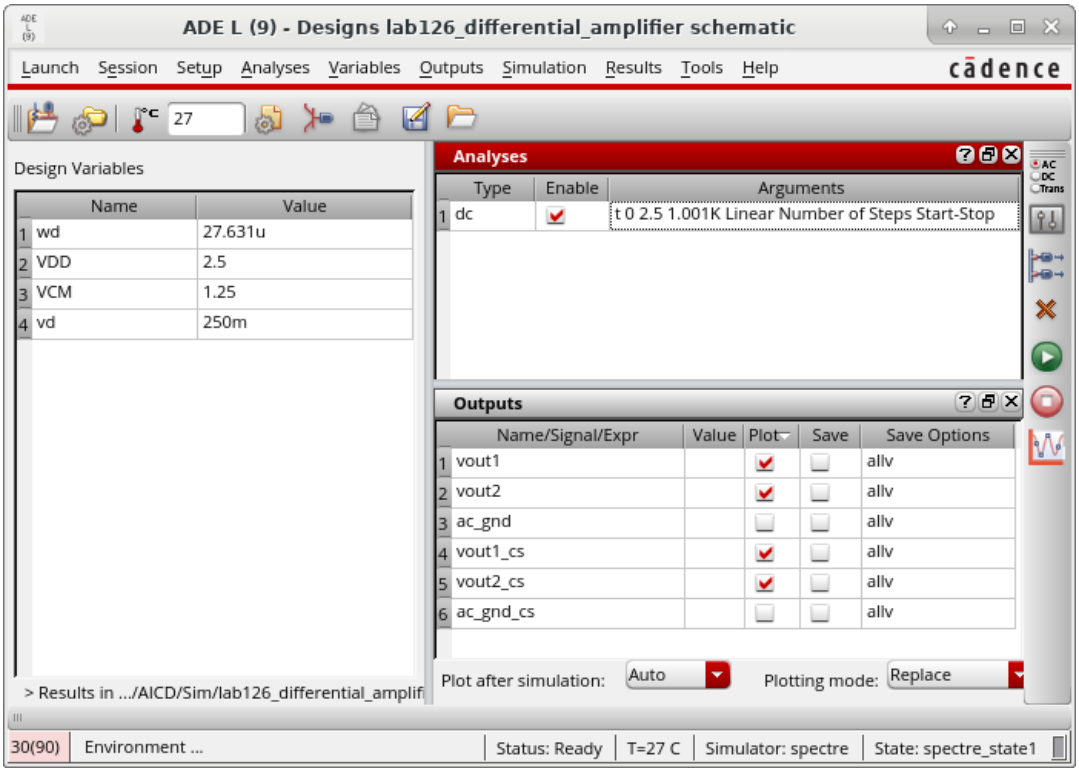
Recall from the introduction: differential amplifiers amplify the difference between their inputs (differential-mode) and reject what is common between their inputs (common-mode). To determine the common-mode output voltage range through simulation, we will apply a fixed differential input voltage of 250mV (vd=0.25V) and sweep the common-mode input voltage (VCM) from 0V to 2.5V. The common-mode range will then be the range over which the common-mode output voltage remains approximately flat at its DC operating point.

In the ADE, set vd = 250 mV. Modify the DC analysis form to make VCM the sweep variable and the 'Start-Stop' option to Start = 0 and Stop = 2.5 for the sweep range. Leave all other properties unchanged. See Figure 24. OK the form.

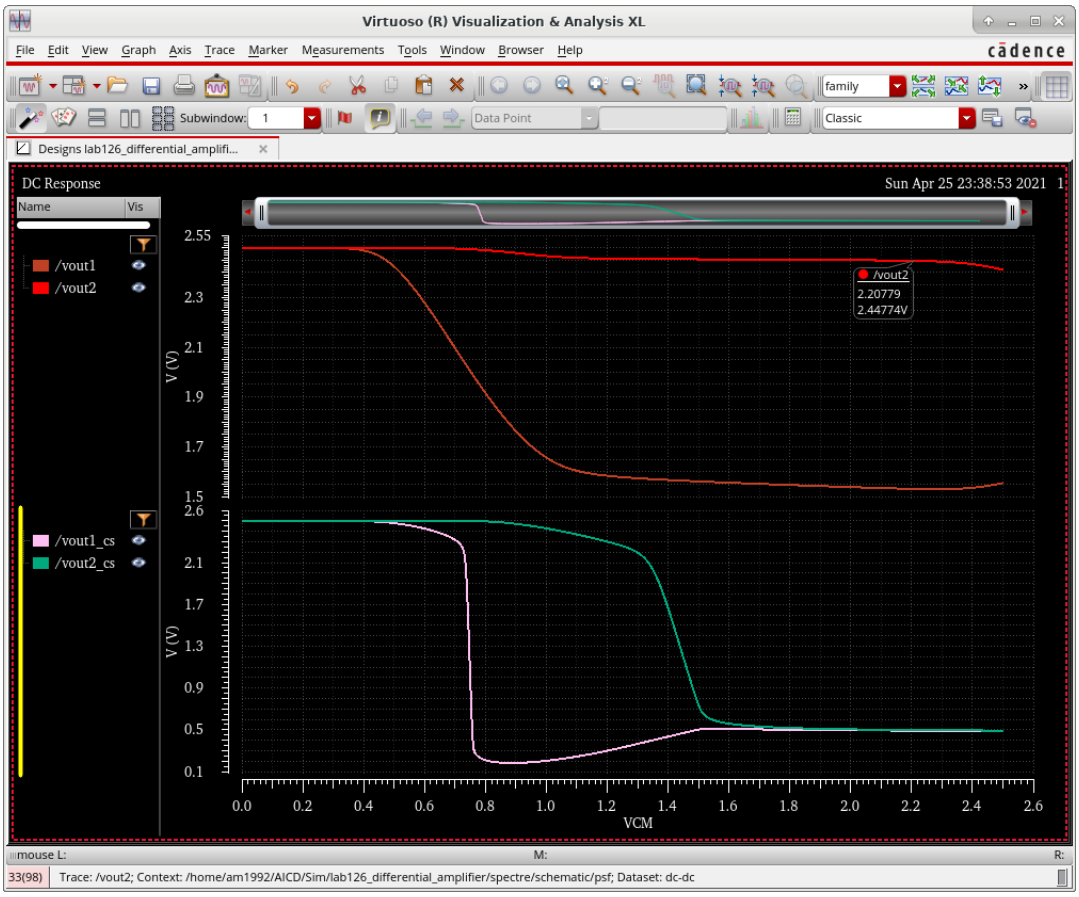
(5.2.9) Run the '**Netlist and Run**' command by clicking on the green play button. When the simulation has completed, separate the traces for the resistor-loaded and current source loaded differential amplifies. Figures 25 and 26.



< Figure 191. The Choosing Analyses Form. >



< Figure 192. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 193. Resistor-Loaded and Current-Source Loaded Differential Amplifier Output Voltages. VD=250mV. VCM=0V to +2.5V >

(5.2.10) **Discussion**. Figure 26 plots the differential output voltages (vout1…, vout2…) of the differential amplifiers as the common-mode input voltage (VCM) is swept. We will use the waveform calculator to plot the common-mode output voltage (VCMO) for each of the amplifiers.

Note that the differential component of the output voltage rides on top of the common-mode component of the output voltage. We can eliminate the differential component by adding the two output voltages (*vout1…, vout2…*) and dividing by two. That is,

$$V_{CMO} = \frac{vout1 + vout2}{2}$$

In the calculator buffer, construct the expression for the common-mode output voltage (V_{CMO}): (VS("/vout1")+VS("/vout2"))/2. Do this for both the resistor-loaded and current source-loaded amplifiers. Add the VCMO plots to the respective VCM sweeps in the waveform window. See Figures 27 and 28.

We observe for the two differential amplifiers, there is a range of the VCM sweep over which the common-mode output voltage (yellow traces) remains approximately flat (at the DC operating point). This region for which there is no change in output voltage for a change in the common mode input VCM identifies the common mode input voltage range.

For the Resistor loaded differential amplifier (top) we see the common mode output voltage remains flat (at ~2V, the DC operating point) from around VCM=1.3V to VCM=2.5V. For the common source loaded amplifier we see the common mode output voltage remains flat (at ~1.3V, the DC operating point) from around VCM=0.8V to VCM=1.3V.

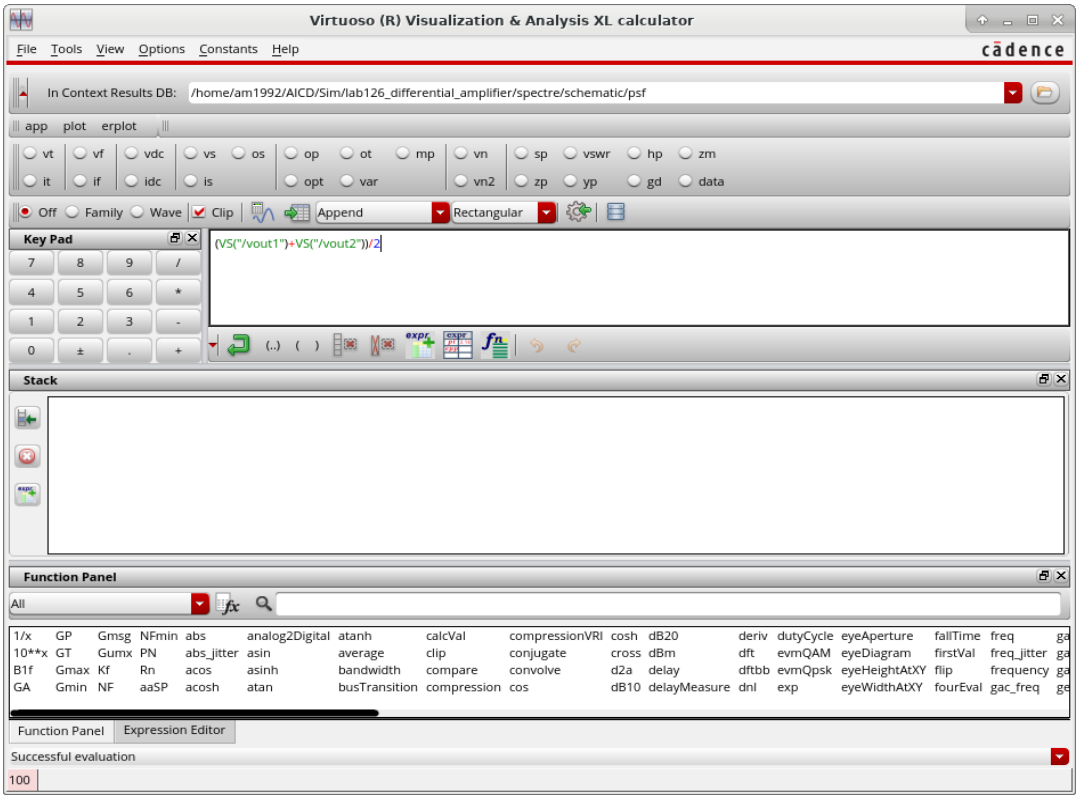
The principal question we want to answer is: what is the optimum value of common-mode input voltage (VCM) we should choose to bias the amplifiers?

This is the value of VCM that maintains the DC operating point for maximum swing (mid-rail, VDD/2) and maximises the differential gain.

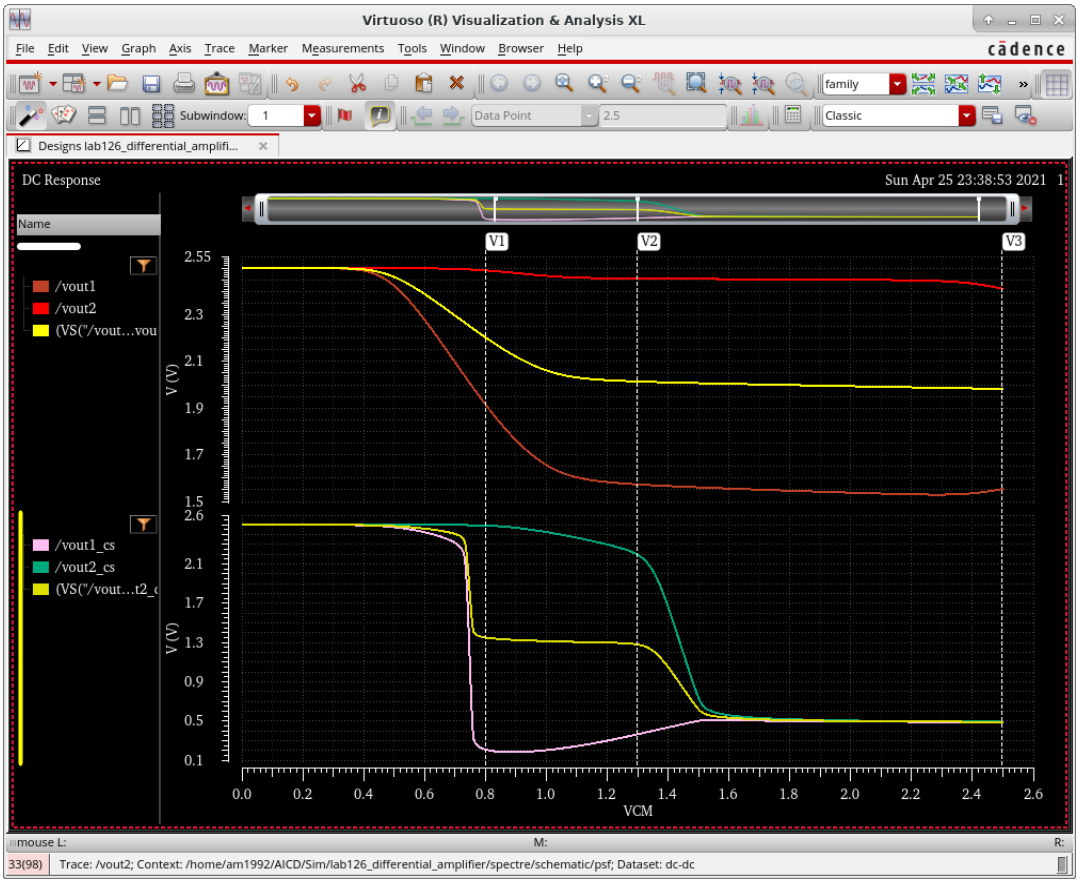
1) For the resistor-loaded amplifier, the combination of the linear load devices (resistors) and relatively low gain means biasing anywhere over the VCMO range (VCM=1.3V to VCM=2.5V) would be fine. However, we note that since we can't set the DC operating point independently of the gain, the DC operating point is fixed at around 2V. This limits the output signal swing (but since the gain is low, this may not be a problem).

2) For the current source loaded amplifier, the range is more limited, but we note that the differential gain is largest at around at VCM=0.9V. At VCM=0.9V the output voltage is very close to mid-rail at around 1.3V. So, the optimum choice for VCM for the current source loaded amplifier is 0.9V.

Make sure you understand how to define the common mode input voltage VCM(I) range, the common-mode output voltage VCM(O) range, and the factors that maximise/limit them!



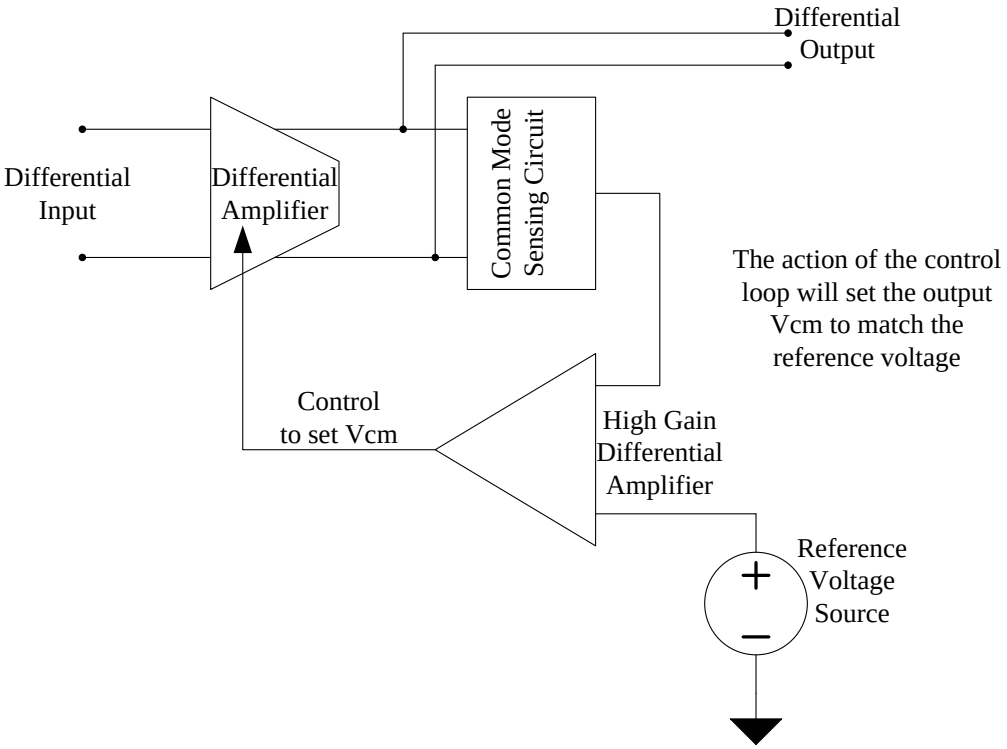
< Figure 194. Calculator Expression for Calculating Common-Mode Output Voltage. >



< Figure 195. Resistor-Loaded and Current-Source Loaded Differential Amplifier Output Voltages. With Calculator Plotted Common-Mode Voltages (yellow traces). VD=250mV. VCM=0V to +2.5V >

5.3 Common-Mode Feedback (CMFB)

In this part, of the lab we will provide a brief demonstration, using ideal components, of how we can use feedback to set the common-mode output voltage of the differential amplifier. This is a much more practical approach to setting the common-mode output voltage than we used in the previous part of this lab. It also makes the circuit robust to variations in process, voltage, and/or temperature (PVT). Figure 29 shows the basic configuration of the common-mode feedback system (CMFB).



< Figure 196. High-Level Diagram of the Common Mode Feedback System.

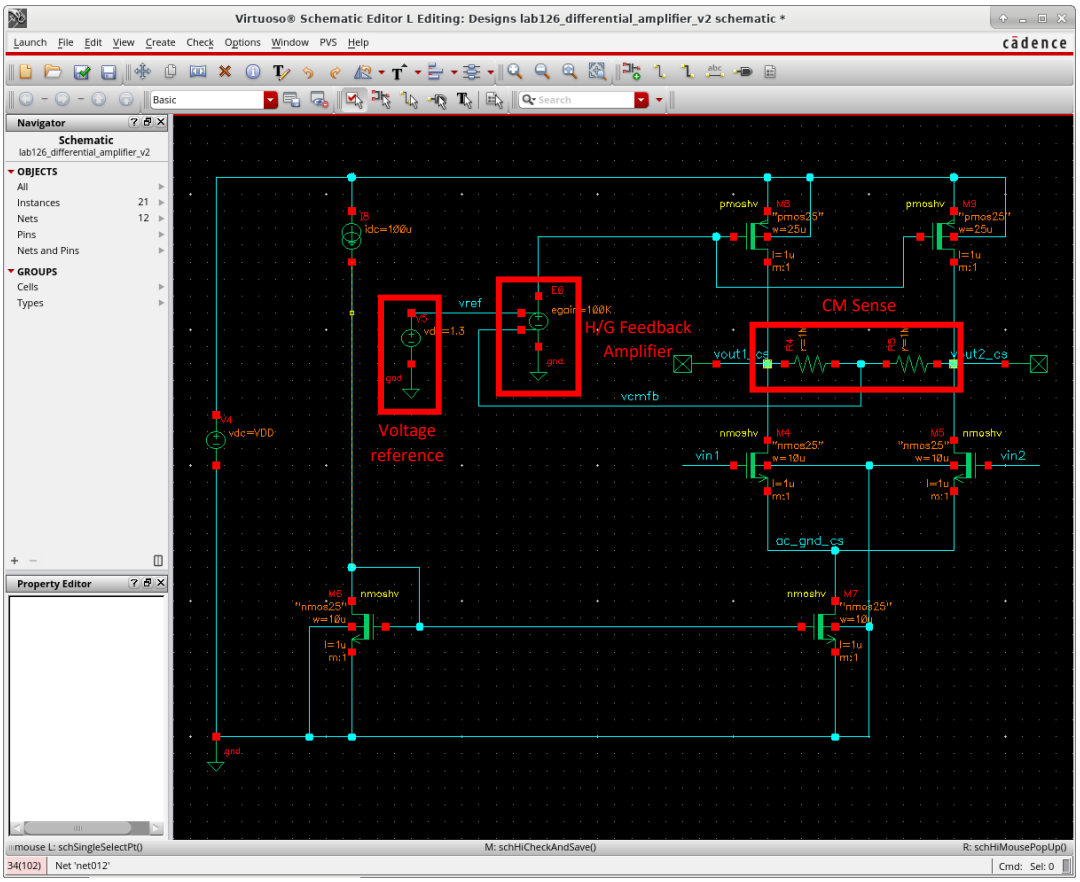
Reproduced from a figure used in a previous version of the Background Material. >

In Figure 29, the differential amplifier (top left) is the amplifier for which we want to set the common-mode output voltage. A sensing circuit (middle) is used to detect the common mode voltage from the outputs of the differential amplifier. The desired common-mode voltage is set by a reference voltage source (bottom right). A high gain feedback amplifier is used to compare the actual common mode output voltage with the desired common-mode output voltage set by the reference. The high gain amplifier is then used to drive the biasing circuitry of the differential amplifier so that the actual common mode output voltage is driven equal to the reference voltage.

(5.3.1) Create a new schematic of the fully differential amplifier. You can build one from scratch or you can copy your existing schematic using the copy command in the library manager. You may also modify your existing schematic by creating another circuit on the same canvas (alongside the two previous circuits you have already built – make sure to change the name of any named nets so as not to create an implicit connection to any of the other circuits on the same canvas).

(5.3.2) We will apply common-mode feedback to the current-source loaded differential amplifier we simulated in part 2 (Figure 13). Build the differential amplifier. Since we will use a feedback amplifier to bias the PMOS current source loads, do not include the PMOS mirror/ biasing circuitry we used in the last part.

(5.3.2) Instantiate and place two 1MΩ resistors (**res**), a Voltage-Controlled Voltage Source (**vcvs**) with a gain of 100K, a voltage reference set to 1.3V (**vdc**), and a ground connection (**gnd**). Source all components from the analogLib ideal component library. See Figure 30.



< Figure 197. The Current-Source Loaded Differential Amplifier Modified for Common-Mode Feedback. >

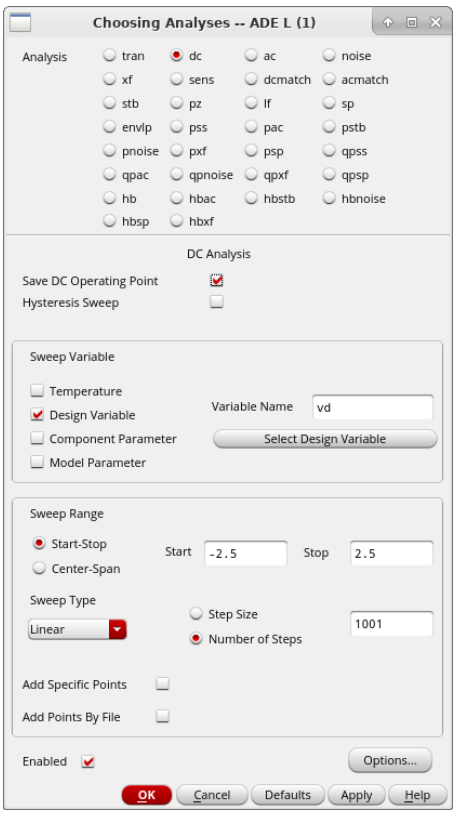
(5.3.3) Note: Here we are using the two 1MΩ resistors as a basic common-mode sense circuit. Since the output voltages are complementary (one goes up, one goes down), the mid-point between the two resistors gives the common voltage between the two outputs. We are using two very large resistors so as not to load the amplifier and reduce its gain. This is not a practical solution – practical designs would use a transistorised circuit design to sense the common mode voltage. However, this setup is useful to demonstrate the principle of common-mode feedback.

(5.3.4) We chose a value of 1.3V for the common-mode output voltage – set by the reference **vdc**. This is just above the mid-rail. Note that the VCM is set at 1.25V, and we want drain voltage of the input pair to be higher than the gate voltage of the input pair to ensure they remain in saturation.

(5.3.5) For the feedback amplifier (**vcvs**) we use a high gain of 100K to ensure high tracking accuracy between the actual output and the reference voltage. **It is important to note that we are applying the feedback to the PMOS current source load devices. It is through this feedback that we are matching the PMOS and NMOS currents for a given VCMO.**

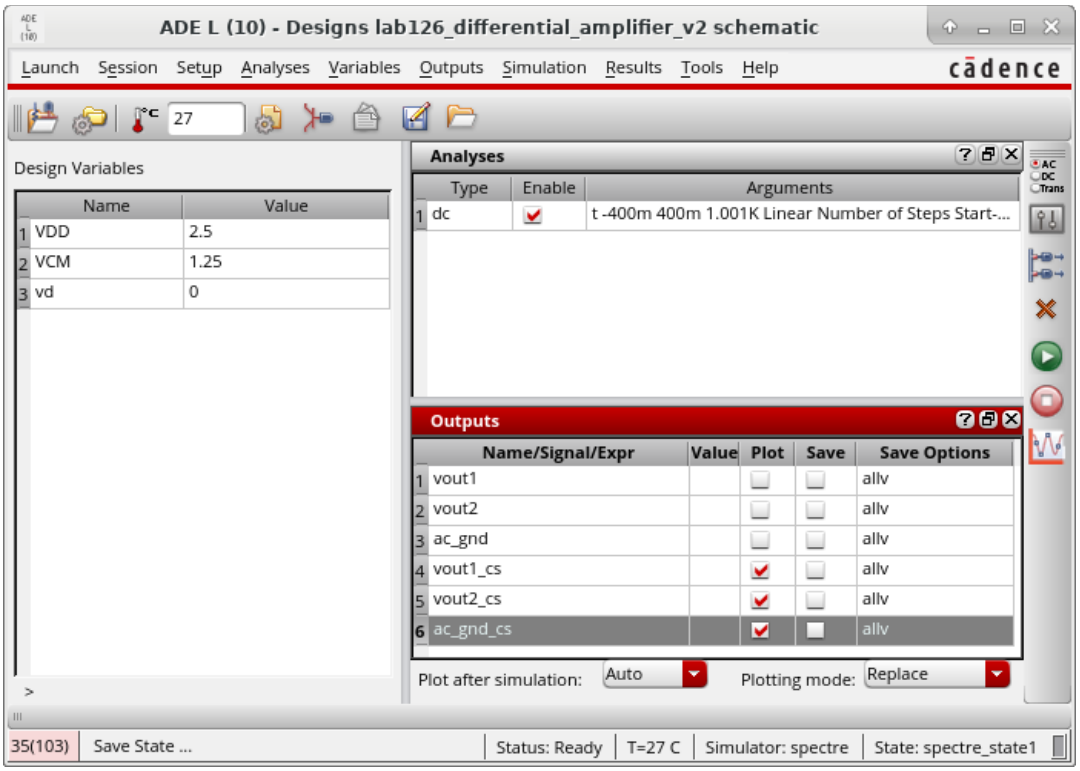
(5.3.6) Launch the ADE. As before, we want to plot the differential output voltage (vout1_cs and vout2_cs) and the AC ground point (ac_gnd_cs) as we sweep v_D from -2.5 to 2.5V. From the ADE window, go to **Menu: Analyses à Choose…** or use the '**Choose Analyses**' button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. From the form, select the 'dc' option button. Check the 'Save DC Operating Point'.

In the choosing analyses form. Check the 'Save DC Operating Point' box. Check the 'Design Variable' box as the sweep variable. Type vd as the variable name. Choose the 'Start-Stop' option button with Start = -2.5 and Stop = 2.5 for the sweep range. Select 'Linear' from the drop-down menu as the sweep type. Choose the 'Number of Steps' option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. See Figure 6. OK the form. The **dc** analysis should now appear in the 'Analyses' list of the ADE window.

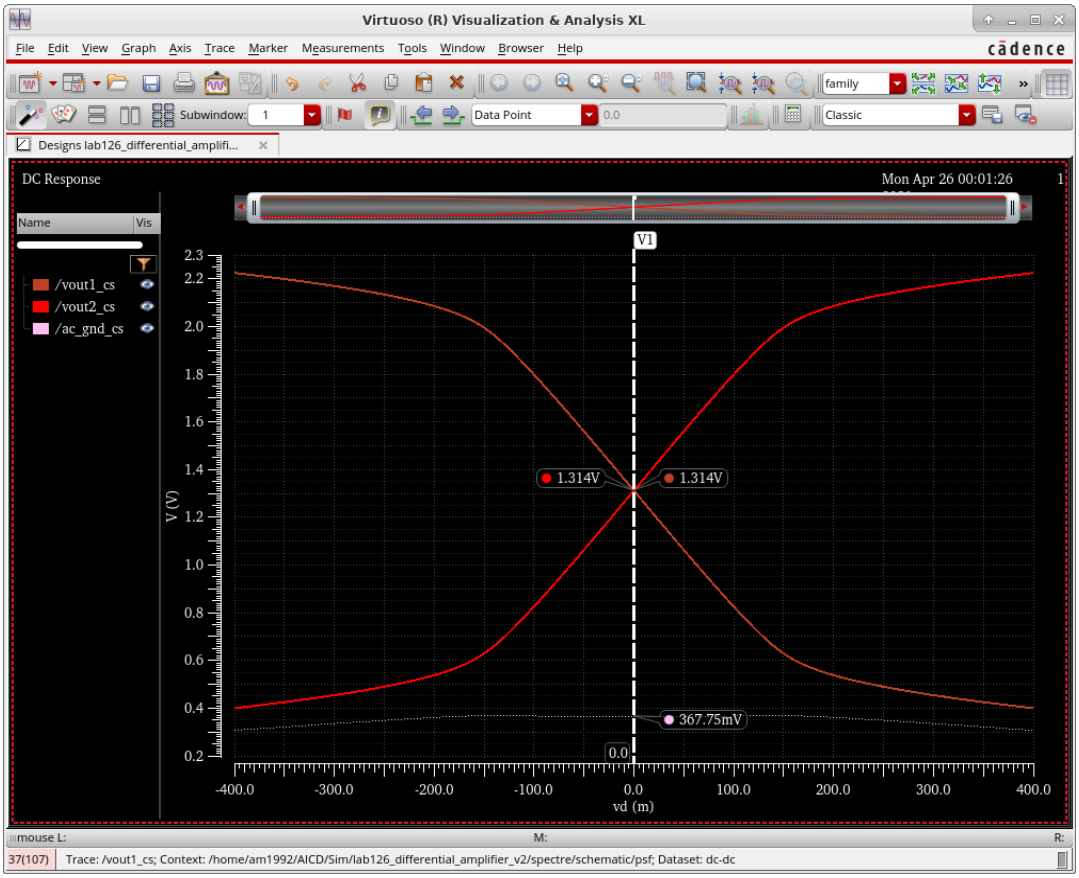


< Figure 198. The Choosing Analyses Form. >

(5.3.7) Select the two output voltages (vout1_cs and vout2_cs) and the AC ground point (ac_gnd) for plotting. In the Analog Design Environment window, select **Menu: Outputs à To Be Plotted à Select On Schematic** to enter the plotting mode of the ADE**. Press the 'ESC' key to exit the plotting mode of the ADE**.



< Figure 199. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 200. Current-Source Loaded Differential Amplifier Output Voltages Set Using Common-Mode Feedback. VCM=1.25V. VD=-0.4V to +0.4V >

(5.3.8) **Discussion**. Referring to Figure 33. Using the common-mode feedback setup we have set the common-mode output voltage to 1.314V (our reference was 1.3V). Note, that in practice we do not need such a high gain (100K) to get a good tracking accuracy. For this simple demonstration, the **VCVS** is also providing the DC biasing for the circuit which is problematic since its gain function cannot be decoupled from its DC biasing function. In a practical feedback circuit, this would may be an issue.

You are encouraged re-simulate the feedback circuit for different values of reference voltage and VCVS gain.

End of Lab!

Tutorial Lab 12.7 (Formatively Assessed):

Corner Analysis

Preliminary Matter: Setting up the Cadence Environment

Please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout before entering these commands into your terminal.

For this course, you will use the Cadence Virtuoso Design Environment (referred to simply as Cadence). Cadence is a Linux based software suite for designing full custom integrated circuits. In order to use Cadence you will first need to set-up the Cadence environment.

8. Open a terminal window and ensure that you are in your home directory using the following command:

```
cd ~
```

Note: 'cd' stands for change directory. The solitary tilde symbol (~) is a shortcut/alias for your home directory (/home/username).

9. Under your home directory, use the following commands to load the Cadence environment:

```
module use /eda/cadence/modules

module load course/EENGM6030
```

Note: The two 'module' commands load the required shell/Cadence environmental variables for this course (paths, names, aliases…) In particular, this includes a reference to the process design kit (PDK) used in this course, the GPDK180. You will need to do this every time you open a terminal, since the sourced environment only applies to the current terminal (until it's closed).

a. ***Note: This step only needs to be completed ONCE - If you already ran this script in lab 12.1, you don't need to run it again here.*** The first time you use Cadence, you will need to run the following command (script):

```
EENGM6030_setup
```

This will create a folder called AICD (short for Analogue Integrated Circuit Design) under your home directory. The AICD folder contains the files and directories needed to run Cadence. Running this setup script more than once will result in a warning message that the AICD folder already exists. It will then exit without making changes.

10. The AICD folder will be used for all your Cadence related activities for this course. **You should always make sure you are in your AICD folder before you launch Cadence.** To run cadence, use the following commands:

```
cd ~/AICD

virtuoso &
```

.

1. Learning Objectives of this Lab:

From the previous lab, you should be able to:

- Create, and simulate in terms of a DC-sweep, a CMOS differential voltage amplifier with resistive and current source loads.
- Understand the need for feedback in order to set the DC operating point (common-mode output voltage) of a fully differential amplifier.
- Understand the logical parts and basic arrangement of the common-mode feedback loop. Simulate a simple common-mode feedback loop using ideal components.

In this lab, you will:

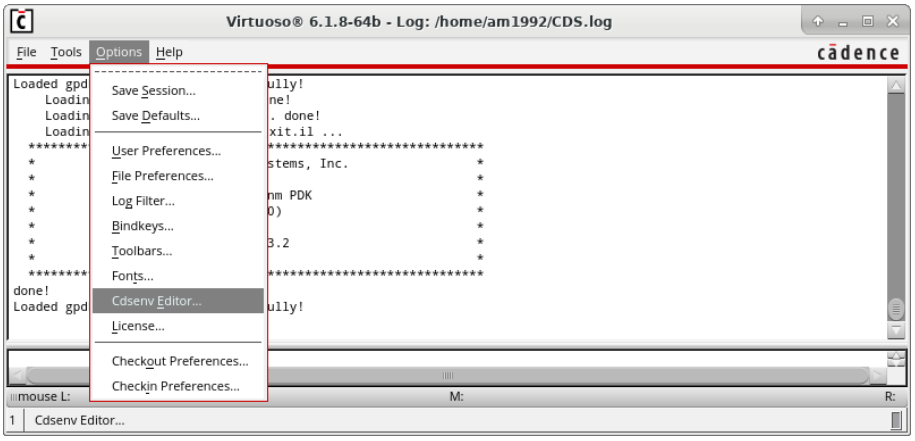
- Undertake a circuit operating point analysis over a temperature range.
- Use a parametric analysis to investigate the operation of a circuit at nominal and extremes of voltage supply.
- Simulate a CMOS FET circuit at its process corners.
- Examine the use of feedback to remove the problems associated with PVT variation.

2. Cadence Documentation, Help, and Licensing

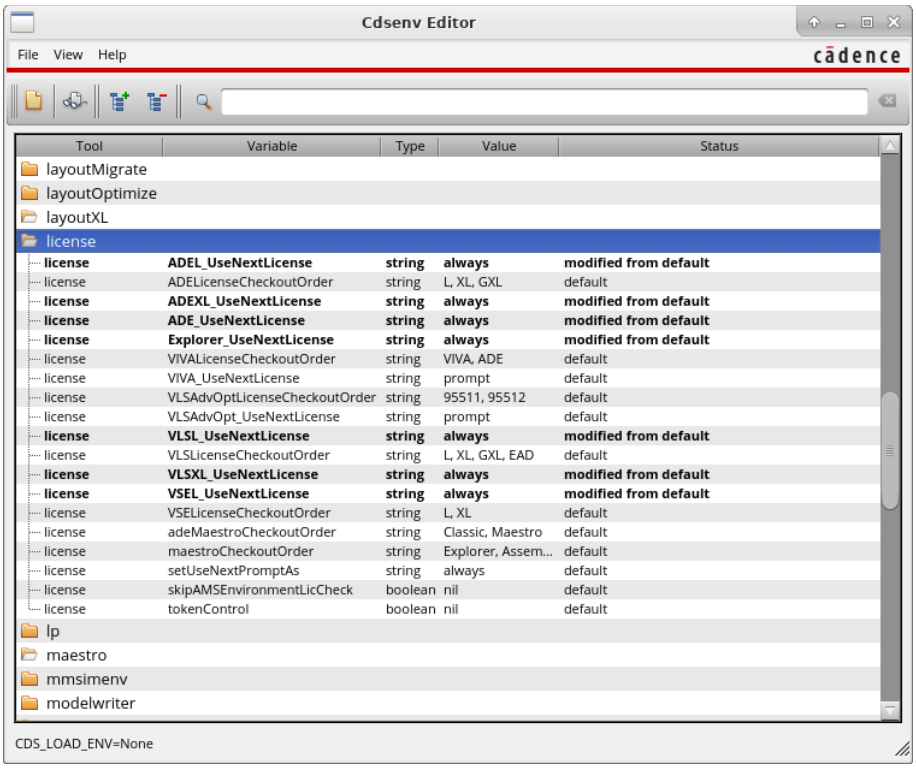
Local and online help is available from the help menus (**Menu: Help → User Guide** or by pressing **F1**), or by typing 'cdnshelp &' at the command prompt (assuming you have setup your terminal environment).

PLEASE NOTE: WHEN LAUNCHING A TOOL IN THE CADENCE DESIGN ENVIRONMENT YOU MAY SOMETIMES GET A WARNING ABOUT A LICENSE BEING UNAVAILABLE FOR THE TOOL, AND REQUESTING PERMISSION TO OBTAIN A HIGHER-TIERED LICENSE. YOU SHOULD SELECT 'ALWAYS' TO THESE MESSAGES.

If you accidently deny a license request, or you cannot access a license, you will need to modify the Cadence environment licence/permissions from the CIW. From the CIW menu go to **Menu: Options → Cdsenv Editor**. Once in the environment editor, go to the license folder, and make sure all licenses (in bold) are set to '**always'** – if any entries are listed as '**never**' you will need to manually change/retype them to '**always**'.



< Figure X.1. The Cadence Environment Menu Option from the CIW Window. >



< Figure X.2. The Cadence Environment Editor. Note, all licenses should be set to 'always'.>

3. Getting Started with Cadence:

For a refresher/introduction on how to start a Cadence session please read through sections 1 through 3 of the tutorial lab 12.1 instruction booklet/handout.

3.1 Dealing with Lock files

PLEASE NOTE: IF A SCHEMATIC ISN'T PROPERLY CLOSED DOWN (FOR EXAMPLE, DUE TO A DROPPED REMOTE CONNECTION), THE SCHEMATIC MIGHT BECOME EDIT LOCKED. IF THIS HAPPENS YOU WILL NOT BE ABLE TO OPEN THE FILE FOR EDITING. YOU WILL ONLY BE ABLE TO OPEN THE FILE FOR READING UNTIL YOU REMOVE ANY/ALL LOCK FILES.

There are two ways you can unlock files:

(6) The first is to manually delete any edit lock files. To do this, go to your '~/AICD/Designs' folder and find the folder representing the schematic e.g., for lab1 this would be '~/AICD/Designs/lab121_vdivider/schematic' if this is what you called your schematic. Delete all the .cdslck lock files in this folder and you should be able to open your schematic for editing.

(7) You can also automatically release edit locks with the command:

```
clsAdminTool -are .
```

To use this command, you have to be in your AICD folder (the full-stop is shorthand for the current directory path). If you are not in your AICD folder, you have to specify the path (relative or absolute) to your AICD directory.

Since this is a Cadence command, you have to ensure that you have set up the terminal environment for Cadence (in particular, the modules must be loaded as in step 2, 'Preliminary Matter'). It is best to use the terminal window that you used to start Cadence, since the environment was already set up. Note that you will not be able enter any commands into your terminal if you did not start cadence with the '&' character. By example, to release any lock files after opening a new terminal, the full command set would be,

```
cd~
module use /eda/cadence/modules
module load course/EENGM6030_2021
clsAdminTool -are AICD
```

For more information on this command, use

```
clsAdminTool -help
```

4. Circuit Simulation Exercises: Part 1

With guidance, you will:

- Undertake a circuit operating point analysis over a temperature range.

The simulation device models (PDKs) we use in analogue CAD are complex. Underlying these models are dozens of equations using several dozen model parameters – including physical variables, environmental variables, process variables, and coefficients of operation.

When designing analogue integrated circuits, we must be aware that the model parameters used in our simulations are often initially set to their nominal/typical 'statistical mean' values. Furthermore, the device/circuit temperature is also initially set to a nominal value, typically, 18–27°C). When realised, however, the circuits we are designing will be subject to significant variation in their manufacturing and processing (e.g., doping concentration profiles, dimensional tolerances, bulk and surface-level defects, etc.), and in their operating environment (chiefly, temperature and power supply). It is realistic to expect that two circuits fabricated on the same wafer may have significantly different performance.

Simulating for only the nominal conditions of process, voltage, and temperature is not sufficient to prove functionality of your design. We must ensure that our designs will function (1) correctly and (2) within specification, over all likely variations of **p**rocess, **v**oltage, and **t**emperature (PVT). Fortunately, simulation device models include information on the statistical variation of their process parameters. We use such information during the design process using two types of analyses – **Corner Analysis** (a form of worst-case/extreme-case analysis) and **Monte-Carlo Analysis** (a form of random analysis).

Internally, device model libraries are divided into sections. In the case of the corner analysis, the worst-case variations of process parameters are grouped according to their effect on **PMOS and NMOS device speed** – specifically, carrier mobility and transconductance. These groupings are placed in separate sections within the device model library:

- **NN**. The typical mean values of the model parameters that give rise to the average speed (nominal or typical) behaviour for PMOS/NMOS devices are grouped together in a section designated **NN** (Nominal NMOS / Nominal PMOS).

- **SS**. The worst-case combination of model parameters that give rise to the slowest NMOS and slowest PMOS devices (poor mobility/transconductance) are grouped together in a section of the device model designated **SS** (slow NMOS / slow PMOS).

- **FF**. The worst-case variation of process parameters that give rise to the fastest NMOS and fastest PMOS devices (high mobility/transconductance) are grouped together in a section of the device model designated **FF** (fast NMOS / fast PMOS).

- **SF** and **FS**. There are two other worst-case parameter combinations: slow-NMOS/fast-PMOS devices (**S/F**), and fast-NMOS/slow-PMOS devices (**F/S**).

Each of the four worst-case conditions represents a 'process corner' in our simulations:

Section	NMOS	PMOS
NN	Nominal	Nominal
SS	Slow	Slow
SF	Slow	Fast
FS	Fast	Slow
FF	Fast	Fast

< Table 1. Nominal Process Parameters plus Four 'Worst-Case' Process Corners used in the GPDK180 PDK >

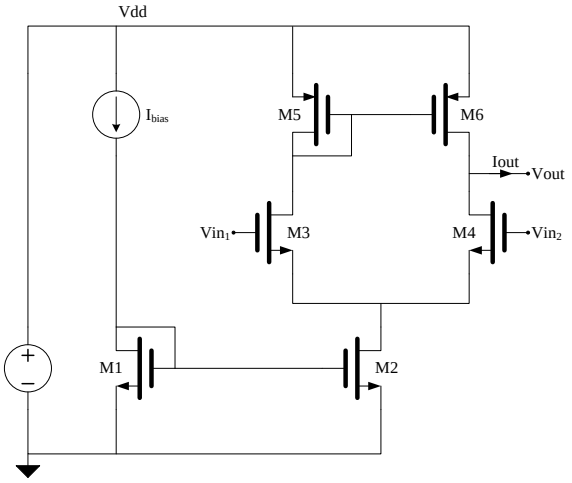
Note: Some PDK suppliers may use a different designation. For example, the AMS 0.35um PDK you used previously uses **TM** (typical-mean), **WS** (worst-case speed) and **WP** (worst-case power). In this case, **TM**=**NN**, **WS**=**SS** and **WP**=**FF** (fast devices come at the cost of using more power which is why WP=FF). Additionally, **WO**=**SF** (worst-case 'one' switching means slow PMOS) and **WZ=FS** (worst-case 'zero' switching means slow NMOS). uses this designation. The principle of table 1, however, is the same.

For a full PVT corner analysis, we must also vary the power supply and temperature in addition to the process corners, so that we can observe the ***simultaneous*** effect of process (NN, SS, FF, SF, FS), voltage (VDD +/- 10%), and temperature (-40°C to +125°C) on our circuits.

For the curious, the GPDK180 model library is based on the BSIM3v3 SPICE model (*BSIM* = **B**erkeley **S**hort-channel **I**nsulated-gate field-effect-transistor **M**odel). For a full list of the model equations and parameters you are referred to APPENDIX A of the BSIM3v3 manual https://link.springer.com/content/pdf/bbm%3A978-0-306-47050-9%2F1.pdf.

4.1 The Differential Transconductance Amplifier (TCA)

(4.1.1) Create the schematic for the NFET-inut differential transconductance amplifier in Figure 1. A summary of the circuit's operation is given in section 4.4 of the background material.

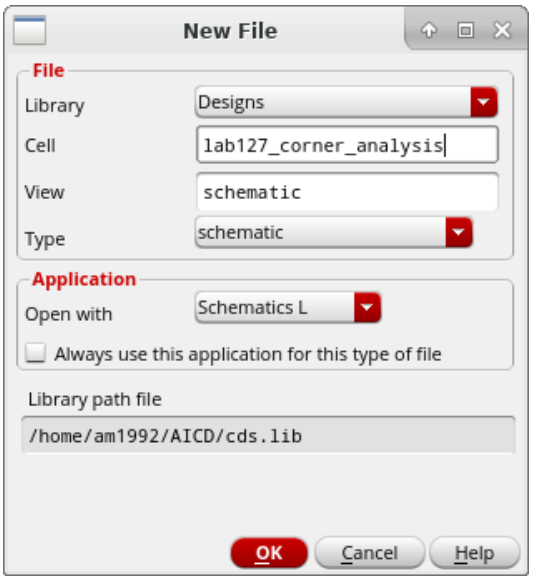


< Figure 201. Current Mirror Loaded, NFET Input, Differential Transconductance Amplifier (TCA). Reproduced from Figure 26, section 4.4 of the background material. >

(4.1.2) From the Library Manager window, select the 'Designs' library and choose **Menu: Files à New à Cell View** to invoke the New File form (Figure 2).

In the 'Cell' text field, choose a meaningful cell name (e.g., lab127_corner_analysis). In the 'View' text field, type 'schematic' as the view name and from the 'Type' drop-down menu choose 'schematic' as the view type. In the 'Application' section of the dialog box, choose 'Schematics L' from the 'Open with' drop down menu. OK the form.

(4.1.3) You should be presented with a blank **Virtuoso Schematic Editor L** schematic entry window for the new cellview. This cellview will not be saved in the Library Manager until you check and save it in the Schematic Editor. From the Schematic Editor window, choose **Menu: File à Check and Save**. (Or you can also **Menu: Check à Current_Cellview** followed by **Menu: File à Save** to perform the same command.)



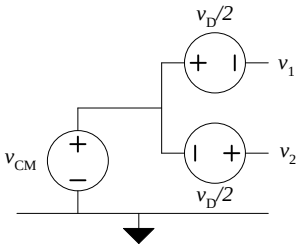
< Figure 202. The New File Form >

(4.1.4) In your schematic editor window, from the GPDK180 library, instantiate and place four **nmoshv** NMOS FETs (two for the input pair, and two for the current mirror tail bias) and two **pmoshv** PMOS FETs (for the active current-mirror load).

*Recall, there are several ways of rotating/flipping a part. When adding an instance to your schematic you can rotate and/or flip the item before you place it on the canvas using the **Rotate**, **Sideways**, and **Upside-Down** buttons on the Add Instance form. Alternatively, you can rotate/flip the components after placing them by selecting the component and using the **rotate** or **flip** commands from the menu/toolbar. You can also flip or rotate a component by using **F3** with the **move/stretch** commands. In general, pressing F3 while a command is in progress gives you a more detailed set of options for that command. When moving an item using the **Move** or **Stretch** commands, pressing F3 will bring up a dialog box with more detailed options including **Rotate**, **Sideways**, and **Upside-Down** buttons.*

(4.1.5) Instantiate and place an ideal **vdc** source for the supply voltage and an ideal **idc** current source for the tail current reference. All ideal components are from the analogLib library.

(4.1.6) To drive the circuit, use a single **vdc** for the common-mode input voltage (VCM) and add two split **vdc** inputs for to the differential-mode input voltage (vd). This will enforce the common-mode voltage (CMV) principle rather than having separate VCMs driven by the same parameter for the two inputs (**note the opposing orientation of the two v_D/2 supplies – if you orient them in the same direction make one of the supplies negative, -vd/2**). See Figure 3.



< Figure 203. Common-mode voltage driver circuit.

v_1 = VCM - vd/2. v_2 = VCM + vd/2. v_D = (v_1 - v_2). VCM = (v_1 + v_2)/2.>

(4.1.7) Parameterise your design. Set the tail current reference to 100 uA (**DC Current** = 100u) For the DC supply voltage, set the voltage attribute to the string **DC Voltage** = VDD. For the input set the common-mode voltage component to the string **DC Voltage** = VCM, and the two differential-mode components to the string **DC Voltage** = vd/2 and - vd/2. Dimension all PMOS and NMOS devices 10um/1um (**width** = *10u*, **length** = *1u*). Leave all other attributes unchanged.

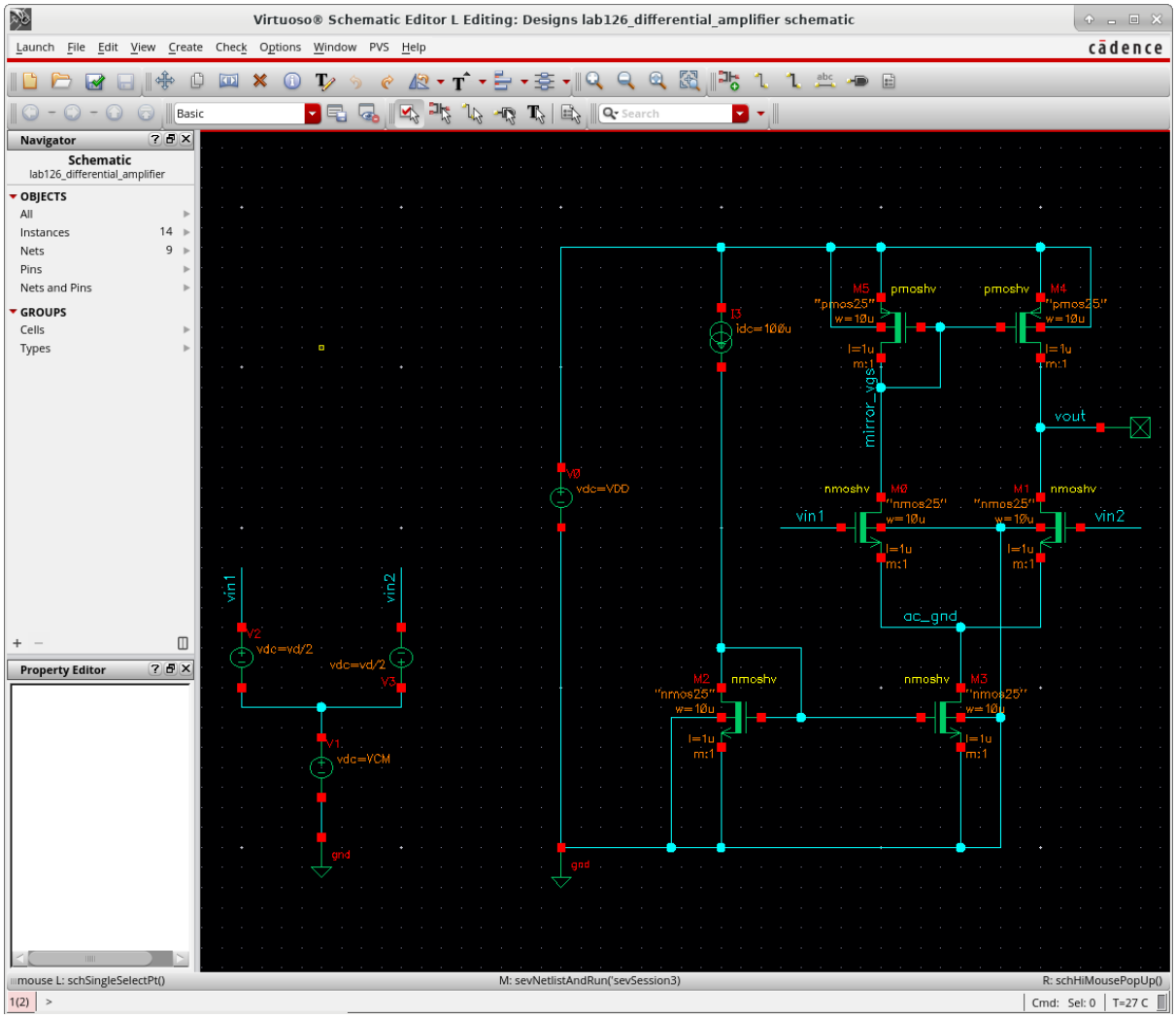
(4.1.8) Wire the components as in Figure 4 using the **Wire (narrow)** command.

(4.1.9) Using the **Create Wire Name** command: name the two input nets vin1 and vin2 (the gates of the differential input pair), name the output net vout, the drain of the mirror load mirror_vgs, and name the AC ground point ac_gnd. Use the **noConn** connector from the **basic** library to avoid floating-net warnings. Keep the schematic neat.

(4.1.10) To connect the inputs to the amplifier to the driver circuit, we will create an implicit connection using wire names. Name the outputs of the common-mode driver circuit vin1 and vin2. This will create a connection between the vin1 of the driver circuit and vin1 of the amplifier, and a connection between vin2 of the driver and vin2 of the amplifier. See Figure 4.

REMEMBER: FOR AN NMOS DEVICE, YOU MUST ALWAYS CONNECT THE BODY/BULK TERMINAL TO THE MOST NEGATIVE NET (WIRE) OF THE CIRCUIT. IN FIGURE 4, THIS IS THE GROUND TERMINAL. FOR A PMOS DEVICE, THE BODY/BULK TERMINAL MUST ALWAYS BE CONNECTED TO THE MOST POSITIVE NET (WIRE) OF THE CIRCUIT, USUALLY THE SUPPLY VDD. THIS IS NECESSARY TO ENSURE THAT THE xMOS CHANNELS WILL BE PROPERLY ESTABLISHED.

(4.1.11) Check and save your schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.



< Figure 204. The Differential Transconductance Amplifier Schematic as in Figure 1. >

4.2 Simulating the Amplifier over a Temperature Range

(4.2.1) We will first simulate the DC operating point of the amplifier. Ordinarily, we would do this using the 'Save DC Operating Point' option on the DC analyses form, and then annotate the schematic. However, here we want to observe how the DC operating point varies with temperature so we will do this using a DC Sweep Analysis – with temperature as the sweep variable.

(4.2.2) From the Schematic Editor window, launch the Analogue Design Environment (ADE) – **Menu: Launch à ADE L**. Copy the design variables from the schematic into the ADE. For this circuit, these are the DC voltages from the three **vdc** components – VDD, VCM, and vd. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the VDD, VCM, and vd variables. in the 'Design Variables' list of the ADE. Initialise VDD *= 2.5V*, VCM *= 1.25V,* and vd = 0V. These values are used only to set the DC operating point of the amplifier circuit.

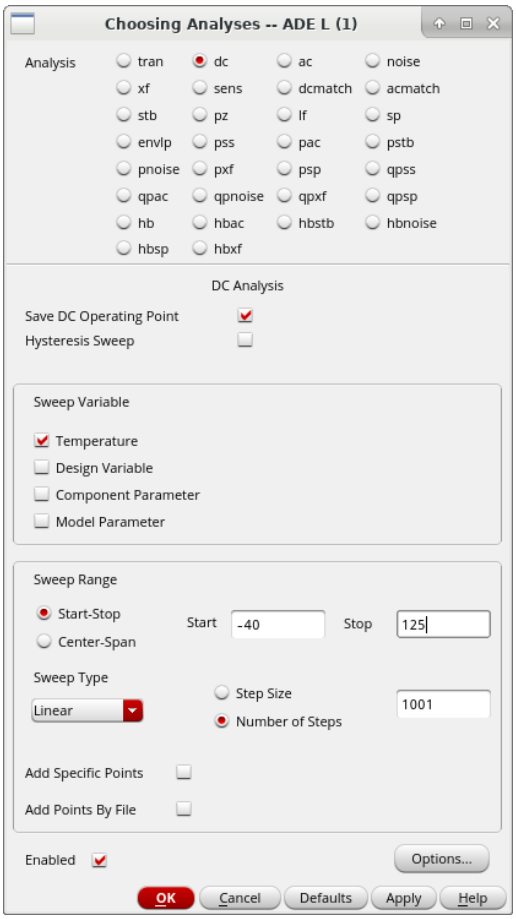
(4.2.3) We want to plot the various nets/nodes around the differential amplifier as we sweep the temperature from -40°C to +125°C. This is the full temperature range for industrial applications. See table 2.

Electronic Application	**Temperature Range (°C)**
Commercial	0 to +70
Hi-Temperature Commercial	-10 to +100
Extended Commercial	-20 to +85
Industrial	-40 to +85
Automotive	-40 to +105
Full-Range Industrial	-40 to +125
Power Supply	-40 to +130
Military	-55 to +125

< Table 2. Temperature Ranges for Different Applications. >

(4.2.4) From the ADE window, go to **Menu: Analyses à Choose…** or use the '**Choose Analyses**' button on the right-side of the ADE window (top button). The 'Choosing Analyses – ADE L' form will open. From the form, select the '**dc**' analysis option button.

In the choosing analyses form. Check the ‘Save DC Operating Point’ box. Check the ’**Temperature**’ box as the sweep variable. Choose the ‘Start-Stop’ option button with Start = -40 and Stop = 125 for the sweep range. Select ‘Linear’ from the drop-down menu as the sweep type. Choose the ‘Number of Steps’ option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the ‘Enabled’ box near the bottom of the form. OK the form. See Figures 5 and 6. The **dc** analysis should now appear in the ‘Analyses’ list of the ADE window.



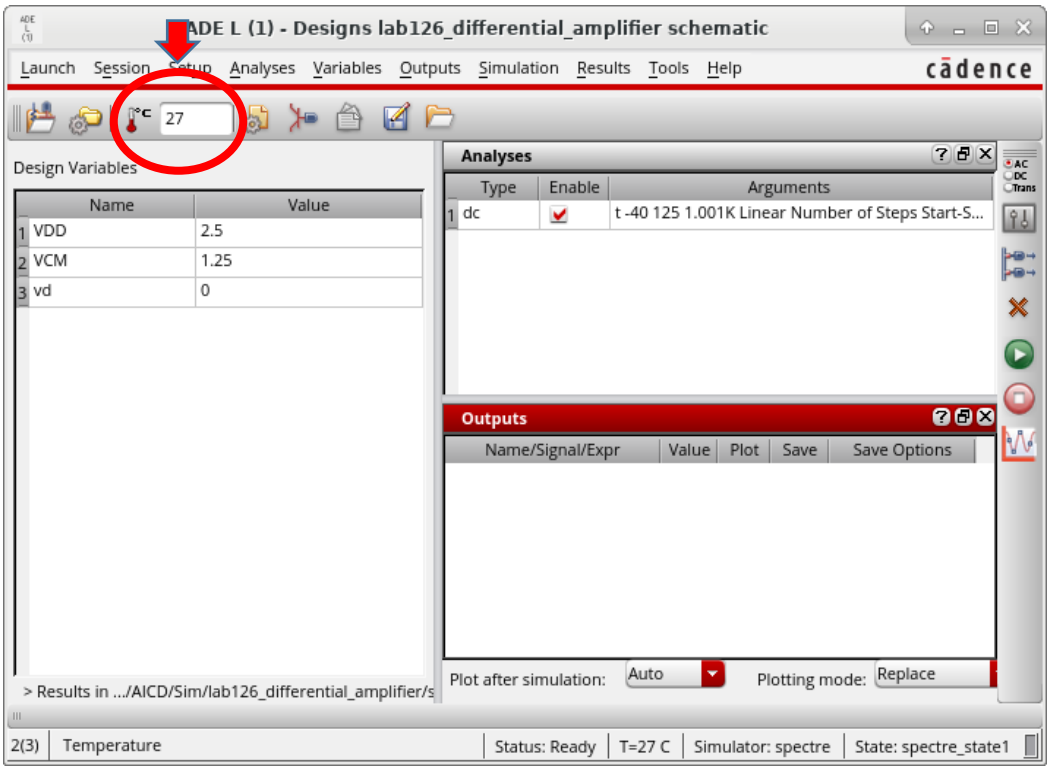
< Figure 205. The Choosing Analyses Form. >

(4.2.5) At this stage, don’t select any outputs to be plotted as we will use the Direct-Plot form instead – post simulation. In the Analog Design Environment window, run the ‘**Netlist and Run**’ command by clicking on the green play button. Only the text output window will open (since we aren’t plotting any waveforms yet).

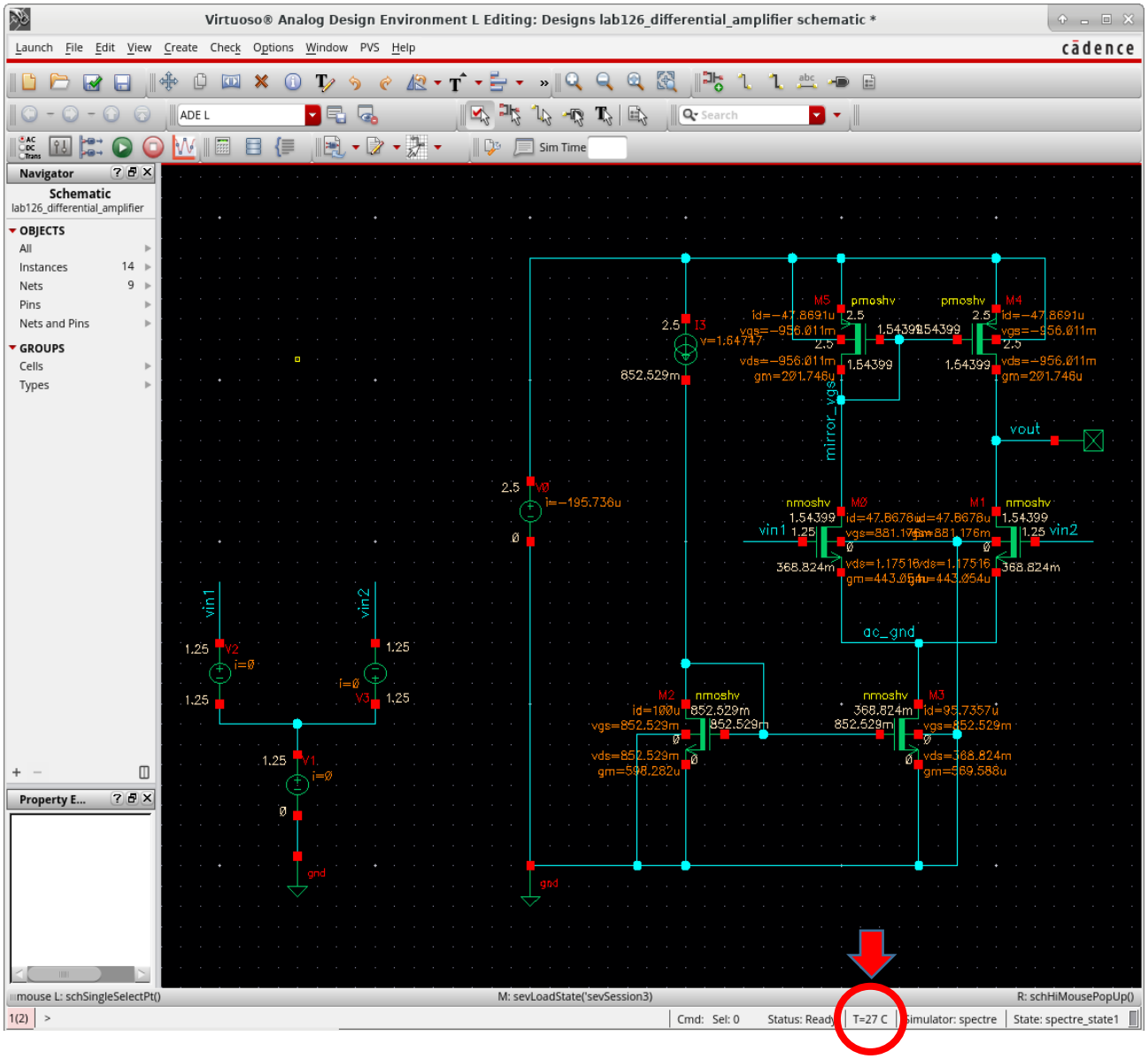
(4.2.6) Annotate the schematic, **Menu: Results à Annotate à DC node voltages…** and **Menu: Results à Annotate à DC operating points…** See Figure 7.

(4.2.7) **Discussion**. In all of the previous labs, we have simulated our circuits at a temperature of 27°C – this is the nominal temperature. This value is set in the top right of the ADE in the icon toolbar. This sets the temperature of all devices in the schematic. This will not affect the ideal components as these are not dependent on temperature. For an individual simulation you can, change the temperature on the toolbar. You can also change the temperature units (Celsius, Fahrenheit, or Kelvin) by clicking on the thermometer icon. You can also see the temperature on the status bar at the bottom of the schematic editor window.

Note that when you annotate a schematic, the annotations are for the nominal temperature (as well design variables) set in the ADE. In this case, nominal DC output operating point vout is 1.54399V (at VDD=2.5V, Temp=27°C, and Process=NN).

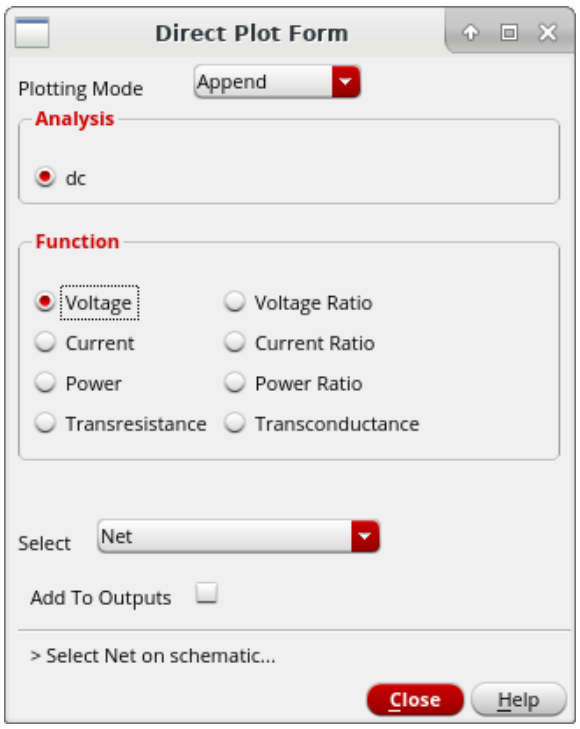


< Figure 206. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 207. Annotated. Differential Transconductance Amplifier Schematic >

(4.2.8) From the ADE window, select **Menu: Results → Direct Plot → Main Form** to invoke the Direct Plot Form (this may also invoke an empty waveform window).



< Figure 208. The Direct Plot Form for DC Analysis. >

Note: Recall, that the Direct Plot command of the ADE is a more general approach to plotting outputs after a simulation has run. Using this option, you can quickly probe the circuit – selecting and plotting the waveforms at different nets (voltages) and nodes (currents) one by one – without having to set them up in the ADE beforehand. You can also plot common expressions quickly without having to use the calculator. By default, all voltages (nets) are saved and are always available for plotting using the Direct Plot form.

Note: By default, not all currents (nodes) are saved and therefore may not be available for plotting using the Direct Plot form (MOSFET nodes are not available unless saved). To make all currents available for plotting, we can change the default behaviour of the simulator to save all device currents ***Menu: Outputs → Save All*** *and check the 'all' option in the 'Select device currents (currents)' section, or use select* ***Menu: Outputs → To Be Saved → Select On Schematic*** *and choose the nodes you want to save (you will have to re-run the simulation). These will then be available for plotting from the Direct plot form.*

Note: In the 'Analysis' section of the Direct Plot Form there will be an option button for every analysis type setup and run in the ADE. In this case, since we have only setup and run a DC analysis, there is only a ***dc*** *option present. Associated with each analysis type, will be a number of plotting options; for the ac analysis we have the function options (e.g., voltage, current, power…) and the modifier options (there are no modifier options for dc analysis - the modifier options available can be very different for the different analysis types (ac, dc, trans, stb…).*

Note: At the very top of the form is the plotting mode drop-down menu, with the usual options – Replace, Append, New Sub-Window, and New Window. Leave it on the 'Append' option.

(4.2.9) From the Direct Plot Form:

1) Select the **voltage** option in the function section of the direct plot form. Then, in the schematic window click on the output net *vout.* You should now have a plot of the dc voltage at this point over the temperature range -40°C to +125°C. See Figure 9.

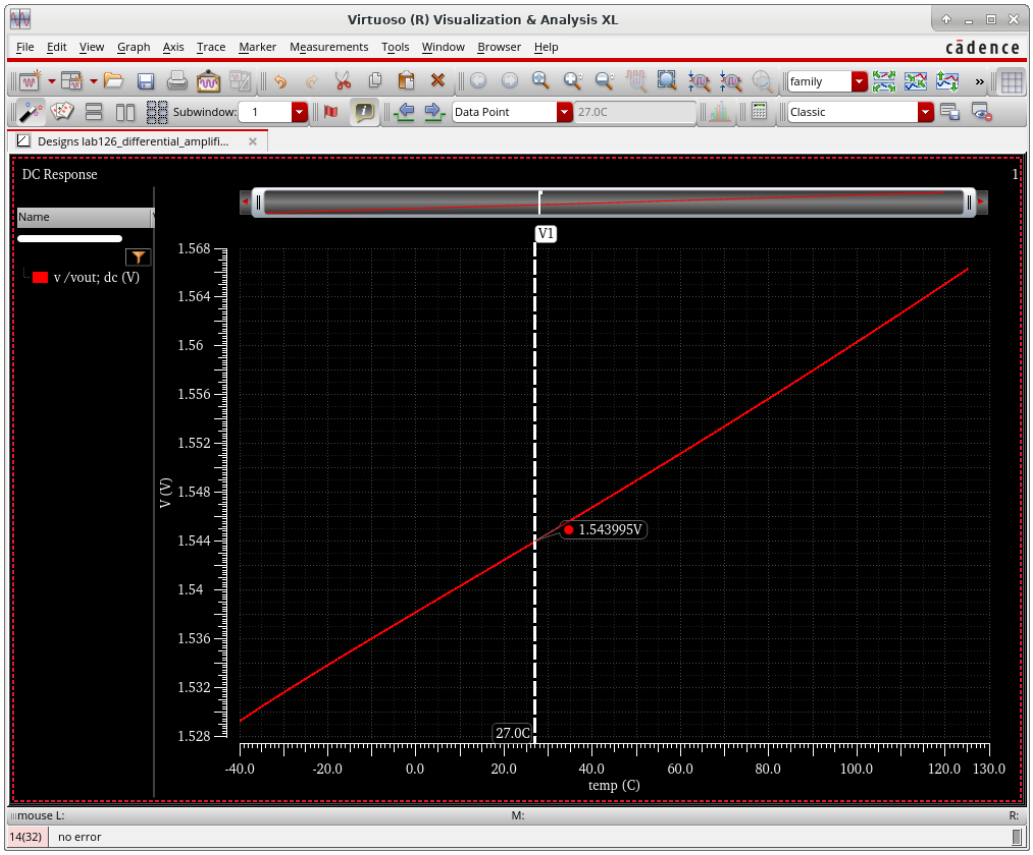
2) Select the **current** option in the function section of the direct plot form. Then, in the schematic window click on the current node for the drain of the tail device. You should now have a plot of the dc node currents over the temperature range -40°C to +125°C. See Figure 10. If the tail current is not available to plot you will need to follow the instructions above for it to be saved and re-run the simulation.

(4.2.10) From the schematic editor window, **press the 'ESC' key to exit the plotting mode of the Direct Plot Form**. Or simply close down the Direct Plot Form. Split the traces for clarity. Right-click on the empty area of the graph window and select Split Current Strip à trace from the context menu. A vertical marker has been added at the nominal temperature 27°C

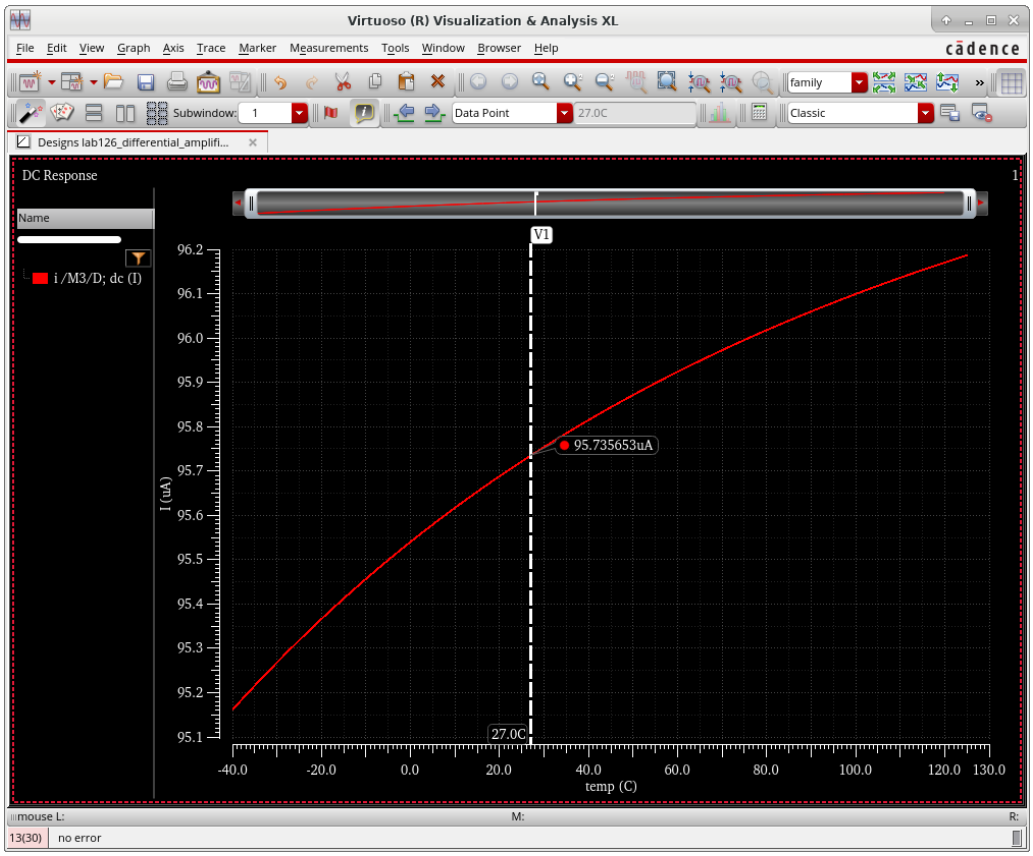
(4.2.11) **Discussion**. Referring to Figures 9 and 10. A marker has been placed on the temperature sweep at 27°C. This identifies the nominal value (1.54399V) we saw previously on our schematic annotation (see Figure 7).

For the NFET-input differential transconductance amplifier the change in operating point v_{out} over the 165°C sweep (-40 to +125°C) is around 40mV. This would not be considered significant – a change of 2 to 3% over the full range or Nominal +/- 1.5%. Similarly for the tail current, the nominal value (at VDD=2.5V, Temp-27°C, and Process=NN) is 95.736uA. This varies by +/-0.6uA over the full temperature range (a change of Nominal +/-0.6%).

This Differential Transconductance Amplifier is therefore relatively robust to temperature variation – though bear in mind this is most likely due to the large 10um/1um device sizes.



< Figure 209. DC operating point v_{out} of the TCA for temperature -40°C to +125°C >

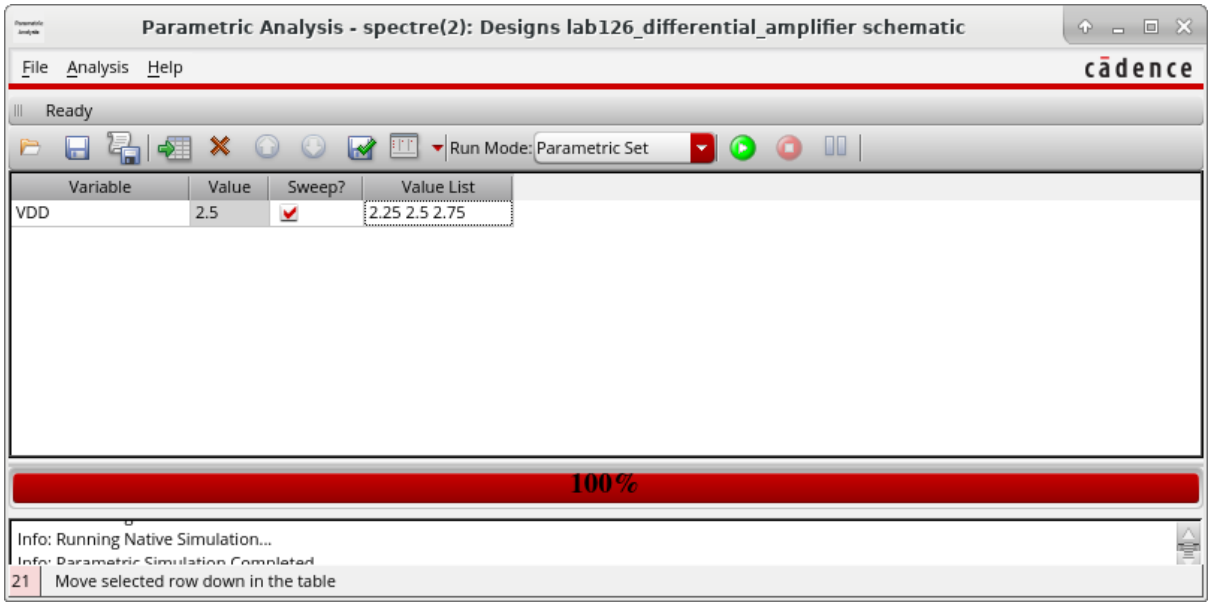


< Figure 210. DC operating point I_{TAIL} of the TCA for temperature -40°C to +125°C >

4.3 Simulating the Amplifier over a Temperature and Supply Voltage Range

(4.3.1) As well as observing the effects of varying temperature, we want to observe how our design will perform when there is a variation on the power supply voltage. In our simulations we will assume a standard VDD variation of +/-10%. We will setup a parametric analysis so we can observe the ***simultaneous*** effect of temperature and VDD variation on our circuit.

(4.3.2) To sweep VDD +/-10% we will use the parametric analysis tool. From the ADE window, choose **Menu: Tools → Parametric Analysis…** to invoke the Parametric Analysis tool. Note, that a parametric sweep can be specified as a range to sweep from the tool bar (Run Mode → Sweeps and Ranges) or as an explicit list of values over which to iterate (Run Mode → Parametric Set).



< Figure 211. Parametric Analysis Tool. For VDD = 2.5V, set up the parametric set VDD (+/-10%) = 2.25V, 2.5V, and 2.75V. This will run the swept DC-analysis (temp -40°C to 125°C) 3 times to create a family of curves. >

(4.3.3) In the Parametric Analysis window, select 'Parametric Set' from the '**Run Mode'** drop down menu in the toolbar.

In the '**Variable'** column (main window), you should get a drop-down menu of the available design variables. Select VDD as the parametric sweep variable. Note, only the design variables in the 'Design Variables' section of the ADE will be available in the drop-down list along with the system variable *temp*. So, make sure you have copied the design variables from the schematic into the ADE using the **Menu: Variables → Copy From Cellview…** command.

The '**Value**' column will list the nominal value of the parametric variable as set up in the ADE, in this case VDD will be 2.5V. In the '**Sweep**' column, make sure the option box is checked to enable the sweep.

In the '**Value List**' column enter the list of values – separated by spaces - for the VDD values that you want to simulate. For VDD +/-10%, this would be 2.25V, 2.5V, 2.75V. See Figure 11.

(4.3.4) The parametric analysis is run from the parametric analysis window not from the ADE window. In the Parametric Analysis window, run the '**Run Selected Sweeps**' command by clicking on the green play button. The simulation should run properly and both the text output window as well as the waveform output window should open automatically.

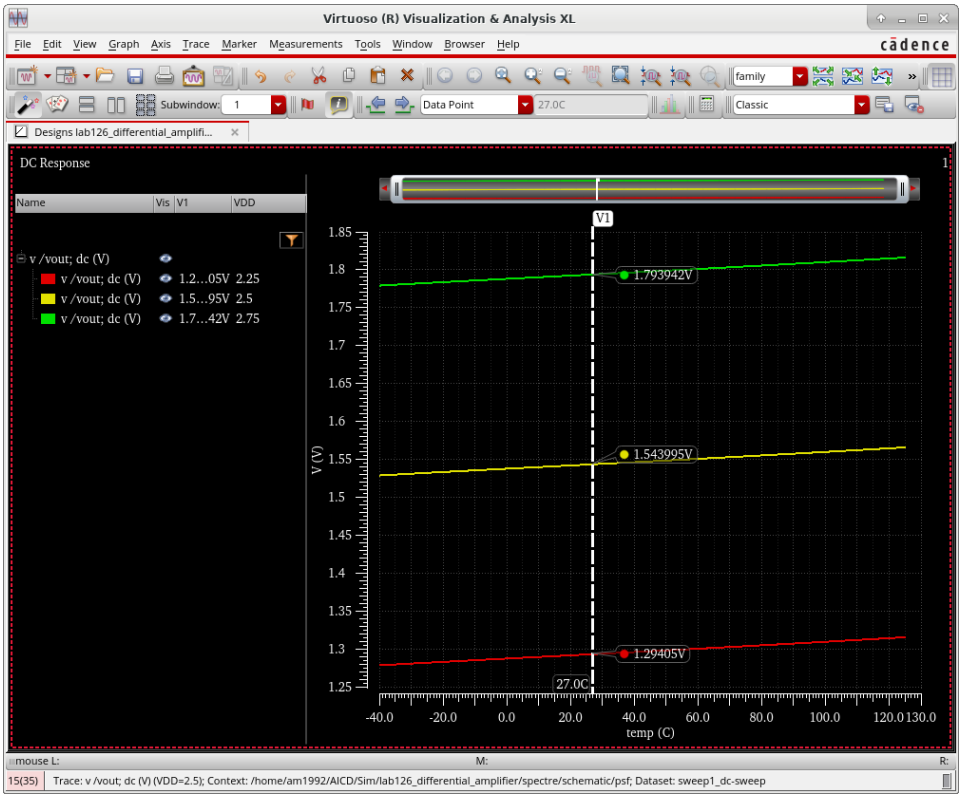
With the above settings, the ADE will perform a swept-DC analysis of *temp* from -40°C to 125°C, a total of 3 times – one simulation for each value of *VDD* (2.25V, 2.5V, 2.75*V*). See Figures 12 and 13.

(4.3.5) **Discussion**. When we add the effect of VDD variation (+/- 10%) to the temperature variation (-40 to 125°C), the DC operating point at v_{out} falls to 1.28V at its lowest (at VDD=2.25V, Temp=-40°C, and Process=NN) and rises to 1.81V at its highest (at VDD=2.75V, Temp=125°C, and Process=NN). This gives a total variation of around 0.5V over the full range of temperature and VDD variation. Under nominal conditions v_{out} is 1.55V (at VDD=2.5V).

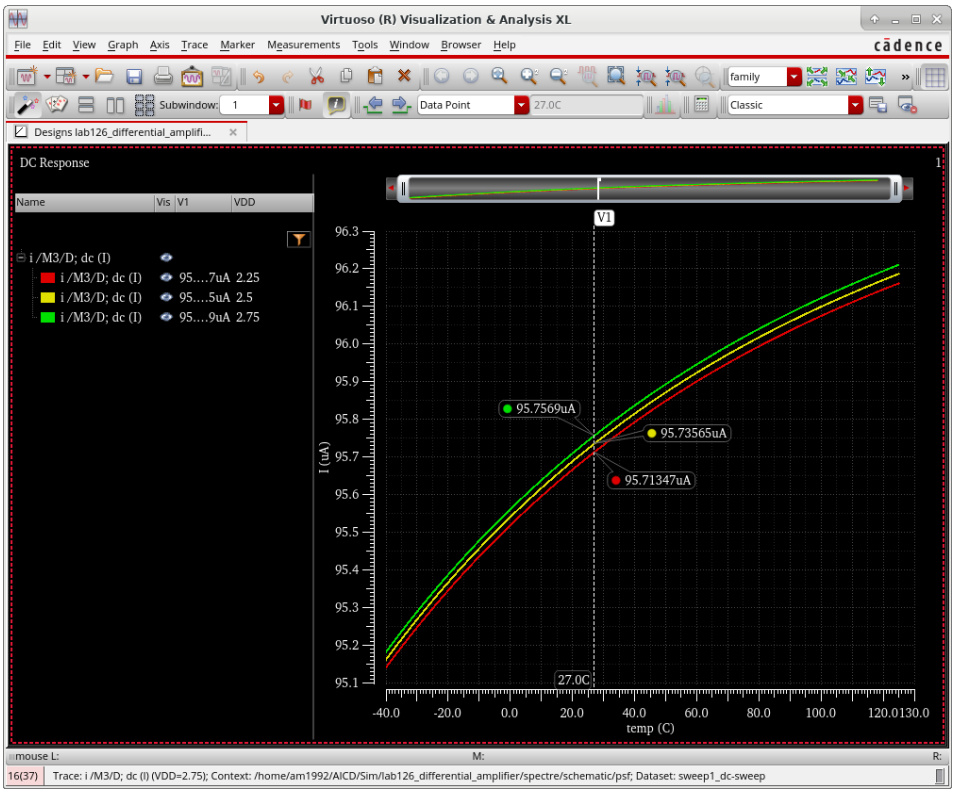
We want the DC output voltage to remain at approximately mid-rail for maximum signal-swing (VDD/2). As a ratio of vout/VDD we get 1.55V/2.5V for the nominal case, and 1.28V/2.25V and 1.81V/2.75V for the corner cases. The DC output voltage remains close to 55% to 65% of VDD in all cases (relatively close to mid-rail, 50% of VDD).

The current mirror loaded TCA shows a satisfactory level of robustness to changes in temperature and supply voltage over a wide range. However, whether we can say that this level of robustness is acceptable will depend on the required specifications for the application.

Note, we have not yet introduced any variation of the process parameters.



< Figure 212. DC operating point v_{out} of the TCA for temperature -40°C to +125°C for VDD = 2.25V, 2.5V, and 2.75V >



< Figure 213. DC operating point I_{TAIL} of the TCA for temperature -40°C to +125°C for VDD = 2.25V, 2.5V, and 2.75V >

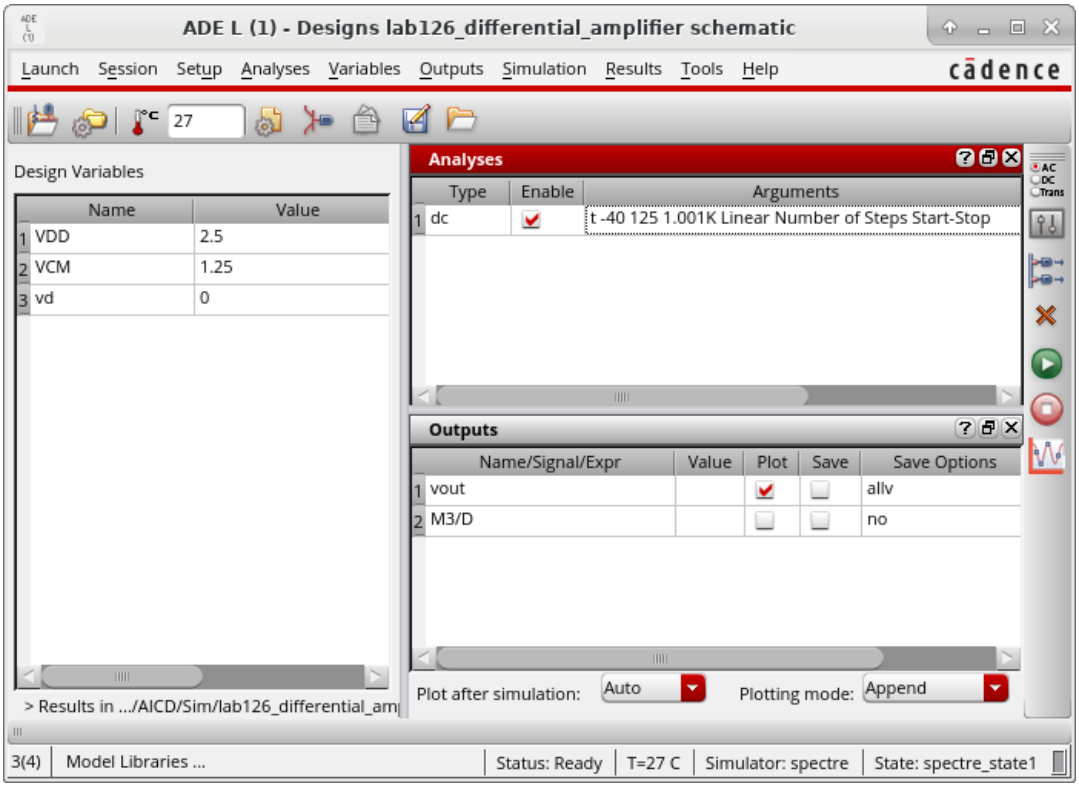
4.4 Simulating the Amplifier at Process Corner, Supply Voltage, and Temperature (PVT)

(4.4.1) We will setup a parametric analysis so we can observe the ***simultaneous*** effect of process (NN, SS, FF, SF, FS), voltage (2.25, 2.5V, 2.75V), and temperature (-40°C to +125°C) – PVT variation – on our circuit.

(4.4.2) While there are ways of automating the changing of process corners, we will be doing this manually (conducting multiple runs of the simulator) once for each process corner (SS, SF, FS, FF) plus nominal NN. This will require five parametric sweeps in total. Since this can be quite repetitive, we will add the outputs we wish to observe – the output net vout and the tail current node I_{TAIL} – to the ADE.

(4.4.3) From the ADE, select **Menu: Outputs → To Be Plotted → Select On Schematic** to enter the plotting mode of the ADE**.** Select the output net vout and the drain node current of the tail device (M3/D) for plotting. For now, only select vout for plotting. Note that your transistor names may be different – it depends on the order in which you instantiated the devices.

Make sure the Plotting Mode is set to 'Append'. This will ensure the output plots are added to the waveform window automatically on each run of the simulator. **Press the 'ESC' key to exit the plotting mode of the ADE**. See Figure 14.

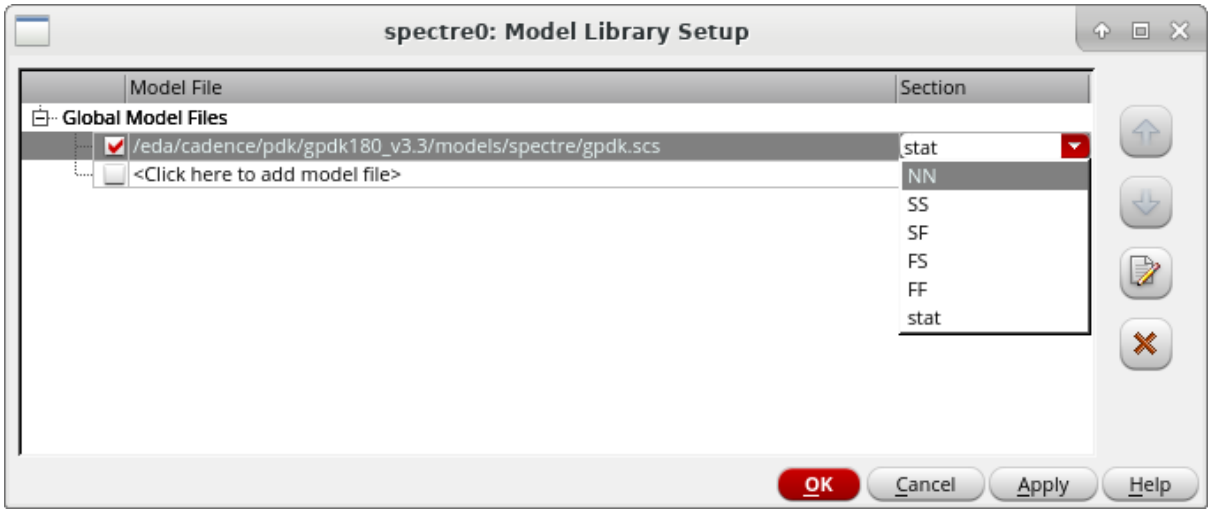


< Figure 214. The ADE with the Design Variables, Analyses and Outputs. >

Setting the Model Library Sections

(4.4.5) The Model Library file contains the model files that describe the NMOS and PMOS devices during simulation. As mentioned in the introduction, the device library models are divided into sections. There are typically six sections – stat, NN, SS, FS, SF, and FF that group together the model parameter variations according to their effect on NMOS and PMOS transistor speed.

(4.4.6) Model library files settings. From the ADE, go to **Menu: Setup → Model Libraries** to invoke the Model Library Setup form. See Figure 15. The main column lists the model files loaded by the Spectre simulator for use in our simulations (Recall that Spectre is the name of the SPICE simulator that the ADE runs). In our case, this is the GPDK.scs model file for the GPDK180 technology library (the '.scs' extension stands for 'spectre circuit simulator'). Note that this is the main model file; it links to several other files in the technology library (e.g., cmos.scs, diode.scs, bipolar.scs, capacitor.scs, resistor.scs, …).



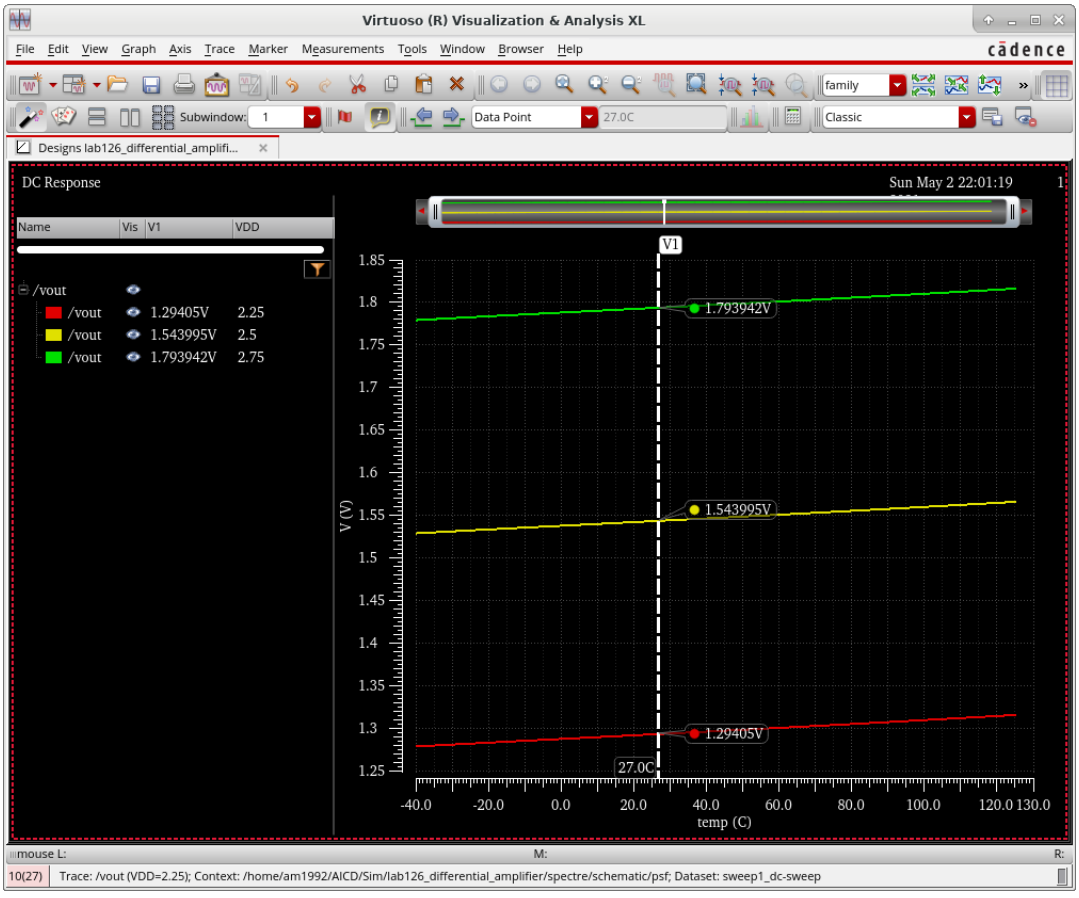
< Figure 215. The Model Library Setup Form. >

(4.4.7) In the 'section' column, alongside the model file, is a drop-down menu that lists the sections of the model file.

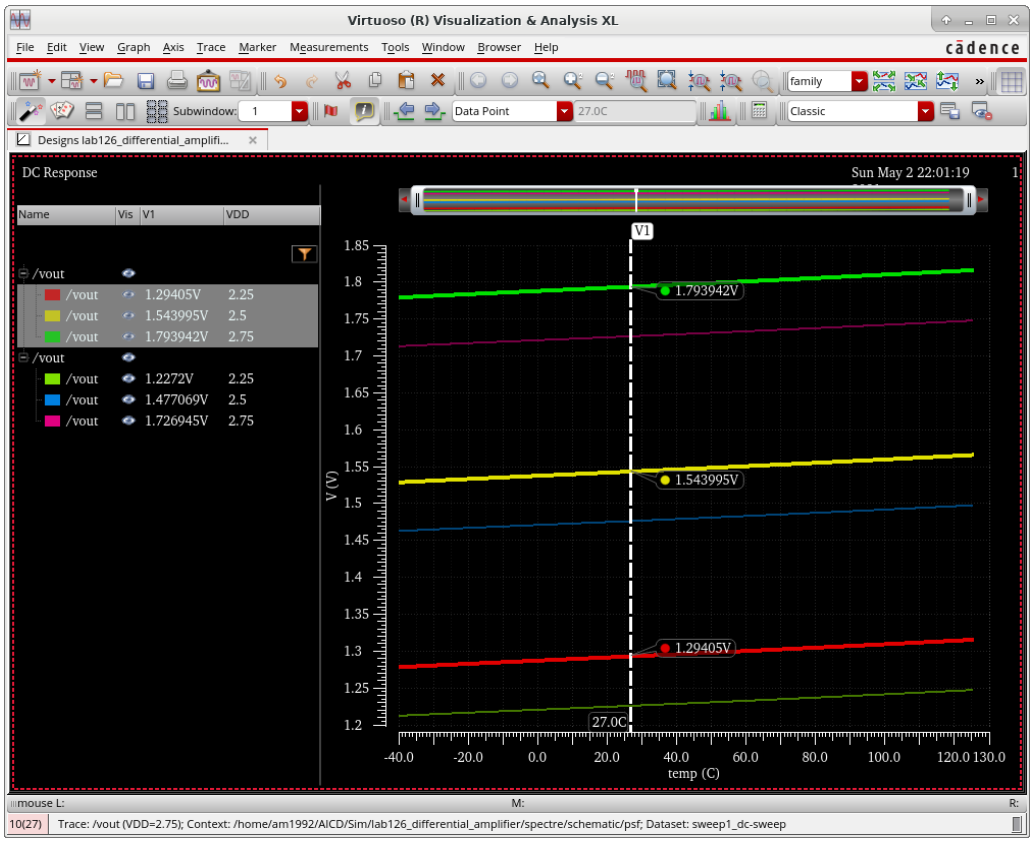
By default, this is set to **stat** – this is used for statistical analyses/ simulations (such as a Monte-Carlo analysis). However, to make use of the statistical analysis section requires additional functionality to instruct the simulator how the model parameters should be varied. This is available in the ADE-XL version of the ADE, but is not available in the basic ADE-L version which we use in this course. By default, **stat** behaves identical to the **NN** (nominal) option if no information on how to vary the parameters is supplied.

(4.4.8) Set the process section to **NN** (nominal/nominal) for the typical-mean performance. Apply or OK the form. From the Parametric Analysis window, run the '**Run Selected Sweeps**' command by clicking on the green play button. The waveform window should open and plot the output net vout for the **NN** process parameter case, across a temperature sweep -40°C to 125°C, at the voltage corners for VDD +/-10% (2.25V, 2.5V, and 2.75V).

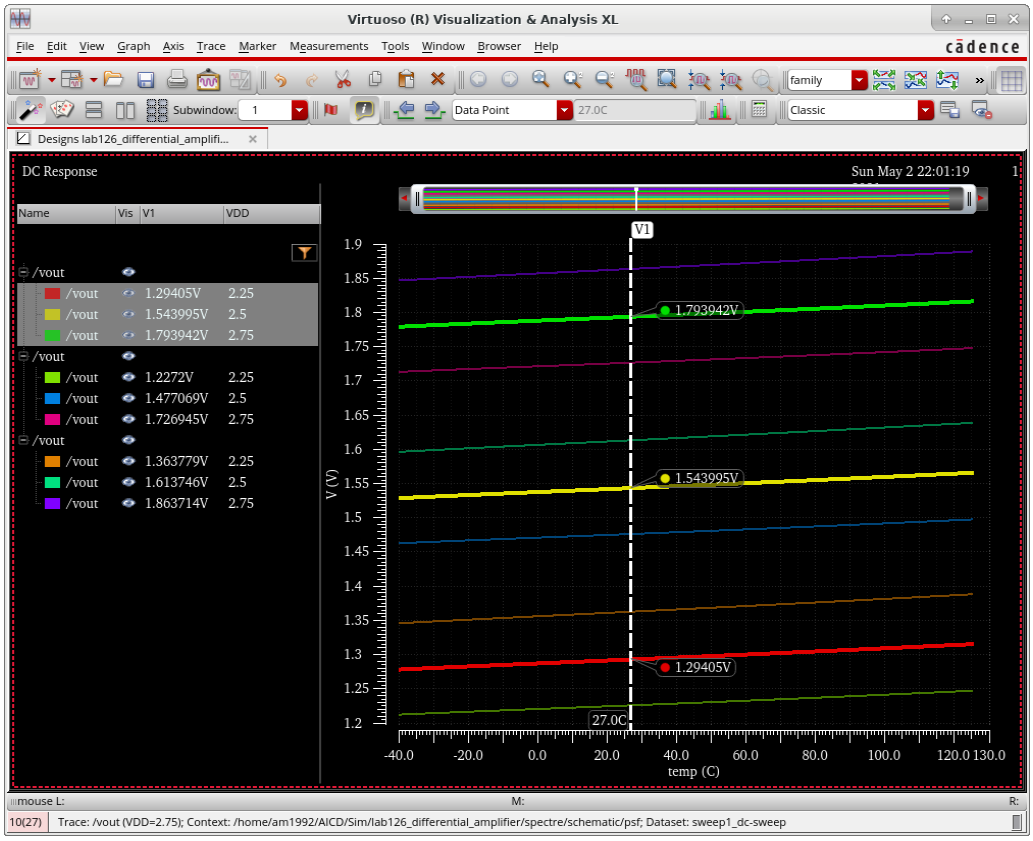
(4.4.9) Repeat the previous step for all process corners **SS**, **FF**, **SF**, **FS** (the order is not important here). For each corner, change the process section, apply or OK the form, and re-run the simulation. **Remember to run the simulation from the parametric tool not the ADE.** Ensure your ADE plotting mode is set to 'Append' so that each previous corner simulation is retained.



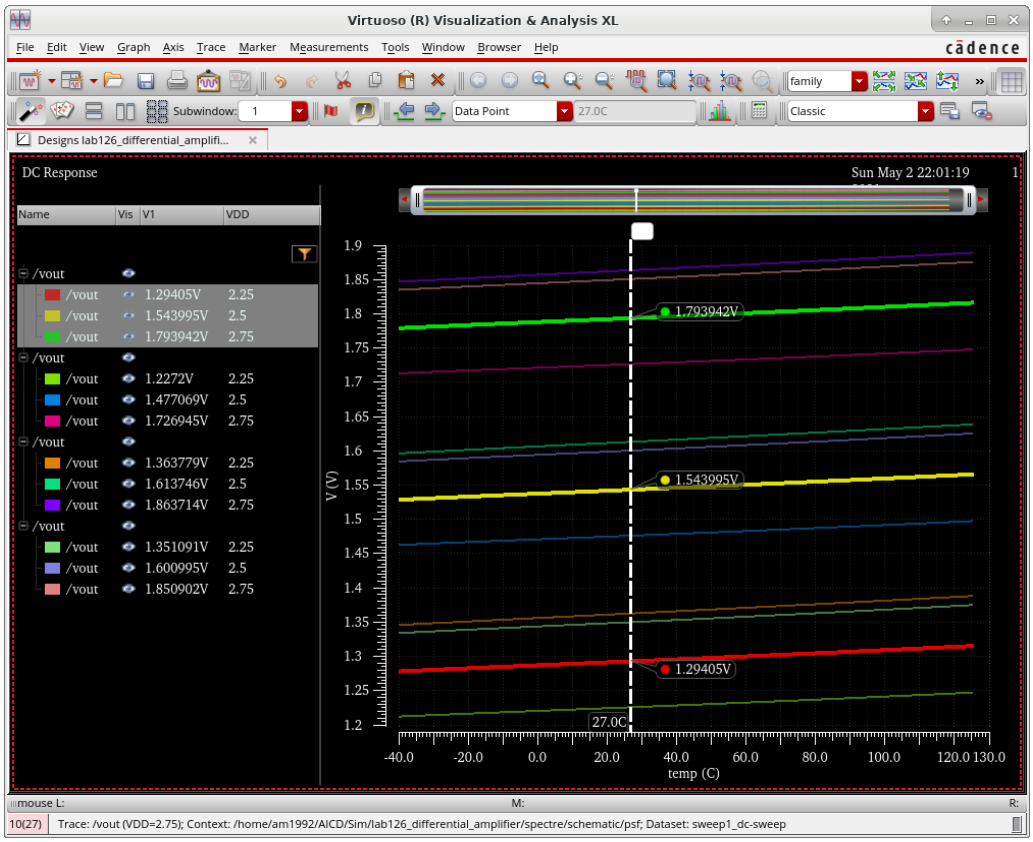
< Figure 216. DC operating point v_{out} of the TCA for temperature -40°C to +125°C for VDD = 2.25V, 2.5V, and 2.75V. For NN Process Section. >



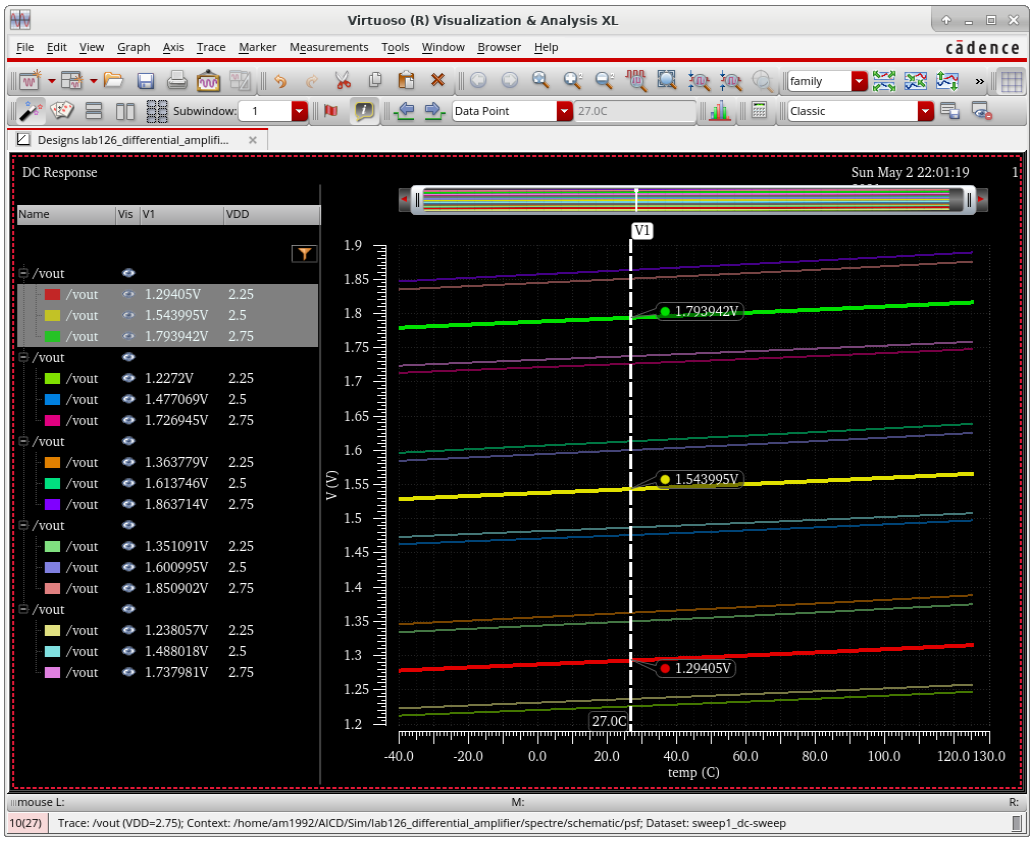
< Figure 217. DC operating point v_{out} of the TCA for temperature -40°C to +125°C for VDD = 2.25V, 2.5V, and 2.75V. For NN+SS Process Sections. >



< Figure 218. DC operating point v_{out} of the TCA for temperature -40°C to +125°C for VDD = 2.25V, 2.5V, and 2.75V. For NN+SS+FF Process Sections. >

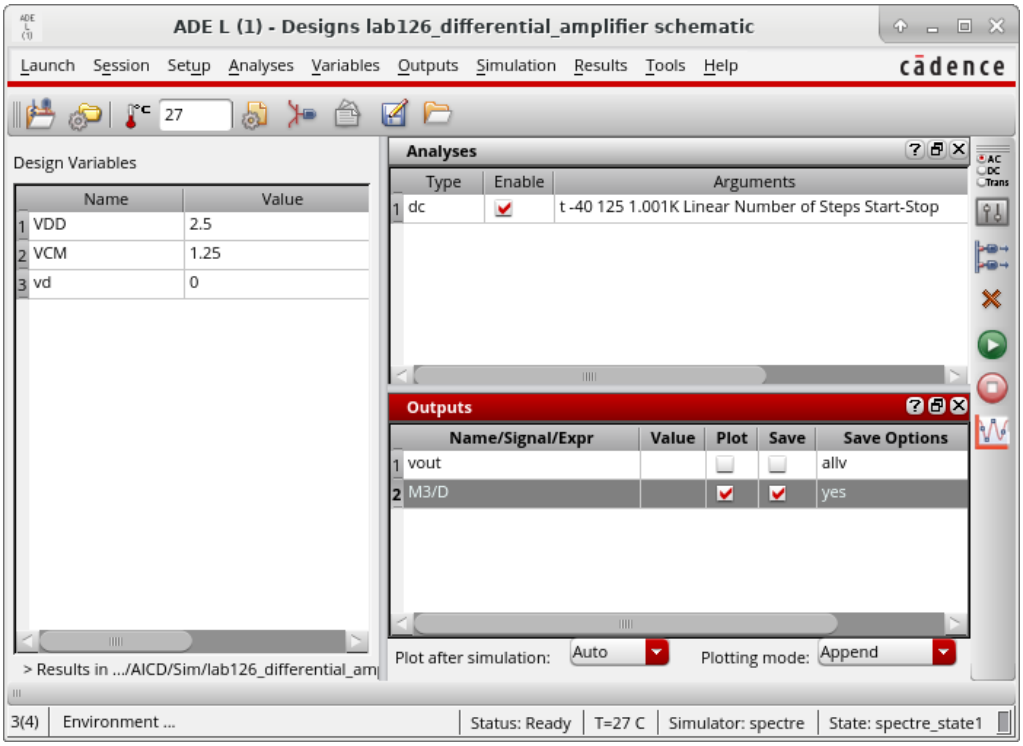


< Figure 219. DC operating point v_{out} of the TCA for temperature -40°C to +125°C for VDD = 2.25V, 2.5V, and 2.75V. For NN+SS+FF+SF Process Sections. >

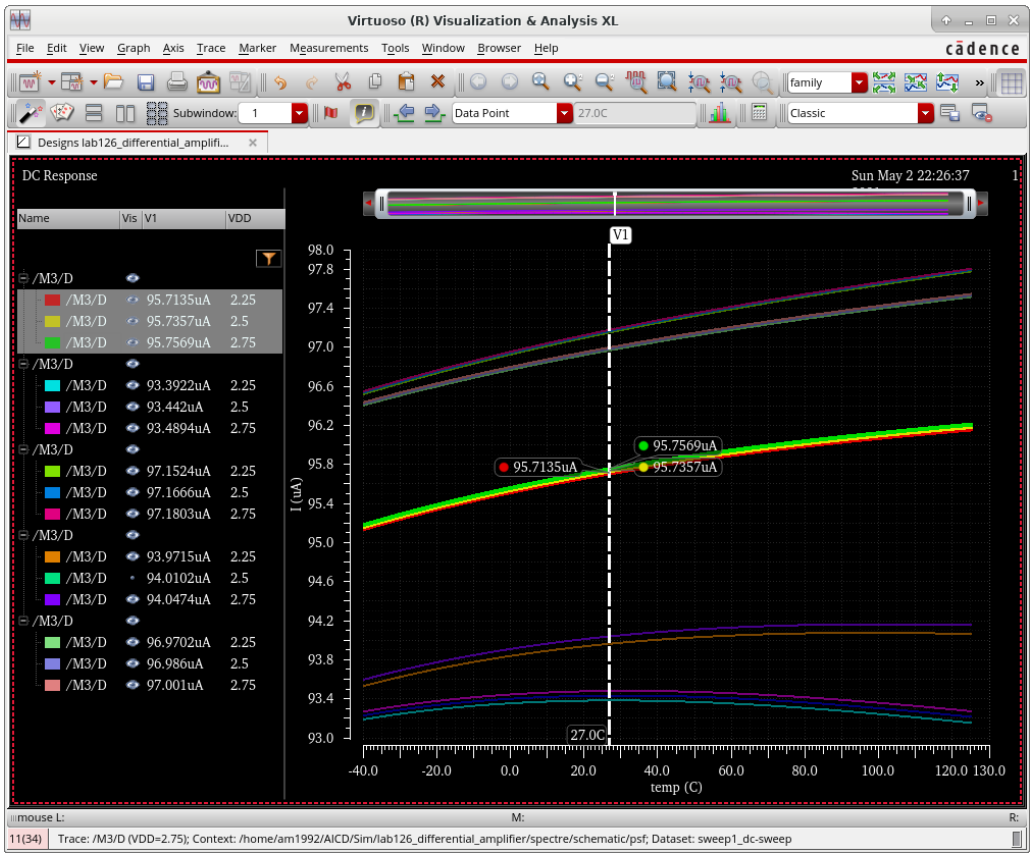


< Figure 220. DC operating point v_{out} of the TCA for temperature -40°C to +125°C for VDD = 2.25V, 2.5V, and 2.75V. For NN+SS+FF+SF+FS Process Sections. >

(4.4.10) Repeat the process again for the tail current (M3/D) output.



< Figure 221. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 222. DC operating point I_{TAIL} of the TCA for temperature -40°C to +125°C for VDD = 2.25V, 2.5V, and 2.75V. NN+SS+FF+SF+FS Process Sections. >

(4.4.11) **Discussion**. When determining the worst-case PVT variations (the corner cases), we are looking for the highest and lowest values of the circuit parameter under investigation across all the generated curves/plots. These can (literally) be found in the corners of the plot window.

For the circuit to have 'passed' the corner analysis, that is, deemed sufficiently robust to PVT variations, the highest and lowest values (i.e., the worst-case values) of the circuit parameter (v_{out}) must be within the range given in the specifications. If a parameter falls outside the specifications, the circuit has failed the test/analysis.

In the above simulations, over the full PVT variation (Figure 21), the DC operating point at v_{out} varies from 1.54V at nominal (VDD=2.5V, Temp=27°C, and Process=NN), to 1.21V at its lowest corner-case (VDD=2.25V, Temp=-40°C, and Process=SS), to 1.88V at its highest corner-case (VDD=2.75V, Temp=125°C, and Process=FF).

For the tail current (Figure 23), we get 95.7uA at nominal (VDD=2.5V, Temp-27°C, and Process=NN), 93.1uA at its lowest corner-case (VDD=2.25V, Temp=125°C, and Process=SS), and 97.8uA at its highest corner-case (VDD=2.75V, Temp=125°C, and Process=FF).

This gives a total v_{out} variation of ~ 1.54V +/- 0.34V and a tail current variation of ~ 95.7uA +/- 3uA. This would be considered acceptable in most applications (though, this would depend on the exact specifications of application). We see that the current mirror loaded TCA circuit is inherently self-biasing and robust to PVT (helped by the large NMOS/PMOS device sizes 10um/1um). Other circuits such as the differential amplifier with PMOS current source loads are very susceptible to variations in PVT and require an active feedback approach to maintain the proper bias.

In the next part of this lab, we will examine the role of feedback as a way of mitigating the adverse effects of PVT variation.

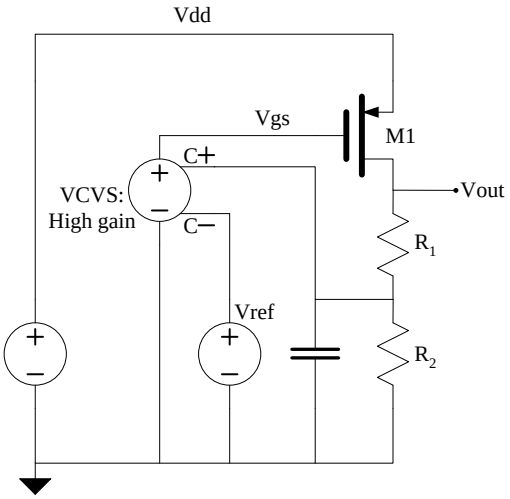
5. Circuit Simulation Exercises: Part 2

With guidance, you will:

- Examine the use of feedback to remove the problems associated with PVT variation.

5.1 The Controllable Voltage Reference Circuit

(5.1.1) Create the schematic for the IC voltage reference circuit in Figure 24. A summary of the circuit's operation is given in section 4.6 of the background material.



< Figure 223. IC Controllable Voltage Reference Circuit

Reproduced from Figure 30, section 4.6 of the background material. >

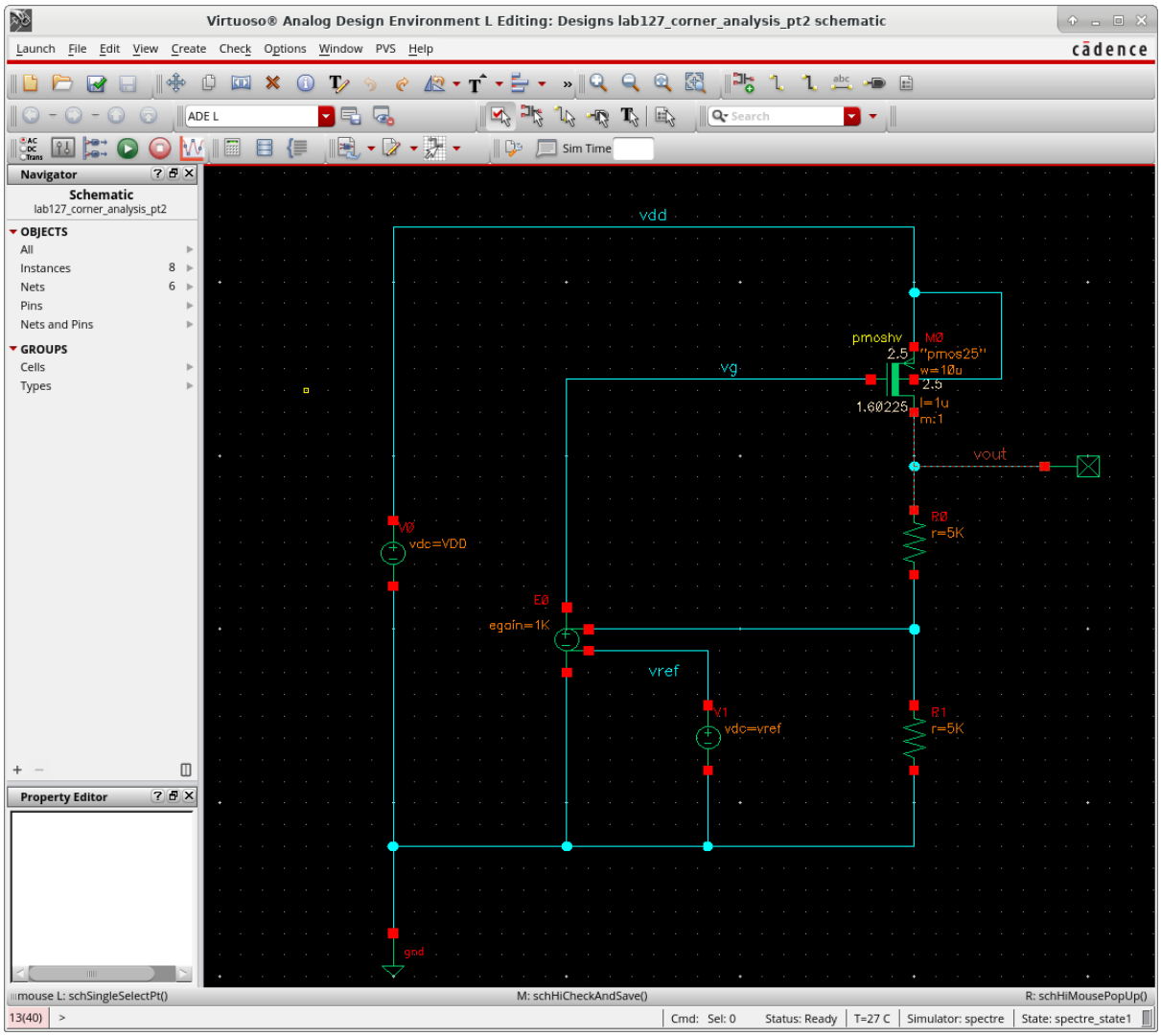
(5.1.2) You can either create the schematic from scratch (New File) or reuse the schematic of this circuit we created in lab 12.4, part 2 (lab124_vregulator_pt2). If creating a new file or copying the old cell, give the schematic a meaningful name e.g., lab127_corner_analysis_pt2.

(5.1.3) In the schematic editor window, construct the circuit given in Figure 24 (25). From the analogLib library instantiate and place two DC voltage sources (**vdc**), two resistors (**res**), a voltage-controlled voltage source (**vcvs**), and a ground reference (**gnd**). Also instantiate one high-voltage PMOS FET (**pmoshv**) from the GPDK180 library.

Parameterise your design. For the DC supply voltage, set the voltage attribute to the string **DC Voltage** = VDD. For the input, set the reference to the string **DC Voltage** = vref.

Dimension the PMOS device 10um/1um (**width** = 10u, **length** = 1u). Set the VCVS to a gain of 1K (**Voltage Gain** = 1K). Set the two resistors to 5KΩ (**Resistance** = 5K). Since we are simulating with DC you can ignore the capacitor. Leave all other attributes unchanged.

(5.1.4) Wire the components as in Figure 24 (25) using the **Wire (narrow)** command. **Remember to connect the body/bulk terminal of the PMOS device to the most positive net in the circuit (VDD=2.5V)**. Using the **Create Wire Name** command, name the DC supply net vdd, the PMOS gate net vg, and the reference input net vref.



< Figure 224. Controllable Voltage Reference Circuit.

(5.1.5) Check and save your schematic using the **Check and Save** command. Remember to use the output area of the CIW to diagnose any problems with your schematic and/or simulations if there are any warning or error messages when checking, saving, or simulating your work.

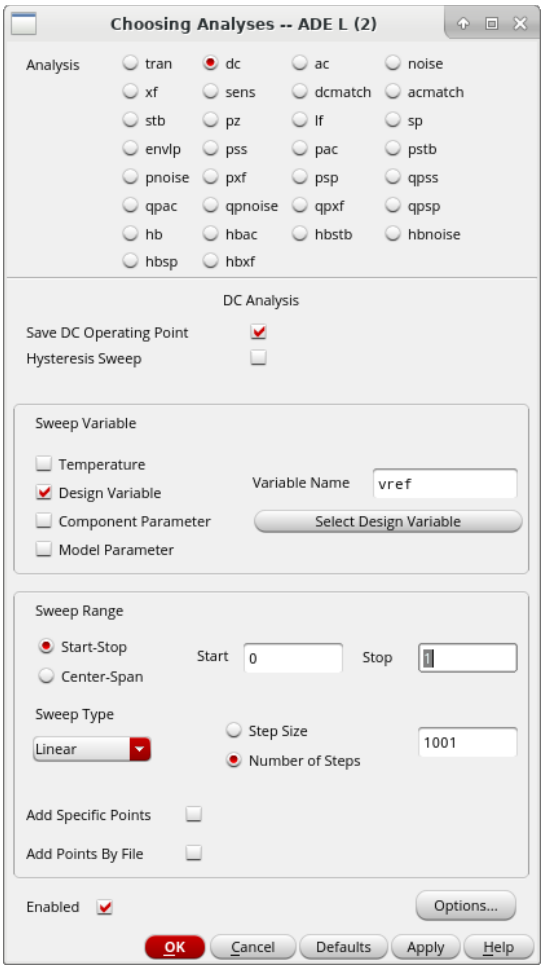
5.2 DC Sweep Analysis of the Volage Reference Circuit

(5.2.1) We will simulate the voltage reference using a DC sweep. We will sweep the input reference vref from 0 to 1V and observe how accurately the output vout tracks the input. From the Schematic Editor window, launch the Analogue Design Environment (ADE) from **Menu: Launch → ADE L**.

(5.2.2) Copy the design variables from the schematic into the ADE. From the ADE window, choose **Menu: Variables à Copy From Cellview**. You should now see the VDD and vref variables in the 'Design Variables' list of the ADE. Initialise VDD = *2.5*V and vref = *0.8*V.

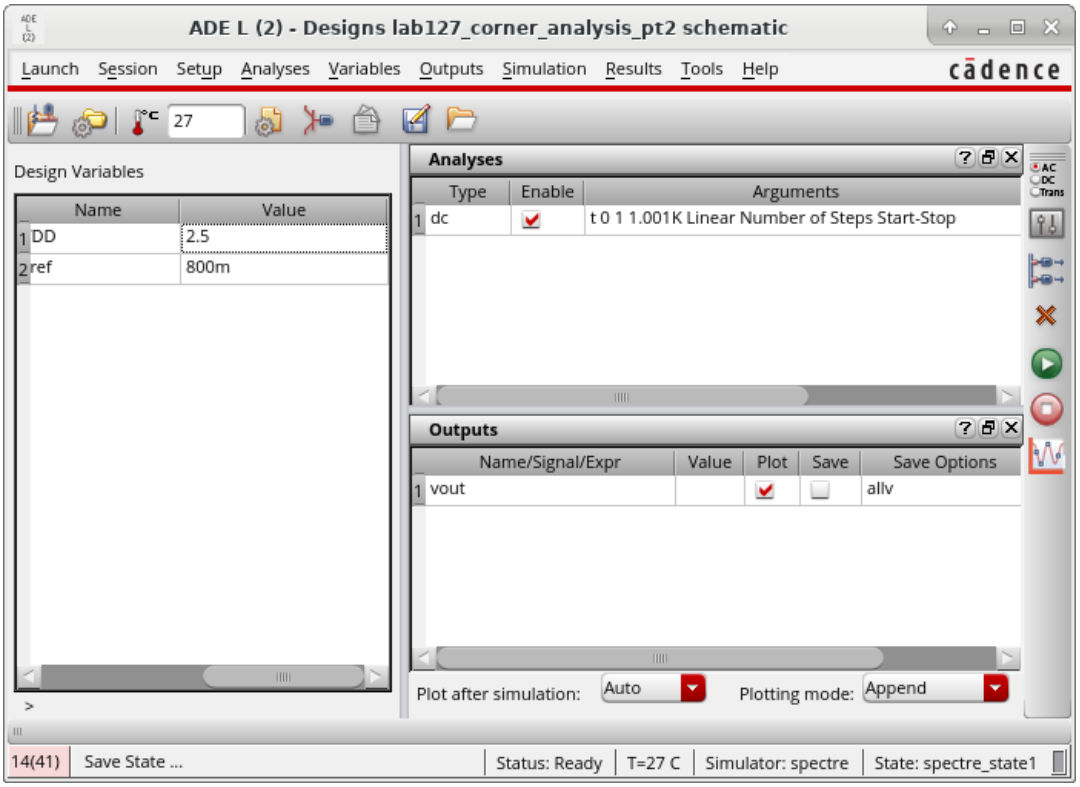
(5.2.3) From the ADE window, go to **Menu: Analyses à Choose…** or use the **Choose Analyses** button on the right-side of the ADE window (top button) to invoke the 'Choosing Analyses – ADE L' form. Select the '**dc**' option button.

In the choosing analyses form. Check the 'Save DC Operating Point' box. Check the 'Design Variable' box as the sweep variable. Type vref as the variable name. Choose the 'Start-Stop' option button with Start = 0 and Stop = 1 for the sweep range. Select 'Linear' from the drop-down menu as the sweep type. Choose the 'Number of Steps' option button with 1001 steps in the sweep. Ensure that the analysis is enabled by checking the 'Enabled' box near the bottom of the form. See Figure 26. OK the form. The **dc** analysis should now appear in the 'Analyses' list of the ADE window.

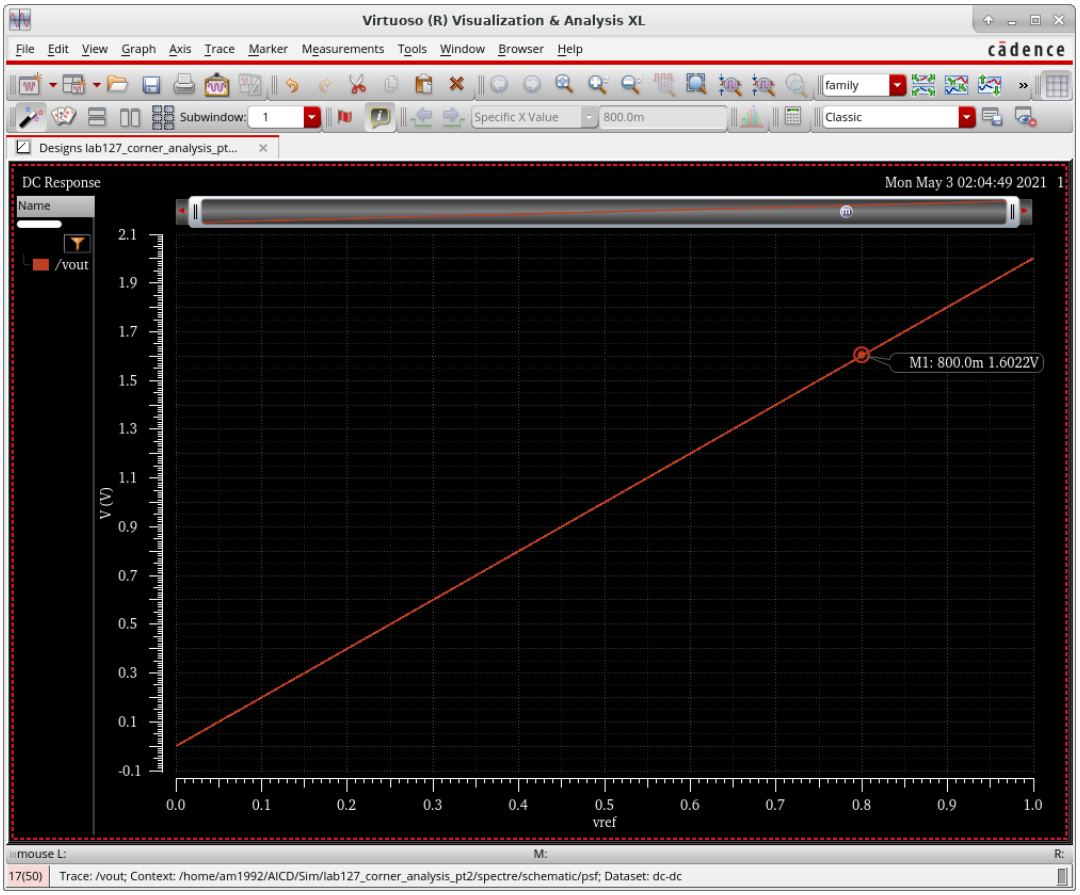


< Figure 225. The Choosing Analyses Form. >

(5.2.4) From the ADE, select **Menu: Outputs → To Be Plotted → Select On Schematic** to enter the plotting mode of the ADE**.** Select only the output net vout for plotting. **Make sure the Plotting Mode is set to 'Append'**. This will ensure the output plots are added to the waveform window automatically on each run of the simulator. **Press the 'ESC' key to exit the plotting mode of the ADE**.



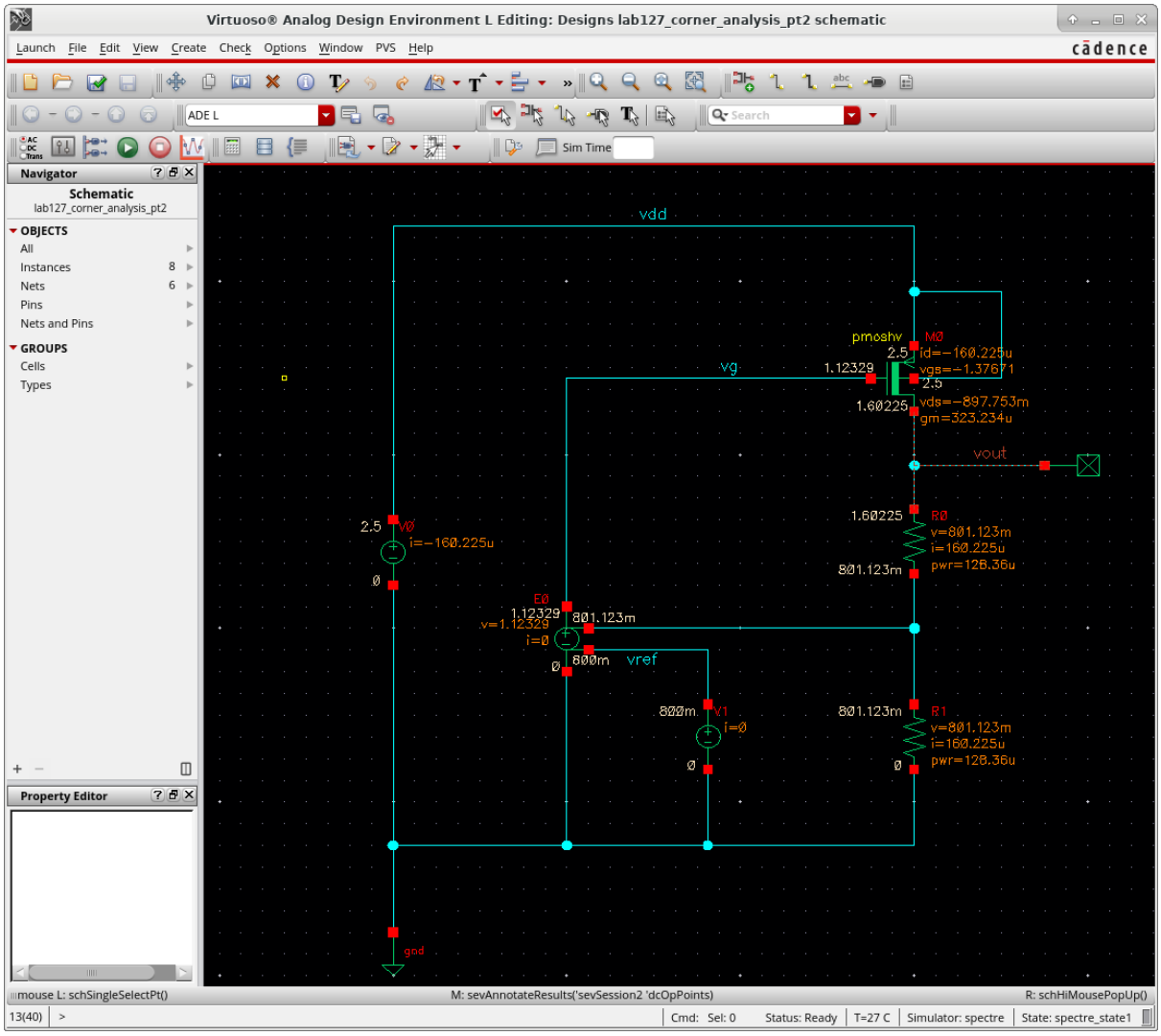
< Figure 226. The ADE with the Design Variables, Analyses and Outputs. >



< Figure 227. Output Voltage vout Tracking as input reference is swept from 0 to 1V. >

(5.2.5) In the Analog Design Environment window, run the '**Netlist and Run**' command by clicking on the green play button. The simulation should run without error. The text output window should appear but there will be no waveform window.

(5.2.6) From the ADE window choose **Menu: Results à Annotate à DC node voltages…** If you now look in the schematic window, you should see the node voltages annotated directly on the schematic. Again, from the ADE window, choose **Menu: Results à Annotate à DC operating points…** You should now see (along with the node voltages) additional DC operating point information annotated on the schematic including the DC bias currents. See Figure 29.



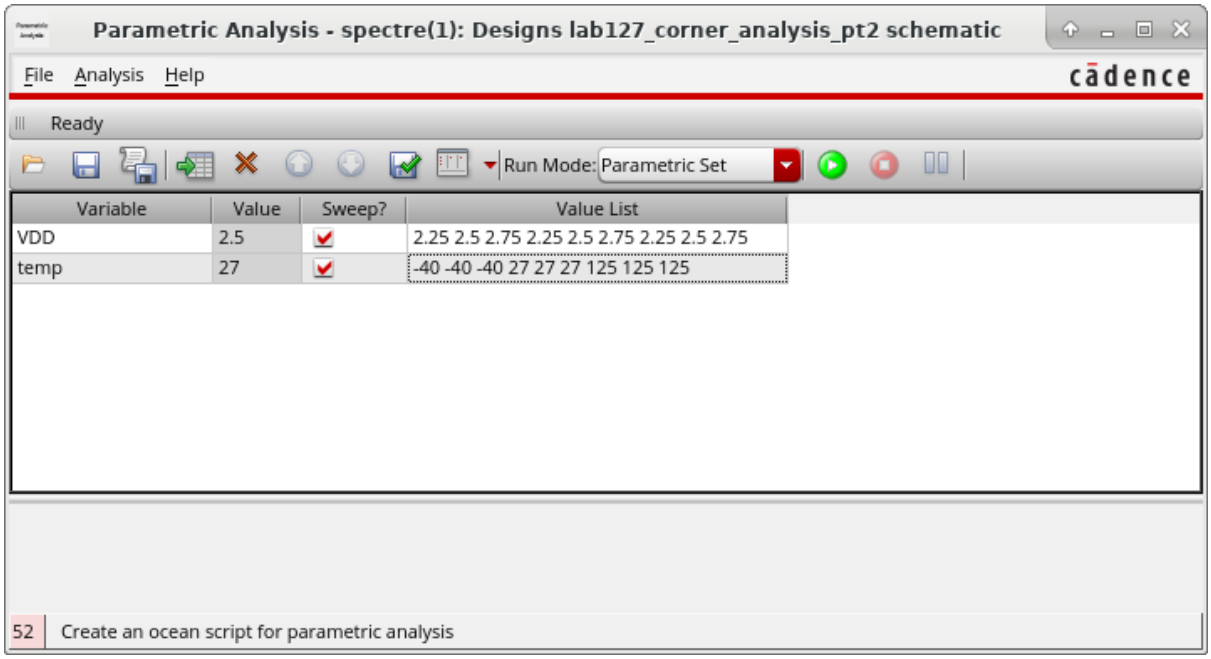
< Figure 228. Controllable Voltage Reference Circuit with Node Voltage and DC Operating Point Annotations. >

(5.2.7) **Discussion**. Refer to section 4.6 of the background material for a discussion on circuit operation. The important point to note is that the high gain of the VCVS regulates the output voltage(vout) to be twice that of the voltage reference (vref). The factor of two occurs due to the even division of the feedback signal across the potential divider. The feedback acts to maintain the same voltage on both inputs to the VCVS, this requires an output voltage across the resistors equal to 2*vref so that vcvs(+) equals vcvs(-). The high gain of the VCVS is used to overcome any non-ideal behaviour of the PMOS – or any other disturbance – such as PVT variation. The higher the gain the more closely the output will track the voltage reference (by a factor of two.) Note that the PMOS device adds an inversion in the loop – so the input to the VCVS is the negative terminal and the feedback signal goes to the positive terminal.

5.3 DC Sweep Analysis of the Volage Reference Circuit over Process, Voltage and Temperature (PVT)

(5.3.1) We will setup a PVT corner analysis so we can observe the ***simultaneous*** effect of process (NN, SS, FF, SF, FS), voltage (VDD +/-10%), and temperature (-40°C to 125°C) as we sweep the input reference from 0 to 1V.

(5.3.2) Since we are sweeping the input reference voltage vref from within the ADE itself – we will use the parametric analysis tool to vary the power supply voltage (VDD – 2.25V, 2.5V, and 2.75V) and temperature (-40°C, 27°C, and 125°C). Note that here we are using the nominal and corner values of both voltage and temperature (we are not sweeping the temperature linearly over the full range as we were before). See Figure 30. See (5.3.3) below for an explanation of how the value list is set up.



< Figure 229. Parametric Analysis Tool. Set up for a corner combination of VDD (+/- 10%) = 2.25V, 2.5V, and 2.75V and Temperature -40°C, 27 °C, and 125°C) >

(5.3.3) Note that when setting up the parametric list/set in the parametric analysis tool, you have to explicitly set the combinations of VDD and temperature. **The parametric tool will not itself iterate over all combinations.** The number of items/values in each list must match and each pair of values in the lists will be simulated in the order listed. For example, in Figure 30, the combination of {VDD, temp} simulated by the parametric tool will be:

{2.25, -40°C },

{2.5V, -40°C },

{2.75V, -40°C },

{2.25V, 27°C },

{2.5V, 27°C },

{2.75V, 27°C },

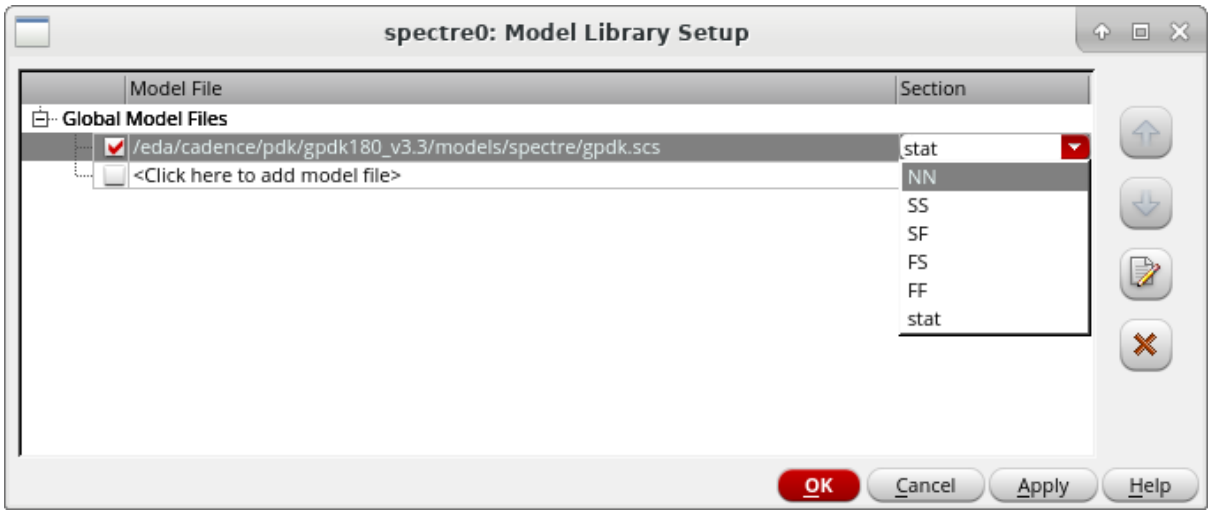
{2.25V, 125°C },

{2.5V, 125°C },

{2.75V, 125°C }.

This covers all corner combinations of VDD and temperature.

(5.3.4) Model library files settings. From the ADE, go to **Menu: Setup → Model Libraries** to invoke the Model Library Setup up form. Set the process section to **NN** (nominal/nominal) for the typical-mean performance. From the Parametric Analysis window, run the '**Run Selected Sweeps**' command by clicking on the green play button. The waveform window should open and plot the output net vout for the **NN** process parameter case, across temperature corners (-40°C, 27°C, and 125°C) at the VDD voltage corners (2.25V, 2.5V, and 2.75V).

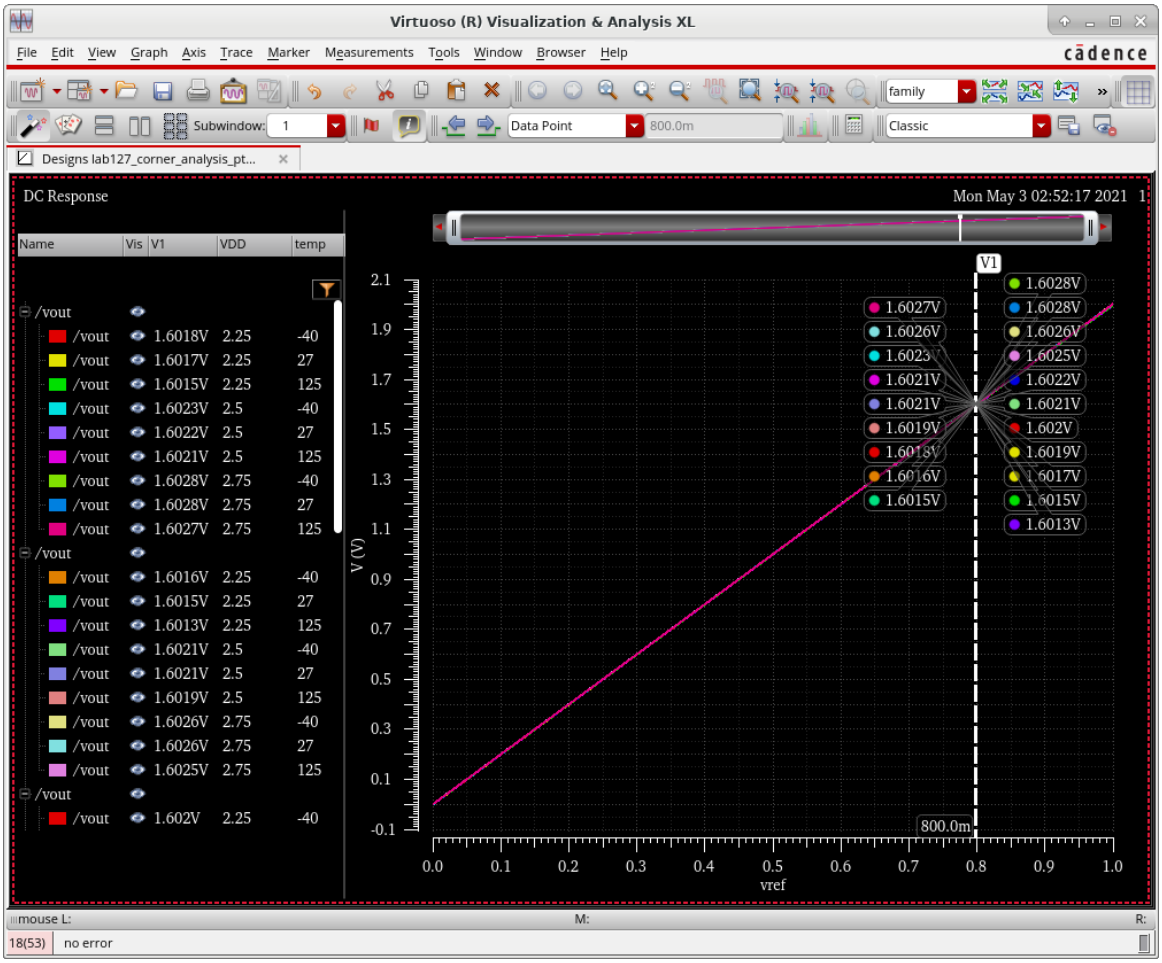


< Figure 230. Section Selection Drop-down Menu in the Model Library Setup Form. >

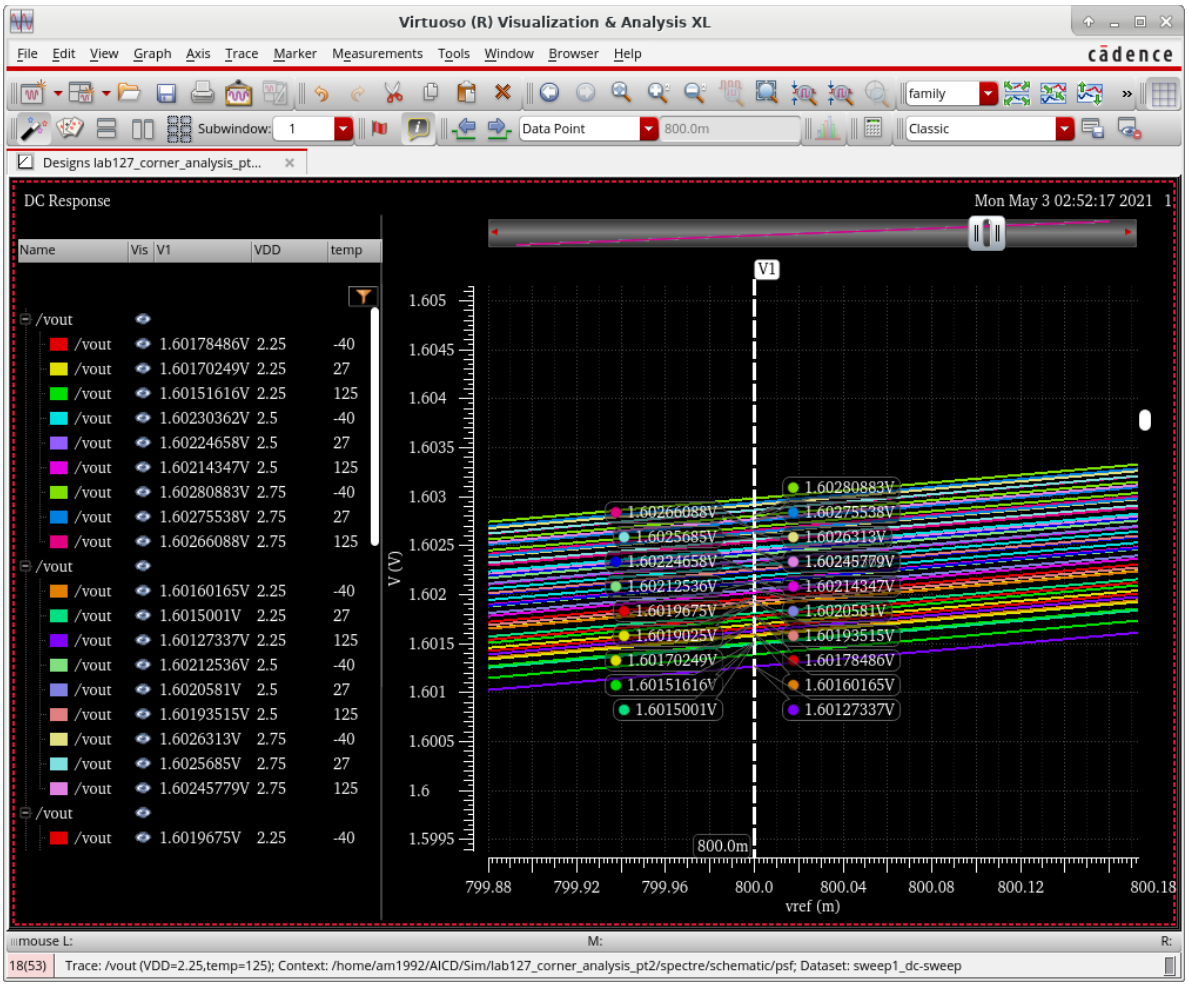
(5.3.5) Make sure the plotting mode is set to **Append**. Repeat the previous step for all process corners **SS**, **FF**, **SF**, **FS** (the order is not important here). For each corner, change the process section, apply or OK the form, and re-run the simulation. **Remember to run the simulation from the parametric tool not the ADE.**

(5.3.6) **Discussion**. Referring to Figures 32 and 33. With the feedback amplifier set to a gain of 1K we have an accurate tracking of the input voltage reference (multiplied by 2), even in the face of full PVT variation. For an input refence of 0.8V, the worst-case the output voltage is 1.6028V. This represents a tracking accuracy of:

$$T.A = \left(1 - \frac{Vout - (2 * Vref)}{(2 * Vref)}\right) \times 100\% = \left(1 - \frac{1.6028V - 1.6V}{1.6V}\right) \times 100\% = 98.25\%$$



< Figure 231. DC operating point v_{out} of the voltage regulator. Temperature -40°C, 27 °C, and +125°C. VDD = 2.25V, 2.5V, and 2.75V. NN+SS+FF+SF+FS Process Sections. VCVS Gain=1K>



< Figure 232. As above. Close-up around 0.8V. Temperature -40°C, 27 °C, and +125°C. VDD = 2.25V, 2.5V, and 2.75V. NN+SS+FF+SF+FS Process Sections. VCVS Gain=1K>

(5.3.7) Repeat steps 5.3.1 to 5.3.5 for a feedback amplifier **VCVS** gain of 100K.

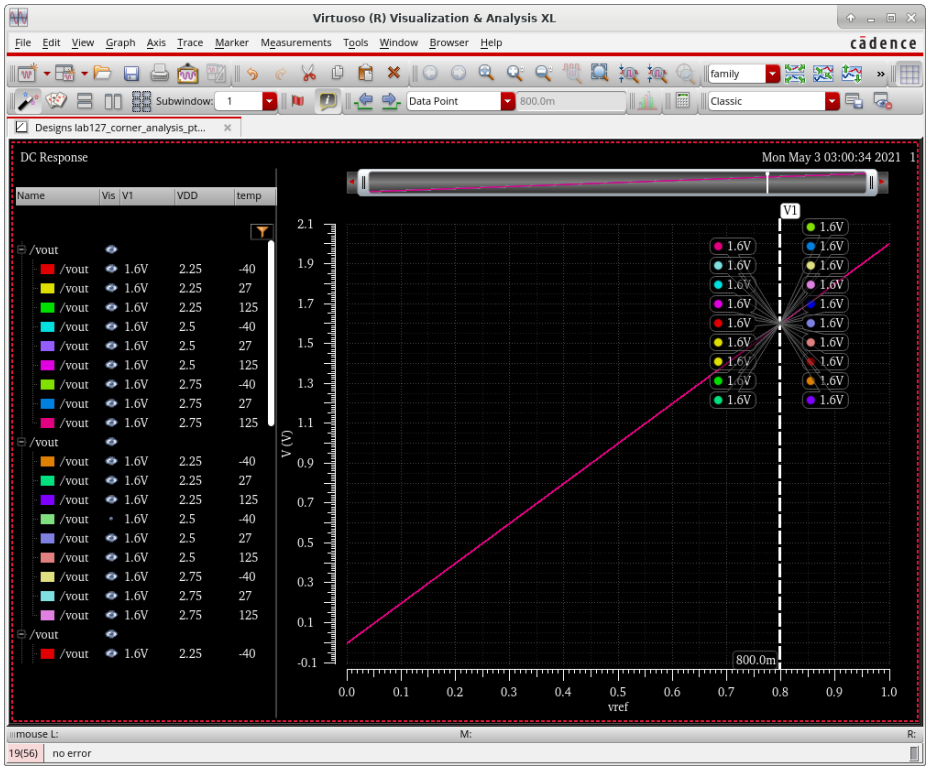
(5.3.8) **Discussion**. Referring to Figures 33 and 34. Increasing the feedback gain to 100K, the worst-case output voltage is 1.600028V. We therefore increase the tracking accuracy to,

$$T.A = \left(1 - \frac{1.600028V - 1.6V}{1.6V}\right) \times 100\% = 99.9825\%$$

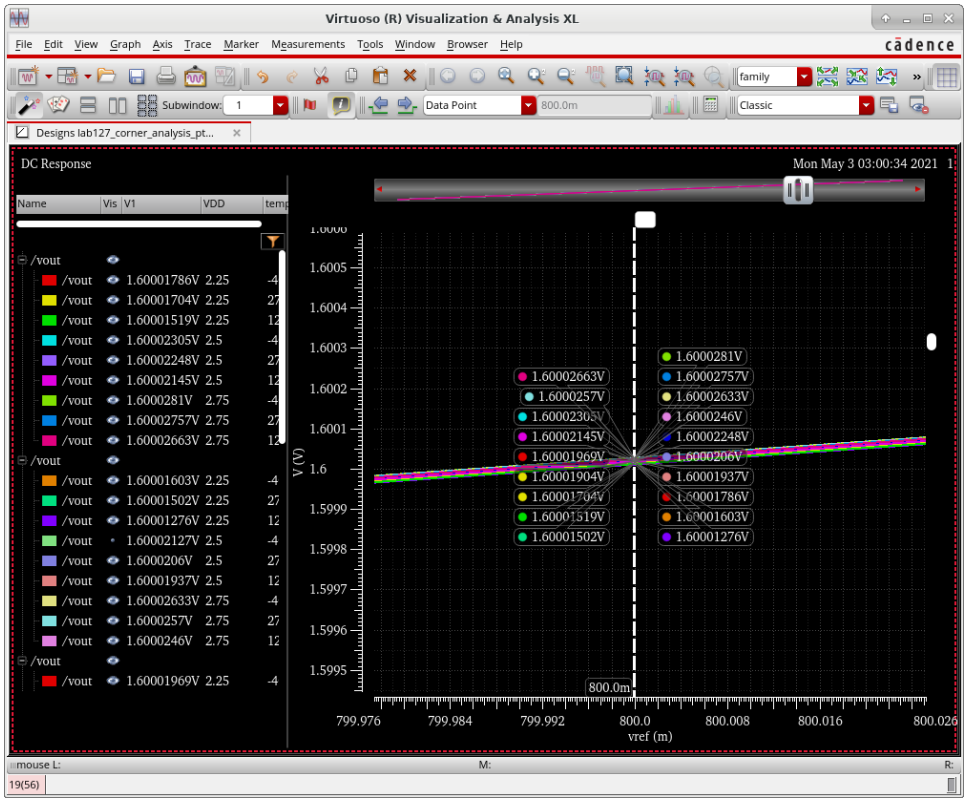
It is not surprising that increasing the gain by a factor of x100 improves the tracking accuracy by the same amount. The higher the gain the better the tracking accuracy and the less impact PVT variation has on circuit performance.

Note from lab 12.4, however, that there is a trade-off between tracking accuracy and stability – the higher the gain the worse the stability margin.

Analogue design is about trading one performance metric for another.



< Figure 233. DC operating point v_{out} of the voltage regulator. Temperature -40°C, 27 °C, and +125°C. VDD = 2.25V, 2.5V, and 2.75V. NN+SS+FF+SF+FS Process Sections. VCVS Gain=100K>



< Figure 234. As above. Close-up around 0.8V. Temperature -40°C, 27 °C, and +125°C. VDD = 2.25V, 2.5V, and 2.75V. NN+SS+FF+SF+FS Process Sections. VCVS Gain=100K> **End of Lab!**